

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЧЕРКАСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ БОГДАНА ХМЕЛЬНИЦЬКОГО

Авраменко В. С., Розломій І. О.

ОРГАНІЗАЦІЯ БАЗ ДАНИХ І ЗНАНЬ

Курс лекцій

Черкаси 2019

Рецензент:

Е. В. Фауре, доктор технічних наук, проректор з науково-дослідної роботи та міжнародних зв'язків Черкаського державного технологічного університету.

В. І. Салапатов, кандидат технічних наук, доцент кафедри програмного забезпечення автоматизованих систем Черкаського національного університету імені Богдана Хмельницького.

*Рекомендовано до друку рішенням Вченої ради
Черкаського національного університету імені Богдана Хмельницького
(Протокол № від р.)*

Авраменко В. С., Розломій І. О.

Організація баз даних і знань. Курс лекцій. Черкаси: Черкаський національний університет імені Богдана Хмельницького, 2019. 257 с.

У лекціях розглянуті головні концепції проектування і побудови баз даних, основні моделі баз даних, архітектури СКБД, сучасні напрямки розвитку технологій баз даних, питання, які пов'язані з експлуатацією баз даних. Всі головні положення лекцій розглядаються з використанням навчальних прикладів і ілюструються графічними матеріалами.

Курс лекцій призначений для студентів ЧНУ імені Богдана Хмельницького, які навчаються за спеціальностями «121 Інженерія програмного забезпечення», «122 Комп'ютерні науки», «123 Комп'ютерна інженерія», «124 Системний аналіз» та для студентів інших споріднених спеціальностей.

УДК 681.3.07

© Авраменко В. С.

© Розломій І. О.

ЗМІСТ

ВСТУП.....	8
1. Еволюція систем керування даними	9
1.1. Роль пристроїв зовнішньої пам'яті для зберігання інформації	9
1.2. Файлові системи	10
1.2.1. Структури файлів.....	10
1.2.2. Логічна структура файлових систем.....	11
1.2.3. Авторизація доступу до файлів	12
1.2.4. Синхронізація багатокористувацького режиму доступу	12
1.2.5. Області застосування файлів	13
1.3. Потреби інформаційних систем	13
1.3.1. Структури даних	15
1.3.2. Цілісність даних.....	16
1.3.3. Мови запитів	17
1.3.4. Транзакції, журналізація і багатокористувацький режим	18
1.3.5. СКБД як незалежний системний компонент	19
2. ПОНЯТТЯ МОДЕЛІ ДАНИХ. РІЗНОВИДИ МОДЕЛЕЙ ДАНИХ.	21
2.1. Поняття бази даних	21
2.2. Модель даних	22
2.3. Ранні моделі даних	23
2.3.1. Модель даних інвертованих таблиць.....	23
2.3.2. Ієрархічна модель даних	24
2.3.3. Мережева модель даних.....	26
2.4. Неформальний вступ до реляційної моделі даних	27
2.4.1. Реляційні структури даних	28
2.4.2. Маніпулювання реляційними даними	28
2.4.3. Цілісність в реляційній моделі даних	30
2.5. Сучасні моделі даних	31
2.5.1. Об'єктно-орієнтована модель даних	31
2.5.2. Справжня реляційна модель	32
3. РЕЛЯЦІЙНА МОДЕЛЬ ДАНИХ	34
3.1. Вступ	34
3.2. Основні поняття реляційних баз даних	34
3.2.1. Сутність і атрибути.....	35
3.2.2. Тип даних і домен	36
3.2.3. Відношення	36
3.2.4. Первинний і зовнішній ключ.....	38
3.2.5. Зв'язок	40
3.3. Фундаментальні властивості відношень	41
3.3.1. Відсутність кортежів-дублікатів	41
3.3.2. Відсутність впорядкованості кортежів і атрибутів	41
3.3.3. Атомарність значень атрибутів, перша нормальна форма відношення	42
3.4. Реляційна модель даних.....	45
3.4.1. Загальна характеристика.....	45
3.4.2. Взаємозв'язок файлів (таблиць).....	45

3.4.3. Цілісність суті і посилань	47
3.5. Індування.....	49
4. ЗАСОБИ МАНПУЛЮВАННЯ РЕЛЯЦІЙНИМИ ДАНИМИ	52
4.1. Вступ	52
4.2. Огляд реляційної алгебри Кодда.....	52
4.2.1. Загальна інтерпретація реляційних операцій.....	53
4.2.2. Замкнутість реляційної алгебри і операція перейменування	54
4.3. Особливості теоретико-множинних операцій реляційної алгебри.....	55
4.3.1 Операції об'єднання, перетину, взяття різниці.....	55
4.3.2. Операція розширеного декартового добутку.....	58
4.4. Спеціальні реляційні операції	59
4.4.1. Операція обмеження.....	59
4.4.2. Операція взяття проєкції.....	60
4.4.3. Операція з'єднання відношень.....	61
4.5. Приклади використання реляційних операторів	64
5. ПРОЕКТУВАННЯ РЕЛЯЦІЙНИХ БАЗ ДАНИХ НА ОСНОВІ НОРМАЛІЗАЦІЇ.....	66
5.1. Вступ	66
5.2. Етапи розробки бази даних.....	67
5.2.1. Концептуальна модель предметної області	69
5.2.2. Логічна модель даних.....	70
5.2.3. Фізична модель даних	71
5.3. Критерії оцінки якості логічної моделі даних	71
5.4. Перша Нормальна Форма	74
5.4.1. Перша Нормальна Форма (1НФ).....	74
5.4.2. Аномалії поновлення.....	74
5.5. Функціональні залежності	76
5.6. Друга нормальна форма	76
5.6.1. Друга нормальна форма (2НФ)	76
5.6.2. Аналіз декомпозиції відношень	78
5.7. Третя Нормальна Форма (3НФ)	79
6. НОРМАЛЬНІ ФОРМИ БІЛЬШ ВИСОКИХ ПОРЯДКІВ.....	81
6.1. Нормальна форма Бойса-Кодда.....	81
6.2. Четверта нормальна форма (4НФ)	85
6.3. П'ята нормальна форма	88
6.4. Загальна схема процедури нормалізації	90
6.5. Денормалізація бази даних	91
6.5.1. Загальні відомості про денормалізації.....	91
6.5.2. Коли потрібна денормалізація?	95
6.5.3. Як визначити, коли денормалізація виправдана?	96
6.5.4. Як грамотно реалізувати денормалізацію?	97
7. КОНЦЕПТУАЛЬНЕ ПРОЕКТУВАННЯ БАЗ ДАНИХ	98
7.1. Обмеженість реляційної моделі при проектуванні БД	98
7.2. Концептуальна модель предметної області	100
7.3. Модель «Сутність-Зв'язок».....	100
7.3.1. Сутності	101

7.3.2. Зв'язки	102
7.3.3. Атрибути.....	103
7.3.4. Потужність зв'язків.....	104
7.3.5. Сильні і слабкі зв'язки.....	105
7.3.6. Атрибути зв'язків. Обов'язкові і необов'язкові зв'язки	105
7.3.7. Слабкі сутності. Складні зв'язки	106
7.3.8. Рекурсивні зв'язки.....	107
7.4. Розширена модель «сутність – зв'язок».....	108
7.5. Проблеми побудови моделей «Сутність – Зв'язок»	111
7.6. Приклад побудови моделі «Сутність – Зв'язок».....	112
7.7. Семантична модель «Сутність-Зв'язок» Баркера.....	114
7.7.1. Нотація моделі «Сутність-Зв'язок» Баркера	114
7.7.2. Приклад розробки простої ER-моделі в нотації Баркера.....	117
8. ЛОГІЧНЕ ПРОЕКТУВАННЯ БАЗ ДАНИХ	122
8.1. Етапи логічного проектування	122
8.2. Спрощення концептуальної моделі	122
8.3. Методика перетворення ER-діаграм в реляційні структури	126
8.3.1. Сутності і атрибути	126
8.3.2. Зв'язки	127
8.4. Перевірка відношень за допомогою правил нормалізації	133
8.5. Приклад створення логічної моделі бази даних	134
9. ФІЗИЧНЕ ПРОЕКТУВАННЯ БАЗИ ДАНИХ	136
9.1. Етапи фізичного проектування баз даних	136
9.2. Організація зберігання інформації.....	138
9.3. Хешування.....	139
9.4. Індксація	140
9.5. В-дерева	141
9.6. Інвертовані файли.....	142
10. МОВА SQL. ФОРМУВАННЯ ЗАПИТІВ ДО БАЗИ ДАНИХ	144
10.1. Оператори SQL	144
10.2. Приклади використання операторів маніпулювання даними	145
10.2.1. Приклади використання операторів INSERT і UPDATE.....	145
10.2.2. Приклади використання оператора SELECT	145
10.2.3. Синтаксис оператора вибірки даних (SELECT)	156
10.2.4. Порядок виконання оператора SELECT.....	160
11. ФУНКЦІЇ ТА КОМПОНЕНТИ СКБД	163
11.1. Функції СКБД	163
11.2. Компоненти СКБД.....	164
11.3. Архітектурні рішення доступу до БД	166
11.3.1. Файл-сервер.....	167
11.3.2. Клієнт-сервер	168
12. РОЗПОДІЛЕНА ОБРОБКА ДАНИХ	171
12.1. Основні поняття і визначення	171
12.2. Управління паралельною обробкою	172
12.3. Багатокористувацькі СКБД	174
12.4. Проектування багатокористувацьких баз даних	177
12.5. Проектування розподілених баз даних.....	178

13. ТРАНЗАКЦІЇ І ЦІЛІСНІСТЬ БАЗ ДАНИХ	182
13.1. Обмеження цілісності.....	182
13.2. Класифікація обмежень цілісності.....	185
13.2.1. Класифікація обмежень цілісності за способами реалізації.....	185
13.2.2. Класифікація обмежень цілісності за часом перевірки	187
13.2.3. Класифікація обмежень цілісності по області дії.....	187
13.3. Реалізація декларативних обмежень цілісності засобами SQL.....	191
13.3.1. Загальні принципи реалізації обмежень засобами SQL.....	191
14. ТРАНЗАКЦІЇ І ПАРАЛЕЛІЗМ.....	194
14.2. Конфлікти між транзакціями.....	197
14.3. Блокування	199
14.4. Дозвіл тупикових ситуацій	200
14.5. Метод тимчасових міток.....	201
14.6. Реалізація ізольованості транзакцій засобами SQL.....	202
15. ЗАХИСТ І ВІДНОВЛЕННЯ ДАНИХ	204
15.1. Захист інформації в базах даних	204
15.2. Види відновлення даних	207
15.2.1. Індивідуальний відкат транзакції.....	208
15.2.2. Відновлення після м'якого збою	208
15.2.3. Відновлення після жорсткого збою	210
15.3. Шифрування.....	210
16. АДМІНІСТРУВАННЯ БАЗ ДАНИХ.....	212
16.1. Цілі адміністрування та його актуальність для сучасних БД.....	212
16.2. Процедура адміністрування.....	212
16.3. Резервування і відновлення БД	216
16.4. Оптимізація роботи БД	216
16.5. Безпека даних	217
16.6. Підтримка заходів забезпечення безпеки в мові SQL	217
17. БАЗИ ЗНАНЬ	221
17.1. Коли дані стають знаннями	221
17.2. Постулати систем баз даних	223
17.3. Моделі зображення знань	224
17.3.1. Формально-логічна модель.....	224
17.3.2. Продукційна модель	225
17.3.3. Семантичні мережі	228
17.3.4. Фреймова модель.....	231
17.4. Механізми виведення даних	231
17.4.1. Індуктивне виведення.....	231
17.4.2. Виведення за аналогією	232
17.5. Інтелект людини і штучний інтелект.....	233
17.6. Застосування баз знань.....	233
17.6.1. Експертні системи	234
17.6.2. Інші способи застосування штучного інтелекту	235
17.6.3. Погляд у майбутнє.....	235
18. ПЕРСПЕКТИВИ РОЗВИТКУ БД ТА СКБД.....	236

18.1. Революція 2005 року в реляційних базах даних	236
18.2. Інтелектуальний аналіз даних	240
18.3. Об'єктно-орієнтовані бази даних.....	242
18.4. Темпоральні бази даних.....	243
18.5. Дедуктивні бази даних	243
18.6. Бази даних NoSQL	243
18.6.1. Рух NOSQL.....	243
18.6.2. Документоорієнтовані бази даних	245
18.6.3. Графові бази даних	246
18.7. Web-технології і бази даних	247
19. СЛОВНИК ТЕРМІНІВ БАЗ ДАНИХ.....	248

ВСТУП

Розвиток комп'ютерних технологій, пов'язаних зі зберіганням і обробкою даних, привів до появи з середини 1960-х рр. поняття бази даних і спеціалізованого програмного забезпечення, що одержало назву систем керування базами даних (СКБД) – DataBase Management Systems (DBMS).

На сьогоднішній день не залишилося жодної галузі знань, де не застосовувалися б комп'ютерні бази даних і системи управління базами даних. У переліку сучасного програмного забезпечення бази даних входять в десятку найбільш затребуваних програмних продуктів і служать джерелом непоганого заробітку для професійних програмістів. Без зберігання і обліку даних сьогодні обійтися вельми складно. Численні магазини, склади, страхові агентства, відділи кадрів, бухгалтерії, навчальні заклади і багато інших підприємств і організацій мають гостру потребу в розроблених спеціально для них баз даних.

Бази даних завжди були важливою темою при вивченні інформаційних систем. Але саме в останні роки, завдяки бурхливому розвитку Інтернету і пов'язаного з цим технологічного прориву, знання технології баз даних стало одним з найбільш популярних шляхів до кар'єри. Технологія баз даних дозволяє зробити інтернет-додаток чимось більшим, ніж просто засіб для публікації брошур, що було характерно для ранніх додатків. У той же час інтернет-технології забезпечують стандартизований і доступний спосіб доставки вмісту бази даних користувачам. Жодне з цих нових обставин не скасовує необхідності в класичних додатках баз даних, які були незамінні в бізнесі до появи Інтернету, – вони лише посилюють важливість знань про базах даних.

Багато студентів знаходять цей предмет цікавим і захоплюючим, хоча часом він може бути важким. Проектування і розробка баз даних вимагають одночасно і мистецтва, і інженерних навичок. Розуміння вимог користувача і втілення цих вимог у ефективної логічну структуру бази даних є мистецтвом. Перетворення логічної структури в фізичну базу даних з функціонально завершеними, високопродуктивними додатками є інженерну задачу.

Тому створити ефективну базу даних вкрай непросто, навіть якщо вона призначена для обслуговування незначних обсягів даних і підлягає експлуатації на домашньому комп'ютері. І тільки при роботі з базами даних від користувачів потрібні особливі навички і вміння, які приходять тільки з досвідом. Складність проекту зростає на порядок, коли виникає завдання розробити життєздатний комерційний продукт, з яким будуть одночасно працювати десятки користувачів. Саме тому головне завдання цих лекцій – надати студентам знання про порядок проектування реляційних баз даних і розробці програм, що використовують ці бази даних.

Лекції не обмежують студентів у виборі цільової СКБД, тому вашу базу даних можна розгорнути як на основі найпростіших настільних систем (наприклад, Microsoft Access), так і на фундаменті професійних багатокористувацьких клієнт-серверних систем, таких як InterBase, FireBird, Blackfish SQL, Microsoft SQL Server, Oracle, MySQL тощо.

1. ЕВОЛЮЦІЯ СИСТЕМ КЕРУВАННЯ ДАНИМИ

У цій вступній лекції ми обговоримо передумови виникнення в комп'ютерах пристроїв зовнішньої пам'яті, а також обґрунтуємо принципову важливість для організації інформаційних систем дискових пристроїв. Будуть розглянуті особливості організації та основне функціональне призначення одного з ключових компонентів сучасних операційних систем – систем управління файлами. В розділі 1.3. Потреби інформаційних систем буде показано, чому можливостей файлових систем недостатньо для створення інформаційних програмних систем. Буде продемонстровано, що природні вимоги інформаційних систем до засобів управління даними у зовнішній пам'яті призводять до необхідності наявності систем керування базами даних (СКБД). В ході цього аналізу будуть визначені основні риси, якими повинні володіти СКБД.

1.1. Роль пристроїв зовнішньої пам'яті для зберігання інформації

У найширшому сенсі *інформаційна система* представляє собою програмний комплекс, функції якого полягають у підтримці надійного зберігання інформації в пам'яті комп'ютера, виконанні специфічних для даного застосування перетворень інформації та/або обчислень, наданні користувачам зручного і легко освоюється інтерфейсу. Зазвичай обсяги даних, з якими доводиться мати справу таким системам, досить великі, а самі дані мають досить складну структуру. Класичними прикладами інформаційних систем є банківські системи, системи резервування авіаційних або залізничних квитків, місць в готелях і т. д.

Про надійне і довготривале зберігання інформації можна говорити тільки при наявності запам'ятовуючих пристроїв, що зберігають інформацію після виключення електроживлення. Оперативна (основна) пам'ять цією властивістю зазвичай не володіє. У перші десятиліття розвитку обчислювальної техніки використовувалися два види пристроїв зовнішньої пам'яті: магнітні стрічки та магнітні барабани. При цьому ємність магнітних стрічок була досить велика, але за своєю природою вони забезпечували послідовний доступ до даних. Ємність магнітної стрічки пропорційна її довжині. Щоб отримати доступ до необхідної порції даних, потрібно в середньому перемотати половину її довжини. Але чисто механічну операцію перемотування можна виконати дуже швидко. Тому швидкий довільний доступ до даних на магнітній стрічці неможливий.

Магнітний барабан був масивний металевий циліндр з намагніченою зовнішньою поверхнею і нерухомим пакетом магнітних головок. Такі пристрої забезпечували можливість досить швидкого довільного доступу до даних, але дозволяли зберігати порівняно невеликий обсяг даних. Швидкий довільний доступ здійснювався завдяки високій швидкості обертання барабана і наявності окремої головки на кожен доріжку магнітної поверхні; обмеженість обсягу була обумовлена наявністю лише однієї магнітної поверхні.

Зазначені обмеження не дуже істотні для систем чисельних розрахунків. Для збереження проміжних результатів ідеально підходять магнітні стрічки: при виконанні процедури установки контрольної точки дані послідовно скидаються на стрічку, а при необхідності перезапуску від збереженої контрольної точки дані також послідовно з стрічки зчитуються.

Друга традиційна потреба «чисельних програмістів» – максимально великий обсяг оперативної пам'яті. Велика оперативна пам'ять потрібно, по-перше, для того щоб забезпечити програмі швидкий доступ до великої кількості оброблюваних даних. По-друге, складні обчислювальні програми самі можуть мати великий об'єм.

Однак для інформаційних систем, в яких обсяг постійно збережених даних визначається специфікою бізнес-додатків, а потреба в поточних даних визначається користувачем додатків, одних тільки магнітних барабанів і стрічок недостатньо. Ємність магнітного барабана просто не дозволяє тривалий час зберігати дані великого обсягу. Що ж стосується стрічок, то уявіть собі стан людини, яка, стоячи біля квиткової каси, повинна дочекатися повної перемотування магнітної стрічки. Природним вимогам до таких систем є забезпечення високої середньої швидкості виконання операцій при наявності великих обсягів даних.

Саме вимоги до пристроїв зовнішньої пам'яті з боку бізнес-додатків викликали появу пристроїв зовнішньої пам'яті зі знімними пакетами магнітних дисків і рухливими головками читання/запису, що стало революцією в історії обчислювальної техніки. Ці пристрої пам'яті володіли істотно більшою ємністю, ніж магнітні барабани (за рахунок наявності декількох магнітних поверхонь), забезпечували задовільну швидкість доступу до даних в режимі довільної вибірки, а можливість зміни дискового пакету на пристрої дозволяла мати архів даних практично необмеженого обсягу.

З появою *магнітних дисків* почалася історія систем управління даними у зовнішній пам'яті. До цього кожна прикладна програма, якою потрібно було зберігати дані в зовнішній пам'яті, сама визначала розташування кожної порції даних на магнітному диску і виконувала обміни між оперативною і зовнішньою пам'яттю за допомогою програмно-апаратних засобів низького рівня (машинних команд або викликів відповідних програм операційної системи). Такий режим роботи не дозволяв або дуже ускладнював підтримку на одному зовнішньому носії декількох архівів довготривалого зберігання. Крім того, кожній прикладній програмі доводилося вирішувати проблеми іменування частин даних і структуризації даних у зовнішній пам'яті.

1.2. Файлові системи

Історичним кроком став перехід до використання *систем керування файлами*. З точки зору прикладної програми *файл* – це іменована область зовнішньої пам'яті, в яку можна записувати і з якої можна зчитувати дані. Правила іменування файлів, спосіб доступу до даних, що зберігаються в файлі, і структура цих даних залежать від конкретної системи управління файлами і від типу файлу. Система управління файлами бере на себе розподіл зовнішньої пам'яті, відображення імен файлів у відповідні адреси зовнішньої пам'яті і забезпечення доступу до даних.

У цьому розділі ми розглянемо історію файлових систем, їх основні риси та області раціонального використання. Перша розвинена файлова система була розроблена фахівцями IBM в середині 60-х рр. для серії комп'ютерів «360». У цій системі підтримувалися як чисто послідовні, так і індексного-послідовні файли, а реалізація багато в чому спиралася на можливості тільки що з'явилися до цього часу контролерів керування дисковими пристроями. Контролери забезпечували можливість обміну з дисковими пристроями порціями даних довільного розміру, а також індексний доступ до записів файлів, і ці функції контролерів активно використовувалися в файловій системі OS/360.

Файлова система OS/360 забезпечила майбутніх розробників унікальним досвідом використання дискових пристроїв з рухомими головками, який має відображатися у всіх сучасних файлових системах.

1.2.1. Структури файлів

Практично у всіх сучасних комп'ютерах основними пристроями зовнішньої пам'яті є магнітні диски з рухомими головками, і саме вони служать для зберігання файлів. Як зазначалося раніше, апаратура магнітних дисків допускає виконання обміну з дисками порціями даних довільного розміру. Однак можливість обмінюватися з магнітними дисками порціями, розміри яких менше повного об'єму блоку, в даний час в файлових системах не використовується. Це пов'язано з двома обставинами.

По-перше, зчитування або запис тільки частини блоку не призводить до істотного виграшу в сумарному часу обміну. По-друге, для роботи з частинами блоків файлова система повинна забезпечити буфери оперативної пам'яті відповідного розміру, що істотно ускладнює розподіл оперативної пам'яті.

Тому у всіх сучасних файлових системах явно чи неявно виділяється рівень, що забезпечує роботу з *базовими файлами*, які представляють собою набори блоків, послідовно нумерованих в адресному просторі файлу і відображаються на фізичні блоки диска (рис. 1.1). Розмір логічного блоку файлу збігається з розміром фізичного блоку диска або кратний йому; зазвичай розмір

логічного блоку вибирається рівним розміру сторінки віртуальної пам'яті, підтримуваної апаратурою комп'ютера разом з операційною системою.

У деяких файлових системах базовий рівень був доступний користувачеві, але частіше він прикривався деякими більш високим рівнем, стандартним для користувачів. Існують два основні підходи. При першому підході, властивому, наприклад, файловим системам операційних систем компанії DEC RSX і VMS, користувачі представляють файл як послідовність записів. Кожен запис – це послідовність байтів, що має постійний або змінний розмір. Можна читати або писати записи послідовно або позиціонувати файл на запис з вказаним номером. Деякі файлові системи дозволяють структурувати записи на поля і оголошувати якісь поля ключами запису.

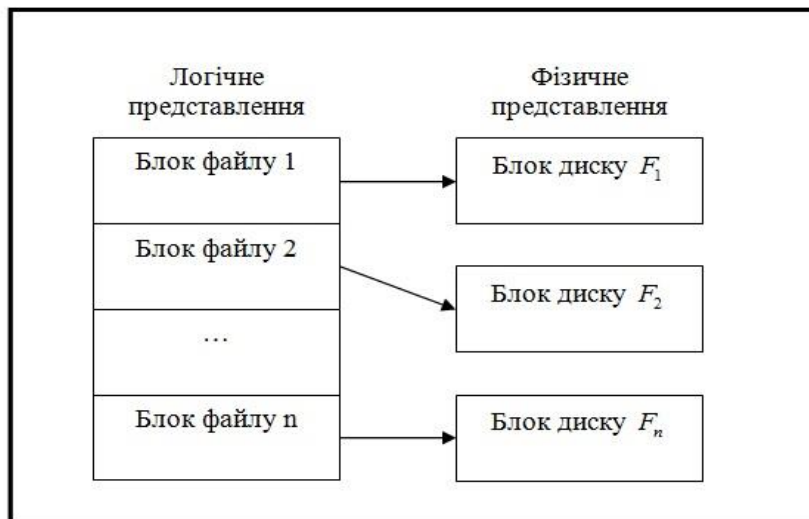


Рис. 1.1. Схематичне зображення базового файлу

У таких файлових системах можна отримати вибірку запису з файлу по її заданому ключу. В цьому випадку файлова система підтримує в тому ж (або іншому, службовому) базовому файлі додаткові, невидимі користувачеві, службові структури даних.

1.2.2. Логічна структура файлових систем

У всіх сучасних файлових системах забезпечується багаторівневе іменування файлів за рахунок наявності у зовнішній пам'яті **каталогів** – додаткових файлів зі спеціальною структурою. Кожен каталог містить імена каталогів та/або файлів, що зберігаються в даному каталозі. Таким чином, повне ім'я файлу складається зі списку імен каталогів плюс назва файлу в каталозі, безпосередньо містить цей файл.

Підтримка багаторівневої схеми іменування файлів забезпечує декілька переваг, основним з яких є проста і зручна схема логічної класифікації файлів і генерації їхніх імен. Можна зіставити каталог або ланцюжок каталогів з користувачем, підрозділом, проектом і т. д. І потім утворювати в цьому каталозі файли або каталоги, не побоюючись колізій з іменами інших файлів або каталогів.

Різниця між способами іменування файлів в різних файлових системах полягає в тому, з чого починається цей ланцюжок імен. У будь-якому випадку перше ім'я повинно відповідати кореневому каталогу файлової системи. Питання полягає в тому, як зіставити це ім'я кореневому каталогу – де його шукати? У зв'язку з цим є два радикально різних підходи.

У багатьох системах управління файлами потрібно, щоб кожен архів файлів (повне дерево каталогів) цілком розташовувалось на одному дисковому пакеті або логічному диску – розділі фізичного дискового пакета, логічно яка подається у вигляді окремого диска за допомогою засобів операційної системи. У цьому випадку повне ім'я файлу починається з імені дискового пристрою, на якому встановлений відповідний диск. Такий спосіб іменування використовувався в файлових системах компаній IBM і DEC. Дуже близькі до цього і файлові системи, реалізовані в операційних системах сімейства Windows компанії Microsoft. Можна назвати таку організацію підтримкою ізольованих файлових систем.

Інший крайній варіант був реалізований в файлових системах операційної системи Multics. У цій системі був реалізований цілий ряд оригінальних ідей, але ми зупинимося тільки на особливостях організації архіву файлів.

У файловій системі Multics користувачам забезпечувалася можливість представляти всю сукупність каталогів і файлів у вигляді єдиного дерева. Повне ім'я файлу починалося з імені кореневого каталогу, і користувач не зобов'язаний був піклуватися про встановлення на дисковий пристрій будь-яких конкретних дисків. Сама система, виконуючи пошук файлу по його імені, запитувала у оператора установку необхідних дисків. Таку файлову систему можна назвати повністю централізованою.

Але в таких системах виникають суттєві проблеми, якщо потрібно перенести піддерево файлової системи на іншу обчислювальну установку. Оскільки файли і каталоги будь-якого логічного піддерева можуть бути фізично розкидані по різних дисковим пакетам і навіть магнітних стрічок, для такого перенесення потрібна спеціальна утиліта, що збирає всі об'єкти необхідного піддерева на одному зовнішньому носії, що не входить до складу штатних пристроїв централізованої файлової системи. Звичайно, навіть при наявності такої утиліти виконання процедури фізичної збірки потребує суттєвого часу.

Компромісне рішення застосовується в файлових системах ОС UNIX. На базовому рівні в цих файлових системах підтримуються ізольовані архіви файлів. Один з таких архівів оголошується кореневою файловою системою. Це робиться на етапі генерації операційної системи, і після запуску операційна система «знає», на якому дисковому пристрої (фізичному або логічному) розташовується коренева файлова система. Після запуску системи можна «змонтувати» кореневу файлову систему і ряд ізольованих файлових систем в одну загальну файлову систему. Технічно це здійснюється за допомогою створення в кореневій файловій системі спеціальних порожніх каталогів (точок монтування).

1.2.3. Авторизація доступу до файлів

Оскільки файлова система є загальним сховищем файлів, що належать, взагалі кажучи, різним користувачам, системи управління файлами повинні забезпечувати **авторизацію** доступу до файлів. У загальному вигляді підхід полягає в тому, що по відношенню до кожного зареєстрованого користувача даної обчислювальної системи для кожного існуючого файлу вказуються дії, які дозволені або заборонені даному користувачеві. Застосування мандатного способу захисту тягне за собою істотні накладні витрати, пов'язані з потребою зберігання надлишкової інформації і використанням цієї інформації для перевірки правочинності доступу.

Тому в більшості сучасних систем керування файлами застосовується підхід до захисту файлів, вперше реалізований в ОС UNIX (так званий **дискреційний** підхід). У цій системі кожному зареєстрованому користувачу відповідає пара цілочисельних ідентифікаторів: ідентифікатор групи, до якої відноситься користувач, і його власний ідентифікатор. Цими ж ідентифікаторами забезпечується кожен процес, запущений від імені даного користувача і має можливість звертатися до системних викликів файлової системи.

Відповідно, при кожному файлі зберігається повний ідентифікатор користувача (власний ідентифікатор плюс ідентифікатор групи), який створив цей файл, і позначається, які дії з файлом може виробляти він сам, які дії з файлом доступні для інших користувачів тієї ж групи і що можуть робити з файлом користувачі інших груп. Для кожного файлу контролюється можливість виконання трьох дій: читання, запис і виконання.

1.2.4. Синхронізація багатокористувацького режиму доступу

Якщо операційна система підтримує багатокористувацький режим, може виникнути ситуація, коли два або більше користувачів одночасно намагаються працювати з одним і тим же файлом. Якщо всі ці користувачі збираються тільки читати файл, нічого поганого не станеться. Але якщо хоча б один з них буде змінювати файл, для коректної роботи цієї групи потрібна взаємна синхронізація.

У файлових системах зазвичай застосовувався наступний підхід. В операції відкриття файлу (першої та обов'язкової операції, з якої повинен починатися сеанс роботи з файлом) крім інших параметрів вказувався режим роботи (читання або зміна). Якщо до моменту виконання цієї операції від імені деякого процесу А файл вже був відкритий деяким іншим процесом В, причому існуючий режим відкриття був несумісний з необхідним режимом (сумісні тільки режими читання), то в залежності від особливостей системи або процесу А повідомлялося про неможливість відкриття файлу в потрібному режимі, або процес А блокувався до тих пір, поки процес В не виконає операцію закриття файлу.

1.2.5. Області застосування файлів

Після короткого екскурсу в історію і сучасний стан файлових систем обговоримо можливі області їх застосування. Перш за все, звичайно, файли використовуються для зберігання текстових даних: документів, текстів програм і т. д.

Файли, що містять тексти програм, використовуються як вхідні файли компіляторів, які, в свою чергу, формують файли, що містять об'єктні модулі. З точки зору файлової системи об'єктні файли також мають дуже простий структурою – послідовність записів або байтів. Система програмування накладає на таку структуру більш складну і специфічну для цієї системи структуру об'єктного модуля. Підкреслимо, що логічна структура об'єктного модуля файлової системи невідома; ця структура підтримується інструментами системи програмування.

Аналогічно іде справа з файлами, які формувались редакторами зв'язків (редактор зв'язків повинен розуміти логічну структуру файлів об'єктних модулів) і містять образи виконуваних програм. Логічна структура таких файлів залишається відомою тільки редактору зв'язків і завантажувачу – програмою операційної системи. Загальна схема взаємодії програмних компонентів при побудові програми показана на рис. 1.2.

Ми коротко окреслили способи використання файлів в процесі розробки програм, але можна сказати, що ситуація аналогічна і в інших випадках: наприклад, при утворенні і використанні файлів, що містять графічну, аудіо- та відеоінформацію.

Таким чином, файлові системи зазвичай забезпечують зберігання слабо структурованої інформації, залишаючи подальшу структурування прикладним програмам.

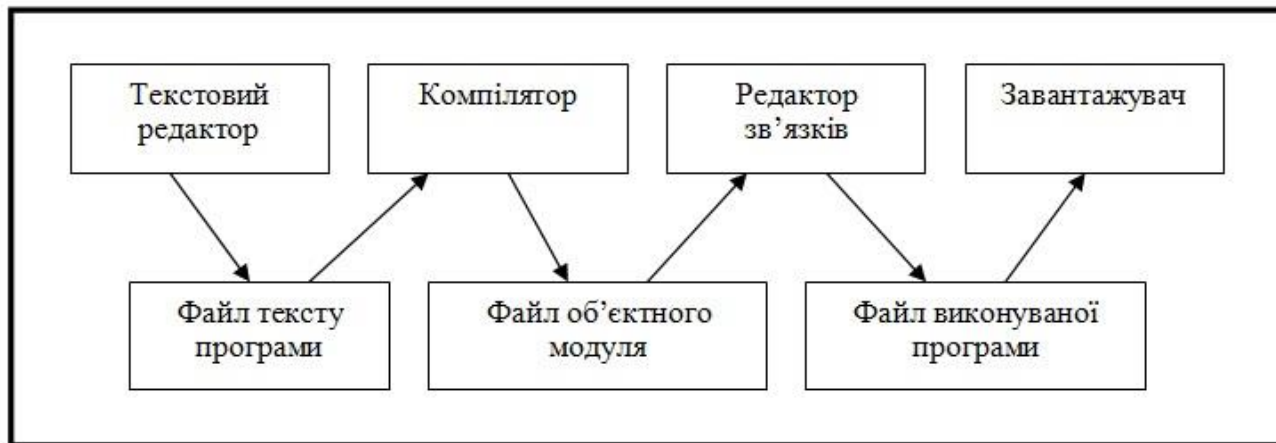


Рис. 1.2. Зв'язки між програмними компонентами в логічній структурі файлів

1.3. Потреби інформаційних систем

Чи задовольняють розглянуті вище базові можливості файлових систем потреби інформаційних систем? Типова інформаційна система (ІС), головним чином, орієнтована на зберігання, вибір і модифікацію даних відповідної прикладної області. Структура таких даних часто дуже складна, і, хоча структури даних різні в різних інформаційних системах, між ними часто буває багато спільного.

На початковому етапі використання обчислювальної техніки для побудови ІС проблеми структуризації даних вирішувалися індивідуально в кожній ІС. Проводилися необхідні надбудови над файловими системами (бібліотеки програм), як це робиться в компіляторах, редакторах тощо (рис. 1.3). Але оскільки ІС вимагають складних структур даних, ці додаткові індивідуальні засоби керування даними були істотною частиною ІС і практично повторювалися від однієї системи до іншої.

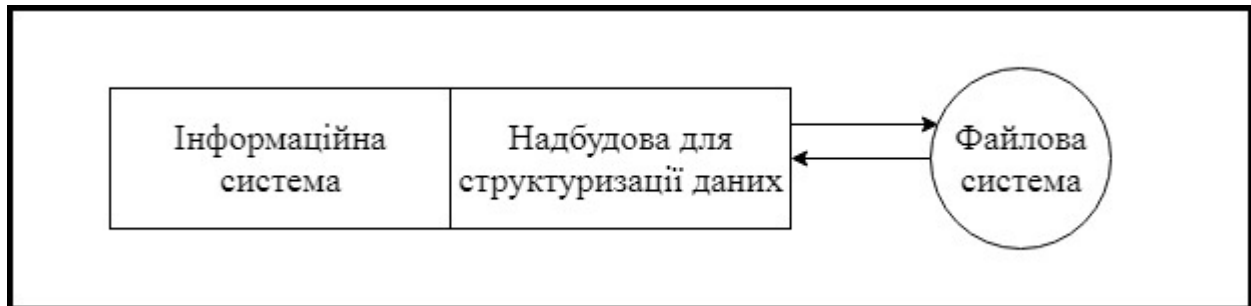


Рис. 1.3. Примітивна схема структуризації даних в інформаційній системі

Прагнення виділити загальну частину інформаційних систем, відповідальну за управління складно структурованими даними, стало першою спонукальною причиною створення **систем керування базами даних** (СКБД). Дуже скоро стало зрозуміло, що неможливо обійтися загальною бібліотекою програм (рис. 1.4), що реалізує над стандартної базової файлової системою більш складні методи зберігання даних.

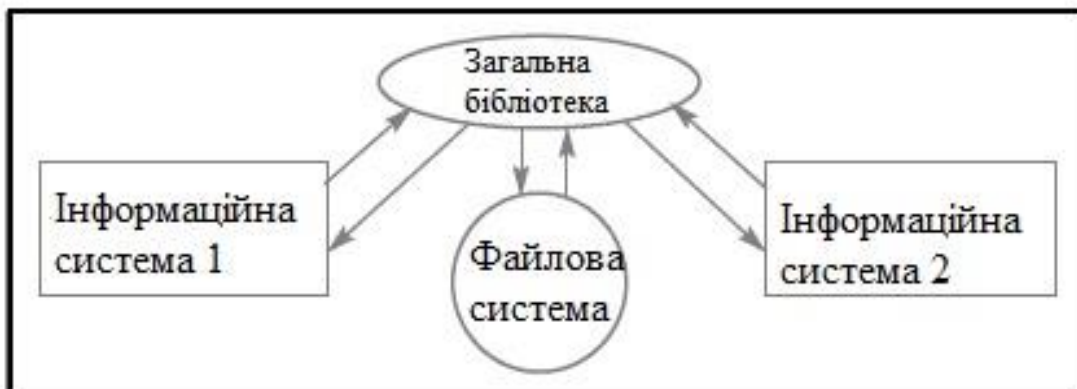


Рис. 1.4. Дві інформаційні системи із загальною бібліотекою

Пояснимо це на прикладі. Припустимо, що потрібно реалізувати просту інформаційну систему, яка підтримує облік службовців деякої організації. Система повинна виконувати наступні дії:

- видавати списки службовців по відділам;
- підтримувати можливість переведення службовця з одного відділу в інший;
- забезпечувати засоби підтримки прийому на роботу нових співробітників і звільнення працюючих службовців.

Крім того, для кожного відділу повинна підтримуватися можливість отримання:

- імені керівника відділу;
- загальної чисельності відділу;
- загальної суми зарплати службовців відділу, середнього розміру зарплати і т. д.

Для кожного службовця повинна підтримуватися можливість отримання:

- номера посвідчення по повному імені службовця (для простоти припустимо, що імена всіх службовців різні);
- повного імені за номером посвідчення;
- інформації про відповідність службовця займаній посаді та про розмір його зарплати.

1.3.1. Структури даних

Припустимо, що ми вирішили розробити цю ІС на файлову систему і користуватися одним файлом СЛУЖБОВЦІ, розширивши базові можливості файлової системи за рахунок спеціальної бібліотеки функцій. Оскільки мінімальної інформаційної одиницею в нашому випадку є службовець, в цьому файлі повинна міститися один запис для кожного службовця. Щоб можна було задовольнити зазначені вище вимоги, запис про службовця повинна мати такі поля:

- повне ім'я службовця (СЛУ_ІМЯ);
- номер його посвідчення (СЛУ_НОМЕР);
- дані про відповідність службовця займаній посаді (СЛУ_СТАТ; для простоти «так» або «ні»);
- розмір зарплати (СЛУ_ЗАРП);
- номер відділу (СЛУ_ОТД_НОМЕР).

Оскільки ми вирішили обмежитися одним файлом СЛУЖБОВЦІ, та ж запис повинна містити ім'я керівника відділу (СЛУ_ОТД_РУК). Інакше було б неможливо, наприклад, отримати ім'я керівника відділу з відомим номером.

Щоб ІС могла ефективно виконувати свої базові функції, необхідно забезпечити багатоключевий доступ до файлу СЛУЖБОВЦІ по унікальним ключам СЛУ_ІМЯ і СЛУ_НОМЕР (ключ називається унікальним, якщо його значення гарантовано різні у всіх записах файлу). Очевидно, що в іншому випадку для виконання найбільш часто використовуваних операцій отримання даних про конкретний службовця знадобиться послідовний перегляд в середньому половини записів файлу.

Крім того, повинна забезпечуватися можливість ефективного вибору всіх записів із загальним значенням СЛУ_ОТД_НОМЕР, тобто доступ за неунікальним ключем. Якщо не підтримувати спеціальний механізм доступу, то для отримання даних про відділ у цілому в загальному випадку буде потрібно повний перегляд файлу. Необхідна загальна структура файлу СЛУЖБОВЦІ показана на рис. 1.5. Але навіть в цьому випадку, щоб отримати чисельність відділу або загальний розмір зарплати, система повинна буде вибрати всі записи про службовців зазначеного відділу і порахувати відповідні загальні значення.

Таким чином, ми бачимо, що при реалізації навіть такої простої інформаційної системи на базі файлової системи виникають такі труднощі:

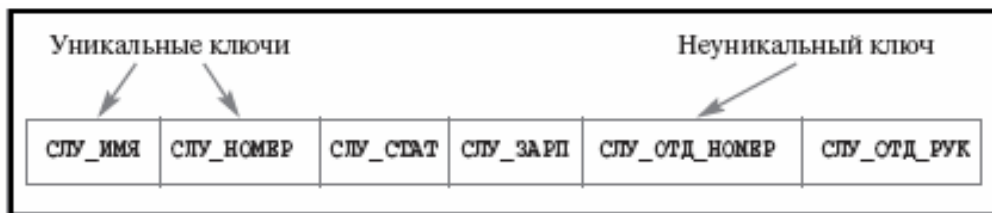


Рис. 1.5. Структура файлу СЛУЖБОВЦІ на рівні додатку (випадок одного файлу)

- потрібне створення досить складної надбудови для багатоключового доступу до файлів;
- веде до суттєвої надмірності даних (для кожного службовця повторюється ім'я керівника його відділу);
- слід дотримуватися масової вибірки і обчислень для отримання сумарної інформації про відділи.

Крім того, якщо в ході експлуатації системи буде потрібно, наприклад, забезпечити операцію видачі списків службовців, які отримують зазначену зарплату, то або доведеться при виконанні кожної такої операції повністю переглядати файл, або потрібно буде реструктурувати файл СЛУЖБОВЦІ, оголошуючи ключовим і поле СЛУ_ЗАРП.

Для поліпшення ситуації можна було б підтримувати два багато ключових файлів: СЛУЖБОВЦІ і ВІДДІЛИ. Перший файл повинен був би містити поля СЛУ_ІМЯ, СЛУ_НОМЕР,

СЛУ_СТАТ, СЛУ_ЗАРП і СЛУ_ОТД_НОМЕР, а другий – ОТД_НОМЕР, ОТД_РУК (номер посвідчення службовця, який є керівником відділу), ОТД_СЛУ_ЗАРП (загальний розмір зарплати службовців даного відділу) і ОТД_РАЗМЕР (загальне число службовців у відділі). Структура цих файлів показана на рис. 1.6. Введення цих двох файлів дозволило б подолати більшість незручностей, перелічених вище. Кожен з файлів містив би тільки не дубльовані інформацію, не з'являлася необхідність в динамічних обчисленнях сумарної інформації по відділах. Але зауважимо, що при такому переході наша інформаційна система повинна володіти деякими новими особливостями, що зближують її з СКБД.

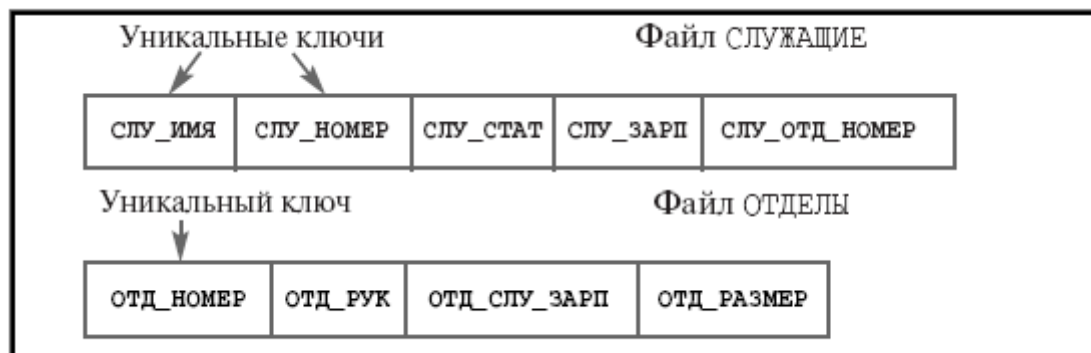


Рис. 1.6. Структура файлу СЛУЖБОВЦІ і ВІДДІЛИ на рівні додатку (випадок двох файлів)

1.3.2. Цілісність даних

Тепер спроектована система повинна «знати», що вона працює з двома інформаційно пов'язаними файлами (це крок у бік схеми бази даних), повинна мати інформацію про структуру і сенсі кожного поля. Наприклад, системі повинно бути відомо, що у полів СЛУ_ОТД_НОМЕР в файлі СЛУЖБОВЦІ і ОТД_НОМЕР в файлі ВІДДІЛИ один і той же сенс – це номер відділу.

Крім того, система повинна враховувати, що в ряді випадків зміна даних в одному файлі має автоматично викликати модифікацію другого файлу, щоб загальний вміст файлів було узгодженим. Наприклад, якщо на роботу приймається новий службовець, то потрібно додати запис в файл СЛУЖБОВЦІ, а також належним чином змінити поля ОТД_СЛУ_ЗАРП і ОТД_РАЗМЕР в запису файлу ВІДДІЛИ, відповідної відділу цього службовця. Більш точно, система повинна керуватися наступними правилами:

1. Якщо у файлі СЛУЖБОВЦІ міститься запис із значенням поля СЛУ_ОТД_НОМЕР, рівним n , то і в файлі ВІДДІЛИ повинен міститися запис із значенням поля ОТД_НОМЕР, також рівним n .
2. Якщо у файлі ВІДДІЛИ міститься запис із значенням поля ОТД_РУК, рівним m , то і в файлі СЛУЖАЩИЕ повинен міститися запис із значенням поля СЛУ_НОМЕР, також рівним m ; в наступних лекціях ми побачимо, що правила (1) і (2) є окремими випадками загального правила посилальної цілісності: поле СЛУ_ОТД_НОМЕР містить «посилання» на записи таблиці ВІДДІЛИ, і поле ОТД_РУК має «посилання» на записи таблиці СЛУЖБОВЦІ.
3. При будь-якому коректному стані інформаційної системи значення поля ОТД_СЛУ_ЗАРП будь-якого запису ОТД_К файлу ВІДДІЛИ має дорівнювати сумі значень поля СЛУ_ЗАРП всіх тих записів файлу СЛУЖБОВЦІ, в яких значення поля СЛУ_ОТД_НОМЕР збігається зі значенням поля ОТД_НОМЕР записи ОТД_К.
4. При будь-якому коректному стані інформаційної системи значення поля ОТД_РАЗМЕР будь-якого запису ОТД_К файлу ВІДДІЛИ має дорівнювати числу всіх тих записів файлу СЛУЖБОВЦІ, в яких значення поля СЛУ_ОТД_НОМЕР збігається зі значенням поля ОТД_НОМЕР записи ОТД_К; в наступних лекціях ми побачимо, що правила (3) і (4) являють собою приклади загальних обмежень цілісності бази даних.

Поняття *узгодженості*, або *цілісності*, даних є ключовим поняттям баз даних. Фактично, якщо ІС підтримує узгоджене зберігання даних в декількох файлах, можна говорити про те, що вона підтримує базу даних (БД). Якщо ж деяка допоміжна система керування даними дозволяє працювати з декількома файлами, забезпечуючи їх узгодженість, можна назвати її *системою керування базами даних* (СКБД).

Вже тільки вимога підтримки цілісності даних в декількох файлах не дозволяє при побудові інформаційної системи обійтися бібліотекою функцій: така система повинна володіти деякими власними даними (їх прийнято називати *метаданими*), що визначають цілісність даних. У нашому прикладі інформаційна система повинна окремо зберігати метадані про структуру файлів СЛУЖБОВЦІ і ВІДДІЛИ, а також правила, що визначають умови цілісності даних в цих файлах.

1.3.3. Мови запитів

Але забезпечення цілісності даних – це далеко не все, що зазвичай потрібно від СКБД. У нашому прикладі користувачеві ІС буде не дуже просто отримати, наприклад, загальну чисельність відділу, в якому працює Петро Іванович Сидоров. Доведеться спочатку дізнатися номер відділу, в якому працює зазначений службовець, а потім встановити чисельність цього відділу. Було б набагато простіше, якби СКБД дозволяла сформулювати такий запит на мові, ближчому користувачам. Такі мови називаються *мовами запитів до баз даних*. Наприклад, на мові запитів SQL (*structured query language* – мова структурованих запитів) Наш запит можна було б висловити в наступній формі (запит1):

```
SELECT ОТД_РАЗМЕР
FROM СЛУЖБОВЦІ, ВІДДІЛИ
WHERE СЛУ_ІМЯ = 'ПЕТРО ІВАНОВИЧ СИДОРОВ' AND
СЛУ_ОТД_НОМЕР = ОТД_НОМЕР;
```

Це приклад запиту на мові SQL з напівз'єднання: з одного боку, запит адресується до двох файлів – СЛУЖБОВЦІ і ВІДДІЛИ, але з іншого боку, дані вибираються тільки з файлу ВІДДІЛИ. Умова СЛУ_ОТД_НОМЕР = ОТД_НОМЕР всього лише «обмежує» цікавий для нас набір записів про відділи до запису, якщо Петро Іванович Сидоров дійсно працює на даному підприємстві. Якщо ж Петро Іванович Сидоров не працює на підприємстві, то умова СЛУ_ІМЯ = 'ПЕТРО ІВАНОВИЧ СИДОРОВ' не буде задовольнятися ні для одного запису файлу СЛУЖБОВЦІ, і тому запит поверне порожній результат.

Можлива й інша формулювання того ж запиту (запит2):

```
SELECT ОТД_РАЗМЕР
FROM ВІДДІЛИ
WHERE ОТД_НОМЕР =
(SELECT СЛУ_ОТД_НОМЕР
FROM СЛУЖБОВЦІ
WHERE СЛУ_ІМЯ = 'ПЕТРО ІВАНОВИЧ СИДОРОВ');
```

Це приклад запиту на мові SQL з вкладеним підзапитом. У вкладеному підзапиті вибирається значення поля СЛУ_ОТД_НОМЕР із запису файлу СЛУЖБОВЦІ, в якій значення поля СЛУ_ІМЯ дорівнює строковій константі 'ПЕТРО ІВАНОВИЧ СИДОРОВ'. Якщо такий запис існує, то вона єдина, оскільки поле СЛУ_ІМЯ є унікальним ключем файлу СЛУЖБОВЦІ. Тоді результатом виконання підзапиту буде єдине значення – номер відділу, в якому працює Петро Іванович Сидоров. У зовнішньому запиті це значення буде ключем доступу до файлу ВІДДІЛИ, і знову буде обрана тільки одна запис, оскільки поле ОТД_НОМЕР є унікальним ключем файлу ВІДДІЛИ. Якщо ж на даному підприємстві Петро Іванович Сидоров не працює, то підзапит поверне порожній результат, і зовнішній запит теж поверне порожній результат.

Наведені приклади показують, що при формулюванні запиту з використанням SQL можна не замислюватися про те, як буде виконуватися цей запит. Серед метаданих бази даних міститиметься інформація про те, що поле СЛУ_ІМЯ є ключовим для файлу СЛУЖБОВЦІ (тобто по заданому значенню імені службовця можна швидко знайти відповідний запис або переконатися в тому, що запис з таким значенням поля СЛУ_ІМЯ в файлі відсутній), а поле ОТД_НОМЕР – ключове для файлу ВІДДІЛИ (і більш того, обидва ключа в відповідних файлах є унікальними), і система сама скористається цим. Можна формально довести, що формулювання запит1 і запит2 еквівалентні, тобто незалежно від стану даних завжди виробляють один і той же результат.

Якщо ж, наприклад, виникне потреба в отриманні списку службовців, які не відповідають займаній посаді, то досить звернутися до системи із запитом (запит3):

```
SELECT СЛУ_ІМЯ, СЛУ_НОМЕР  
FROM СЛУЖБОВЦІ  
WHERE СЛУ_СТАТ = "НІ";
```

і система сама виконає необхідний повний перегляд файлу СЛУЖБОВЦІ, оскільки поле СЛУ_СТАТ не є ключовим, і іншого способу виконання не існує.

1.3.4. Транзакції, журналізація і багатокористувацький режим

Уявімо собі, що в первісній реалізації ІС, заснованої на використанні бібліотек розширених методів доступу до файлів, обробляється операція прийняття на роботу нового службовця. Дотримуючись вимог узгодженої зміни файлів, ІС вставляє новий запис в файл СЛУЖБОВЦІ і збирається модифікувати відповідний запис файлу ВІДДІЛИ (або вставляти в цей файл новий запис, якщо службовець є першим у своєму відділі), але саме в цей момент відбувається (наприклад) аварійне вимкнення живлення комп'ютера.

Очевидно, що після перезапуску системи її база даних буде знаходитися в неузгоджені стані (точно будуть порушені правила (3) і (4), а може бути, і правила (1) і (2)). Буде потрібно з'ясувати це (а для цього потрібно явно перевірити відповідність даних у файлах СЛУЖБОВЦІ і ВІДДІЛИ) і привести дані в узгоджене стан. Перевірку і корекцію можна виконати, наприклад, наступним чином.

1. Згрупувати записи файлу СЛУЖБОВЦІ за значеннями поля СЛУ_ОТД_НОМЕР.
2. Для кожної групи перевірити, чи існує в файлі ВІДДІЛИ запис, значення поля ОТД_НОМ якої дорівнює значенню поля СЛУ_ОТД_НОМЕР записів даної групи.
3. Якщо такого запису в файлі ВІДДІЛИ немає, то виключити групу з файлу СЛУЖБОВЦІ і перейти до обробки наступної групи; інакше порахувати кількість записів в групі і обчислити сумарне значення заробітної плати; оновити отриманими значеннями поля ОТД_РАЗМЕР і ОТД_СЛУ_ЗАРП відповідного запису файлу ВІДДІЛИ і перейти до обробки наступної групи.

Справжні СКБД беруть таку роботу на себе, підтримуючи **транзакційне управління і журналізацію змін бази даних**. Прикладна система не зобов'язана піклуватися про підтримку коректності стану бази даних, хоча і повинна знати, які ланцюжка операцій зміни даних є допустимими.

Уявімо тепер, що в інформаційній системі потрібно забезпечити паралельну (наприклад, багатотермінальну) роботу з базою даних службовців і відділів. Якщо спиратися тільки на використання файлів, то для забезпечення коректності на весь час модифікації будь-якого з двох файлів доступ інших користувачів до цього файлу буде блокований (згадайте можливості файлових систем щодо синхронізації паралельного доступу). Таким чином, зарахування на роботу Петра Івановича Сидорова істотно загальмує отримання інформації про службовця Іван Сидорович Петрове, навіть якщо вони працюють в різних відділах. Справжні СКБД забезпечують набагато більш тонку синхронізацію паралельного доступу до даних.

1.3.5. СКБД як незалежний системний компонент

До сих пір ми не виділяли СКБД зі складу інформаційної системи, маючи на увазі загальну організацію системи, подібну до тієї, яка показана на рис. 1.7. Тут видно два дефекти. По-перше, очевидно, що СКБД повинна підтримувати досить розвинену функціональність. Повторювати цю функціональність в кожній інформаційній системі нерозумно. З іншого боку, незрозуміло, яким чином можна забезпечити готовий до використання компонент СКБД, який можна було б вбудовувати в інформаційні системи. По-друге, вже має бути зрозуміло, що набір файлів можна назвати базою даних тільки при наявності метаданих. На рис. 1.7 метадані є приналежністю інформаційної системи, і тому, наприклад, файли СЛУЖБОВЦІ і ВІДДІЛИ можна ефективно використовувати тільки через нашу гіпотетичну систему реєстрації службовців.



Рис. 1.7. СКБД в складі інформаційної системи

Припустимо, що підприємству потрібна ще й інформаційна бухгалтерська система. Очевидно, що для її роботи також будуть потрібні дані про службовців і відділах. При показаній вище організації системи можливі два варіанти виконання завдання, жоден з яких не є задовільним.

1. Впровадити бухгалтерську систему до складу системи реєстрації службовців. Але ж, як правило, бухгалтерські системи купуються у вигляді готових і окремих продуктів, не пристосованих до подібного «впровадження».
2. Скопіювати метадані системи реєстрації службовців в бухгалтерську систему. Але метадані (як і дані) не обов'язково є статичними. Структура бази даних може з часом змінюватися, можуть зникати одні правила цілісності і з'являтися інші. Як погоджувати копії метаданих, підтримувані незалежними інформаційними системами?

Так ми приходимо до організації системи, показаної на рис. 1.8.

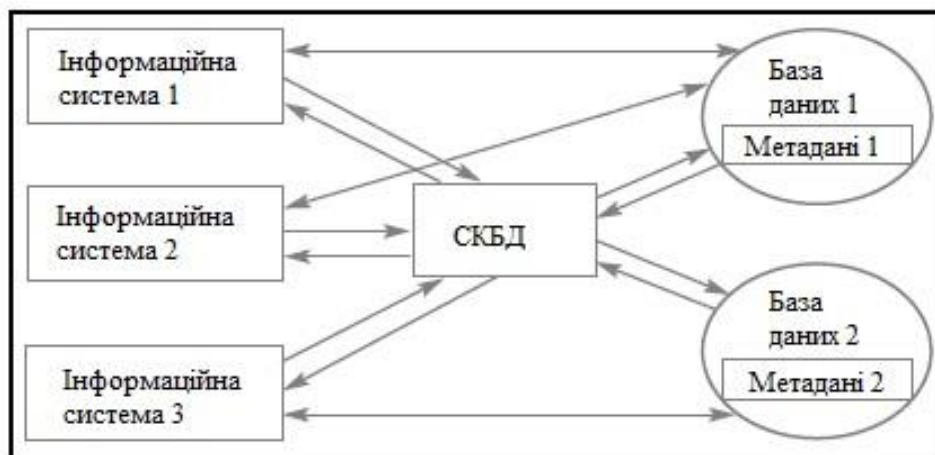


Рис. 1.8. Окрема СКБД і бази даних з метаданими

Тут ми бачимо три ІС, які через одну СКБД працюють з двома різними базами даних, причому перша і друга системи працюють із загальною базою даних. Це можливо, оскільки метадані кожної бази даних містяться в самих базах даних, і досить лише вказати СКБД, з якою базою даних бажає працювати для цієї програми.

Оскільки СКБД функціонує окремо від додатків, і її робота з базами даних регулюється метаданими, спільне використання однієї бази даних двома ІС не викличе втрати узгодженості даних, то доступ до даних буде належним чином синхронізуватися.

Зауважимо, що рис. 1.8 впритул наближає нас до найбільш поширеної архітектури *«клієнт-сервер»*. СКБД відіграє роль «сервера», обслуговуючого кількох «клієнтів» – прикладних інформаційних систем.

Таким чином, СКБД вирішують безліч проблем, які важко або взагалі неможливо вирішити при використанні файлових систем. При цьому існують додатки, для яких цілком достатньо файлів; додатки, для яких необхідно вирішувати, який рівень роботи з даними у зовнішній пам'яті для них потрібно, і додатки, для яких, безумовно, потрібні бази даних.

2. ПОНЯТТЯ МОДЕЛІ ДАНИХ. РІЗНОВИДИ МОДЕЛЕЙ ДАНИХ

Історію технології БД прийнято відраховувати з початку 1960-х рр., коли з'явилися перші спроби створення спеціальних програмних засобів управління базами даних. За минулі десятиліття виникали і використовувалися різні підходи до організації баз даних. Для опису та порівняння деяких з них скористаємося поняттям моделі даних (структури бази даних), запропонованим в 1969 р. Едгаром Коддом [1]. Кодд ввів це поняття для опису конкретного реляційного підходу до організації БД. Відповідно, він говорив про реляційної моделі даних, різних теоретичних і реалізаційних аспектів якої в основному присвячений цей курс. Однак поняття моделі даних виявилось зручним не тільки для опису реляційного підходу і порівняння реалізацій реляційних СКБД, але і для подання і зіставлення інших підходів до організації баз даних.

2.1. Поняття бази даних

Можливо, ви ще не знаєте, що входить в поняття бази даних, але те, що ви ними постійно користуєтеся абсолютно точно. Кожен раз, коли ви щось шукаєте в системі пошуку, ви використовуєте базу даних. Коли ви вводите свої логін і пароль для входу на який-небудь сервіс, вони порівнюються зі значеннями, які зберігаються в базі даних цього сервісу.

Незважаючи на те, що ми постійно використовуємо бази даних, для багатьох залишається незрозумілим, що ж це таке насправді. І пов'язано це частково з тим, що одні й ті ж терміни, що відносяться до баз даних, використовуються людьми для визначення абсолютно різних речей.

Комп'ютери були створені для вирішення обчислювальних завдань, проте з часом вони все частіше стали використовуватися для побудови систем обробки документів, а точніше, що міститься в них інформації. Такі системи зазвичай і називають інформаційними.

Іншими словами, інформаційна система вимагає створення в пам'яті ЕОМ динамічно оновлюваної моделі зовнішнього світу з використанням єдиного сховища – бази даних.

База даних – набір відомостей, що зберігаються деяким впорядкованим способом. Можна порівняти базу даних з шафою, в якому зберігаються документи. Іншими словами, база даних це сховище даних. Самі по собі бази даних не представляли б інтересу, якби не було систем управління базами даних.

Система керування базами даних – це сукупність мовних і програмних засобів, яка здійснює доступ до даних, дозволяє їх створювати, змінювати і видаляти, забезпечує безпеку даних і т.д. Загалом СКБД – це система, що дозволяє створювати бази даних і маніпулювати даними з них. А здійснює цей доступ до даних СКБД за допомогою спеціальної мови – SQL.

SQL – це мова структурованих запитів, основним завданням якого є надання простого способу зчитування і запису інформації в базу даних.

Найпростіша схема роботи з базою даних виглядає приблизно так (рис. 2.1):



Рис. 2.1. Найпростіша схема роботи з базою даних

За характером використання СКБД ділять на:

- однокористувацькі, призначені для створення і використання БД на персональному комп'ютері;
- розраховані на багато користувачів, призначені для роботи з єдиною БД декількох комп'ютерів, об'єднаних в локальні або глобальні мережі.

Розподіл за характером використання можна представити наступною схемою (рис. 4.2).

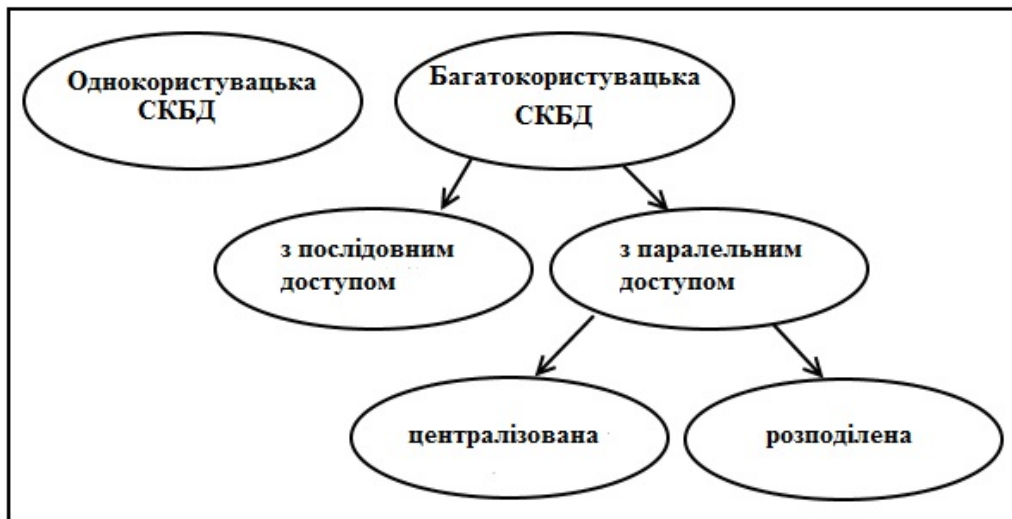


Рис.2.2. Характер використання СКБД

Найбільш відомі однокористувацькі СКБД – Microsoft Visual FoxPro і Access, багатокористувацькі – MS SQL Server, Oracle та MySQL.

2.2. Модель даних

У моделі даних (структурі даних) описується деякий набір родових понять і ознак, якими повинні володіти всі конкретні СКБД і керовані ними бази даних, якщо вони ґрунтуються на цій моделі. Наявність моделі даних дозволяє порівнювати конкретні реалізації, використовуючи один спільну мову.

Модель даних – це засіб абстракції, що дозволяє відобразити інформаційний зміст даних. У самому простому вигляді модель даних може бути представлена простим переліком тієї інформації, яка повинна зберігатися в інформаційній системі. Але так як дані, що представляють цю інформацію, практично завжди мають складну структуру і тісно взаємопов'язані, то необхідно представити будь-яким чином також і їх структуру. Інакше кажучи, модель даних являє собою сукупність правил, що можуть бути використані для опису структури даних.

Моделей для відображення однієї і тієї ж інформації можна придумати безліч. Критерієм вибору буде максимум кількості інформації, що витягується з даних. Щоб зрозуміти процес побудови моделі даних необхідно знати ряд термінів, які застосовуються при описі і поданні даних.

Створюючи базу даних, ми прагнемо впорядкувати інформацію за різними ознаками для того, щоб потім отримувати від неї необхідні нам дані в будь-якому поєднанні. Зробити це можливо, тільки якщо дані структуровані.

Структурування – це набір угод про способи представлення даних. Зрозуміло, що структурувати інформацію можна по-різному. Залежно від структури розрізняють ієрархічну, мережеву, реляційну, об'єктно-орієнтовану і гібридну модель даних (баз даних). Найпопулярнішою на сьогоднішній день є реляційна структура, тому про інших опишемо лише загальні їх характеристики.

Хоча поняття моделі даних було введено Коддом, найбільш поширене трактування моделі даних, очевидно, належить Крістоферу Дейту, який відтворює її стосовно до реляційних БД практично у всіх своїх книгах (див., наприклад, [2]). Згідно Дейту реляційна модель складається з трьох частин, що описують різні аспекти реляційного підходу: структурної частини, маніпуляційної частини і цілісної частини.

У структурної частини моделі даних фіксуються основні логічні структури даних, які можуть застосовуватися на рівні користувача при організації БД, відповідних даної моделі. Наприклад, в моделі даних SQL основним видом структур бази даних є таблиці, а в об'єктній моделі даних – об'єкти раніше визначених типів.

Маніпуляційна частина моделі даних містить специфікацію одного або декількох мов, призначених для написання запитів до БД. Ці мови можуть бути абстрактними, що не володіють точно проробленим синтаксисом (що властиво мовами реляційної алгебри і реляційного числення, що використовуються в реляційній моделі даних), або закінченими виробничими мовами (як у випадку моделі даних SQL). Основне призначення маніпуляційної частини моделі даних – забезпечити еталонний «модельний» мову БД, рівень виразності якого повинен підтримуватися в реалізаціях СКБД, відповідних даної моделі.

Нарешті, в цілісній частині моделі даних (яка явно виділяється не у всіх відомих моделях) специфікуються механізми обмежень цілісності, які обов'язково повинні підтримуватися в усіх реалізаціях СКБД, відповідних даної моделі. Наприклад, в цілісній частині реляційної моделі даних категорично потрібна підтримка обмеження первинного ключа в будь-якої змінної відношення, а аналогічну вимогу до таблиць в моделі даних SQL відсутній.

У цій лекції ми застосуємо поняття моделі даних для огляду як підходів, що передували появі реляційних баз даних, так і підходів, які виникли пізніше і в наш час.

2.3. Ранні моделі даних

Почнемо з розгляду загальних підходів до організації трьох типів ранніх систем, а саме, систем, заснованих на інвертованих списках, ієрархічних і мережових систем управління базами даних. В цілому ранні системи можна охарактеризувати наступним чином:

1. Ці системи активно використовувалися протягом багатьох років, задовго до появи працездатних реляційних СКБД.
2. Усі ранні системи не ґрунтувалися на будь-яких абстрактних моделях. Поняття моделі даних фактично увійшло в побут фахівців в області БД тільки разом з реляційним підходом. Абстрактні уявлення ранніх систем з'явилися пізніше на основі аналізу та виявлення загальних ознак у різних конкретних систем.
3. У ранніх системах доступ до БД проводився на рівні записів. Користувачі цих систем здійснювали явну навігацію в БД, використовуючи мови програмування, розширені функціями СКБД. Інтерактивний доступ до БД підтримувався тільки шляхом створення відповідних прикладних програм з власним інтерфейсом.
4. Навігаційна природа ранніх систем і доступ до даних на рівні записів змушували користувачів самих виробляти всю оптимізацію доступу до БД, без будь-якої підтримки системи.
5. Після появи реляційних систем більшість ранніх систем були оснащені «реляційними» інтерфейсами. Однак в більшості випадків це не зробило їх по-справжньому реляційними системами, оскільки залишалася можливість маніпулювати даними в природному для них режимі.

2.3.1. Модель даних інвертованих таблиць

До числа найбільш відомих і типових представників систем, в основі яких лежить ця модель даних, відносяться СКБД Datascom/DB, виведена на ринок в кінці 1960-х рр. компанією Applied Data Research, Inc. (ADR) і належить в даний час компанії Computer Associates, і Adabas (ADApable DAtabase System), яка була розроблена компанією Software AG в 1971 р. СКБД Datascom/DB і до сих пір є її основним продуктом.

Організація доступу до даних на основі інвертованих таблиць використовується практично у всіх сучасних реляційних СКБД, але в цих системах користувачі не мають безпосереднього доступу до інвертованих таблицями (індексам). Коли ми будемо розглядати внутрішні інтерфейси реляційних СКБД, можна буде побачити, що вони дуже близькі до призначених для користувача інтерфейсів систем, заснованих на інвертованих таблицях.

Структури даних. База даних в моделі інвертованих таблиць схожа на БД в моделі SQL, але з тією відмінністю, що користувачам видно і збережені таблиці, і шляхи доступу до них. При цьому:

1. Рядки таблиць упорядковуються системою в деякій фізичній, видимій користувачам послідовності.
2. Фізична упорядкованість рядків всіх таблиць може визначатися і для всієї БД (так робиться, наприклад, в Datacom / DB).
3. Для кожної таблиці можна визначити довільне число ключів пошуку, для яких будуються індекси. Ці індекси автоматично підтримуються системою, але явно видно користувачам.

Маніпулювання даними. Підтримуються два класи операцій:

1. Операції, що встановлюють адресу записи і розбиваються на два підкласу:
 - прямі пошукові оператори (наприклад, встановити адресу першого запису таблиці по деякому шляху доступу);
 - оператори, що встановлюють адресу записи при вказівці відносної позиції від попередньої записи по деякому шляху доступу.
2. Операції над адресованими записами.

Ось типовий набір операцій:

- LOCATE FIRST – знайти перший запис таблиці *T* у фізичному порядку (повертається адреса записи);
- LOCATE FIRST WITH SEARCH KEY EQUAL – знайти перший запис таблиці *T* із заданим значенням ключа пошуку *k* (Повертається адреса записи);
- LOCATE NEXT – знайти перший запис, наступну за записом з заданим адресою в заданому шляху доступу (повертається адреса записи);
- UPDATE – оновити запис з вказаною адресою;
- DELETE – видалити запис з вказаною адресою;
- STORE – включити запис в зазначену таблицю (операція генерує і повертає адресу записи).

Обмеження цілісності. Загальні правила визначення цілісності БД відсутні. У деяких системах підтримуються обмеження унікальності значень деяких полів, але в основному вся підтримка цілісності даних покладається на прикладну програму.

2.3.2. Ієрархічна модель даних

Типовим представником (найбільш відомим і поширеним) є СКБД IMS (Information Management System) компанії IBM. Перша версія системи з'явилася в 1968 р.

Ієрархічні структури даних. Ієрархічна БД складається з упорядкованого набору кількох примірників одного типу дерева. Тип дерева складається з одного «кореневого» типу запису і упорядкованого набору з нуля або більше типів піддерев (кожне з яких є певним типом дерева). Тип дерева в цілому являє собою ієрархічно організований набір типів записи.

На рис. 2.3 показаний приклад типу дерева (схеми ієрархічної БД). Тут тип запису Відділ є предком для типів записів Керівник і Службовці, а Керівник і Службовці – нащадки типу запису Відділ. Сенс полів типів записів в основному повинен бути зрозумілий за їхніми іменами. Поле Рук_Отдел типу записи Керівник містить номер відділу, в якому працює службовець, який є даними керівником (передбачається, що він працює не обов'язково в тому ж відділі, яким керує). Між типами записи підтримуються зв'язки.

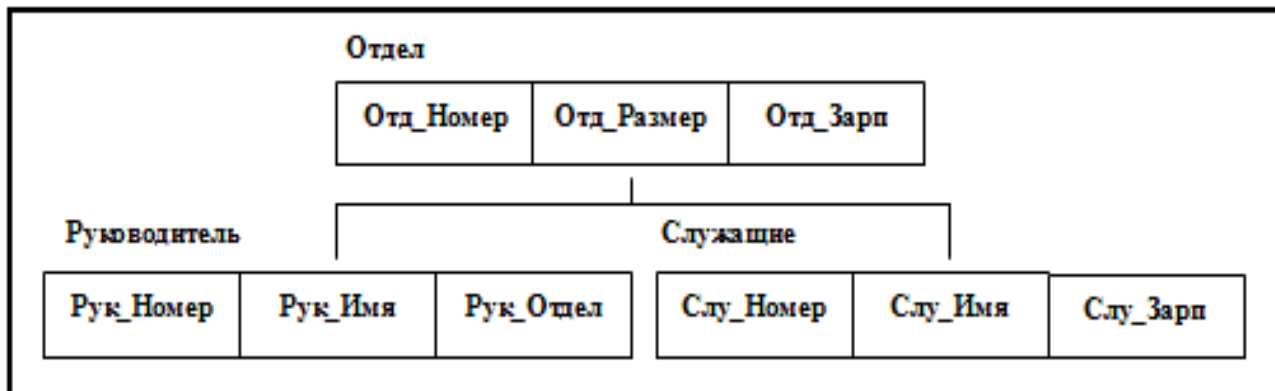


Рис. 2.3. Приклад схеми ієрархічної БД

База даних з такою схемою могла б виглядати так, як показано на рис. 2.4 (ми покажемо один екземпляр дерева).

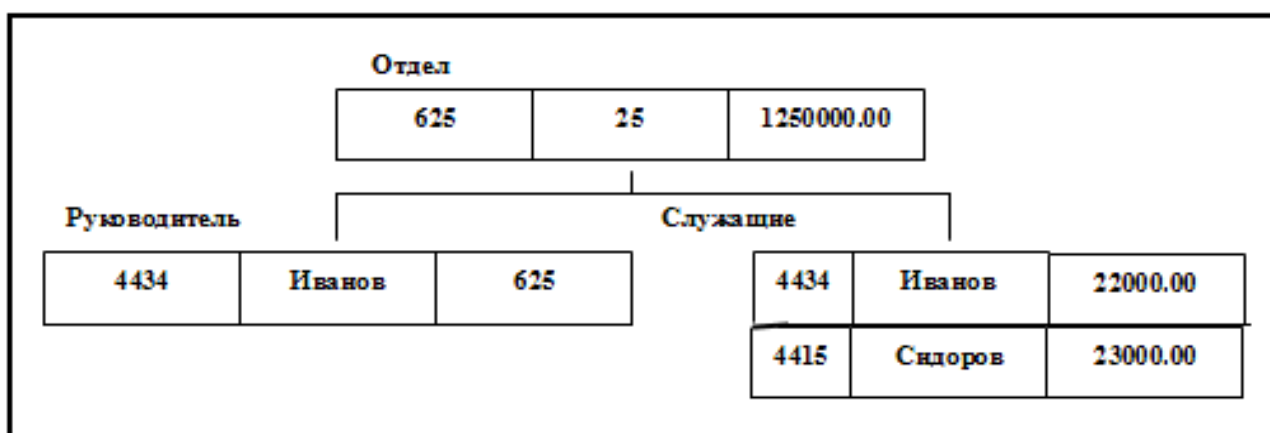


Рис. 2.4. Приклад ієрархічної бази даних

Всі екземпляри даного типу нащадка із загальним примірником типу предка називаються близнюками. Для ієрархічної бази даних визначається повний порядок обходу дерева: зверху-вниз, зліва-направо. Зауважимо, що в термінології IMS замість терміна «запис» використовувався термін «сегмент», а під записом бази даних розумілося все дерево сегментів.

Маніпулювання даними. Прикладами типових операцій маніпулювання ієрархічно організованими даними можуть бути наступні:

- знайти зазначений примірник типу дерева БД (наприклад, відділ 310);
- перейти від одного примірника типу дерева до іншого;
- перейти від екземпляра одного типу записи до примірника іншого типу записи всередині дерева (наприклад, перейти від відділу до першого співробітника);
- перейти від одного запису до іншого в порядку обходу ієрархії;
- вставити новий запис у вказану позицію;
- видалити поточний запис.

Обмеження цілісності. В ієрархічній моделі даних автоматично підтримується цілісність посилань між предками і нащадками. Основне правило: ніякої нащадок не може існувати без свого батька. Зауважимо, що аналогічна підтримка цілісності по посиланнях між записами без зв'язку «предок-нащадок», не забезпечується. Прикладом такої «зовнішньої» посилання є вміст поля Рук_Отдел в екземплярі типу записи Керівник.

2.3.3. Мережева модель даних

Типовим представником систем, заснованих на мережевій моделі даних, є СКБД IDMS (Integrated Database Management System), розроблена компанією Cullinet Software, Inc. і спочатку орієнтована на використання на основному комплекті компанії IBM. Архітектура системи заснована на пропозиціях Data Base Task Group (DBTG) організації CODASYL (COConference on DATA SYstems Languages), яка відповідала за визначення мови програмування COBOL. Звіт DBTG був опублікований в 1971 р., і незабаром після цього з'явилося кілька систем, що підтримують архітектуру CODASYL, серед яких була присутня і СКБД IDMS. В даний час IDMS належить компанії Computer Associates. До відомих мережевих систем управління базами даних відносяться: DBMS, IDMS, TOTAL, VISTA, МЕРЕЖА, КОМПАС та ін.

Мережеві структури даних. Мережевий підхід до організації даних є розширенням ієрархічного підходу. В ієрархічних структурах запис-нащадок повинен мати в точності одного предка; в мережевій структурі даних у нащадка може бути будь-яке число предків (рис. 2.5). Така мережа дозволяє реалізовувати відношення М: М («багато до багатьох») на всіх зв'язках.

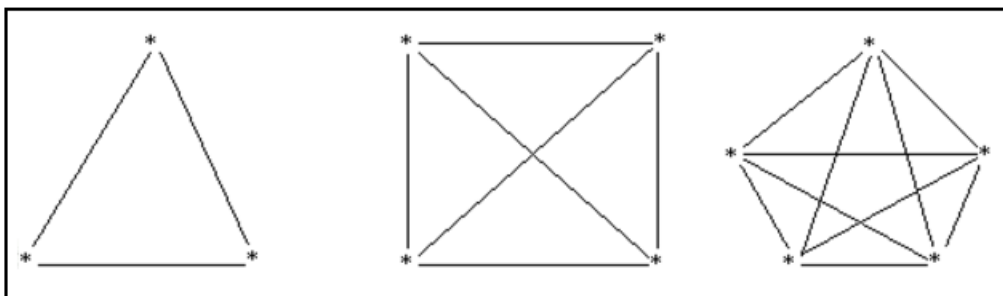


Рис. 2.5. Подання зв'язків в мережевій моделі даних

Мережева БД складається з набору записів і набору зв'язків між цими записами, а якщо говорити більш точно, з набору примірників кожного типу із заданого в схемі БД набору типів запису і набору примірників кожного типу із заданого набору типів зв'язку.

Тип зв'язку визначається для двох типів запису: предка і нащадка. Примірник типу зв'язку складається з одного примірника типу запису предка і упорядкованого набору примірників типу запису нащадка. Для даного типу зв'язку L з типом запису предка P і типом запису нащадка C повинні виконуватися наступні дві умови:

- кожен екземпляр типу запису P є предком тільки в одному екземплярі типу зв'язку L ;
- кожен екземпляр типу запису C є нащадком не більше ніж в одному екземплярі типу зв'язку L .

На формування типів зв'язку не накладаються особливі обмеження; можливі, наприклад, такі ситуації:

- тип запису нащадка в одному типі зв'язку $L1$ може бути типом запису предка в іншому типі зв'язку $L2$ (як в ієрархії);
- даний тип запису P може бути типом запису предка в будь-якій кількості типів зв'язку;
- даний тип запису P може бути типом запису нащадка в будь-якій кількості типів зв'язку;
- може існувати будь-яке число типів зв'язку з одним і тим же типом запису предка і одним і тим же типом запису нащадка; і якщо $L1$ і $L2$ – два типи зв'язку з одним і тим же типом запису предка P і одним і тим же типом запису нащадка C , то правила, за якими утворюється споріднення, в різних зв'язках можуть відрізнятися;
- типи записів X і Y можуть бути предком і нащадком в одній зв'язку і нащадком і предком – в іншому;
- предок і нащадок можуть бути одного типу запису.

На рис. 2.6 показані три типи записів (Відділ, Службовці і Керівник) і три типи зв'язку (Складається із службовців, має керівника і є службовцем).

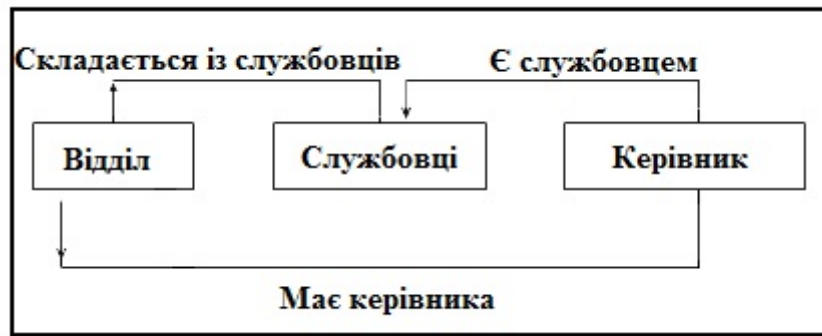


Рис. 2.6. Приклад схеми мережевої бази даних

У типі зв'язку *Складається із службовців* типом запису-предком є Відділ, А типом запису-нащадком – службовці (Екземпляр цього типу зв'язку зв'язує екземпляр типу запису Відділ з багатьма екземплярами типу запису службовці, Які відповідають усім службовцям даного відділу). У типі зв'язку *має керівника* типом записи-предком є Відділ, А типом записи-нащадком – керівник (Екземпляр цього типу зв'язку зв'язує екземпляр типу запису Відділ з одним екземпляром типу запису керівник, Відповідним керівнику даного відділу). Нарешті, в типі зв'язку *є службовцям* типом запису-предком є керівник, А типом запису-нащадком – службовці (Екземпляр цього типу зв'язку зв'язує екземпляр типу запису керівник з одним екземпляром типу запису службовці, Відповідним тому службовцю, яким є даний керівник).

Маніпулювання даними. Ось приблизний набір операцій маніпулювання даними:

- знайти певний запис в наборі однотипних записів (наприклад, службовця з ім'ям Хоменко);
- перейти від предка до першого нащадку за деякою зв'язку (наприклад, до першого службовцю відділу 625);
- перейти до наступного нащадку у деякому зв'язку (наприклад, від Іванова до Сидорову);
- створити новий запис;
- знищити запис;
- модифікувати запис;
- включити в зв'язок;
- виключити з зв'язку і т.д.

Обмеження цілісності. Є (необов'язкова) можливість вимагати для конкретного типу зв'язку відсутність нащадків, які не беруть участі ні в одному екземплярі цього типу зв'язку (як в ієрархічній моделі).

2.4. Неформальний вступ до реляційної моделі даних

Як вже говорилося на початку цієї лекції, основні ідеї реляційної моделі даних були запропоновані Едгаром Коддом в 1969 р. Слід зауважити, що, незважаючи на загальновизнану важливість цієї і наступних робіт Кодда, присвячених реляційної моделі даних, ці роботи писалися на ідейному рівні, що не були (за теперішніми мірками) глибоко технічно опрацьованими, у багатьох важливих місцях допускали неоднозначне тлумачення, і тому ці роботи неможливо було використовувати як безпосереднє керівництво для реалізації СКБД, що підтримує реляції іонну модель.

За минулі десятиліття реляційна модель розвивалася в двох напрямках. Перший напрямок заклав знаменитий експериментальний проект компанії IBM System R. У цьому проекті виникла мова SQL, спочатку заснований на ідеях Кодда (який також працював в IBM), але такий, що порушує деякі принципи приписи реляційної моделі. До теперішнього часу в чинному стандарті мови SQL, по суті, специфікована деяка власна, закінчена модель даних, огляд якої ми наведемо в наступному розділі цієї лекції.

Другий напрямок, починаючи з 1990-х рр., Очолив Крістофер Дейт, до якого пізніше приєднався Хью Дарвен. Обидва цих вчених також працювали в компанії IBM і до 1990-х рр. внесли великий вклад в розвиток мови SQL. Однак в 1990-і рр. Дейт і Дарвен прийшли до висновку, що спотворення реляційної моделі даних, властиві мові SQL, досягли настільки високого рівня, що прийшов час запропонувати альтернативу, що спирається на неспотворені ідеї Едгара Кодда і забезпечує всі можливості як SQL, так і об'єктно-орієнтованого підходу до організації баз даних і СКБД.

Нові ідеї Дейта і Дарвена були вперше викладені в їх Третньому маніфесті [3], А пізніше на основі цих ідей була специфікована модель даних [4]. Автори вважають, що в роботі [4], насправді, наводиться лише сучасна і повна інтерпретація ідей Кодда. У наступних лекціях під час обговорення реляційної моделі ми будемо використовувати, в основному, інтерпретацію Дейта і Дарвена.

У цій же лекції ми спочатку наведемо в даному розділі короткий і неформальний огляд основних ідей реляційної моделі в тому вигляді, в якому вона була запропонована Коддом, а в наступному розділі також коротко і неформально обговоримо пропозиції Дейта і Дарвена.

2.4.1. Реляційні структури даних

Основна ідея Кодда полягала в тому, щоб вибрати в якості родової логічної структури зберігання даних структуру, яка, з одного боку, була б досить зручною для більшості додатків і, з іншого боку, допускала б можливість виконання над базою даних ненавігаційній операцій. Ієрархічні і, особливо, мережеві структури даних є навігаційними за своєю природою. Ненавігаційному використанню таблиць заважає впорядкованість їх стовпців і, особливо, рядків.

По суті, Кодд запропонував використовувати в якості родової структури БД «таблиці», в яких і стовпці, і рядки не є впорядкованими. Легко бачити, що така «таблиця» з множини стовпців $\{A_1, A_2, \dots, A_n\}$, в якій кожен стовпець A_i може містити значення з множини $T_i = \{V_{i1}, V_{i2}, \dots, V_{im}\}$ (всі множини кінцеві), в математичному сенсі являє собою відношення над множинами $\{T_1, T_2, \dots, T_n\}$. Нагадаємо, що в математиці відношенням над множинами $\{T_1, T_2, \dots, T_n\}$ називається підмножина декартового добутку цих множин. Тому для позначення родової структури Кодд став використовувати термін відношення (relation), а для позначення елементів відношення – термін кортеж. Відповідно, модель даних отримала назву реляційної моделі.

Схема БД в реляційної моделі даних – це набір іменованих заголовків відношень виду $H_i = \{ \langle A_{i1}, T_{i1} \rangle, \langle A_{i2}, T_{i2} \rangle, \dots, \langle A_{ini}, T_{ini} \rangle \}$ T_i називається доменом атрибуту A_i . За Коддом, кожен домен T_i є підмножиною значень деякого базового типу даних $T_i +$, а значить, до її елементів застосовні всі операції цього базового типу. В кінці 1960-х рр. базовими типами даних вважалися типи даних поширених тоді мов програмування (в IBM найбільш популярними мовами були PL1 і COBOL).

Реляційна база даних в кожен момент часу являє собою набір іменованих відношень, кожне з яких має заголовком, таким як він визначений в схемі БД, і тілом. Ім'я відношення R_i збігається з ім'ям заголовка цього відношення HR_i тіло відношення BR_i – це множина кортежів виду $\{ \langle A_{i1}, T_{i1}, v_{i1} \rangle, \langle A_{i2}, T_{i2}, v_{i2} \rangle, \dots, \langle A_{ini}, T_{ini}, v_{ini} \rangle \}$, $det_{ij} \in T_i$.

2.4.2. Маніпулювання реляційними даними

Оскільки в реляційній моделі даних заголовок і тіло будь-якого відношення являють собою множини, то до відношень застосовні звичайні теоретико-множинні операції: об'єднання, перетин, віднімання, взяття декартового добутку. Нагадаємо, що для двох множин $S_1\{s_1\}$ і $S_2\{s_2\}$ результатом операції об'єднання цих двох множин $S_1 \cup S_2$ є множина $S\{s\}$ така, що $s \in S_1$ або $s \in S_2$. Результатом операції перетину $S_1 \cap S_2$ є множина $S\{s\}$ така, що $s \in S_1$ і $s \in S_2$. Результатом операції віднімання $S_1 - S_2$ є множина $S\{s\}$ така, що $s \in S_1$ і $s \notin S_2$. На рис. 2.7 ці

операції проілюстровані в інтуїтивній графічній формі. Про операцію взяття декартового добутку вже говорилося вище.

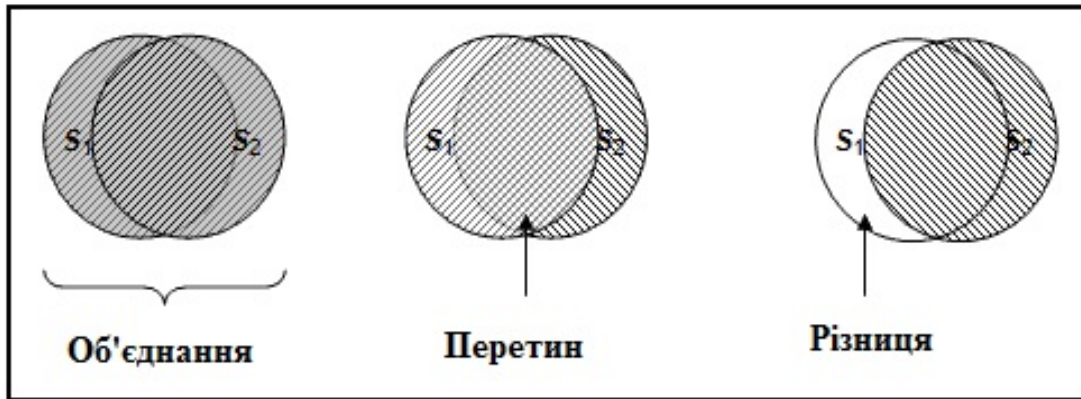


Рис. 2.7. Ілюстрація результатів теоретико-множинних операцій

Ці операції застосовні до будь-якого тіла відношення, але результатом не буде відношення, якщо у відношень-операндів не збігаються заголовки. Кодд запропонував як засіб маніпулювання реляційними базами даних спеціальний набір операцій, які гарантовано виробляють відношення. Цей набір операцій прийнято називати реляційною алгеброю Кодда, хоча цей набір операцій і не є алгеброю в математичному сенсі цього терміна, оскільки деякі бінарні операції цього набору застосовні не до довільних пар відношень.

В алгебрі Кодда є десять операцій: об'єднання (UNION), перетин (INTERSECT), віднімання (MINUS), взяття розширеного декартового добутку (TIMES), перейменування атрибутів (RENAME), проекція (PROJECT), обмеження (WHERE), з'єднання (JOIN), розподіл (DIVIDE BY) і присвоювання. Якщо не вдаватися в деякі тонкощі, які ми розглянемо в інших лекціях, то майже всі операції запропонованого вище набору володіють очевидною і простою інтерпретацією.

1. При виконанні операції об'єднання (UNION) двох відношень з однаковими заголовками виробляється відношення, що включає всі кортежі, що входять хоча б в одне з відношень-операндів.
2. Операція перетину (INTERSECT) двох відношень з однаковими заголовками виробляє відношення, що включає всі кортежі, що входять в обидва відношення-операнди.
3. Відношення, що є різницею (MINUS) двох відношень з однаковими заголовками, включає всі кортежі, що входять до відношення-перший операнд, такі, що жоден з них не входить до відношення, яке є другим операндом.
4. При виконанні декартового добутку (TIMES) двох відношень, перетин заголовків яких порожній, виробляється відношення, кортежі якого виробляються шляхом об'єднання кортежів першого і другого операндів.
5. Операція перейменування (RENAME) виробляє відношення, тіло якого збігається з тілом операнда, але імена атрибутів змінені. Ця операція дозволяє виконувати перші три операції над відношеннями заголовки яких майже збігаються (співпадаючими у всьому, крім імен атрибутів) і виконувати операцію TIMES над відношеннями, перетин заголовків яких не є порожнім.
6. Результатом обмеження (WHERE) відношення за деякою умовою є відношення, що включає кортежі відношення-операнда, яке задовольняє цій умові.
7. При виконанні проекції (PROJECT) відношення на задану підмножину множини його атрибутів виробляється відношення, кортежі якого є відповідними підмножинами кортежів відношення-операнда.
8. При з'єднанні (JOIN) двох відношень за деякою умовою утворюється результуюче відношення, кортежі якого виробляються шляхом об'єднання кортежів першого і другого відношення і задовольняють цій умові.

9. У операції реляційного поділу (DIVIDE BY) два операнда – бінарне і унарне відношення. Результуюче відношення складається з унарних кортежів, що включають значення першого атрибута кортежів першого операнда таких, що множина значень другого атрибута (при фіксованому значенні першого атрибута) включає множину значень другого операнда.
10. Операція присвоювання ($:=$) дозволяє зберегти результат обчислення реляційного вираження в існуючому відношенні БД.

2.4.3. Цілісність в реляційній моделі даних

Кодд запропонував два декларативних механізми підтримки цілісності реляційних баз даних, які затверджені в реляційній моделі даних і повинні підтримуватися в будь-якій СКБД, що її реалізує: обмеження цілісності сутності, або обмеження первинного ключа і обмеження посилювальної цілісності, або обмеження зовнішнього ключа.

Обмеження цілісності сутності визначене в такий спосіб: для заголовка будь-якого відношення бази даних повинен бути явно чи неявно визначений первинний ключ, який є такою мінімальною підмножиною заголовка відношення, що в будь-якому тілі цього відношення, яке може з'явитися в базі даних, значення первинного ключа в будь-якому кортежі цього тіла є унікальним, тобто відрізняється від значення первинного ключа в будь-якому іншому кортежі. Під мінімальним первинним ключем розуміється те, що якщо з множини атрибутів первинного ключа видалити хоча б один атрибут, то обмеження цілісності зміниться, тобто в БД зможуть з'являтися тіла відношення, які не допускалися вихідним первинним ключем.

Якщо первинний ключ не оголошується явно, то в якості первинного ключа відношення приймається весь його заголовок. Зрозуміло, що оскільки за визначенням будь-яке тіло відношення з заданим заголовком є множиною, отже, в ньому відсутні дублікати, і первинний ключ, що співпадає з заголовком відношення, завжди має властивість унікальності.

Щоб пояснити сенс обмеження посилювальної цілісності, потрібно спочатку ввести поняття зовнішнього ключа. В принципі при використанні реляційної моделі даних можна зберігати всі дані, відповідні предметної області в одній таблиці. Приклад такої бази даних демонструвався в лекції 1 на рис. 1.5, де в одному файлі (інтуїтивному аналогу відношення) зберігалася інформація і про службовців, і про відділи, в яких вони працюють. Як було показано в лекції 1, такий підхід призводить до надмірності зберігання (дані про відділ повторюються в кожному записі про службовця цього відділу) і ускладнює виконання деяких операцій.

На рис. 1.6 для зберігання інформації про службовців і відділах використовувалося два файли, в одному з яких зберігалися дані, індивідуальні для кожного службовця, а в другому – дані про відділи. Для можливості отримання повної інформації про службовців і відділах, в яких вони працюють, в файлі СЛУЖБОВЦІ містилося поле СЛУ_ОТД_НОМЕР, що містить для кожного службовця його унікальний номер відділу. У той же час, в файлі ВІДДІЛИ було поле ОТД_НОМЕР, Що є унікальним ключем цього файлу. Насправді, ввівши файли СЛУЖБОВЦІ і ВІДДІЛИ, а також забезпечивши зв'язок між ними за допомогою полів СЛУ_ОТД_НОМЕР і ОТД_НОМЕР, ми змогли забезпечити табличне представлення ієрархії ВІДДІЛ-СЛУЖБОВЦІ. Якщо говорити в термінах реляційної моделі даних, то щодо ВІДДІЛИ поле ОТД_НОМЕР є первинним ключем, а в відношенні СЛУЖБОВЦІ поле СЛУ_ОТД_НОМЕР є зовнішнім ключем, що посилюється на відношення ВІДДІЛИ.

Більш точно, зовнішнім ключем відношення R_1 , що посилюється на відношення R_2 , називається підмножина заголовка HR_1 , яке збігається з первинним ключем відношення R_2 (з точністю до імен атрибутів). Тоді обмеження посилювальної цілісності реляційної моделі даних можна сформулювати наступним чином: в будь-якому тілі відношення R_1 , яке може з'явитися в базі даних, для «без порожнього» значення зовнішнього ключа, що посилюється на відношення R_2 , в будь-якому кортежі цього тіла повинен знайтися кортеж в тілі відношення R_2 , яке міститься в базі даних, з збігається значенням первинного ключа. Легко помітити, що це майже те ж саме обмеження, про який говорилося в підрозділі «Ієрархічна модель даних»: ніякого нащадка не може існувати без свого батька.

2.5. Сучасні моделі даних

Історія сучасних моделей даних почалася з 1989 р., коли група відомих фахівців в області мов програмування баз даних опублікувала статтю під назвою «Маніфест систем об'єктно-орієнтованих баз даних» [5]. До цього часу вже існувало кілька реалізацій об'єктно-орієнтованих СКБД (ООСКБД), але кожна з них спиралася на деяке розширення об'єктної моделі будь-якого об'єктно-орієнтованої мови програмування (Smalltalk, Object Lisp, C++).

В [5] не пропонує єдину об'єктно-орієнтовану модель даних, але виділявся набір вимог до ООСКБД. Базовими вимогами було подолання невідповідності між типами даних, що використовуються в мовах програмування, і типами даних, підтримуваними в набрали на той час силу реляційних (вірніше, SQL-орієнтованих) СКБД, а також надання СКБД можливостей зберігати в БД дані довільно складної структури. Ці вимоги супроводжувалися твердженнями про обмеженість реляційної моделі даних і мови SQL і потреби використовувати більш розвинені моделі даних.

Під впливом цієї роботи в 1991 р. виник консорціум ODMG (Object Database Management Group), завданням якого була розробка стандарту об'єктно-орієнтованої моделі даних. Протягом більш ніж десятирічного існування ODMG опублікувала три базових версії стандарту, остання з яких називається ODMG 3.0 [6]. На цей документ ми і будемо спиратися в подальшому викладі.

У відповідь на публікацію [5] група дослідників, близьких до індустрії баз даних, в 1990 р. опублікувала документ «Маніфест систем баз даних третього покоління» [7]. Погоджуючись з авторами [5] щодо потреби забезпечення розвиненої системи типів даних в СКБД, автори [7] стверджували, що можна добитися аналогічних результатів, не роблячи революцію в області технології баз даних, а еволюційно розвиваючи технологію SQL-орієнтованих СКБД.

За публікацією [7] послідувала поява об'єктно-реляційних продуктів провідних компаній-постачальників SQL-орієнтованих СКБД (Informix Universal Server, Oracle8, IBM DB2 Universal Database). У 1999 р. був прийнятий стандарт мови SQL (SQL: 1999), в якому був зафіксований ряд нових рис мови, які надають йому риси повноцінної моделі даних. В останньому стандарті SQL: 2003 ця модель уточнена і розширена.

Отже, на початку 1990-х рр. були проголошені два маніфести, кожен з яких претендував на роль програми майбутнього розвитку технології баз даних. У першому маніфесті реляційна модель даних відкидалася повністю, а в другому замінювалося ще незрілою на той час моделлю даних SQL, яка вже тоді була далека від реляційної моделі. На захист реляційної моделі даних в її первозданному вигляді встали Крістофер Дейт і Х'ю Дарвен, що опублікували в 1995 р. статтю, під назвою «Третій маніфест» [3].

Пізніше Дейт і Дарвен написали книгу, перше видання якої вийшло в 1998 р. під назвою «Foundation for Object / Relational Databases: The Third Manifesto» [4; 5], Дрюге – в 2000 р. під назвою «Foundation for Future Database Systems: The Third Manifesto» (виданий переклад другого видання на російську мову [8] (Основи майбутніх систем баз даних. Третій маніфест. М: Янус-К, 2004) і третій – під назвою «Databases, Types and the Relational Model: The Third Manifesto» в 2006. У цих книгах дуже докладно викладається підхід авторів до побудови СКБД на основі, як вони стверджують, істинних ідей Едгара Кодда, викладених ним у своїх перших статтях про реляційну модель даних. Вони назвали її істинною реляційною моделлю.

2.5.1. Об'єктно-орієнтована модель даних

У моделі даних SQL і істинної реляційної моделі даних база даних являє собою набір іменованих контейнерів даних одного родового типу: таблиць або відношень відповідно. В об'єктно-орієнтованій моделі даних база даних – це набір об'єктів (контейнерів даних) довільного типу.

Типи і структури даних об'єктної моделі. В об'єктній моделі даних вводяться два різновиди типів: літеральні і об'єктні типи. *Літеральні типи даних* – це звичайні типи даних, прийняті в традиційних мовах програмування. Вони підрозділяються на базові скалярні числові

типи, символні і булеві типи (атомарні літерали), конструюються типи записів (структур) і колекцій.

Літеральний тип запису – це традиційний визначений користувачем структурний тип, подібний структурному типу мови C або типу запису мови Pascal. Відмінність полягає лише в тому, що в об'єктній моделі атрибут типу запису може визначатися не тільки на літеральному, але і на об'єктному типі, тобто значення літерального типу запису може в якості компонентів включати об'єкти.

Об'єктні типи в об'єктній моделі даних за змістом ближче всього до поняття класу в об'єктно-орієнтованих мовах програмування. У кожного об'єктного типу є операція створення і ініціалізації нового об'єкта цього типу.

Атрибутами називаються властивості об'єкта. Значеннями атрибутів можуть бути і літерали, і об'єкти, але тільки тоді, коли не потрібно зворотне посилання. Зв'язки – це інверсні властивості. В цьому випадку значенням властивості може бути тільки об'єкт. Зв'язки визначаються між атомарними об'єктними типами. В об'єктній моделі ODMG підтримуються тільки бінарні зв'язки, тобто зв'язку між двома типами. Зв'язки можуть бути різновидів «один-до-одного», «один-до-багатьох» і «багато-до-багатьох» в залежності від того, скільки примірників відповідного об'єктного типу може брати участь в зв'язку.

Маніпулювання даними в об'єктній моделі. У стандарті ODMG в якості базового засобу маніпулювання об'єктними базами даних пропонується мову OQL (Object Query Language). OQL не є обчислювально повною мовою. Вона являє собою просту мову запитів.

Оператори мови OQL можуть викликатися з будь-якої мови програмування, для якого в стандарті ODMG визначені правила зв'язування. І, навпаки, в запитах OQL можуть бути присутніми виклики операцій, запрограмованих на цих мовах.

Що ж стосується посилювальної цілісності, то вона підтримується, якщо між двома атомарними об'єктними типами визначається зв'язок виду «один-до-багатьох». В цьому випадку об'єкти на стороні зв'язку «один» розглядаються як предки, а об'єкти на стороні зв'язку «багато» – як нащадки, і ООСКБД зобов'язана стежити за тим, щоб не утворювалися нащадки без предків.

Основною перевагою об'єктно-орієнтованої моделі даних в порівнянні з реляційною є можливість відображення інформації про складних взаємозв'язках об'єктів. Об'єктно-орієнтована модель дозволяє також ідентифікувати окремі записи в базі і визначати функції їх обробки. З огляду на ці переваги, сьогодні вже деякі реляційні СКБД доповнюють функціями, що дозволяють скористатися перевагами об'єктної технології. Такі гібридні БД поєднують в собі можливості реляційних і об'єктно-орієнтованих, тому їх часто називають об'єктно-реляційними.

Основний недолік об'єктно-орієнтованої моделі полягає в складності розуміння її суті і низькій швидкості виконання запитів. В даний час об'єктно-орієнтовані бази даних досить складні, і тому їх комерційне використання йде повільно. Але у цих моделей є потенціал, а, отже, і майбутнє. А тому дослідження в області об'єктної орієнтації стають головним напрямком в теорії СКБД.

2.5.2. Справжня реляційна модель

Дейт і Дарвен дуже докладно і ретельно розробили пропонований ними варіант реляційної моделі даних. Тому в короткому огляді істинної реляційної моделі, що пропонується в цьому розділі, ми опишемо тільки її найзагальніші і зовнішні риси.

Типи і структури даних істинної реляційної моделі. Крістофер Дейт і Х'ю Дарвен поставили перед собою важке завдання: показати, що на основі ідей Едгара Кодда можна реалізувати СКБД, що забезпечують можливості по частині подання та зберігання даних складної структури, не менші тих, які забезпечують об'єктні і SQL-орієнтовані СКБД. Цьому заважав, перш за все, теза Кодда про нормалізацію відношень: в реляційній базі даних повинні міститися тільки відношення, атрибути яких визначені на «доменах, елементи яких є атомарними (не складовими) значеннями» [1].

В істинно реляційній моделі дуже велика увага приділяється типам даних. Пропонуються три категорії типів даних: скалярні типи, кортежні типи і типи відношень.

Скалярний тип даних – це звичний інкапсульований тип, реальна внутрішня структура якого прихована від користувачів. Пропонуються механізми визначення нових скалярних типів і операцій над ними. Типом атрибута визначається скалярного типу може бути будь-який визначений до цього моменту скалярний тип, будь кортежних тип і тип відношення. Деякі базові скалярні типи даних повинні бути визначені в системі.

Кортежних тип – це безіменний тип даних, який визначається за допомогою генератора типу TUPLE з зазначенням множини пар <імя_атрибута, тип_атрибута> (заголовка кортежу). Типом атрибута кортежних типу може бути будь-який визначений до цього моменту скалярний тип, будь кортежних тип і тип відношення. Значним кортежних типу є кортеж, який представляє собою множину триплетів <імя_атрибута, тип_атрибута, значення_атрибута>, яке відповідає заголовку кортежу цього кортежних типу.

Тип відношення – це безіменний тип даних, який визначається за допомогою генератора типу RELATION з зазначенням деякого заголовка кортежу. Значним типу відношення є заголовок відношення, що співпадає з заголовком кортежу цього типу відношення, і тіло відношення, що представляє собою множину кортежів, що відповідають цьому заголовку. Кортежні типи і типи відношень не є інкапсульованими: є можливість прямого доступу до атрибутів.

База даних в істинній реляційній моделі – це набір довготривало збережених іменованих змінних відношення, кожна з яких визначена на деякому типі відношення. У кожен момент часу кожна змінна відношення бази даних містить деяке значення відношення відповідного типу.

Маніпулювання даними в істинній реляційній моделі. В якості еталонного засобу маніпулювання даними в істинній реляційній моделі можна використовувати згадувану раніше реляційну алгебру Кодда. Однак Дейт і Дарвен запропонували нову реляційну алгебру, названу ними Алгеброю А, яка ґрунтується на реляційних аналогах булевих операцій кон'юнкції, диз'юнкції і заперечення. Через її операції виражаються всі операції алгебри Кодда.

Обмеження цілісності в істинній реляційній моделі. У число обов'язкових вимог істинної реляційної моделі входить вимога визначення хоча б одного можливого ключа для кожної змінної відношення (можливий ключ – це одне з підмножин заголовка змінної відношення, що володіє властивостями первинного ключа).

3. РЕЛЯЦІЙНА МОДЕЛЬ ДАНИХ

3.1. Вступ

Науковий базис теорії відношень, покладеної Коддом в основу реляційної моделі, закладався ще на рубежі XIX і XX століть. Авторами основ майбутньої реляційної теорії є два дослідники: Чарльз Содерс Пірс (1839-1914) і Ернст Шредер (1841-1902). Незважаючи на настільки глибоке коріння реляційної теорії, перша реляційна база даних з'явилася на світ через сімдесят років. Датою народження реляційної БД можна вважати червень 1970 року. Саме тоді Кодд (на той момент часу співробітник однієї з лабораторій корпорації IBM) опублікував свою знамениту статтю «Реляційна модель даних для великих спільно використовуваних банків даних», в якій вперше прозвучав настільки популярний сьогодні термін «реляційна модель». За свій внесок в науку Едгар Кодд був удостоєний премії імені Тюрінга. Ця знаменна подія відбулася в 1981 році.

Першопричиною виникнення нового на той час підходу до проектування баз даних послужили суттєві обмеження попередніх моделей. Ні мережева, ні ієрархічна моделі не були здатні просто і доступно описувати які мають враховуватися дані. Кодд зумів об'єднати, на перший погляд, несумісні речі – з одного боку, реляційна модель спиралася на математичні викладки, а з іншого боку, була зрозуміла пересічному користувачеві.

Модель даних не представляє істотного інтересу для звичайного користувача, але без неї не обійтися розробнику БД. Адже це не що інше, як креслення майбутньої БД. Жоден будівельник не почне будувати будинок без креслення, так і жоден програміст не почне створення фізичної БД без її докладної схеми. Фундамент реляційної моделі побудований на понятті сутності і зв'язків між сутностями.

Будь-яка галузь знань, будь то математика, радіоелектроніка, фізика, або інформатика в міру свого розвитку формує свій набір термінів і визначень. З точки зору складності семантики, системи управління базами даних (в загальному) і реляційна модель (зокрема) не є винятком з правил. Ця порівняно нова галузь знань практично миттєво встигла обзавестися специфічним набором слів. Багато терміни перекочували в БД з словникового запасу програмістів, частина визначень запозичена з теорії множин.

3.2. Основні поняття реляційних баз даних

Будь-який більш-менш підготовлений користувач знає, що бази даних призначені для зберігання і обробки інформації. База даних міської бібліотеки повинна містити інформацію про авторів та їхні твори. Зіткнувшись з базою даних авіакомпанії з високим ступенем ймовірності, можна стверджувати, що в ній ми знайдемо інформацію про розклад польотів, марках літаків і містах, в які вони розраховують доставити пасажирів.

Те, що здається елементарним для стороннього спостерігача, для професійного розробника БД може стати дуже непростою справою. На самому першому етапі проектування майбутньої БД він буде стикатися з двома ключовими питаннями:

1. Що необхідно зберігати в БД?
2. Які зв'язки між даними слід підтримувати в БД?

Для отримання відповіді на перше питання розробник повинен скласти початковий перелік типів сутностей. Тип сутності (entity) – це клас однотипних об'єктів, про які слід зберігати інформацію в БД. На заключній стадії проектування типи суті перетворюються в реляційні таблиці. У 90% випадків сутність являє собою якийсь об'єкт, який відповідає на питання «Хто?» або «Що?».

Виділимо в такому значенні реляційних баз даних: сутність, тип даних, домен, атрибут, кортеж, відношення, первинний ключ. Для початку покажемо зміст цих понять на прикладі відношення (сутності) СЛУЖБОВЦІ, що містить інформацію про службовців деякого підприємства (рис. 3.1).

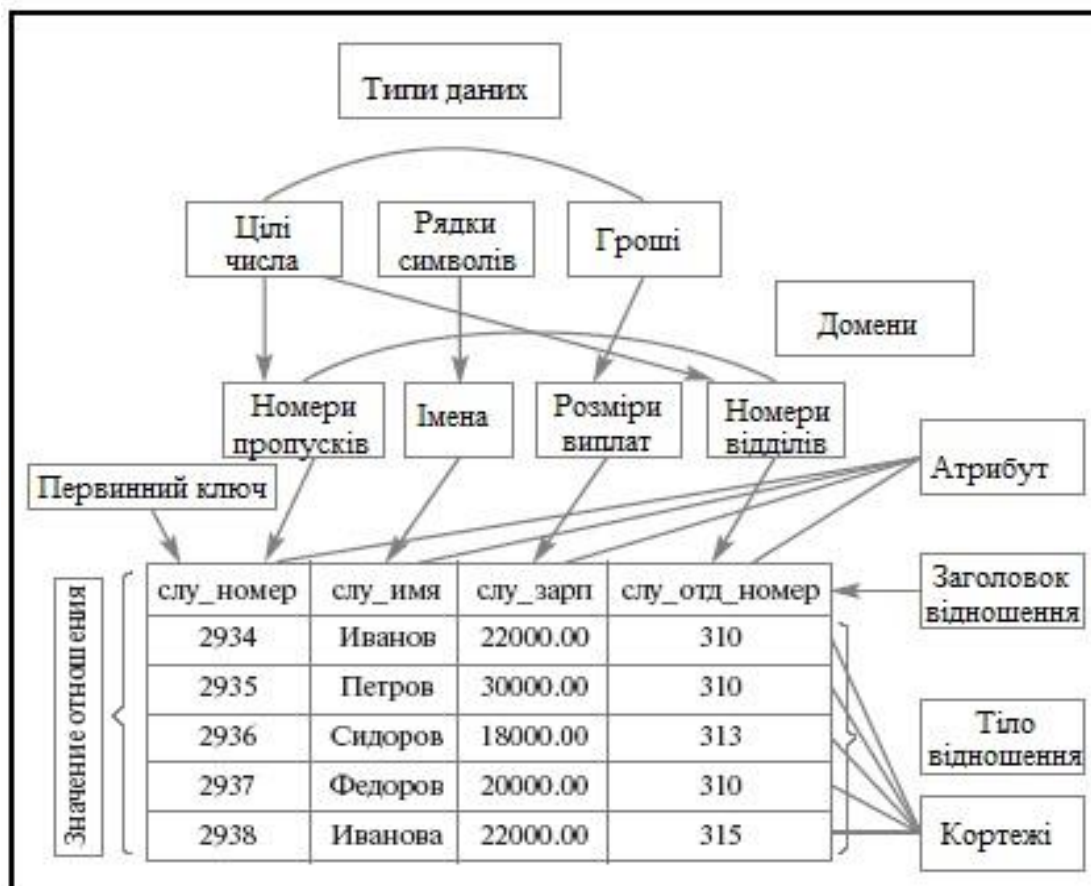


Рис. 3.1.Співвідношення основних понять реляційного підходу

3.2.1. Сутність і атрибути

У базі даних бібліотеки на роль сутностей претендують такі одухотворені і неживі об'єкти, як автор, книга, видавництво, читач. Зовсім не обов'язково, щоб сутність відповідала об'єктів реального світу, навпаки, вона може моделювати і абстрактні поняття, виражені в особливих відносинах між сутностями. Саме в такому випадку сутність спритно маскується під дієслово. Як правило, це відбувається, коли обліку підлягають деякі події. Наприклад, коли читач отримує (це і є дієслово) книгу з бібліотечного фонду, логіка бази даних вимагає зберегти інформацію про те, кому, коли і яка з книг була видана. При більш детальному розгляді з'ясовується, що при побудові початкового списку ми просто не виявили такий тип сутності, як читачка бібліотечна картка, в яку і робляться всі перераховані позначки.

Якщо тип сутності – це клас однотипних об'єктів, то окремий об'єкт, що входить в цей клас, в реляційній моделі називають екземпляром сутності. Примірник сутності (або просто сутність) – це конкретна реалізація об'єкта. Для сутності «автор» – це Едгар Кодд, Для сутності «книга» – це «Реляційна модель даних». Тобто, примірник сутності – це те, про що (або про кого) слід зберігати інформацію в базі даних.

У будь-якої сутності є свої характеристики. У сутності «читач» – це, наприклад, ім'я, домашня адреса і номер телефону. Відмінні властивості сутності називають атрибутами.

Атрибути можуть бути простими (наприклад, назва міста), а може бути і складовими (тобто, включати в себе кілька простих атрибутів). В якості складового атрибута виступає адреса читача. Тут знайдеться місце поштовим індексом, місту, вулиці, будинку і номером квартири.

Атрибут може бути похідним, тобто, з отриманими в результаті проведення якоїсь операції над іншими атрибутами. Подібні атрибути часто зустрічаються при побудові різних бухгалтерських систем, наприклад, значення стягується з співробітників підприємства податку становить певний відсоток від заробітної плати цих службовців. Зустрічаються і багатозначні атрибути, вони описують ті властивості сутності, в яких одночасно зберігається кілька значень.

Наприклад, в однієї людини може бути кілька адрес проживання, кілька контактних телефонних номерів і т. п. Є й особливий різновид атрибутів, без яких неможлива побудова реляційної моделі, – атрибути, однозначно ідентифікують екземпляр сутності. На роль такого атрибута в змозі претендувати індивідуальний номер платника податків або унікальний заводський номер літака.

Побудувавши вичерпний список типів сутностей і підготувавши перелік їх атрибутів, розробник моделі бази даних повинен задуматися над особливостями реалізації типів сутностей. Для цього в першу чергу слід розібратися з особливостями фізичного представлення кожного з атрибутів типу сутності, для цього призначені тип даних і домен.

3.2.2. Тип даних і домен

Відправною точкою процесу побудови будь-якої системи зберігання і обробки даних вважається вибір типу даних. Незалежно від того, яку мову програмування ви вивчали, при побудові нової програми найпершим кроком стає розгляд типів даних, наявних у розпорядженні системи.

Значення даних, що зберігаються в реляційній базі даних, також є типізованими, тобто, відомий тип кожного значення, що зберігається. Поняття типу даних в реляційній моделі даних повністю відповідає поняттю типу даних в мовах програмування. Нагадаємо, що традиційне (нечітке) визначення типу даних складається з трьох основних компонентів: визначення множини значень даного типу; визначення набору операцій, які можна застосувати до значень типу; визначення способу зовнішнього представлення значень типу (літералів).

Зазвичай в сучасних реляційних базах даних допускається зберігання символьних, числових даних, спеціалізованих числових даних (таких, як «гроші»), а також спеціальних «темпоральних» даних (дата, час, часовий інтервал). Крім того, в реляційних системах підтримується можливість визначення користувачами власних типів даних. У прикладі на рис. 3.1 ми маємо справу з даними трьох типів: рядки символів, цілі числа і «гроші».

Поняття домену більш специфічно для баз даних, хоча і є аналогії з підтипами в деяких мовах програмування. У загальному вигляді домен визначається шляхом завдання деякого базового типу даних, до якого відносяться елементи домену, і довільного логічного виразу, який застосовується до елемента цього типу даних (обмеження домену).

Найбільш правильною інтуїтивною трактуванням поняття **домену** є його сприйняття як допустимого потенційного, обмеженого підмножини значень даного типу. Наприклад, домен ІМЕНА в нашому прикладі визначено на базовому типі символьних рядків, але в число його значень можуть входити тільки ті рядки, які можуть представляти імена. Зокрема, для можливості подання українських імен такі рядки не можуть починатися з м'якого знаку і не можуть бути довгими, наприклад, 20 символів.

Якщо деякий атрибут відношення визначається на деякому домені (як, наприклад, на рис. 3.1 атрибут СЛУ_ІМЯ визначається на домені ІМЕНА), то в подальшому обмеження домену грає роль обмеження цілісності, який накладається на значення цього атрибута.

Відзначимо також семантичне навантаження поняття домену: дані вважаються порівнянними тільки в тому випадку, коли вони відносяться до одного домену. Так, значення доменів НОМЕРИ ПРОПУСКІВ і НОМЕРА ВІДДІЛІВ відносяться до типу цілих чисел, але не є порівнянними.

3.2.3. Відношення

Поняття відношення є найбільш фундаментальним в реляційному підході до організації баз даних. Це відображено і в загальній назві підходу – термін реляційний походить від *relation* (відношення). Однак сам термін відношення є виключно неточним, оскільки, кажучи про будь-які зберігаються дані, ми повинні мати на увазі тип цих даних, значення цього типу і змінні, в яких зберігаються значення. Відповідно, для уточнення терміну відношення виділяються поняття заголовка відношення, значення відношення, змінної відношення і поняття кортежу.

Заголовком (або схемою) відношення називається кінцева множина впорядкованих пар виду $\langle A, T \rangle$, де A називається іменем атрибута, а T позначає ім'я деякого базового типу або раніше визначеного домену. За визначенням потрібно, щоб всі імена атрибутів в заголовку відношення були різні. У прикладі на рис. 3.1 заголовком відношення СЛУЖБОВЦІ є множина пар $\langle \text{Слу_номер}, \text{номера_пропусков} \rangle$, $\langle \text{слу_імя}, \text{імена} \rangle$, $\langle \text{слу_зарп}, \text{размери_виплат} \rangle$, $\langle \text{слу_отд_номер}, \text{номера_отделов} \rangle$.

Кортеж, що відповідає даній схемі відношення, – це множина пар {ім'я атрибута, значення}, яке містить одне входження кожного імені атрибута, що належить схемою відношення. «Значення» є допустимим значенням домену даного атрибута (або типу даних, якщо поняття домену не підтримується). Відношенню СЛУЖБОВЦІ відповідають, наприклад, наступний кортеж: $\{ \langle \text{Слу_номер}, 2934 \rangle, \langle \text{слу_імя}, \text{Іванов} \rangle, \langle \text{слу_зарп}, 22.000 \rangle, \langle \text{слу_отд_номер}, 310 \rangle \}$.

Ступінь або «арність» кортежу – це число елементів (атрибутів) кортежу, збігається з «арністю» відповідної схеми відношення. Наприклад, ступінь відношення СЛУЖБОВЦІ дорівнює чотирьом, тобто, воно є 4-арним.

Кардинальність – кількість кортежів відношення (потужність відношення).

Тілом відношення називається довільна множина кортежів. Одна з можливих тіл відношення СЛУЖБОВЦІ показана на рис. 3.1.

Відношення – це множина кортежів, що відповідають одній схемі відношення. Звичайним життєвим поданням відношення є двовимірна таблиця, заголовком якої є схема відношення, а рядками – кортежі відношення. В цьому випадку імена атрибутів іменують стовпці цієї таблиці. Тому іноді говорять «стовпець таблиці», маючи на увазі «атрибут відношення» (рис. 3.2). Коли ми перейдемо до розгляду практичних питань організації реляційних БД і засобів управління, ми будемо використовувати цю термінологію, якої дотримуються в більшості реляційних СКБД.

Ім'я поля	Атрибут(стовпець, поле)	Таблиця READER			
	PEOPLES_KEY	SNAME	FNAME	PHONENUM	BDAY
	1	Петров	Василий	34-74-454	1/01/1970
	2	Акулов	Алексей	76-15-011	11/12/1978
	3	Козлова	Татьяна	83-13-116	21/05/1980
	4	Козлов	Олег	83-13-116	5/10/1977
	5	Миронов	Николай	22-57-962	11/12/1969

Рис. 3.2. Реляційна таблиця

Реляційна база даних – це набір відношень, імена яких збігаються з іменами схем відношень в схемі БД. Основні властивості відношення:

1. Унікальність імен стовпців усередині таблиці.
2. Кожен рядок зберігає інформацію про окрему сутність.
3. У відношенні немає однакових кортежів, рядки таблиці відрізняються один від одного хоча б єдиним значенням.
4. Кортежі не впорядковані.
5. Атрибути не впорядковані і розрізняються по найменуванню.
6. Всі значення атрибутів атомарні.

Як видно, основні структурні поняття реляційної моделі даних мають дуже просту інтуїтивну інтерпретацію, хоча в теорії реляційних БД всі вони визначаються абсолютно формально і точно.

3.2.4. Первинний і зовнішній ключ

Реляційні бази даних, як ми вже знаємо, складаються з таблиць. Кожна таблиця складається з стовпців (їх називають полями або атрибутами) і рядків (їх називають записами або кортежами). Таблиці в реляційних базах даних мають ряд властивостей. Основними є такі:

1. У таблиці не може бути двох однакових рядків. В математиці таблиці, що володіють такою властивістю, називають відношеннями – по-англійськи *relation*, звідси і назва – реляційні.
2. Стовпці розташовуються в певному порядку, який створюється при створенні таблиці. У таблиці може не бути жодного рядка, але обов'язково повинен бути хоча б один стовпець.
3. У кожного стовпця є унікальне ім'я (в межах таблиці), і все значення в одному стовпці мають один тип (число, текст, дата тощо).
4. На перетині кожного стовпця і рядка може перебувати тільки атомарне значення (одне значення, яке не перебуває з групи значень). Таблиці, що задовольняють цій умові, називають нормалізованими.

Все буде зрозуміліше на прикладі. Припустимо, ми захотіли створити прощену модель системи прийому замовлень. У замовника є зареєстроване ім'я, адреса, адреса доставки замовлення, дата замовлення, замовлений товар. Ця інформація і повинна зберігатися в базі даних.

Теоретично (на папері) ми можемо все це розташувати в одній таблиці, наприклад, так:

Customer Name	(Ім'я замовника)
Customer Address	(Адреса замовника)
ShipTo Address	(Адреса доставки)
Order Date	(Дата замовлення)
Product Ordered	(Замовлений товар)
Quantity Ordered	(Замовлена кількість)
Unit Price	(Ціна за одиницю товару)

Цю інформацію можна було б зберігати в кожного запису в одній таблиці, що дуже неефективно. В цьому випадку для кожного замовлення і для кожного товару в замовленні довелося б зберігати одну і ту ж інформацію: ім'я замовника, адреса замовника, адреса доставки і дату замовлення. Щоб уникнути повторення розіб'ємо дані на окремі таблиці:

Таблиця Замовників	Таблиця Замовлень	Таблиця Компоненти замовлення
ім'я замовника адреса замовника	адреса доставки дата замовлення	замовлений товар замовлену кількість товару ціна за одиницю товару

При такій розбивці таблиця «Замовники» містить всю інформацію про замовника, таблиця «Замовлення» – всю інформацію про замовлення, таблиця «Компоненти» – всю інформацію по кожному товару в замовленні. Очевидно, що дані не дублюються. Однак зовсім неясно, які записи в інших таблицях відповідають конкретній запису цієї таблиці. Для вирішення цього питання були введені реляційні поняття «Первинний Ключ» (скорочено РК – Primary Key) і «Зовнішній Ключ» (FF – Foreign Key).

Простий ключ складається з одного атрибута (поля). Складний – з двох і більше.

Потенційний ключ – простий або складний ключ, який унікально ідентифікує кожну запис таблиці. При цьому потенційний ключ повинен володіти критерієм ненадлишковості: при видаленні будь-якого з полів набір полів перестає унікально ідентифікувати запис.

З безлічі всіх потенційних ключів таблиці вибирають первинний ключ, всі інші ключі називають альтернативними.

Первинний ключ – це атрибут (поле) або група атрибутів, однозначно ідентифікує екземпляр сутності. Зазвичай це покажчик на запис в таблиці, що представляє собою поле (або поля), значення якого не може повторюватися в таблиці або бути порожнім. При строгому дотриманні канонів розробки реляційної бази даних для кожної таблиці бази повинен бути

первинний ключ. В кожному відношенні завжди повинен бути один і тільки один первинний ключ.

Ключі можуть бути складними, тобто містять кілька атрибутів. Розглянемо кандидатів на роль первинного ключа сутності Співробітник:

співробітник
Табельний номер
Прізвище
ім'я
По батькові
вага
дата народження
Номер паспорта

Тут можна виділити наступні потенційні ключі:

1. Табельний номер;
2. Номер паспорта;
3. Прізвище + Ім'я + батькові.

Для того щоб стати первинним, потенційний ключ повинен задовольняти ряду вимог:

1. **Унікальність.** Два примірника не повинні мати однакових значень можливого ключа. Потенційний ключ № 3 (Прізвище + Ім'я + батькові) є поганим кандидатом, оскільки в організації можуть працювати повні тезки.
2. **Компактність.** Складний можливий ключ не повинен містити жодного атрибута, видалення якого не приводило б до втрати унікальності. Для забезпечення унікальності ключа № 3 доповнимо його атрибутами Дата народження і Вага. Якщо поєднання атрибутів Прізвище + Ім'я + батькові + Дата народження досить для однозначної ідентифікації співробітника, то Вага виявляється зайвим, тобто, ключ Прізвище + Ім'я + батькові + Дата народження + Вага не є компактным.

При виборі первинного ключа перевага повинна віддаватися більш простим ключам, тобто ключам, що містять меншу кількість атрибутів. У наведеному прикладі ключі № 1 і 2 краще ключа № 3.

Атрибути ключа не повинні містити нульових значень. Значення атрибутів ключа не повинно змінюватися протягом усього часу існування екземпляра сутності. Співробітниця організації може вийти заміж і змінити як прізвище, так і паспорт. Тому ключі № 2 і 3 не підходять на роль первинного ключа.

Кожна сутність повинна мати принаймні один потенційний ключ. Багато сутностей мають тільки один потенційний ключ. Такий ключ стає первинним. Деякі суті можуть мати більше одного можливого ключа. Тоді один з них стає первинним, а інші – альтернативними ключами.

Альтернативний ключ (Alternate Key) – це потенційний ключ, який не став первинним.

Зовнішній ключ – це покажчик на запис таблиці, що представляє собою поле (або поля), значення якого повторює значення поля первинного ключа іншої таблиці, пов'язаного з даними. Іншими словами, зовнішній ключ містить «заслання» на первинний ключ іншої таблиці. Первинний і зовнішній ключі складають основу взаємозв'язків таблиць реляційної бази даних. Збігаються значення первинного та зовнішнього ключів вказують на наявність зв'язку між записами (є «покажчиками» на ці записи). Записи зовнішнього ключа в «Таблиці А» відсилають до запису первинного ключа в «Таблиці Б», а первинний ключ «Таблиці Б» на записи зовнішнього ключа в «Таблиці А».

Первинні ключі можуть бути логічними (природними) і сурогатними (штучними). Сурогатний ключ являє собою додаткове поле в базі даних. Як правило, це порядковий номер запису.

Щоб таблиці попереднього прикладу задовольняли вимогам «реляційності», в них необхідно додати деякі поля для визначення первинних і зовнішніх ключів.

Таблиця замовників:

номер замовника (ID_Customer) – первинний ключ
 ім'я замовника
 адреса замовника

Таблиці замовлень:

номер замовлення (ID_Order) – первинний ключ
 номер замовника (ID_Customer) – зовнішній ключ
 адреса доставки
 дата замовлення

Таблиці компоненти замовлення:

номер замовлення (ID_Order) – перший компонент первинного ключа і зовнішній ключ
 замовлений товар – другий компонент первинного ключа
 замовлену кількість товару
 ціна за одиницю товару

Оскільки не можна гарантувати, що в таблиці «Замовники» не повториться ім'я замовника, то в якості первинного ключа додано поле «номер замовника». Номер замовлення доданий в таблицю «Замовлення» в якості первинного ключа, оскільки в таблиці немає поля, яке можна вважати унікальним. У таблицю замовлення введений зовнішній ключ – поле номер замовника – для зв'язку з таблицею замовника. Зовнішній ключ «номер замовлення» введений в таблицю «Компоненти» для зв'язку його з таблицею «Замовлення». Це ж поле стало першою компонентою складеного первинного ключа (номер замовлення і замовлений товар).

3.2.5. Зв'язок

Реляційна модель призначена не тільки для опису переліків типів сутностей, а й для відображення особливості взаємодії між ними. Взаємодія виникає там, де між типами сутностей виявляються відношення, виражені у вигляді дієслова, наприклад, «продавець оформив замовлення». Тут «продавець» і «замовлення» – це дві сутності, а зв'язок між ними проглядається в дієслові «оформив».

Розрізняють три типи зв'язку між сутностями: «один до одного», «один до багатьох» і «багато до багатьох». Поява в реляційній БД зв'язку «один до одного» – вкрай рідкісний випадок. Найчастіше він свідчить про те, що розробник не цілком коректно визначився з атрибутами і замість одного з атрибутів створив зайвий тип сутності. На практиці найбільш поширена зв'язок «один до багатьох», наприклад, автор написав багато книг, завод випустив багато автомобілів, в одному відділі працює багато співробітників (рис. 3.3).



Рис. 3.3. Відношення (зв'язок) «один до багатьох»

Зв'язок «багато до багатьох» зустрічається рідше: офіси різних компаній розміщені в різних містах, безліч видавництв опублікувало твори різних авторів і т.п. Реляційна модель безпосередньо не підтримує відношення «багато до багатьох», тому його доводиться розбивати на два відношення «один до багатьох». До питання побудови зв'язків класу «багато до багатьох» ми повернемося в лекції при описі процесу ER-моделювання, а поки обговоримо ще одну особливість зв'язку.

Реляційна модель дозволяє пов'язувати сутність саму з собою – задавати рекурсивний зв'язок. Такий, на перший погляд, дещо незвичний спосіб взаємодії виявляється досить корисним при «виращуванні» деревоподібних конструкцій, наприклад, що описують організаційну структуру підприємства, заводу або університету. Тут чітко проглядається ієрархія, в якій одна і та ж сутність може бути головною по відношенню до підлеглих елементів і одночасно перебувати в підпорядкованому стані до вищого за рангом вузлу. Так, дочірніми елементами по відношенню до деканату факультету виступають кафедри цього факультету, в свою чергу деканат підпорядковується ректорату навчального закладу.

3.3. Фундаментальні властивості відношень

Зупинимося тепер на деяких важливих властивостях відношень, які впливають з наведених раніше визначень.

3.3.1. Відсутність кортежів-дублікатів

Те властивість, що тіло будь-якого відношення ніколи не містить кортежів-дублікатів, випливає з визначення тіла відношення як множини кортежів. У класичній теорії множин за визначенням будь-яка множина складається з різних елементів.

Саме з цієї властивості випливає наявність у кожного значення відношення первинного ключа – мінімальної множини атрибутів, що є підмножиною заголовка даного відношення, складене значення яких унікально визначає кортеж відношення. Дійсно, оскільки в будь-який час все кортежі тіла будь-якого відношення різні, у будь-якого значення відношення властивістю унікальності має, принаймні, повний набір його атрибутів. Однак в формальному визначенні первинного ключа потрібне забезпечення його «мінімальності», тобто, в набір атрибутів первинного ключа не повинні входити такі атрибути, які можна відкинути без шкоди для основного властивості – однозначного визначення кортежу.

Визначаючи змінну відношення, проектувальник моделює частину предметної області, дані з якої буде містити база даних. І звичайно, проектувальник повинен знати природу цих даних. Наприклад, йому повинно бути відомо, що ніякі два службовців ні в який момент часу не можуть мати посвідчення з одним і тим же номером. Тому він може (і навіть повинен, як буде показано трохи пізніше) явно оголосити {СЛУ_НОМЕР} можливим ключем щодо на рис. 3.1.

Тепер пояснимо, чому проектувальнику слід явно оголошувати первинний і можливі ключі змінних відношення. Справа в тому, що в результаті цього оголошення СКБД отримує інформацію, яка в подальшому буде використовуватися як обмеження цілісності. СКБД ніколи не допустить появи в змінні відношення значення-відношення, що містить два кортежу з однаковим значенням атрибута СЛУ_НОМЕР. Тим самим оголошення первинних ключів дають СКБД можливість підтримувати цілісність бази даних навіть у разі спроб занесення в неї некоректних даних. Нарешті, повернемося до властивості мінімальності первинного. Як зазначалося вище, це властивість є критично важливим, і важливість проявляється саме при трактуванні первинних ключів як обмежень цілісності.

3.3.2. Відсутність впорядкованості кортежів і атрибутів

Звичайно, формально властивість відсутності впорядкованості кортежів у значенні відношення також є наслідком визначення тіла відношення як множини кортежів. Однак на цю властивість можна подивитися і з іншого боку. Так, ту обставину, що тіло відношення є множиною кортежів, полегшує побудову повного механізму реляційної моделі даних, включаючи базові засоби маніпулювання даними – реляційні алгебру та обчислення.

Досить часто у користувачів реляційних СКБД і розробників інформаційних систем викликає роздратування той факт, що вони не можуть зберігати кортежі відношення на фізичному рівні в потрібному їм порядку. І посилання на вимоги реляційної теорії тут не дуже доречні. Можна було б розробити іншу теорію, в якій допускалися б впорядковані «відношення». Однак

зберігати впорядковані списки кортежів в умовах інтенсивно оновлюваної бази даних набагато складніше технічно, а підтримка впорядкованості тягне за собою істотні накладні витрати.

Відсутність вимоги до підтримання порядку на множині кортежів відношення надає СКБД додаткову гнучкість при зберіганні баз даних у зовнішній пам'яті і при виконанні запитів до бази даних. Це не суперечить тому, що при формулюванні запиту до БД, наприклад, на мові SQL можна зажадати сортування результуючої таблиці відповідно до значень деяких стовпців. Такий результат, взагалі кажучи, є не відношенням, а деяким впорядкованим списком кортежів, і він може бути тільки остаточним результатом, до якого вже не можна адресувати запити.

Атрибути відношення не впорядковані, оскільки за визначенням заголовки відношення є множина пар <Ім'я атрибута, ім'я домену>. Для посилання на значення атрибута в кортежі відношення завжди використовується ім'я атрибута. Легко помітити явну аналогію між заголовками відношення і структурними типами в мовах програмування. Навіть в мові програмування C з його практично необмеженими можливостями роботи з покажчиками наполегливо рекомендується звертатися до полів структур тільки за їхніми іменами.

Аналогічними практичними міркуваннями виправдовується і відсутність впорядкованості атрибутів в заголовку відношення. В цьому випадку СКБД сама приймає рішення про те, в якому фізичному порядку слід зберігати значення атрибутів кортежів (хоча зазвичай один і той же фізичний порядок підтримується для всіх кортежів кожного відношення). Крім того, це властивість полегшує виконання операції модифікації схем існуючих відношень не тільки шляхом додавання нових атрибутів, а й шляхом видалення існуючих.

3.3.3. Атомарність значень атрибутів, перша нормальна форма відношення

В основу розробки реляційної бази даних покладено наступний принцип: елемент даних, будучи збережений в одному місці бази, не повинен повторюватися в інших її місцях. Це дає відразу дві переваги: скорочення обсягу необхідного дискового простору, а також для спрощення супроводу даних і усунення аномалій в організації зберігання даних. При розробці реляційної бази даних цей принцип реалізується шляхом розбиття даних на частини – пов'язані одна з одною таблиці (файли), і називається нормалізацією.

Нормалізація – це розбиття таблиці на дві або більше таблиці, які мають кращі властивості при включенні, зміні і видаленні даних. Остаточна мета нормалізації зводиться до отримання такого проекту бази даних, в якому кожен факт з'являється лише в одному місці, тобто виключена надмірність інформації.

Процес нормалізації зводиться до послідовного приведення структури даних до нормальних форм – формалізованим вимогам до організації даних.

Відомі шість нормальних форм. На практиці зазвичай обмежуються приведенням даних до третьої нормальної форми.

Значення всіх атрибутів відношення є атомарними. Це впливає з визначення домену як потенційного безлічі значень скалярного типу даних. Головне в атомарності значень атрибутів полягає в тому, що реляційна СКБД не повинна забезпечувати користувачам явною видимістю внутрішньої структури значення. З усіма значеннями можна звертатися тільки за допомогою операцій, визначених у відповідному типі даних.

Прийнято говорити, що в реляційних базах даних допускаються тільки нормалізовані відношення, або відношення, представлені в першій нормальній формі.

Спочатку дамо формальне визначення першої нормальної форми (1НФ). Схеми відношення R знаходиться в 1НФ, якщо значення в $\text{dom}(A)$ є атомарними для кожного атрибута A в R . Тут $\text{dom}(A)$ – допустимі значення домена (область визначення значень) стовпця A .

Іншими словами, таблиця знаходиться в першій нормальній формі, якщо кожен атрибут відношень зберігає одне-єдине значення (атомарний) і не бути ні списком, ні множиною значень.

Згідно з визначенням відношення, будь-яке відношення автоматично вже знаходиться в 1НФ. Нагадаємо коротко властивості відношення (це і будуть властивості 1НФ):

1. У відношенні немає однакових кортежів.
2. Кортежі не впорядковані.

3. Атрибути не впорядковані і розрізняються по найменуванню.
4. Всі значення атрибутів атомарні.

Слід зауважити, що, незважаючи на простоту даного визначення, однозначно визначити поняття атомарності часто виявляється досить важко, якщо заздалегідь невідомі семантика атрибута і його роль в обробці даних, що зберігаються. Атрибут, який є атомарним в одному додатку, може виявитися складовим в іншому. Загальний принцип тут такий: значення не атомарному, якщо воно використовується по частинах.

Наведемо простий приклад. В БД відділу кадрів підприємства в таблиці, що зберігає особисті відомості про співробітників, є атрибут «домашній-адреса», в якому адреса зберігається в форматі: місто, вулиця, будинок [/ корпус], [квартира] (слідуючи загальноприйнятої нотації, тут в квадратних дужках вказані опціональні фрагменти адреси, які можуть бути відсутніми). В даному випадку адреса зберігається у вигляді єдиної текстового рядка, оскільки малоймовірно, щоб треба було вибрати співробітників, скажімо, за номером квартири. Таким чином, в контексті БД відділу кадрів адреса є атомарним поняттям, і його розподіл на складові частини не має сенсу, тому що тільки внесе в БД зайву громіздкість.

Однак той же адреса для додатка, призначеного для сортування пошти в поштовому відділенні, атомарним не є, оскільки бажано згрупувати конверти в окремі стопки по вулицях, так як кожен вулицю обслуговує свій листоноша. Крім того, з метою оптимізації переміщень листоноші в межах вулиці, кожен стопку бажано впорядкувати за номерами будинків, щоб зробити рознести пошту за один прохід по вулиці без повернення.

Отже, перша нормальна форма вимагає, щоб кожне поле таблиці БД було неподільним (атомарним) і не містило повторюваних груп. У реляційній моделі відношення завжди знаходиться в першій нормальній формі за визначенням поняття відношення.

А як на практиці застосовувати це правило? Насправді все це дуже просто.

Розглянемо ці принципи на класичному прикладі – таблиці з бази даних продажів книгарні. Для простоти припустимо, що одну книгу пише тільки один автор.

Дані можуть групуватися в таблиці (відношення) різними способами. При проектуванні БД в якості відправної точки може використовуватися одне універсальне відношення, до якого включаються всі необхідні атрибути. Воно може містити всі дані, які передбачається розміщувати в БД. Спочатку ми, швидше за все, так би і зробили – розмістили б цю інформацію в одній універсальній таблиці наступним чином:

Покупець	Купівля	Дата купівлі	Сума	ТелПокупця
ТОВ «Астра»	Іванов «Мікрософт Офіс» – 2 екз	05.01.10	400	111-11-11
Астра ТОВ	Іванов «Мікрософт Офіс» – 1 прим Дерк «Довідник по РНР» – 1 прим Дерк «Довідник по Java» – 1 прим	06.01.10	800	111-11-11
Аврора з-д	Дейт «Бази даних» – 1 прим	30.12.09	50	222-22-22

Неподільність (Атомарність) означає, що в таблиці не повинно бути полів, які можна розбити на більш дрібні поля. Наприклад, якщо в одному полі ми об'єднаємо назва книги і кількість примірників, то вимога неподільності дотримуватися не буде. Перша нормальна форма вимагає, щоб ми розбили ці дані по двох полях.

Виконаємо вимоги 1NF про атомарності даних, і створимо відповідну нормалізовану таблицю. Тоді ми отримуємо таблицю ПРОДАЖ в такому вигляді:

Покупець	Купівля	Автор	К	Ціна	Дата покуп	Сума	ТелПокуп
ТОВ Астра	«Мікрософт Офіс»	Іванов	2	200	05.01.10	400	111-11-11
Астра ТОВ	«Мікрософт Офіс»	Іванов	1	200	06.01.10	200	111-11-11
Астра ТОВ	«Довідник по РНР»	Дерк	1	300	06.01.10	300	111-11-11
Астра ТОВ	«Довідник по Java»	Дерк	1	300	06.01.10	300	111-11-11
Аврора з-д	"Бази даних"	Дейт	1	50	30.12.09	50	222-22-22

Тепер можемо вважати, що кожне значення кожного з атрибутів нашого відношення є атомарним. Отже, таблиця знаходиться в 1НФ.

Під поняттям повторювані групи мають на увазі поля, що містять однакові за змістом значення. Тому не можна розміщувати покупку декількох книг в трьох і більше полях, які є списком або множиною значень типу:

Покупець	Покупка1	Покупка2	Покупка3	...
ТОВ «Астра»	«Мікрософт Офіс»	«Мікрософт Офіс»	«Довідник по РНР»	...
...	...	...	...	...

Наведемо ще один приклад ненормалізованих відношень, показаного на рис. 3.4. Можна сказати, що тут ми маємо бінарне відношення, в якому значеннями атрибута ВІДДІЛИ є відношення. Зауважимо, що вихідне відношення СЛУЖБОВЦІ є нормалізованим варіантом відношення ВІДДІЛИ-СЛУЖБОВЦІ. Нормалізований варіант показаний на рис. 3.5.

Нормалізовані відношення становлять основу класичного реляційного підходу до організації баз даних. Вони володіють деякими обмеженнями (не всяку інформацію зручно представляти у вигляді плоских таблиць), але істотно спрощують маніпулювання даними. Розглянемо, наприклад, два ідентичних оператора занесення кортежу:

- зарахувати службовця Науменко (пропуск номер 3000, зарплата 25000.00) до відділу 320;
- зарахувати службовця Науменко (пропуск номер 3000, зарплата 25000.00) до відділу 310.

НОМЕР_ОТДЕЛА	ОТДЕЛ		
	СЛУ_НОМЕР	СЛУ_ИМЯ	СЛУ_ЗАРП
310	2934	Иванов	22000.00
	2935	Петров	30000.00
	2937	Федоров	20000.00
313	2936	Сидоров	18000.00
315	2938	Иванова	22000.00

Рис. 3.4. Ненормалізоване відношення ВІДДІЛИ-СЛУЖБОВЦІ

СЛУ_НОМЕР	СЛУ_ИМЯ	СЛУ_ЗАРП	СЛУ_ОТД_НОМЕР
2934	Иванов	22000.00	310
2935	Петров	30000.00	310
2936	Сидоров	18000.00	313
2937	Федоров	20000.00	310
2938	Иванова	22000.00	315

Рис. 3.5. Нормалізований варіант відношення ВІДДІЛИ-СЛУЖБОВЦІ

Якщо інформація про службовців представлена у вигляді відношення СЛУЖБОВЦІ, То обидва оператори будуть виконуватися однаково (вставити кортеж в відношення СЛУЖБОВЦІ). Якщо ж працювати з ненормалізованими відношеннями ВІДДІЛИ-СЛУЖБОВЦІ, То перший оператор призведе до простою вставці кортежу, а другий – до додавання кортежу в значення-відношення атрибута ВІДДІЛ кортежу з первинним ключем 310.

При роботі з ненормалізованими відношеннями аналогічні труднощі будуть виникати і при виконанні операцій видалення та модифікації кортежів.

3.4. Реляційна модель даних

Коли в попередніх розділах ми говорили про основні поняття реляційних баз даних, ми не спиралися на будь-яку конкретну реалізацію. Ці міркування в рівній мірі відносяться до будь-якої системи, при побудові якої використовувався реляційний підхід. Іншими словами, ми використовували поняття так званої реляційної моделі даних.

Модель даних (в контексті області баз даних) описує якийсь набір родових понять і ознак, якими повинні володіти всі конкретні СКБД і керовані ними бази даних, якщо вони ґрунтуються на цій моделі. Наявність моделі даних дозволяє порівнювати конкретні реалізації, використовуючи один спільну мову. Хоча поняття моделі даних є загальним, і можна говорити про ієрархічної, мережевої та інших моделях даних, потрібно відзначити, що в області баз даних це поняття було введено Едгаром Коддом стосовно до реляційних систем і найбільш ефективно використовується саме в даному контексті.

3.4.1. Загальна характеристика

Хоча поняття реляційної моделі даних першим ввів основоположник реляційного підходу Едгар Кодд, найбільш поширене трактування реляційної моделі даних, очевидно, належить відомому популяризаторові ідей Кодда Крістоферу Дейта, який відтворює її (з різними уточненнями) практично у всіх своїх книгах. Згідно з трактуванням Дейта, реляційна модель складається з трьох частин, що описують різні аспекти реляційного підходу: структурної частини, маніпуляційної частини і цілісної частини.

У структурної частини моделі фіксується, що єдиною основою структурою даних, що використовується в реляційних БД, є нормалізоване n -арне відношення. Визначаються поняття доменів, атрибутів, кортежів, заголовка, тіла і змінної відношення. У двох попередніх розділах цієї лекції ми розглядали саме поняття і властивості структурної складової реляційної моделі.

У маніпуляційній частині моделі визначаються два фундаментальні механізми маніпулювання реляційними БД – реляційна алгебра і реляційне числення. Перший механізм базується в основному на класичній теорії множин (з деякими уточненнями і доповненнями), а другий – на класичному логічному апараті обчислення предикатів першого порядку. Ми розглянемо ці механізми більш детально в наступних лекціях, а поки лише зауважимо, що основною функцією маніпуляційної частини реляційної моделі є забезпечення заходів реляційної будь-якого конкретного мови реляційних БД: мова називається реляційною, якщо вона має не меншу виразність і потужність, ніж реляційна алгебра або реляційне обчислення.

В основу розробки реляційної бази даних покладено наступний принцип: елемент даних, будучи збережений в одному місці бази, не повинен повторюватися в інших її місцях. Це дає відразу дві переваги: скорочення обсягу необхідного дискового простору, а також для спрощення супроводу даних і усунення аномалій в організації зберігання даних. При розробці реляційної бази даних цей принцип реалізується шляхом розбиття даних на частини – пов'язані один з одним файли (таблиці), і називається нормалізацією.

3.4.2. Взаємозв'язок файлів (таблиць)

Між будь-якими двома таблицями реляційної бази даних можна встановити три типи зв'язків: Один-до-Одного; Один-до-Багатьох (або батько-діти) і зворотні Багато-до-Одного; Багато-до-Багатьох. Ці відношення характеризують яке число записів однієї таблиці пов'язано з яким числом записів іншої таблиці.

Реляційні зв'язки (відношення) між таблицями призначені для розбивки таблиць на самодостатні частини. Розглянемо приклад. Припустимо, нам треба створити таблицю – Студенти. Таблиця призначена для того щоб вказати, які іспити були здані конкретним студентом (рис. 3.6).

№	Фамилия	Имя	Отчество	Экзамен
1	Иванов	Иван	Иванович	Математика
2	Иванов	Иван	Иванович	Физика
3	Петров	Петр	Петрович	Рус. Яз.
4	Петров	Петр	Петрович	Литература
5	Сидоров	Сидор	Сидорович	Сопр.мат.
6	Сидоров	Сидор	Сидорович	Теор.вер.

Рис. 3.6. Таблица Студенти

Відразу кидається в очі недолік такого проектування: дані з полів «Прізвище», «Ім'я» і «По батькові» багаторазово повторюються. Користувачеві доведеться вводити велику кількість дублюючих даних, а таблиця виходить «розпухлою», переповненій цими даними. Виправити становище нескладно, потрібно лише розбити цю таблицю на дві різні таблиці, які мають реляційний зв'язок Один-до-Багатьох (рис. 3.7).

Як ви можете помітити, надмірність даних зникла – в одній таблиці представлені тільки дані по студентах, в іншій – дані по іспитів. Зв'язок між таблицями організована по ключовому полю «№» таблиці зі студентами. У таблиці з іспитами, замість повних даних про студента вписується тільки його номер. Студент може здати скільки завгодно багато іспитів, користувач же просто вибере його зі списку, і в таблицю потрапить його номер. Таку таблицю легше заповнювати, і розмір її буде теж менше.



Рис. 3.7. Реляційний зв'язок Один-до-Багатьох

Зв'язок, представлена тут відношенням Один-до-Багатьох, означає, що з однієї таблиці може мати зв'язок з безліччю записів з іншої таблиці. Однак є можливість і того, що запис з першої таблиці не матиме жодних зв'язків з іншою таблицею – студент може ще не здати жодного іспиту. При створенні зв'язків, як правило, одна таблиця називається головною (master), інша – підлеглої (details). У нашому випадку головною є таблиця зі студентами. Таблиця зі списком зданих іспитів – підпорядкована.

Відношення Один-до-Одного означає, що рівно один запис в одній таблиці пов'язаний з рівно одним записом іншої таблиці або такий зв'язок відсутній. Це відношення корисно тоді, коли поля деякої таблиці не завжди вимагають наявності в них інформації. При зосередженні таких полів в одній таблиці відбувається марна трата дискового простору під порожні поля записів, які не потребують додаткової інформації. Тому вигідно створити другу таблицю з відношенням Один-до-Одного з першої таблицею, в якій зберігалися б необов'язкові поля з інформацією в них (рис. 3.8).

Зв'язок Один-до-Одного ще використовують для того, щоб відокремити головну інформацію від другорядних даних. Наприклад, дані про студентів, такі як прізвище, група, можуть часто використовуватися для самих різних звітів. Однак домашня адреса і телефон студентів потрібні далеко не завжди, тому вони винесені в іншу таблицю.

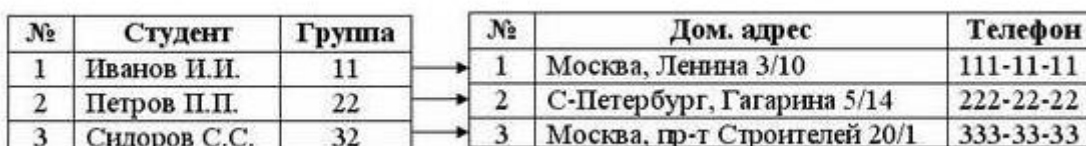


Рис. 3.8. Реляційний зв'язок Один-до-Одного

Якби ми об'єднали ці таблиці в одну, то отримали б таблицю з надлишком даних. У більшості багатокористувацьких системах також застосовують зв'язок Один-до-Одного, коли розбивають таблицю на дві різні таблиці, в якій зберігається два поля, одне з яких змінюється вручну, а друге змінюється автоматично.

Найважче мати справу з відношенням Багато-до-Багатьох. Воно означає, що деяка множина записів одного файлу пов'язано з деякою множиною записів іншого файлу.

Поширимо наш приклад на промисловий концерн, який купує деталі і виробляє вироби. Одна деталь може входити в різні вироби, а один виріб може складатися з багатьох деталей.

Таблиця деталі:

Номер деталі – первинний ключ

опис деталі

Таблиця вироби:

номер виробу – первинний ключ

Клацніть, щоб подивитися

Так як в реляційних БД немає відношення Багато-до-Багатьох, то, не вдаючись у теоретичні подробиці, зазначимо, що в цьому випадку необхідно створити третю таблицю, яку прийнято називати Сполучною таблицею. Як видно з прикладу Сполучна таблиця породжує два відношення Один-до-Багатьох:

Таблиця Деталі:

Номер деталі – первинний ключ (--- >>)

опис деталі

Таблиця Деталь-Виріб:

Номер деталі – 1-й компонент первинного ключа і зовнішній ключ (<< ---)

Номер виробу – 2-й компонент первинного ключа і зовнішній ключ (<< ---)

кількість деталей

Таблиця Вироби:

Номер виробу – первинний ключ (--- >>)

Клацніть, щоб подивитися

У таблиці Деталь-Виріб є складним первинним ключем і два зовнішні ключа. Між таблицею Деталі й таблицею Деталь-Виріб встановлено відношення Один-до-Багатьох. Цей же тип відношення встановлений між таблицею Виріб і таблицею Деталь-Виріб. Сполучний файл став ніби «посередником» між двома таблицями, пов'язаними відношенням Багато-до-Багатьох.

3.4.3. Цілісність суті і посилань

У цілісній частині реляційної моделі даних фіксуються дві базові вимоги цілісності, які повинні підтримуватися в будь-якій реляційній СКБД.

Перша вимога називається вимогою цілісності сутності (entity integrity). Об'єкту або сутності реального світу в реляційних БД відповідають кортежі відношень. Саме вимога полягає в тому, що будь-який кортеж будь-якого значення-відношення будь-якої змінної відношення повинен бути відрізнити від будь-якого іншого кортежу цього значення відношення по складовим значенням заздалегідь певної множини атрибутів змінної відношення, тобто, іншими словами, будь-яка змінна відношення повинна володіти первинним ключем. Як ми бачили в попередньому розділі, це вимога автоматично задовольняється, якщо в системі не порушуються базові властивості відношень.

Насправді, вимога цілісності сутності повністю звучить наступним чином: у будь-якої змінної відношення повинен існувати первинний ключ, і ніяке значення первинного ключа в кортежі значення-відношення не має містити невизначених значень. Щоб це формулювання було повністю зрозумілим, ми повинні хоча б коротко обговорити поняття невизначеного значення (NULL).

Звичайно, теоретично будь-який кортеж, що заноситься в збережене відношення, повинен містити всі характеристики модельованої ним сутності реального світу, які ми хочемо зберегти в базі даних. Однак на практиці не всі ці характеристики можуть бути відомі до того моменту, коли

потрібно зафіксувати сутність в базі даних. Простим прикладом може бути процедура прийняття на роботу людину, розмір заробітної плати якого ще не визначено. В цьому випадку службовець відділу кадрів, який заносить в відношення СЛУЖБОВЦІ кортеж, що описує нового службовця, просто не може забезпечити значення атрибута СЛУ_ЗАРП (Будь-яке значення домену РАЗМЕРИ_ВИПЛАТ буде невірною характеризувати зарплату нового службовця).

Едгар Кодд [9] запропонував використовувати в таких випадках невизначені значення (NULL). Невизначене значення не належить ніякому типу даних і може бути присутнім серед значень будь-якого атрибута, визначеного на будь-якому типі даних (якщо це явно не заборонено при визначенні атрибута).

Перша з вимог – вимога цілісності сутності – означає, що первинний ключ повинен повністю ідентифікувати кожну сутність, а тому в складі будь-якого значення первинного ключа не допускається наявність невизначених значень.

Друга вимога, яка називається вимогою цілісності по посиланнях (referential integrity), є більш складною. Очевидно, що при дотриманні нормалізованості відношень складні сутності реального світу представляються в реляційній БД у вигляді декількох кортежів декількох відношень.

Вимога цілісності по посиланнях, або вимога цілісності зовнішнього ключа, полягає в тому, що для кожного значення зовнішнього ключа, що з'являється в кортежі значення – відношення, повинен знайтися кортеж з таким же значенням первинного ключа, або значення зовнішнього ключа повинно бути повністю невизначеним (тобто, ні на що не вказувати). Для нашого прикладу це означає, що якщо для службовця вказано номер відділу, то цей відділ повинен існувати. Зауважимо, що, як і первинний ключ, зовнішній ключ повинен специфікуватися при визначенні змінної відношення і являє собою обмеження на допустимі значення – відношення цієї змінної. Іншими словами, визначення зовнішнього ключа є визначення обмеження цілісності бази даних.

Обмеження цілісності суті і за посиланнями повинні підтримуватися СКБД. Для дотримання цілісності сутності досить гарантувати відсутність в будь-якій змінній відношення значень – відношення, що містять кортежі з одним і тим же значенням первинного ключа і забороняти входження в значення первинного ключа невизначених значень. З цілісністю по посиланнях справа дещо складніша.

Зрозуміло, що при оновленні посилається відношення (вставці нових кортежів або модифікації значення зовнішнього ключа в існуючих кортежі) досить стежити за тим, щоб не з'являлися некоректні значення зовнішнього ключа. Але як бути при видаленні кортежу з відношення, на яку веде посилання?

Тут існують три підходи, кожен з яких підтримує цілісність по посиланнях.

Перший підхід полягає в тому, що взагалі забороняється проводити видалення кортежу, для якого існують посилання (тобто, спочатку потрібно або видалити посилаються кортежі, або відповідним чином змінити значення їх зовнішнього ключа).

Обмежити дію означає, що коли користувач намагається видалити запис з первинним ключем або змінити значення первинного ключа, то дія дозволено, якщо тільки немає зовнішніх ключів, які посилаються на даний первинний ключ. Наприклад, видаляються лише ті постачальники, які ще не здійснювали поставок. Інакше операція видалення відкидається.

При **другому підході** при видаленні кортежу, на який є посилання, у всіх посилань кортежу значення зовнішнього ключа автоматично стає повністю невизначеним.

Нарешті, **третій підхід (каскадне видалення)** полягає в тому, що при видаленні кортежу з відношення, на яке веде посилання, з посилаючого відношення автоматично видаляються всі посилаючи кортежі.

При каскадуванні дії слід враховувати, що каскадування поширюється на записи всіх залежних файлів. При створенні програми обробки даної ситуації необхідно знати взаємозв'язку файлів і програмувати каскадування на всю «глибину» зв'язків, щоб виключити можливість «повисання» зв'язку.

Що повинно відбуватися при спробі оновлення первинного ключа цільової сутності, на яку посилається деякий зовнішній ключ? Цей випадок для визначеності знову розглянемо докладніше. Є ті ж три можливості, як і при видаленні.

1. Оновлюються первинні ключі лише тих сутностей, які ще не пов'язані з іншими сутностями. Наприклад, оновлюються первинні ключі лише тих Постачальників, які ще не здійснювали Постачань. Інакше операція оновлення відкидається.
2. Коли користувач намагається змінити значення первинного ключа, то обнуляються значення зовнішніх ключів, що посилаються на первинний ключ (якщо значення зовнішніх ключів допускають нульове значення). Така можливість, звичайно, не застосовується, якщо даний зовнішній ключ не повинен містити NULL-значень. При обнуленні обнуляються тільки окремі зовнішні ключі, що посилаються на первинний ключ, який змінюють.
3. Операція оновлення «каскадується» з тим, щоб оновити первинні і зовнішні ключі у всіх пов'язаних сутності.

У розвинених реляційних СКБД зазвичай можна вибрати спосіб підтримки цілісності по посиланнях для кожного випадку визначення зовнішнього ключа. Звичайно, для прийняття такого рішення необхідно аналізувати вимоги конкретної прикладної області.

На закінчення хотілося б відзначити, що потенційні слухачі (читачі) цього курсу працюють або працюватимуть з будь-якої SQL-орієнтованої СКБД. Будь-яка компанія, яка виробляє подібні СКБД, називає їх реляційними системами.

Дуже важливо чітко розуміти, які властивості таких систем дійсно є реляційними, а що в них не повною мірою відповідає вихідним, ясним і суворим ідеям реляційного підходу і навіть суперечить їм. Це допоможе більш правильно організовувати бази даних і будувати додатки в середовищі SQL-орієнтованої СКБД. Вже згадана реляційна модель даних призначена, перш за все, для оцінки відповідності різних реалізацій СКБД загального реляційному підходу.

3.5. Індексування

Індекс являє собою засіб прискорення пошуку записів в таблиці, а також інших операцій, що використовують пошук: витяг, модифікацію, сортування тощо. Таблицю, для якої використовують індекс, називають індексованою.

Індекс містить відсортовану по колонці або декільком колонкам інформацію і вказує на рядки, в яких зберігається конкретне значення колонки. Індекс виконує роль змісту таблиць, перегляд якого передуює зверненню до записів таблиці. У деяких системах індекси зберігаються в індексних файлах окремо від табличних.

Рішення проблеми організації фізичного доступу до інформації залежить в основному від наступних факторів:

- виду вмісту в поле ключа записів індексного файлу;
- типу використовуваних посилань (покажчиків) на запис основної таблиці;
- методу пошуку потрібних записів.

Індексний файл – це файл особливого типу, в якому кожен запис складається з двох значень: дане і RID-покажчик (Record Identifier). На практиці найчастіше використовуються два методи пошуку: послідовний і бінарний (заснований на поділі інтервалу пошуку навпіл).

Пошук необхідних записів при індексації може відбуватися за однорівневою або дворівневою схемою індексації. При однорівневій схемі в індексному файлі зберігаються короткі записи, що мають два поля: поле вмісту старшого ключа (хеш-коду ключа) адресується блоку і поле адреси початку цього блоку (рис. 3.9.).

У кожному блоці запису розташовуються в порядку зростання значення ключа (або згортки). Старшим ключем кожного блоку є ключ його останнього запису.

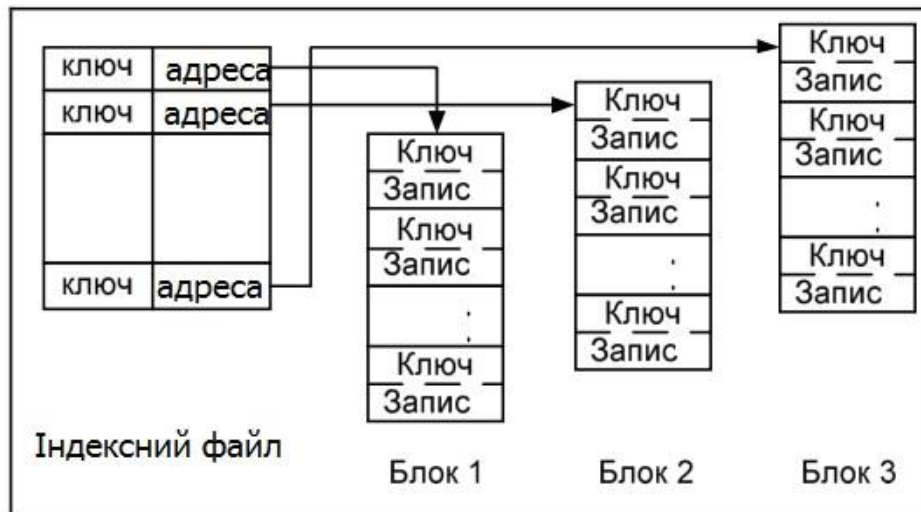


Рис. 3.9. Однорівнева схема індексації

Якщо в індексному файлі зберігаються хеш-коди ключових полів індексованої таблиці, то алгоритм пошуку потрібного запису включає три етапи.

1. Освіта згортки значення ключового поля шуканого запису.
2. Пошук в індексному файлі запису блоку, значення першого поля якого більше отриманої згортки (це гарантує знаходження шуканої згортки в даному блоці).
3. Послідовний перегляд записів блоку до збігу згорток шуканого запису і запису блоку файлу. У разі колізій згорток (кільком різних ключам може відповідати одне значення хеш-функції, тобто, одна адреса) йде пошук запису, значення ключа якого збігається зі значенням ключа шуканого запису.

Недолік однорівневої схеми – ключі (згортки) записів зберігаються разом із записами, що призводить до збільшення часу пошуку записів через велику довжину перегляду (значення даних в записах доводиться пропускати).

У дворівневій схемі ключі (згортки) записів відокремлені від вмісту записів (рис. 3.10).

В даному випадку індекс основної таблиці розподілений по сукупності файлів: одному файлу головного індексу і множини файлів з блоками ключів.

На практиці при створенні індексу для деякої таблиці бази даних вказують поле таблиці, яке вимагає індексації. Ключові поля таблиці в багатьох СКБД індексуються автоматично. Індексні файли, створювані по ключових полях таблиці, називаються фалами первинних індексів.

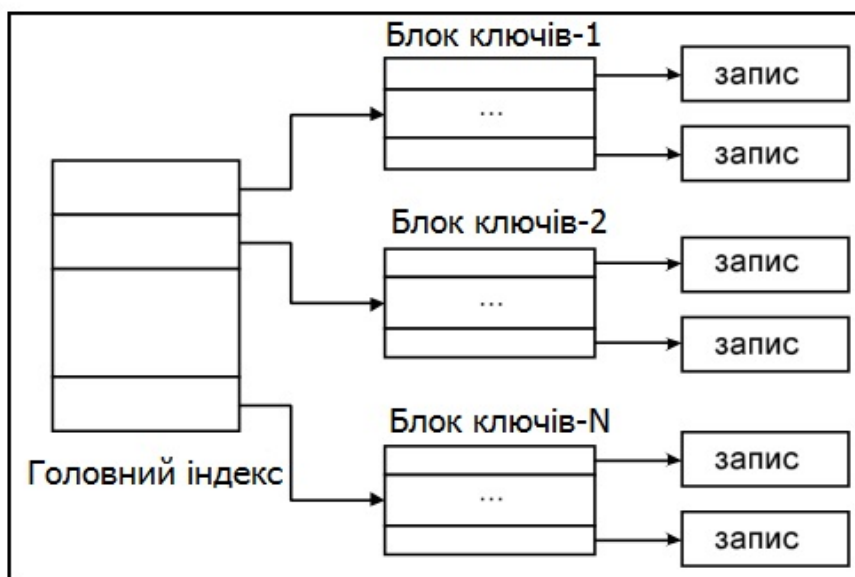


Рис. 3.10. Дворівнева схема індексації

Індекси, що створюються не для ключових полів, називаються вторинними (для користувача) індексами. Введення цих індексів не змінює фізичного розташування записів таблиці, але впливає на послідовність перегляду записів. Індексні файли, створювані для підтримки вторинних індексів таблиці, називаються файлами вторинних індексів.

Деякі СКБД підтримують Групові та Групові хешовані індекси.

Кластеризація – приміщення в один блок записів таблиць, які з великою ймовірністю будуть часто піддаватися з'єднанню.

Кластеризований індекс – спеціальна техніка індексування, при якій дані в таблиці фізично розташовуються в індексованому порядку. Використання кластеризованого індексу значно прискорює виконання запитів по індексованій колонці. Для кожної таблиці може існувати тільки один кластеризований індекс. При створенні кластеризованого індексу за первинним ключем автоматично знімається кластеризація за первинним ключем.

Хешування – альтернативний спосіб зберігання даних в заздалегідь заданому порядку з метою прискорення пошуку (прямий доступ). Хешування позбавляє від необхідності підтримувати і переглядати індекси.

Кластеризований хешований індекс значно прискорює операції пошуку і сортування, але додавання і видалення рядків сповільнюється через необхідність реорганізації даних для відповідності індексу. Хешування застосовується в тому випадку, коли необхідний прямий доступ (без індексів), наприклад, при бронюванні авіаквитків, місць в готелях, прокаті машин, а також електронних грошових переказах. Однак недоліком хешування є необхідність знаходження відповідної хеш-функції, необхідність виконання операції згортки (вимагає певного часу), можливі колізії (згортка різних значень може дати однаковий хеш-код) і проміжки між записами невизначеної довжини.

При хешуванні RID-показчик обчислюється за допомогою деякої хеш-функції і називається хеш-кодом. В поле ключа індексного файлу можна зберігати значення ключових полів індексованої таблиці або згортку ключа (хеш-код). Довжина хеш-коду завжди постійна і має досить малу величину (наприклад, 4 байта), а це істотно знижує час пошукових операцій.

Загальним недоліком індексних схем є необхідність зберігання індексів, до яких потрібно звертатися для виявлення записів.

4. ЗАСОБИ МАНІПУЛЮВАННЯ РЕЛЯЦІЙНИМИ ДАНИМИ

4.1. Вступ

У попередній лекції згадувалися три складові реляційної моделі даних. Дві з них – структурна і цілісна частини – були розглянуті детально, а маніпуляційній частині реляційної моделі даних присвячується ця лекція.

Цій темі приділимо велику увагу, оскільки розуміння формальних механізмів маніпулювання реляційними даними виключно важливо для розуміння технології баз даних взагалі. У цій лекції після невеликого введення буде розглянуто варіант реляційної алгебри Кодда [1], яка запропонована Крістофером Дейтом близько 15 років тому. Потім ми обговоримо новий «мінімальний» варіант алгебри, запропонований Дейтом і Дарвенем в кінці 1990-х рр. Після цього ми перейдемо до реляційного обчислення, досить докладно розглянемо один з варіантів реляційного числення кортежів і коротко обговоримо особливості обчислення доменів.

Як уже зазначалося в попередній лекції, в маніпуляційній складовій реляційної моделі даних визначаються два базових механізми маніпулювання реляційними даними – заснована на теорії множин реляційна алгебра і засноване на математичній логіці (точніше, на численні предикатів першого порядку) реляційне числення. У свою чергу, зазвичай виділяються два види реляційного числення – числення кортежів і числення доменів.

Всі ці механізми мають одну важливу властивість: вони замкнені відносно поняття відношення. Це означає, що вирази реляційної алгебри і формули реляційного числення визначаються над відношеннями реляційних БД і результатом їх «обчислення» також є відношення. В результаті будь-який вираз або формула можуть інтерпретуватися як відношення, що дозволяє використовувати їх в інших виразах або формулах.

Конкретна мова маніпулювання реляційними БД називається реляційно повною, якщо будь-який запит формулюється за допомогою одного вираження реляційної алгебри або однієї формули реляційного числення, може бути сформульований за допомогою одного оператора цієї мови.

Відомо, що механізми реляційної алгебри і реляційного числення еквівалентні, тобто для будь-якого допустимого вираження реляційної алгебри можна побудувати еквівалентну, що виробляє такий же результат, формулу реляційного обчислення і навпаки.

Вирази реляційної алгебри будуються на основі алгебраїчних операцій, і подібно до того, як інтерпретуються арифметичні і логічні вирази, вирази реляційної алгебри також має процедурну інтерпретацію. Іншими словами, запит, поданий на мові реляційної алгебри, то, можливо вирахувати на основі виконання елементарних алгебраїчних операцій з урахуванням їх пріоритетності та можливої наявності дужок. Мови реляційного числення є більшою мірою непроцедурними, або декларативними.

Оскільки механізми реляційної алгебри і реляційного числення еквівалентні, в конкретній ситуації для перевірки ступеня реляційної деякої мови БД можна користуватися будь-яким з цих механізмів. Зауважимо, що вкрай нечасто алгебра або числення приймається в якості повної основи будь-якої мови БД. Зазвичай (наприклад, в разі мови SQL) мова ґрунтується на деякій суміші алгебраїчних і логічних конструкцій. Проте, знання алгебраїчних і логічних основ мов баз даних часто застосовується на практиці.

У цій лекції ми не будемо вводити будь-які суворі синтаксичні конструкції при розгляді механізмів реляційної алгебри і реляційного числення, а в основному обмежимося розглядом матеріалу на змістовному рівні.

4.2. Огляд реляційної алгебри Кодда

Основна ідея реляційної алгебри полягає в тому, що відношення є множинами, засоби маніпулювання відношеннями можуть базуватися на традиційних теоретико-множинних операціях, доповнених деякими спеціальними операціями, специфічними для реляційних баз даних.

Існує багато підходів до визначення реляційної алгебри, які розрізняються наборами операцій і способами їх інтерпретації, але, в принципі, є більш-менш рівносильними. В даному розділі ми опишемо розширений початковий варіант алгебри, який був запропонований Коддом (будемо називати її «алгеброю Кодда»). У цьому варіанті набір основних алгебраїчних операцій складається з восьми операцій, які діляться на два класи – теоретико-множинні операції та спеціальні реляційні операції. До складу теоретико-множинних операцій входять операції [10]:

- об'єднання відношень;
 - перетину відношень;
 - взяття різниці відношень;
 - взяття декартового добутку відношень.
- Спеціальні реляційні операції включають:
- обмеження відношень;
 - проекцію відношень;
 - з'єднання відношень;
 - поділ відношень.

Крім того, до складу алгебри включається операція присвоювання, що дозволяє зберегти в базі даних результати обчислення алгебраїчних виразів, і операція перейменування атрибутів, що дає можливість коректно сформулювати заголовок (схему) результуючого відношення.

4.2.1. Загальна інтерпретація реляційних операцій

Якщо не вдаватися в деякі тонкощі, які ми розглянемо в наступних розділах, то майже для всіх операцій запропонованого вище набору є очевидна і проста інтерпретація.

При виконанні операції об'єднання (**UNION**) двох відношень з однаковими заголовками виробляється відношення, що включає всі кортежі, що входять хоча б в одне з відношень-операндів.

Операція перетину (**INTERSECT**) двох відношень з однаковими заголовками виробляє відношення, що включає всі кортежі, що входять в обидва відношень-операнда.

Відношення, що є різницею (**MINUS**) двох відношень з однаковими заголовками, включає всі кортежі, що входять до відношення-перший операнд, такі, що жоден з них не входить у відношення, яке є другим операндом.

При виконанні декартового добутку (**TIMES**) двох відношень, перетин заголовків в яких порожньо, виробляється відношення, кортежі якого виробляються шляхом об'єднання кортежів першого і другого операндів.

Результатом обмеження (**WHERE**) відношення по деякому умові є відношення, що включає кортежі відношення-операнда, яке задовольняє цій умові.

При виконанні проекції (**PROJECT**) відношення на задану підмножину множини його атрибутів виробляється відношення, кортежі якого є відповідними підмножинами кортежів відношення-операнда.

При з'єднанні (**JOIN**) двох відношень за деякою умовою утворюється результуюче відношення, кортежі якого виробляються шляхом об'єднання кортежів першого і другого відношення і задовольняють цій умові.

В операції реляційного поділу (**DIVIDE BY**) два операнда – бінарне і унарне відношення. Результуюче відношення складається з унарних кортежів, що включають значення першого атрибута кортежів першого операнда таких, що множина значень другого атрибута (при фіксованому значенні першого атрибута) включає множину значень другого операнда.

Операція перейменування (**RENAME**) виробляє відношення, тіло якого збігається з тілом операнда, але імена атрибутів змінені.

Операція присвоювання (**: =**) дозволяє зберегти результат обчислення реляційного виразу в існуючому відношенні БД.

Оскільки результатом будь-якої реляційної операції (крім операції присвоювання, яка не виробляє значення) є якесь відношення, можна утворювати реляційні вирази, в яких замість

відношення-операнда деякої реляційної операції знаходиться вкладений реляційний вираз. У побудові реляційного вираження можуть брати участь всі реляційні операції, крім операції присвоювання. Обчислювальна інтерпретація реляційного вираження диктується встановлених пріоритетів операцій, показаних на рис. 4.1. Обчислення виразу проводиться зліва направо з урахуванням пріоритетів операцій і дужок.

Операція	Пріоритет
RENAME	4
WHERE	3
PROJECT	3
TIMES	2
JOIN	2
INTERSECT	2
DIVIDE BY	2
UNION	1
MINUS	1

Рис. 4.1. Таблиця пріоритетів операцій традиційної реляційної алгебри

4.2.2. Замкнутість реляційної алгебри і операція перейменування

Як було зазначено раніше, кожне значення-відношення характеризується заголовком (або схемою) і тілом (або безліччю кортежів). Тому, якщо нам дійсно потрібна алгебра, операції якої замкнуті щодо поняття відношення, то кожна операція повинна виробляти відношення в повному розумінні, тобто він повинен мати і тілом, і заголовком. Тільки в цьому випадку можна буде будувати вкладені вирази.

Тема відношення є множина пар $\langle \text{ім'я-атрибути}, \text{ім'я-домену} \rangle$. Якщо подивитися на загальний огляд реляційних операцій, наведений у попередньому підрозділі, то видно, що домени атрибутів результуючого відношення однозначно визначаються доменами відношень-операндів. Однак з іменами атрибутів результату не завжди все так просто.

Наприклад, уявімо собі, що у відношень-операндів операції декартового добутку є однойменні атрибути з однаковими доменами. Яким був би заголовок результуючого відношення? Оскільки це множина, в ній не повинні міститися однакові елементи. Але і втратити атрибут в результаті неприпустимо. А це означає, що в такому випадку взагалі неможливо коректно виконати операцію декартового добутку.

Аналогічні проблеми можуть виникати і у випадках інших двомісних операцій. Для вирішення проблем в число операцій реляційної алгебри вводиться операція перейменування. Її слід застосовувати в тому випадку, коли виникає конфлікт іменування атрибутів у відношеннях-операнда однієї реляційної операції. Тоді до одного з операндів спочатку застосовується операція перейменування, а потім основна операція виконується вже без всяких проблем. Більш строго ми визначимо операцію перейменування в наступній лекції, а поки лише зауважимо, що результатом цієї операції є відношення, що збігається в усьому з відношенням-операндом, крім того, що ім'я зазначеного атрибута змінено на задане ім'я.

У подальшому викладі ми будемо припускати застосування операції перейменування у всіх конфліктних ситуаціях.

4.3. Особливості теоретико-множинних операцій реляційної алгебри

Хоча в основі теоретико-множинної частини реляційної алгебри Кодда лежить класична теорія множин, відповідні операції реляційної алгебри володіють деякими особливостями.

4.3.1 Операції об'єднання, перетину, взяття різниці

Почнемо з операції об'єднання відношень (все, що буде сказано з приводу об'єднання, вірно і для операцій перетину і взяття різниці відношень). Сенса операції об'єднання в реляційній алгебрі в цілому залишається теоретико-множинним. Ще раз нагадаємо, що в теорії множин [10]:

- результатом об'єднання двох множин $A\{a\}$ і $B\{b\}$ є така множина $C\{c\}$, що для кожного z або існує такий елемент a , що належить множині A , що $c=a$, або існує такий елемент b , що належить множині B , що $c = b$;
- перетином множин A і B є така множина $C\{c\}$, що для будь-якого c існують такі елементи a , що належить множині A , і b , що належить множині B , що $c = a=b$;
- різницею множин A і B є така множина $C\{c\}$, що для будь-якого c існує такий елемент a , що належить множині A , що $c=a$, і не існує такий елемент b , що належить B , що $c=b$.

Але якщо в теорії множин операція об'єднання осмислена для будь-яких двох множин-операндів, то в разі реляційної алгебри результатом операції об'єднання повинно бути відношення. Якщо в реляційній алгебрі допустити можливість теоретико-множинного об'єднання двох довільних відношень (з різними заголовками), то, звичайно, результатом операції буде множина, але множина різнотипних кортежів, тобто не відношення. Якщо виходити з вимоги замкнутості реляційної алгебри щодо поняття відношень, то така операція об'єднання є безглуздою.

Ці міркування підводять до поняття сумісності відношень з об'єднання: два відношення сумісні з об'єднання в тому і тільки в тому випадку, коли мають однакові заголовки. У розгорнутій формі це означає, що в заголовках обох відношень міститься один і той же набір імен атрибутів, і однойменні атрибути визначені на одному і тому ж домені.

Якщо два відношення сумісні з об'єднання, то при звичайному виконанні над ними операцій об'єднання, перетину і взяття різниці результатом операції є відношення з коректно певним заголовком, що збігається з заголовком кожного з відношень-операндів.

Для ілюстрації операцій об'єднання, перетину і взяття різниці припустимо, що в базі даних є два відношення `СЛУЖАЩИЕ_В_ПРОЕКТЕ_1` і `СЛУЖАЩИЕ_В_ПРОЕКТЕ_2` з однаковими схемами `{СЛУ_НОМЕР, СЛУ_ІМЯ, СЛУ_ЗАРП, СЛУ_ОТД_НОМЕР}`. Кожне з відношень містить дані про службовців, що беруть участь у відповідному проекті. На рис. 4.2 [10] показано приблизне наповнення кожного з двох відношень (деякі службовці беруть участь в обох проектах).

Тоді виконання операції `СЛУЖАЩИЕ_В_ПРОЕКТЕ_1 UNION СЛУЖАЩИЕ_В_ПРОЕКТЕ_2` дозволить отримати інформацію про всі службовців, що беруть участь в обох проектах. Операція перетину `СЛУЖАЩИЕ_В_ПРОЕКТЕ_1 INTERSECT СЛУЖАЩИЕ_В_ПРОЕКТЕ_2` дозволить отримати дані про службовців, які одночасно беруть участь у двох проектах.

СЛУЖАЩИЕ_В_ПРОЕКТЕ_1			
СЛУ_НОМЕР	СЛУ_ИМЯ	СЛУ_ЗАРП	СЛУ_ОТД_НОМЕР
2934	Иванов	22000.00	310
2935	Петров	30000.00	310
2936	Сидоров	18000.00	313
2937	Федоров	20000.00	310
2938	Иванова	22000.00	315

СЛУЖАЩИЕ_В_ПРОЕКТЕ_2			
СЛУ_НОМЕР	СЛУ_ИМЯ	СЛУ_ЗАРП	СЛУ_ОТД_НОМЕР
2934	Иванов	22000.00	310
2935	Петров	30000.00	310
2939	Сидоренко	18000.00	313
2940	Федоренко	20000.00	310
2941	Иваненко	22000.00	315

Рис. 4.2. Зразкове наповнення двох відношень

Нарешті, операція СЛУЖАЩИЕ_В_ПРОЕКТЕ_1 MINUS СЛУЖАЩИЕ_В_ПРОЕКТЕ_2 виробить відношення, що містить кортежі службовців, які беруть участь тільки в першому проекті (рис. 4.3) [10].

СЛУЖАЩИЕ_В_ПРОЕКТЕ_1 UNION СЛУЖАЩИЕ_В_ПРОЕКТЕ_2			
СЛУ_НОМЕР	СЛУ_ИМЯ	СЛУ_ЗАРП	СЛУ_ОТД_НОМЕР
2934	Иванов	22000.00	310
2935	Петров	30000.00	310
2939	Сидоренко	18000.00	313
2940	Федоренко	20000.00	310
2941	Иваненко	22000.00	315
2936	Сидоров	18000.00	313
2937	Федоров	20000.00	310
2938	Иванова	22000.00	315

СЛУЖАЩИЕ_В_ПРОЕКТЕ_1 INTERSECT СЛУЖАЩИЕ_В_ПРОЕКТЕ_2			
СЛУ_НОМЕР	СЛУ_ИМЯ	СЛУ_ЗАРП	СЛУ_ОТД_НОМЕР
2934	Иванов	22000.00	310
2935	Петров	30000.00	310

СЛУЖАЩИЕ_В_ПРОЕКТЕ_1 MINUS СЛУЖАЩИЕ_В_ПРОЕКТЕ_2			
СЛУ_НОМЕР	СЛУ_ИМЯ	СЛУ_ЗАРП	СЛУ_ОТД_НОМЕР
2936	Сидоров	18000.00	313
2937	Федоров	20000.00	310
2938	Иванова	22000.00	315

Рис. 4.3. Результати виконання операцій UNION, INTERSECT і MINUS

Операція об'єднання проводиться тільки з таблицями, сумісними з об'єднання, наприклад, з ідентичною структурою. В результаті ми отримаємо сумарну таблицю, яка містить рядки з двох вихідних відношень.

Представлена на рис. 4.4 операція об'єднання підсумовує записи з таблиць STUDENTS_A і STUDENTS_B в результуюче відношення. Якби вихідні таблиці включали однакові за змістом рядки, то в результаті операції об'єднання з підсумкової множини були б видалені записи-дублікати.

Таблиця STUDENTS_A		Таблиця STUDENTS_B			
SURNAME	FNAME	SURNAME	FNAME		
Орехов	Владимир	Алфёров	Николай		
Петровский	Александр	Остапенко	Владимир		
Кульгина	Оксана	Юров	Олег		

 $\cup$

SURNAME	FNAME
Орехов	Владимир
Петровский	Александр
Кульгина	Оксана
Алфёров	Николай
Остапенко	Владимир
Юров	Олег

 $=$

SURNAME	FNAME
Орехов	Владимир
Петровский	Александр
Кульгина	Оксана
Алфёров	Николай
Остапенко	Владимир
Юров	Олег

Рис. 4.4. Демонстрація операції об'єднання

Операція перетину може проводитися з таблицями, сумісними з об'єднання, дія дозволяє виявити рядки, загальні для двох таблиць. Припустимо, ми повинні визнати, які зі студентів успішно склали іспит з вищої математики (таблиця STUDENTS_MATH) і іспит з СКБД (таблиця STUDENTS_DBMS). Перетин двох відношень дасть запитований результат (рис. 4.5).

Таблиця STUDENTS_MATH		Таблиця STUDENTS_DBMS		Перетин	
SURNAME	FNAME	SURNAME	FNAME	SURNAME	FNAME
Орехов	Владимир	Бобров	Валентин	Петровский	Александр
Петровский	Александр	Петровский	Александр	Кульгина	Оксана
Кульгина	Оксана	Кульгина	Оксана	Самойлов	Евгений
Самойлов	Евгений	Самойлов	Евгений	Кузьмина	Ирина
Кузьмина	Ирина	Кузьмина	Ирина	Алфёров	Николай
Яковенко	Константин	Ищенко	Владимир	Остапенко	Владимир
Щукин	Александр	Алфёров	Николай		
Алфёров	Николай	Остапенко	Владимир		
Остапенко	Владимир				
Юров	Олег				

 $\cap$

SURNAME	FNAME
Петровский	Александр
Кульгина	Оксана
Самойлов	Евгений
Кузьмина	Ирина
Алфёров	Николай
Остапенко	Владимир

Рис. 4.5. Демонстрація операції перетину

Операція різниці (віднімання) дозволяє з'ясувати, які з рядків першої таблиці відсутні в другій таблиці. Як і додавання, віднімання може здійснюватися тільки з таблицями, сумісними з об'єднання. Припустимо, що у нас є дві таблиці: в першій знаходяться дані про студентів навчальної групи, а в другій – дані про студентів, які успішно склали екзаменаційну сесію. Для того щоб обчислити, кому слід з'явитися на перездачу, віднімаємо з першої таблиці другу (рис. 4.6).

Таблиця STUDENTS_A		Таблиця STUDENTS_C			
SURNAME	FNAME	SURNAME	FNAME		
Орехов	Владимир	Орехов	Владимир		
Петровский	Александр	Петровский	Александр		
Кульгина	Оксана	Кульгина	Оксана		
Самойлов	Евгений	Самойлов	Евгений		
Кузьмина	Ирина	Кузьмина	Ирина		
Яковенко	Константин	Ищенко	Владимир		
Ищенко	Владимир	Алфёров	Николай		
Алфёров	Николай	Остапенко	Владимир		
Остапенко	Владимир	Юров	Олег		
Юров	Олег				

=

Різниця	
Яковенко	Константин

Рис. 4.6. Демонстрація операції віднімання

4.3.2. Операція розширеного декартового добутку

В теорії множин декартовий добуток може бути отриманий для будь-яких двох множин, та елементами результуючої множини є пари, складені з елементів першої і другої множини. Якщо говорити більш точно, декартовим добутком множин A $\{a\}$ і B $\{b\}$ є така множина пар C $\{<c1, c2>\}$, що для кожного елемента $<c1, c2>$ множини C існує такий елемент a множини A , що $c1 = a$, і такий елемент b множини B , що $c2 = b$.

Оскільки відношення є множинами, для будь-яких двох відношень можливе отримання прямого добутку. Але результат не буде відношенням. Елементами результату будуть не кортежі, а пари кортежів.

Тому в реляційній алгебрі використовується спеціалізована форма операції взяття декартового добутку – розширений декартовий добуток відношень. При взятті розширеного декартового добутку двох відношень елементом результуючого відношення є кортеж, який являє собою об'єднання одного кортежу першого відношення і одного кортежу другого відношення.

Наведемо більш точне визначення операції розширеного декартового добутку. Нехай є два відношення $R1\{A1, a2, \dots, an\}$ і $R2\{B1, b2, \dots, bm\}$. Тоді результатом операції $R1$ TIMES $R2$ є відношення $R\{A1, a2, \dots, an, b1, b2, \dots, bm\}$, тіло якого є множиною кортежів виду $\{Ra1, ra2, \dots, ran, rb1, rb2, \dots, rbm\}$.

Декартовий добуток відкриває набір операцій, здійснюваних з двома підмножинами. Відношення, що отримується в результаті декартового добутку, являє собою з'єднання рядків з одного відношення з рядками з іншого відношення. В результаті добутку кожному рядку з однієї таблиці приєднується один рядок з іншої таблиці.

На практиці подібна операція затребувана нечасто, адже в результуючій множині виявляються всі можливі поєднання рядків з двох таблиць. Однак з будь-якого правила є й винятки. Припустимо, що студенти хочуть отримати дані про те, які навчальні дисципліни їм належить вивчати, тоді без декартового добутку не обійтися (рис. 4.7).

Таблица STUDENTS				Добуток таблиц			
SURNAME	FNAME	×	Таблица DISCIPLINE	=	SURNAME	FNAME	DISCIPLINE
Орехов	Владимир		Операционные системы		Орехов	Владимир	Операционные системы
Петровский	Александр		Языки программирования		Орехов	Владимир	Языки программирования
Кульгина	Оксана		Методы программирования		Орехов	Владимир	Методы программирования
			СУБД		Орехов	Владимир	СУБД
					Петровский	Александр	Операционные системы
					Петровский	Александр	Языки программирования
					Петровский	Александр	Методы программирования
					Петровский	Александр	СУБД
					Кульгина	Оксана	Операционные системы
					Кульгина	Оксана	Языки программирования
					Кульгина	Оксана	Методы программирования
					Кульгина	Оксана	СУБД

Рис. 4.7. Демонстрація операції декартового добутку

4.4. Спеціальні реляційні операції

В цьому розділі докладніше розглянемо спеціальні реляційні операції реляційної алгебри, такі, як обмеження, проекція, з'єднання і поділ.

4.4.1. Операція обмеження

Операція обмеження **WHERE** вимагає наявності двох операндів: обмежуваного відношення і простої умови обмеження. Проста умова обмеження може мати:

- вид (***a comp-op b***), Де *a* і *b* – імена атрибутів обмежені відношення; атрибути *a* і *b* повинні бути визначені на одному і тому ж домені, для значень базового типу даних якого підтримується операція порівняння ***comp-op***, А бо на базових типах даних, над значеннями яких можна виконувати цю операцію порівняння;
- або вид (***a comp-op const***), Де *a* – ім'я атрибута обмеженого відношення, а ***const*** – літерально задана константа; атрибут *a* повинна бути визначена на домені або базовому типі, для значень якого підтримується операція порівняння ***comp-op***.

Операцією порівняння ***comp-op*** можуть бути "***=***", "***<***", "***>***", "***>=***", "***<=***", "***<>***". В результаті виконання операції обмеження виробляється відношення, заголовок якого збігається з заголовком відношення-операнда, а в тіло входять ті кортежі відношення-операнда, для яких значенням умови обмеження є ***true***. Тим самим, якщо в деяких кортежі містяться невизначені значення, і з цієї причини обчислення простої умови дає значення ***unknown***, То ці кортежі не ввійдуть в результуюче відношення.

Для позначення виклику операції обмеження використовується конструкцію ***A WHERE comp***, де ***A*** – відношення, яке обмежується, а ***comp*** – проста умова порівняння. Нехай ***comp1*** і ***comp2*** – дві прості умови обмеження. Тоді за визначенням:

- ***A WHERE (comp1 AND comp2)*** позначає те ж саме, що і ***(A WHERE comp1) INTERSECT (A WHERE comp2)***;
- ***A WHERE (comp1 OR comp2)*** позначає те ж саме, що і ***(A WHERE comp1) UNION (A WHERE comp2)***;
- ***A WHERE NOT comp1*** позначає те ж саме, що і ***A MINUS (A WHERE comp1)***.

Ці угоди дозволяють задіяти операції обмеження, в яких умовою обмеження є довільний булевий вираз, складений з простих умов з використанням логічних зв'язків AND, OR, NOT і дужок.

Результат виконання операції СЛУЖАЩИЕ_В_ПРОЕКТЕ_1 WHERE (СЛУ_ЗАРП > 20000.00 AND (СЛУ_ОТД_НОМ = 310 OR СЛУ_ОТД_НОМ = 315)) (Отримати дані з відношення СЛУЖАЩИЕ_В_ПРОЕКТЕ_1 про службовців, які працюють у відділах 310 і 315 і отримують зарплату, що перевищує 20 000.00 грн.) показаний на рис. 4.8 [10].

СЛУ_НОМЕР	СЛУ_ИМЯ	СЛУ_ЗАРП	СЛУ_ОТД_НОМЕР
2934	Иванов	22000.00	310
2935	Петров	30000.00	310
2938	Иванова	22000.00	315

Рис. 4.8. Результат операції СЛУЖАЩИЕ_В_ПРОЕКТЕ_1 WHERE

На інтуїтивному рівні операцію обмеження найкраще представляти як взяття деякої «горизонтальної» вибірки з відношення-операнда (вибірки деяких рядків з таблиці). При здійсненні вибірки з початкової множини формується результуюча підмножина, що містить тільки елементи, що задовольняють певній умові. Наприклад, за допомогою операції вибірки ми зможемо отримати список студентів, які народилися в 1990 році (рис. 4.9). Вибірка з таблиці STUDENTS склала три рядки.

Таблица STUDENTS				
S_KEY	SURNAME	FNAME	BIRTHDAY	SPECIALITY
1	Орехов	Владимир	11/04/1989	Математика
2	Петровский	Александр	21/11/1990	Математика
3	Кульгина	Оксана	5/01/1990	Ин. язык
4	Самойлов	Евгений	15/07/1991	Информатика
5	Кузьмина	Ирина	2/03/1990	Физика
6	Яковенко	Константин	12/09/1989	Химия
7	Ищенко	Владимир	09/19/1988	Астрономия

=

Вибірка з таблиці STUDENTS				
2	Петровский	Александр	21/11/1990	Математика
3	Кульгина	Оксана	5/01/1990	Ин. язык
5	Кузьмина	Ирина	2/03/1990	Физика

Рис. 4.9. Демонстрація операції вибірки

4.4.2. Операція взяття проекції

Операція взяття проекції також вимагає наявності двох операндів – проектованого відношення А і підмножини множини імен атрибутів, що входять в заголовок відношення А.

Результатом проекції відношення А на множину атрибутів {a1, a2, ..., an} (PROJECT A {a1, a2, ..., an}) є відношення з заголовком, який визначається множиною атрибутів {a1, a2, ..., an}, і з тілом, що складається з кортежів виду <a1: v1, a2: v2, ..., an: vn> таких, що стосовно А є кортеж, атрибут a1 який має значення v1, атрибут a2 має значення v2, ..., атрибут an має значення vn. Тим самим, при виконанні операції проекції виділяється «вертикальна» вирізка відношення-операнда з природним знищенням потенційно виникають кортежів-дублікатів.

Результат операції PROJECT СЛУЖАЩИЕ_В_ПРОЕКТЕ_1 {СЛУ_ОТД_НОМ} (В яких відділах працюють службовці, дані про яких містяться в відношенні СЛУЖАЩИЕ_В_ПРОЕКТЕ_1) показаний на рис. 4.10 [10].

СЛУ_ОТД_НОМЕР
310
313
315

Рис. 4.10. Результат операції PROJECT СЛУЖАЩИЕ_В_ПРОЕКТЕ_1 {СЛУ_ОТД_НОМ}

На відміну від вибірки (виділяє горизонтальні елементи початкової множини) операція проекції спрямована на отримання вертикальної складової множини. На цей раз повертаються всі рядки з вихідної таблиці, але в нову підмножину увійдуть не всі, а тільки визначені користувачем стовпці таблиці. На рис. 4.11 показана проекція, отримана з таблиці STUDENTS. У підмножині включені поля з прізвищем і ім'ям студентів.

Таблиця STUDENTS					Проекція з таблиці STUDENTS	
S_KEY	SURNAME	FNAME	BIRTHDAY	SPECIALITY		
1	Орехов	Владимир	11/04/1989	Математика	Орехов	Владимир
2	Петровский	Александр	21/11/1990	Математика	Петровский	Александр
3	Кульгина	Оксана	5/01/1990	Ин. язык	Кульгина	Оксана
4	Самойлов	Евгений	15/07/1991	Информатика	Самойлов	Евгений
5	Кузьмина	Ирина	2/03/1990	Физика	Кузьмина	Ирина
6	Яковенко	Константин	12/09/1989	Химия	Яковенко	Константин
7	Ищенко	Владимир	09/19/1988	Астрономия	Ищенко	Владимир

Рис. 4.11. Демонстрація операції проекції

4.4.3. Операція з'єднання відношень

Загальна операція з'єднання (що також називається з'єднанням за умовою) вимагає наявності двох операндів – відношення, які з'єднуються і третього операнда – простої умови. Нехай з'єднуються відношення A і B . Як і в разі операції обмеження, умова з'єднання $comp$ має вигляд або $(a \text{ } comp\text{-op } b)$, або $(a \text{ } comp\text{-op } const)$, де a і b – імена атрибутів відношень A і B , $const$ – літерально задана константа, і $comp\text{-op}$ – допустима в даному контексті операція порівняння. Тоді за визначенням результатом операції з'єднання $A \text{ JOIN } B \text{ WHERE } comp$ сумісних з узяття розширеного декартового добутку відношень A і B є відношення, що отримується шляхом виконання операції обмеження за умовою $comp$ розширеного декартового добутку відношень A і B ($A \text{ JOIN } B \text{ WHERE } comp \equiv (A \text{ TIMES } B) \text{ WHERE } comp$).

Якщо ретельно осмислити це визначення, то стане ясно, що в загальному випадку застосування умови з'єднання істотно зменшить потужність результату проміжного декартового добутку відношень-операндів тільки в тому випадку, якщо умова з'єднання має вигляд $(a \text{ } comp\text{-op } b)$, де a і b – імена атрибутів різних відношень-операндів. Тому на практиці вважають реальними операціями з'єднання саме ті операції, які ґрунтуються на умовах з'єднання наведеного вигляду. У підрозділі, що стосується операції обмеження, ми визначили трактування використання в якості обмежувача умови довільного булевого виразу, який складено з простих умов над атрибутами відношення-операнда і літеральними константами. Звичайно ж, і в операції з'єднання може здаватися довільний логічний вираз, складений з простих умов над атрибутами відношень-операндів і константами. Операцію з'єднання з такою умовою $comp$ розумно вважати операцією дійсного з'єднання, якщо воно має вигляд (або може бути перетворено до виду) $comp1 \text{ AND } (a \text{ } comp\text{-op } b)$, де a і b – імена атрибутів різних відношень-операндів.

Для ілюстрації операцій з'єднання ми трохи змінимо заголовки і тіла відношень, які використовувалися раніше у прикладах цієї лекції. Нехай тепер є відношення СЛУЖБОВЦІ {СЛУ_НОМЕР, СЛУ_ІМЯ, СЛУ_ЗАРП, ПРО_НОМ} (атрибут ПРО_НОМ в кімнаті конференцій проектів, в яких бере участь кожен службовець) і ПРОЕКТИ {ПРО_НОМ, ПРОЕКТ_РУК, ПРО_ЗАРП} (ПРО_НОМ – номер проекту, ПРОЕКТ_РУК – ім'я службовця-керівника проекту, ПРО_ЗАРП – середня заробітна плата службовців, що беруть участь в проекті). Зразковий вміст відношення СЛУЖБОВЦІ і ПРОЕКТИ показаний на рис. 4.12.

Тоді осмисленою операцією з'єднання загального вигляду буде СЛУЖБОВЦІ JOIN ПРОЕКТИ WHERE (СЛУ_ЗАРП > ПРО_ЗАРП) (Видати дані про службовців, які отримують заробітну плату, що перевищує середню заробітну плату будь-якого проекту). Результати цього запиту показані на рис. 4.13 [10].

Існує важливий окремий випадок з'єднання – **еквіз'єднання** (EQUIJOIN) і просте, але важливе розширення операції еквіз'єднання – природне з'єднання (NATURAL JOIN). Операція з'єднання називається операцією еквіз'єднання, якщо умова з'єднання має вигляд ($a=b$), Де a і b – атрибути різних операндів з'єднання. Цей випадок найчастіше зустрічається на практиці, і для нього існують найбільш ефективні алгоритми реалізації.

СЛУЖБОВЦІ			
СЛУ_НОМЕР	СЛУ_ІМЯ	СЛУ_ЗАРП	ПРО_НОМ
2934	Іванов	22400.00	1
2935	Петров	29600.00	1
2936	Сидоров	18000.00	1
2937	Федоров	20000.00	1
2938	Іванова	22000.00	1
2934	Іванов	22400.00	2
2935	Петров	29600.00	2
2939	Сидоренко	18000.00	2
2940	Федоренко	20000.00	2
2941	Іваненко	22000.00	2

ПРОЕКТИ		
ПРО_НОМ	ПРОЕКТ_РУК	ПРО_ЗАРП
1	Іванов	22400.00
2	Іваненко	22400.00

Рис. 4.12. Відношення СЛУЖБОВЦІ і ПРОЕКТИ

Операція природного з'єднання застосовується до пари відношень A і B , що володіють (можливо, складовим) загальним атрибутом c (тобто, атрибутом з одним і тим же ім'ям і певним на одному і тому ж домені). Нехай AB позначає об'єднання заголовків відношень A і B . Тоді природне з'єднання A і B – це спроектований на AB результат еквіз'єднання A і B за умовою $A_c = B_c$. Хоча операція природного з'єднання виражається через операції перейменування, з'єднання загального вигляду і проєкції, для неї зазвичай використовується скорочена форма, яка називається NATURAL JOIN.

СЛУ_НОМЕР	СЛУ_ІМЯ	СЛУ_ЗАРП	ПРО_ІМЯ	ПРО_НОМ1	ПРОЕКТ_РУК	ПРО_ЗАРП
2935	Петров	29600.00	1	1	Іванов	22400.00
2935	Петров	29600.00	2	2	Іваненко	22400.00

Рис. 4.13. Результат операції СЛУЖБОВЦІ JOIN ПРОЕКТИ WHERE (СЛУ_ЗАРП > ПРО_ЗАРП)

На рис. 4.14 [10] наведені результати операцій СЛУЖБОВЦІ JOIN (ПРОЕКТИ RENAME (ПРО_НОМ, ПРО_НОМ1)) WHERE (СЛУ_ЗАРП = ПРО_ЗАРП) (Еквіз'єднання відношень СЛУЖБОВЦІ і ПРОЕКТИ: Знайти всіх службовців, які отримують зарплату, рівну середній заробітній платі в будь-якому проекті) СЛУЖБОВЦІ NATURAL JOIN ПРОЕКТИ (Природне з'єднання – видати повну інформацію про службовців і проектах, в яких вони беруть участь).

Результат операции СЛУЖАЩИЕ JOIN (ПРОЕКТЫ RENAME (ПРО_НОМ, ПРО_НОМ1)) WHERE (СЛУ_ЗАРП = ПРО_ЗАРП)						
СЛУ_НОМЕР	СЛУ_ИМЯ	СЛУ_ЗАРП	ПРО_НОМ	ПРО_НОМ1	ПРОЕКТ_РУК	ПРО_ЗАРП
2934	Иванов	22400.00	1	1	Иванов	22400.00
2934	Иванов	22400.00	2	2	Иваненко	22400.00

Результат операции СЛУЖАЩИЕ NATURAL JOIN ПРОЕКТЫ						
СЛУ_НОМЕР	СЛУ_ИМЯ	СЛУ_ЗАРП	ПРО_НОМ	ПРОЕКТ_РУК	ПРО_ЗАРП	
2934	Иванов	22400.00	1	Иванов	22400.00	
2935	Петров	29600.00	1	Иванов	22400.00	
2936	Сидоров	18000.00	1	Иванов	22400.00	
2937	Федоров	20000.00	1	Иванов	22400.00	
2938	Иванова	22000.00	1	Иванов	22400.00	
2934	Иванов	22400.00	2	Иваненко	22400.00	
2935	Петров	29600.00	2	Иваненко	22400.00	
2939	Сидоренко	18000.00	2	Иваненко	22400.00	
2940	Федоренко	20000.00	2	Иваненко	22400.00	
2941	Иваненко	22000.00	2	Иваненко	22400.00	

Рис. 4.14. Результати операцій еквіз'єднання і природного з'єднання відношень СЛУЖБОВЦІ і ПРОЕКТИ

Всі способи з'єднання відношень (природне об'єднання, зовнішнє об'єднання і т. д.) Засновані на декартовому добутку, тільки тепер після перемноження рядків двох таблиць на результат накладаються різні додаткові обмеження.

Припустимо, в нашому розпорядженні є дві таблиці. У таблиці CHAIR зберігається інформація про кафедри університету, а в таблиці TEACHERS – про викладачів. Продемонструємо неформально найпоширенішу операцію природного з'єднання (злиття). Таблиці з'єднуються за загальними колонками, в таблиці кафедр – це первинний ключ CHAIR_KEY, а в таблиці педагогів – однойменний стовпець зовнішнього ключа (рис. 4.15).

У результуючій таблиці не знайшлося місця для кафедри хімії. Це особливість природного злиття, в таблиці TEACHERS немає жодного запису із зовнішнім ключем, що посилаються на кафедру хімії (значення 4), тому кафедра хімії не потрапила в підсумкову таблицю. Якщо ж нам потрібно побачити і "кинуті" записи, то замість природного злиття необхідно скористатися зовнішнім злиттям. Розрізняють праве і ліве зовнішні з'єднання.

Таблица TEACHERS			Таблица CHAIR			Результат природного злиття																																									
<table><tr><th>SURNAME</th><th>CHAIR_KEY</th></tr><tr><td>Орлов</td><td>1</td></tr><tr><td>Володина</td><td>1</td></tr><tr><td>Шуверов</td><td>3</td></tr><tr><td>Калужный</td><td>2</td></tr><tr><td>Аскеров</td><td>2</td></tr></table>	SURNAME	CHAIR_KEY	Орлов	1	Володина	1	Шуверов	3	Калужный	2	Аскеров	2		▷◁	<table><tr><th>CHAIR_KEY</th><th>CHAIR</th></tr><tr><td>1</td><td>Высшая математика</td></tr><tr><td>2</td><td>Физика</td></tr><tr><td>3</td><td>Информатика</td></tr><tr><td>4</td><td>Химия</td></tr></table>	CHAIR_KEY	CHAIR	1	Высшая математика	2	Физика	3	Информатика	4	Химия	=	<table><tr><th>SURNAME</th><th>CHAIR_KEY</th><th>CHAIR</th></tr><tr><td>Орлов</td><td>1</td><td>Высшая математика</td></tr><tr><td>Володина</td><td>1</td><td>Высшая математика</td></tr><tr><td>Шуверов</td><td>3</td><td>Информатика</td></tr><tr><td>Калужный</td><td>2</td><td>Физика</td></tr><tr><td>Аскеров</td><td>2</td><td>Физика</td></tr></table>	SURNAME	CHAIR_KEY	CHAIR	Орлов	1	Высшая математика	Володина	1	Высшая математика	Шуверов	3	Информатика	Калужный	2	Физика	Аскеров	2	Физика		
SURNAME	CHAIR_KEY																																														
Орлов	1																																														
Володина	1																																														
Шуверов	3																																														
Калужный	2																																														
Аскеров	2																																														
CHAIR_KEY	CHAIR																																														
1	Высшая математика																																														
2	Физика																																														
3	Информатика																																														
4	Химия																																														
SURNAME	CHAIR_KEY	CHAIR																																													
Орлов	1	Высшая математика																																													
Володина	1	Высшая математика																																													
Шуверов	3	Информатика																																													
Калужный	2	Физика																																													
Аскеров	2	Физика																																													

Рис. 4.15. Операція природного злиття

На рис. 4.16 представлено праве зовнішнє з'єднання, при якому кортежі відношення CHAIR (таблиця розташована праворуч), що не мають співпадаючих значень в загальних шпальтах відношення TEACHERS, також включаються в результуюче відношення.

Таблиця TEACHERS			Таблиця CHAIR			Результат правого зовнішнього з'єднання				
SURNAME	CHAIR_KEY	▷C	CHAIR_KEY	CHAIR	=	SURNAME	CHAIR_KEY	CHAIR		
Орлов	1		1	Высшая математика		Орлов	1	Высшая математика		
Володина	1		2	Физика		Володина	1	Высшая математика		
Шуверов	3		3	Информатика		Калужный	2	Физика		
Калужный	2		4	Химия		Аскеров	2	Физика		
Аскеров	2					Шуверов	3	Информатика		
								NULL	4	Химия

Рис. 4.16. Демонстрація правого зовнішнього з'єднання

В одному з рядків результуючої таблиці в полі SURNAME з'явився визначник NULL. Це сталося через те, що в таблиці педагогів не зберігається інформація про співробітників кафедри хімії. Якщо згадати введені нами наприкінці попередньої лекції визначення зовнішнього ключа відношення, то має стати зрозуміло, що основний зміст операції природного з'єднання складається в можливості відновлення складної сутності, виконати декомпозицію через вимоги першої нормальної форми. Операція природного з'єднання не включається до складу набору операцій даної реляційної алгебри Кодда, але має дуже важливе практичне значення.

4.5. Приклади використання реляційних операторів

Приклад 4.1. Отримати прізвища студентів, які здали дисципліну з кодом 3 [11].

Рішення.

((R3 JOIN R1) Код дисципліни = 3) [Прізвище студента]

Приклад 4.2. Отримати прізвища студентів, які здали математику.

Рішення.

((R2 WHERE Назва дисципліни = Математика) JOIN R3) JOIN R1 [Прізвище студента]

Приклад 4.3. Отримати прізвища студентів, які здали всі дисципліни.

Рішення.

((R3 [Особистий номер, Код дисципліни] DIVIDEBY R2 [Код дисципліни]) join R1) [Прізвище студента]

Приклад 4.4. Отримати прізвища студентів, що не здали дисципліну з кодом 3.

Рішення.

$((R1 \text{ [Особистий номер] MINUS } \{(R1 \text{ join } R3) \text{ WHERE Код дисципліни} = 3\} \text{ [Особистий номер]})$
 $\text{JOIN } R1) \text{ [Прізвище студента]}$

Рішення даного завдання можна отримати також і в такий спосіб:

- 1) $S1 = R1 \text{ [Особистий номер]}$ – отримання списку ідентифікаторів всіх студентів;
- 2) $S2 = R1 \text{ JOIN } R3$ – з'єднання даних про студентів та оцінках;
- 3) $S3 = S2 \text{ WHERE Код дисципліни} = 3$ – в даних про студентів та оцінках залишається тільки інформація про здачу дисципліни з кодом 3;
- 4) $S4 = S3 \text{ [Особистий номер]}$ – отримання списку ідентифікаторів студентів, які здали дисципліну з кодом 3;
- 5) $S5 = S1 \text{ MINUS } S4$ – отримання списку ідентифікаторів студентів, що не здали дисципліну з кодом 3;
- 6) $S6 = S5 \text{ JOIN } R1$ – з'єднання списку ідентифікаторів студентів, що не здали дисципліну з кодом 3 з даними про студентів;
- 7) $S7 = S6 \text{ [Прізвище студента]}$ – прізвища студентів, що не здали дисципліну з кодом 3.

На завершення лекції відзначимо кілька моментів. Перш за все, зауважимо, що алгебра Кодда була представлена не в її оригінальній формі, а з деякими істотними корективами, внесеними Крістофером Дейтом. Однією з найбільш значних коректив було додавання тривіальної на перший погляд операції перейменування атрибутів. Коли Едгар Кодд в кінці 1960-х рр. вперше опублікував свою алгебру, основна увага в ній приділялася тому, як конструюються результуючі множини кортежів, тобто, що представляють собою тіла результатів операцій. Набагато менше уваги приділялося заголовкам відношень-результатів. Фактично Кодд намагався застосувати для іменування атрибутів результатів операцій точкову нотацію, використовуючи для уточнення імен атрибутів імена вихідних відношень-операндів. При наявності довільно складних і довгих виразів алгебри цей шлях, в кращому випадку, вів до породження довгих і важких для сприйняття імен. Очевидно, що введення операції перейменування атрибутів дозволяє легко впоратися з цією проблемою.

5. ПРОЕКТУВАННЯ РЕЛЯЦІЙНИХ БАЗ ДАНИХ НА ОСНОВІ НОРМАЛІЗАЦІЇ

5.1. Вступ

Ця лекція відкриває серію з трьох лекцій, присвячених проектуванню реляційних баз даних. У даній лекції йтиметься про нормалізацію схем відношень з урахуванням тільки функціональних залежностей між атрибутами відношень. Ці «перші кроки» нормалізації дозволяють отримати схему бази даних, в яких всі змінні відношень знаходяться в третій нормальній формі, зазвичай розцінює задовільною для більшої частини додатків.

При проектуванні бази даних вирішуються дві основні проблеми.

1. Яким чином відобразити об'єкти предметної області в абстрактні об'єкти моделі даних, щоб це відображення не суперечило семантиці предметної області і було, по можливості, найкращим (ефективним, зручним тощо)? Часто цю проблему називають проблемою логічного проектування баз даних.
2. Як забезпечити ефективність виконання запитів до бази даних, тобто, яким чином, маючи на увазі особливості конкретної СКБД, розташувати дані у зовнішній пам'яті, створення яких додаткових структур (наприклад, індексів) вимагати і т. д.? Цю проблему зазвичай називають проблемою фізичного проектування баз даних.

У цій та наступній лекціях буде розглянуто класичний підхід, при якому весь процес проектування бази даних здійснюється в термінах реляційної моделі даних методом послідовних наближень до задовільного набору схем відношень. Вихідною точкою є представлення предметної області у вигляді одного або декількох відношень, і на кожному етапі проектування проводиться деякий набір схем відношень, що володіють «поліпшеними» властивостями.

Процес проектування являє собою процес нормалізації (декомпозиції) схем відношень, причому кожна наступна нормальна форма має властивості, в деякому сенсі, кращими, ніж попередня.

Нормалізація – це покроковий, оборотний процес заміни вихідної схеми іншою схемою, в якій таблиці мають більш просту і логічну структуру, що володіє кращими властивостями при включенні, зміні і видаленні даних. Для чого це потрібно?

По-перше, для усунення надмірності даних.

По-друге, при роботі з такими таблицями можуть виникнути так звані аномалії оновлення. Наприклад, якщо ми вилучимо з цієї таблиці четверте повідомлення, то разом з ним зникне і інформація про тему. Така ситуація є аномалією видалення. Якщо ми вирішимо змінити назву теми, то нам доведеться переглянути

Кожній нормальній формі відповідає певний набір обмежень, і відношення знаходиться в деякій нормальній формі, якщо задовольняє властивого їй набору обмежень. Прикладом може служити обмеження першої нормальної форми – значення всіх атрибутів відношення атомарні. Оскільки вимога першої нормальної форми є базовою вимогою класичної реляційної моделі даних, ми будемо вважати, що вихідний набір відношення вже відповідає цій вимозі.

В теорії реляційних баз даних звичайно виділяється наступна послідовність нормальних форм:

- перша нормальна форма (1НФ);
- друга нормальна форма (2НФ);
- третя нормальна форма (3НФ);
- нормальна форма Бойса-Кодда (БКНФ);
- четверта нормальна форма (4НФ);
- п'ята нормальна форма, або нормальна форма проєкції-з'єднання (5НФ або PJ/NF).

Основні властивості нормальних форм полягають у наступному:

- кожна наступна нормальна форма в деякому сенсі краща за попередню нормальну форму;
- при переході до наступної нормальної форми властивості попередніх нормальних форм зберігаються.

Для реляційних баз даних необхідно, щоб її таблиці знаходилися в 1НФ. Нормальні форми більш високих рівнів можуть використовуватися розробниками на свій розсуд. Однак грамотний фахівець буде прагнути до того, щоб довести рівень нормалізації бази даних хоча б до 3НФ, тим самим виключивши надмірність даних і аномалії оновлення. Треба сказати, що НФБК, 4НФ і 5НФ використовуються на практиці вкрай нечасто.

У цій лекції ми обговоримо перші кроки процесу нормалізації, в яких враховуються функціональні залежності між атрибутами відношень. Хоча ми і називаємо ці кроки першими, саме вони мають основну практичну важливість, оскільки дозволяють отримати схему реляційної бази даних, в більшості випадків задовольняє потреби додатків.

5.2. Етапи розробки бази даних

Метою розробки будь-якої бази даних є зберігання та використання інформації про будь-якої предметної області. Для реалізації цієї мети є наступні інструменти:

1. Реляційна модель даних – зручний спосіб представлення даних предметної області.
2. Мова SQL – універсальний спосіб маніпулювання такими даними.

Однак очевидно, що для однієї і тієї ж предметної області реляційні відношення можна спроектувати безліччю різних способів. Наприклад, можна спроектувати кілька відношень з великою кількістю атрибутів, або навпаки, рознести все атрибути по великому числу дрібних відношень. Як визначити, за якими ознаками потрібно поміщати атрибути в ті чи інші відношення?

У даній лекції розглядаються способи «хорошого» або «правильного» проектування реляційних відношень. Спочатку ми обговоримо, що означає «хороші» або «правильні» моделі даних. Потім будуть введені поняття першої, другої і третьої нормальних форм відношень (1НФ, 2НФ, 3НФ) і показано, що «хорошими» є відношення в третій нормальній формі.

При розробці бази даних зазвичай виділяється кілька рівнів моделювання, за допомогою яких відбувається перехід від предметної області до конкретної реалізації бази даних засобами конкретної СКБД. Можна виділити такі рівні:

1. Сама предметна область.
2. Концептуальна модель предметної області.
3. Логічна модель даних.
4. Фізична модель даних.
5. Власне, база даних і додатки.

Основні етапи (рівні) проектування можна відобразити за допомогою схеми, представленої на рис. 5.1.

На етапі формулювання й аналізу вимог (аналіз предметної області) встановлюються цілі організації, визначаються вимоги до БД. Ці вимоги документуються у формі доступній кінцевому користувачеві і проектувальнику БД. Зазвичай при цьому використовується методика інтерв'ювання персоналу різних рівнів управління. На цьому етапі розробник (адміністратор бази даних) за результатами проведення досліджень предметної області, а також об'єднуючи приватні уявлення про вміст бази даних, отримані в результаті опитування користувачів, і свої власні уявлення про дані, які можуть знадобитися в майбутніх додатках, створює узагальнене неформальний опис бази даних.

Етап концептуального (інфологічного) проектування полягає в описі і синтезі інформаційних вимог користувачів в початковий проект БД. Результатом цього етапу є високорівневе представлення інформаційних вимог користувачів на основі різних підходів. Такий опис предметної області називається концептуальною (інфологічною) моделлю даних.

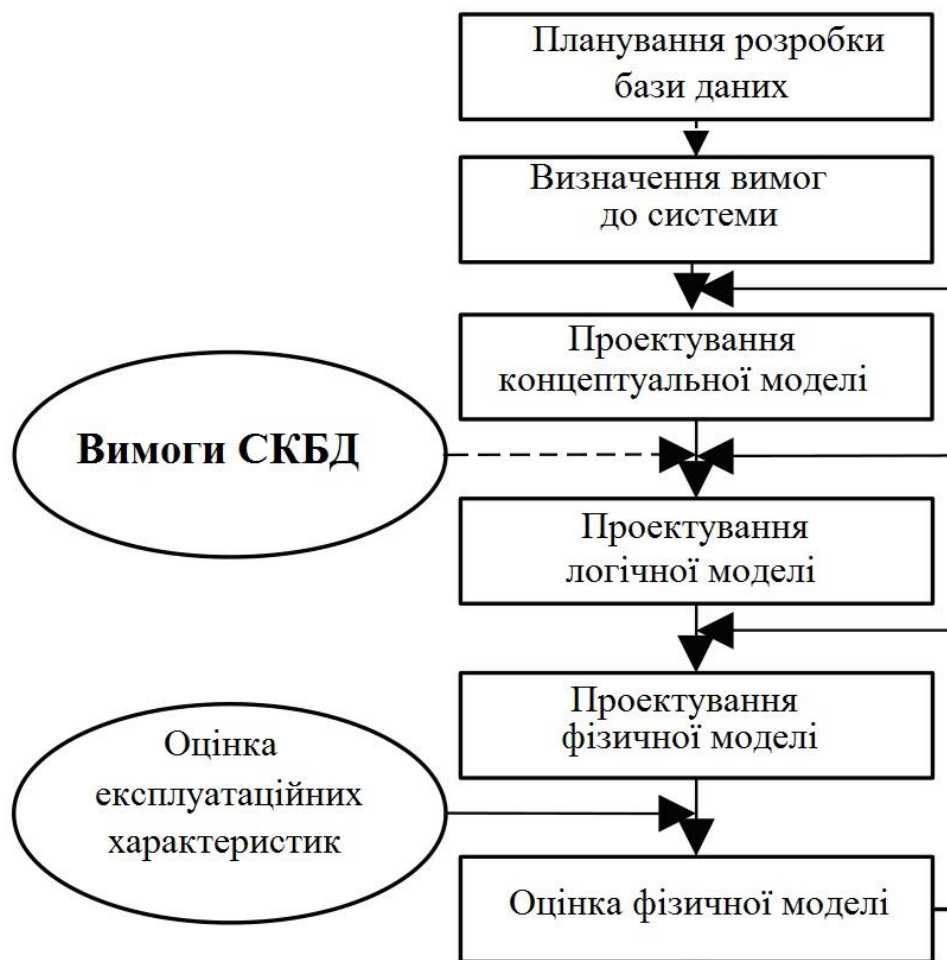


Рис. 5.1. Схема проектування БД

Етап побудови логічної (датологічної) моделі починається з вибору моделі даних. При виборі моделі даних важливу роль відіграє її простота, наочність і порівняння природної структури даних з моделлю, що її представляє. Але найчастіше цей вибір визначається успіхом (або наявністю) тієї чи іншої СКБД. Тобто, розробник вибирає СКБД, а не модель даних.

Версія концептуальної моделі, яка може бути забезпечена конкретної СКБД, називається логічною моделлю. Іноді процес визначення концептуальної і логічної моделей називається визначенням структури даних.

На етапі фізичного проектування вирішуються питання, пов'язані з продуктивністю системи, визначаються структури зберігання даних і методи доступу, а також техніка індексування. Тобто, фізична модель є комп'ютерно-орієнтованою. На даному етапі ми прив'язуємося до конкретної СКБД і розписуємо схему даних більш детально, із зазначенням типів, розмірів полів і обмежень. Крім розробки таблиць і індексів, на цьому етапі також проводиться визначення основних запитів.

За допомогою СКБД надається можливість програмам і користувачам здійснювати доступ до збережених даних лише за їхніми іменами, не піклуючись про фізичне розташування цих даних. Потрібні дані відшукуються СКБД на зовнішніх запам'ятовуючих пристроях з фізичної моделі даних.

На етапі оцінки фізичної моделі проводиться оцінка експлуатаційних характеристик. Тут можна перевірити ефективність виконання запитів, швидкість пошуку, правильність і зручність виконання операцій з БД, цілісність даних і ефективність витрати ресурсів комп'ютера. При незадовільних експлуатаційних характеристиках можливе повернення до перегляду фізичної та логічної моделей даних, вибору СКБД і типу комп'ютера.

Розглянемо кожен з етапів докладніше.

5.2.1. Концептуальна модель предметної області

Предметна область – це частина реального світу, дані про яку ми хочемо відобразити в базі даних. Наприклад, в якості предметної області можна вибрати бухгалтерію будь-якого підприємства, відділ кадрів, банк, магазин тощо. Предметна область нескінченна і містить як суттєво важливі поняття і дані, так і малозначні або взагалі не значущі дані. Так, якщо в якості предметної області вибрати «облік товарів на складі», то поняття «накладна» і «рахунок-фактура» є суттєво важливими поняттями. А то, що співробітниця, приймаюча накладні, має двох дітей – це для обліку товарів неважливо. Однак, з точки зору відділу кадрів, дані про наявність дітей є суттєво важливими. Таким чином, важливість даних залежить від вибору предметної області.

Аналіз предметної області (Системний аналіз) передбачає складання опису предметної області, яке має на увазі формулювання і аналіз вимог, що пред'являються до змісту і процесу обробки даних усіма відомими і потенційними користувачами БД. На етапі системного аналізу необхідно провести докладний словесний опис інформаційних об'єктів предметної області і реальних зв'язків, які присутні між описаними об'єктами.

Предметна область це множина фрагментів, які характеризуються великою кількістю об'єктів, безліччю процесів, які використовують об'єкти, а також безліччю користувачів, які характеризуються єдиним поглядом на предметну область.

Об'єктом називається явище зовнішнього світу. Це або щось реально існуюче – людина, товар, виріб, або процес – облік народжуваності, отримання товарів, випуск виробів. Кожен об'єкт володіє величезною кількістю властивостей.

Наприклад, об'єкт «Людина» має властивості: зростання, ім'я, дата народження тощо, об'єкт – «Виріб» має властивості: якість, дата виготовлення, зовнішній вигляд тощо.

Між об'єктами існують численні зв'язки. Наприклад, Людина купує, продає, виробляє Виріб, а Виріб створюється, купується, продається Людиною.

Модель предметної області (Концептуальна модель). Модель предметної області – це наші знання про предметну область. Знання можуть бути як у вигляді неформальних знань в мозку експерта, так і виражені формально за допомогою будь-яких засобів. В якості таких засобів можуть виступати текстові описи предметної області, набори посадових інструкцій, правила ведення справ в компанії і т.п.

Концептуальна модель моделює опис основних сутностей (таблиць) і зв'язків між ними без урахування прийнятої моделі БД, синтаксису цільової СКБД і апаратної платформи. Часто на такій моделі будуть відображатися імена сутностей (таблиць) без вказівки їх атрибутів. Подання користувача включає в себе дані, необхідні конкретному користувачеві для прийняття рішень або виконання деякого завдання.

Досвід показує, що текстовий спосіб представлення моделі предметної області вкрай неефективний. Набагато більш інформативними і корисними при розробці баз даних є описи предметної області, виконані за допомогою спеціалізованих графічних нотацій. Є велика кількість методик опису предметної області. З найбільш відомих можна назвати методику структурного аналізу SADT і засновану на ньому IDEF0, діаграми потоків даних Гейне-Сарсона, методику об'єктно-орієнтованого аналізу UML, і ін. Модель предметної області описує швидше процеси, що відбуваються в предметній області і дані, використовувані цими процесами. Від того, наскільки правильно змодельована предметна область, залежить успіх подальшої розробки додатків.

В даний час для проектування БД активно використовуються CASE-засоби, в основному орієнтовані на використання ERD (Entity – Relationship Diagrams, діаграми «сутність-зв'язок»). З їх допомогою визначаються важливі для предметної області об'єкти (сутності), відносини один з одним (зв'язки) і їх властивості (атрибути). Слід зазначити, що кошти проектування ERD в основному орієнтовані на реляційні бази даних (РБД). ERD були вперше запропоновані П. Ченом в 1976 р. Основні елементи ERD перераховані нижче. Надалі багатьма авторами були розроблені свої варіанти подібних моделей: нотація (notation – система позначення, записи) Мартіна, нотація IDEF1X, нотація Баркера, але всі вони базуються на графічних діаграмах, запропонованих Ченом.

Сутність (таблиця, в РБД – відношення) реальний або уявний об'єкт, що має істотне значення для розглянутої предметної області, інформація про який підлягає зберіганню. Якщо

висловлюватися точніше, то це не об'єкт, а набір об'єктів (клас) з однаковими властивостями. Приклади сутностей: працівник, деталь, відомість, результати задачі іспиту і т. д. Примірник сутності (запис, рядок, в РБД – кортеж) – унікально ідентифікований об'єкт.

Зв'язок – деяка асоціація між двома сутностями, значима для розглянутої предметної області. Прикладами зв'язків можуть бути родинні стосунки «батько-син», виробничі – «начальник-підлеглий» або довільні – «мати у власності», «мати властивість».

Атрибут (стовпець, поле) – властивість сутності або зв'язку.

Більшість сучасних CASE-засобів моделювання даних, як правило, підтримує кілька графічних нотацій побудови інформаційних моделей. Зокрема, система ERwin фірми Computer Associates підтримує дві нотації: IDEF1X і IE (Information Engineering – інформаційне проектування). Дані нотації є взаємно-однозначними, тобто, перехід від однієї нотації до іншої і назад виконується без втрати якості моделі. Відмінність між ними полягає лише в формі відображення елементів моделі. При використанні будь-якого CASE-засобу спочатку будується інфологічна модель БД у вигляді діаграми із зазначенням сутностей і зв'язків між ними.

5.2.2. Логічна модель даних

На наступному, більш низькому рівні знаходиться логічна модель даних предметної області. Логічна модель описує поняття предметної області, їх взаємозв'язок, а також обмеження на дані, що накладаються предметною областю. Приклади понять – «співробітник», «відділ», «проект», «зарплата». Приклади взаємозв'язків між поняттями – «співробітник числиться рівно в одному відділі», «співробітник може виконувати кілька проектів», «над одним проектом може працювати кілька співробітників». Приклади обмежень – «вік співробітника не менше 16 і не більше 60 років».

Логічна модель даних є початковим прототипом майбутньої бази даних. Логічна модель будується в термінах інформаційних одиниць, але зазвичай без прив'язки до конкретної СКБД і інших фізичних умов реалізації. Але найчастіше цей вибір визначається наявністю тієї чи іншої СКБД. Тобто, розробник вибирає СКБД, а не модель даних. На підставі отриманої логічної моделі переходять до фізичної моделі даних.

Більш того, логічна модель даних необов'язково повинна бути виражена засобами саме реляційної моделі даних. Основним засобом розробки логічної моделі даних в даний момент є різні варіанти ER-діаграм (Entity-Relationship, діаграми сутність-зв'язок). Одну і ту ж ER-модель можна перетворити як в реляційну модель даних, так і в модель даних для ієрархічних і мережевих СКБД, або в постреляційних моделях даних.

Рішення, прийняті на попередньому рівні, при розробці моделі предметної області, визначають деякі межі, в яких можна розвивати логічну модель даних. Наприклад, модель предметної області складського обліку містить поняття «склад», «накладна», «товар». При розробці відповідної реляційної моделі ці терміни обов'язково повинні бути використані, але різних способів реалізації тут багато – можна створити одне відношення, в якому будуть присутні в якості атрибутів «склад», «накладна», «товар», а можна створити три окремих відношення, по одному на кожне поняття.

Якщо в процесі логічного проектування обрана конкретна СКБД, то високорівневе представлення даних перетвориться в структури конкретної моделі даних (в нашому випадку – реляційної) або використовуваної СКБД. Отримана логічна структура БД може бути оцінена кількісно за допомогою різних характеристик (число звернень до логічних записів, обсяг даних в кожному додатку, загальний обсяг даних і т.д.). На основі цих оцінок логічна структура може бути вдосконалена з метою досягнення більшої ефективності. Логічна модель даних є початковим прототипом майбутньої бази даних. Логічна модель будується в термінах інформаційних одиниць обраної моделі БД (ієрархічна, реляційна, мережева), але, в такому випадку, без прив'язки до конкретної СКБД.

При розробці логічної моделі даних виникають питання: чи добре спроектовані відношення? Чи правильно вони відображають модель предметної області, а, отже, і саму предметну область?

5.2.3. Фізична модель даних

На ще більш низькому рівні знаходиться фізична модель даних. Фізична модель даних описує дані засобами конкретної СКБД. Ми будемо вважати, що фізична модель даних реалізована засобами саме реляційної СКБД, хоча, як вже сказано вище, це необов'язково. Відношення, розроблені на стадії формування логічної моделі даних, перетворюються в таблиці, атрибути стають стовпцями таблиць, для ключових атрибутів створюються унікальні індекси, домени перетворюються в типи даних, прийняті в конкретній СКБД.

Обмеження, наявні в логічній моделі даних, реалізуються різними засобами СКБД, наприклад, за допомогою індексів, декларативних обмежень цілісності, тригерів, збережених процедур. При цьому знову-таки рішення, прийняті на рівні логічного моделювання, визначають деякі межі, в межах яких можна розвивати фізичну модель даних. Точно також, в межах цих кордонів можна приймати різні рішення. Наприклад, відношення, що містяться в логічній моделі даних, повинні бути перетворені в таблиці, але для кожної таблиці можна додатково оголосити різні індекси, що підвищують швидкість доступу до даних. Багато що тут залежить від конкретної СКБД.

За допомогою СКБД нада можливість програмам і користувачам здійснювати доступ до збережених даних лише за їхніми іменами, не піклуючись про фізичне розташування цих даних. Потрібні дані відшукуються СКБД на зовнішніх запам'ятовуючих пристроях з фізичної моделі даних.

При розробці фізичної моделі даних виникають питання: чи добре спроектовані таблиці? Чи правильно вибрані індекси? Наскільки багато програмного коду у вигляді тригерів і збережених процедур необхідно розробити для підтримки цілісності даних?

І, нарешті, як результат попередніх етапів з'являється власне сама база даних. База даних реалізована у конкретній програмно-апаратній основі, і вибір цієї основи дозволяє істотно підвищити швидкість роботи з базою даних. Наприклад, можна вибирати різні типи комп'ютерів, змінювати кількість процесорів, обсяг оперативної пам'яті, дискові підсистеми тощо. Дуже велике значення має також налаштування СКБД в межах обраної програмно-апаратної платформи.

Але знову рішення, прийняті на попередньому рівні – рівні фізичного проектування, визначають межі, в межах яких можна приймати рішення щодо вибору програмно-апаратної платформи і налаштування СКБД.

Таким чином ясно, що рішення, прийняті на кожному етапі моделювання і розробки бази даних, будуть позначатися на подальших етапах. Тому особливу роль відіграє прийняття правильних рішень на ранніх етапах моделювання.

5.3. Критерії оцінки якості логічної моделі даних

Наведемо деякі принципи побудови хороших логічних моделей даних. Хороших в тому сенсі, що рішення, прийняті в процесі логічного проектування, приводили б до хорошим фізичним моделям і в кінцевому підсумку до хорошої роботи бази даних.

Для того щоб оцінити якість прийнятих рішень на рівні логічної моделі даних, необхідно сформулювати деякі критерії якості в термінах фізичної моделі і конкретної реалізації і подивитися, як різні рішення, прийняті в процесі логічного моделювання, впливають на якість фізичної моделі і на швидкість роботи бази даних.

Звичайно, таких критеріїв може бути дуже багато і вибір їх в достатній мірі довільний. Ми розглянемо деякі з таких критеріїв, які є безумовно важливими з точки зору отримання якісної бази даних:

1. Адекватність бази даних предметної області.
2. Легкість розробки і супроводу бази даних.
3. Швидкість виконання операцій оновлення даних (вставка, оновлення, видалення кортежів).
4. Швидкість виконання операцій вибірки даних.

Адекватність бази даних предметної області. База даних повинна адекватно відображати предметну область. Це означає, що повинні бути виконані такі умови:

1. Стан бази даних в кожен момент часу має відповідати стану предметної області.
2. Зміна стану предметної області має приводити до відповідної зміни стану бази даних
3. Обмеження предметної області, відображені в моделі предметної області, повинні деяким чином відбиватися і враховуватися бази даних.

Легкість розробки і супроводу бази даних. Практично будь-яка база даних, за винятком зовсім елементарних, містить певну кількість програмного коду у вигляді збережених процедур і тригерів.

Збережені процедури – це процедури і функції, що зберігаються безпосередньо в базі даних в відкомпілюваному вигляді і які можуть запускатися користувачами або додатками, що працюють з базою даних. Збережені процедури зазвичай пишуться або на спеціальній процедурній розширенні мови SQL (наприклад, PL/SQL для ORACLE або Transact-SQL для MS SQL Server), або на деякому універсальній мові програмування, наприклад, C++, з включенням в код операторів SQL у відповідності зі спеціальними правилами такого включення. Основне призначення процедур – реалізація бізнес-процесів предметної області.

Тригери – це збережені процедури, пов'язані з деякими подіями, що відбуваються під час роботи бази даних. В якості таких подій виступають операції вставки, оновлення та видалення рядків таблиць. Якщо в базі даних визначено певний тригер, то він запускається автоматично завжди при виникненні події, з яким цей тригер пов'язаний.

Дуже важливим є те, що користувач не може обійти тригер. Тригер спрацьовує незалежно від того, хто з користувачів і яким способом ініціював подію, яка викликала запуск тригера. Таким чином, основне призначення тригерів – автоматична підтримка цілісності бази даних.

Тригери можуть бути як досить простими, наприклад, підтримують кількість посилань цілісність, так і досить складними, що реалізують будь-які складні обмеження предметної області або складні дії, які повинні відбутися при настанні деяких подій. Наприклад, з операцією вставки нового товару в накладну може бути пов'язаний тригер, який виконує наступні дії: перевіряє, чи є необхідна кількість товару, при наявності товару додає його в накладну і зменшує дані про наявність товару на складі, при відсутності товару формує замовлення на поставку відсутнього товару і тут же посилає замовлення по електронній пошті постачальнику.

Очевидно, що чим більше програмного коду у вигляді тригерів і збережених процедур містить база даних, тим складніше її розробка і подальший супровід.

Швидкість операцій оновлення даних (Вставка, оновлення, видалення). На рівні логічного моделювання ми визначаємо реляційні відношення і атрибути цих відношень. На цьому рівні ми не можемо визначати будь-які фізичні структури зберігання (індекси, хешування і т.п.). Єдине, чим ми можемо управляти – це розподілом атрибутів з різних відношень. Можна описати мало відношень з великою кількістю атрибутів, або багато відношень, кожне з яких містить мало атрибутів. Таким чином, необхідно спробувати відповісти на питання – чи впливає кількість відношень і кількість атрибутів у відношеннях на швидкість виконання операцій оновлення даних. Таке питання, звичайно, не є достатньо коректним, тому що швидкість виконання операцій з базою даних сильно залежить від фізичної реалізації бази даних. Проте, основними операціями, що змінюють стан бази даних, є операції вставки, оновлення та видалення записів. У базах даних, які потребують постійних змін (складський облік, системи продажів квитків тощо) продуктивність визначається швидкістю виконання великої кількості невеликих операцій вставки, оновлення та видалення.

Розглянемо операцію вставки запису в таблицю. Вставка запису провадиться в одну з вільних сторінок пам'яті, виділеної для даної таблиці. СКБД постійно зберігає інформацію про наявність і розташування вільних сторінок. Якщо для таблиці не створені індекси, то операція вставки виконується фактично з однаковою швидкістю незалежно від розміру таблиці і від кількості атрибутів в таблиці. Якщо в таблиці є індекси, то при виконанні операції вставки запису індекси повинні бути перебудовані. Таким чином, швидкість виконання операції вставки зменшується при збільшенні кількості індексів у таблиці і мало залежить від числа рядків в таблиці.

Розглянемо операції оновлення та видалення записів з таблиці. Перш, ніж оновити або видалити запис, її необхідно знайти. Якщо таблиці не індексована, то єдиним способом пошуку є

послідовне сканування таблиці в пошуку потрібного запису. В цьому випадку, швидкість операцій оновлення і видалення істотно збільшується зі збільшенням кількості записів в таблиці і не залежить від кількості атрибутів. Але насправді неіндексовані таблиці практично ніколи не використовуються.

Для кожної таблиці зазвичай оголошується один або кілька індексів, відповідний потенційним джерелам. За допомогою цих індексів пошук запису провадиться дуже швидко і практично не залежить від кількості рядків і атрибутів в таблиці (хоча, звичайно, деяка залежність є). Якщо для таблиці оголошено кілька індексів, то при виконанні операцій оновлення і видалення ці індекси повинні бути перебудовані, на що витрачається додатковий час. Таким чином, швидкість виконання операцій оновлення і видалення також зменшується при збільшенні кількості індексів у таблиці і мало залежить від числа рядків в таблиці.

Можна припустити, що чим більше атрибутів має таблиця, тим більше для неї буде оголошено індексів. Ця залежність, звичайно, не пряма, але при однакових підходах до фізичного моделювання зазвичай так і відбувається. Таким чином, можна прийняти допущення, що чим більше атрибутів мають відношення, розроблені в ході логічного моделювання, тим повільніше будуть виконуватися операції оновлення даних, за рахунок витрати часу на перебудову більшої кількості індексів.

Швидкість операцій вибірки даних. Одне з призначень бази даних – надання інформації користувачам. Інформація витягується з реляційної бази даних за допомогою оператора SQL – SELECT. Однією з найбільш дорогих операцій при виконанні оператора SELECT є операція з'єднання таблиць.

Таким чином, чим більше взаємопов'язаних відношень було створено в ході логічного моделювання, тим більша ймовірність того, що при виконанні запитів ці відношення будуть з'єднуватися, і, отже, тим повільніше будуть виконуватися запити. Таким чином, збільшення кількості відношень призводить до уповільнення виконання операцій вибірки даних, особливо, якщо запити заздалегідь невідомі.

Весь процес проектування БД характеризується як ітеративний, при цьому кожен етап розглядається як сукупність ітеративних процедур, в результаті виконання яких отримують відповідну модель.

Взаємодія між етапами проектування та словникової системою необхідно розглядати окремо. Процедури проектування можуть використовуватися незалежно в разі відсутності словникової системи. Сама словникова система може розглядатися як елемент автоматизації проектування.

Адміністратор БД може підключити до системи будь-яке число нових користувачів (нових додатків), доповнивши, якщо треба, логічну модель. Зазначені зміни фізичної та логічної моделей не будуть помічені існуючими користувачами системи (виявляться «прозорими» для них), так само як не будуть помічені і нові користувачі.

Проектування БД можна проводити за допомогою автоматизованих систем розробки додатків, так званих CASE-систем (Computer Aided Software Engineering). Автоматизовані системи розробки додатків є програмні засоби, що підтримують процеси створення і супроводу інформаційних систем, такі як аналіз і формулювання вимог, проектування додатків, генерація коду, тестування, управління конфігурацією і проектом. Основна мета CASE систем полягає в тому, щоб відокремити процес проектування програмного забезпечення від його кодування і наступних етапів розробки (тестування, документування тощо), а також автоматизувати весь процес створення програмних систем.

Процес розробки баз даних за допомогою CASE систем на етапі концептуального проектування зазвичай проводиться за допомогою ER-моделі. Результатом проектування практично завжди (або до недавнього часу) була реляційна модель даних. Прикладами CASE систем можуть служити: ERWin (Logic Works), Rational Rose (Rational Software) (OODBMS), S-Designer (SPD), DataBase Designer, Developer / Designer 2000 (Oracle Corp.), MS Visual Studio.

5.4. Перша Нормальна Форма

5.4.1. Перша Нормальна Форма (1НФ)

Поняття першої нормальної форми вже обговорювалося в лекції 2. Перша нормальна форма (1НФ) – це звичайне відношення. Відповідно до нашого визначення відношень, будь-яке відношення автоматично вже знаходиться в 1НФ. Нагадаємо коротко властивості відношень (це і будуть властивості 1НФ):

1. У відношенні немає однакових кортежів.
2. Кортежі не впорядковані.
3. Атрибути не впорядковані і розрізняються по найменуванню.
4. Всі значення атрибутів атомарні.

В ході логічного моделювання на першому кроці запропоновано зберігати дані в одному відношенні, що має, наприклад, такі атрибути:

СОТРУДНИКИ_ОТДЕЛИ_ПРОЕКТИ (*Н_СОТР*, *ФАМ*, *Н_ОТД*, *ТЕЛ*, *Н_ПРО*, *ПРОЕКТ*, *Н_ЗАДАН*), де:

Н_СОТР – табельний номер співробітника

ФАМ – прізвище співробітника

Н_ОТД – номер відділу, в якому значиться співробітник

ТЕЛ – телефон співробітника

Н_ПРО – номер проекту, над яким працює співробітник

ПРОЕКТ – найменування проекту, над яким працює співробітник

Н_ЗАДАН – номер завдання, над яким працює співробітник

Так як кожен співробітник в кожному проекті виконує рівно одне завдання, то в якості потенційного ключа відношення необхідно взяти пару атрибутів {*Н_СОТР*, *Н_ПРО*}.

У поточний момент стан предметної області відображається такими фактами:

1. Співробітник Іванов, який працює в 1 відділі, виконує в першому проекті «Вайбер» завдання 1 і в другому проекті «Скайп» завдання 1.
2. Співробітник Петров, який працює в 1 відділі, виконує в першому проекті «Вайбер» завдання 2.
3. Співробітник Сидоров, який працює у 2 відділі, виконує в першому проекті «Вайбер» завдання 3 і в другому проекті «Скайп» завдання 2.

Цей стан відбивається в таблиці 5.1 (курсивом виділено ключові атрибути).

Таблиця 5.1. Відношення **СОТРУДНИКИ_ОТДЕЛИ_ПРОЕКТИ**

<i>Н_СОТР</i>	<i>ФАМ</i>	<i>Н_ОТД</i>	<i>ТЕЛ</i>	<i>Н_ПРО</i>	<i>ПРОЕКТ</i>	<i>Н_ЗАДАН</i>
1	Іванов	1	11-22-33	1	Вайбер	1
1	Іванов	1	11-22-33	2	Скайп	1
2	Петров	1	11-22-33	1	Вайбер	2
3	Сидоров	2	33-22-11	1	Вайбер	3
3	Сидоров	2	33-22-11	2	Скайп	2

5.4.2. Аномалії поновлення

Навіть одного погляду на таблицю відношення **СОТРУДНИКИ_ОТДЕЛИ_ПРОЕКТИ** досить, щоб побачити, що дані зберігаються в ній з великою надмірністю. У багатьох рядках повторюються прізвища співробітників, номери телефонів, найменування проектів. Крім того, в даному відношенні зберігаються разом незалежні один від одного дані: і дані про співробітників, і про відділи, і про проекти, і про роботи по проектам. Поки ніяких дій з відношенням не

проводиться, це не страшно. Але як тільки стан предметної області змінюється, то, при спробах відповідним чином змінити стан бази даних, виникає велика кількість проблем.

Історично ці проблеми отримали назву аномалії оновлення. Аномалії визначені як протиріччя між моделлю предметної області і фізичною моделлю даних, підтримуваних засобами конкретної СКБД.

Ми будемо дотримуватися інтуїтивного поняття аномалії як неадекватності моделі даних предметної області, (що говорить насправді про те, що логічна модель даних просто невірна!) Або як необхідність додаткових зусиль для реалізації всіх обмежень визначених у предметної області (додатковий програмний код у вигляді тригерів або збережених процедур).

Так як аномалії виявляють себе при виконанні операцій, що змінюють стан бази даних, то розрізняють наступні види аномалій:

- аномалії вставки (INSERT);
- аномалії оновлення (UPDATE);
- аномалії видалення (DELETE).

Відносно СОТРУДНИКИ_ОТДЕЛИ_ПРОЕКТИ можна навести приклади таких аномалій:

Аномалії вставки (INSERT). У відношення СОТРУДНИКИ_ОТДЕЛИ_ПРОЕКТИ не можна вставити дані про співробітника, який поки не бере участь ні в одному проекті. Дійсно, якщо, наприклад, у другому відділі з'являється новий співробітник, скажімо, Хоменко, і він поки не бере участь ні в одному проекті, то ми повинні вставити у відношення кортеж (4, Хоменко, 2, 33-22-11, null, null, null). Це зробити неможливо, так як атрибут Н_ПРО (номер проекту) входить до складу потенційного ключа, і, отже, не може містити null-значень.

Точно також можна вставити дані про проект, над яким поки не працює жоден співробітник.

Причина аномалії – зберігання в одному відношенні різномірної інформації (і про співробітників, і про проекти, і про роботи по проекту).

Висновок – логічна модель даних неадекватна моделі предметної області. База даних, заснована на такій моделі, працюватиме неправильно.

Аномалії оновлення (UPDATE). Прізвища співробітників, найменування проектів, номери телефонів повторюються в багатьох кортежах відношення. Тому якщо співробітник змінює прізвище, або проект змінює найменування, або змінюється номер телефону, то такі зміни необхідно одночасно виконати у всіх місцях, де це прізвище, найменування або номер телефону зустрічаються, інакше відношення стане некоректним (наприклад, один і той же проект в різних кортежі буде називатися по-різному). Таким чином, оновлення бази даних одним дією реалізувати неможливо. Для підтримки відношення в цілісному стані необхідно написати тригер, який при оновленні запису коректно виправляв би дані і в інших місцях.

Причина аномалії – надмірність даних, також породжена тим, що в одному відношенні зберігається різномірна інформація.

Висновок – збільшується складність розробки бази даних. База даних, заснована на такій моделі, працюватиме правильно тільки при наявності додаткового програмного коду у вигляді тригерів.

Аномалії видалення (DELETE). При видаленні деяких даних може відбутися втрата іншої інформації. Наприклад, якщо закрити проект «Вайбер» і видалити всі рядки, в яких він зустрічається, то будуть втрачені всі дані про співробітника Петрові. Якщо видалити співробітника Сидорова, то буде втрачена інформація про те, що у відділі номер 2 знаходиться телефон 33-22-11. Якщо за проектом тимчасово припинені роботи, то при видаленні даних про роботи за цим проектом будуть видалені і дані про сам проект (найменування проекту). При цьому якщо був співробітник, який працював тільки над цим проектом, то будуть втрачені і дані про це працівника.

Причина аномалії – зберігання в одному відношенні різномірної інформації (і про співробітників, і про проекти, і про роботи по проекту).

Висновок – логічна модель даних неадекватна моделі предметної області. База даних, заснована на такій моделі, працюватиме неправильно.

5.5. Функціональні залежності

В основі подальшого процесу проектування лежить метод нормалізації, тобто, декомпозиції відношень, що знаходиться в попередній нормальній формі, на два або більше відношення, які задовольняють вимогам наступної нормальної форми. Ми обговоримо перші кроки процесу нормалізації, в яких враховуються функціональні залежності між атрибутами відношень. Хоча ми і називаємо ці кроки першими, саме вони мають основну практичну важливість, оскільки дозволяють отримати схему реляційної бази даних, в більшості випадків задовольняє потреби додатків.

У нашому прикладі відношення СОТРУДНИКИ_ОТДЕЛИ_ПРОЕКТИ знаходиться в 1НФ, при цьому, як було показано вище, логічна модель даних не адекватна моделі предметної області. Таким чином, першою нормальної форми недостатньо для правильного моделювання даних.

Для усунення зазначених аномалій (а насправді для правильного проектування моделі даних) застосовується метод нормалізації відношень. Нормалізація заснована на понятті функціональної залежності атрибутів відношення. Тому для подальшого викладу нам будуть потрібні ще кілька визначень.

Визначення 5.1. Функціональна залежність. Відносно R атрибут Y функціонально залежить від атрибута X (X функціонально визначає Y; X і Y можуть бути складовими) в тому і тільки в тому випадку, якщо кожному значенню X відповідає в точності одне значення Y: $RX \rightarrow RY$

Визначення 5.2. Повна функціональна залежність. Функціональна залежність $RX \rightarrow RY$ називається повною, якщо атрибут (поле) Y знаходиться в повній функціональній залежності від складеного атрибута (поля) X і не залежить функціонально від будь-якої підмножини поля X.

Визначення 5.3. Транзитивна функціональна залежність. Функціональна залежність $RX \rightarrow RY$ називається транзитивною (тобто не прямою), якщо існує такий атрибут Z, що є функціональні залежності $RX \rightarrow RZ$ і $RZ \rightarrow RY$ і відсутня функціональна залежність $RZ \rightarrow RX$ (При відсутності останнього вимоги ми мали б «нецікаві» транзитивні залежності в будь-якому відношенні, що володіє декількома ключами.)

Визначення 5.4. Неключовий атрибут. Неключовим атрибутом називається будь-який атрибут відношення, що не входить до складу ключа (зокрема, первинного).

Визначення 5.5. Взаємно незалежні атрибути. Два або більше атрибута взаємно незалежні, якщо жоден з цих атрибутів не є функціонально залежним від інших.

Тепер ми, мабуть, досить підковані теоретично, щоб приступити до розгляду нашої основної теми – нормалізації відношень.

5.6. Друга нормальна форма

5.6.1. Друга нормальна форма (2НФ)

Ми зробили перший крок у приведенні нашої бази в нормалізований стан – привели таблицю до 1НФ. Але чи зручно з нею працювати? Тепер наше відношення виглядає дещо коректніше. Однак навіть побіжний погляд на нього говорить про те, що воно поки що далеко від досконалості. Очевидно, що повторюються значення, яких чимало в попередній таблиці, є потенційним джерелом проблем.

По-перше, при введенні їх значень легко помилитися. Наприклад, досить змінити всього одну букву в назві проекту, і формально це буде вже зовсім інший проект. Знайти подібні помилки в об'ємній таблиці – завдання не з простих.

По-друге, назва проекту або номер телефону може змінитися, хоч це відбувається і не так часто (проте, нехтувати такою можливістю не варто). Тоді доведеться знову ж у всій таблиці змінювати відповідні значення.

По-третє, не можна нехтувати і ефективністю зберігання інформації. З нашої таблиці можна як мінімум тричі дізнатися дані про відділ 1 (наприклад, контактний телефон), хоча нам вистачило

б і одного разу. В даному прикладі це дрібниці, але на практиці зустрічаються бази великих обсягів.

Таким чином, боротьба з надмірністю даних вигідна з усіх боків – як з боку підвищення зручності користування даними і підтримки їх в цілісному вигляді, так і з боку ефективності обробки та зберігання даних апаратними засобами сервера баз даних. Саме на усунення цієї надмірності і спрямована подальша нормалізація відношень.

Схема відношення R знаходиться у 2НФ щодо множини функціональних залежностей F, якщо вона знаходиться в 1НФ і кожен неключовий атрибут повністю залежить від кожного ключа для R (тобто, немає неключових атрибутів, залежних від частини складного ключа).

Тепер неформальне визначення, спробуємо дати інтуїтивно зрозумілий аналог вищесказаного. Іншими словами, відношення знаходиться у 2НФ, якщо воно знаходиться в 1НФ, і при цьому всі неключові атрибути залежать тільки від ключа цілком, а не від якоїсь його частини.

Якщо атрибут не залежить повністю від первинного ключа, то він внесений помилково. Нормалізація проводиться шляхом знаходження існуючого відношення, до якого відноситься даний атрибут, або створенням нового відношення, в який атрибут повинен бути поміщений.

Зауваження. Якщо потенційний ключ відношення є простим, то відношення автоматично перебуває в 2НФ.

Відношення СОТРУДНИКИ_ОТДЕЛИ_ПРОЕКТИ не знаходиться у 2НФ, тому що є атрибути, які залежать від частини складного ключа. Залежність атрибутів, що характеризують співробітника від табельного номера співробітника є залежністю від частини складного ключа:

$H_СОТР \rightarrow \Phi АМ$

$H_СОТР \rightarrow H_ОТД$

$H_СОТР \rightarrow ТЕЛ$

Залежність найменування проекту від номера проекту є залежністю від частини складного ключа:

$H_ПРО \rightarrow ПРОЕКТ$

Для того щоб усунути залежність атрибутів від частини складного ключа, потрібно зробити декомпозицію відношення на кілька відношень. При цьому ті атрибути, які залежать від частини складного ключа, виносяться в окреме відношення.

Відношення СОТРУДНИКИ_ОТДЕЛИ_ПРОЕКТИ розіб'ємо на три відношення – СОТРУДНИКИ_ОТДЕЛИ, ПРОЕКТИ і ЗАВДАННЯ.

Відношення СОТРУДНИКИ_ОТДЕЛИ (H_СОТР, ФАМ, H_ОТД, ТЕЛ) (табл. 5.2).

Функціональні залежності. Залежність атрибутів, що характеризують співробітника від табельного номера співробітника:

$H_СОТР \rightarrow \Phi АМ$

$H_СОТР \rightarrow H_ОТД$

$H_СОТР \rightarrow ТЕЛ$

Залежність номера телефону від номера відділу: $H_ОТД \rightarrow ТЕЛ$

Таблиця 5.2. Відношення СОТРУДНИКИ_ОТДЕЛИ

$H_СОТР$	ФАМ	$H_ОТД$	ТЕЛ
1	Іванов	1	11-22-33
2	Петров	1	11-22-33
3	Сидоров	2	33-22-11

Відношення ПРОЕКТИ (H_ПРО, ПРОЕКТ) (табл. 5.3).

Функціональні залежності: $H_ПРО \rightarrow ПРОЕКТ$

Таблиця 5.3. Відношення ПРОЕКТИ

$H_ПРО$	ПРОЕКТ
1	Вайбер
2	Скайп

Відношення ЗАВДАННЯ (Н_СОТР, Н_ПРО, Н_ЗАДАН) (табл. 5.4). Функціональні залежності: {Н_СОТР, Н_ПРО} → Н_ЗАДАН

Таблиця 5.4. Відношення ЗАВДАННЯ

Н_СОТР	Н_ПРО	Н_ЗАДАН
1	1	1
1	2	1
2	1	2
3	1	3
3	2	2

5.6.2. Аналіз декомпозиції відношень

Відношення, отримані в результаті декомпозиції, знаходяться у 2НФ. Дійсно, відношення СОТРУДНИКИ_ОТДЕЛИ і ПРОЕКТИ мають прості ключі, отже, автоматично знаходяться в 2НФ, відношення ЗАВДАННЯ має складний ключ, але єдиний неключовий атрибут Н_ЗАДАН функціонально залежить від усього ключа {Н_СОТР, Н_ПРО}.

Частина аномалій поновлення усунена. Так, дані про співробітників і проектах тепер зберігаються в різних відношеннях, тому при появі співробітників, які не беруть участі ні в одному проекті, просто додаються кортежі в відношення СОТРУДНИКИ_ОТДЕЛИ. Точно також, при появі проекту, над яким не працює жоден співробітник, просто вставляється кортеж в відношення ПРОЕКТИ.

Прізвища співробітників і найменування проектів тепер зберігаються без надмірності. Якщо співробітник змінить прізвище або проект змінить назву, то таке оновлення буде вироблено в одному місці.

Якщо за проектом тимчасово припинені роботи, але потрібно, щоб сам проект зберігся, то для цього проекту видаляються відповідні кортежі щодо ЗАВДАННЯ, а дані про сам проект і дані про співробітників, які брали участь у проекті, залишаються у відношеннях ПРОЕКТИ і СОТРУДНИКИ_ОТДЕЛИ.

Проте, частину аномалій усунути не вдалося.

Решта аномалії вставки (INSERT). У відношення СОТРУДНИКИ_ОТДЕЛИ не можна вставити кортеж (4, Хоменко, 1, 33-22-11), тому що при цьому вийде, що два співробітника з 1-го відділу (Іванов і Хоменко) мають різні номери телефонів, а це суперечить моделі предметної області. У цій ситуації можна запропонувати два рішення, в залежності від того, що реально відбулося в предметної області.

Інший номер телефону може бути введений з двох причин – по помилку людини, що вводить дані про нового співробітника, або тому що номер у відділі дійсно змінився. Тоді можна написати тригер, який при вставці запису про співробітника перевіряє, чи збігається телефон з уже наявними телефоном у іншого співробітника цього ж відділу. Якщо номери відрізняються, то система повинна задати питання, чи залишити старий номер у відділі або замінити його новим. Якщо потрібно залишити старий номер (новий номер введений помилково), то кортеж з даними про нового співробітника буде вставлений, але номер телефону буде у нього буде той, який вже є в відділі (в даному випадку, 11-22-33). Якщо ж номер в відділі дійсно змінився, то кортеж буде вставлений з новим номером, і одночасно будуть змінені номери телефонів у всіх співробітників цього ж відділу.

Причина аномалії – надмірність даних, породжена тим, що в одному відношенні зберігається різномірна інформація (про співробітників і про відділи).

Висновок – збільшується складність розробки бази даних. База даних, заснована на такій моделі, працюватиме правильно тільки при наявності додаткового програмного коду у вигляді тригерів.

Решта аномалії оновлення (UPDATE). Одні і ті ж номери телефонів повторюються в багатьох кортежах відношення. Тому якщо у відділі змінюється номер телефону, то такі зміни необхідно одночасно виконати у всіх місцях, де цей номер телефону зустрічаються, інакше відношення стане некоректним. Таким чином, оновлення бази даних одним дією реалізувати неможливо. Необхідно написати тригер, який при оновленні запису коректно виправляє номери телефонів в інших місцях.

Причина аномалії – надмірність даних, також породжена тим, що в одному відношенні зберігається різнорідна інформація.

Висновок – збільшується складність розробки бази даних. База даних, заснована на такій моделі, працюватиме правильно тільки при наявності додаткового програмного коду у вигляді тригерів.

Решта аномалії видалення (DELETE). При видаленні деяких даних як і раніше може відбутися втрата іншої інформації. Наприклад, якщо видалити співробітника Сидорова, то буде втрачена інформація про те, що у відділі номер 2 знаходиться телефон 33-22-11.

Причина аномалії – зберігання в одному відношенні різнорідної інформації (і про співробітників, і про відділи).

Висновок – логічна модель даних неадекватна моделі предметної області. База даних, заснована на такій моделі, працюватиме неправильно.

Зауважимо, що при переході до другої нормальної форми відношення стали майже адекватними предметній області. Залишилися також труднощі в розробці бази даних, пов'язані з необхідністю написання тригерів, що підтримують цілісність бази даних. Ці труднощі тепер пов'язані тільки з одним відношенням СОТРУДНИКИ_ОТДЕЛИ.

5.7. Третя Нормальна Форма (3НФ)

Схема відношення R знаходиться в 3НФ щодо множини функціональних залежностей F, якщо вона знаходиться в 2НФ і жоден з первинних атрибутів в R не є транзитивно залежним від ключа для R.

Тепер неформальний виклад визначення: щоб привести відношення до 3НФ, необхідно усунути функціональні залежності між неключовими атрибутами відношення. Іншими словами, факти, збережені в таблиці, повинні залежати тільки від ключа.

Транзитивна (тобто не пряма) залежність спостерігається в тому випадку, якщо одне з двох неключових полів залежить від первинного ключа, а інше залежить від першого неключового поля.

Атрибути, що залежать від інших неключових атрибутів, нормалізуються шляхом переміщення залежного атрибута і атрибута, від якого він залежить, в нове відношення.

Відношення СОТРУДНИКИ_ОТДЕЛИ не перебуває у 3НФ, тому що є функціональна залежність неключових атрибутів (залежність номера телефону від номера відділу):

$H_OTD \rightarrow TEL$

Для того щоб усунути залежність неключових атрибутів, потрібно зробити декомпозицію відношення на кілька відношень. При цьому ті неключові атрибути, які є залежними, виносяться в окреме відношення.

Відношення СОТРУДНИКИ_ОТДЕЛИ розіб'ємо на два відношення – СПІВРОБІТНИКИ і ВІДДІЛИ.

Відношення СПІВРОБІТНИКИ (H_COTR , ФАМ, H_OTD) (табл. 5.5):

Функціональні залежності атрибутів, що характеризують співробітника від табельного номера співробітника:

$H_COTR \rightarrow ФАМ$

$H_COTR \rightarrow H_OTD$

$H_COTR \rightarrow ТЕЛ$

Таблиця 5.5. Відношення СПІВРОБІТНИКИ

<i>H_COTP</i>	ФАМ	<i>H_OTD</i>
1	Іванов	1
2	Петров	1
3	Сидоров	2

Відношення ВІДДІЛИ (*H_OTD*, ТЕЛ) (табл. 5.6):

Функціональні залежність номера телефону від номера відділу:

H_OTD → ТЕЛ

Таблиця 5.6. Відношення ВІДДІЛИ

<i>H_OTD</i>	ТЕЛ
1	11-22-33
2	33-22-11

Звернемо увагу на те, що атрибут *H_OTD*, не був ключовим щодо *СОТРУДНИКИ_ОТДЕЛИ*, стає потенційним ключем щодо ВІДДІЛИ. Саме за рахунок цього усувається надмірність, пов'язана з багаторазовим зберіганням одних і тих же номерів телефонів.

Висновок. Таким чином, всі виявлені аномалії поновлення усунені. Реляційна модель, що складається з чотирьох відношень СПІВРОБІТНИКИ, ВІДДІЛИ, ПРОЕКТИ, ЗАВДАННЯ, що знаходиться в третій нормальній формі, є адекватною описаною моделлю предметної області, і вимагає наявності тільки тих тригерів, які підтримують кількість посилок цілісності. Такі тригери є стандартними і не вимагають великих зусиль в розробці. На практиці, при створенні логічної моделі даних, досвідчені розробники зазвичай відразу будують відношення в 3НФ. Крім того, основним засобом розробки логічних моделей даних є різні варіанти ER-діаграм. Особливість цих діаграм в тому, що вони відразу дозволяють створювати відношення в 3НФ.

Як правило, модель предметної області ніколи не буває правильно розроблена з першого кроку. Експерти предметної області можуть забути про будь-що згадати, розробник може неправильно зрозуміти експерта, під час розробки можуть змінитися правила, прийняті в предметної області, і т.д. Все це може привести до появи нових залежностей, які були відсутні в первісній моделі предметної області. Тут як раз і необхідно використовувати нормалізацію хоча б для того, щоб переконатися, що відношення залишилися в 3НФ і логічна модель не погіршилася.

Підіб'ємо підсумки лекції. Проектування БД процес, як правило, трудомісткий і нешвидкий. Адже потрібно дуже добре вивчити предметну область, щоб врахувати всі нюанси, побажання і вимоги користувачів. Потім всю зібрану інформацію зобразити у вигляді об'єктів, атрибутів і зв'язків. Причому зробити це треба найбільш раціонально.

Сподіваємося, що у вас склалося певне уявлення про процес нормалізації відношень і про його роль при проектуванні баз даних. Зауважимо, що для простоти викладу були обрані кілька надумані приклади і спеціально попередньо їх «зіпсували», щоб надалі піддати нормалізації. На практиці справа зазвичай йде не так. По-перше, крім зовсім вже «зелених» новачків, навряд чи кому прийде в голову починати проектування бази даних з таблиць, які порушують 1НФ.

З складовими ключами на практиці теж доводиться зустрічатися не так вже й часто. Зазвичай складовою ключ підміняють так званим сурогатним ключем штучного походження, який зазвичай генерується самої СКБД, яка і гарантує його унікальність. Складовою ключ в цьому випадку стає альтернативним ключем, а штучний ключ – первинним. Винятком, мабуть, є ситуація з ідентифікують залежностями (зовнішніми ключами), коли має місце міграція первинного ключа батьківської суті в складовою первинний ключ дочірньої сутності.

Таким чином, на практиці призводити відношення до 1НФ і 2НФ доводиться не так вже й часто. З 3НФ ситуація дещо складніша, оскільки транзитивній залежність не завжди так явно кидається в очі. Тому порушення 3НФ – найбільш часта проблема, з якою доводиться мати справу.

6. НОРМАЛЬНІ ФОРМИ БІЛЬШ ВИСОКИХ ПОРЯДКІВ

У попередній лекції були розглянуті три нормальні форми (1НФ, 2НФ, 3НФ). У більшості випадків цього цілком достатньо, щоб розробляти цілком працездатні бази даних. У даній лекції розглядаються нормальні форми більш високих порядків, а саме – нормальна форма Бойса-Кодда (НФБК), четверта нормальна форма (4НФ), п'ята нормальна форма (5НФ).

6.1. Нормальна форма Бойса-Кодда

При приведенні відношень за допомогою алгоритму нормалізації до відношень в 3НФ неявно передбачалося, що всі відношення містять один потенційний ключ. Це не завжди вірно. Тому крім 3NF фахівці виділяють посилений різновид 3NF – нормальну форму Бойса-Кодда. Сенс посилення в тому, що при формулюванні первинних вимог до 3NF Кодд не передбачив можливість збігу трьох подій, при яких визначення 3НФ не зовсім підходить для наступних відношень:

- відношення має два або більше потенційних ключа;
- два і більше потенційних ключа є складовими;
- вони перетинаються, тобто мають хоча б один атрибут.

Можливість спільного виникнення перелічених подій вкрай мала, але все-таки не виключена. Тому і передбачено більш суворе визначення 3NF.

Для відношень, що мають один потенційний ключ (первинний), НФБК є 3НФ. Відношення знаходиться в НФБК, коли кожна нетривіальна і не приведена зліва функціональна залежність має потенційний ключ як детермінанта.

Згадаймо, що ліва частина символічного запису $A \rightarrow B$ називається детермінантом функціональної залежності, тобто детермінантою називається будь-який атрибут, значення якого визначають інші значення всередині цього рядка.

Таблиця (відношення) приведена до нормальної форми Бойса-Кодда, якщо кожен детермінант таблиці є потенційним ключем. Очевидно, якщо в таблиці є тільки один потенційний ключ, то форми 3НФ і НФБК еквівалентні. Розмірковуючи інакше, можна сказати, що форма НФБК може бути порушена, тільки в тому випадку, якщо в таблиці є більше одного потенційного ключа.

Більшість проектувальників розглядають нормальну форму Бойса-Кодда як особливий випадок 3НФ. Насправді, якщо ви використовуєте представлену нами раніше технологію, то більшість таблиць відповідають вимогам НФБК, якщо вони відповідають вимогам 3НФ. Так як же таблиця, приведена до нормальної форми 3НФ, може не бути приведеною до нормальної форми НФБК? Щоб відповісти на це питання, необхідно згадати, що транзитивна залежність має місце, коли один не первинний (неключовий) атрибут залежить від іншого не первинного атрибута.

Інакше кажучи, таблиця приведена до 3НФ, якщо вона приведена до 2НФ, і в ній відсутні транзитивні залежності. А що станеться в тому випадку, якщо неключовий атрибут є детермінантою ключового атрибута? Така обставина не порушує умов 3НФ, але не відповідає правилам НФБК, оскільки НФБК вимагає, щоб кожен детермінант в таблиці був потенційним ключем.

Згадувати про нормальну форму Бойса-Кодда слід тільки в ситуації, коли в таблицях ми активно застосовуємо складові ключі. Якщо ж при проектуванні БД розробник спирається на ідею штучних ключів (наприклад, автоінкрементний), про існування 3НФБК можна забути.

Розглянемо наступний приклад відношення, що містить два ключі.

Приклад 6.1. Нехай потрібно зберігати дані про постачання деталей деякими постачальниками. Припустимо, що найменування постачальників є унікальними. Крім того, кожен постачальник має свій унікальний номер. Дані про постачання можна зберігати відносно «Поставки» (табл. 6.1).

Таблиця 6.1. Відношення «Поставки»

Номер постачальника PNUM	Найменування постачальника PNAME	Номер деталі DNUM	Обладнання, що поставляється кількість VOLUME
1	фірма 1	1	100
1	фірма 1	2	200
1	фірма 1	3	300
2	фірма 2	1	150
2	фірма 2	2	250
3	фірма 3	1	1000

Дане відношення містить два потенційних ключа – {PNUM, DNUM} і {PNAME, DNUM}. Видно, що дані зберігаються в відношенні з надмірністю – у разі зміни найменування постачальника, це найменування потрібно змінити у всіх кортежах, де воно зустрічається. Чи можна цю аномалію усунути за допомогою алгоритму нормалізації, описаного в попередній лекції? Для цього потрібно виявити наявні функціональні залежності (як зазвичай, курсивом виділено ключові атрибути):

PNUM → *PNAME* – найменування постачальника залежить від номера постачальника.

PNAME → *PNUM* – номер постачальника залежить від найменування постачальника.

{*PNUM*, *DNUM*} → *VOLUME* – поставляється кількість залежить від першого ключа відношення.

{*PNUM*, *DNUM*} → *PNAME* – найменування постачальника залежить від першого ключа відношення.

{*PNAME*, *DNUM*} → *VOLUME* – поставляється кількість залежить від другого ключа відношення.

{*PNAME*, *DNUM*} → *PNUM* – номер постачальника залежить від другого ключа відношення.

Дане відношення не містить неключових атрибутів, залежних від частини складного ключа (див. визначення 2НФ). Дійсно, від частини складного ключа залежать атрибути PNAME і PNUM, але вони самі є ключовими. Таким чином, відношення знаходиться в 2НФ.

Крім того, відношення не містить транзитивних залежних один від одного неключових атрибутів, так як неключовий атрибут всього один – VOLUME (див. визначення 3НФ). Таким чином, показано, що відношення «Поставки» знаходиться в 3НФ.

Таким чином, описаний раніше алгоритм нормалізації не придатний до цього відношення. Проблему вирішує нормальна форма, яку історично прийнято називати нормальною формою Бойса-Кодда, і яка є уточненням 3НФ в разі наявності декількох можливих ключів, які перекриваються. Очевидно, однак, що аномалія даного відношення усувається шляхом декомпозиції його на два наступних відношення – «Постачальники» (табл. 6.2) і «Поставки-2» (табл. 6.3).

Таблиця 6.2. Відношення «Постачальники»

Номер постачальника PNUM	Найменування постачальника PNAME
1	фірма 1
2	фірма 2
3	фірма 3

Таблиця 6.3. Відношення «Поставки-2»

Номер постачальника PNUM	Номер деталі DNUM	Обладнання, що поставляється кількість VOLUME
1	1	100
1	2	200
1	3	300
2	1	150
2	2	250
3	1	1000

Визначення 6.1. Відношення R знаходиться в нормальній формі Бойса-Кодда (НФБК) тоді і тільки тоді, коли множина атрибутів (детермінанти, визначники) Всіх функціональних залежностей є потенційними ключами.

Якщо відношення знаходиться в НФБК, то воно автоматично знаходиться і в 3НФ. Дійсно, це відразу випливає з визначення 3НФ.

Відношення «Поставки» не знаходиться в НФБК, тому що є залежності (PNUM → PNAME і PNAME → PNUM), Детермінанти яких не є потенційними ключами.

Для того щоб усунути залежність від детермінантів, які не є потенційними ключами, необхідно провести декомпозицію, виносячи ці детермінанти і залежні від них частини в окреме відношення. Відношення «Постачальники» і «Поставки-2», отримані в результаті декомпозиції знаходяться в НФБК. Наведена декомпозиція відношення «Поставки» на відношення «Постачальники» і «Поставки-2» не є єдиною можливою. Альтернативною декомпозицією є декомпозиція на наступні відношення – «Постачальники» (табл. 6.4) і «Поставки-3» (табл. 6.5).

Таблиця 6.4. Відношення «Постачальники»

Номер постачальника PNUM	Найменування постачальника PNAME
1	фірма 1
2	фірма 2
3	фірма 3

Таблиця 6.5 Відношення «Поставки-3»

Найменування постачальника PNAME	Номер деталі DNUM	Обладнання, що поставляється кількість VOLUME
фірма 1	1	100
фірма 1	2	200
фірма 1	3	300
фірма 2	1	150
фірма 2	2	250
фірма 3	1	1000

На перший погляд, така декомпозиція гірша, ніж перша. Дійсно, найменування постачальників і раніше повторюються, і при зміні найменування постачальника, це найменування доведеться змінювати одночасно в декількох місцях (тим більше відразу в двох відношеннях). Здається, що ситуація стала ще гірше, ніж була до декомпозиції. Однак таке відчуття виникає від

того, що ми інтуїтивно вважаємо, що найменування постачальників можуть змінюватися, а номери – ні. Якщо ж припустити, що номери постачальників теж можуть змінюватися, то перша декомпозиція виходить така ж «погана» як і друга – повторювані номери доведеться міняти одночасно в декількох місцях і також відразу в двох відношеннях.

Насправді ніякого протиріччя тут немає. Відносно «Поставки-3» атрибут «Найменування постачальника» (PNAME) є зовнішнім ключем, що служить для зв'язку із відношенням «Постачальники». Тому, у разі зміни найменування постачальника, це зміна проводиться щодо «Постачальники» і каскадно (див. Стратегії підтримки посилювальної цілісності) поширюється на відношення «Поставки-3» абсолютно так, як зміна номера постачальника каскадно поширюється на відношення «Поставки-2». Тому, формально обидві декомпозиції зовсім рівноправні. У реальній роботі розробник вибере, звичайно, першу декомпозицію, але тут важливо підкреслити, що його вибір заснований зовсім на інших міркуваннях, що не мають відношення до формальної теорії нормальних форм.

Відношення «Поставки-2», отримане в результаті декомпозиції має всього один потенційний ключ. Тому, для аналізу відношення «Поставки-2» не потрібно залучати визначення НФБК, досить визначення 3НФ. Хоча відношення «Постачальники» має два потенційних ключа, але, так як інших атрибутів в ньому немає, воно вже так просто влаштовано, що спростити його далі не можна. Виникає питання, чи є нетривіальні приклади відношень в НФБК, які не перебувають в 3НФ і не такі прості, як відношення «Постачальники»?

Приклад 6.2. Припустимо, що нам, як і раніше необхідно враховувати поставки, але кожен акт поставки повинен мати певний унікальний номер (назвемо його «наскрізний номер поставки»). Відношення може мати такий вигляд (табл. 6.6).

Таблиця 6.6 Відношення «Поставки-3-номером»

Номер постачальника <i>PNUM</i>	Номер деталі <i>DNUM</i>	Обладнання, що поставляється кількість VOLUME	Наскрізний номер поставки NN
1	1	100	1
1	2	200	2
1	3	300	3
2	1	150	4
2	2	250	5
3	1	1000	6

Одним потенційних ключів даного відношення є, як і раніше, пара атрибутів {PNUM, DNUM}. Іншим ключем, в силу унікальності наскрізного номера, є атрибут NN. В даному відношенні є наступні функціональні залежності.

Залежність атрибутів від першого ключа відношення:

{PNUM, DNUM} → VOLUME,

{PNUM, DNUM} → NN,

Залежність атрибутів від другого ключа відношення:

NN → PNUM,

NN → DNUM,

NN → VOLUME,

Залежності, які є наслідком залежностей від ключів відношення:

{PNUM, DNUM} → {VOLUME, NN},

NN → {PNUM, DNUM},

NN → {PNUM, VOLUME},

NN → {DNUM, VOLUME},

NN → {PNUM, DNUM, VOLUME}.

Як можна помітити, детермінанти всіх залежностей є потенційними ключами, тому дане відношення знаходиться в НФБК. Особливістю даного відношення є те, що воно має два абсолютно незалежних потенційних ключа.

Нормалізація схеми бази даних сприяє більш ефективному виконанню системою управління базами даних операцій оновлення бази даних, оскільки скорочується кількість перевірок і допоміжних дій, що підтримують цілісність бази даних. При проектуванні реляційної бази даних майже завжди домагаються другий нормальної форми всіх вхідних в базу даних відношень. У часто оновлюваних базах даних зазвичай намагаються забезпечити третю нормальну форму відношень. На нормальну форму Бойса-Кодда увагу звертають набагато рідше, оскільки на практиці ситуації, в яких у відношень є кілька складових перекриваються можливих ключів, зустрічаються нечасто.

6.2. Четверта нормальна форма (4НФ)

Якщо третя нормальна форма покликана для боротьби з транзитивними залежностями, то четверта нормальна форма (4НФ) складається в конфронтації з іншим злом реляційної моделі – багатозначними залежностями. Багатозначна залежність – це зв'язок «багато до багатьох».

Розглянемо наступний приклад. Нехай потрібно враховувати дані про абітурієнтів, які вступають до ВНЗ. При аналізі предметної області були виділені наступні вимоги:

1. Кожен абітурієнт має право складати іспити на кілька факультетів одночасно.
2. Кожен факультет має свій список предметів для.
3. Один і той же предмет може здаватися на кількох факультетах.

Абітурієнт зобов'язаний здавати всі предмети, зазначені для факультету, на який він надходить, незважаючи на те, що він, може бути, вже здавав такі ж предмети на іншому факультеті. Припустимо, що нам потрібно зберігати дані про те, які предмети повинен здавати кожен абітурієнт. Спробуємо зберігати дані в одному відношенні «Абітурієнти-Факультети-Предмети» (табл. 6.7).

Таблиця 6.7. Відношення «Абітурієнти-Факультети-Предмети»

абітурієнт	факультет	предмет
Іванов	математичний	Математика
Іванов	математичний	Інформатика
Іванов	фізичний	Математика
Іванов	фізичний	фізика
Петров	математичний	Математика
Петров	математичний	Інформатика

В даний момент в відношенні зберігається інформація про те, що абітурієнт Іванов надходить на два факультети (математично і фізичний), а абітурієнт Петров – тільки на математичний. Крім того, можна зробити висновок, що на математичному факультеті потрібно здавати математику і інформатику, а на фізичному – математику і фізику.

Здається, що в відношенні є аномалія оновлення, пов'язана з тим, що дублюються прізвища абітурієнтів, найменування факультетів і найменування предметів. Однак ця аномалія легко усувається стандартним способом – винесенням всіх найменувань в окремі відношення, залишаючи в вихідному відношенні тільки відповідні номери (табл. 6.8-6.11).

Таблиця 6.8. Модифіковане відношення «Абітурієнти-Факультети-Предмети»

номер Абітурієнта	номер Факультету	номер Предмету
1	1	1
1	1	2
1	2	1
1	2	3
2	1	1
2	1	2

Таблиця 6.9 Відношення «Абітурієнти»

номер Абітурієнта	абітурієнт
1	Іванов
2	Петров

Таблиця 6.10. Відношення «Факультети»

номер Факультету	факультет
1	математичний
2	фізичний

Таблиця 6.11. Відношення «Предмети»

номер Предмету	предмет
1	Математика
2	Інформатика
3	фізика

Тепер кожне найменування зустрічається тільки в одному місці.

І все-таки як у вихідному, так і в модифікованому відношенні є аномалії оновлення, що виникають при спробі вставити або видалити кортежі.

Аномалія вставки. При спробі додати в відношення «Абітурієнти-Факультети-Предмети» новий кортеж, наприклад, (Сидоров, Математичний, Математика), ми зобов'язані додати також і кортеж (Сидоров, Математичний, Інформатика), тому що всі абітурієнти математичного факультету зобов'язані мати один і той же список предметів для. Відповідно, при спробі вставити в модифікований кортеж (3, 1, 1), ми зобов'язані вставити в нього також і кортеж (3, 1, 2).

Аномалія видалення. При спробі видалити кортеж (Іванов, Математичний, Математика), ми зобов'язані видалити також і кортеж (Іванов, Математичний, Інформатика) по тій же самій причині.

Таким чином, вставка і видалення кортежів не може бути виконана незалежно від інших кортежів відношення.

Крім того, якщо ми видалимо кортеж (Іванов, Фізичний, Математика), а разом з ним і кортеж (Іванов, Фізичний, Фізика), то буде втрачена інформація про предмети, які повинні здаватися на фізичному факультеті.

Декомпозиція відношення «Абітурієнти-Факультети-Предмети» для усунення зазначених аномалій не може бути виконана на основі функціональних залежностей, тому що це відношення не містить ніяких функціональних залежностей. Це відношення є повністю ключовим, тобто ключем відношення є вся множина атрибутів. Але ясно, що якась взаємозв'язок між атрибутами є. Цей взаємозв'язок описується поняттям багатозначною залежності.

Визначення 6.2. Нехай R – відношення, і X, Y, Z – деякі з його атрибутів (або непересічні множини атрибутів). Тоді атрибути (множини атрибутів) Y і Z багатозначно залежать від X (позначається $X \twoheadrightarrow Y \mid Z$), тоді і тільки тоді, коли з того, що в відношенні R містяться кортежі $r1 = (x, y, z1)$ і $r2 = (x, y1, z)$ слід, що у відношенні R міститься також і кортеж до $r3 = (x, y, z)$.

Відносно «Абітурієнти-Факультети-Предмети» є багатозначна залежність Факультет $\twoheadrightarrow$ Абітурієнт | Предмет. Словами це можна висловити так – для кожного факультету (для кожного значення з X) кожен вступник на нього абітурієнт (значення з Y) здає один і той же список предметів (набір значень з Z), і для кожного факультету (для кожного значення з X) кожен який вводить на факультеті іспит (значення з Z) здається одним і тим же списком абітурієнтів (набір значень з Y). Саме наявність цієї залежності не дозволяє незалежно вставляти і видаляти кортежі. Кортежі зобов'язані вставлятися і віддалятися одночасно цілими наборами.

Зауваження. Якщо по відношенню до R є не менше трьох атрибутів X, Y, Z і є функціональна залежність $X \rightarrow Y$, тобто і багатозначна залежність $X \twoheadrightarrow Y \mid Z$. Таким чином, поняття багатозначної залежності є узагальненням поняття функціональної залежності.

Визначення 6.3. Багатозначна залежність $X \twoheadrightarrow Y \mid Z$ називається нетривіальною багатозначною залежністю, якщо не існує функціональних залежностей $X \rightarrow Y$ і $X \rightarrow Z$.

Визначення 6.4. Відношення R знаходиться в четвертій нормальній формі (4НФ) тоді і тільки тоді, коли відношення знаходиться в НФБК і не містить нетривіальних багатозначних залежностей.

Декомпозиція відношення R на проєкції $R1 = R[X, Y]$ і $R2 = R[X, Z]$ буде декомпозицією без втрат тоді і тільки тоді, коли є багатозначна залежність $X \twoheadrightarrow Y \mid Z$ (теорема Фейджина [13]).

Відношення «Абітурієнти-Факультети-Предмети» знаходиться в НФБК, але не в 4НФ. Згідно з теоремою Фейджина, це відношення можна без втрат декомпонувати на два відношення – «Факультети-Абітурієнти» і «Факультети-Предмети» (табл. 6.12, 6.13).

Таблиця 12 Відношення «Факультети-Абітурієнти»

факультет	абітурієнт
математичний	Іванов
фізичний	Іванов
математичний	Петров

Таблиця 13 Відношення «Факультети-Предмети»

факультет	предмет
математичний	Математика
математичний	Інформатика
фізичний	Математика
фізичний	Фізика

В отриманих відношеннях усунені аномалії вставки і видалення, характерні для відношень «Абітурієнти-Факультети-Предмети».

Зауважимо, що отримані відношення залишилися повністю ключовими, і в них, як і раніше немає функціональних залежностей.

Відношення з нетривіальними багатозначними залежностями виникають, як правило, в результаті природного з'єднання двох відношень за загальним полем, яке не є ключовим ні в одному з відношень. Фактично це призводить до спроби зберігати в одному відношенні інформацію про дві незалежні сутності. В якості ще одного прикладу можна навести ситуацію, коли співробітник може мати багато робіт і багато дітей. Зберігання інформації про роботи і дітей в одному відношенні призводить до виникнення нетривіальної багатозначної залежності Працівник $\twoheadrightarrow$ Робота | Діти.

6.3. П'ята нормальна форма

Всі розглянуті раніше приклади використовували декомпозицію вихідного відношення на дві проекції (декомпозиція без втрати). Однак існують відношення, для яких не можна виконати декомпозицію без втрати на дві проекції, але які можна піддати декомпозиції без втрати на три або більше проекцій. Для звичайного розробника БД п'ята нормальна форма (5НФ) представляє скоріше теоретичний, ніж практичний інтерес. Приведення таблиці до вищої міри нормалізації – вкрай рідкісний випадок.

Найчастіше виникає залежність атрибутів відношення, коли вони попарно залежні один від одного. Наприклад, відношення з атрибутами «ЗАМОВЛЕННЯ», «ТОВАР», «ПОСТАЧАЛЬНИК» є таким, оскільки є функціональна залежність товару від постачальника, а також залежність замовлення від товару і постачальника. Така функціональна залежність називається залежністю з'єднання. Змістовно її можна визначити наступним чином: замовлення містить багато товарів, товар постається безліччю постачальників, постачальник міститься у великій кількості замовлень.

Для визначення п'ятої нормальної форми слід попередньо ввести поняття залежності з'єднання, яке, в свою чергу базується на понятті декомпозиції без втрат.

Як відомо декомпозицією відношення R називається заміна R на сукупність відношень $\{R_1, R_2, \dots, R_n\}$ таку, що кожне з них є проекція R , і кожен атрибут R входить хоча б в одну з проекцій декомпозиції.

Наприклад, для відношення R з атрибутами $\{a, b, c\}$ існують такі основні варіанти декомпозиції:

$\{A\}, \{b\}, \{c\}; \{A\}, \{b, c\}; \{A, b\}, \{c\}; \{B\}, \{a, c\}; \{A, b\}, \{b, c\}; \{A, b\}, \{a, c\}; \{B, c\}, \{a, c\};$
 $\{A, b\}, \{b, c\}, \{a, c\}$

Розглянемо тепер відношення R' , яке виходить в результаті операції природного з'єднання (NATURAL JOIN), застосованої до відношень, отриманих в результаті декомпозиції R .

Залежні з'єднання – це властивість декомпозиції, яка викликає генерацію помилкових рядків при зворотному з'єднанні декомпозиції відношень за допомогою операції природного з'єднання.

Декомпозиція називається **декомпозицією без втрат**, якщо R' в точності збігається з R . Неформально кажучи, при декомпозиції без втрат відношення «розділяється» на відношення-проекції таким чином, що з отриманих проекцій можлива «збірка» вихідного відношення за допомогою операції природного з'єднання.

Далеко не кожна декомпозиція є декомпозицією без втрат. Проілюструємо це на прикладі відношення R з атрибутами $\{a, b, c\}$.

Пояснимо на прикладі. Припустимо, що в нашій базі даних існує якесь відношення R – «ПОСТАЧАЛЬНИК-КНИГА-ВИДАВНИЦТВО» (табл. 6.8), що зберігає дані про постачальника (SUPPLIER), книзі, отриманої від цього постачальника, (BOOK) і видавництві, надрукованих цю книгу (PUBLISHER). Цілком ймовірно, що в процесі роботи з базою даних на основі вихідної таблиці нам доведеться побудувати два запити:

1. Перший запит поверне дані про постачальників і книгах (ми його зберегли в вигляді таблиці BOOK_SUPPLIER) (табл. 6.9).
2. На базі другого запиту ми отримали інформацію про книгах і видавництвах (таблиця PUBLISHER_BOOK) (табл. 6.9).

Таблиця 6.8. Відношення «ПОСТАЧАЛЬНИК-КНИГА-ВИДАВНИЦТВО»

SUPPLIER	BOOK	PUBLISHER
Постачальник1	Програмування в C ++	Видавництво2
Постачальник1	Мова структурованих запитів SQL	Видавництво 4
Постачальник2	Програмування в C ++	Видавництво 4
Постачальник2	Основи реляційних баз даних	Видавництво 3
Постачальник3	Мова структурованих запитів SQL	Видавництво 1

Таблиця 6.9. Декомпозиція таблиці «ПОСТАЧАЛЬНИК-КНИГА- ВИДАВНИЦТВО» на дві таблиці: BOOK_SUPPLIER, PUBLISHER_BOOK

SUPPLIER	BOOK
Постачальник1	Програмування в С ++
Постачальник1	Мова структурованих запитів SQL
Постачальник2	Програмування в С ++
Постачальник2	Основи реляційних баз даних
Постачальник3	Мова структурованих запитів SQL

BOOK	PUBLISHER
Програмування в С ++	Видавництво 2
Мова структурованих запитів SQL	Видавництво 4
Програмування в С ++	Видавництво 4
Основи реляційних баз даних	Видавництво 3
Мова структурованих запитів SQL	Видавництво 1

Зверніть увагу, що дані в нових таблицях цілком коректні і відповідають значенням, що містяться у вихідній таблиці.

А тепер спробуємо вирішити зворотну задачу – відновимо вихідну таблицю на основі даних з BOOK_SUPPLIER і PUBLISHER_BOOK. Для цього створюємо звичайний запит на мові SQL, який об'єднує обидві таблиці за загальним для них полем BOOK (відношення R '):

```
SELECT BOOK_SUPPLIER.SUPPLIER, BOOK_SUPPLIER.BOOK,
PUBLISHER_BOOK.PUBLISHER
FROM BOOK_SUPPLIER
INNER JOIN PUBLISHER_BOOK ON BOOK_SUPPLIER.BOOK =
PUBLISHER_BOOK.BOOK
```

Чи не станемо коментувати текст запиту. Для тих, хто поки не знайомий з мовою SQL, доведеться трохи потерпіти до лекції, в якій ви дізнаєтеся про базову мовою реляційних БД. Головне – результат (рис. 6.10), після з'єднання двох таблиць ми отримали чотири хибних (що не існують в реальному таблиці) рядки (записи), які відзначені сірим фоном.

Таблиця 6.10. Генерація помилкових рядків

SUPPLIER	BOOK	PUBLISHER
Постачальник2	Програмування в С ++	Видавництво 2
Постачальник1	Програмування в С ++	Видавництво 2
Постачальник3	Мова структурованих запитів SQL	Видавництво 4
Постачальник1	Мова структурованих запитів SQL	Видавництво 4
Постачальник2	Програмування в С ++	Видавництво 4
Постачальник1	Програмування в С ++	Видавництво 4
Постачальник2	Основи реляційних баз даних	Видавництво 3
Постачальник3	Мова структурованих запитів SQL	Видавництво 1
Постачальник1	Мова структурованих запитів SQL	Видавництво 1

Очевидно, що R 'не збігається з R, а значить така декомпозиція не є декомпозицією без втрат. Розглянемо тепер декомпозицію R1 = {SUPPLIER, BOOK}, R2 = {SUPPLIER, PUBLISHER}:

R1

SUPPLIER	BOOK
Постачальник1	Програмування в С ++
Постачальник1	Мова структурованих запитів SQL
Постачальник2	Програмування в С ++
Постачальник2	Основи реляційних баз даних
Постачальник3	Мова структурованих запитів SQL

R2

SUPPLIER	PUBLISHER
Постачальник 1	Видавництво 2
Постачальник 1	Видавництво 4
Постачальник 2	Видавництво 4
Постачальник 2	Видавництво 3
Постачальник 3	Видавництво 1

Така декомпозиція є декомпозицією без втрат, в чому читач може переконатися самостійно.

Розглядаючи п'яту нормальну форму, необхідно пам'ятати, що для не дозволено обов'язково повинен бути присутнім хоча б один потенційний ключ, який дозволить без втрат даних провести поділ відношення на два відношення і більше.

Визначення 6.5. Відношення знаходиться в п'ятій нормальній формі (5НФ, яку іноді називають проекційно-сполучною нормальною формою – ПСНФ) тоді і тільки тоді, коли воно знаходиться в 4НФ і кожна нетривіальна залежність визначається її потенційним ключем.

Тобто, якщо змінна відношення знаходиться в 5НФ, то єдиними в ній є ті залежності з'єднання, які визначаються її потенційними ключами, і тоді єдиними можливими декомпозиції будуть декомпозиції, які засновані на цих потенційних ключах.

У деяких випадках над відношенням зовсім неможливо виконати декомпозицію без втрат. Існують також приклади відношень, для яких не можна виконати декомпозицію без втрат на дві проекції, але які можна піддати декомпозиції без втрат на три або більшу кількість проекцій.

П'ята нормальна форма орієнтована на роботу з залежними сполуками. Зазначені залежні з'єднання між трьома атрибутами зустрічаються нечасто. А залежні з'єднання між чотирма, п'ятьма і більше атрибутами вказати практично неможливо.

Базу даних, створену в процесі нормалізації таблиць, можна отримати і альтернативним шляхом – за допомогою методики Пітера Чена «сутність-зв'язок». Дійсно, пропозиція побудови концептуальної моделі БД на основі ER-моделювання та методика нормальних форм Едгара Кодда не суперечать один одному. Схожість результатів не означає, що у своїй професійній діяльності слід віддавати перевагу тільки одній з методик моделювання, забувши про існування іншої. Навпаки, програмісту необхідно володіти всіма підходами, використовуючи в своїй роботі сильні сторони кожної з методик. Теорії реляційних баз даних і нормалізації не стоять на місці. У роботі одного з провідних фахівців з теорії баз даних – Кріса Дейта вже згадується шоста нормальна форма, правда із застереженням, що якщо мова ведеться «про класичних операціях проекції і з'єднання, то п'ята нормальна форма є останньою нормальною формою» [13].

6.4. Загальна схема процедури нормалізації

Для опису загальної схеми нормалізації (передбачається, що відношення (таблиця) вже знаходиться в 1НФ) виконати поетапно наступні процедури:

1. Змінну відношення в 1НФ слід розбити на такі проекції, які дозволять виключити всі функціональні залежності, які не є непривідними. В результаті буде отриманий набір змінних відношення в 2НФ.
2. Отримані змінні відношення в 2НФ слід розбити на такі проекції, які дозволять виключити всі існуючі транзитивні функціональні залежності. В результаті буде отриманий набір змінних відношення в 3НФ.
3. Отримані змінні відношення в 3НФ слід розбити на проекції, що дозволяють виключити всі функціональні залежності, що залишилися, в яких детермінанти не є потенційними ключами. В результаті такого приведення буде отримано набір змінних відношень в НФБК. Примітка. Правила 1-3 можуть бути об'єднані в одне: «Початкову змінну відношення слід розбити на проекції, що дозволяють виключити всі функціональні залежності, в яких детермінанти не є потенційними ключами».
4. Отримані змінні відношення в НФБК слід розбити на проекції, що дозволяють виключити всі багатозначні залежності, які не є також функціональними. В результаті буде отриманий

набір змінних відношення в 4НФ. Примітка. На практиці такі багатозначні залежності зазвичай виключаються перед виконанням етапів 1-3.

5. Отримані змінні відношення в 4НФ слід розбити на проекції, що дозволяють виключити всі залежності з'єднання (якщо вдасться їх виявити), які не визначаються потенційними ключами. В результаті буде отриманий набір змінних відношення в 5НФ.

З приводу наведених вище правил можна зробити кілька додаткових зауважень.

Процес розбиття на проекції на кожному етапі повинен бути виконаний без втрат і з збереженням залежностей (там, де це можливо).

Існує неформальний набір наступних альтернативних визначень НФБК, 4НФ і 5НФ:

- змінна відношення R знаходиться в НФБК тоді і тільки тоді, коли кожна функціональна залежність, що задовольняє змінну відношення R , визначається її потенційними ключами;
- змінна відношення R знаходиться в 4НФ тоді і тільки тоді, коли кожна багатозначна залежність, що задовольняє змінну відношення R , визначається її потенційними ключами;
- змінна відношення R знаходиться в 5НФ тоді і тільки тоді, коли кожна залежність з'єднання, що задовольняє змінну відношення R , визначається її потенційними ключами.

Аномалії оновлення були викликані саме тими функціональними залежностями, багатозначними залежностями або залежностями з'єднання, які не визначались потенційними ключами. Мається на увазі, що всі згадані тут функціональні, багатозначні залежності і залежності з'єднання є нетривіальними.

6.5. Денормалізація бази даних

6.5.1. Загальні відомості про денормалізацію

До сих пір в цій та попередній лекції в основному передбачалося, що повна нормалізація до 3НФ (і зрідка аж до 5НФ) дуже бажана. Але на практиці часто можна чути твердження, що для досягнення високої продуктивності системи іноді слід виконати денормалізацію («відкат назад» до тієї чи іншої нормальної форми). При цьому використовуються доводи, подібні таким.

1. Повна нормалізація призводить до появи великої кількості логічно незалежних змінних відношення (і передбачається, що розглянуті змінні відношення є базовими).
2. Велика кількість логічно незалежних змінних відношення призводить до появи великої кількості окремо збережених фізичних файлів.
3. Велика кількість окремо збережених фізичних файлів призводить до появи великої кількості операцій введення-виведення.

Строго кажучи, ці доводи, звичайно ж, не вірні, оскільки у визначенні реляційної моделі ніде не стверджується, що базові змінні відношення повинні перебувати у взаємно-однозначним дотриманням збереженими файлами. Тому денормалізацію в разі необхідності слід виконувати на рівні збережених файлів, але не на рівні базових змінних відношення.

Однак в деякому відношенні ці доводи все ж вірні для сучасних продуктів SQL. Нагадаємо, що нормалізація змінної відношення R означає її заміну множиною таких проекцій $R_1, R_2, \dots, R_n$, що результатом зворотного з'єднання проекцій $R_1, R_2, \dots, R_n$ обов'язково буде значення R . Кінцевою метою нормалізації є скорочення ступеня надмірності даних за рахунок приведення проекцій $R_1, R_2, \dots, R_n$ до максимально високого рівня нормалізації.

Тепер можна перейти до визначення поняття денормалізації. Нехай R_1, R_2, R_n є множиною змінних відношення. Тоді денормалізації цих змінних відношення називається така заміна змінних відношення їх з'єднанням R , що для всіх можливих значень i ($i = 1, \dots, n$) виконання проекції R по атрибутам R_i обов'язково знову призводить до створення значень R_i .

Кінцевою метою денормалізації є збільшення ступеня надмірності даних за рахунок приведення змінної відношення R до більш низького рівня нормалізації в порівнянні з вихідними змінними відношення $R_1, R_2, \dots, R_n$. Точніше, переслідується мета скоротити кількість з'єднань, які потрібно виконувати в додатку на етапі виконання, оскільки деякі з цих сполук вже виконані заздалегідь в складі робіт з проектування бази даних.

У разі денормалізації колишній підхід (застосовувався при нормалізації), створений на підставі строго наукової і логічної теорії, замінюється чисто прагматичних і суб'єктивним підходом.

Друга очевидна складність пов'язана з проблемами надмірності і аномаліями оновлення, які виникають через те, що доводиться мати справу з не повністю нормалізованими змінними відношення.

І найголовніша проблема формулюється в такий спосіб. Коли мова йде про те, що денормалізація «сприяє досягненню високої продуктивності», фактично мається на увазі, що вона сприяє досягненню високої продуктивності деяких конкретних додатків. Це відноситься до «правильної» денормалізації, тобто до денормалізації, яка виконується тільки на фізичному рівні, а також до того типу денормалізації, яку іноді доводиться здійснювати в сучасних продуктах SQL.

Будь-яка обрана фізична структура, яка прекрасно підходить для одних додатків з точки зору їх продуктивності, може виявитися абсолютно непридатною для інших. Наприклад, припустимо, що кожна базова змінна відношення відображається на один фізичний файл, а кожен файл, що зберігається, складається з фізично суміжного набору збережених записів, по одній для кожного кортежу відповідної змінної відношення.

Пам'ятайте, що швидкість обробки даних може мати більше значення, ніж недоліки деякої аномалії даних. Крім того, деякі аномалії можуть мати суто теоретичне значення. У сучасних базах даних коректну нормалізацію часто дуже важко забезпечити. Протиріччя між ефективністю проекту, інформаційними потребами і швидкістю обробки інформації найчастіше дозволяється пошуком компромісу, який, як правило, полягає в деякій денормалізації.

Закріпимо наші знання про денормалізації на спрощеному класичному прикладі – таблиці з бази даних продажів книгарні. Для простоти припустимо, що одну книгу пише тільки один автор.

Дані можуть групуватися в таблиці (відношенні) різними способами. При проектуванні БД в якості відправної точки може використовуватися одне універсальне відношення, до якого включаються всі необхідні атрибути. Спочатку воно може містити всі дані, які розмістили в одній універсальній таблиці наступним чином:

Покупець	Купівля	Дата купівлі	Сума	ТелПокупця
ТОВ Астра	Іванов «Мікрософт Офіс» - 2 екз	05.01.10	400	111-11-11
Астра ТОВ	Іванов «Мікрософт Офіс» - 1 прим Дерк «Довідник по РНР» - 1 прим Дерк «Довідник по Java» - 1 прим	06.01.10	800	111-11-11
Аврора з-д	Дейт «Бази даних» - 1 прим	30.12.09	50	222-22-22

Виконаємо вимоги 1НФ про атомарність даних, і створимо відповідну нормалізовану таблицю. Тоді ми отримуємо таблицю Продаж в такому вигляді:

Покупець	Купівля	Автор	К	Ціна	Дата купівлі	Сума	ТелПокуп
ТОВ Астра	«Мікрософт Офіс»	Іванов	2	200	05.01.10	400	111-11-11
Астра ТОВ	«Мікрософт Офіс»	Іванов	1	200	06.01.10	200	111-11-11
Астра ТОВ	«Довідник по РНР»	Дерк	1	300	06.01.10	300	111-11-11
Астра ТОВ	«Довідник по Java»	Дерк	1	300	06.01.10	300	111-11-11
Аврора з-д	«Бази даних»	Дейт	1	50	30.12.09	50	222-22-22

Неподільність (Атомарність) означає, що в таблиці не повинно бути полів, які можна розбити на більш дрібні поля. Тепер можемо вважати, що кожне значення кожного з атрибутів нашого відношення є атомарним. Отже, таблиця знаходиться в 1НФ.

Під поняттям повторювані групи мають на увазі поля, що містять однакові за змістом значення. Тому не можна розмішувати покупку декількох книг в трьох і більше полях, які є списком або безліччю значень типу:

Покупець	Покупка1	Покупка2	Покупка3	...
ТОВ «Астра»	«Мікрософт Офіс»	«Мікрософт Офіс»	«Довідник по РНР»	...
...	...	...	...	...

Для приведення цієї схеми в 2НФ необхідно поділити вихідне відношення (таблицю Продаж) на два, в яких все неключових атрибути будуть повністю функціонально залежати від ключа, тобто виносимо їх в окремі таблиці.

Покупець	ТелПокупця		Автор
ТОВ «Астра»	111-11-11		Іванов
«Астра» ТОВ	111-11-11		Дерк
Аврора з-д	222-22-22		Дейт
книга		Ціна	ISBN
Мікрософт Офіс		200	1-111-11111-1
Справочник по РНР		300	2-222-22222-2
Справочник по Java		300	3-333-33333-3
Бази даних		50	4-444-44444-4

У кожену таблицю додамо поля в якості штучних (сурогатних ключів) для зв'язку між таблицями (РК – первинний ключ, FK – зовнішній ключ). Після того як ми визначили первинні ключі для таблиць, вони придбали такий вигляд:

Таблиця Customers (Покупці)

ID_Customer-РК	Покупці	ТелПокупця
1	ООО Астра	111-11-11
2	Аврора з-д	222-22-22

Таблиця Authors (Автори)

ID_Autor-РК	Автор
1	Іванов
2	Дерк
3	Дейт

Таблиця Books (Книги)

ID_Book-РК	ID_Autor-FK	Книга	Ціна	ISBN
1	1	Мікрософт Офіс	200	1-111-11111-1
2	2	Справочник по РНР	300	2-222-22222-2
3	2	Справочник по Java	300	3-333-33333-3
4	3	Бази даних	50	4-444-44444-4

Міркуючи аналогічним чином щодо таблиці Продаж, отримаємо таку структуру:

Таблиця Orders (Продажі)

ID_Order	ID_Customer-FK	Дата покупки	Всього
1	1	05.01.10	400
2	1	06.01.10	800
3	2	30.12.09	50

Таблиця OrdersDetail (Подробиці продажу)

ID_Order-FK	ID_Book-FK	Кількість	Сума
1	1	2	400
2	1	1	200
2	2	1	300
2	3	1	300
3	4	1	50

Отже, ми мінімізували кількість надлишкової інформації, встановили зв'язки між новими таблицями, і тепер наша структура відповідає другій нормальній формі. Але в таблиці OrdersDetail поле Сума залежить від поля Кількість. Аналогічне розрахункове поле – Сума таблиці Orders. Прибираємо їх.

Ось тепер наші таблиці знаходяться в 3 нормальній формі. І остаточно таблиці Orders і OrdersDetail тепер виглядає наступним чином:

Таблиця Orders (Продаж)

ID_Order	ID_Customer-FK	Дата покупки
1	1	05.01.10
2	1	06.01.10
3	2	30.12.09

Таблиця OrdersDetail (Подробиці продажу)

ID_Order-FK	ID_Book-FK	Кіл-ть
1	1	2
2	1	1
2	2	1
2	3	1
3	4	1

Друга назва процесу нормалізації – «формальне проектування бази даних». Ключовим словом в цьому визначенні є слово «формальне». Проводячи нормалізацію, ми добиваємося скорочення (а в ідеалі – повної відсутності) надмірності інформації, що зменшує ймовірність появи аномалій введення, оновлення та видалення, полегшує адміністрування бази і зменшує її розмір. З іншого боку, нормалізація ускладнює введення-виведення інформації в базу – доводиться створювати складні запити, які, до того ж, можуть відпрацьовувати досить повільно.

Проводячи формальну нормалізацію для нашого прикладу, ми не врахували те, що ціни на книги можуть змінюватися, а зі зміною цін «попливуть» наші розрахункові поля Сума і Разом. У нас є два виходи: вводити таблицю історії цін, і розраховувати суми продажів через неї або піти на часткову денормалізацію.

Перший варіант ускладнить і сповільнить висновок даних про суми продажів, тому ми підемо іншим шляхом і повернемо назад поля Сума і Разом (проведемо композицію декількох відношень (таблиць) в одну). Наша підсумкова структура бази даних тепер виглядає так:

Таблиця Authors (Автори)

ID_Autor-PK	Автор
1	Иванов
2	Дерк
3	Дейт

Таблиця Customers (Покупці)

ID_Customer-PK	Покупець	ТелПокупця
1	ООО Астра	111-11-11
2	Аврора з-д	222-22-22

Таблиця Books (Книги)

ID_Book-PK	ID_Autor-FK	Книга	Ціна	ISBN
1	1	Мікрософт Офіс	200	1-111-11111-1
2	2	Справочник по PHP	300	2-222-22222-2
3	2	Справочник по Java	300	3-333-33333-3
4	3	Бази даних	50	4-444-44444-4

Таблиця Orders (Продажі)

ID_Order	ID_Customer-FK	Дата покупки	Всього
1	1	05.01.10	400
2	1	06.01.10	800
3	2	30.12.09	50

Таблиця OrdersDetail (Подробиці продажу)

ID_Order-FK	ID_Book-FK	Кількість	Сума
1	1	2	400
2	1	1	200
2	2	1	300
2	3	1	300
3	4	1	50

Підіб'ємо підсумки денормалізації, відповівши на питання:

1. Коли потрібна денормалізація?
2. Як визначити, коли денормалізація виправдана?
3. Як грамотно реалізувати денормалізацію.

6.5.2. Коли потрібна денормалізація?

Розглянемо деякі поширені ситуації, в яких денормалізація може виявитися корисною.

Велика кількість з'єднань таблиць. У запитах до повністю нормалізованої бази нерідко доводиться з'єднувати до десятка, а то й більше, таблиць. А кожне з'єднання – операція дуже ресурсномістка. Як наслідок, такі запити вимагають великих ресурсів сервера і виконуються повільно.

У такій ситуації може допомогти:

1. Денормалізація шляхом скорочення кількості таблиць. Краще об'єднувати в одну кілька таблиць, які мають невеликий розмір, що містять рідко змінну (умовно-постійну, або нормативно-довідкову) інформацію, причому інформацію, за змістом тісно пов'язану між собою.

2. У загальному випадку, якщо у великій кількості запитів потрібно об'єднувати більше п'яти або шести таблиць, слід розглянути варіант денормалізації бази даних.

3. Денормалізація шляхом введення додаткового поля в одну з таблиць. При цьому з'являється надмірність даних, потрібні додаткові дії для збереження цілісності БД.

Розрахункові значення. Найчастіше повільно виконуються і споживають багато ресурсів запити, в яких виробляються якісь складні обчислення, особливо при використанні угруповань і агрегатних функцій (Sum, Max і т.п.). Іноді має сенс додати в таблицю 1-2 додаткових стовпчика, що містять часто використовувані (і складно обчислювані) розрахункові дані.

Припустимо, що необхідно визначити загальну вартість кожного замовлення (як в вище наведеному прикладі). Для цього спочатку слід визначити вартість кожного продукту (за формулою «кількість одиниць продукту» × «ціна одиниці продукту»). Після цього необхідно згрупувати вартості на замовлення.

Виконання цього запиту є досить складним і, якщо в базі даних зберігаються відомості про велику кількість замовлень, може зайняти багато часу. Замість виконання такого запиту можна на етапі розміщення замовлення визначити його вартість і зберегти її в окремому стовпці таблиці

замовлень. У цьому випадку для отримання необхідного результату достатньо витягти з даного стовпчика попередньо розраховані значення.

Створення стовпця, що містить попередньо розраховані значення, дозволяє значно заощадити час при виконанні запиту, однак вимагає своєчасної зміни даних в цьому стовпці.

Довгі поля. Якщо в базі даних є великі таблиці, що містять довгі поля (Blob, Long і т.п.), то серйозно прискорити виконання запитів до такої таблиці ми зможемо, якщо винесемо довгі поля в окрему таблицю. Наприклад, створити в базі каталог фотографій, в тому числі зберігати в blob-полях і самі фотографії (професійної якості, з високим дозволом, і відповідного розміру). З точки зору нормалізації абсолютно правильною буде така структура таблиці:

- ID фотографії;
- ID автора;
- ID моделі фотоапарата;
- сама фотографія (blob-поле).

А тепер уявімо, скільки часу буде працювати запит, що підраховує кількість фотографій, зроблених будь-яким автором?

Правильним рішенням (хоча і таким, що порушує принципи нормалізації) в такій ситуації буде створити ще одну таблицю, що складається всього з двох полів – ID фотографії та blob-поле з самої фотографією. Тоді вибірки з основної таблиці (в якій величезного blob-поля зараз вже немає) йтимуть миттєво, ну а коли необхідно подивитися саму фотографію, то, звичайно, необхідно почекати.

6.5.3. Як визначити, коли денормалізація виправдана?

Витрати і вигоди. Один із способів визначити, наскільки виправдані ті чи інші кроки денормалізації, – провести аналіз в термінах витрат і можливих вигод. У скільки обійдеться денормалізована модель даних? Визначити вимоги (чого хочемо досягти):

- визначити вимоги до даних (що потрібно дотримуватися);
- знайти мінімальний крок, що задовольняє ці вимоги;
- підрахувати витрати на реалізацію;
- реалізувати.

Витрати включають в себе фізичні аспекти, такі як дисковий простір, ресурси, необхідні для управління цією структурою, і втрачені можливості через тимчасових затримок, пов'язаних з обслуговуванням цього процесу.

За денормалізація потрібно платити. У денормалізованій базі даних підвищується надмірність даних, що може підвищити продуктивність, але зажадає більше зусиль для контролю за пов'язаними даними. Ускладниться процес створення додатків, оскільки дані будуть повторюватися і їх важче буде відстежувати. Крім того, здійснення посилювальної цілісності виявляється не простою справою – пов'язані дані виявляються розділеними по різних таблицях.

До переваг відноситься більш висока продуктивність при виконанні запиту і можливість отримати при цьому більш швидку відповідь. Крім того, можна отримати і інші переваги, у тому числі збільшення пропускної здатності, рівня задоволеності клієнтів і продуктивності, а також більш ефективне використання інструментарію зовнішніх розробників.

Частота запитів і стійкість продуктивності. Наприклад, при зверненні до БД 70% з 1000 запитів, щодня генеруються підприємством, являють собою запити рівня зведених, а не детальних даних. При використанні таблиці зведених даних запити виконуються приблизно за 6 секунд замість 4 хвилин, тобто час обробки менше на 2730 хвилин. Навіть з поправкою на ті 105 хвилин, які необхідно щотижня витрачати на підтримку таблиць зведених даних, в результаті економиться 2625 хвилин в тиждень, що повністю виправдовує створення таблиці зведених даних. Згодом може трапитися так, що велика частина запитів буде звернена не до зведеними даними, а до детальним даними. Чим менше число запитів, що використовують таблицю зведених даних, тим простіше від неї відмовитися, не зачіпаючи інші процеси.

Інші критерії. Перераховані вище критерії не єдині, які слід враховувати, приймаючи рішення про те, чи слід робити наступний крок в оптимізації БД. Необхідно враховувати й інші

фактори, в тому числі пріоритети бізнесу і потреби кінцевих користувачів. Користувачі повинні розуміти, як з технічної точки зору на архітектуру системи впливає вимога користувачів, що бажають, щоб всі запити виконувалися за кілька секунд. Найпростіше добитися цього розуміння – окреслити витрати, пов'язані зі створенням таких таблиць і їх управлінням.

6.5.4. Як грамотно реалізувати денормалізацію?

Зберегти детальні таблиці. Щоб не обмежувати можливості бази даних, важливі для бізнесу, необхідно дотримуватися стратегії співіснування, а не заміни, тобто зберегти детальні таблиці для глибинного аналізу, додавши до них денормалізовані структури. Наприклад, лічильник відвідувань. Для бізнесу необхідно знати кількість відвідувань веб-станиці. Але для аналізу (за періодами, по країнах тощо) ймовірно, що знадобляться детальні дані – таблиця з інформацією про кожному відвідуванні.

Використання тригерів. Можна денормалізувати структуру бази даних і при цьому продовжувати користуватися перевагами нормалізації, якщо користуватися тригерами баз даних для збереження цілісності інформації, ідентичності дублюються даних.

Наприклад, при додаванні обчислюваного поля на кожен з стовпців, від яких обчислюється поле залежить, додається тригер, що викликає єдину збережену процедуру (це важливо!), яка і записує потрібні дані в обчислюване поле. Треба тільки не пропустити жоден з стовпців, від яких залежить обчислюване поле.

Програмна підтримка. Наприклад, в MySQL версії 4.1 тригерів і збережених процедур не передбачено. Тому дбати про забезпечення несуперечності даних в денормалізованій базі повинні розробники додатків. За аналогією з тригерами, повинна бути одна функція, оновлююча все поля, що залежать від змінюваного поля.

Підведемо підсумки. При денормалізації важливо зберегти баланс між підвищенням швидкості роботи бази даних і збільшенням ризику появи суперечливих даних, між полегшенням роботи програмістам, які пишуть Select-запити, і ускладненням завдання тих, хто забезпечує наповнення і оновлення бази даних. Тому проводити денормалізацію бази даних треба дуже акуратно, дуже вибірково, і тільки там, де без цього ніяк не обійтися.

Якщо заздалегідь не можна підрахувати плюси і мінуси денормалізації, то спочатку необхідно реалізувати модель з нормалізованими таблицями, і лише потім, для оптимізації проблемних запитів проводити денормалізацію.

7. КОНЦЕПТУАЛЬНЕ ПРОЕКТУВАННЯ БАЗ ДАНИХ

Широке поширення реляційних СКБД та їх використання в найрізноманітніших додатках показує, що реляційна модель даних достатня для моделювання різноманітних предметних областей. Однак проектування реляційної бази даних (РБД) в термінах відношень на основі коротко розглянутого нами в двох попередніх лекціях механізму нормалізації часто представляє собою дуже складний і незручний для проектувальника процес.

При проектуванні інформаційних систем (ІС) з базами даних, що входять до їх складу, зручно користуватися класифікацією моделей, зображеною на рис. 7.1. Усі моделі даних, які використовуються на трьох етапах проектування, діляться на три види. На першому етапі досліджується предметна область, виявляються в ній об'єкти і процеси, які треба буде відобразити в ІС при розв'язанні задач, для яких розробляється ІС. Модель, яка використовується на цьому етапі, служить для наочного представлення семантичних зв'язків у предметній області. Строга формалізація структури даних на цьому етапі не обов'язкова. Такі моделі називаються **інфологічними (концептуальними)**. Нині найпоширенішою інфологічною моделлю є модель «сутність-зв'язок».

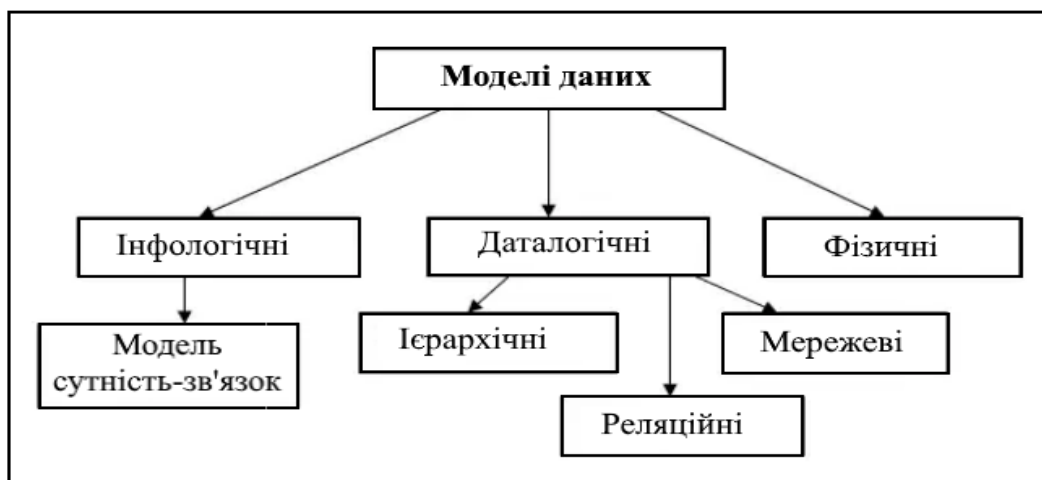


Рис.7.1. Моделі даних

Після того як закінчено дослідження предметної області і детально поставлена задача проектування можна переходити до другого етапу, на якому проектується **база даних**. На цьому етапі використовуються формальні моделі даних, в які треба перетворити інфологічну модель. Такі моделі, які безпосередньо використовуються в базах даних, називаються **даталогічними**. Найбільше поширення серед даталогічних моделей отримали реляційні моделі баз даних.

Базу даних незалежно від її даталогічної моделі можна по-різному розмістити на різних зовнішніх носіях. Наприклад, можна використати жорсткий диск або твердотільну зовнішню пам'ять. Для опису фізичного розміщення бази даних служить **фізична модель**.

7.1. Обмеженість реляційної моделі при проектуванні БД

При використанні в проектуванні обмеженість реляційної моделі проявляється в таких аспектах.

1. Модель не забезпечує достатніх коштів для подання сенсу даних. Семантика реальної предметної області повинна незалежним від моделі способом представлятися в голові проектувальника. Зокрема, це відноситься до відзначеної нами раніше проблеми уявлення обмежень цілісності, що виходять за межі обмежень первинного та зовнішнього ключа.
2. У багатьох прикладних областях важко моделювати предметну область на основі плоских таблиць. У ряді випадків на самій початковій стадії проектування дизайнерові доводиться нелегко, оскільки від нього вимагається описати предметну область у вигляді однієї (можливо, навіть ненормалізованої) таблиці.

3. Хоча весь процес проектування відбувається на основі врахування залежностей, реляційна модель не надає будь-які формалізовані кошти для подання цих залежностей.
4. Незважаючи на те, що процес проектування починається з виділення деяких істотних для додатка об'єктів предметної області («сутностей») і виявлення зв'язків між цими сутностями, реляційна модель даних не пропонує жодного механізму для поділу сутностей і зв'язків.

Потреба проектувальників баз даних в більш зручних і потужних засобах моделювання предметної області спричинила виникнення напрямку семантичних моделей даних. Хоча будь-яка розвинена семантична модель даних, як і реляційна модель, включає структурну, маніпуляційну та цілісну частини, головним призначенням семантичних моделей є забезпечення можливості вираження семантики даних.

Перш ніж ми коротко розглянемо особливості двох поширених семантичних моделей, зупинимося на можливих областях їх застосування. Найчастіше на практиці семантичне моделювання використовується на першій стадії проектування бази даних. При цьому в термінах семантичної моделі проводиться концептуальна схема бази даних, яка потім вручну перетворюється до реляційної (або якої-небудь іншої) схеми. Цей процес виконується під управлінням методик, в яких досить чітко обумовлено всі етапи такого перетворення. Основною перевагою даного підходу є відсутність потреби в додаткових програмних засобах, що підтримують семантичне моделювання. Потрібно лише знання основ обраної семантичної моделі і правил перетворення концептуальної схеми в реляційну схему.

Слід зауважити, що багато початківців проектувальників баз даних недооцінюють важливість семантичного моделювання вручну. Найчастіше це сприймається як додаткова і зайва робота. Ця точка зору абсолютно невірна. По-перше, побудова потужної і наочної концептуальної схеми БД дозволяє більш повно оцінити специфіку предметної області, яка моделюється і уникнути можливих помилок на стадії проектування схеми реляційної БД. По-друге, на етапі семантичного моделювання проводиться важлива документація (у вигляді вручну намальованих діаграм і коментарів до них), яка може виявитися дуже корисною не тільки при проектуванні схеми реляційної БД, але і при експлуатації, супроводі та розвитку вже заповненої БД.

Відсутність такого роду документації суттєво ускладнює внесення навіть невеликих змін в схему існуючої реляційної БД. Звичайно, це стосується випадків, коли проектувана БД містить не надто мале число таблиць. Швидше за все, без семантичного моделювання можна обійтися, якщо число таблиць не перевищує десяти, але воно абсолютно необхідно, якщо БД включає більше сотні таблиць. Для справедливості зауважимо, що процедура створення концептуальної схеми вручну з її подальшим перетворенням в реляційну схему БД скрутна в разі великих БД.

Історія систем автоматизації проектування баз даних (CASE-засобів) почалася з автоматизації процесу побудови діаграм, перевірки їх формальної коректності, забезпечення засобів довготривалого зберігання діаграм та іншої проектною документації. Наявність електронного архіву проектною документації допомагає при експлуатації, адмініструванні та супроводі бази даних. Але система, яка обмежується підтримкою малювання діаграм, перевіркою їх коректності та зберіганням, нагадує текстовий редактор, що підтримує введення, редагування і перевірку синтаксичної коректності конструкцій деякого мови програмування, але існуючий окремо від компілятора. Здається, обґрунтованим бажанням є розширити такий редактор функціями компілятора, і це дійсно можливо, оскільки відома техніка компіляції конструкцій мови програмування в коді цільового комп'ютера. Але якщо є чітка методика перетворення концептуальної схеми БД в реляційну схему, то чому б не виконати програмну реалізацію відповідного «компілятора» і не включити її до складу системи проектування баз даних.

Ця ідея, здалася розумною виробникам CASE-засобів проектування БД. Переважна більшість подібних систем, представлених на ринку, забезпечує автоматизоване перетворення діаграмних концептуальних схем баз даних, представлених в тій чи іншій семантичній моделі даних, в реляційні схеми, специфіковані найчастіше на мові SQL.

Як правило, CASE-засоби, що автоматизують перетворення концептуальної схеми БД в реляційну, виробляють реляційну схему бази даних у третій нормальній формі. Нормалізація вищого рівня ускладнює програмну реалізацію і нечасто потрібна на практиці.

Починаючи з цієї лекції, ми переходимо до використання термінів таблиця, рядок і стовпець замість строгих реляційних термінів відношення, атрибут і таблиця, оскільки тут під реляційними базами даних розуміються, головним чином, SQL-орієнтовані бази даних, для яких ця спрощена термінологія більш звична.

7.2. Концептуальна модель предметної області

При проектуванні ІС дані треба представити у вигляді простої моделі, що відображає сенс даних, їх взаємозв'язок і не прив'язується при цьому до конкретного типу бази даних. Такі моделі дістали назву *інфологічних (концептуальних)*.

Концептуальна модель предметної області – це знання про предметну область у вигляді понять (концептів). Знання можуть бути як у вигляді неформальних знань, так і виражені формально за допомогою деяких засобів. Такими засобами можуть виступати текстові описи предметної області, набори посадових інструкцій тощо.

Досвід показує, що текстовий спосіб представлення моделі предметної області є неефективним. Набагато інформативнішим і кориснішим при розробці баз даних є опис предметної області, виконаний за допомогою спеціалізованих графічних нотацій. Є велика кількість методик опису предметної області. Концептуальна модель БД – відображає інформаційний зміст даних як основних понять і відношення між ними. Концептуальна модель не зачіпає фізичного стану даних, у тому числі архітектури даних, методів доступу, форматів фізичних даних.

Концептуальна модель предметної області описує процеси, що відбуваються у предметній області й дані, які використовуються цими процесами. Від того, наскільки правильно змодельована предметна область, залежить успіх подальшої розробки програм.

Побудова моделі предметної області розпочинається з виявлення абстракцій, що існують у реальному світі та належать модельованій предметній області. Концептуальна модель відображає семантику предметної області у вигляді сукупності понять (сутностей), їх характеристик (атрибутів) і зв'язків (асоціативних відношень між сутностями). Варто відзначити, що концептуальна модель створюється в першу чергу для полегшення сприйняття інформації звичайним користувачем.

З концептуального проектування починається створення концептуальної схеми БД, в основі якої лежить концептуальна модель даних. Концептуальна модель представляє загальний погляд на дані. Розрізняють два головних підходи до моделювання даних при концептуальному проектуванні:

- семантичні моделі;
- об'єктні моделі.

Семантичні моделі головну увагу приділяють структурі даних. Найбільш поширеною семантичною моделлю є модель «сутність – зв'язок» (Entity Relationship model, ER-модель). **ER-модель** складається із сутностей, зв'язків, атрибутів, доменів атрибутів, ключів. Моделювання даних відображає логічну структуру даних, так само, як блок-схеми алгоритмів відображають логічну структуру програми.

Об'єктні моделі головну увагу приділяють поведінці об'єктів даних і засобам маніпуляції даними. Головне поняття таких моделей – об'єкт, тобто сутність, яка має стан і поведінку. Стан об'єкта визначається сукупністю його атрибутів, а поведінка об'єкта визначається сукупністю операцій специфікованих для нього.

Зближення цих моделей реалізується в розширеному ER-моделюванні (Extended Entity Relationship model, EER-модель).

7.3. Модель «Сутність-Зв'язок»

ER-моделювання являє собою низхідний підхід до проектування БД, який починається з визначення найбільш важливих даних, які називаються сутностями (entities), і зв'язків (relationships) між даними, які повинні бути представлені в моделі. Потім в модель заноситься

інформація про властивості сутностей і зв'язків, яка називається атрибутами (attributes), а також всі обмеження, які відносяться до сутностей, зв'язків і атрибутів. ER-модель дає графічне представлення логічних об'єктів і їх відношень в структурі БД. Послідовність проведення ER-модельовання показана на рис. 7.2.



Рис. 7.2. Етапи побудови моделі «Сутність – Зв'язок»

Вперше поняття ER-моделі запровадив Пітер Чен у 1976 р. Підхід П. Чена дозволив концептуальне моделювання перевести в практичну площину проектування БД. У подальшому діаграми Чена набули розвитку у багатьох інших роботах з ER-модельовання. До них належать такі моделі:

- «пташина лапка», розроблена К.М. Бахманом;
- IDEF1X, розроблена Т. Ремесм;
- на основі UML;
- модель Баркера і багато інших моделей.

7.3.1. Сутності

Сутність дозволяє моделювати клас однотипних об'єктів. Сутність має унікальне ім'я у межах системи, що моделюється. Оскільки сутність відповідає деякому класу однотипних об'єктів, то передбачається, що в системі існує багато екземплярів даної сутності. Об'єкт, якому відповідає сутність, має набір атрибутів, які характеризують його властивості. При цьому набір атрибутів повинен бути таким, щоби можна було розрізняти конкретні екземпляри сутності.

Приклад. Сутність **Викладач** може мати такі атрибути: *Табельний номер, Прізвище, Ім'я, По батькові, Посада, Вчений ступінь*.

Набір атрибутів, що однозначно ідентифікує конкретний екземпляр сутності, називають *ключовим*. У наведеному прикладі для сутності **Викладач** ключем буде *Табельний номер*, оскільки для всіх викладачів табельні номери різні. Екземпляром сутності **Викладач** буде опис конкретного викладача. Загальноприйняте позначення сутності – прямокутник (рис. 7.3).



Рис. 7.3. Представлення сутностей і атрибутів в ER-діаграмах

7.3.2. Зв'язки

Між сутностями встановлюються зв'язки, які вказують яким чином сутності співвідносяться або взаємодіють між собою. Розрізняють такі зв'язки:

- між двома сутностями (бінарний зв'язок);
- між трьома сутностями (тернарний зв'язок);
- між N сутностями (N-арний зв'язок);
- між однією сутністю (рекурсивний зв'язок).

Найбільш поширеними є бінарні зв'язки. Зв'язок показує яким чином екземпляри сутностей зв'язані між собою. Бінарні зв'язки бувають:

1:1 (один до одного); 1:M (один до багатьох); N:M (багато до багатьох).

На рис. 7.4, 7.5 показані відображення цих зв'язків у різних ER-моделях.

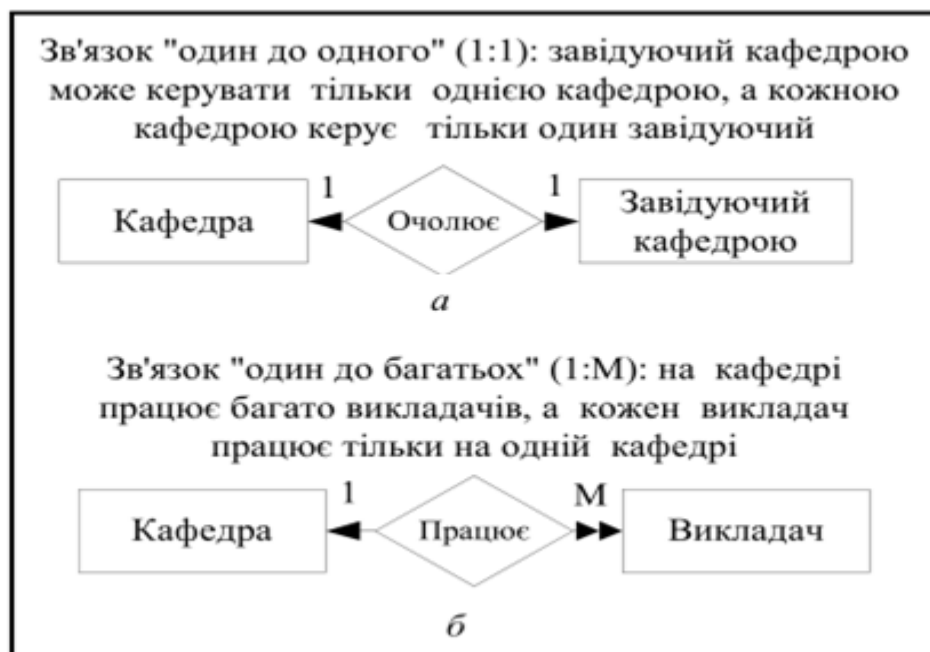


Рис. 7.4. Представлення зв'язків між відношеннями на діаграмі Чена



Рис. 7.4. Представлення зв'язків між відношеннями на діаграмі Чена (продовження)

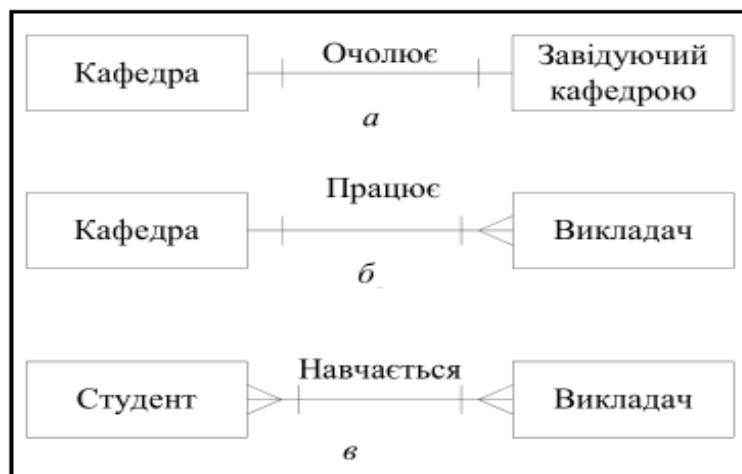


Рис. 7.5. Представлення зв'язків між відношеннями на діаграмі «пташина лапка»

7.3.3. Атрибути

Атрибути являють собою властивості сутності. Значення кожного атрибута вибирають з відповідної множини значень, яка включає всі потенційні значення, які можуть бути присвоєні атрибуту. Ця множина значень називається доменом.

Приклад. Атрибут *Оцінка* може приймати чотири значення: 2, 3, 4, 5. Ці значення і складають домен цього атрибута.

Атрибути залежно від складності значень поділяються на певні категорії (табл. 7.1).

Таблиця 7.1. Типи атрибутів

Тип	Властивість
Простий	Атрибут, який не може бути поділений на інші атрибути. Приклад. <i>Прізвище; посада</i>
Складовий	Атрибут, який може бути поділений на інші атрибути. Приклад. <i>Адреса; прізвище, ім'я, по батькові</i>
Однозначний	Атрибут, який може приймати тільки одне значення. Приклад. <i>Табельний номер, номер залікової книжки</i>
Багатозначний	Атрибут, який може приймати багато значень. Приклад. <i>Телефон, Адреса (постійне місце проживання)</i>
Похідний	Атрибут, який не зберігається в БД, а обчислюється за допомогою певного алгоритму Приклад. <i>Вік (обчислюється по даті народження), кількість студентів в групі</i>

Приклад. Розглянемо сутність *Студент* (рис. 7.6).

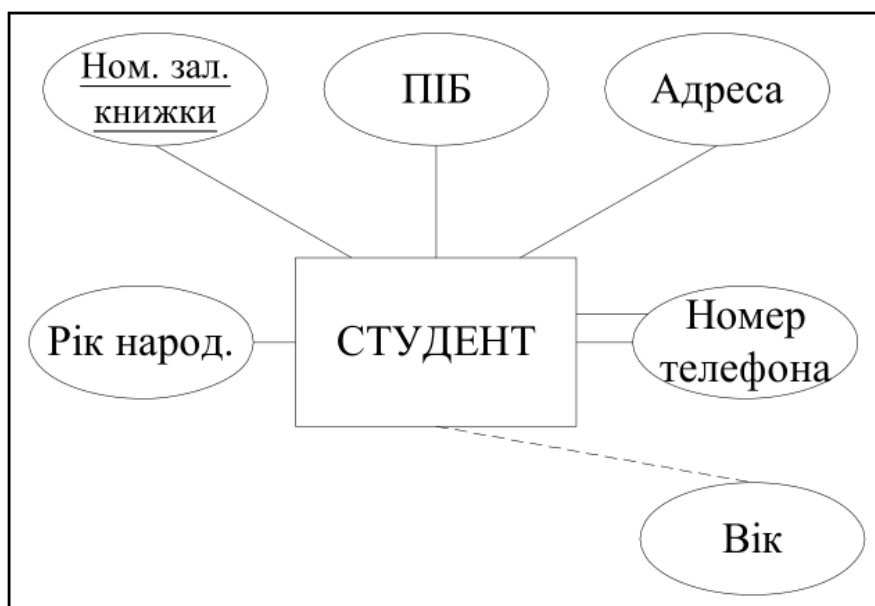


Рис. 7.6. ER-діаграма сутності *Студент*

Атрибути *Номер залікової книжки*, *Рік народження* є простими.

Атрибути *ПІБ* і *Адреса* є складовими. *ПІБ* може бути поділений на атрибути: *прізвище*, *ім'я*, *по батькові*, а *Адреса* – на *індекс*, *місто*, *вулиця*, *будівля*, *квартира*.

Атрибут *Вік* є похідним: він обчислюється за значенням атрибута *Рік народження* (зображується пунктирною лінією).

Атрибут *Номер залікової книжки* є однозначним: він не може приймати два значення для одного студента.

Атрибут *Номер телефону* є багатозначним: він може приймати декілька значень для одного студента (зображується подвійною лінією).

Атрибут або набір атрибутів сутності, які застосовуються для ідентифікації екземпляра сутності, називаються **потенційним ключем**. Сутність може містити декілька потенційних ключів. В нашому прикладі в якості потенційних ключів можуть бути такі атрибути: *Номер залікової книжки*, *Прізвище*, *Ім'я по Батькові*.

Потенційний ключ, який вибрано для однозначної ідентифікації кожного екземпляра сутності певного типу, називається **первинним ключем**. Первинний ключ вибирається за умови гарантії унікальності його значень, а також мінімальної довжини атрибутів, які входять в його склад. В прикладі в якості первинного ключа служить *Номер залікової книжки*.

7.3.4. Потужність зв'язків

Потужність зв'язку (кардинальність) відображає певне число екземплярів сутностей, які зв'язані з одним екземпляром зв'язаної сутності. В моделі Чена потужність зв'язку відображається вказівкою відповідних чисел поруч з сутностями у форматі (x, y). Перше число визначає мінімальне значення потужності зв'язку, а друге – його максимальне значення. Потужність вказує на число екземплярів у зв'язаній сутності.

Відомості про максимальне і мінімальне значення потужності зв'язку може застосовуватися у прикладному програмному забезпеченні або за допомогою тригерів. На рівні таблиць СКБД не може оперувати з потужністю зв'язків.

Приклад. Розглянемо зв'язок *Група-Студент* (рис. 7.7).

Потужність (1,30) поруч із сутністю *Група* вказує, що в групі може займатися від 1 до 30 студентів. Потужність (1,1) поруч із сутністю *Студент* вказує, що студент може займатися тільки в одній групі (мінімум 1, максимум 1). У моделі «пташина лапка» числовий діапазон значень потужності не відображається в ER-діаграмах.

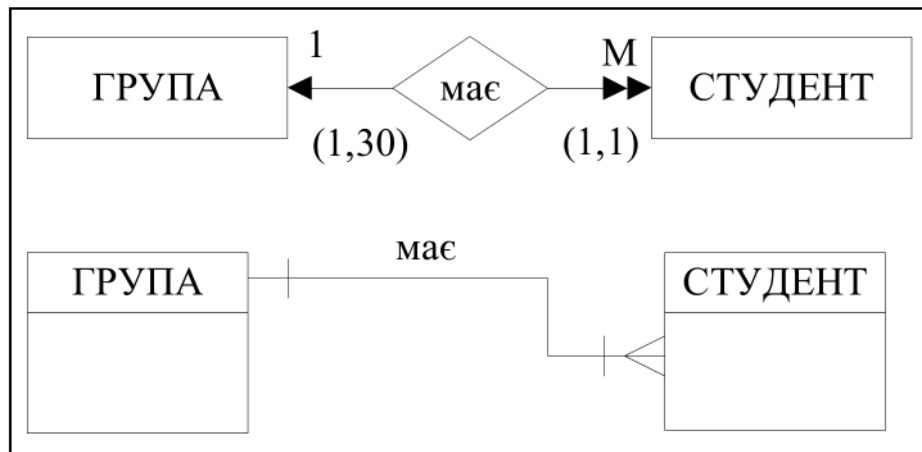


Рис. 7.7. Зв'язок і потужність в ER-діаграмах

7.3.5. Сильні і слабкі зв'язки

Якщо сутність може існувати незалежно від інших сутностей, то вона є *незалежною від існування*. Якщо сутність залежить від існування інших сутностей, то вона є *залежною від існування*. Наприклад, сутності **Студент** і **Група** можуть існувати незалежно одна від одної, а сутність **Нагорода студента** є залежною від сутності **Студент** й існувати без неї не може.

Якщо одна сутність незалежна від існування іншої сутності, то зв'язок між ними називається *не ідентифікаційним зв'язком* або *слабким зв'язком*. На ER-діаграмах «пташина лапка» слабкий зв'язок відображається штриховою лінією.

Ідентифікаційний зв'язок або **сильний зв'язок** має місце у тому випадку, коли одна зв'язана сутність залежить від існування іншої. На ER-діаграмах «пташина лапка» сильний зв'язок відображається суцільною лінією.

Приклад. Розглянемо зв'язки **Група-Студент** і **Студент-Нагорода** (рис. 7.8).

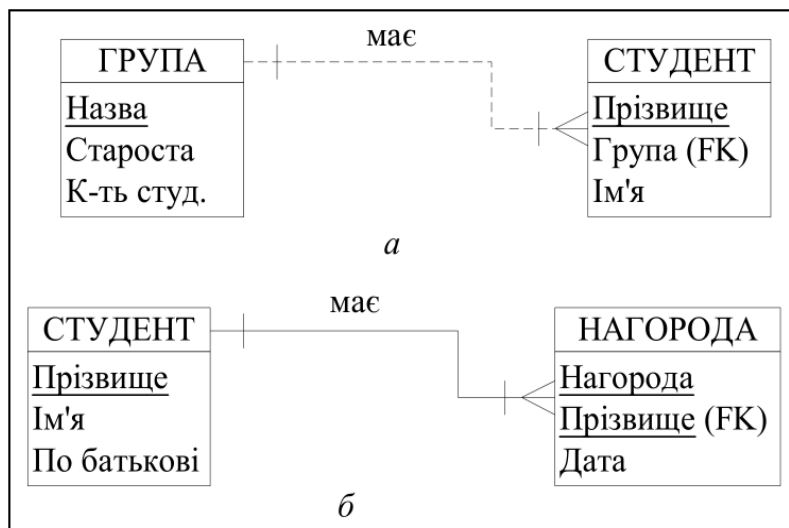


Рис. 7.8. Не ідентифікаційний (а) та ідентифікаційний (б) зв'язки

Сутність **Студент** не залежить від сутності **Група**, в цьому випадку зв'язок між сутностями відображається штриховою лінією, а атрибут *Назва групи* в сутності **Студент** є зовнішнім ключем (Foreign Key, FK).

Сутність **Нагорода** залежить від сутності **Студент**, в цьому випадку зв'язок між сутностями відображається суцільною лінією, а атрибут *Прізвище студента* в сутності **Нагорода** є частиною первинного ключа (Primary Key, PK) і одночасно зовнішнім ключем (FK).

7.3.6. Атрибути зв'язків. Обов'язкові і необов'язкові зв'язки

Так само як і сутності зв'язки можуть мати атрибути.

Приклад. Атрибути *День* і *Аудиторія* належать до зв'язку між сутностями *Студент* і *Дисципліна* (рис. 7.9).

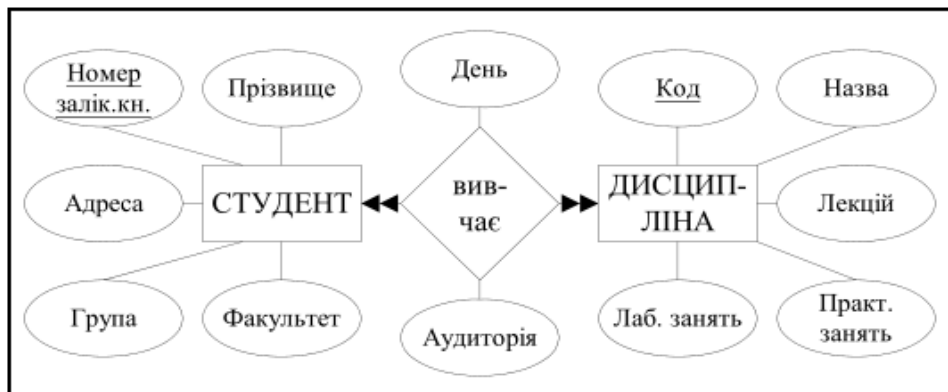


Рис. 7.9. ER-діаграма з атрибутами зв'язку *День* і *Аудиторія*

Участь сутності у зв'язку може бути обов'язковою або необов'язковою. Якщо один екземпляр сутності не потребує наявності відповідного екземпляра сутності в окремому зв'язку, то участь сутності у зв'язку є **необов'язковою**. Наприклад, у зв'язку сутностей *Студент-Нагорода* – не кожен студент має нагороди. Тобто не кожен екземпляр в таблиці *Студент* потребує обов'язкової наявності екземпляра сутності в таблиці *Нагорода*. Сутність *Нагорода* розглядається як необов'язкова по відношенню до сутності *Студент*. Необов'язкова сутність позначається невеликим колом з боку необов'язкової сутності. Існування необов'язковості вказує на те, що для необов'язкової сутності мінімальне значення потужності зв'язку дорівнює 0.

Участь сутності у зв'язку буде **обов'язковою**, якщо кожен екземпляр сутності обов'язково потребує відповідного екземпляра сутності в окремому зв'язку. При обов'язковому зв'язку для обов'язкової сутності мінімальна потужність зв'язку дорівнює 1.

Приклад. Зв'язок між сутностями *Студент* і *Нагорода* є необов'язковим (рис. 7.10). Необов'язково кожен студент має нагороду, але якщо є нагорода, то вона обов'язково зв'язана з певним студентом.

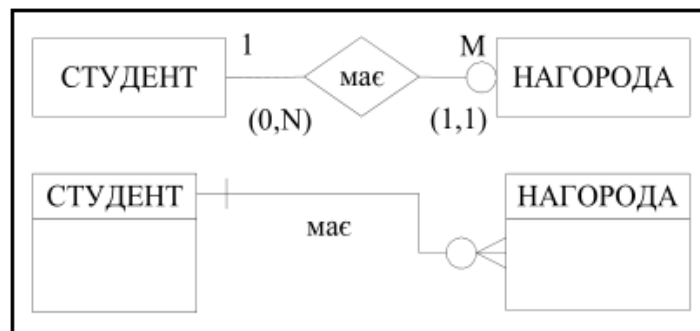


Рис. 7.10. Необов'язкова сутність *Нагорода* в ER-діаграмах П. Чена і «пташина лапка»

7.3.7. Слабкі сутності. Складні зв'язки

Слабкою сутністю називається сутність, яка задовольняє таким умовам:

- залежності від існування сутності з якою вона зв'язана;
- первинний ключ цієї сутності частково або повністю отриманий з іншої сутності.

Слабка сутність на діаграмі Чена позначається подвійним прямокутником, а на діаграмі «пташина лапка» невеликими сегментами в кожному з кутів прямокутника.

Приклад. Сутність *Нагорода* є слабкою по відношенню до сутності *Студент*: вона залежить від існування цієї сутності і в її первинний ключ входить первинний ключ сутності *Студент* (рис. 7.11).

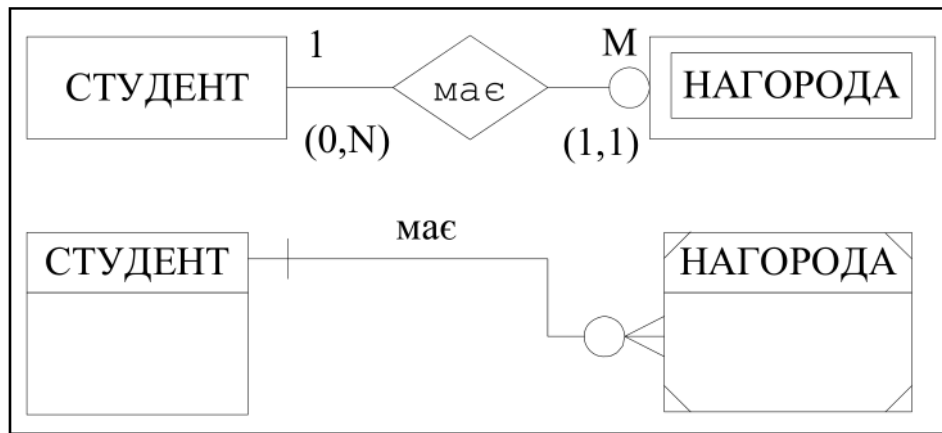


Рис. 7.11 Слабка сутність на діаграмах П. Чена і «пташина лапка»

Складні зв'язки. Використання зв'язків більш високого порядку дає можливість у багатьох випадках краще відобразити семантику проблемної області.

Приклад. Сутності *Викладач*, *Дисципліна* і *Екзамен* утворюють тернарний зв'язок (рис. 7.12).

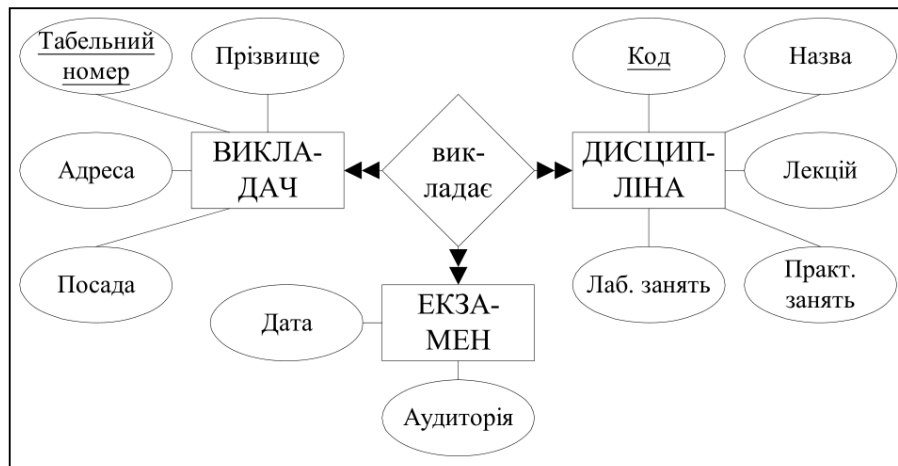


Рис. 7.12. Тернарний зв'язок між трьома сутностями на діаграмі П. Чена

7.3.8. Рекурсивні зв'язки

Рекурсивний зв'язок має місце, коли є зв'язок між екземплярами одного і того ж набору сутностей.

Приклад. Розглянемо можливі варіанти рекурсивних зв'язків (рис. 7.13).

Зв'язок 1:1 представляє висловлювання: «викладач може бути одружений тільки з одним співробітником». Зв'язок 1:M представляє таке висловлювання: «викладач, якщо він є завідуючим кафедрою, керує декількома викладачами, а викладачі мають тільки одного керівника – завідуючого кафедрою». Зв'язок M:N представляє висловлювання: «адміністратор має декілька підлеглих-адміністраторів, і в свою чергу адміністратор має декілька керівників-адміністраторів».

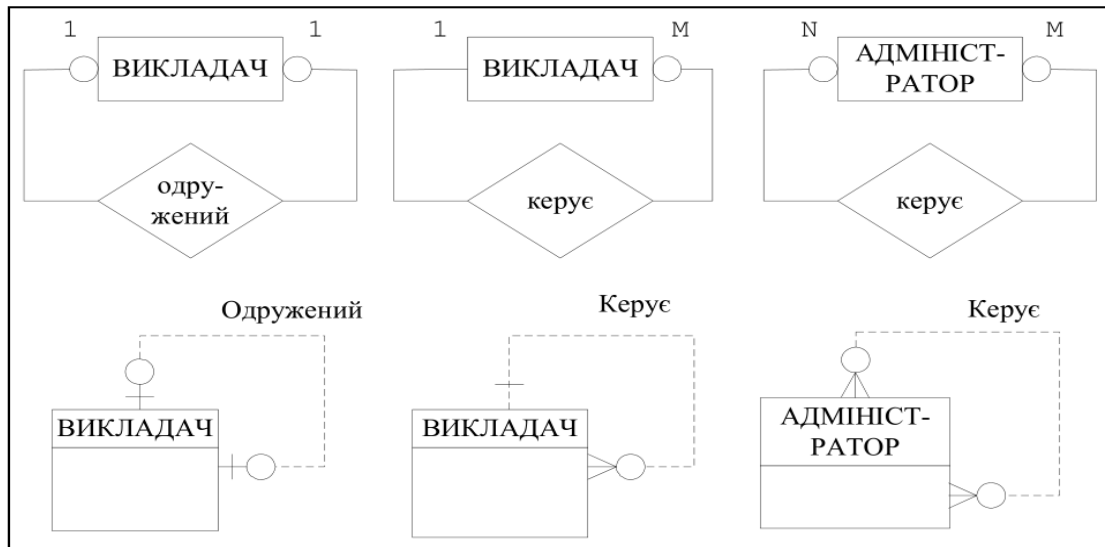


Рис. 7.13. Представлення рекурсивних зв'язків на діаграмі П. Чена і моделі «пташина лапка»

Між двома сутностями може бути декілька зв'язків зрізними змістовними навантаженнями.

Приклад. Між сутностями *Викладач* і *Студент* можна встановити такі змістовні зв'язки: *Викладає* і *Керівництво дипломним проектуванням*. Викладач викладає для багатьох студентів, для кожного студента викладає багато викладачів. Кожен студент обов'язково повинен мати одного керівника дипломного проекту, але необов'язково кожен викладач керує дипломниками (рис. 7.14).



Рис. 7.14. Різні зв'язки між двома сутностями

7.4. Розширена модель «сутність – зв'язок»

Для задоволення нових потреб, що висуваються більш складними застосуваннями, в семантичне моделювання були введені додаткові концепції, що розширюють його можливості. Така модель має назву *розширеної ER-моделі* (Enhanced Entity Relationship, EER-модель). Вона включає всі концепції ER-моделі плюс концепції *уточнення*, *узагальнення*, *агрегування* і *композиції*. Додаткові концепції базуються на таких поняттях, як *суперклас* і *підклас*. Суперклас може мати декілька підкласів.

Наприклад, підкласи *Викладач*, *Керівник*, *Лаборант* є членами суперкласу *Співробітник*. Це означає, що кожен екземпляр підкласу є в той же час і екземпляром суперкласу. Зв'язок між суперкласом і підкласом відноситься до типу 1:1.

Використання понять суперклас і підклас дозволяє визначити для підкласів власні атрибути і атрибути, що наслідуються від суперкласу. Так, наприклад, підклас *Викладач* повинен мати ті ж атрибути, що і всі *Співробітники*. Однак він має і свої власні атрибути, які не визначені для інших категорій працівників університету. До цих атрибутів можна віднести *вчене звання*, *номер диплому про вчене звання*, *кількість навчально-методичних праць* тощо. При відсутності підкласів для об'єкту *Співробітник* слід було б вводити атрибути, які б мали невизначене значення для

інших співробітників (наприклад для лаборантів). Підклас може мати свої власні зв'язки, які не підходять для всіх екземплярів суперкласу.

Наприклад, **Викладач** може мати підкласи **Професор**, **Доцент**, **Асистент**. Підклас наслідую не тільки атрибути, але і всі зв'язки суперкласу.

Уточнення це процес збільшення різниці між окремими екземплярами об'єкта за рахунок визначення їхніх відмінних характеристик. Цей процес є низхідним. Наприклад, перехід від об'єкта **Співробітник** до об'єктів **Викладач** і **Керівник**.

Узагальнення це процес зведення відмінностей між об'єктами до мінімуму шляхом виділення їх спільних характеристик. Цей процес є висхідним. Наприклад, перехід від об'єктів **Викладач** і **Керівник** до об'єкта **Співробітник**.

У процесі проведення уточнення або узагальнення можуть застосовуватися обмеження:

- ступеня участі;
- неперетинання.

Підкласи набору сутностей можуть перетинатися і не перетинатися. Якщо підкласи суперкласу *не перетинаються*, то це означає, що кожен екземпляр сутності може бути елементом тільки одного з підкласів (позначається *Or*). Зв'язки, які не перетинаються позначаються символом «G». Наприклад, співробітник може працювати або на посаді доцента, або на посаді професора, і не може бути одночасно і професором, і доцентом.

Якщо підкласи суперкласу *перетинаються*, то це означає, що будь-який екземпляр сутності може бути елементом декількох з підкласів (позначається *And*). Зв'язки, які перетинаються позначаються символом «Gs». Наприклад, завідуючий кафедрою проводить заняття, і одночасно виконує обов'язки викладача і керівника. На рис. 7.15 показана ієрархія сутностей з підкласами, що перетинаються і не перетинаються.

Приклад. Відношення суперклас – підклас.

Не кожен елемент суперкласу повинен бути елементом одного з підкласів.

Зв'язок суперкласу з підкласом з *обов'язковою участю*, вказує на те, що кожен елемент суперкласу повинен бути також елементом підкласу.

Приклад. Студент обов'язково займається або на денній, або на заочній, або на вечірній формі навчання, або екстернатом (рис. 7.16).

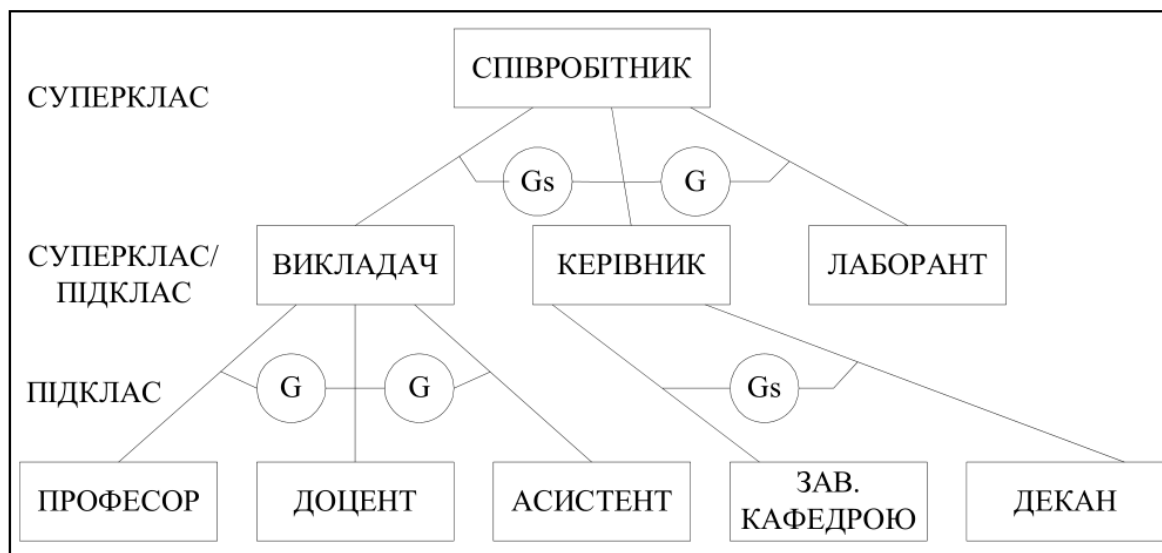


Рис. 7.15. Діаграма з використанням понять суперклас і підклас

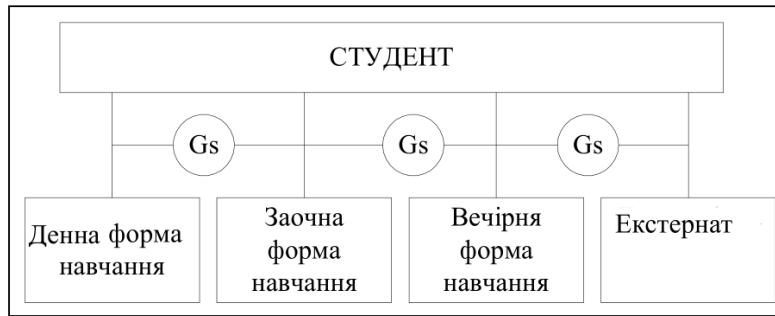


Рис. 7.16. Діаграма зв'язку суперкласу з підкласом з обов'язковою участю

Зв'язок суперкласу з підкласом з *необов'язковою участю*, вказує на те, що деякі елементи суперкласу можуть не належати жодному з підкласів. Наприклад, можуть бути викладачі, які не займають посади професора, доцента або асистента (наприклад старший викладач).

Введення понять суперкласів і підкласів дозволяє уникнути опису різних екземплярів сутності, які можуть мати різні атрибути. Це може привести до появи великої кількості незаповнених атрибутів. До того ж індивідуальні атрибути можуть показати зв'язки, які притаманні тільки для конкретних екземплярів, а не всім екземплярам сутностей.

Приклад. Екземпляри сутності **Викладач**, які належать до викладачів на посаді професора, у порівнянні з викладачами на посаді асистента, мають такі додаткові атрибути: вчене звання, вчений ступінь тощо. Крім того, вони можуть бути зв'язані індивідуальними атрибутами із сутністю **Аспірант**.

Один підклас може бути зв'язаний з декількома суперкласами, які в свою чергу є підкласами одного спільного для них суперкласу.

Приклад. Підкласи **Викладач** і **Керівник** наслідують суперклас **Співробітник**. Підклас **Завідуючий кафедрою** є одночасно екземпляром суперкласу **Викладач** і суперкласу **Керівник** (рис. 7.17).

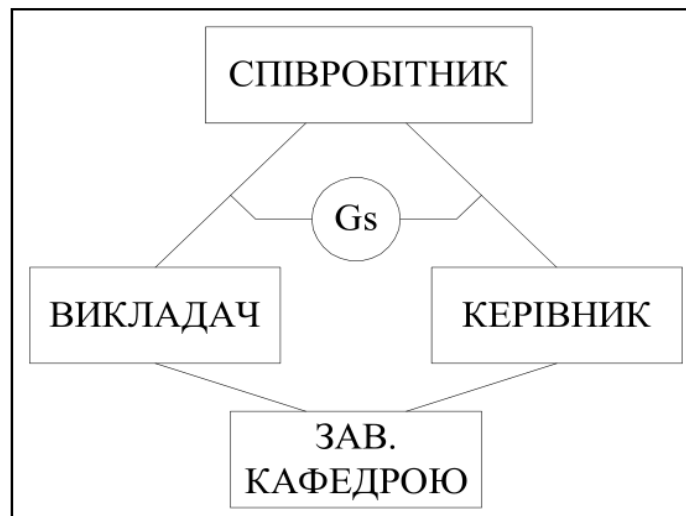


Рис. 7.17. Діаграма з підкласом, що сумісно використовується

Підклас є *категорією*, якщо він зв'язаний одразу з декількома суперкласами різних типів. Категорія може бути з повною участю і з частковою участю екземплярів. Повна участь передбачає, що кожен екземпляр всіх суперкласів повинен бути представлений у даній категорії. При частковій участі присутність в категорії всіх екземплярів всіх суперкласів необов'язкова.

Введення понять суперкласів і підкласів дозволяє ввести в проект більший обсяг семантичної інформації. Наприклад, твердження «Асистент є викладачем» дозволяє встановити певні ієрархічні відношення між об'єктами **Асистент** і **Викладач**. EER-діаграми повинні використовуватися, якщо структура даних є занадто складною, для того щоби її можна було легко представити з використанням ER-діаграм.

Для моделювання інших видів зв'язків вводиться поняття агрегування і композиції.

Агрегування являє собою зв'язок типу «входить в склад» або «включає» між двома сутностями, одна з яких представляє «ціле», а інша – «частину».

Композиція – це особлива форма агрегування, яка представляє залежність між сутностями, що характеризуються повною приналежністю і співпаданням термінів існування між «цілим» і «частиною».

7.5. Проблеми побудови моделей «Сутність – Зв'язок»

При недостатньому розумінні суті встановлених зв'язків може бути створена модель, яка не буде повною мірою відображати зв'язки між реальними об'єктами. Визначають *дефекти з'єднання*, які виникають при невірній інтерпретації змісту деяких зв'язків: дефекти розгалуження і дефекти розриву.

Дефекти розгалуження мають місце, коли модель вірно відображає зв'язки між сутностями, але шлях між окремими сутностями визначений неоднозначно. Цей дефект виникає в тому випадку, коли два або більше зв'язків типу 1:М виходять з однієї сутності.

Приклад. Розглянемо такі зв'язки: на факультеті займається багато студентів, у склад факультету входить багато груп (рис. 7.18). Ці зв'язки вірно відображають зміст предметної області, але при спробі з'ясувати, в яких групах займаються конкретні студенти, виникають проблеми. Із сутності **Факультет** виходять два зв'язки 1:М.

Усунути цю проблему можна шляхом перебудови моделі для представлення вірної взаємодії цих сутностей (рис. 7.19).

Отже, тепер відповідь на попереднє питання не є проблемою.

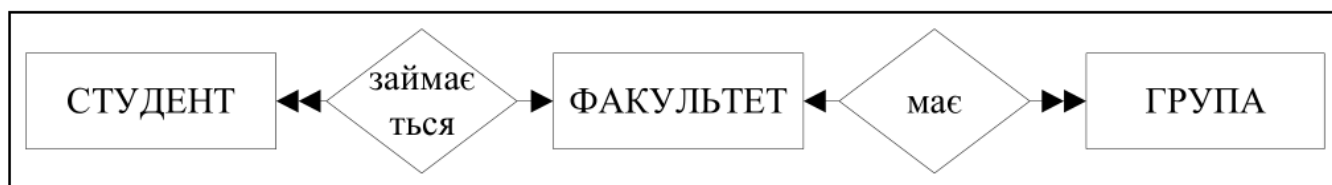


Рис. 7.18. Приклад дефекту розгалуження в ER-моделі



Рис. 7.19. Усунення дефекту розгалуження в прикладі ER-моделі

Дефекти розриву виникають у тому випадку, коли в моделі передбачається наявність зв'язку між декількома сутностями. Цей дефект виникає у разі, коли існує один або декілька зв'язків з мінімальною потужністю рівною 0, яка визначає необов'язкову участь, і ці зв'язки складають частину шляху між взаємозв'язаними сутностями.

Приклад. Розглянемо такі зв'язки: в склад факультету входить багато кафедр, кожна кафедра може відповідати за декілька комп'ютерних класів (від 0 до М) (рис. 7.20). Тобто, деякі кафедри можуть не бути відповідальними за комп'ютерний клас. У свою чергу комп'ютерний клас може підпорядковуватися певній кафедрі, а може підпорядковуватися безпосередньо факультету (факультетський комп'ютерний клас). Ці зв'язки вірно відображають зміст предметної області, але при спробі з'ясувати, які комп'ютерні класи підпорядковані певному факультету, виникають проблеми. Зв'язок між сутностями **Кафедра** і **Комп'ютерний клас** передбачає необов'язкову участь сутностей, і він є частиною шляху між сутностями **Факультет** і **Кафедра**.

Усунути цю проблему можна шляхом введення додаткового зв'язку між сутностями **Факультет** і **Комп'ютерний клас** (рис. 7.21).



Рис. 7.20. Приклад дефекту розриву в ER-моделі

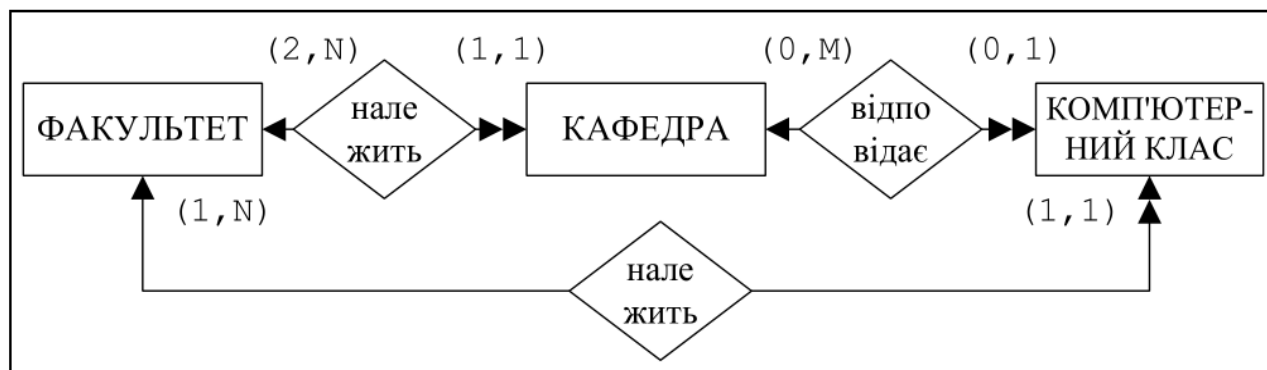


Рис. 7.21. Усунення дефекту розриву в прикладі ER-моделі

7.6. Приклад побудови моделі «Сутність – Зв'язок»

Процес концептуального проектування БД є ітеративний й заснований на операціях і процедурах, що повторюються. Спочатку створюється базова ER-модель певної предметної області. При дослідженні цієї моделі як правило з'являються додаткові сутності, атрибути і зв'язки. Після цього ER-модель буде змінюватися. Процес змін повторюється до тих пір, поки концептуальна модель не буде відображати предметну область.

Розробники БД на основі опитування фахівців визначають сутності, атрибути, зв'язки у предметній області, що розглядається, дослідженні документів, які застосовуються в роботі на підприємстві.

Розглянемо приклад створення ER-моделі предметної області ЗАКЛАД ВИЩОЇ ОСВІТИ (ЗВО).

Задачі інформаційної системи. Мета створення бази даних полягає у такому:

- забезпечення адміністрації університету довідковою інформацією по факультетах, кафедрах, спеціальностях, викладачах, студентах і дисциплінах, що викладаються у ЗВО;
- контроль успішності студентів.

Аналіз предметної області. У результаті дослідження й аналізу предметної області були визначені такі сутності, атрибути і первинні ключі (табл. 5.2).

Таблиця 7.2. Сутності, атрибути і первинні ключі предметної області ВНЗ

Сутність	Атрибути	Первинний ключ
ФАКУЛЬТЕТ	Код, Назва, Декан	Код факультету
СПЕЦІАЛЬНІСТЬ	Код, Назва, Вимоги	Код спеціальності
ГРУПА	Код, Назва, Кількість студентів, Староста	Код групи
СТУДЕНТ	Номер залікової книжки, Прізвище, Адреса, Рік народження	Номер залікової книжки
КАФЕДРА	Код, Назва, Завідуючий кафедрою, Кількість викладачів	Код кафедри
ВИКЛАДАЧ	Табельний номер, Прізвище, Посада Науковий ступінь	Табельний номер викладача
ДИСЦИПЛІНА	Код, Назва, Кількість годин, Семестр	Код дисципліни

Зв'язки сутностей визначаються на основі бізнес-правил, які побудовані з урахуванням організаційної структури та операцій, що виконуються в системі:

- в університеті існує декілька факультетів;
- на факультеті навчаються групи студентів за певними спеціальностями;
- у склад групи входять студенти;
- факультет об'єднує декілька кафедр;
- на кожній кафедрі працює декілька викладачів;
- на кожній спеціальності викладається ряд дисциплін, які проводять викладачі з різних кафедр;
- з кожної дисципліни своєї спеціальності студенти складають іспит або залік;
- не кожен викладач читає дисципліни (наприклад, асистент) і не з кожної дисципліни є викладач(наприклад, нова дисципліна, по якій викладача ще не призначено).

Для спрощення концептуальної моделі бази даних цілий ряд об'єктів і бізнес-правил ЗВО залишився нерозглянутим.

Дослідження предметної області виявило, що всі сутності є сильними, а зв'язки між сутностями – не ідентифікуючими. Зв'язок між сутностями *Студент* і *Дисципліна* має атрибут *Оцінка*.

Побудова ER-діаграми. На основі задач, які були поставлені перед інформаційною системою, і на основі аналізу предметної області, побудована ER-діаграма ЗВО (рис. 7.22).

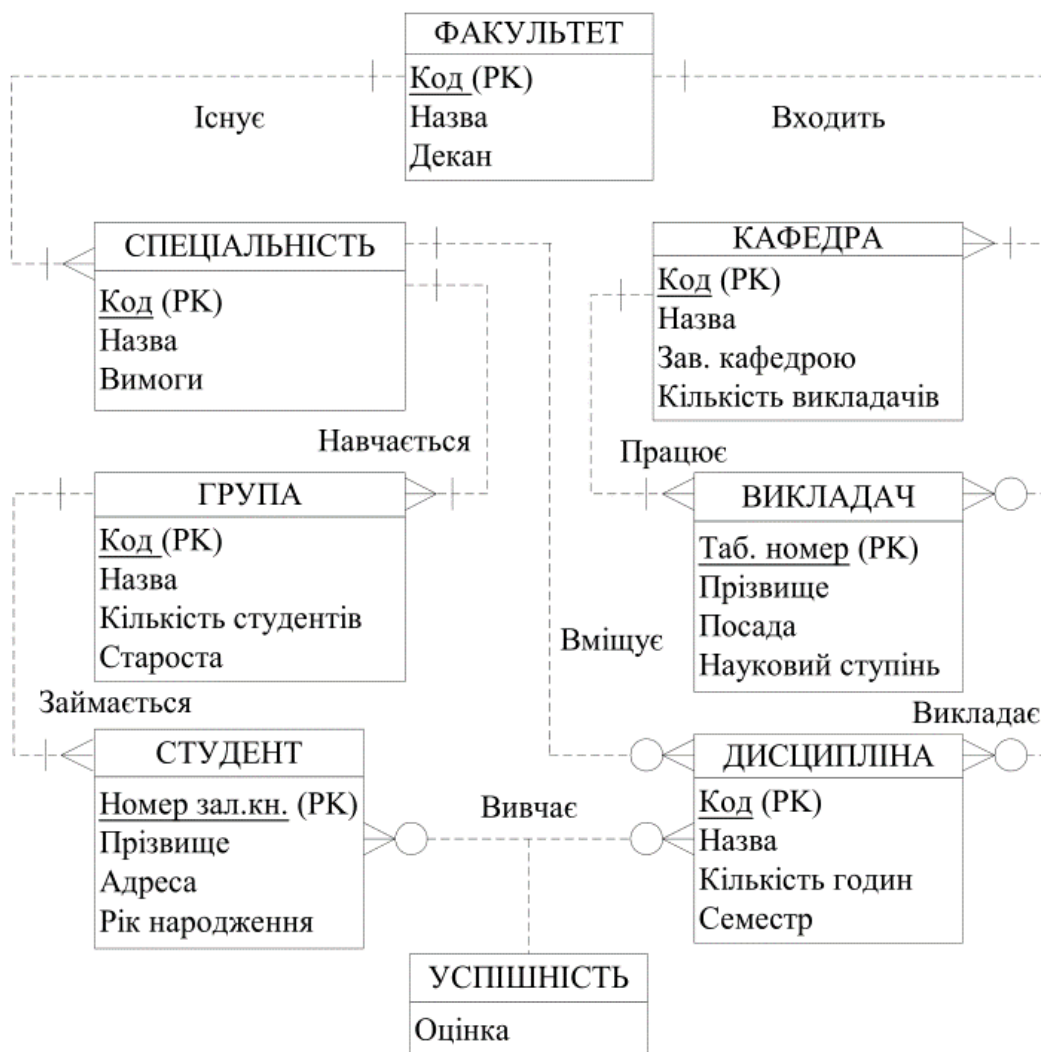


Рис. 7.22. ER-діаграма предметної області ЗАКЛАД ВИЩОЇ ОСВІТИ

Слід звернути увагу на те, що розроблений концептуальний проект не є єдиним проектом, який відповідає поставленій задачі. Можливі варіанти розробки системи із застосуванням інших зв'язків між сутностями, або із застосуванням розширеної ER-моделі.

Застосування ER-діаграм дозволяє забезпечити просте і наглядне уявлення про головні логічні об'єкти БД і про зв'язки, які між цими об'єктами існують. Також до переваг ER-діаграми слід віднести те, що вони добре інтегрують з реляційною моделлю.

Недоліком ER-моделей є те, що вони мають недостатні можливості для представлення відношень і обмежень, можуть бути складні при наявності багатьох об'єктів, не мають засобів для опису операцій маніпулювання даними.

7.7. Семантична модель «Сутність-Зв'язок» Баркера

7.7.1. Нотація моделі «Сутність-Зв'язок» Баркера

Серед безлічі різновидів ER-моделей одна з найбільш популярних і розвинених застосовувалася в системі CASE компанії Oracle, яка базується на нотації Баркера. Кожна сутність в моделі зображується у вигляді прямокутника, що містить ім'я сутності (рис. 7.23):

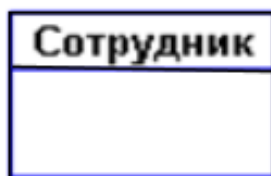


Рис. 7.23. Графічне зображення сутності

Кожен атрибут забезпечується ім'ям, унікальним в межах суті. Найменування атрибута повинно бути виражено іменником в однині (можливо, з характеризують прикметниками).

Прикладами атрибутів сутності «Співробітник» можуть бути такі атрибути як «Табельний номер», «Прізвище», «Ім'я», «По батькові», «Посада», «Зарплата» і т.п.

Атрибути зображуються у межах прямокутника, що визначає сутність (рис. 7.24):



Рис. 7.24. Атрибути сутності

Ключ сутності – мінімальний набір атрибутів, за значеннями яких можна однозначно знайти необхідний екземпляр сутності. Мінімальність означає, що виключення з набору будь-якого атрибута не дозволяє ідентифікувати сутність, що залишилися.

Наприклад, для сутності **Розклад** ключем є атрибут **НОМЕР_РЕЙСА** або набір: **ПУНКТ_ВІДПРАВЛЕННЯ**, **ЧАС_ВИЛЕТУ** і **ПУНКТ_ПРИЗНАЧЕННЯ** (за умови, що з пункту в пункт вилітає в кожен момент часу один літак). Сутність може мати кілька різних ключів. Ключові атрибути зображуються на діаграмі підкресленням (рис. 7.25):



Рис. 7.25. Сутність з ключовим атрибутом

Між сутностями можуть бути встановлені зв'язки, що показують, яким чином сутності співвідносяться або взаємодіють між собою. Зв'язок може існувати між двома різними сутностями або між сутністю і їй же самій (рекурсивна зв'язок). Вона показує, як пов'язані екземпляри сутностей між собою. Якщо зв'язок встановлюється між двома сутностями, то вона визначає взаємозв'язок між екземплярами однієї й іншої сутності. Зв'язки дозволяють по одній сутності знаходити інші сутності, пов'язані з нею.

Рекурсивний зв'язок з'являється в тому випадку, коли одна і та ж сутність виступає неодноразово в різних ролях. Рекурсивний зв'язок також іноді називається одиничним зв'язком. На наступному прикладі зображений рекурсивний зв'язок, що зв'язує сутність МУЖЧИНА з нею ж самою. Кінець зв'язку з ім'ям «син» визначає той факт, що кілька людей можуть бути синами одного батька. Кінець зв'язку з ім'ям «батько» означає, що не у кожного чоловіка повинні бути сини (рис. 7.26).

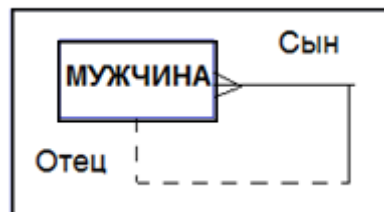


Рис. 7.26. Приклад рекурсивного типу зв'язку

Якби призначенням бази даних було тільки збереження окремих, не пов'язаних між собою даних, то її структура могла б бути дуже простою. Проте одна з основних вимог до організації бази даних – це забезпечення можливості відшукування одних сутностей за значеннями інших, для чого необхідно встановити між ними певні зв'язки. А так як в реальних базах даних нерідко містяться сотні або навіть тисячі сутностей, то теоретично між ними може бути встановлено більше мільйона зв'язків. Наявність такої великої кількості зв'язків і визначає складність інфологічних моделей.

На рис. 7.27 діаграма включає два пов'язаних типу сутності. Наприклад, зв'язки між сутностями можуть виражатися наступними фразами – «СПІВРОБІТНИК може мати кілька ДІТЕЙ», «кожен СПІВРОБІТНИК зобов'язаний числитися рівно в одному ВІДДІЛІ».

Графічно зв'язок зображується лінією, що з'єднує дві сутності.



Рис. 7.27. Приклад рекурсивного типу зв'язку

Кожна зв'язок має два кінця і одне або два найменування. Найменування зазвичай виражається в невизначеному дієслівній формі: «мати», «належати» і т.п. Кожне з найменувань ставиться до свого кінця зв'язку. Іноді найменування не пишуть зважаючи на їх очевидності. Кожна зв'язок може мати один з наступних типів зв'язку (рис. 7.28).

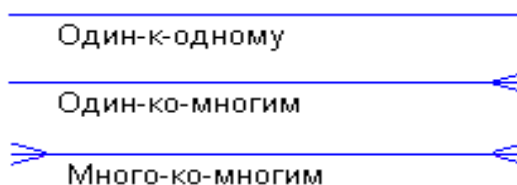


Рис. 7.28. Типи зв'язків між сутностями

Зв'язок типу один-до-одного (1:1) означає, що один екземпляр першої суті (лівою) пов'язаний з одним екземпляром другої суті (правою). Зв'язок один-до-одного найчастіше свідчить про те, що насправді ми маємо всього одну сутність, неправильно (або з якоюсь метою) розділену на дві.

Зв'язок типу один-до-багатьох (1: M), $\Leftarrow$ – «вороняча лапка») означає, що один екземпляр першої суті (лівою) пов'язаний декількома екземплярами другої сутності (правої). Це найбільш часто використовуваний тип зв'язку. Ліва сутність (з боку «один») називається батьківською, права (з боку «багато») – дочірньою.

Зв'язок типу багато-до-багатьох (M: M) означає, що кожен екземпляр першої сутності може бути пов'язаний з декількома екземплярами другої сутності, і кожен примірник другої сутності може бути пов'язаний з декількома екземплярами першої сутності. Тип зв'язку багато-до-багатьох є тимчасовим типом зв'язку, допустимим на ранніх етапах розробки моделі. Надалі цей тип зв'язку повинен бути замінений двома зв'язками типу один-до-багатьох шляхом створення проміжної сутності.

Кожен зв'язок може мати одну з двох модальностей зв'язку (рис. 7.29):

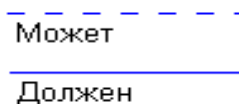


Рис. 7.29. Зображення модальності зв'язків між сутностями

Модальність «може» означає, що екземпляр однієї сутності може бути пов'язаний з одним або декількома екземплярами іншої сутності, а може бути і не пов'язаний ні з одним екземпляром.

Модальність «повинен» означає, що екземпляр однієї сутності може бути пов'язаний не менше ніж з одним екземпляром іншої сутності.

Зв'язок може мати різну модальність з різних кінців.

Описаний графічний синтаксис дозволяє однозначно читати діаграми, користуючись наступною схемою побудови фраз:

<Кожен екземпляр СУТІ 1> <модальності ЗВ'ЯЗКУ> <НАЗВА ЗВ'ЯЗКУ> <ТИП ЗВ'ЯЗКУ> <екземпляр СУТІ 2>.

Кожен зв'язок може бути прочитана як зліва направо, так і справа наліво. Наприклад, зв'язок, представлена на малюнку 7.5, читається так:

Зліва направо: «кожен співробітник може мати кілька дітей».

Справа наліво: «Кожна дитина має належати рівно одному співробітнику».

Нормальні форми ER-схем. Як і в реляційних схемах баз даних, в ER-діаграмах вводиться поняття нормальних форм, причому їх зміст дуже близько відповідає змісту реляційних нормальних форм. Наведемо лише дуже короткі і неформальні визначення трьох перших нормальних форм.

У першій нормальній формі ER-діаграми усуваються повторювані атрибути або групи атрибутів, тобто проводиться виявлення неявних сутностей, «замаскованих» під атрибути.

У другій нормальній формі усуваються атрибути, які залежать тільки від частини унікального ідентифікатора (ключа сутності). Ця частина унікального ідентифікатора визначає окрему сутність.

У третій нормальній формі усуваються атрибути, які залежать від атрибутів, що не входять в унікальний ідентифікатор (ключ сутності). Ці атрибути є основою окремої сутності.

При правильному визначенні сутностей, отримані таблиці будуть одночасно перебувати в 3НФ. Основна перевага методу полягає в тому, модель будується методом послідовних уточнень первинних діаграм.

ER-діаграма повинна охоплювати всі функціональні вимоги бази даних для того, щоб стати справжньою моделлю. У наведеному нижче прикладі відношення між сутностями будуть виглядати досить просто, але в реальності можуть зустрічатися і набагато більш складні відношення.

7.7.2. Приклад розробки простої ER-моделі в нотації Баркера

При розробці ER-моделей необхідно обстежити предметну область (підприємство, підприємство) і виявити:

1. Сутності предметної області, про яких зберігаються дані в організації (підприємстві), наприклад, люди, місця, ідеї, події тощо (будуть представлені у вигляді блоків).
2. Властивості цих сутностей (будуть представлені у вигляді імен атрибутів в цих блоках).
3. Зв'язки між цими сутностями (будуть представлені у вигляді ліній, що з'єднують ці блоки).

ER-діаграми зручні тим, що процес виділення сутностей, атрибутів і зв'язків є ітераційним. Розробивши перший наближений варіант діаграм, ми уточнюємо їх, опитуючи експертів предметної області. При цьому документацією, в якій фіксуються результати бесід, є самі ER-діаграми.

Припустимо, що перед нами стоїть завдання розробити інформаційну систему на замовлення певної оптової торгової фірми. В першу чергу ми повинні вивчити предметну область і процеси, що відбуваються в ній. Для цього ми опитуємо співробітників фірми, читаємо документацію, вивчаємо форми замовлень, накладних і т.п.

Наприклад, в ході бесіди з менеджером з продажу, з'ясувалося, що він (менеджер) вважає, що проектована система повинна виконувати наступні дії:

1. Зберігати інформацію про покупців.
2. Друкувати накладні на відпущені товари.
3. Стежити за наявністю товарів на складі.

Виділимо всі іменники в цих пропозиціях – це будуть потенційні кандидати на сутності й атрибути, і проаналізуємо їх (незрозумілі терміни будемо виділяти знаком питання):

1. **Покупець** – явний кандидат на сутність.
2. **Накладна** – явний кандидат на сутність.
3. **Товар** – явний кандидат на сутність.
4. (?) *Склад* – а взагалі, скільки складів має фірма? Якщо кілька, то це буде кандидатом на нову сутність.
5. (?) *Наявність товару* – це, швидше за все, атрибут, але атрибут будь сутності?

Відразу виникає очевидний зв'язок між сутностями – «покупці можуть купувати багато товарів» і «товари можуть продаватися багатьом покупцям». Перший варіант діаграми виглядає так, як на рис. 7.30.

Хоча зв'язок «Багато до багатьох» (M:N) легко зобразити на діаграмі, її далеко не просто реалізувати фізично, адже реляційні бази підтримують тільки відношення «один-до-багатьох». Пізніше, обговорюючи процес нормалізації бази даних, ми дуже детально розглянемо спосіб вирішення подібних завдань на фізичному рівні. А зараз, поки ми перебуваємо на концептуальному рівні моделювання, лише запам'ятаємо, що в тих випадках, коли між двома типами сутностей виникає зв'язок «багато-до-багатьох», розробник БД створює штучний тип сутності, що виконує функції комутатора між основними сутностями (рис. 7.31).



Рис. 7.30. Зв'язок «багато до багатьох» між сутностями

Додатковий тип сполучною сутності розриває зв'язок «M:N» навпіл, що дозволяє трансформувати для реляційних баз даних відношення «багато-до-багатьох» в пару звичайних зв'язків «один-до-багатьох». Штучно створена сутність-комутатор-якому випадку буде містити два атрибути зовнішніх ключа. Сутності-комутатори призначені для підтримки асоціації між відношенням, які об'єднуються «багато-до-багатьох» типами сутностей, один атрибут комутатора стане тримати зв'язок з розташованим на діаграмі зліва типом «Buer» (Покупець), а інший – з правим типом сутності «Product» (Товар).

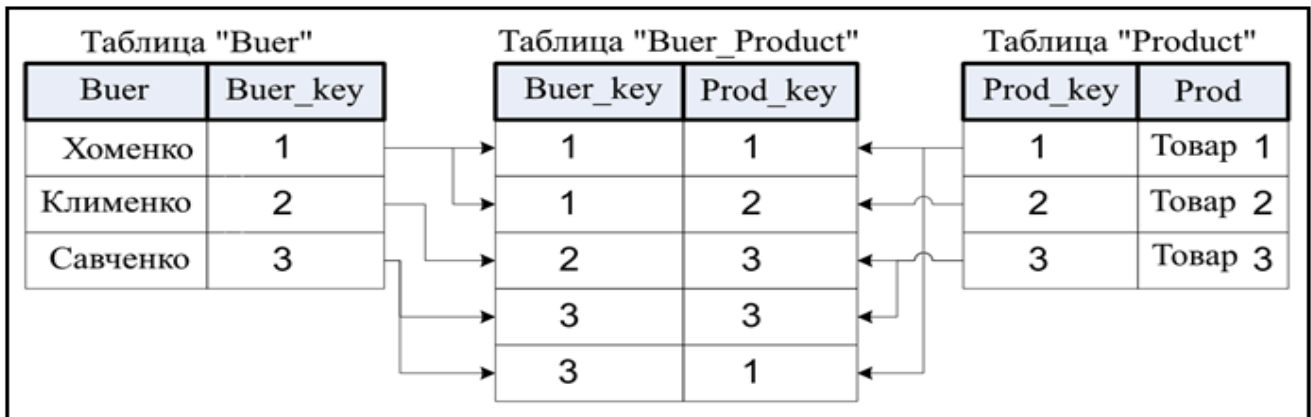


Рис. 7.31. Перетворення відношення «M:N» до двох відношень «1:M»

Задавши додаткові питання менеджеру, ми з'ясували, що фірма має кілька складів. Причому, кожен товар може зберігатися на декількох складах і бути проданим з будь-якого складу. Куди помістити сутності «Накладна» і «Склад» і з чим їх пов'язати? Запитаємо себе, як пов'язані ці сутності між собою і з сутностями «Покупець» і «Товар»?

1. Покупці купують товари, отримуючи при цьому накладні, в які внесені дані про кількість і ціну купленого товару.
2. Кожен покупець може отримати кілька накладних.
3. Кожна накладна зобов'язана виписуватися на одного покупця.
4. Кожна накладна повинна містити кілька товарів (не буває порожніх накладних). Кожен товар, в свою чергу, може бути проданий декільком покупцям через кілька накладних.
5. Крім того, кожна накладна повинна бути виписана з певного складу, і з будь-якого складу може бути виписано багато накладних.

Після уточнення, діаграма буде виглядати наступним чином (рис. 7.32).

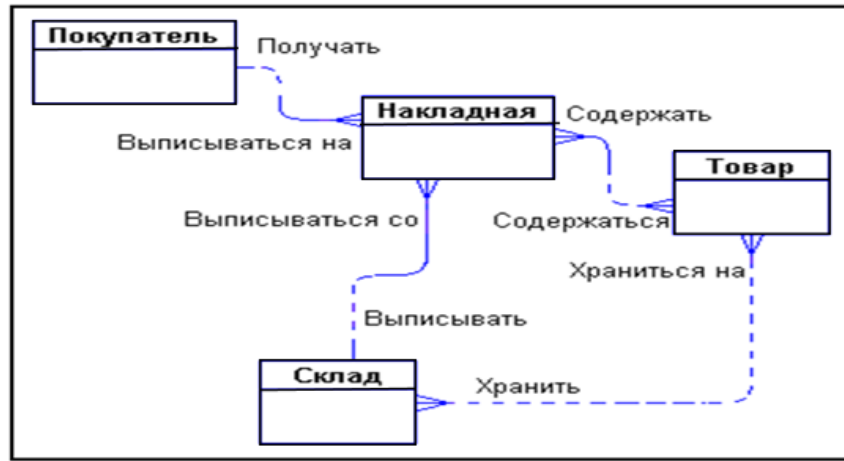


Рис. 7.32. Зв'язок «багато-до-багатьох» між сутностями

Тепер визначимо атрибути сутностей. Розмовляючи з співробітниками фірми, ми з'ясували наступне:

1. Кожен покупець має найменування, адреса, банківські реквізити.
2. Кожен товар має найменування, ціну, а також характеризується одиницями виміру.
3. Кожна накладна виписується з певного складу і має унікальний номер, дату виписки, список товарів з кількостями і цінами, а також загальну суму накладної.
4. Кожен склад має своє найменування.

Знову випишемо всі іменники, які будуть потенційними атрибутами, і проаналізуємо їх:

1. *Юридична особа* – термін риторичний, ми не працюємо з фізичними особами. Чи не звертаємо уваги.
2. *Найменування покупця* – явна характеристика покупця.
3. *Адреса* – явна характеристика покупця.
4. *Банківські реквізити* – явна характеристика покупця.
5. *Найменування товару* – явна характеристика товару.
6. (?) *Ціна товару* – схоже, що це характеристика товару. Чи відрізняється ця характеристика від ціни в накладній?
7. *Одиниця виміру* – явна характеристика товару.
8. *Номер накладної* – явна унікальна характеристика накладної.
9. *Дата накладної* – явна характеристика накладної.
10. (?) *Список товарів в накладній* – список не може бути атрибутом. Ймовірно, потрібно виділити цей список в окрему сутність.
11. (?) *Кількість товару в накладній* – це явна характеристика, але характеристика чого? Це характеристика не просто «товару», а «товару в накладній».
12. (?) *Ціна товару в накладній* – знову ж таки це має бути не просто характеристика товару, а характеристика товару в накладній. Але ціна товару вже зустрічалася вище – це одне й те саме?
13. *Сума накладної* – явна характеристика накладної. Ця характеристика не є незалежною. Сума накладної дорівнює сумі вартостей усіх товарів, що входять в накладну.
14. *Найменування складу* – явна характеристика складу.

В ході додаткової бесіди з менеджером вдалося прояснити різні поняття цін. Виявилося, що кожен товар має деяку поточну ціну. Ця ціна, по якій товар продається в даний момент. Природно, що ця ціна може змінюватися з часом. Ціна одного і того ж товару в різних накладних, виписаних в різний час, може бути різною. Таким чином, є дві ціни – ціна товару в накладній і поточна ціна товару.

З виникають поняттям «Список товарів в накладній» все досить ясно.

Сутності «Накладна» і «Товар» пов'язані один з одним відношенням типу багато-до-багатьох. Такий зв'язок, як ми зазначали раніше, повинна бути розщеплена на дві зв'язку типу один-до-багатьох. Для цього потрібна додаткова сутність.

Цією сутністю і буде сутність «Список товарів в накладній». Зв'язок її з сутностями «Накладна» і «Товар» характеризується наступними фразами:

- «Кожна накладна повинна мати кілька записів зі списку товарів в накладній»;
- «Кожен запис зі списку товарів в накладній зобов'язана включатися рівно в одну накладну»;
- «Кожен товар може включатися в кілька записів зі списку товарів в накладній»;
- «Кожен запис зі списку товарів в накладній повинна бути пов'язана рівно з одним товаром».

Атрибути «Кількість товару в накладній» і «Ціна товару в накладній» є атрибутами сутності «Список товарів в накладній».

Точно також зробимо зі зв'язком, що з'єднує сутності «Склад» і «Товар». Введемо додаткову сутність «Товар на складі». Атрибутом цієї сутності буде «Кількість товару на складі». Таким чином, товар буде значитися на будь-якому складі і кількість його на кожному складі буде своє. Тепер можна внести все це в діаграму (рис. 7.33):

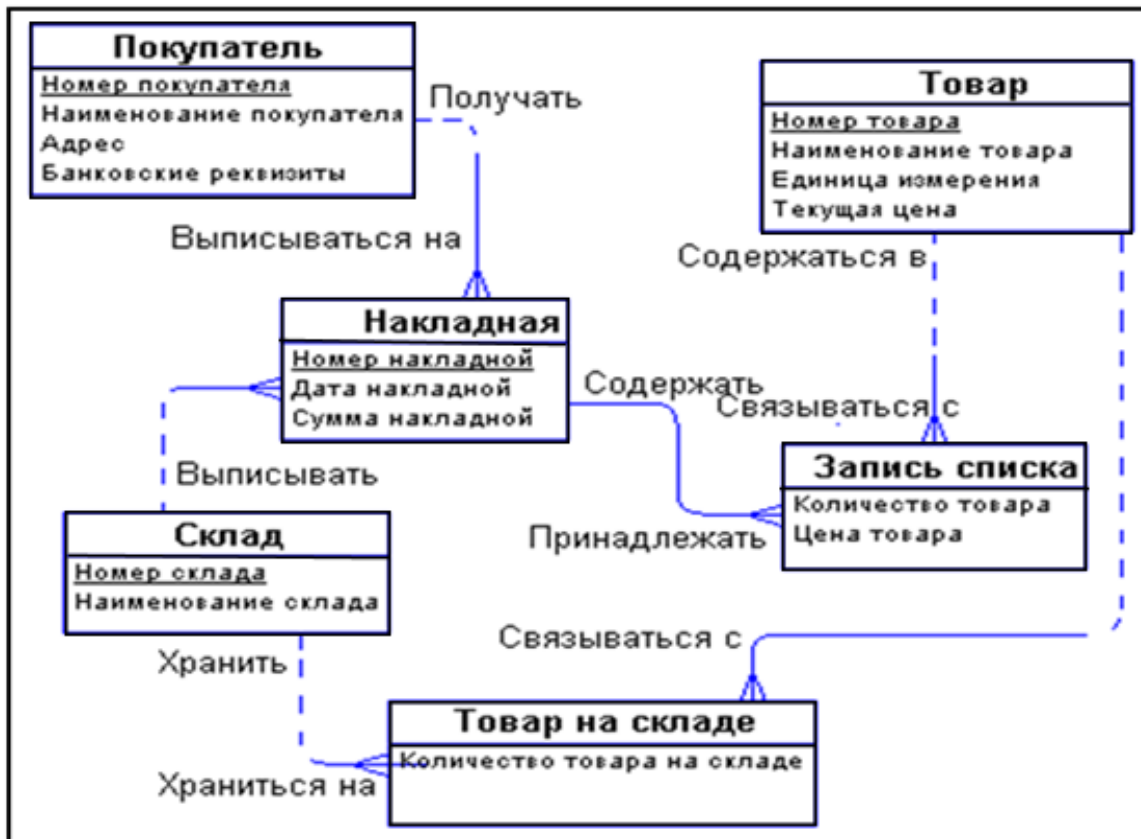


Рис. 7.33. Приклад розробки простої ER-моделі

Висновки. Реальним засобом моделювання даних є не формальний метод нормалізації відношень, а так зване семантичне моделювання.

Як інструмент семантичного моделювання використовуються різні варіанти діаграм сутність-зв'язок (ER – Entity-Relationship).

Діаграми сутність-зв'язок дозволяють використовувати наочні графічні позначення для моделювання сутностей і їх взаємозв'язків.

Різні розробники створили значний перелік програмного забезпечення, що дозволяє автоматизувати етап концептуального моделювання БД. Ці продукти об'єднують під поняттям CASE-системи. Термін CASE (Computer-Aided Software Engineering) можна перевести як автоматизоване проектування і створення програм. Як приклади CASE продуктів варто згадати: ER / Studio (корпорації Embarcadero), ERwinDataModeler (фірми Platinum – CA), PowerDesigner (фірми Sybase), Visio (корпорації Microsoft).

Основна мета CASE-проектуювання полягає не стільки в автоматизації ходу розробки БД, скільки в перетворенні трудомісткого і малозрозумілої для непосвячених «ритуалу» кодування у відносно більш простий процес логічного проектування.

Розробнику часто навіть не потрібно знати мови програмування, досить володіти методологією концептуального проектування БД. В результаті CASE-засіб виявляється доступним навіть початківцю, що не тільки прискорює, а й значно здешевлює вартість одержуваного програмного забезпечення. Адже найчастіше досить натиснути кнопку, і з намальованою концептуальною моделі створюється фізична БД. Але з іншого боку, якість результуючого продукту, згенерованого за допомогою CASE-інструменту, і якість «ручної роботи» професійного програміста сьогодні порівнювати не варто.

У результаті праці розробника баз даних повинна з'явитися БД, що представляє собою деяку абстракцію вельми складної частини реального світу. Новостворена БД повинна із заданою ступенем точності відповідати цьому світу і чітко вирішувати поставлені їй завдання по зберіганню і обробці даних. Саме тому процес розробки БД включає в себе етап концептуального проектування, на якому програміст створює спрощену модель майбутніх даних.

8. ЛОГІЧНЕ ПРОЕКТУВАННЯ БАЗ ДАНИХ

8.1. Етапи логічного проектування

Логічне проектування виконується для певної моделі даних. Для реляційної моделі даних логічне проектування полягає у створенні реляційної схеми, визначенні числа і структури таблиць, формуванні запитів до БД, визначенні типів звітних документів, створенні форм для вводу і редагування даних в БД і рішенні цілого ряду інших задач. Концептуальні моделі за певними правилами перетворюються в логічні моделі даних.

Коректність логічних моделей перевіряється за допомогою *правил нормалізації*, які дозволяють переконатися в структурній узгодженості, логічній цілісності і мінімальній збитковості прийнятої моделі даних. Модель також перевіряється з метою виявлення можливостей виконання *транзакцій*, які будуть задаватися користувачами. Проектування являє собою циклічний процес. Етапи логічного проектування наведені на рис. 8.1.



Рис. 8.1. Етапи логічного проектування бази даних

8.2. Спрощення концептуальної моделі

Першим кроком спрощення концептуальної моделі є попередні перетворення з метою усунення зв'язків, які є несумісними з реляційною моделлю.

На цьому етапі виконуються такі операції:

- вилучення двосторонніх зв'язків M:N;
- вилучення складних зв'язків;
- вилучення багатозначних атрибутів;
- вилучення рекурсивних зв'язків;
- вилучення зв'язків з атрибутами.

Вилучення двосторонніх зв'язків «багато до багатьох». Перетворення зв'язку «багато-до-багатьох» виконується шляхом введення проміжної сутності із заміною одного зв'язку М:N двома зв'язками 1:N з новою сутністю.

Приклад. *Викладач* може викладати багато *Дисциплін*, одну *Дисципліну* викладає багато *Викладачів* (рис. 8.2; а – зв'язок М:N; б – результат перетворення – два зв'язки 1:N).

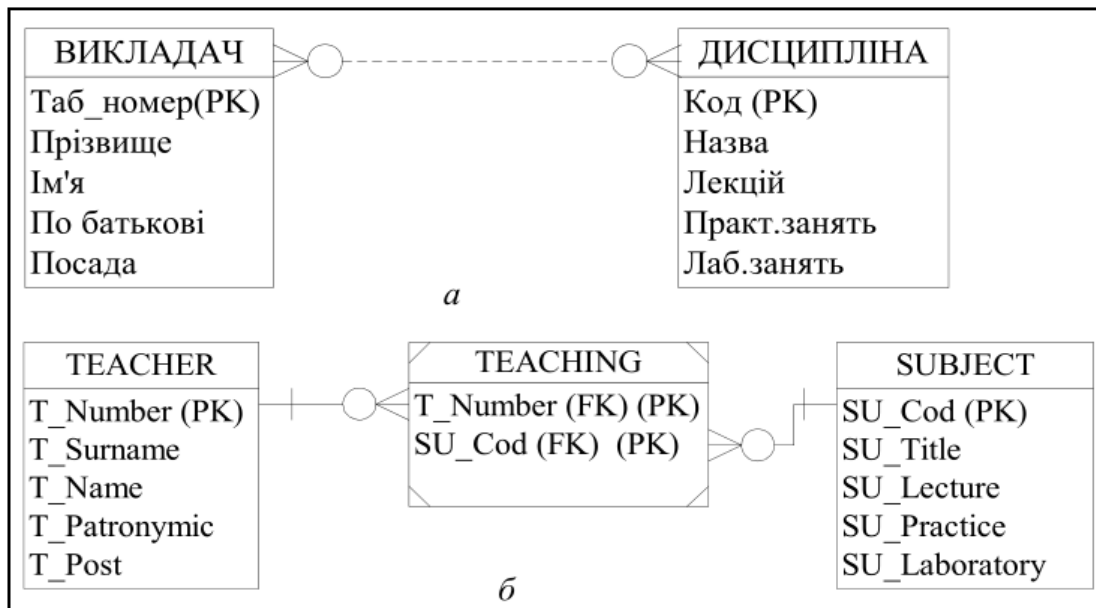


Рис. 8.2. Перетворення зв'язку «багато до багатьох»

У результаті перетворення отримана нова сутність, яка є слабкою і залежить від двох інших сутностей. Її первинний ключ складається з первинних ключів двох сутностей, а кожен атрибут окремо є вторинним ключем.

Вилучення складних зв'язків. Для вилучення складних зв'язків виконуються такі операції:

- у модель вводиться нова сутність;
- складний зв'язок замінюється бінарними зв'язками «один до багатьох» зі знов створеною сутністю;
- кількість бінарних зв'язків дорівнює ступеню складності зв'язку.

Приклад. *Викладач* може викладати багато *Дисциплін*, одну *Дисципліну* викладає багато *Викладачів*. З *Дисципліни* *Викладач* проводить *Екзамен* (рис. 8.3).

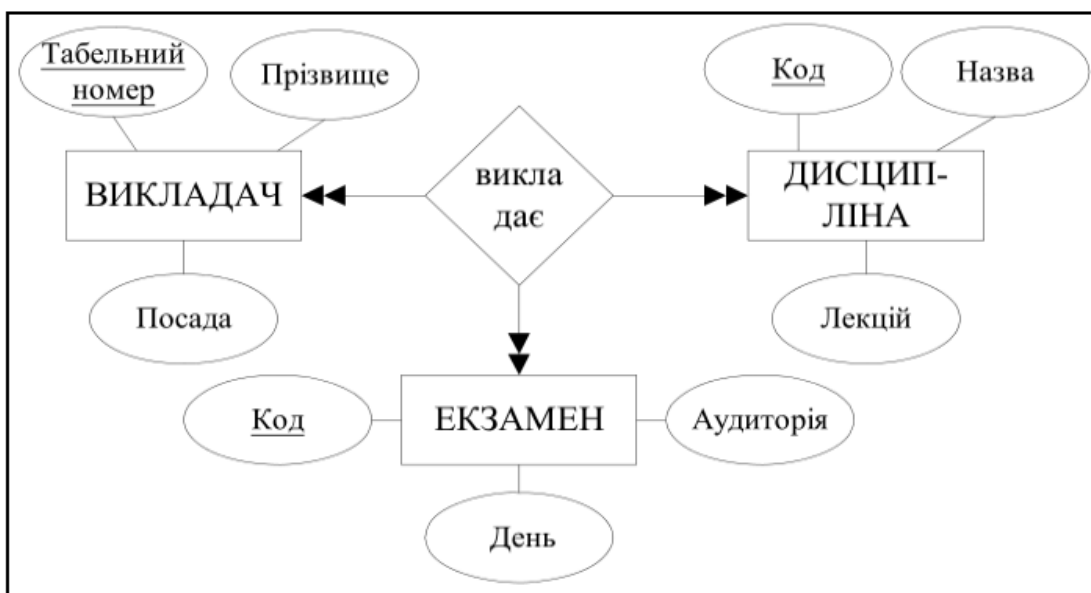


Рис. 8.3. Складний зв'язок *Викладає*

Декомпозиція складного зв'язку на три двосторонні зв'язки і нову сутність *Екзамен* показана на рис. 8.4.

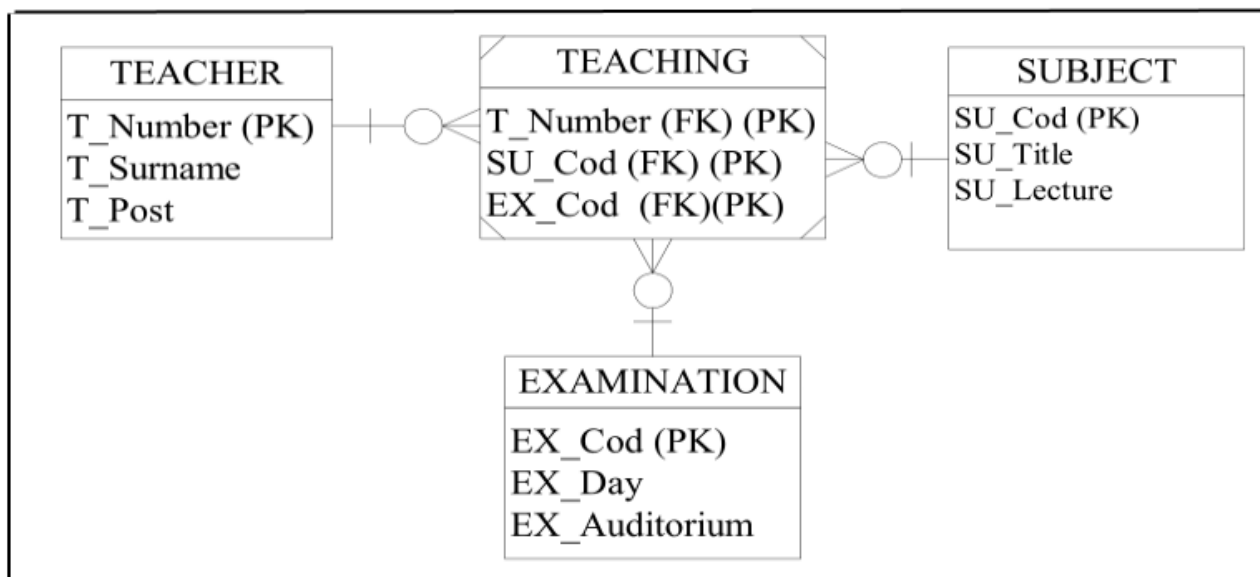


Рис. 8.4. Представлення тернарного зв'язку бінарними зв'язками

Вилучення багатозначних атрибутів. Якщо в концептуальній моделі даних присутній багатозначний атрибут, то може бути виконана декомпозиція цього атрибуту для визначення деякої сутності.

Приклад. *Студент* може мати декілька телефонів (рис.8.5; а – сутність *Студент* з багатозначним атрибутом *Номер телефону*; б – нова сутність *Телефон*).

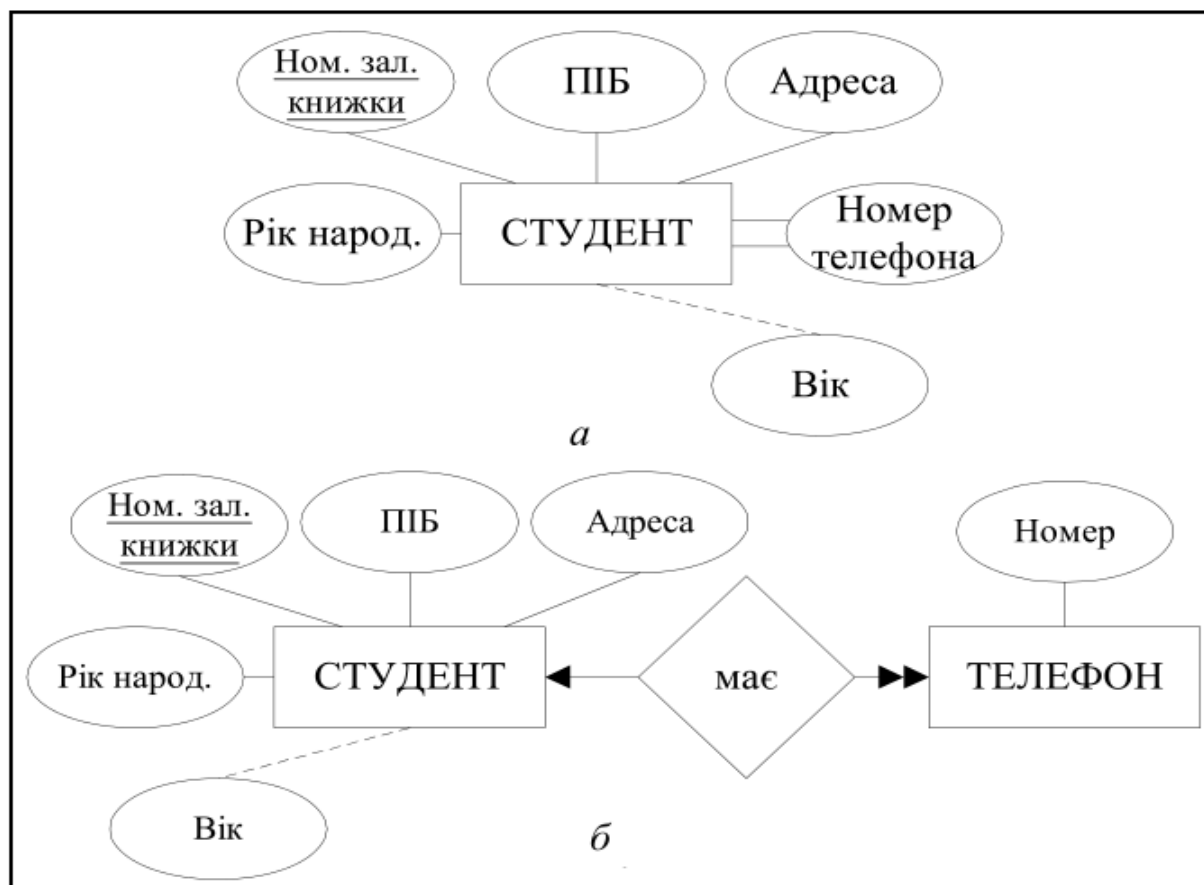


Рис.8.5. Вилучення багатозначного атрибуту

Вилучення рекурсивних зв'язків. На етапі спрощення концептуальної моделі рекурсивні зв'язки 1:1 і 1:M (рис. 8.6) можуть бути перетворені у одне відношення. У випадку, коли є необов'язкова сутність з боку «багато» для зв'язку 1:M для зменшення пустих значень створюється нове відношення. Зв'язок M:N перетворюється на дві сутності (рис.8.7).

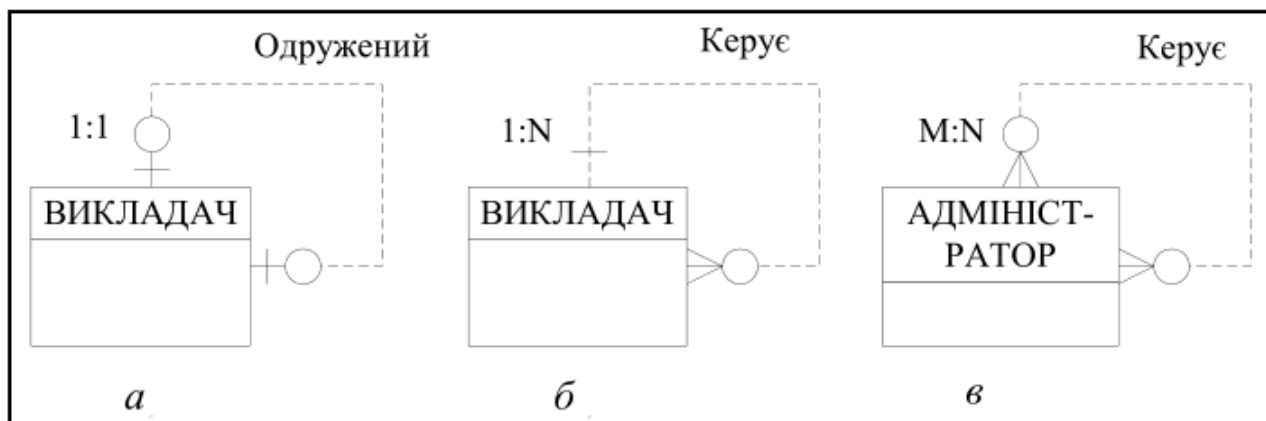


Рис. 8.6. Види рекурсивних зв'язків: а – 1:1; б – 1:N; в – M:N

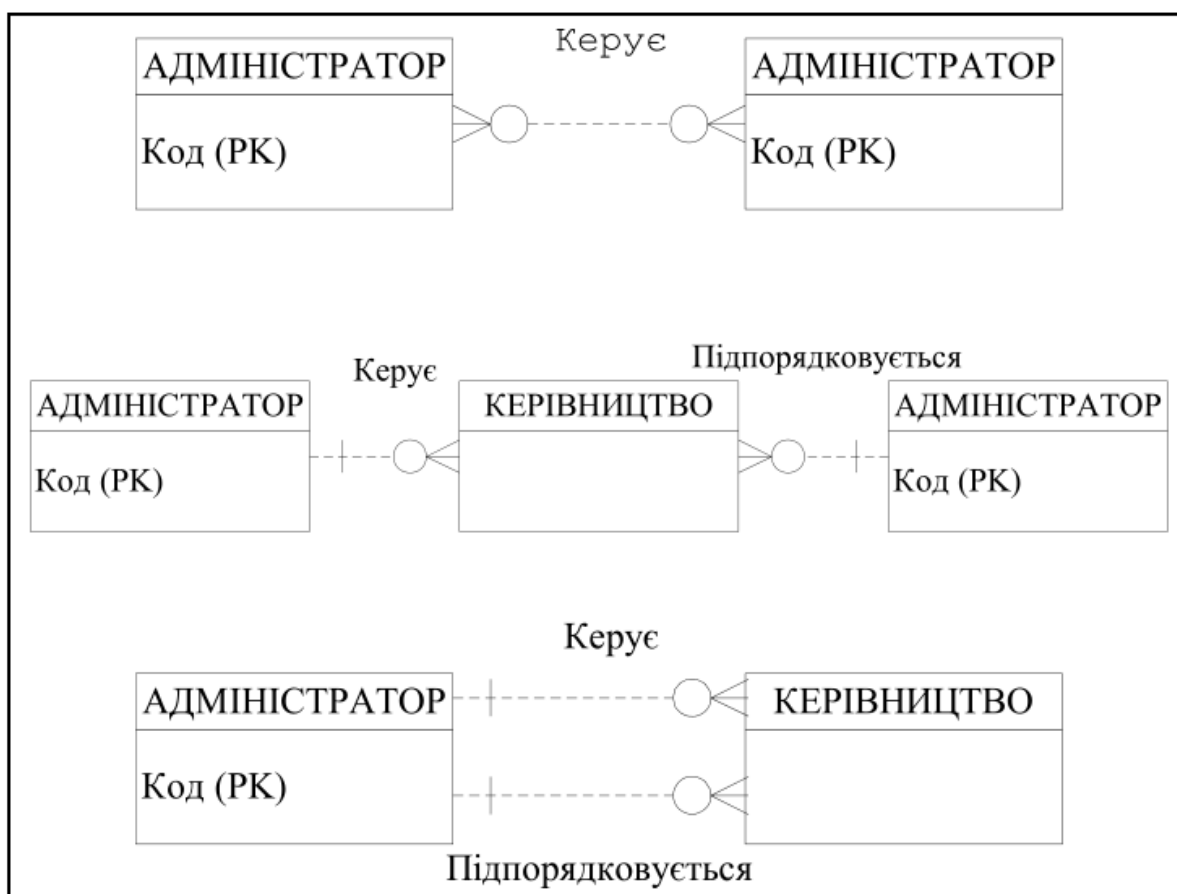


Рис. 8.7. Етапи послідовного перетворення рекурсивного зв'язку M:N

Вилучення зв'язків з атрибутами. Вилучення зв'язків з атрибутами виконується шляхом додавання у модель нової сутності для відношення M:N з атрибутами зв'язку. Для відношення 1:M атрибути зв'язку передаються у сутність «багато» без створення нової сутності.

Приклад. Розглянемо сутності *Студент-Дисципліна* (рис.8.8).

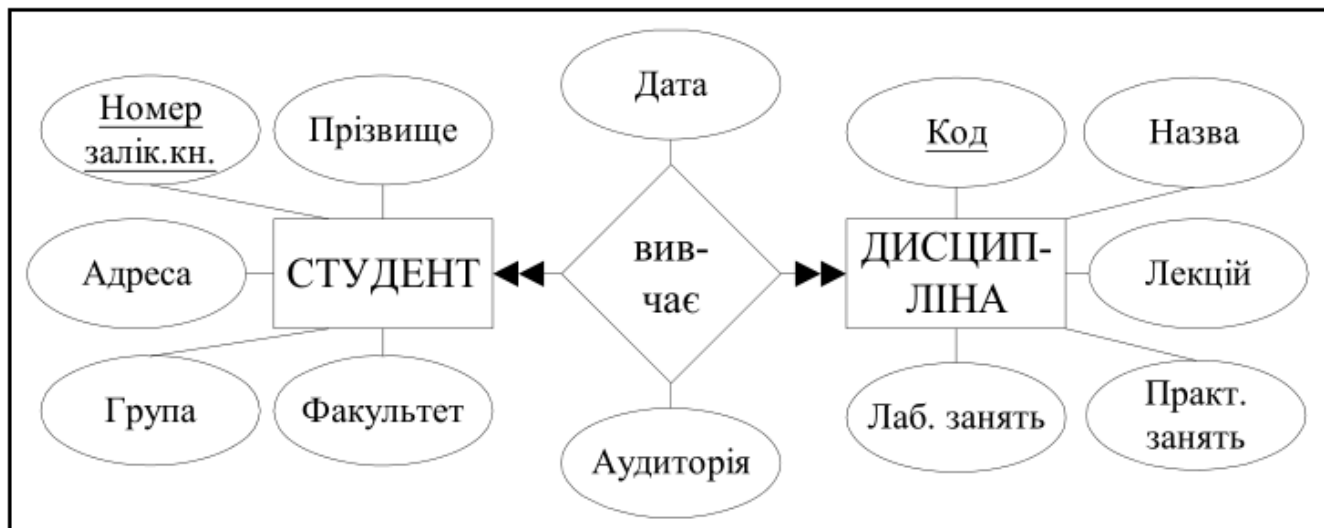


Рис. 8.8. Зв'язок «багато-до-багатьох» з атрибутами зв'язку *Дата* і *Аудиторія*

У результаті перетворення зв'язку з атрибутами отримано реляційну схему (рис. 8.9).

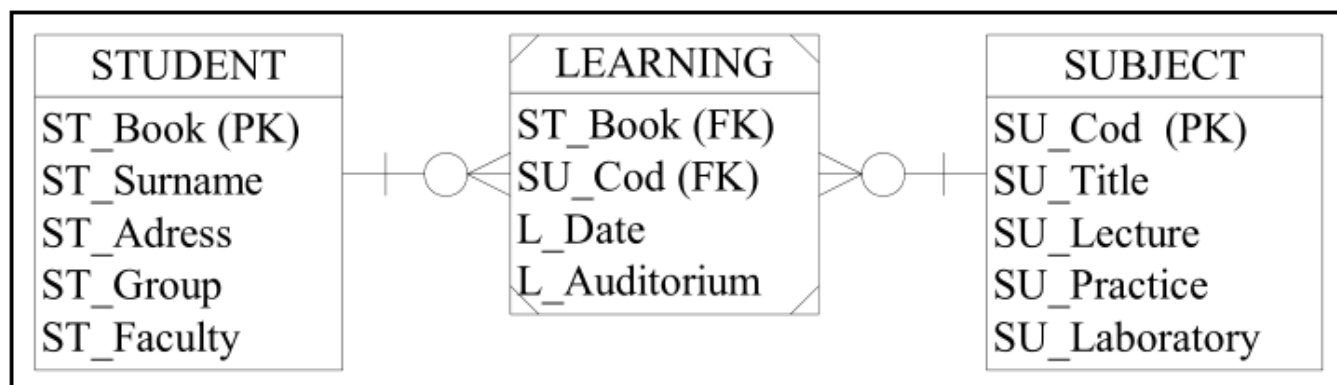


Рис. 8.9. Представлення атрибутів зв'язку *L_Date* і *L_Auditorium* у новому відношенні *Learning*

Спрощення концептуальної моделі передбачає також вилучення збиткових зв'язків. Збиткові зв'язки характеризуються тим, що одна і та ж інформація може бути отримана не тільки через них, але і через інші зв'язки.

Після спрощення в концептуальній моделі можуть бути присутні тільки такі елементи:

- об'єкти і атрибути;
- зв'язки типу 1:1 і 1:M;
- зв'язки типу суперклас-підклас.

8.3. Методика перетворення ER-діаграм в реляційні структури

Для ER-моделі існує алгоритм однозначного перетворення її в реляційну модель даних. Розглянемо правила перетворення ER-моделі в реляційну модель.

8.3.1. Сутності і атрибути

Для кожної сутності створюється відношення, кожен атрибут сутності стає атрибутом відповідного відношення.

Для *сильних сутностей* первинний ключ сутності стає PRIMARY KEY (PK) відповідного відношення.

Приклад. Розглянемо сутність *Студент* (рис.8.10).

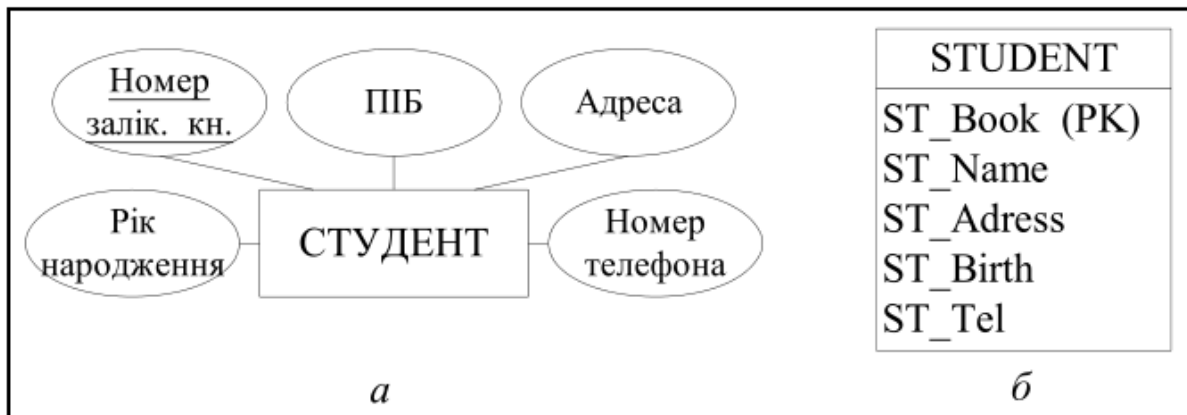


Рис. 8.10. Перетворення сутності *Студент* (а) у відношення *Student* (б)

Для *слабких сутностей* первинний ключ частково або повністю залежить від ключа сутності володаря (декількох володарів), тобто РК визначається тільки тоді, коли визначені всі РК сутностей володарів.

Приклад. Розглянемо перетворення сильної сутності *Студент* і слабкої сутності *Нагорода* (рис. 8.11).

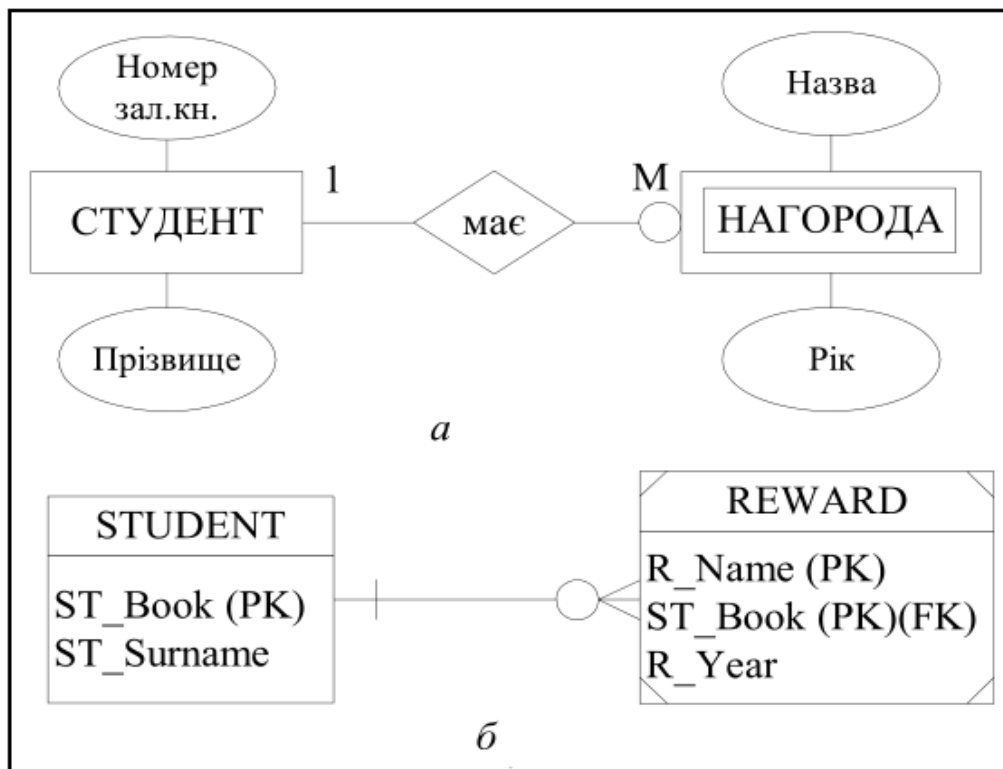


Рис. 8.11. Перетворення сильної сутності *Студент* і слабкої сутності *Нагорода* з ідентифікуючим зв'язком між ними (а) у зв'язані відношення *Student* і *Reward* (б)

8.3.2. Зв'язки

Після перетворення концептуальної моделі залишаються такі типи зв'язків:

- «один-до-одного»;
- «один-до-багатьох»;
- рекурсивні зв'язки.

Для кожного типу зв'язку залежно від умов зв'язування існують свої різновиди. Зв'язки між відношеннями в реляційній моделі реалізуються шляхом використання первинних і зовнішніх ключів.

Зв'язки «один до одного». В концептуальних моделях даних визначають такі обмеження ступеня участі сутностей:

- обов'язкова участь для обох сутностей;
- обов'язкова участь для однієї сутності;
- необов'язкова участь для обох сутностей.

Залежно від обмежень перетворення на реляційну модель будуть різні.

Приклад. Розглянемо можливі варіанти перетворення зв'язку між сутностями *Викладач* і *Дисципліна* (рис. 8.12).

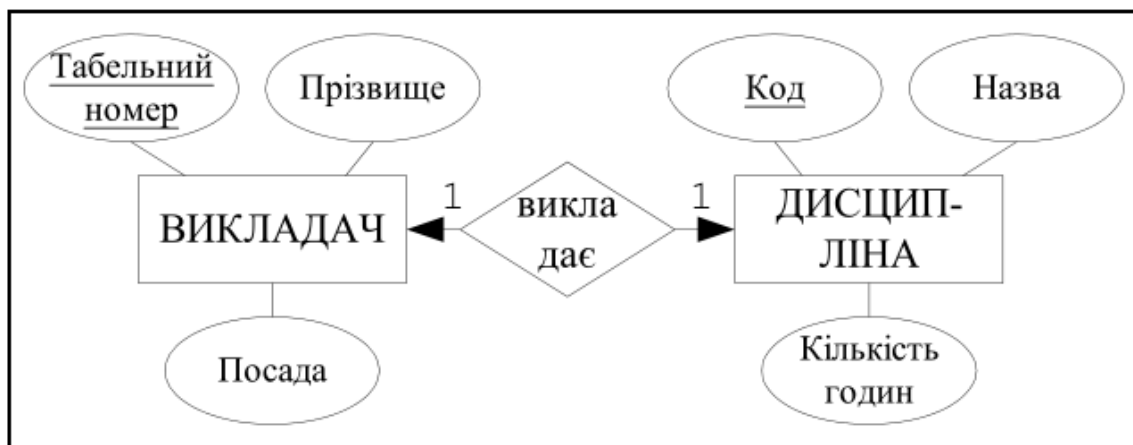


Рис. 8.12. Зв'язок 1:1 між сутностями *Викладач* і *Дисципліна*

1. **Обов'язкова участь для обох сутностей.** Припустимо, що кожен викладач обов'язково викладає одну дисципліну і кожну дисципліну обов'язково викладає один викладач. В цьому випадку реляційна структура буде складатися з одного відношення і мати один з таких варіантів (рис. 8.13).

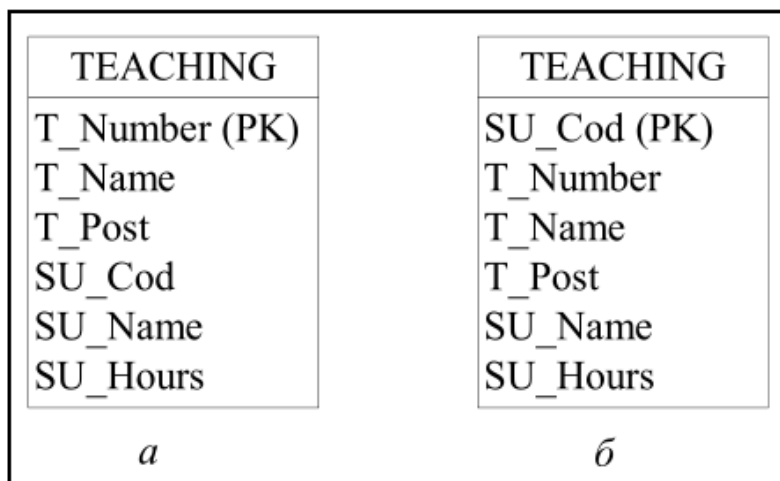


Рис. 8.13. Варіанти відношень для перетворення зв'язку 1:1 з обов'язковою участю для обох сутностей

2. **Обов'язкова участь для однієї сутності.** Припустимо, що кожен викладач обов'язково викладає одну дисципліну, а за кожною дисципліною необов'язково закріплений викладач. В цьому випадку сутність, яка є необов'язковою (*Дисципліна*) виступає в якості батьківської сутності, а обов'язкова сутність визначається як дочірня (*Викладач*). Реляційна структура показана на рис. 8.14.

Зовнішній атрибут SU_Cod також може бути ключем для *Викладача*.

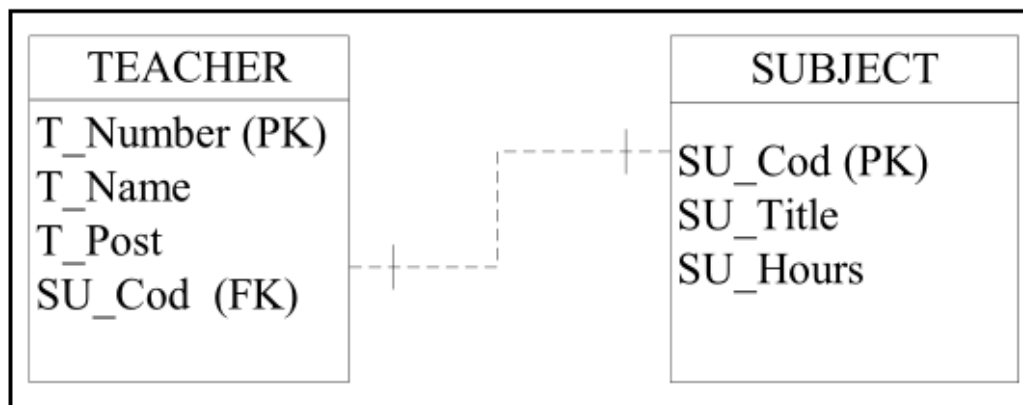


Рис.8.14. Перетворення зв'язку 1:1 з обов'язковою участю сутності Викладач і необов'язковою участю сутності Дисципліна

3. *Необов'язкова участь для обох сутностей.* Припустимо, що необов'язково кожен викладач викладає дисципліни і необов'язково за кожною дисципліною закріплений викладач. В цьому випадку можливі три реляційні структури (рис. 8.15).

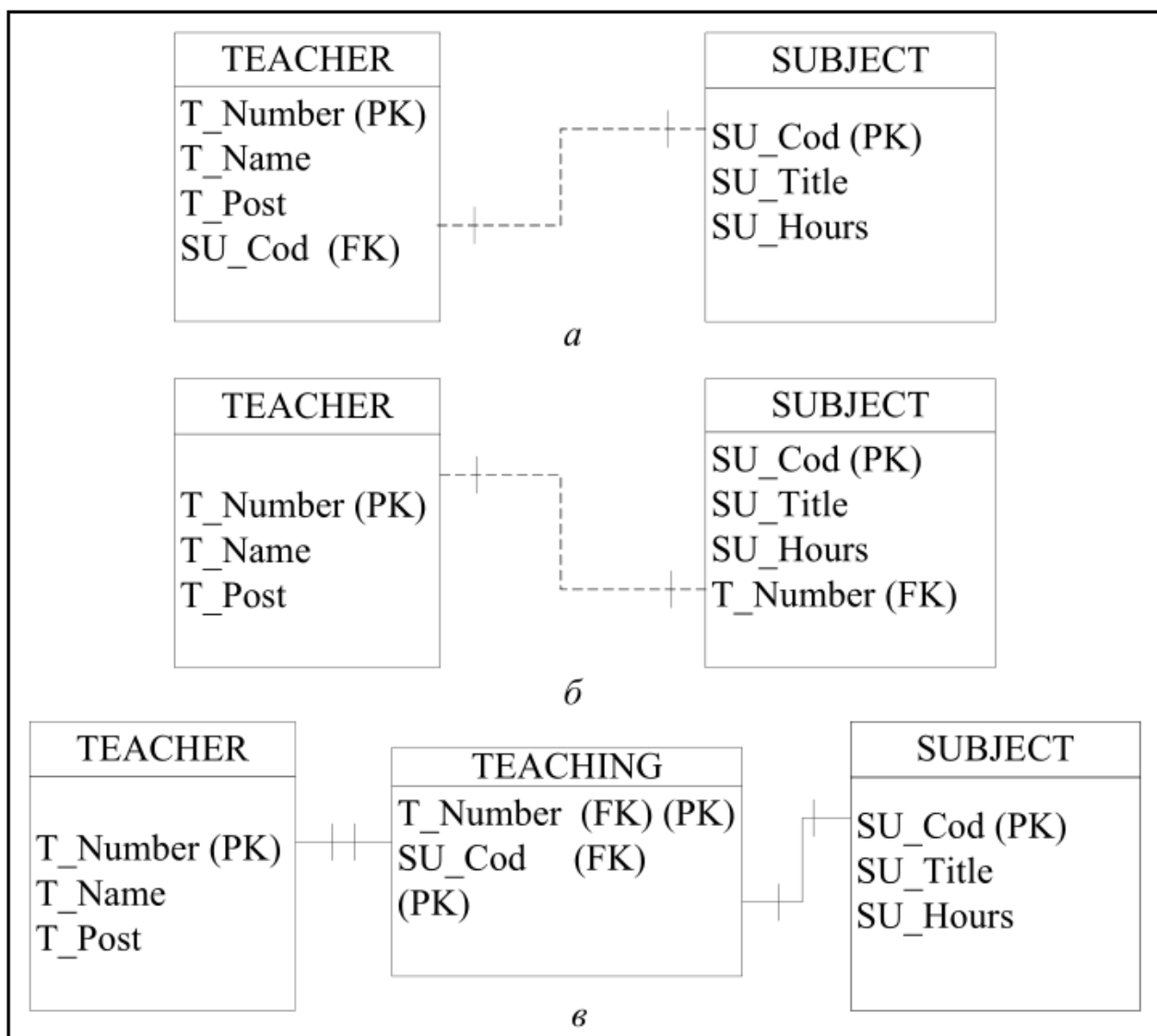


Рис. 8.15. Варіанти реляційних схем відношень для перетворення зв'язку 1:1 з необов'язковою участю для обох сутностей

Зв'язки «один-до-багатьох». У кожне відношення, яке відповідає підлеглий (дочірній) сутності, додається набір атрибутів основної (батьківської) сутності, який складає первинний ключ основної сутності. У відношенні, що відповідає підлеглий сутності, цей набір атрибутів стає зовнішнім ключем (FOREIGN KEY, FK).

Для моделювання необов'язкового типу зв'язку у атрибутів, що відповідають зовнішньому ключу, встановлюється властивість допустимості невизначених значень (NULL). У разі обов'язкового типу зв'язку атрибути набувають властивості відсутності невизначених значень (NOT NULL).

Приклад. Розглянемо можливі варіанти перетворення зв'язку між сутностями *Викладач* і *Дисципліна* (рис. 8.16).

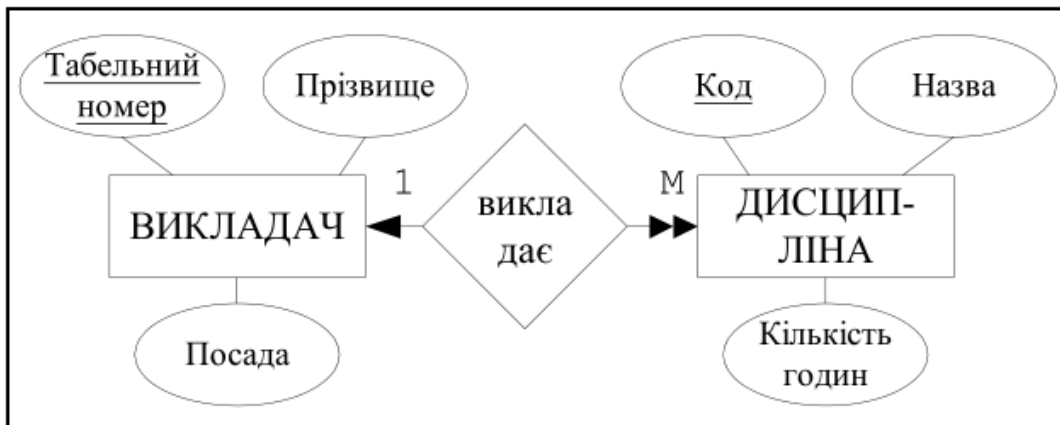


Рис. 8.16. Зв'язок 1:М між сутностями *Викладач* і *Дисципліна*

1. Необов'язкова участь сутності *Викладач* і обов'язкова участь сутності *Дисципліна* (рис. 8.17).

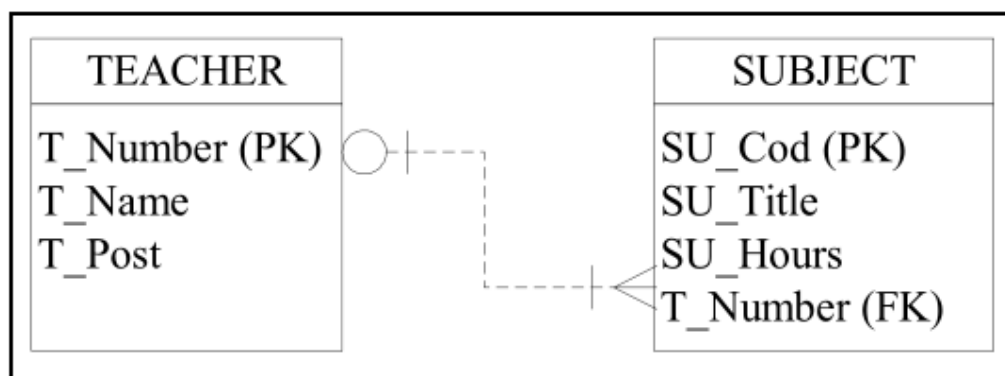


Рис. 8.17. Перетворення зв'язку 1:М з необов'язковою участю сутності *Викладач* і обов'язковою участю сутності *Дисципліна*

2. Необов'язкова участь сутності *Викладач* і необов'язкова участь сутності *Дисципліна* (рис. 8.18).

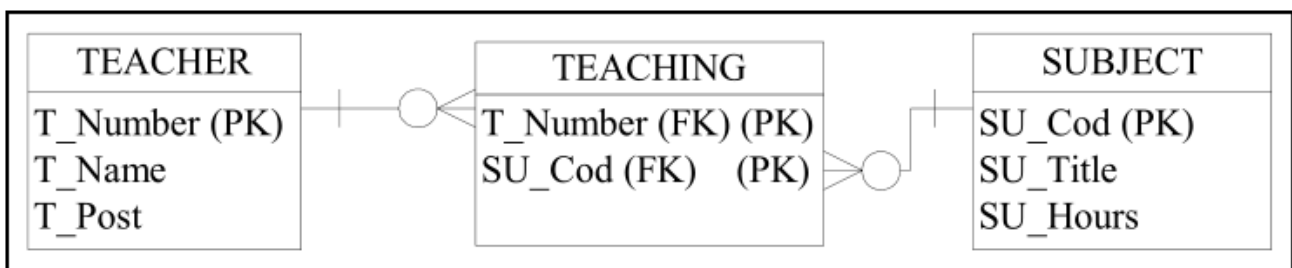


Рис. 8.18. Перетворення зв'язку 1:М з необов'язковою участю сутності *Викладач* і необов'язковою участю сутності *Дисципліна*

Зв'язки «багато-до-багатьох». Для кожного зв'язку M:N необхідно створювати додаткове відношення, яке представляє цей зв'язок і включати в нього всі атрибути, які входять в склад цього зв'язку. Копії атрибутів первинного ключа сутностей, які беруть участь у зв'язку, передаються у нове відношення для використання в якості зовнішніх ключів. Ці зовнішні ключі утворюють також первинний ключ нового відношення.

Приклад. Розглянемо можливі варіанти перетворення зв'язку між сутностями *Викладач* і *Дисципліна* (рис. 8.19).

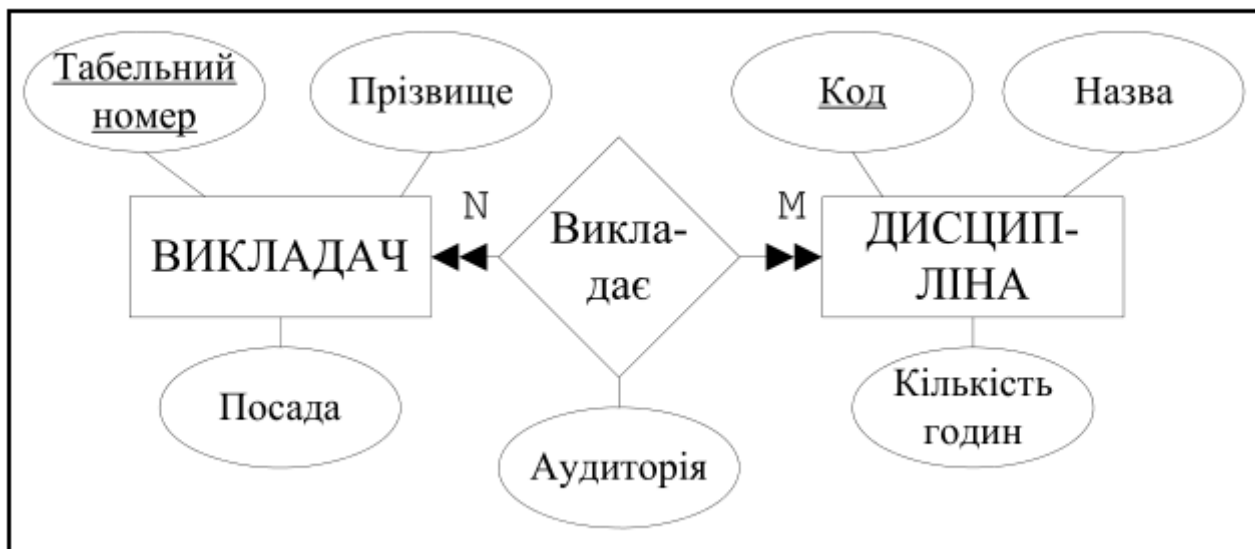


Рис. 8.19. Зв'язок N:M між сутностями *Викладач* і *Дисципліна*

В цьому випадку існує єдина схема перетворення (рис. 8.20).

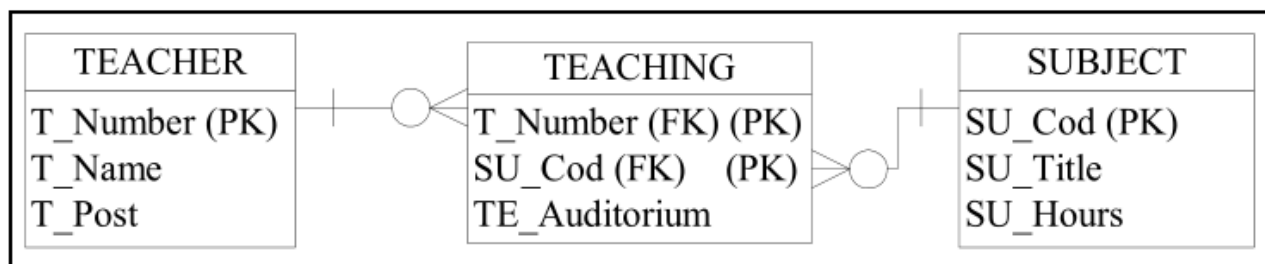


Рис. 8.20. Перетворення зв'язку N:M у реляційну схему

Інші види зв'язків. Для *рекурсивних зв'язків* 1:1 виконуються правила визначені раніше для зв'язку між двома сутностями 1:1. Для рекурсивного зв'язку 1:1 з обов'язковою участю двох сторін, реляційна схема представляється у вигляді одного відношення з двома копіями первинного ключа (див. рис. 8.13). Одна копія відповідає зовнішньому ключу. Для рекурсивного зв'язку 1:1 з обов'язковою участю тільки однієї сторони створюється або одне відношення, або нове відношення, яке відображає цей зв'язок (див. рис. 8.14). Для рекурсивного зв'язку 1:1 з необов'язковою участю обох сторін створюється нове відношення (див. рис. 8.15).

Для *складних типів зв'язків* створюється відношення, яке відображає цей зв'язок і включає всі атрибути, які входять в склад цього зв'язку. Копії атрибутів первинного ключа сутностей, які беруть участь у зв'язку, передаються у нове відношення для використання в якості зовнішніх ключів. Ці зовнішні ключі утворюють також первинний ключ нового відношення (див. рис. 8.3, 8.4).

Для *багатозначного атрибуту* створюється нове відношення, яке відповідає багатозначному атрибуту, і в це нове відношення передається первинний ключ сутності для використання в якості зовнішнього ключа (див. рис. 8.5).

Зв'язки «суперклас – підклас». Для виконання перетворення зв'язку типу суперклас – підклас у реляційну модель необхідно враховувати також обмеження ступеня участі у зв'язку

(обов'язково – Mandatory або необов'язково – Optional) і обмеження неперетинання (And або Or). Можливі чотири сполучення, перетворення яких дає чотири реляційні схеми. На схему також впливає те, чи беруть участь підкласи в різних зв'язках, кількість сутностей в зв'язку тощо. Діапазон можливих варіантів рішення є достатньо великим і конкретна схема вибирається в кожному конкретному випадку з урахуванням багатьох факторів.

Приклад. Розглянемо суперклас **Викладач**, який має атрибути *Табельний номер*, *Прізвище*, *Посада*. Підкласами суперкласу виступають об'єкти **Професор**, **Доцент**, **Асистент** (рис. 8.21). Кожен екземпляр підкласу може бути екземпляром суперкласу, тобто суперклас може мати свої екземпляри (Optional). Кожен викладач обов'язково належить тільки одному підкласу (Or). Ця діаграма перетворюється в реляційну схему відношень показану на рис. 8.22. Зв'язок між відношеннями виконується за допомогою ключа суперкласу (*Табельний номер*).

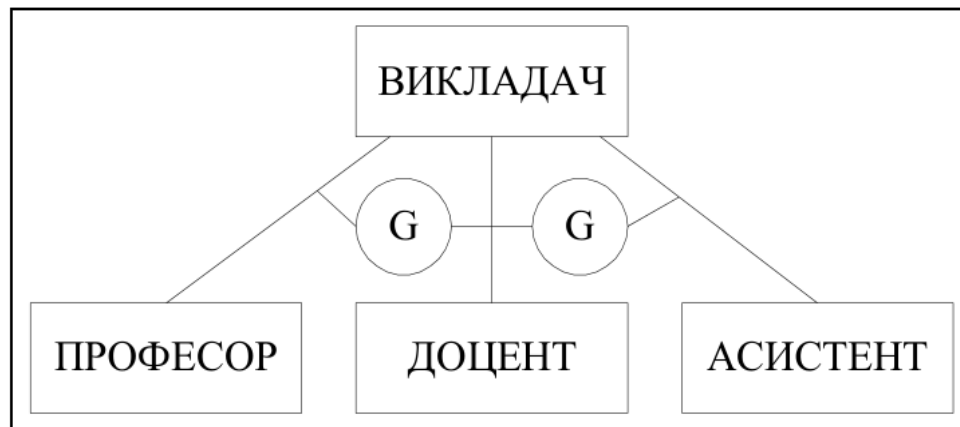


Рис. 8.21. Зв'язок суперклас – підклас з обмеженнями Optional і Or

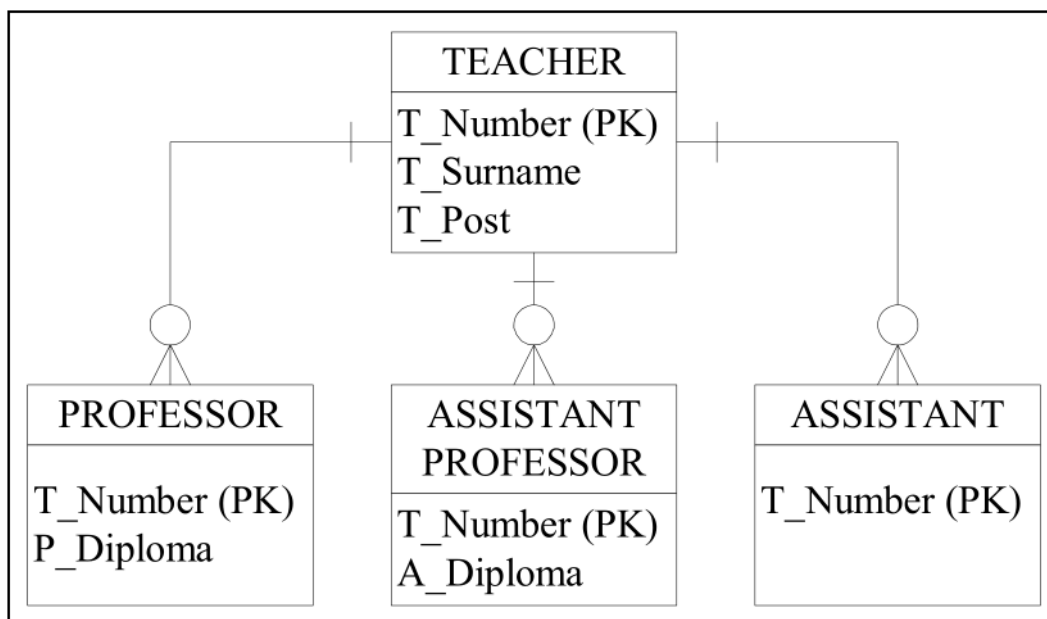


Рис. 8.22. Реляційна схема, яка відповідає попередньому зв'язку суперклас – підклас

Приклад. Розглянемо суперклас **Студент**, який має атрибути *Номер залікової книжки*, *Прізвище*, *Група*. Підкласами суперкласу виступають об'єкти **Очна**, **Заочна**, **Вечірня** і **Дистанційна форми навчання** (рис. 8.23). Кожен екземпляр підкласу є одночасно екземпляром суперкласу (Mandatory). Кожен студент може належати до декількох підкласів, тобто одночасно може займатися на різних формах навчання (And). Ця діаграма перетворюється в наступну реляційну схему відношень (рис. 8.24).

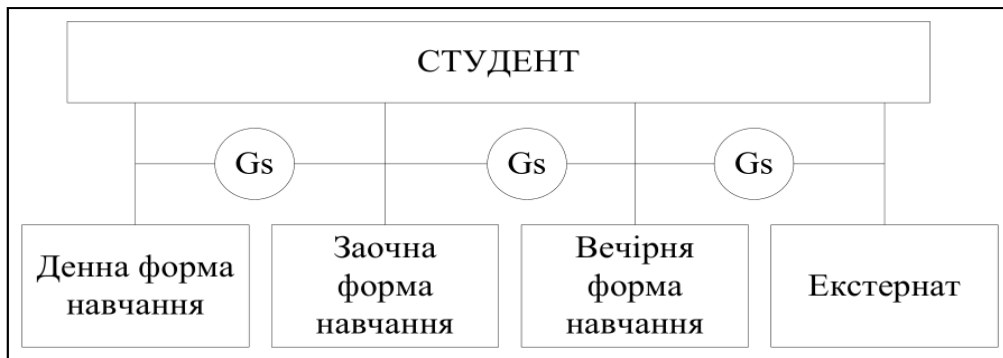


Рис. 8.23. Зв'язок суперклас–підклас з обмеженнями Mandatory і And

STUDENT
ST_Book (PK)
ST_Surname
ST_Group
ST_Confront
ST_Correspond
ST_Nightclasses
ST_Distance

Рис. 8.24. Реляційна схема, яка відповідає попередньому зв'язку суперклас–підклас

8.4. Перевірка відношень за допомогою правил нормалізації

Створений на попередніх етапах набір відношень логічної моделі БД повинен бути перевірений на коректність об'єднання атрибутів у кожному відношенні. Перевірка виконується шляхом застосування до кожного відношення процедури послідовної нормалізації. Атрибути в результаті нормалізації будуть згруповані відповідно до існуючих між ними логічних зв'язків. Для забезпечення коректності логічної моделі, у разі виявлення відношень, які не відповідають вимогам нормалізації, необхідно повернутися на попередні етапи проектування і перебудувати помилково створені елементи моделі.

Приклад. В результаті проектування отримано відношення показана на рис. 8.25.

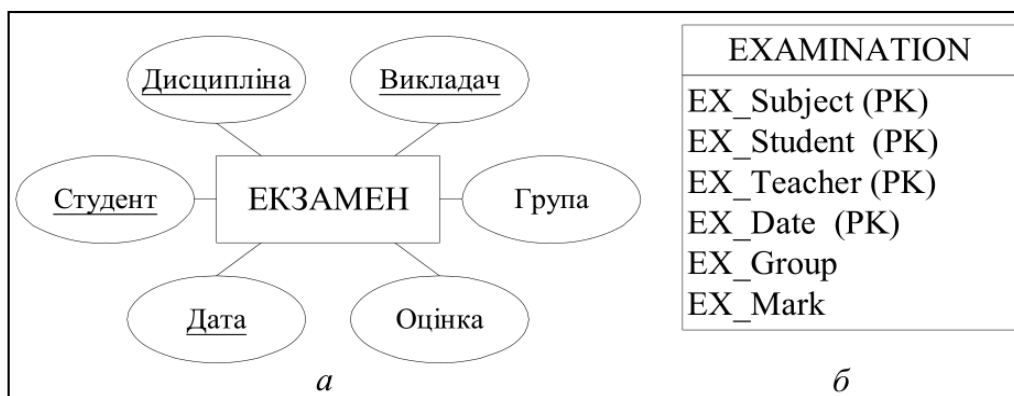


Рис. 8.25. Перетворення сутності Екзамен (а) у відношення Examination (б)

При дослідженні даного відношення були виявлені такі функціональні залежності:

Дисципліна, Викладач, Студент, Дата → Оцінка

Студент → Група

У наведеній схемі існують аномалії і необхідно продовжити нормалізацію. В результаті декомпозиції вихідного відношення буде отримана схема показана на рис. 8.26.

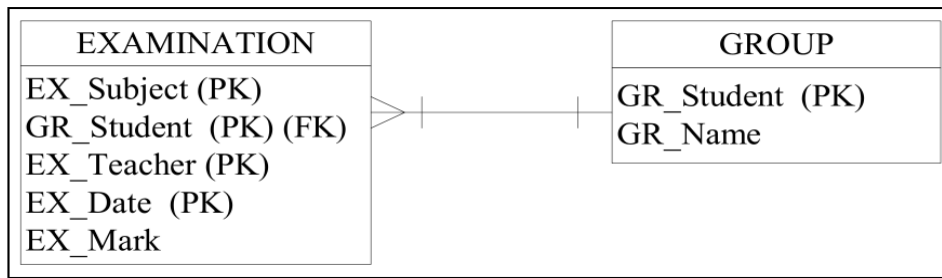


Рис. 8.26. Реляційна схема, яка відповідає сутності *Екзамен*

8.5. Приклад створення логічної моделі бази даних

У попередній лекції був розроблений концептуальний проект бази даних для предметної області ЗАКЛАД ВИЩОЇ ОСВІТИ (ЗВО). ER-діаграма ЗВО відображає всі бізнес правила, які в свою чергу визначають сутності, атрибути, зв'язки тощо. Наступним етапом проектування БД є створення логічної моделі БД на основі створеної ER-моделі. Створення логічної моделі бази даних виконується шляхом застосування правил перетворення ER-діаграми в логічну модель (рис. 8.27).

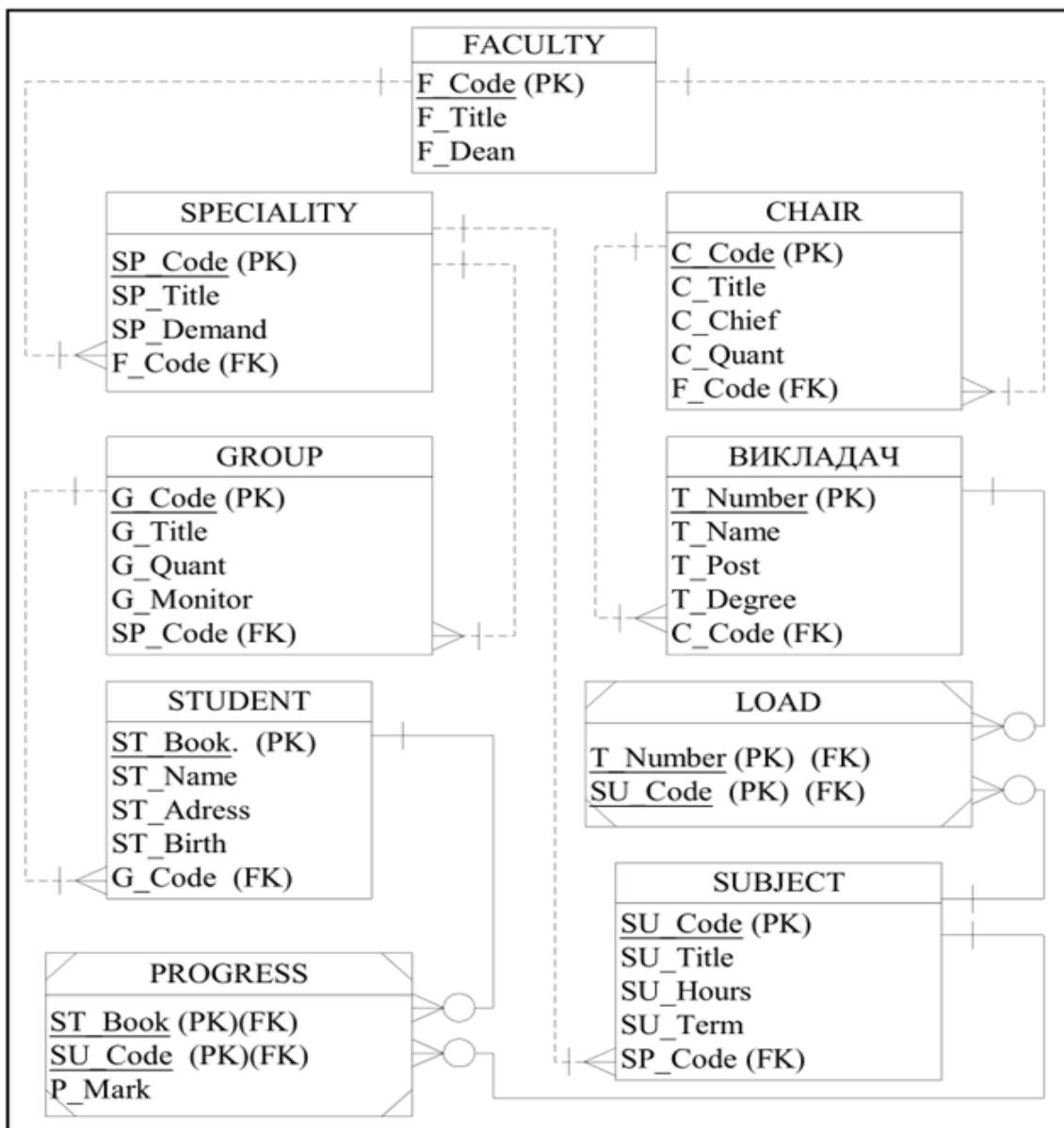


Рис. 8.27. Логічна модель бази даних для предметної області ЗАКЛАД ВИЩОЇ ОСВІТИ

Після створення логічної моделі даних реляційна схема аналізується на коректність об'єднання атрибутів в одному відношенні. Перевірка коректності виконується шляхом застосування послідовної нормалізації до кожного з відношень. Метою цієї перевірки є отримання гарантій того, що схема бази даних щонайменше знаходиться в 3-й нормальній формі або в нормальній формі Бойса-Кодда. Якщо ця умова не виконується, то необхідно повернутися на попередні етапи проектування і перебудувати помилково створені фрагменти моделі. Перевірка логічної моделі бази даних ЗВО показує, що реляційна схема знаходиться в 4-й нормальній формі й корегування моделі не потрібно.

Після перевірки логічної моделі за допомогою правил нормалізації система аналізується на предмет виконання транзакцій користувачів, які задаються на початкових етапах проектування. У разі неможливості виконання певних транзакцій необхідне корегування моделі бази даних (див. рис. 8.1).

Подальша перевірка моделі вимагає перевірки підтримки цілісності даних. На основі матеріалів прикладу можна тільки визначити, що підтримується посилкова цілісність. Всі інші перевірки, включаючи і перевірку транзакцій користувачів, вимагають більш детально опрацьованого проєкту бази даних.

9. ФІЗИЧНЕ ПРОЕКТУВАННЯ БАЗИ ДАНИХ

Фізична організація даних відповідає за їх зберігання, управління, форми представлення і структури даних.

Фізичне проектування бази даних – процес підготовки опису реалізації бази даних на вторинних запам'ятовуючих пристроях і створення на їх основі внутрішньої (фізичної) моделі даних. На цьому етапі розглядаються основні відносини, організація файлів і індексів, призначених для забезпечення ефективного доступу до даних, а також всі пов'язані з цим обмеження цілісності і засоби захисту.

Фізичне проектування є третім і останнім етапом створення проекту бази даних, при виконанні якого проектувальник приймає рішення про способи реалізації розроблюваної бази даних. Під час попереднього етапу проектування була визначена логічна структура бази даних, яка описує відносини і обмеження в даній прикладній області. Хоча ця структура не залежить від конкретної цільової СКБД, вона створюється з урахуванням обраної моделі зберігання даних, наприклад, реляційної, мережевої або ієрархічної.

Однак, приступаючи до фізичного проектування бази даних, перш за все необхідно вибрати конкретну цільову СКБД. Тому фізичне проектування нерозривно пов'язане з конкретною СКБД. Між логічним і фізичним проектуванням існує постійний зворотний зв'язок, так як рішення, що приймаються на етапі фізичного проектування з метою підвищення продуктивності системи, здатні вплинути на структуру логічної моделі даних.

9.1. Етапи фізичного проектування баз даних

Як правило, основною метою фізичного проектування бази даних є опис способу фізичної реалізації логічного проекту бази даних. У разі реляційної моделі даних під цим мається на увазі наступне:

- створення набору реляційних таблиць і обмежень для них на основі інформації, представленої в глобальній логічній моделі даних;
- визначення конкретних структур зберігання даних і методів доступу до них, що забезпечують оптимальну продуктивність СКБД;
- розробка засобів захисту створюваної системи.

Етапи концептуального і логічного проектування великих систем слід відокремлювати від етапів фізичного проектування. На це є кілька причин.

1. Вони пов'язані з абсолютно різними аспектами системи, оскільки відповідають на питання, що робити, а не як робити.
2. Вони виконуються в різний час, оскільки зрозуміти, що треба зробити, слід перш, ніж вирішити, як це зробити.
3. Вони вимагають абсолютно різних навичок і досвіду, тому вимагають залучення фахівців різного профілю.

Проектування бази даних – це ітераційний процес, який має свій початок, але не має кінця і складається з нескінченної низки уточнень. Його слід розглядати перш за все, як процес пізнання. Як тільки проектувальник приходить до розуміння роботи підприємства та сенсу оброблюваних даних, а також висловлює це розуміння засобами обраної моделі даних, набуті знання можуть показати, що потрібне уточнення і в інших частинах проекту.

Особливо важливу роль в загальному процесі успішного створення системи грає концептуальне і логічне проектування бази даних. Якщо на цих етапах не вдасться отримати повне уявлення про діяльність підприємства, то задача визначення всіх необхідних для користувача уявлень або забезпечення захисту бази даних стає надмірно складною або навіть нездійсненною. До того ж може виявитися скрутним визначення способів фізичної реалізації або досягнення прийнятної продуктивності системи. З іншого боку, здатність адаптуватися до змін є одним з ознак вдалого проекту бази даних. Тому цілком має сенс затратити час і енергію, необхідні для підготовки найкращого можливого проекту.

Етапи фізичного проектування баз даних:

1. Перенесення глобальної логічної моделі даних в середовище цільової СКБД.
2. Проектування основних відносин.
3. Реалізація обмежень предметної області.
4. Проектування фізичного представлення бази даних.
5. Аналіз транзакцій.
6. Вибір файлової структури.
7. Визначення індексів.
8. Визначення вимог до дискової пам'яті.

Фізичне проектування баз даних включає шість основних етапів. Концептуальне і логічне проектування охоплює два перших етапи розробки баз даних, а фізичне проектування-етапи 3-8.

Етап 3 стадії фізичного проектування включає розробку основних відношень і реалізацію обмежень предметної області з використанням доступних функціональних коштів цільової СКБД. На цьому етапі має бути також прийнято рішення щодо вибору способів отримання похідних даних, які включені в модель даних.

Етап 4 включає вибір файлової організації та індексів для основних відношень. Як правило, СКБД для персональних комп'ютерів мають фіксовану структуру зовнішньої пам'яті, а інші СКБД надають кілька альтернативних варіантів файлової організації для зберігання даних. З точки зору користувача організація внутрішньої структури зберігання відношень повинна бути абсолютно прозорою – користувач повинен мати можливість отримувати доступ до будь-якого відношення і до окремих його рядками без урахування способу зберігання даних. Це означає, що СКБД повинна забезпечувати повну незалежність фізичного зберігання даних від їх логічної організації. Тільки в цьому випадку внесення змін до фізичної організації бази даних не зробить ніякого впливу на роботу користувачів.

Розробник повинен надати докладні фізичні проекти бази даних з урахуванням застосовуваної СКБД і операційної системи. В проекті реалізації бази даних в СКБД розробник повинен визначити структури файлів, які будуть використовуватися для подання кожного відношення. В проекті реалізації бази даних в операційній системі розробник повинен вказати розташування окремих файлів і забезпечити необхідну їх захист.

На етапі 5 необхідно прийняти рішення про те, як має бути реалізовано кожне подання користувача. Аналіз транзакцій-визначення функціональних характеристик транзакцій, які будуть виконуватися в проєктованій базі даних, і виділення найбільш важливих з них.

А на етапі 6 здійснюється визначення оптимальної файлової структури для зберігання базових відношення та індексів, необхідних для досягнення прийнятної продуктивності. Іншими словами, визначення способу зберігання відношень і кортежів у вторинній пам'яті. Вибір файлової структури – визначення найбільш ефективної файлової структури для кожного базового відношення.

На етапі 7 аналізується також необхідність зниження рівня вимог нормалізації даних в логічній моделі, що може сприяти підвищенню загальної продуктивності системи. Однак ці дії слід робити тільки в разі реальної необхідності, оскільки введення в базу даних надмірності неминуче викличе появу проблем з підтриманням цілісності даних. Вибір індексів-визначення того, чи буде додавання індексів сприяти підвищенню продуктивності системи.

На етапі 8 описаний спосіб організації поточного контролю операційної системи, що дозволяє своєчасно виявляти і усувати всі проблеми продуктивності, які можуть бути вирішені на рівні проекту, а також враховувати нові або змінені вимоги. Визначення вимог до дискового простору – оцінка обсягу дискового простору, необхідного для розміщення бази даних.

Кінцевим підсумком розробки фізичної організації БД є бази даних – файл бази даних і файли пошукових структур. У ПК ці файли можуть бути послідовними або прямими (мається на увазі послідовного або прямого доступу).

9.2. Організація зберігання інформації

Фізична організація даних відповідає за їх зберігання, управління, форми представлення і структури даних. Фізичне проектування являє собою процес визначення характеристик сховища даних і доступу до них в БД. Властивості сховища даних залежать від пристроїв зберігання, засобів доступу до даних, що підтримуються системою і від СКБД. На етапі фізичного проектування визначається місцезнаходження даних на пристроях зберігання, загальна продуктивність системи.

В реляційних БД складні фізичні процеси організації даних приховані від користувача, але вони мають великий вплив на продуктивність роботи з БД.

Процес пошуку і представлення даних користувачу виконується в декілька етапів (рис. 9.1).

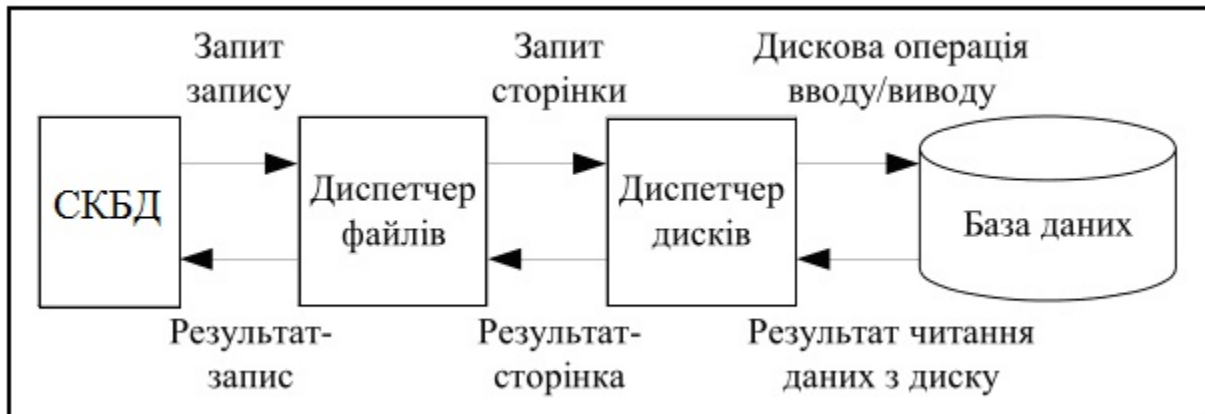


Рис. 9.1. Організація доступу до даних

Спочатку визначається необхідний запис, для знаходження якого викликається диспетчер файлів. Диспетчер файлів визначає сторінку, на якій знаходиться запис, і для її отримання викликає диспетчер дисків. Диспетчер дисків визначає фізичне положення необхідної сторінки на диску і передає її диспетчеру файлів, а той – СКБД. Головними одиницями операцій обміну системи СКБД–БД є сторінки даних. З точки зору СКБД база даних виглядає як набір записів, з точки зору диспетчера файлів БД виглядає як набір сторінок. Кожна сторінка пам'яті має унікальний ідентифікатор. Кожен запис зберігається повністю на одній сторінці.

Для організації зберігання даних застосовується технологія **кластеризації** – фізично близьке розташування записів у просторі пам'яті середовища зберігання БД.

Пам'ять сторінок – організація простору зовнішньої або віртуальної пам'яті в БД, яка передбачає поділ простору пам'яті на сторінки фіксованого розміру. Для розміщення сторінок, що викликаються, в оперативній пам'яті використовуються спеціальні області – **буфери**. Оновлений в буферах зміст сторінок повертається у зовнішню пам'ять. Розмір сторінок, кількість буферів, алгоритми вибору буферів для розміщення сторінок суттєво впливають на ефективність роботи системи.

Компоненти системи БД, які призначені для зберігання даних, мають різну ємкість і різну швидкість доступу до даних. Організація ієрархії пам'яті комп'ютера наведена на рис. 9.2.

Кеш-пам'ять – швидка буферна пам'ять відносно невеликого розміру, яка зберігає останні дані до яких виконувався доступ. Вона реалізується апаратними або програмно-апаратними засобами.

Віртуальна пам'ять – це пам'ять, яка моделюється за допомогою апаратних засобів і програмного забезпечення. Вона дозволяє користувачу оперувати з об'ємами даних, які значно перевершують простір пам'яті, що виділяється в оперативній пам'яті, таким чином нібито вони знаходились в оперативній пам'яті.

Третична пам'ять – запам'ятовуючий пристрій великого об'єму і з низькою вартістю. Найчастіше це стойки з компакт-дисками або касети магнітної стрічки.



Рис. 9.2. Ієрархія пристроїв пам'яті бази даних

Організація зберігання даних має таку ієрархію:

- атрибути відображаються у послідовність байтів постійної або змінної довжини, яка називається *полями*;
- поля об'єднуються в набори даних постійної або змінної довжини, які називаються *записом*;
- записи зберігаються у фізичних *блоках (сторінках)*;
- набори записів, які відповідають відношенням, зберігаються у вигляді *файлів*.

Розрізняють такі засоби фізичної організації файлів даних: послідовний, індексно-послідовний, прямий.

Послідовний файл – файл, до записів якого можливий послідовний доступ у порядку їх фізичного розміщення в пам'яті. Послідовна організація ефективна, коли застосування при кожному зверненні до БД обробляє значну кількість записів.

Індексно-послідовний файл – файл, до записів якого можливий прямий доступ за допомогою створеного для нього *індексу* по заданому ключу, а також послідовний доступ згідно з їх впорядкуванням по цьому ключу індексування. Індексно-послідовна організація ефективна, коли достатньо багато додатків потребують послідовної обробки і достатньо багато додатків потребують прямого доступу.

Файл прямого доступу – файл, в якому забезпечується прямий доступ до його записів за їх безпосередньою або посередньою адресою у середовищі зберігання або за заданими ключем за допомогою будь-якого методу відображення ключа в адресу. Пряма організація необхідна, коли для більшості додатків потрібен прямий доступ до записів.

9.3. Хешування

Хешування – технологія прямого доступу до записів БД за відомими значеннями їхніх ключів. Час доступу суттєво не залежить від кількості записів, що зберігаються. Суть методу хешування полягає в тому, що за значеннями ключа k , або його певних характеристик, обчислюється деяка хеш-функція $h(k)$ і отримане значення береться в якості адреси початку пошуку.

Хеш-структура показана на рис.9.3.

Недоліком більшості хеш-функцій є те, що вони не гарантують отримання унікальної адреси, оскільки кількість можливих значень поля хешування може бути значно більшою за кількість адрес, які доступні для запису. Ситуація, коли різним значенням ключів відповідає одне значення хеш-функції називається *колізією*. Значення ключів, які мають одну й ту ж хеш-функцію

називаються **синонімами**. Для вирішення колізій застосовують спеціальні методи: послідовного перебору, введення області переповнення, багатократне хешування.

Хеш-структури можуть забезпечити відповідь на запитання до БД за одне звернення до диску при умові відсутності синонімів. Хеш-структури мають низьку ефективність при запитах на визначення діапазонів, знаходження екстремумів і деяких інших.



Рис. 9.3. Організація хеш-структури

9.4. Індексація

Незважаючи на високу ефективність хеш-адресації, в файлових структурах далеко не завжди вдається знайти відповідну функцію, тому при організації доступу по первинному ключу широко використовуються індексні файли.

Індекс – структура даних, яка допомагає СКБД швидше знайти окремі записи в файлі і скоротити час виконання запитів користувачів. Основне призначення індексів полягає в забезпеченні ефективного прямого доступу до записів таблиці по ключу. Файл, що містить індексні записи називається **індексним файлом**.

Залежно від організації індексної і основної області розрізняють два типи файлів: зі щільним індексом і з розрідженим індексом. **Щільний індекс** вміщує індексні записи для всіх значень ключа пошуку в даному файлі. **Розріджений індекс** вміщує індексні записи тільки для деяких значень ключа пошуку в даному файлі.

У файлах зі щільним індексом основна область вміщує послідовність записів однакової довжини, які розташовані у довільному порядку, а структура індексного запису має такий вигляд: значення ключа, номер запису. В індексних файлах зі щільним індексом для кожного запису в основній області існує один запис з індексної області. Всі записи в індексній області впорядковані за значенням ключа. Файли зі щільним індексом називаються **індексно-прямими файлами**.

У файлах з розрідженим індексом основна область вміщує послідовність записів однакової довжини, які розташовані у впорядкованому порядку, а структура індексного запису має такий вигляд: значення ключа першого запису блока, номер блока з цим записом.

Розріджений індекс будується для впорядкованих файлів. Файли з розрідженим індексом називаються **індексно-послідовними файлами**.

Індекси доцільно створювати по стовпцю або групі стовпців у таких випадках:

- часто виконується пошук у БД за атрибутами, які перелічені в умові WHERE оператора SELECT;
- часто виконується об'єднання таблиць за певними атрибутами;
- часто виконується сортування таблиць (використання ORDER BY в операторі SELECT).

Індекси не доцільно створювати по стовпцю або групі стовпців у таких випадках:

- містять значення, які часто змінюються, що призводить до частого оновлення індексу і, відповідно, уповільнює роботу;
 - не часто використовуються для пошуку;
 - містять незначну кількість значень.
- Як правило в БД визначають два види індексів:
- індекси, які використовуються запитами для доступу до даних;
 - індекси, які забезпечують посилкову цілісність між таблицями.

Більша частина БД підтримує В-дерева, крім того деякі з них працюють із хеш-таблицями. Деякі системи представляють багатомірні структури даних, такі, як варіанти R-дерев і квадрадерев (*kd*-дерева). Окремі системи підтримують також бітові вектори і багатотабличні індекси з'єднання. Системи, що розташовуються в оперативній пам'яті, крім того підтримують структури даних, які мають невеликі коефіцієнти розгалуження, такі як Т-дерева, бінарні дерева.

9.5. В-дерева

У багатьох СКБД для збереження індексів використовується структура даних, яка називається деревом. Серед дерев найбільш поширені бінарні дерева, В-дерева та їх різновиди (В + -дерева, В * -дерева тощо). В-дерево є збалансованим, впорядкованим за значенням ключа деревом. Найбільш розповсюдженими серед В-дерев є В + -дерева. На рис. 8.4 наведено фрагмент В + -дерева і його зв'язок з БД.

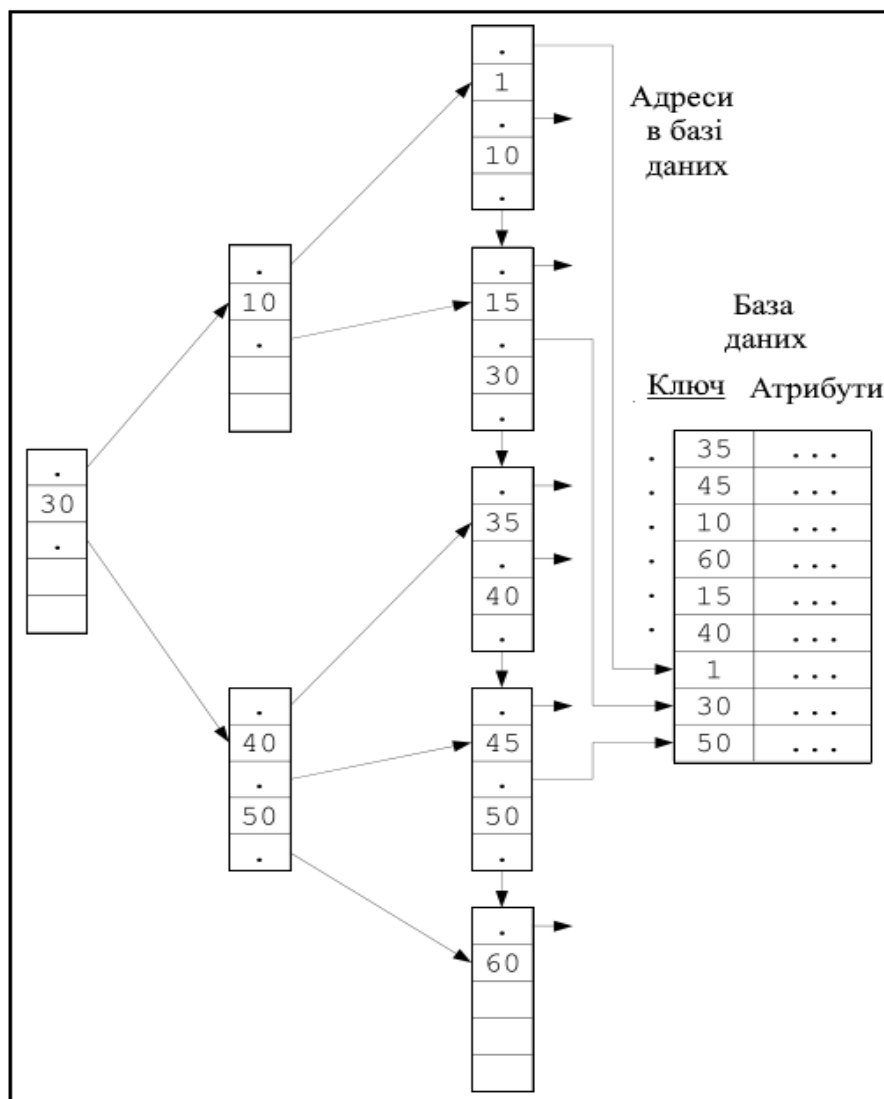


Рис. 9.4. Приклад побудови В + -дерева

9.6. Інвертовані файли

До сих пір ми розглядали структури даних, які використовувалися для прискорення доступу по первинному ключу. Однак досить часто в базах даних потрібно проводити операції доступу за вторинними ключами. Для забезпечення прискореного доступу за вторинними ключами використовуються структури, які називаються *інвертованими списками*. Інвертований список являє собою дворівневу індексну структуру. На першому рівні знаходиться файл або частина файлу, в який впорядковано розташовані значення вторинних ключів. Кожен запис із вторинним ключем має покажчик на номер першого блоку в ланцюгу блоків, які містять номери записів з даним значенням вторинного ключа.

На другому рівні знаходиться ланцюг блоків, які містять номери записів, що вміщують одне і те ж саме значення вторинного ключа. При цьому блоки другого рівня впорядковані за значенням вторинного ключа. На третьому рівні знаходиться безпосередньо файл даних. Пошук даних виконується у такій послідовності:

- на першому рівні знаходиться значення вторинного ключа;
- за знайденим покажчиком зчитується блок другого рівня, який містить номери всіх записів із заданим значенням вторинного ключа;
- у робочу область завантажується зміст всіх записів із заданим значенням вторинного ключа.

На рис. 9.5 представлений приклад інвертованого списку, складеного для вторинного ключа «Номер групи» в списку студентів деякого навчального закладу. Для більш наочного уявлення розмір блоку обмежений п'ятьма записами (цілими числами).

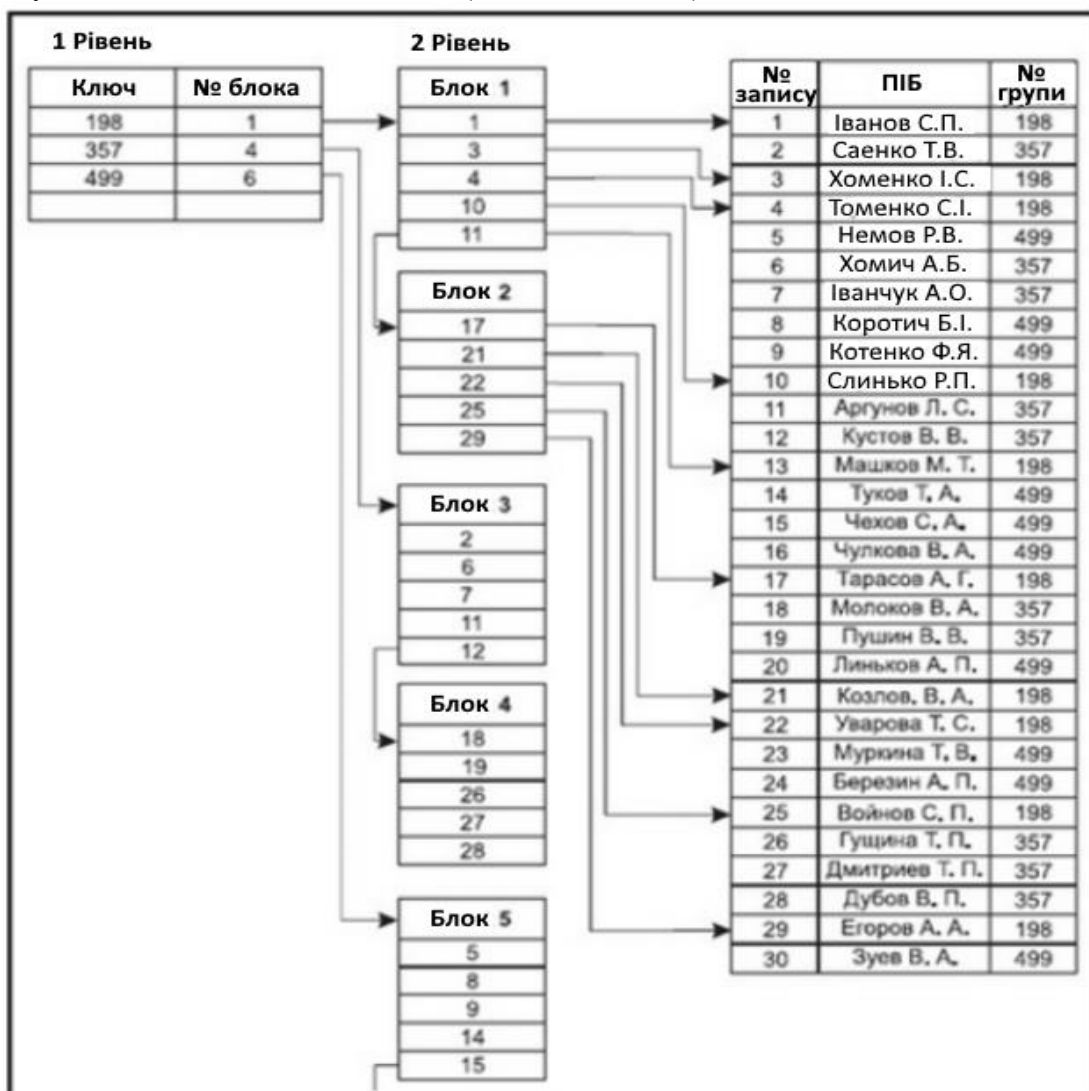


Рис. 9.5. Побудова інвертованого списку за номером групи для списку студентів

Для одного основного файлу може бути створено кілька інвертованих списків за різними вторинним ключам. Слід зазначити, що організація вторинних списків дійсно прискорює пошук записів із заданим значенням вторинного ключа. Але розглянемо питання модифікації основного файлу.

При модифікації основного файлу відбувається наступна послідовність дій:

1. Змінюється запис основного файлу.
2. Виключається старе посилання на попереднє значення вторинного ключа.
3. Додається нове посилання на нове значення вторинного ключа.

При цьому слід зазначити, що два останні кроки виконуються для всіх вторинних ключів, за якими створені інвертовані списки. І, зрозуміло, такий процес вимагає набагато більше тимчасових витрат, ніж просто зміна вмісту записи основного файлу без підтримки всіх інвертованих списків.

Тому не слід безумовно стверджувати, що введення індексних файлів (в тому числі і інвертованих списків) завжди прискорює обробку інформації в базі даних. Ні в якому разі, якщо база даних постійно змінюється, доповнюється, модифікується вміст записів, то наявність великої кількості інвертованих списків або індексних файлів по вторинним ключам може різко сповільнити процес обробки інформації.

Можна дотримуватися наступної позиції: якщо база даних досить стабільна і її вміст практично не змінюється, то побудова вторинних індексів дійсно може прискорити процес обробки інформації.

10. МОВА SQL. ФОРМУВАННЯ ЗАПИТІВ ДО БАЗИ ДАНИХ

Мова Структурованих Запитів (SQL – Structured Query Language) розроблена корпорацією IBM на початку 1970-х років. У 1986 році SQL був вперше стандартизований організацією ANSI.

SQL – це потужна і в той же час нескладна мова для управління базами даних. Вона підтримується практично всіма сучасними базами даних. SQL підрозділяється на два підмножини команд: DDL (Data Definition Language – мова визначення даних) і DML (Data Manipulation Language – мова обробки даних). Команди DDL використовуються для створення нових баз даних, таблиць і стовпців, а команди DML – для читання, запису, сортування, фільтрування, видалення даних.

Поточна версія стандарту мови SQL прийнята в 1992 р. Офіційна назва стандарту – Міжнародний стандарт мови баз даних SQL 1992 (International Standard Database Language SQL), неофіційна назва – SQL / 92, або SQL-92, або SQL2. Документ, що описує стандарт, містить понад 600 сторінок. Ми дамо тільки деякі поняття мови.

Мова SQL стала фактично стандартною мовою доступу до баз даних. Всі СКБД, які претендують на назву «реляційні», реалізують той чи інший діалект SQL. Багато нереляційні системи також мають в даний час засоби доступу до реляційних даних. Метою стандартизації є переносимість додатків між різними СКБД.

Потрібно зауважити, що в даний час, жодна система не реалізує стандарт SQL в повному обсязі. Крім того, у всіх діалектах мови є можливості, які не є стандартними. Таким чином, можна сказати, що кожен діалект – це надмножина деякого підмножини стандарту SQL. Це ускладнює переносимість додатків, розроблених для одних СКБД в інші СКБД. Мова SQL оперує термінами, дещо відмінними від термінів реляційної теорії, наприклад, замість «відношення» використовуються «таблиці», замість «кортежів» – «рядки», замість «атрибутів» – «колонки» або «стовпчики».

Мова SQL є реляційно повною. Це означає, що будь-який оператор реляційної алгебри може бути виражений відповідним оператором SQL.

10.1. Оператори SQL

Основу мови SQL складають оператори, умовно розбиті не кілька груп по виконуваних функціях. Можна виділити наступні групи операторів.

Оператори DDL (Data Definition Language) – оператори визначення об'єктів бази даних:

- CREATE SCHEMA – створити схему бази даних;
- DROP SHEMA – видалити схему бази даних;
- CREATE TABLE – створити таблицю;
- ALTER TABLE – змінити таблицю;
- DROP TABLE – видалити таблицю;
- CREATE DOMAIN – створити домен;
- ALTER DOMAIN – змінити домен;
- DROP DOMAIN – видалити домен;
- CREATE COLLATION – створити послідовність;
- DROP COLLATION – видалити послідовність;
- CREATE VIEW – створити уявлення;
- DROP VIEW – видалити уявлення.

Оператори DML (Data Manipulation Language) – оператори маніпулювання даними:

- SELECT – відібрати рядки з таблиць;
- INSERT – додати рядки в таблицю;
- UPDATE – змінити рядки в таблиці;
- DELETE – видалити рядки в таблиці;

- COMMIT – зафіксувати внесені зміни;
- ROLLBACK – відкотити внесені зміни.

Оператори захисту і управління даними:

- CREATE ASSERTION – створити обмеження;
- DROP ASSERTION – видалити обмеження;
- GRANT – надати привілеї користувачу або додатку на маніпулювання об'єктами;
- REVOKE – скасувати привілеї користувача або додатки.

Крім того, є групи операторів установки параметрів сеансу, отримання інформації про базу даних, оператори статичного SQL, оператори динамічного SQL.

Найбільш важливими для користувача є оператори маніпулювання даними.

10.2. Приклади використання операторів маніпулювання даними

10.2.1. Приклади використання операторів INSERT і UPDATE

INSERT – вставка рядків в таблицю.

Приклад 1. Вставка одного рядка в таблицю:

```
INSERT INTO
P (PNUM, PNAME)
VALUES (4, "Іванов");
```

Приклад 2. Вставка в таблицю кількох рядків, обраних з іншої таблиці (в таблицю TMP_TABLE вставляються дані про постачальників з таблиці P, мають номери, великі 2):

```
INSERT INTO
TMP_TABLE (PNUM, PNAME)
SELECT PNUM, PNAME
FROM P
WHERE P.PNUM > 2;
```

UPDATE – оновлення рядків в таблиці.

Приклад 3. Оновлення декількох рядків в таблиці:

```
UPDATE P
SET PNAME = "Пушник"
WHERE P.PNUM = 1;
```

DELETE – видалення рядків в таблиці

Приклад 4. Видалення кількох рядків у таблиці:

```
DELETE FROM P
WHERE P.PNUM = 1;
```

Приклад 5. Видалення всіх рядків в таблиці:

```
DELETE FROM P;
```

10.2.2. Приклади використання оператора SELECT

Оператор SELECT є фактично найважливішим для користувача і найскладнішим оператором SQL. Він призначений для вибірки даних з таблиць, тобто він, власне, і реалізує одне з основних призначень бази даних – надавати інформацію користувачеві.

Оператор SELECT завжди виконується над деякими таблицями, що входять в базу даних. Насправді в базах даних можуть бути не тільки постійно збережені таблиці, а також тимчасові таблиці і так звані уявлення.

Представлення – SELECT-вирази, що зберігаються в базі. З точки зору користувачів представлення – це таблиця, яка не зберігається постійно в базі даних, а «виникає» в момент звернення до неї. З точки зору оператора SELECT і постійно збережені таблиці, і тимчасові таблиці та подання виглядають абсолютно однаково. Звичайно, при реальному виконанні оператора SELECT системою враховуються відмінності між збереженими таблицями і уявленнями, але ці відмінності приховані від користувача.

Результатом виконання оператора SELECT завжди є таблиця. Таким чином, за результатами дій оператор SELECT схожий на оператори реляційної алгебри. Будь-оператор реляційної алгебри може бути виражений відповідним чином сформульованим оператором SELECT. Складність оператора SELECT визначається тим, що він містить в собі всі можливості реляційної алгебри, а також додаткові можливості, яких в реляційній алгебрі немає.

10.2.2.1. Відбір даних з однієї таблиці

Приклад 6. Вибрати всі дані з таблиці постачальників (ключові слова SELECT FROM):

```
SELECT *
FROM P;
```

В результаті отримаємо нову таблицю, яка містить повну копію даних з вихідної таблиці P.

Приклад 7. Вибрати всі рядки з таблиці постачальників, які задовольняють деякому умові (ключове слово WHERE):

```
SELECT *
FROM P
WHERE P.PNUM > 2;
```

Як умова в розділі WHERE можна використовувати складні логічні вирази, що використовують поля таблиць, константи, порівняння (>, <, = і т.д.), дужки, союзи AND і OR, заперечення NOT.

Приклад 8. Вибрати деякі колонки з вихідної таблиці (вказівка списку відбираються колонок):

```
SELECT P.NAME
FROM P;
```

В результаті отримаємо таблицю з одного колонкою, що містить всі найменування постачальників.

Якщо у вихідній таблиці були присутні кілька постачальників з різними номерами, але однаковими найменуваннями, то в результуючій таблиці будуть рядки з повтореннями – дублікати рядків автоматично, не відкидаються.

Приклад 9. Вибрати деякі колонки з вихідної таблиці, видаливши з результату повторювані рядки (ключове слово DISTINCT):

```
SELECT DISTINCT P.NAME
FROM P;
```

Використання ключового слова DISTINCT призводить до того, що в результуючій таблиці будуть видалені всі повторювані рядки.

Приклад 10. Використання скалярних виразів і перейменувань колонок в запитах (ключове слово AS):

```
SELECT
    TOVAR.TNAME,
    TOVAR.KOL,
    TOVAR.PRICE,
    "=" AS EQU,
    TOVAR.KOL * TOVAR.PRICE AS SUMMA
FROM TOVAR;
```

В результаті отримаємо таблицю з колонками, яких не було у вихідній таблиці TOVAR.

TNAME	KOL	PRICE	EQU	SUMMA
Болт	10	100	=	1000
гайка	20	200	=	4000
гвинт	30	300	=	9000

Приклад 11. Впорядкування результатів запиту (ключове слово ORDER BY):

```
SELECT
  PD.PNUM,
  PD.DNUM,
  PD.VOLUME
FROM PD
ORDER BY DNUM;
```

В результаті отримаємо наступну таблицю, впорядковану по полю DNUM:

PNUM	DNUM	VOLUME
1	1	100
2	1	150
3	1	1000
1	2	200
2	2	250
1	3	300

Приклад 12. Впорядкування результатів запиту по декількох полях зі зростанням або убутанням (ключові слова ASC, DESC):

```
SELECT
  PD.PNUM,
  PD.DNUM,
  PD.VOLUME
FROM PD
ORDER BY
  DNUM ASC,
  VOLUME DESC;
```

В результаті отримаємо таблицю, в якій рядки йдуть в порядку зростання значення поля DNUM, а рядки, з однаковим значенням DNUM йдуть в порядку убутання значення поля VOLUME:

PNUM	DNUM	VOLUME
3	1	1000
2	1	150
1	1	100
2	2	250
1	2	200
1	3	300

Якщо явно не вказані ключові слова ASC або DESC, то за замовчуванням приймається впорядкування по зростанню (ASC).

10.2.2.2. Відбір даних з декількох таблиць

Приклад 13. Природне з'єднання таблиць (спосіб 1 – явне вказівку умов з'єднання):

```
SELECT
  P.PNUM,
  P.PNAME,
  PD.DNUM,
  PD.VOLUME
FROM P, PD
WHERE P.PNUM = PD.PNUM;
```

В результаті отримаємо нову таблицю, в якій рядки з даними про постачальників з'єднані з рядками з даними про постачання деталей:

PNUM	PNAME	DNUM	VOLUME
1	Іванов	1	100
1	Іванов	2	200
1	Іванов	3	300
2	Петров	1	150
2	Петров	2	250
3	Сидоров	1	1000

Сполучаються таблиці перераховані в розділі FROM оператора, умова з'єднання наведено в розділі WHERE. Розділ WHERE, крім умови з'єднання таблиць, може також містити і умови відбору рядків.

Приклад 14. Природне з'єднання таблиць (спосіб 2 – ключові слова JOIN USING):

```
SELECT
  P.PNUM,
  P.PNAME,
  PD.DNUM,
  PD.VOLUME
FROM P JOIN PD USING PNUM;
```

Ключове слово USING дозволяє явно вказати, за якими із загальних колонок таблиць буде проводитися з'єднання.

Приклад 15. Природне з'єднання таблиць (спосіб 3 – ключове слово NATURAL JOIN):

```
SELECT
  P.PNUM,
  P.PNAME,
  PD.DNUM,
  PD.VOLUME
FROM P NATURAL JOIN PD;
```

У розділі FROM не вказано, по яких полях здійснюється з'єднання. NATURAL JOIN автоматично з'єднує по всім однаковим полям у таблицях.

Приклад 16. Природне з'єднання трьох таблиць:

```
SELECT
  P.PNAME,
  D.DNAME,
  PD.VOLUME
FROM
  P NATURAL JOIN PD NATURAL JOIN D;
```

В результаті отримаємо наступну таблицю:

PNAME	DNAME	VOLUME
Іванов	Болт	100
Іванов	гайка	200
Іванов	гвинт	300
Петров	Болт	150
Петров	гайка	250
Сидоров	Болт	1000

Приклад 17. Прямий добуток таблиць:

```
SELECT
  P.PNUM,
  P.PNAME,
  D.DNUM,
  D.DNAME
FROM P, D;
```

В результаті отримаємо наступну таблицю:

PNUM	PNAME	DNUM	DNAME
1	Іванов	1	Болт
1	Іванов	2	Гайка
1	Іванов	3	Гвинт
2	Петров	1	Болт
2	Петров	2	Гайка
2	Петров	3	Гвинт
3	Сидоров	1	Болт
3	Сидоров	2	Гайка
3	Сидоров	3	Гвинт

Так як не вказано умова з'єднання таблиць, то кожен рядок першої таблиці з'єднається з кожним рядком другої таблиці.

Приклад 18. З'єднання таблиць за довільною умовою.

Розглянемо деяку компанію, в якій зберігаються дані про постачальників і поставляються деталі. Нехай постачальникам (P, табл. 10.1) і деталей (D, табл. 10.2) присвоєно якийсь статус. Бізнес компанії організований таким чином, що постачальники мають право поставляти тільки ті деталі, статус яких не вище статусу постачальника.

Сенс цього може бути в тому, що хороший постачальник з високим статусом може поставляти більше різновидів деталей, а поганий постачальник з низьким статусом може постачати тільки обмежений список деталей, важливість яких (статус деталі) не надто висока.

Таблиця 10.1. Відношення (таблиця) P (Постачальники)

PNUM	PNAME	PSTATUS
1	Іванов	4
2	Петров	1
3	Сидоров	2

Таблиця 10.2. Відношення (таблиця) D (Деталі)

DNUM	DNAME	DSTATUS
1	Болт	3
2	гайка	2
3	гвинт	1

Відповідь на питання «які постачальники мають право поставляти які деталі?» дає наступний запит:

```
SELECT
```

```

P.PNUM,
P.PNAME,
P.PSTATUS,
D.DNUM,
D.DNAME,
D.DSTATUS
FROM P, D
WHERE P.PSTATUS >= D.DSTATUS;

```

У результаті отримаємо наступну таблицю:

PNUM	PNAME	PSTATUS	DNUM	DNAME	DSTATUS
1	Іванов	4	1	Болт	3
1	Іванов	4	2	Гайка	2
1	Іванов	4	3	Гвинт	1
2	Петров	1	3	Гвинт	1
3	Сидоров	2	2	Гайка	2
3	Сидоров	2	3	Гвинт	1

10.2.2.3. Використання імен кореляції (аліасів, псевдонімів)

Іноді доводиться виконувати запити, в яких таблиця з'єднується сама з собою, або одна таблиця з'єднується двічі з іншою таблицею. При цьому використовуються імена кореляції (аліаси, псевдоніми), які дозволяють розрізняти копії таблиць, які сполучаються. Імена кореляції вводяться в розділі FROM і йдуть через пробіл після імені таблиці. Імена кореляції повинні використовуватися в якості префікса перед ім'ям стовпця і відокремлюються від імені стовпця точкою. Якщо в запиті вказуються одні й ті ж поля з різних примірників однієї таблиці, вони повинні бути перейменовані для усунення неоднозначності в найменуваннях колонок результуючої таблиці. Визначення імені кореляції діє тільки під час виконання запиту.

Приклад 19. Відібрати всі пари постачальників таким чином, щоб перший постачальник в парі мав статус, більший статусу другої постачальника:

```

SELECT
  P1.PNAME AS PNAME1,
  P1.PSTATUS AS PSTATUS1,
  P2.PNAME AS PNAME2,
  P2.PSTATUS AS PSTATUS2
FROM
  P P1, P P2
WHERE P1.PSTATUS1 > P2.PSTATUS2;

```

В результаті отримаємо наступну таблицю:

PNAME1	PSTATUS1	PNAME2	PSTATUS2
Іванов	4	Петров	1
Іванов	4	Сидоров	2
Сидоров	2	Петров	1

Приклад 20. Розглянемо ситуацію, коли деякі постачальники (назвемо їх контрагенти) можуть виступати як в якості постачальників деталей, так і в якості одержувачів. Таблиці, що зберігають дані, можуть мати такий вигляд:

Таблиця 10.3. Відношення (таблиця) CONTRAGENTS

Номер контрагента NUM	Найменування контрагента NAME
1	Іванов
2	Петров
3	Сидоров

Таблиця 10.4. Відношення (таблиця) DETAILS (Деталі)

Номер деталі DNUM	Найменування деталі DNAME
1	Болт
2	Гайка
3	Гвинт

Таблиця 10.5. Відношення (таблиця) CD (Поставки)

Номер постачальника PNUM	Номер отримувача CNUM	Номер деталі DNUM	Обладнання, що поставляється кількість VOLUME
1	2	1	100
1	3	2	200
1	3	3	300
2	3	1	150
2	3	2	250
3	1	1	1000

У таблиці CD (поставки) поля PNUM і CNUM є зовнішніми ключами, що посилаються на потенційний ключ NUM в таблиці CONTRAGENTS.

Відповідь на питання «хто кому що в якій кількості поставляє» дається наступним запитом:

```
SELECT
  P.NAME AS PNAME,
  C.NAME AS CNAME,
  DETAILS.DNAME,
  CD.VOLUME
FROM
  CONTRAGENTS P,
  CONTRAGENTS C,
  DETAILS,
  CD
WHERE
  P.NUM = CD.PNUM AND
  C.NUM = CD.CNUM AND
  D.DNUM = CD.DNUM;
```

В результаті отримаємо наступну таблицю:

Найменування постачальника PNAME	Найменування отримувача CNAME	Найменування деталі DNAME	Обладнання, що поставляється кількість VOLUME
Іванов	Петров	Болт	100
Іванов	Сидоров	Гайка	200
Іванов	Сидоров	Гвинт	300
Петров	Сидоров	Болт	150
Петров	Сидоров	Гайка	250
Сидоров	Іванов	Болт	1000

Цей же запит може бути виражений дуже великою кількістю способів, наприклад, так:

```
SELECT
  P.NAME AS PNAME,
  C.NAME AS CNAME,
  DETAILS.DNAME,
  CD.VOLUME
FROM
  CONTRAGENTS P,
  CONTRAGENTS C,
  DETAILS NATURAL JOIN CD
WHERE
  P.NUM = CD.PNUM AND
  C.NUM = CD.CNUM;
```

10.2.2.4. Використання агрегатних функцій в запитах

Приклад 21. Отримати загальну кількість постачальників (ключове слово COUNT):

```
SELECT COUNT (*) AS N
FROM P;
```

В результаті отримаємо таблицю з одним стовпцем і одним рядком, що містить кількість рядків з таблиці P:

N
3

Приклад 22. Отримати загальне, максимальне, мінімальне та середнє кількості поставляються деталей (ключові слова SUM, MAX, MIN, AVG):

```
SELECT
  SUM (PD.VOLUME) AS SM,
  MAX (PD.VOLUME) AS MX,
  MIN (PD.VOLUME) AS MN,
  AVG (PD.VOLUME) AS AV
FROM PD;
```

В результаті отримаємо наступну таблицю з одним рядком:

SM	MX	MN	AV
2000	1000	100	333.33333333

10.2.2.5. Використання агрегатних функцій з угрупованнями

Приклад 23. Для кожної деталі отримати сумарне поставляється кількість (ключове слово GROUP BY):

```
SELECT
  PD.DNUM,
  SUM (PD.VOLUME) AS SM
GROUP BY PD.DNUM;
```

Цей запит буде виконуватися наступним чином. Спочатку рядки вихідної таблиці будуть згруповані так, щоб в кожну групу потрапили рядки з однаковими значеннями DNUM. Потім усередині кожної групи буде підсумовано поле VOLUME. Від кожної групи в результуючу таблицю буде включений один рядок:

DNUM	SM
1	1250
2	450
3	300

У списку відібраних полів оператора SELECT, що містить розділ GROUP BY можна включати тільки агрегатні функції і поля, які входять в умову угруповання. Наступний запит видасть синтаксичну помилку:

```
SELECT
  PD.PNUM,
  PD.DNUM,
  SUM (PD.VOLUME) AS SM
GROUP BY PD.DNUM;
```

Причина помилки в тому, що в список полів, які відбираються включено поле PNUM, яке не входить в розділ GROUP BY. І дійсно, в кожну отриману групу рядків може входити кілька рядків з різними значеннями поля PNUM. З кожної групи рядків буде сформовано по одному підсумковому рядку. При цьому немає однозначної відповіді на питання, яке значення вибрати для поля PNUM в підсумковому рядку.

Деякі діалекти SQL не вважають це за помилку. Запит буде виконаний, але передбачити, які значення будуть внесені в поле PNUM в результуючій таблиці, неможливо.

Приклад 24. Отримати номери деталей, що поставляється, сумарна кількість яких перевищує 400 (ключове слово HAVING):

Умова, що сумарна кількість, що поставляється повинна бути більше 400 не може бути сформульовано в розділі WHERE, тому що в цьому розділі можна використовувати агрегатні функції.

Умови, які використовують агрегатні функції повинні бути розміщені в спеціальному розділі HAVING:

```
SELECT
  PD.DNUM,
  SUM (PD.VOLUME) AS SM
GROUP BY PD.DNUM
HAVING SUM (PD.VOLUME)> 400;
```

У результаті отримаємо наступну таблицю:

DNUM	SM
1	1250
2	450

В одному запиті можуть зустрітися як умови відбору рядків в розділі WHERE, так і умови відбору груп в розділі HAVING. Умови відбору груп не можна перенести з розділу HAVING в розділ WHERE. Аналогічно і умови відбору рядків не можна перенести з розділу WHERE в розділ HAVING, за винятком умов, що включають поля зі списку угрупування GROUP BY.

10.2.2.6. Використання підзапитів

Дуже зручним засобом, що дозволяє формувати запити більш зрозумілим чином, є можливість використання підзапитів, вкладених в основний запит.

Приклад 25. Отримати список постачальників, статус яких менше максимального статусу в таблиці постачальників (порівняння з підзапитом):

```
SELECT *  
FROM P  
WHERE P.STATYS <  
(SELECT MAX (P.STATUS)  
FROM P);
```

Так як поле P.STATUS порівнюється з результатом підзапиту, то підзапит повинен бути сформульований так, щоб повертати таблицю, що складається рівно з одного рядка і однієї колонки.

Результат виконання запиту буде еквівалентний результату наступної послідовності дій:

Виконати один раз вкладений підзапит і отримати максимальне значення статусу. Просканувати таблицю постачальників P, кожен раз порівнюючи значення статусу постачальника з результатом підзапиту, і відібрати тільки ті рядки, в яких статус менше максимального.

Приклад 26. Використання предиката IN. Отримати список постачальників, що поставляють деталь номер 2:

```
SELECT *  
FROM P  
WHERE P.PNUM IN  
(SELECT DISTINCT PD.PNUM  
FROM PD  
WHERE PD.DNUM = 2);
```

В даному випадку вкладений підзапит може повертати таблицю, що містить кілька рядків. Результат виконання запиту буде еквівалентний результату наступної послідовності дій:

1. Виконати один раз вкладений підзапит і отримати список номерів постачальників, що поставляють деталь номер 2.
2. Просканувати таблицю постачальників P, щоразу перевіряючи, чи міститься номер постачальника в результаті підзапиту.

Приклад 27. Використання предиката EXIST. Отримати список постачальників, що поставляють деталь номер 2:

```
SELECT *  
FROM P  
WHERE EXIST  
(SELECT *  
FROM PD  
WHERE  
PD.PNUM = P.PNUM AND  
PD.DNUM = 2);
```

Результат виконання запиту буде еквівалентний результату наступної послідовності дій:

1. Просканувати таблицю постачальників P, кожен раз виконуючи підзапит з новим значенням номера постачальника, узятим з таблиці P.
2. В результат запиту включити тільки ті рядки з таблиці постачальників, для яких вкладений підзапит повернув непорожня множина рядків.

На відміну від двох попередніх прикладів, вкладений підзапит містить параметр (зовнішнє посилання), який передається з основного запиту – номер постачальника P.PNUM. Такі підзапити називаються корелюється (correlated). Зовнішнє посилання може набувати різних значень для кожного рядка-кандидата, що оцінюється за допомогою підзапиту, тому підзапит повинен виконуватися заново для кожного рядка, що відбирається в основному запиті. Такі підзапити характерні для предиката EXIST, але можуть бути використані і в інших підзапитах.

Може здатися, що запити, що містять корелюється підзапити будуть виконуватися повільніше, ніж запити з некорельованих підзапитах. Насправді це не так, тому що то, як користувач, сформулював запит, не визначає, як цей запит буде виконуватися. Мова SQL є непроцедурного, а декларативним. Це означає, що користувач, який формулює запит, просто описує, яким повинен бути результат запиту, а як цей результат буде отримано – за це відповідає сама СКБД.

Приклад 28. Використання предиката NOT EXIST. Отримати список постачальників, які не постачають деталь номер 2:

```
SELECT *
FROM P
WHERE NOT EXIST
(SELECT *
FROM PD
WHERE
PD.PNUM = P.PNUM AND
PD.DNUM = 2);
```

Також, як і в попередньому прикладі, тут використовується корелюється підзапит. Відмінність в тому, що в основному запиті будуть відібрані ті рядки з таблиці постачальників, для яких вкладений підзапит не видасть жодного рядка.

Приклад 29. Отримати імена постачальників, що поставляють всі деталі:

```
SELECT DISTINCT PNAME
FROM P
WHERE NOT EXIST
(SELECT *
FROM D
WHERE NOT EXIST
(SELECT *
FROM PD
WHERE
PD.DNUM = D.DNUM AND
PD.PNUM = P.PNUM));
```

Даний запит містить два вкладених підзапиту і реалізує реляційну операцію ділення відношень.

Найбільш внутрішній підзапит параметризований двома параметрами (D.DNUM, P.PNUM) і має наступний сенс: відібрати всі рядки, що містять дані про постачання постачальника з номером PNUM деталі з номером DNUM. Заперечення NOT EXIST говорить про те, що даний постачальник не поставляє дану деталь. Зовнішній до нього підзапит, сам є вкладеним і параметризованим параметром P.PNUM, має сенс: відібрати список деталей, які не поставляються постачальником PNUM. Заперечення NOT EXIST говорить про те, що для постачальника з номером PNUM не повинно бути деталей, які не поставлялися б цим постачальником. Це в точності означає, що в зовнішньому запиті відбираються тільки постачальники, що поставляють всі деталі.

10.2.2.6. Використання об'єднання, перетину і різниці

Приклад 30. Отримати імена постачальників, що мають статус, більший 3 або постачають хоча б одну деталь номер 2 (об'єднання двох підзапитів – ключове слово UNION):

```

SELECT P.PNAME
FROM P
WHERE P.STATUS > 3
UNION
SELECT P.PNAME
FROM P, PD
WHERE P.PNUM = PD.PNUM AND
      PD.DNUM = 2;

```

Результуючі таблиці об'єднуються запитів повинні бути сумісні, тобто мати однакову кількість стовпців і однакові типи стовпців в порядку їх перерахування. Хай не буде для об'єднуються таблиці мали б однакові імена колонок. Це відрізняє операцію об'єднання запитів в SQL від операції об'єднання в реляційної алгебри. Найменування колонок в результуючому запиті будуть автоматично взяті з результату першого запиту в об'єднанні.

Приклад 31. Отримати імена постачальників, що мають статус, більший 3 і одночасно поставляють хоча б одну деталь номер 2 (перетин двох підзапитів – ключове слово INTERSECT):

```

SELECT P.PNAME
FROM P
WHERE P.STATUS > 3
INTERSECT
SELECT P.PNAME
FROM P, PD
WHERE P.PNUM = PD.PNUM AND
      PD.DNUM = 2;

```

Приклад 32. Отримати імена постачальників, що мають статус, більший 3, за винятком тих, хто поставляє хоча б одну деталь номер 2 (різниця двох підзапитів – ключове слово EXCEPT):

```

SELECT P.PNAME
FROM P
WHERE P.STATUS > 3
EXCEPT
SELECT P.PNAME
FROM P, PD
WHERE P.PNUM = PD.PNUM AND
      PD.DNUM = 2;

```

10.2.3. Синтаксис оператора вибірки даних (SELECT)

10.2.3.1. BNF-нотація

Наведемо синтаксис оператора вибірки даних (оператора SELECT) більш точно. При описі синтаксису операторів звичайно використовуються умовні позначення, відомі як стандартні форми Бекуса-Наура (BNF).

У BNF позначеннях використовуються наступні елементи:

1. Символ « $\therefore$ » означає рівність за визначенням. Зліва від знаку розміщується поняття, яке визначається, праворуч – власне визначення поняття.
2. Ключові слова записуються прописними буквами. Вони зарезервовані і складають частину оператора.
3. Мітки-заповнювачі конкретних значень елементів і змінних записуються курсивом.
4. Необов'язкові елементи оператора укладені в квадратні дужки [].
5. Вертикальна риса | вказує на те, що всі попередні їй елементи списку є необов'язковими і можуть бути замінені будь-яким іншим елементом списку після цієї риси.
6. Фігурні дужки {} вказують на те, що все знаходиться всередині них є єдиним цілим.
7. Три крапки « $\dots$ » означає, що попередня частина оператора може бути повторена будь-яку кількість разів.

8. Три крапки, всередині якого знаходиться кома «...» вказують, що попередня частина оператора, що складається з декількох елементів, розділених комами, може мати довільне число повторень. Кому не можна ставити після останнього елемента. Зауваження: дана угода не входить в стандарт BNF, але дозволяє більш точно описати синтаксис операторів SQL.
9. Круглі дужки () є елементом оператора.

10.2.3.2. Синтаксис оператора вибірки

У досить спрощеному вигляді оператор вибірки даних має наступний синтаксис (для деяких елементів ми дамо НЕ BNF-визначення, а словесний опис):

оператор вибірки:: =

табличне вираження

[ORDER BY { {Ім'я стовпця-результату [ASC | DESC]} | {Позитивне ціле [ASC | DESC]} } ., ..];

табличне вираження:: =

Select-вираз

[{UNION | INTERSECT | EXCEPT} [ALL] {Select-вираз | TABLE Ім'я таблиці | Конструктор значень таблиці}]

Select-вираз:: =

SELECT[ALL | DISTINCT] { { {Скалярний вираз | Функція агрегування | Select-вираз} [AS Ім'я стовпця]} ., .. } | { {Ім'я таблиці | Ім'я кореляції} . *} | *

FROM{ {Ім'я таблиці [AS] [Ім'я кореляції] [(Ім'я стовпця., ..)] } | {Select-вираз [AS] Ім'я кореляції [(Ім'я стовпця., ..)] } | Поєднана таблиця} ., .. [WHERE Умовний вираз] [GROUP BY { {Ім'я таблиці | Ім'я кореляції} .} Ім'я стовпця} ., ..] [HAVING Умовний вираз]

Select-вираз в розділі SELECT, що використовується в якості значення для стовпчика, відбирається, має повертати таблицю, що складається з одного рядка і одного стовпця, тобто скалярний вираз.

Умовний вираз в розділі WHERE має обчислюватися для кожного рядка, що є кандидатом в результуюче безліч рядків. В цьому умовному вираженні можна використовувати вкладені запити. Синтаксис умовних виразів, допустимих в розділі WHERE розглядається нижче.

Розділ HAVING містить умовний вираз, що обчислюється для кожної групи, яка визначається списком угруповання в розділі GROUP BY. Це умовний вираз може містити функції агрегування, обчислювані для кожної групи. Умовний вираз, сформульоване в розділі WHERE, може бути перенесено в розділ HAVING. Перенесення умов з розділу HAVING в розділ WHERE неможливий, якщо умовний вираз містить агрегатні функції. Перенесення умов з розділу WHERE в розділ HAVING є поганим стилем програмування – ці розділи призначені для різних за змістом умов (умови для рядків і умови для груп рядків).

Якщо в розділі SELECT присутні агрегатні функції, то вони обчислюються по-різному в залежності від наявності розділу GROUP BY. Якщо розділ GROUP BY відсутній, то результат запиту повертає не більше одного рядка. Агрегатні функції обчислюються по всіх рядках, що задовольняє умовного виразу в розділі WHERE. Якщо розділ GROUP BY присутній, то агрегатні функції обчислюються окремо для кожної групи, визначеної в розділі GROUP BY.

Скалярний вираз – в якості скалярних виразів в розділі SELECT можуть виступати або імена стовпців таблиць, що входять в розділ FROM, або прості функції, які повертають скалярні значення.

функція агрегування:: =

COUNT(*) | { {COUNT | MAX | MIN | SUM | AVG} ([ALL | DISTINCT] Скалярний вираз)}

Конструктор значень таблиці:: =

VALUES Конструктор значень рядка., ..

Конструктор значень рядка:: =

елемент конструктора | (Елемент конструктора., ..) | Select-вираз

Зауваження. Select-вираз, що використовується в конструкторі значень рядка, зобов'язана повертати рівно один рядок.

елемент конструктора:: =

Вираз для обчислення значення | NULL | DEFAULT

10.2.3.3. Синтаксис з'єднаних таблиць

У розділі FROM оператора SELECT можна використовувати з'єднані таблиці. Нехай в результаті деяких операцій ми отримуємо таблиці A і B. Такими операціями можуть бути, наприклад, оператор SELECT або інша поєднана таблиця. Тоді синтаксис з'єднаної таблиці має наступний вигляд:

поєднана таблиця:: =

перехресне з'єднання

| Природне з'єднання | З'єднання за допомогою предиката | З'єднання за допомогою імен стовпців |
з'єднання об'єднання

Тип з'єднання:: =

INNER

| LEFT [OUTER] | RIGTH [OUTER] | FULL [OUTER]

перехресне з'єднання:: =

Таблиця ACROSS JOIN Таблиця B

природне з'єднання:: =

Таблиця A[NATURAL] [Тип з'єднання] JOIN Таблиця B

З'єднання за допомогою предиката:: =

Таблиця A[Тип з'єднання] JOIN Таблиця B ON Предикат

З'єднання за допомогою імен стовпців:: =

Таблиця A[Тип з'єднання] JOIN Таблиця B USING (Ім'я стовпця., ..)

з'єднання об'єднання:: =

Таблиця AUNION JOIN Таблиця B

Опишемо використовувані терміни.

CROSS JOIN – Перехресне з'єднання повертає просто декартовий добуток таблиць. Таке з'єднання в розділі FROM може бути замінено списком таблиць через кому.

NATURAL JOIN – Природне з'єднання проводиться за всіма стовпцями таблиць A і B, які мають однакові імена. У результуючу таблицю однакові стовпці вставляються тільки один раз.

JOIN $\perp$ ON – З'єднання за допомогою предиката з'єднує рядки таблиць A і B за допомогою вказаного предиката.

JOIN $\perp$ USING – З'єднання за допомогою імен стовпців з'єднує відношення подібно до природного з'єднання по тим загальним стовпцям таблиць A і B, які вказані в списку USING.

OUTER – Ключове слово OUTER (зовнішній) не є обов'язковими, воно не використовується ні в яких операціях з даними.

INNER – Тип з'єднання «внутрішнє». Внутрішній тип з'єднання використовується за умовчанням, коли тип явно не заданий. У таблицях A і B з'єднуються тільки ті рядки, для яких знайдено збіг.

LEFT (OUTER) – Тип з'єднання «ліве (зовнішнє)». Ліве з'єднання таблиць A і B включає в себе всі рядки з лівої таблиці A і ті рядки з правої таблиці B, для яких виявлено збіг. Для рядків з таблиці A, для яких не знайдено відповідності в таблиці B, в стовпці, що витягають із таблиці B, заносяться значення NULL.

RIGHT (OUTER) – Тип з'єднання «праве (зовнішнє)». Праве з'єднання таблиць A і B включає в себе всі рядки з правої таблиці B і ті рядки з лівої таблиці A, для яких виявлено збіг. Для рядків з таблиці B, для яких не знайдено відповідності в таблиці A, в стовпці, що витягають із таблиці A заносяться значення NULL.

FULL (OUTER) – Тип з'єднання «повне (зовнішнє)». Це комбінація лівого і правого з'єднань. В повне з'єднання включаються всі рядки з обох таблиць. Для співпадаючих рядків поля заповнюються реальними значеннями, для незбіжних рядків поля заповнюються відповідно до правил лівого і правого з'єднань.

UNION JOIN – З'єднання об'єднання є зворотним по відношенню до внутрішнього з'єднання. Воно включає тільки ті рядки з таблиць А і В, для яких не знайдено збігів. У них використовуються значення NULL для стовпців, отриманих з іншої таблиці. Якщо взяти повне зовнішнє з'єднання і видалити з нього рядки, отримані в результаті внутрішнього сполучення, то вийде з'єднання об'єднання.

Використання з'єднаних таблиць часто полегшує сприйняття оператора SELECT, особливо, коли використовується природне з'єднання. Якщо не використовувати з'єднані таблиці, то при виборі даних з декількох таблиць необхідно явно вказувати умови з'єднання в розділі WHERE. Якщо при цьому користувач вказує складні критерії відбору рядків, то в розділі WHERE змішуються семантично різні поняття – як умови зв'язку таблиць, так і умови відбору рядків (див. Приклади 13, 14, 15 цієї лекції).

10.2.3.4. Синтаксис умовних виразів розділу WHERE

Умовний вираз, що використовується в розділі WHERE оператора SELECT має обчислюватися для кожного рядка-кандидата, що відбирається оператором SELECT. Умовний вираз може повертати одне з трьох значень істинності: TRUE, FALSE або UNKNOWN. Рядок-кандидат відбирається в результуюче безліч рядків тільки в тому випадку, якщо для неї умовний вираз повернуло значення TRUE.

Умовні вирази мають наступний синтаксис (з метою спрощення викладу наведені не всі можливі предикати):

умовний вираз:: = [(*[NOT]* {Предикат порівняння | Предикат between | Предикат in | Предикат like | Предикат null | Предикат кількісного порівняння | Предикат exist | Предикат unique | Предикат match | Предикат overlaps} [*{AND | OR}* Умовний вираз] *[)]* [*IS [NOT]* {TRUE | FALSE | UNKNOWN}]]

предикат порівняння:: =

Конструктор значень рядка {= | < | > | <= | >= | <>} *Конструктор значень рядка*

Приклад 33. Порівняння поля таблиці і скалярного значення:

POSTAV.VOLUME > 100

Приклад 34. Порівняння двох сконструйованих рядків:

(PD.PNUM, PD.DNUM) = (1, 25)

Цей приклад еквівалентний умовного виразу

PD.PNUM = 1 AND PD.DNUM = 25

предикат between:: =

Конструктор значень рядка [NOT] BETWEEN

Конструктор значень рядка AND *Конструктор значень рядка*

Приклад 35. PD.VOLUME BETWEEN 10 AND 100

предикат in:: =

Конструктор значень рядка [NOT] IN {(Select-вираз) | (Вираз для обчислення значення, ..)}

Приклад 36.

R.PNUM IN (1, 2, 3, 5)

предикат like:: =

Вираз для обчислення значення рядка-пошуку [NOT] LIKE

Вираз для обчислення значення рядка-шаблону [ESCAPE Символ]

Зауваження. Предикат LIKE здійснює пошук рядка-пошуку в рядку-шаблоні. У рядку-шаблоні дозволяється використовувати два трафаретних символу:

Символ підкреслення «_» може використовуватися замість будь-якого одиничного символу в рядку-пошуку.

Символ відсотка "%" може замінювати набір будь-яких символів в рядку-пошуку (число символів в наборі може бути від 0 і більше).

предикат null:: =

Конструктор значень рядка IS [NOT] NULL

Зауваження. Предикат NULL застосовується спеціально для перевірки, не дорівнює чи перевіряти вираз null-значенням.

Предикат кількісного порівняння:: =

Конструктор значень рядка {= | < | > | <= | >= | <>} {ANY | SOME | ALL} (Select-вираз)

Квантори ANY і SOME є синонімами і повністю взаємозамінні.

Якщо вказано один з кванторів ANY і SOME, то предикат кількісного порівняння повертає TRUE, якщо порівнювальне значення збігається хоча б з одним значенням, повернутому в підзапиті (select-вираженні).

Якщо вказано квантор ALL, то предикат кількісного порівняння повертає TRUE, якщо порівнюване значення збігається з кожним значенням, що повертається в підзапиті (select-вираженні).

Приклад 37.

P.PNUM = SOME (SELECT PD.PNUM FROM PD WHERE PD.DNUM = 2)

предикат exist:: =

EXIST(Select-вираз)

зауваження. Предикат EXIST повертає значення TRUE, якщо результат підзапиту (select-вирази) не пустили.

предикат unique:: =

UNIQUE(Select-вираз)

зауваження. Предикат UNIQUE повертає TRUE, якщо в результаті підзапиту (select-вирази) немає співпадаючих рядків.

предикат match:: =

Конструктор значень рядка MATCH [UNIQUE] [PARTIAL | FULL] (Select-вираз)

Предикат MATCH перевіряє, чи буде значення, визначене в конструкторі рядка збігатися зі значенням будь-якого рядка, отриманої в результаті підзапиту.

предикат overlaps:: =

Конструктор значень рядка OVERLAPS Конструктор значень рядка

Предикат OVERLAPS, є спеціалізованим предикатом, що дозволяє визначити, чи буде вказаний період часу перекривати інший період часу.

10.2.4. Порядок виконання оператора SELECT

Для того щоб зрозуміти, як виходить результат виконання оператора SELECT, розглянемо концептуальну схему його виконання. Ця схема є саме концептуальною, тому що гарантується, що результат буде таким, як якби він виконувався крок за кроком відповідно до цієї схеми. Насправді, реально результат виходить більш досконалими алгоритмами, якими «володіє» конкретна СКБД.

Стадія 1. Виконання одиночного оператора SELECT

Якщо в операторі присутні ключові слова UNION, EXCEPT і INTERSECT, то запит розбивається на кілька незалежних запитів, кожний з яких виконується окремо:

Крок 1 (FROM). Обчислюється прямий декартовий добуток всіх таблиць, зазначених в обов'язковому розділі FROM. В результаті кроку 1 отримуємо таблицю А.

Крок 2 (WHERE). Якщо в операторі SELECT присутній розділ WHERE, то сканується таблиця А, отримана при виконанні кроку 1. При цьому для кожного рядка з таблиці А обчислюється умовний вираз, наведений в розділі WHERE. Тільки ті рядки, для яких умовний вираз повертає значення TRUE, включаються в результат. Якщо розділ WHERE опущений, то відразу переходимо до кроку 3. Якщо в умовному виразі беруть участь вкладені підзапити, то вони

обчислюються відповідно до даної концептуальної схеми. В результаті кроку 2 отримуємо таблицю В.

Крок 3 (GROUP BY). Якщо в операторі SELECT присутній розділ GROUP BY, то рядки таблиці В, отриманої на другому кроці, групуються відповідно до списку угруповання, наведеними в розділі GROUP BY. Якщо розділ GROUP BY опущений, то відразу переходимо до кроку 4. В результаті кроку 3 отримуємо таблицю С.

Крок 4 (HAVING). Якщо в операторі SELECT присутній розділ HAVING, то групи, що не задовольняють умовному виразу, наведеного в розділі HAVING, виключаються. Якщо розділ HAVING опущений, то відразу переходимо до кроку 5. В результаті кроку 4 одержуємо таблицю D.

Крок 5 (SELECT). Кожна група, отримана на кроці 4, генерує один рядок результату в такий спосіб. Обчислюються всі скалярні вирази, зазначені в розділі SELECT. За правилами використання розділу GROUP BY, такі скалярні вирази повинні бути однаковими для всіх рядків усередині кожної групи. Для кожної групи обчислюються значення агрегатних функцій, наведених в розділі SELECT. Якщо розділ GROUP BY був відсутній, але в розділі SELECT є агрегатні функції, то вважається, що є всього одна група. Якщо немає ні розділу GROUP BY, ні агрегатних функцій, то вважається, що є стільки груп, скільки рядків відібрано до даного моменту. В результаті кроку 5 отримуємо таблицю E, яка містить стільки колонок, скільки елементів наведено в розділі SELECT і стільки рядків, скільки відібрано груп.

Стадія 2. Виконання операцій UNION, EXCEPT, INTERSECT

Якщо в операторі SELECT присутні ключові слова UNION, EXCEPT і INTERSECT, то таблиці, отримані в результаті виконання 1-ї стадії, об'єднуються, віднімаються або перетинаються.

Стадія 3. Впорядкування результату

Якщо в операторі SELECT присутній розділ ORDER BY, то рядки отриманої на попередніх етапах таблиці упорядковуються відповідно до списку упорядкування, наведеному в розділі ORDER BY.

Як насправді виконується оператор SELECT. Якщо уважно розглянути наведений вище концептуальний алгоритм обчислення результату оператора SELECT, то відразу зрозуміло, що виконувати його безпосередньо в такому вигляді надзвичайно не вигідно. Навіть на самому першому кроці, коли обчислюється декартовий добуток таблиць, наведених в розділі FROM, може вийти таблиця величезних розмірів, причому практично більшість рядків і колонок з неї буде відкинута на наступних кроках.

Насправді в РСКБД є оптимізатор, функцією якого є знаходження такого оптимального алгоритму виконання запиту, який гарантує отримання правильного результату.

Схематично роботу оптимізатора можна представити у вигляді послідовності декількох кроків:

Крок 1 (Синтаксичний аналіз). Запит, який поступив піддається синтаксичному аналізу. На цьому кроці визначається, чи правильно взагалі (з точки зору синтаксису SQL) сформульований запит. В ході розбору виробляється деякий внутрішньо уявлення запиту, що використовується на наступних кроках.

Крок 2 (Перетворення в канонічну форму). Запит у внутрішньому поданні піддається перетворенню в деяку канонічну форму. При перетворенні до канонічної форми використовуються як синтаксичні, так і семантичні перетворення. Синтаксичні перетворення (наприклад, приведення логічних виразів до кон'юнктивної або диз'юнктивної нормальної форми, заміна виразів «x AND NOT x» на «FALSE» і т.п.) дозволяють отримати нове внутрішньо уявлення запиту, синтаксично еквівалентну вихідного, але стандартне в деякому сенсі. Семантичні перетворення використовують додаткові знання, якими володіє система, наприклад, обмеження цілісності. В результаті семантичних перетворень виходить запит, синтаксично еквівалентний вихідному, але дає той же самий результат.

Крок 3 (Генерація планів виконання запиту і вибір оптимального плану). На цьому кроці оптимізатор генерує безліч можливих планів виконання запиту. Кожен план будується як комбінація низькорівневих процедур доступу до даних з таблиць, методам з'єднання таблиць. З

усіх згенерованих планів вибирається план, що володіє мінімальною вартістю. При цьому аналізуються дані про наявність індексів у таблиць, статистичних даних про розподіл значень в таблицях і т.п. Вартість плану це, як правило, сума вартостей виконання окремих низькорівневих процедур, які використовуються для його виконання. У вартість виконання окремої процедури можуть входити оцінки кількості звернень до дисків, ступінь завантаженості процесора і інші параметри.

Крок 4. (Виконання плану запиту). На цьому кроці план, обраний на попередньому етапі, передається на реальне виконання.

Багато в чому якість конкретної СКБД визначається якістю її оптимізатора. Хороший оптимізатор може підвищити швидкість виконання запиту на кілька порядків. Якість оптимізатора визначається тим, які методи перетворень він може використовувати, який статистичної та іншою інформацією про таблиці наявними документами, які методи для оцінки вартості виконання плану він знає.

11. ФУНКЦІЇ ТА КОМПОНЕНТИ СКБД

Незалежно від того, на основі якого підходу спроектована сучасна БД, її існування немислимо без системи управління базами даних. Прямо чи опосередковано СКБД використовують адміністратори, розробники БД, програмісти і звичайні користувачі. Для цього СКБД надає певний набір інструментів, що спрощують проектування, адміністрування БД і забезпечують доступ до даних.

Система управління базами даних (Database Management System, DBMS) – це комплекс програмних засобів, за допомогою якого можна створювати і підтримувати базу даних, а також здійснювати до неї контрольований доступ користувачів.

Існує безліч показників, за якими можна класифікувати СКБД. Основна класифікаційна ознака – модель реалізації даних, існують мережева, ієрархічна, реляційна і об'єктно-орієнтована модель.

Розрізняють персональні і розраховані на багато користувачів СКБД. Персональні системи призначені для створення невеликих БД, що встановлюються на одному комп'ютері, тому їх часто називають настільними. На противагу персональним, розраховані на багато користувачів системи призначені для обслуговування БД, що знаходяться в спільному використанні декількома користувачами. Є й інші класифікаційні ознаки, наприклад, по способам розробки додатків БД (ручне кодування і автоматична генерація форм), за можливостями визначення даних, особливостям обробки транзакцій, використовуваної ОС, за економічними параметрами. У цій лекції ми обговоримо основні функції СКБД і як вона їх виконує.

11.1. Функції СКБД

Ключові функції СКБД були сформульовані Коддом ще на початку 1980-х років. На першому етапі їх було вісім, пізніше Кодд і інші дослідники неодноразово розширювали і уточнювали цей перелік. Так що тепер можна говорити про декілька десятків функцій, служб і сервісів, якими повинна мати сучасна СКБД.

У цій лекції ми прокоментуємо тільки основоположні функції СКБД:

1. **Доступність даних.** Надання користувачеві можливості вставляти, редагувати, видаляти та видавати дані їх БД. При здійсненні будь-якої з операцій користувач не повинен вникати в особливості фізичної реалізації системи, тобто, всі операції повинні бути прозорі.
2. **Системний опис даних.** СКБД повинна надавати системний каталог, в якому міститься: опис збережених в БД даних; опис зв'язків між даними; обмеження цілісності даних; реєстраційні дані користувачів і інша службова інформація. Завдяки метаданих БД стає доступною зовнішнім додаткам, спрощується розуміння сенсу даних, посилюються заходи безпеки, може виконуватися аудит інформації.
3. **Управління паралельною обробкою.** Реалізація механізму одночасного багатокористувацького (паралельного) доступу до оброблюваних даних з гарантією коректного поновлення цих даних. Уміння надати кільком користувачам одночасний доступ до ресурсів, що розділяються – чи не найскладніше завдання, яке вирішується СКБД. СКБД повинна зуміти уникнути конфлікту спільного доступу двох або більшого числа користувачів до одних і тих же рядках таблиці, або, в крайньому разі, виключити будь-які небажані наслідки при виникненні конфлікту.
4. **Транзакції.** СКБД гарантує, що БД буде завжди знаходитися в несуперечливому стані незалежно від будь-яких збоїв при проведенні операцій оновлення даних. Для цього операції з даними (в першу чергу вставки, редагування і видалення даних) об'єднуються в єдиний блок, званий транзакцією. Всі оператори транзакції повинні бути виконані коректно і повністю, тільки тоді в БД будуть зафіксовані зміни. В іншому випадку здійснюється автоматичний відкат транзакції, тобто, стан БД буде відновлено на момент часу, що передуює викликом транзакції. Далі ми розглянемо багато прикладів транзакцій, а поки обмежимося одним, найбільш близьким кожному з нас. Уявіть собі, що ви знімаєте якусь

суму готівки в одному з численних банкоматів вашого міста. Ви вже ввели свій код доступу, вказали необхідну суму, ця сума грошей списана з вашого електронного рахунку. І раптом з-за збою живлення або відмови комунікаційного обладнання, або з якоїсь іншої технічної причини команда на видачу грошей в банкомат не дійшла. Що в результаті? Ви втратили свої гроші? На щастя ні. Програмне забезпечення банківських платіжних систем «побачить», що одна з операцій транзакції завершена некоректно. Тому всі зміни, зроблені зазначеної транзакцією, підлягають скасуванню – списані електронні гроші знову повернуться до вас на банківський рахунок. Вам залишається повторити операцію отримання готівки в іншому банкоматі немає. Програмне забезпечення банківських платіжних систем «побачить», що одна з операцій транзакції завершена некоректно. Тому всі зміни, зроблені зазначеної транзакцією, підлягають скасуванню – списані електронні гроші знову повернуться до вас на банківський рахунок. Вам залишається повторити операцію отримання готівки в іншому банкоматі немає. Програмне забезпечення банківських платіжних систем «побачить», що одна з операцій транзакції завершена некоректно. Тому всі зміни, зроблені зазначеної транзакцією, підлягають скасуванню – списані електронні гроші знову повернуться до вас на банківський рахунок. Вам залишається повторити операцію отримання готівки в іншому банкоматі.

5. **Забезпечення цілісності даних.** Всі дані, що зберігаються в БД, повинні бути коректними і не повинні суперечити одні одним. Це означає, що дані в таблицях можуть модифікуватися тільки відповідно до встановлених правил. У найзагальнішому випадку можна говорити про існування трьох правил підтримки цілісності даних: цілісність доменів, цілісність відношень, цілісність зв'язків між відношеннями. Крім того, розробник має можливість описувати свої власні бізнес-правила, які ми будемо називати корпоративними обмеженнями.
6. **Відновлення даних.** У разі непередбачених помилок і збоїв, що призвели до пошкодження або руйнування даних, СКБД повинна мати можливість відновлювати пошкоджені дані. В першу чергу ця функція реалізується за допомогою процедур резервного копіювання.
7. **Обмін даними.** СКБД повинна підтримувати сучасні мережеві технології і надавати доступ до БД віддаленим персональним комп'ютерам.
8. **Контроль за доступом до даних.** Доступ до даних можуть здійснювати тільки зареєстровані в СКБД користувачі відповідно до призначеними адміністратором СКБД їм правами.

Ще раз відзначимо, що список обов'язків СКБД на цьому далеко не закінчується. Сучасні системи надають нам засоби моніторингу, сервіси статистичного аналізу, утиліти експорту/імпорту даних, розвинені засоби проектування БД і прикладного програмного забезпечення, інструменти реорганізації файлів даних і індексних даних, служби аудиту та багато іншого. Однак всі додаткові сервіси та служби виконують допоміжну роль, а основи життєдіяльності СКБД визначаються перерахованими функціями.

11.2. Компоненти СКБД

СКБД – складний вид програмного забезпечення, над створенням якого працюють великі колективи висококваліфікованих програмістів. На сучасному ринку програмного забезпечення конкурує близько двох десятків комерційних СКБД. З малих систем, розрахованих на одного користувача, сьогодні найбільшою популярністю користуються: Microsoft Access і Microsoft FoxPro. У перелік багатокористувацьких СКБД, що набули широкого визнання, входять: Oracle, Microsoft SQL Server, InterBase, FireBird, MySQL, Postgre, Informix, DB3.

Незважаючи на те, що всі перераховані програмні продукти призначені для вирішення практично одних і тих же завдань, що входять в СКБД програмні компоненти і взаємозв'язки між ними далеко не ідентичні. Тому будь-яка спроба узагальнити всі відомі технічні рішення побудови СКБД і на основі цього узагальнення побудувати структурну схему типової СКБД, нехай навіть високого ступеня абстракції, кілька наївна. Проте, ми спробуємо це зробити.

Для того щоб СКБД могла надавати мінімальний набір з розглянутих восьми сервісів, вона повинна складатися з набору компонентів, представлених на рис. 11.1.

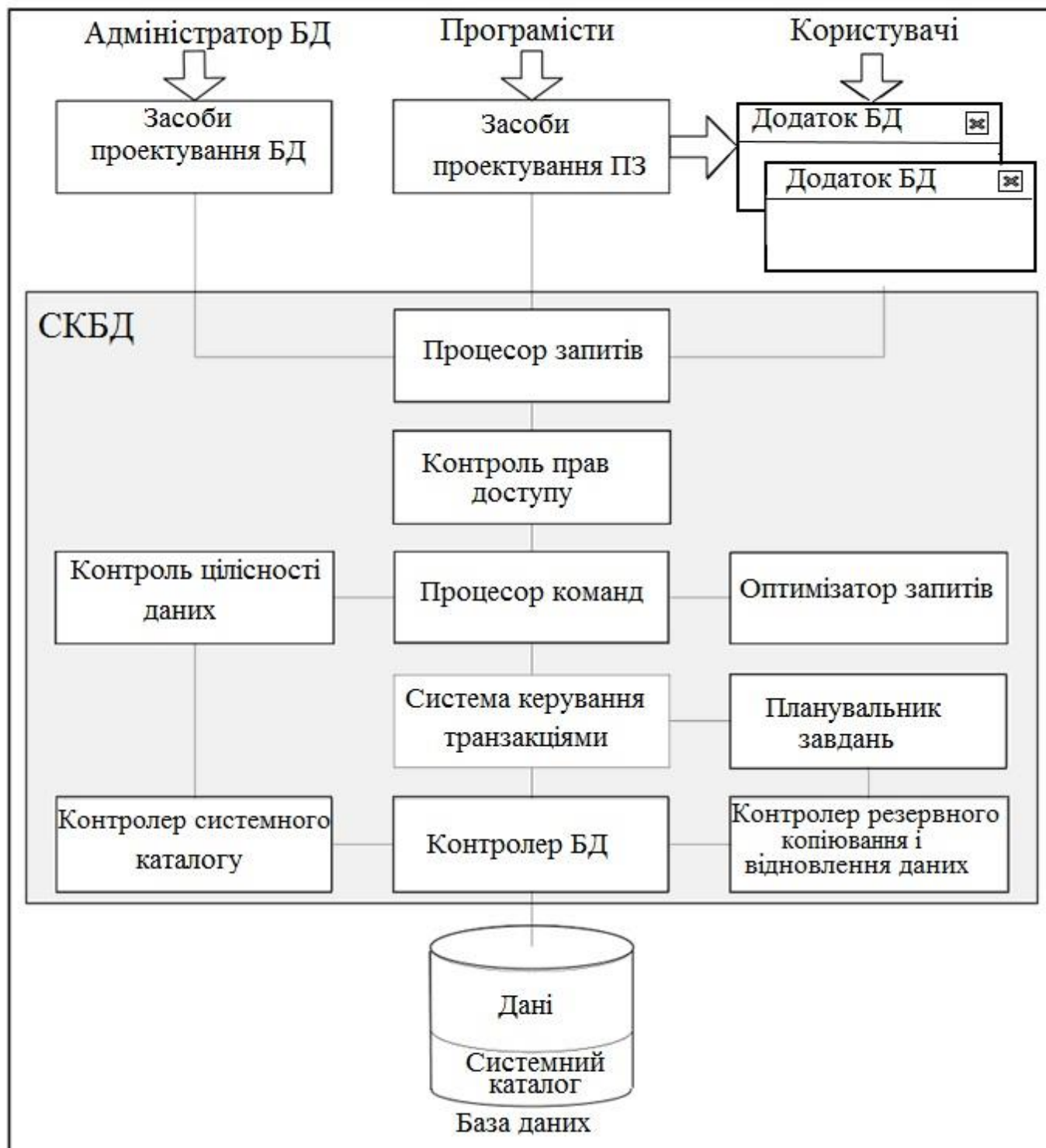


Рис. 11.1. Узагальнена структура СКБД

Над верхньому рівні системи розташовані споживачі послуг СКБД:

- адміністратор БД;
- програмісти;
- користувачі.

Адміністратор БД відповідає за планування і фізичну реалізацію проекту. Він створює основні об'єкти БД, визначає правила підтримки цілісності і несуперечності даних, керує

політикою безпеки, аналізує процес експлуатації проекту, одним словом підтримує життєдіяльність БД. У розпорядженні адміністратора є різноманітні засоби проектування БД, ці кошти можуть входити до складу СКБД у вигляді додаткових програмних модулів, а можуть бути поставлені і сторонніми розробниками.

Програмісти спільно з адміністратором працюють над фізичним створенням проекту БД. Але прикладних програмістів більшою мірою цікавить не сама концепція проекту БД, а способи донесення цієї концепції до кінцевого користувача. Тому область інтересів програмістів зміщена в бік розробки клієнтських додатків і звітів. Основним інструментом прикладного програміста виступають численні середовища проектування, як правило, 4-го покоління (4 Generation Level, 4GL). В першу чергу це програмні пакети Clarion Softvelocity, Embarcadero RAD Studio і Microsoft Visual Studio. Крім того, деякі СКБД (наприклад, Microsoft Access і FoxPro) мають вбудовані засобами проектування, але зазвичай можливості таких засобів суттєво обмежені.

Основним споживачем послуг СКБД виступає звичайний користувач – в його інтересах створюються БД і прикладне програмне забезпечення. Серед користувачів є фахівці, чиї поради здатні допомогти розробникам БД, а на іншому кінці спектру знаходяться користувачі, насилу знаходять потрібні кнопки на клавіатурі. Останні можуть навіть і не розуміти, що вони працюють з БД, розташованої на іншому комп'ютері.

Основним засобом спілкування між людьми і базою даних виступає структурована мова запитів SQL. Тому засоби проектування БД, ПЗ і клієнтські програми БД відправляють на адресу СКБД інструкції на мові SQL. Ці команди надходять на процесор запитів СКБД, який перетворює їх в набір низькорівневих команд, зрозумілих ядру СКБД.

До бази даних можуть звертатися різні категорії користувачів, від керівника підприємства до випадкового відвідувача. Одним з них дозволяється переглядати і редагувати будь-які дані, іншим можуть бути доступні лише вибіркові відомості, третім взагалі не слід навіть знати про існування БД. Саме тому в складі більшості СКБД є модуль, який відповідає за контроль прав доступу. У найпростішому випадку модуль стежить за тим, щоб до БД приєднувалися тільки авторизовані користувачі, для цього отримані під час реєстрації пароль і логін потенційного клієнта звіряються з зберігаються в системному каталозі обліковими записами. В сучасних багатокористувацьких СКБД система безпеки значно складніше і включає в себе комплекс програмних, а іноді і апаратних засобів захисту.

Переконавшись, що інструкція надійшла від довіреної особи, модуль контролю прав доступу передає її в розпорядження процесора команд. В першу чергу процесор переконується, що надійшла команда не суперечить обмеженням цілісності даних. Це зона відповідальності модуля контролю цілісності даних. На час перевірки команди на відповідність обмеженням цілісності доменів, сутностей і зв'язків задіюється контролер системного каталогу. Саме він має можливість збирати метадані, в яких ховається технічний опис нашої БД. Зрозумівши, що загрози цілісності немає, процесор передає команду оптимізатора запитів. Завдання оптимізатора – знайти найбільш ефективний спосіб виконання надійшли команд. Нарешті, оптимізована команда компілюється і передається системі управління транзакціями.

Система управління транзакціями, по-перше, відповідає за повне і коректне виконання блоку команд і, по-друге, спільно з планувальником завдань забезпечує паралельну багатокористувацьку обробку даних. Нарешті блок команд передається в розпорядження контролера баз даних. Завдання модуля полягає в організації взаємодії СКБД з файлами БД і файлами системного каталогу. При цьому для здійснення стандартних операцій введення-виведення задіюються можливості операційної системи.

11.3. Архітектурні рішення доступу до БД

Пошуки ефективних архітектурних рішень організації доступу до БД почалися з перших днів роботи з БД. Всі рішення, так чи інакше, відштовхувалися від поточного стану електронної обчислювальної техніки. За часів домінування великих обчислювальних машин доступ до найперших БД базувався на принципі телеобробки. База даних і СКБД розміщувалася в пам'яті центральної ЕОМ, до цієї ЕОМ підключалися термінали. Так як термінальні пристрої не володіли

власними обчислювальними можливостями, то вся обробка даних здійснювалася тільки в процесорі центральної ЕОМ. Системи на основі телеобробки домінували до початку 1970-х років, але з винаходом мікропроцесорів стали поступово поступатися своїми позиціями і зараз практично не використовуються.

11.3.1. Файл-сервер

Бурхливий розвиток мікропроцесорної техніки в 70-х роках ХХ століття призвів до виникнення порівняно дешевих мікро-ЕОМ. Тепер керівники підприємств замість покупки однієї великої дорогої ЕОМ почали купувати кілька недорогих машин, менш продуктивних, але цілком прийнятних для вирішення завдань, що стоять перед їх колективами.

Мікро-ЕОМ об'єднувалися в найпростіші однорангові локальні мережі, в яких кожна з машин володіла рівними правами зі своїми сусідами. Одна з ЕОМ (з накопичувачами на жорстких магнітних дисках найбільшого розміру) призначалася файловим сервером. На виданому в загальне користування мережевому каталозі сервера крім звичайних документів розміщували і файли баз даних (рис. 11.2).



Рис. 11.2. Архітектура «файл-сервер»

Для того щоб цією БД могла скористатися котрась із робочих станцій, вона зверталася до файлового серверу, перекачувала всі файли БД в свою пам'ять, вносила правки і повертала файли на колишнє місце. Такий спосіб на багатокористувацького доступу до БД володів всього однією перевагою – простотою реалізації і безліччю недоліків:

1. На кожній робочій станції необхідно встановлювати повну копію СКБД.
2. Під час роботи з даними мережевий трафік зростає до пікових значень.
3. Управління паралельною обробкою даних і підтримку цілісності реалізувати було практично неможливо, в той час як один користувач зберігав свою частину даних, інший вже вносив до неї зміни.

Як приклад програмних продуктів, здатних працювати в режимі файл-серверних систем, можна привести такі настільні СКБД, як Access і FoxPro корпорації Microsoft, а також різні версії dBase.

11.3.2. Клієнт-сервер

В даний час архітектурне рішення «файл-сервер» застосовується лише для підтримки малих баз даних з числом користувачів, що не перевищує 5-6 чоловік. У великих підприємствах домінує архітектура «клієнт-сервер». Поява клієнт-серверних систем тісно пов'язана з парадигмою відкритих систем, активно еволюціонують з початку 1980-х років. Програмісти прийшли до висновку про необхідність розподілу завдань між елементами системи, в даному випадку між двома типами процесів, які можуть виконуватися на різних комп'ютерах, об'єднаних в обчислювальну мережу. Процес сервера відповідає за надання будь-яких послуг клієнтського процесу. Клієнтський процес запитує у сервера певні послуги і ресурси.

У клієнт-серверних системах БД розміщується на окремому, як правило, найбільш продуктивному комп'ютері, на цьому ж комп'ютері розгортається сервер СКБД. На клієнтських станціях досить встановити порівняно невимогливе до ресурсів для користувача ПЗ і налаштувати мережевий доступ до сервера СКБД. Робота клієнт-серверних систем принципово відрізняється від роботи в системах «файл-сервер». Тепер, замість перекачування файлів з БД, клієнтський комп'ютер відправляє серверу запит, побудований на основі мови SQL. Отримавши і обробивши інструкцію SQL, сервер повертає клієнтському комп'ютеру результати її виконання, наприклад, вибірку певних даних (рис. 11.3).

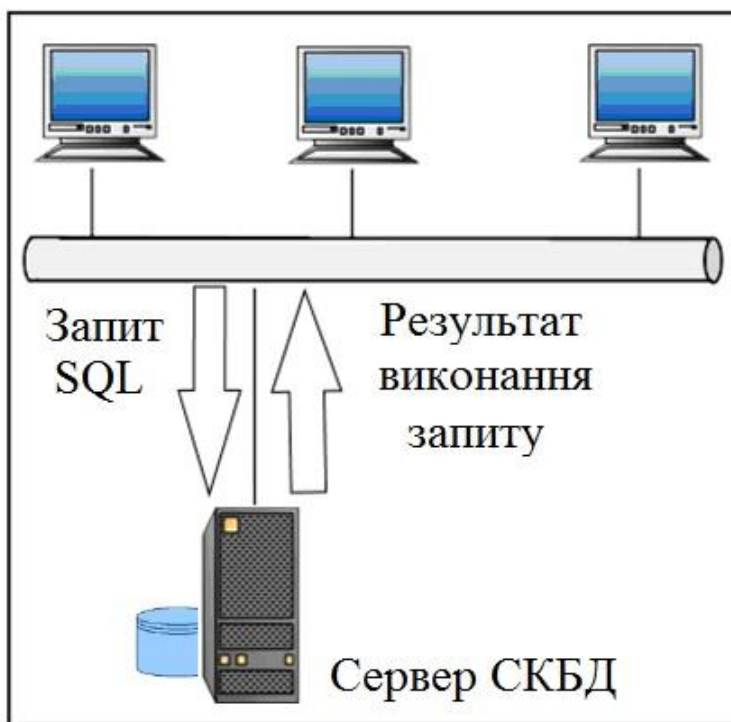


Рис. 11.3. Архітектура «клієнт-сервер»

На цей час існує більше десятка СКБД, призначених для роботи за моделлю клієнт-сервер. Найбільш популярні з них: InterBase фірми Embarcadero Technologies, Oracle, Microsoft SQL Server, Postgre, MySQL, Informix Universal Server, NetWare SQL фірми Novell і т. д. На сьогоднішній день домінує клієнт-серверне побудова БД. Причин тому кілька:

1. Значно підвищується доступність БД. Сервер являє собою відкриту систему, тому клієнтські комп'ютери можуть функціонувати під управлінням різних ОС і з різним ПЗ.
2. Виділений сервер СКБД в змозі забезпечити паралельну багато користувачів обробку даних.
3. Основні правила підтримки цілісності і несуперечності даних описуються на одному сервері СКБД, клієнтські програми ніяким чином не в змозі обійти ці правила.
4. Економно витрачається пропускна здатність комп'ютерних мереж.
5. Завдяки централізованого зберігання даних порівняно просто підтримувати єдині для всіх правила безпеки БД.

6. Наявність стандарту на основну мову спілкування SQL забезпечує широкі можливості доступу до сервера БД з програмного забезпечення різних виробників.
7. Спрощені питання обслуговування і адміністрування БД.

Передбачено кілька підходів до розподілу обов'язків між сервером і клієнтом. У найпростішому випадку (рис. 11.4, а) працює модель тонкий клієнт (thin client). Це варіант, коли клієнтське ПЗ максимально спрощено і являє собою тільки інтерфейсну частину, що складається з форм введення і перегляду даних. Тонкий клієнт фактично відповідає тільки за презентацію даних, тому можна сказати, що він не вміє «думати», і вся програмна логіка зосереджена на стороні сервера.

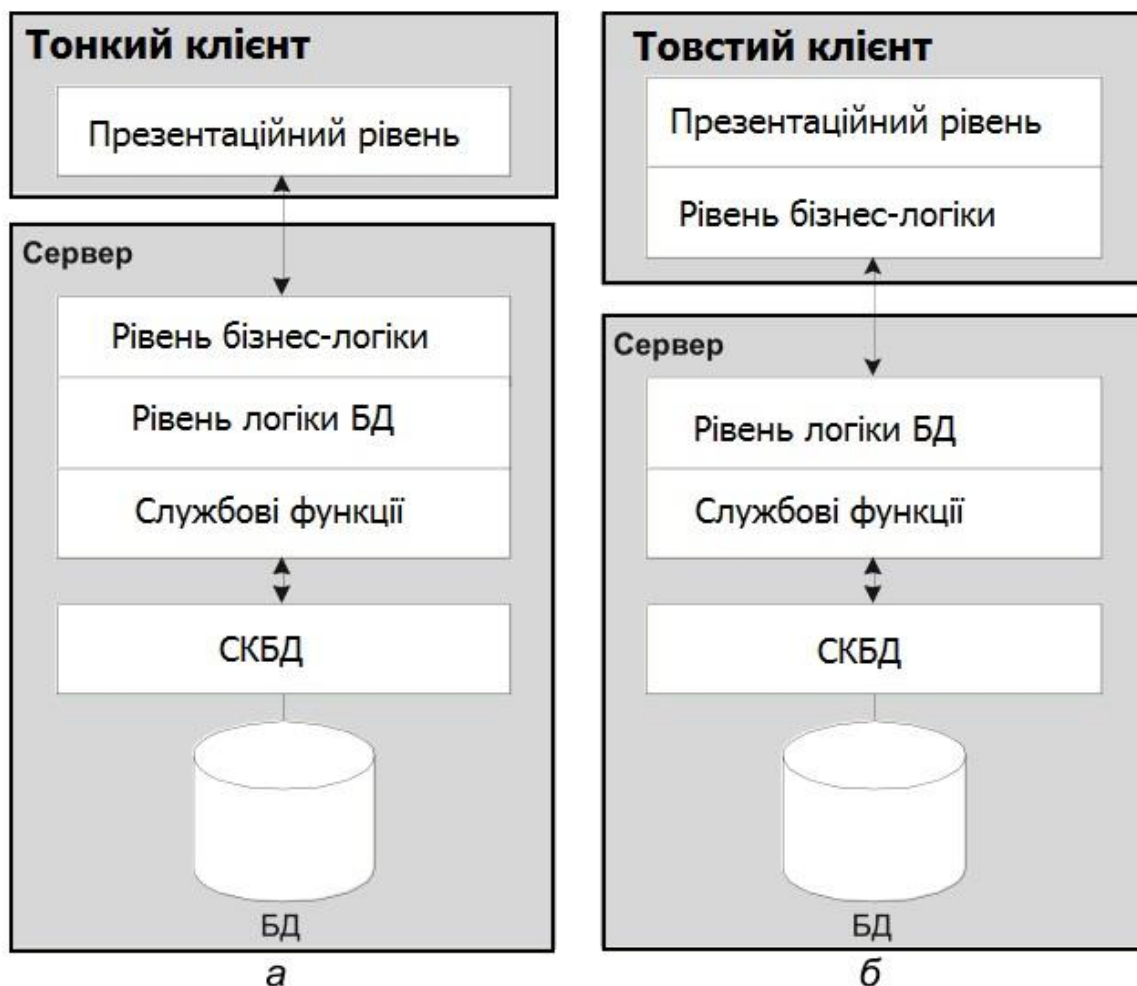


Рис. 11.4. Популярні моделі розподілу функцій між сервером і клієнтом

Є й зворотне рішення – товстий клієнт (fat client). На противагу тонкому клієнтові, товстий клієнт намагається максимально полегшити роботу сервера (див. рис. 11.4, б), наприклад, реалізує додаткові бізнес-правила, сортує отримані дані на стороні клієнта, виконує допоміжні розрахунки, перевіряє синтаксис інструкцій SQL і т. п.

Проектування повнофункціонального товстого клієнтського додатка значно складніше, ніж розробка спрощеного тонкого клієнта. Але, в кінцевому рахунку, ми отримуємо істотну надбавку в продуктивності, а це вельми важливо в великих багатокористувацьких БД.

11.3.3. Багаторівневі рішення

Передбачено й більш складні моделі розподілу функцій між сервером і клієнтом. Наприклад, між сервером СКБД і клієнтом може з'явитися ще одна додаткова ланка – сервер додатків (рис. 11.5). Сервер додатків зазвичай відповідає за дотримання бізнес-правил БД, для цього реалізується кілька прикладних функцій, які представляються клієнту у вигляді окремих служб. Клієнт не може звернутися до СКБД безпосередньо, замість цього він запитує, що цікавить його сервіс у сервера додатків.



Рис. 11.5. Розподіл функцій між сервером і клієнтом – сервер додатків

Багаторівневі проекти БД можуть створюватися на фундаменті кількох технологій. Сьогодні величезною популярністю користується істотно вдосконалена в Delphi 2010 розподілена модель DataSnap. У Delphi є цілий ряд компонентів, що спеціалізуються на побудові багатоланкових проектів DataSnap.

При створенні проміжної ланки між клієнтом і сервером розробники часто користуються технологією Архітектура брокера загальних об'єктних запитів (Common Object Request Broker Architecture, CORBA). Технологія CORBA створювалася спеціально для систем з розподіленою обробкою інформації. Ядром системи виступає брокер об'єктних запитів (Object Request Broker, ORB), що виконує посередницьку функцію між клієнтом і сервером.

Родзинка CORBA в тому, що інтерфейсна частина всіх описаних в ній компонентів відокремлена від їх реалізації. Завдяки цьому зв'язуються за посередництва CORBA сервер і клієнтські програми можуть розроблятися на будь-яких мовах програмування і функціонувати під управлінням різних ОС, аби дотримувалася єдина умова – вони повинні вміти спілкуватися мовою визначення інтерфейсів (Interface Definition Language, IDL).

Багаторівневі проекти БД можна створювати і на альтернативних технологіях, зокрема на основі розподіленої багатокомпонентної моделі об'єктів (Distributed Component Object Model, DCOM), протоколі простого доступу до об'єктів (Simple Object Access Protocol, SOAP) призначеного для обміну даними в розподіленій децентралізованій середовищі, технології сокетів і т. д.

Незважаючи на те, що трехланкова архітектура дозволяє створювати гнучкі і універсальні рішення, вона використовується значно рідше, ніж двухланкова. Причин тому кілька, але основна з них в тому, що створення багаторівневих клієнт-серверних проектів під силу тільки добре підготовленим розробникам, як наслідок подібні проекти коштують значно дорожче в порівнянні з класичною архітектурою «клієнт-сервер».

12. РОЗПОДІЛЕНА ОБРОБКА ДАНИХ

12.1. Основні поняття і визначення

Режими використання БД у загальному вигляді показані на рис. 12.1.

Якщо з БД працюють одночасно декілька користувачів, то в цьому випадку СУБД повинна забезпечувати коректну паралельну роботу всіх користувачів над одними і тими ж даними. Розрізняють розподілену обробку і розподілені БД.

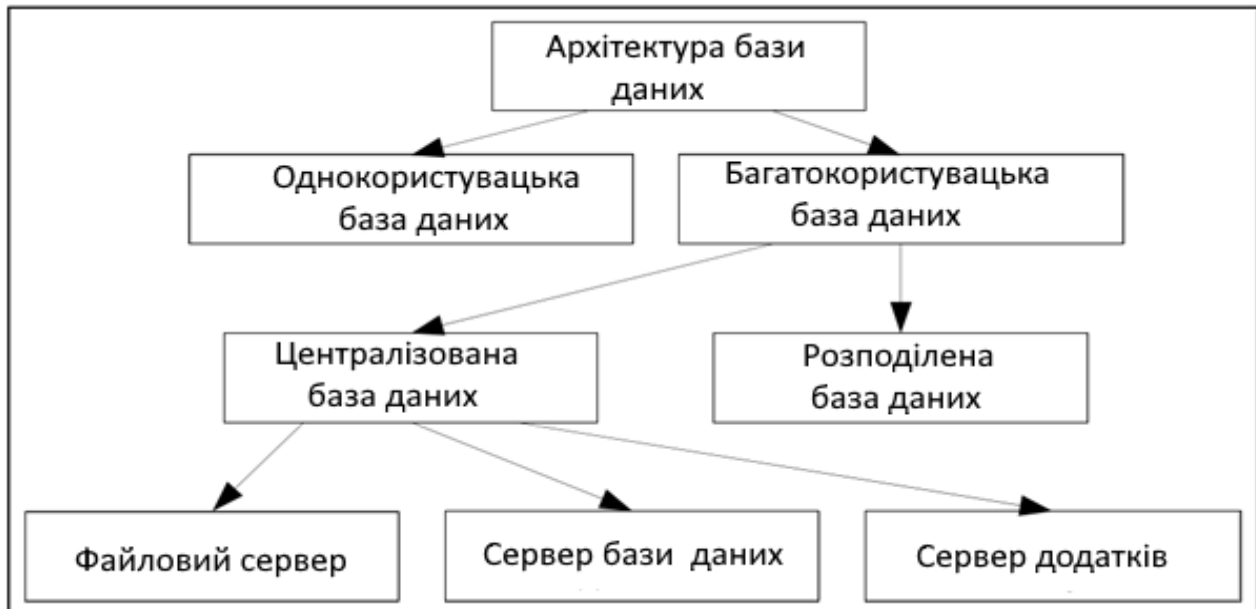


Рис. 12.1. Режими роботи з базою даних

Розподілена обробка – це обробка з використанням централізованої бази даних, доступ до якої може виконуватись з різних комп'ютерів мережі (рис. 12.2, а). Ця топологія часто називається «клієнт-сервер». В цій системі одні вузли – клієнти, а інші – сервери.

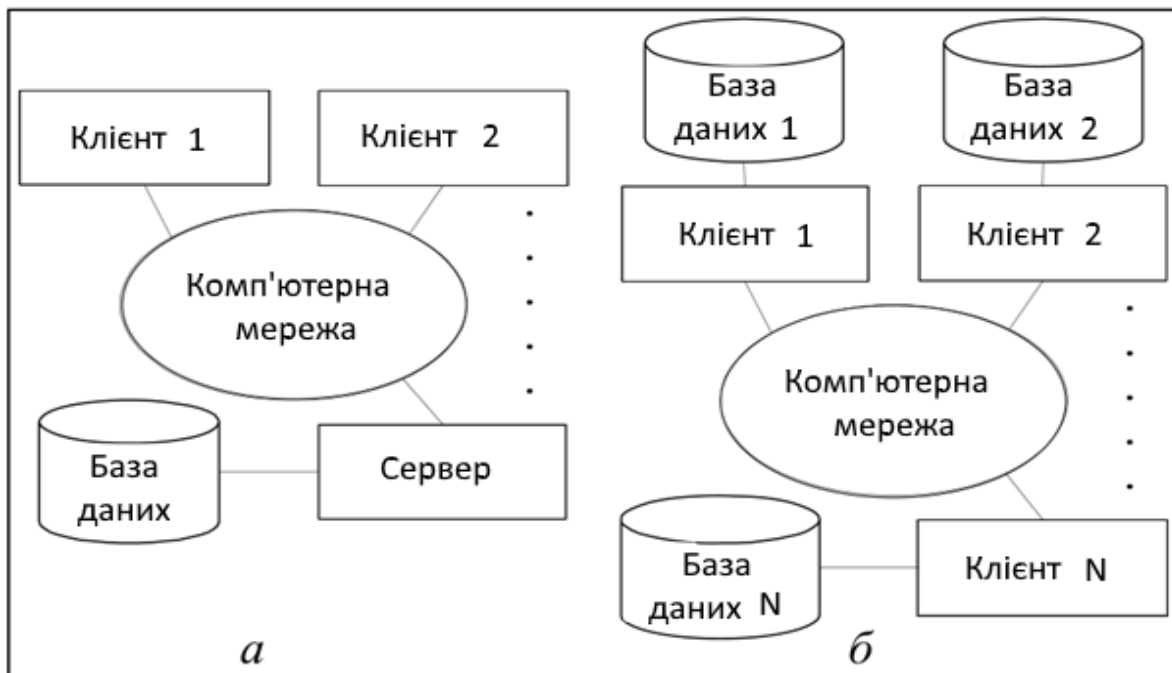


Рис. 12.2. Структура інформаційної системи:
а – розподілена обробка; б – розподілена база даних

Суть клієнт-серверної архітектури в тому, щоб розділити функції між двома підсистемами: клієнтом, який відправляє запит на виконання яких-небудь дій, і сервером, який виконує цей запит. Взаємодія між клієнтом і сервером відбувається за допомогою стандартних спеціальних протоколів, таких як TCP/IP і z39.50. Насправді протоколів дуже багато, вони розрізняються по рівнях.

Сервер – комп'ютер, який надає деякі послуги іншим комп'ютерам, обмін повідомленнями з якими здійснюється за допомогою мережі, що їх з'єднує. Послуги полягають у наданні комп'ютеру, який звертається, ресурсів сервера (файлів, обчислювальних ресурсів тощо) шляхом виконання вказаної програми і видачі результатів її роботи.

По суті, сервер є набором програм, які контролюють виконання різних процесів. Відповідно, цей набір програм встановлений на якомусь комп'ютері. Тому часто комп'ютер, на якому встановлений сервер, і називають сервером.

Клієнт – це процес, який посилає запит на обслуговування. Тобто, клієнтом називають будь-який процес, який користується послугами сервера. Клієнтом може бути як користувач, так і програма. Основне завдання клієнта – виконання додатку і здійснення зв'язку з сервером, коли цього вимагає додаток. Тобто клієнт повинен надавати користувачеві інтерфейс для роботи із додатком, реалізовувати логіку його роботи і при необхідності відправляти завдання серверу.

Взаємодія між клієнтом і сервером починається за ініціативою клієнта. Клієнт запрошує вид обслуговування, встановлює сеанс, отримує потрібні йому результати і повідомляє про закінчення роботи.

Розподілена база даних – це набір логічно зв'язаних між собою роздільних даних і їх описів, які фізично розподілені в деякій комп'ютерній мережі (див. рис. 12.2, б). Розподілена СКБД, в якій управління кожним із вузлів виконується зовсім автономно називається **мультібазовою системою**.

Розподілена СКБД – це програмна система, яка призначена для управління розподіленими базами даних і яка забезпечує прозорий доступ користувачів до розподіленої інформації.

Якщо всі вузли розподіленої системи використовують той самий тип СКБД, то така система називається **гомогенною**. Якщо вузли розподіленої системи використовують різні типи СКБД, які обробляють різні моделі даних, то така система називається **гетерогенною**.

12.2. Управління паралельною обробкою

В багатокористувацьких системах з БД одночасно можуть працювати декілька користувачів або прикладних програм. Для збереження цілісності даних і забезпечення безпеки БД в цих умовах застосовуються транзакції, які забезпечують роботу кожного користувача з узгодженим станом БД.

Транзакція – неподільна з точки зору впливу на БД послідовність операторів маніпулювання даними, яка розглядається СКБД як єдине ціле. Або транзакція успішно виконується, і СКБД фіксує зміни БД, які були зроблені цією транзакцією, у зовнішній пам'яті, або, у разі невдачі, жодна зміна не відображається на стані БД.

Транзакція розглядається як логічна одиниця роботи з БД. Для того щоб використання механізмів обробки транзакцій дозволило забезпечити цілісність даних й ізолюваність користувачів, транзакція повинна мати такі властивості: атомарність (Atomicity), узгодженість (Consistency), ізолюваність (Isolation), довготерміновість (Durability). Транзакції, які мають ці властивості називаються ACID-транзакціями. Властивості транзакції означають таке:

- **атомарність** означає, що транзакція виконується, як єдина операція доступу до БД і виконується або повністю або не виконується зовсім;
- **узгодженість** гарантує взаємну цілісність даних, тобто виконання обмежень цілісності БД після завершення роботи транзакції;
- **ізолюваність** означає, що транзакції, які конкурують за доступ до БД, фізично обробляються послідовно, ізолювано одна від одної, але для користувачів це виглядає так, ніби вони виконуються паралельно;

- **довготерміновість** означає, що коли транзакція виконана успішно, то всі зміни, які вона зробила в даних, не будуть втрачені ні за яких обставин.

Для обробки паралельних транзакцій застосовується серіалізація транзакцій і метод тимчасових міток.

Серіалізація транзакцій – процедура, яка забезпечує підтримку незалежного виконання транзакцій. Це означає, що дія двох паралельно діючих транзакцій буде така сама, як і їх послідовна дія: спочатку перша, а потім друга, або навпаки – спочатку друга, а потім перша. У ході виконання транзакції користувач бачить тільки узгоджені дані і не бачить неузгоджених проміжних даних. Для підтримки паралельної роботи складається спеціальний план.

Для реалізації серіалізації транзакцій застосовується механізм **блокувань**. Блокування передбачає встановлення режиму доступу (монопольного або сумісного) до деякого ресурсу даних, що дозволяє виключити доступ до нього одночасно з даною транзакцією інших транзакцій, в результаті якого може бути порушена логічна цілісність даних БД. Об'єктом блокування може бути вся БД, окремі таблиці, сторінки, рядки.

Для підвищення ступеня паралельності доступу декількох користувачів до однієї БД використовуються такі блокування:

1. **Нежорстке** блокування або роздільне блокування (Shared – S-блокування). Об'єкт блокується для виконання операції читання; об'єкти в цьому випадку не змінюються у ході виконання транзакції і доступні іншим транзакціям також, але тільки в режимі читання.
2. **Жорстке** блокування або монопольне (exclusive – X-блокування). Об'єкт блокується для виконання операції запису, модифікації або вилучення. В цьому випадку виконується монопольне блокування об'єкта і об'єкт залишається недоступним іншим транзакціям до моменту завершення роботи даної транзакції.

Застосування різних типів блокувань призводить до тупиків. **Тупикова ситуація** виникає тоді, коли дві і більш транзакції одночасно знаходяться у стані очікування, причому для продовження роботи кожна з транзакцій очікує завершення роботи іншої транзакції.

Приклад. Нехай транзакція 1 в момент часу t_1 блокує ресурс **A**, а транзакція 2 в момент часу t_2 блокує ресурс **B** (рис.12.3). В момент часу t_3 транзакції 1 потрібен ресурс **B** і вона очікує його звільнення транзакцією 2. В момент часу t_4 транзакції 2 потрібен ресурс **A** і вона очікує його звільнення транзакцією 1. Транзакція 1 чекає транзакцію 2, транзакція 2 чекає транзакцію 1.

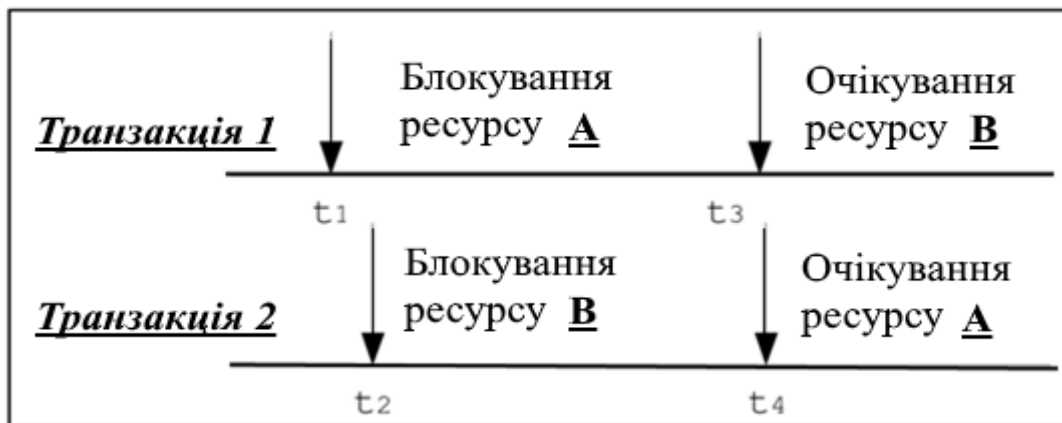


Рис.12.3. Взаємне блокування транзакцій

Основою визначення тупикових ситуацій є побудова графа очікування транзакцій. Алгоритм виходу із тупика передбачає визначення транзакції-жертви. Після вибору такої транзакції виконується її відкат. Для серіалізації транзакцій також застосовується **двофазне блокування**, яке полягає у такому:

- перед виконанням операцій з будь-яким об'єктом транзакція блокує цей об'єкт (накопичення захватів);
- після зняття блокування транзакція не повинна накладувати ніяких інших блокувань (вивільнення захватів).

12.3. Багатокористувацькі СКБД

В якості сервера може розглядатися програма, яка надає деякі послуги по запитах інших програм, що називаються клієнтами. Взаємодія між клієнтами і сервером здійснюється за допомогою передачі повідомлень відповідно до заданого протоколу.

В технології роботи з БД виділяють функції:

- вводу і відображення даних (*представлення даних*);
- прикладні функції, які визначають основні алгоритми розв'язання прикладних задач (*додатки*);
- *обробки* даних у додатках;
- управління інформаційними ресурсами (СКБД).

Представлення даних визначає те, що користувач бачить на своєму екрані. Тут визначаються екранні зображення, операції читання і запису даних, управління діалогом.

Прикладні функції (бізнес-логіка) визначають логіку роботи прикладних програм. Код прикладних програм пишеться з використанням процедурних мов програмування.

Функції **обробки даних** пов'язані з обробкою даних всередині додатків. Даними керує СКБД. Для забезпечення доступу до даних використовується мова SQL, яка найчастіше вбудовується в мови, які використовуються для створення коду додатка.

Функції **управління інформаційними ресурсами** – це СКБД, яка забезпечує зберігання і управління БД.

Залежно від того, де розташовані ці компоненти по відношенню одна до одної розрізняють монолітне виконання (найчастіше для персональних БД), дво- і тривірневе.

Дворівнева архітектура характеризується тим, що всі функції розподіляються між двома процесами, які виконуються на двох платформах: на клієнті і на сервері. В дворівневій архітектурі в свою чергу можлива реалізація таких моделей:

- модель файлового сервера;
- модель віддаленого доступу до даних;
- модель сервера бази даних.

У моделі **файлового сервера** (рис. 12.4) додатки розташовуються і виконуються на робочих станціях.

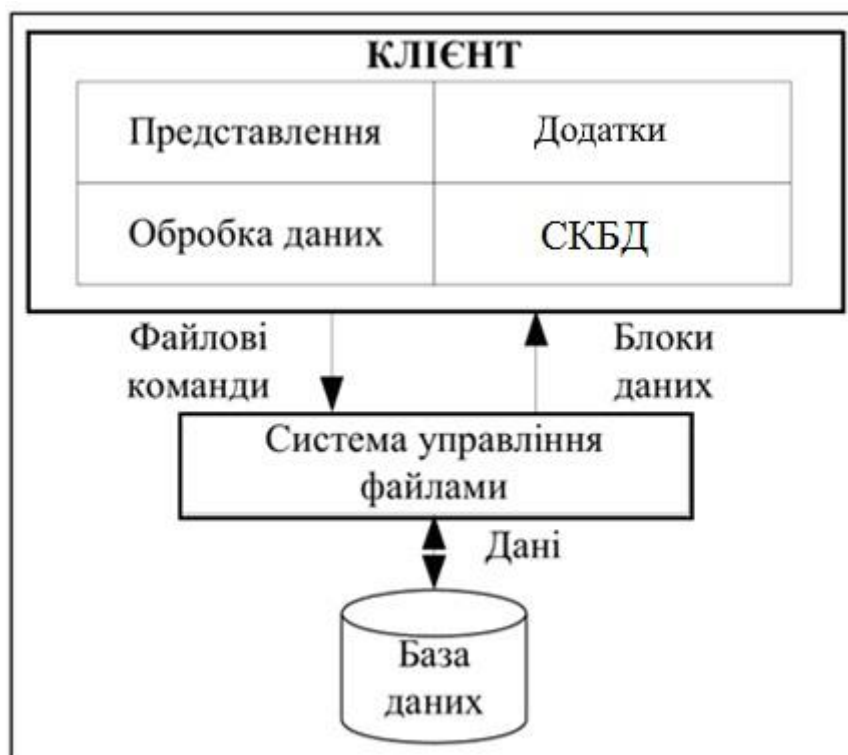


Рис. 12.4. Модель файлового сервера

На файловому сервері зберігаються тільки файли БД (файли даних, індекси тощо) і деякі технологічні файли, які необхідні для роботи додатків і самої СКБД. Клієнт звертається до СКБД на мові SQL, СКБД перекладає запит у послідовність файлових команд і передає файловому серверу. На кожен команду сервер передає блок даних. Обробка інформації виконується на робочій станції за допомогою СКБД. Системи цього класу коштують недорого, прості в установці й освоєнні. До недоліків можна віднести таке:

- на кожній робочій станції знаходиться копія СКБД;
- низька продуктивність систем, оскільки всі проміжні дані передаються по мережі, а обробка виконується на робочих станціях, потужність яких значно поступається потужності сервера;
- складність розподіленої обробки.

У *моделі віддаленого доступу* до даних (рис. 12.5) БД і СКБД знаходяться на сервері, додатки розташовуються і виконуються на робочих станціях. Клієнт звертається до серверу на мові SQL. В цій архітектурі сервер виконує функції обробки транзакцій, даних і запитів. Сервер не перевантажений виконанням додатків.

Значно зменшується завантаження мережі у порівнянні з сервером файлів, оскільки по мережі від клієнта до сервера передаються команди на мові SQL, а не файлові команди, обсяг яких значно більший. Від сервера до клієнта передаються дані, які відповідають запиту, а не блоки файлів.

Недоліками моделей є таке:

- запити на мові SQL при інтенсивній роботі можуть значно завантажити мережу;
- прикладні функції додатків необхідно повторювати для кожного клієнтського комп'ютера;
- сервер виконує пасивну роль і тому функції управління інформаційними ресурсами повинні виконуватись клієнтом.

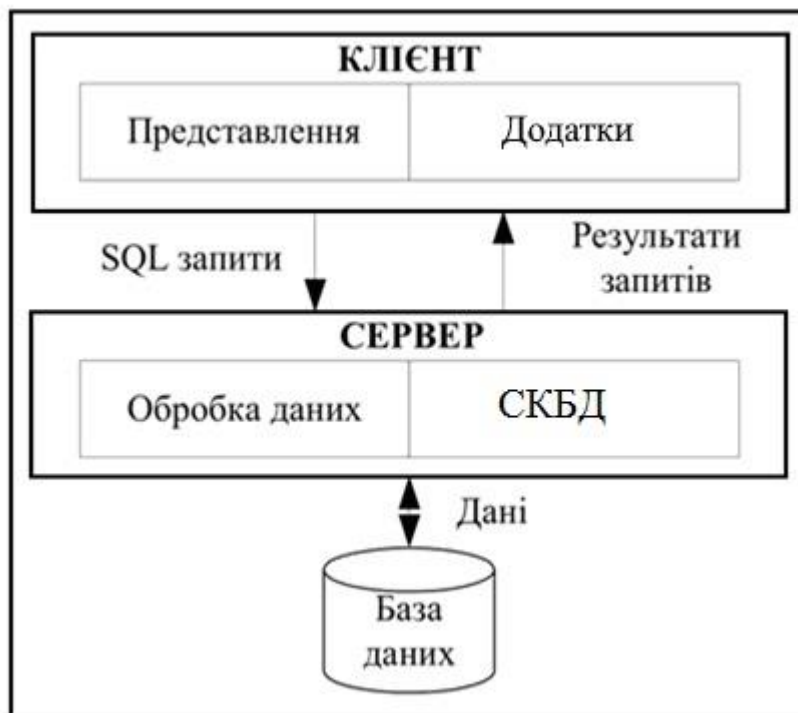


Рис. 12.5. Модель віддаленого доступу

Модель сервера бази даних (рис. 12.6) є подальшим розвитком моделі віддаленого доступу. Ця модель розширена механізмами процедур, що зберігаються і механізмами тригерів, які створюються на розширенні мови SQL.

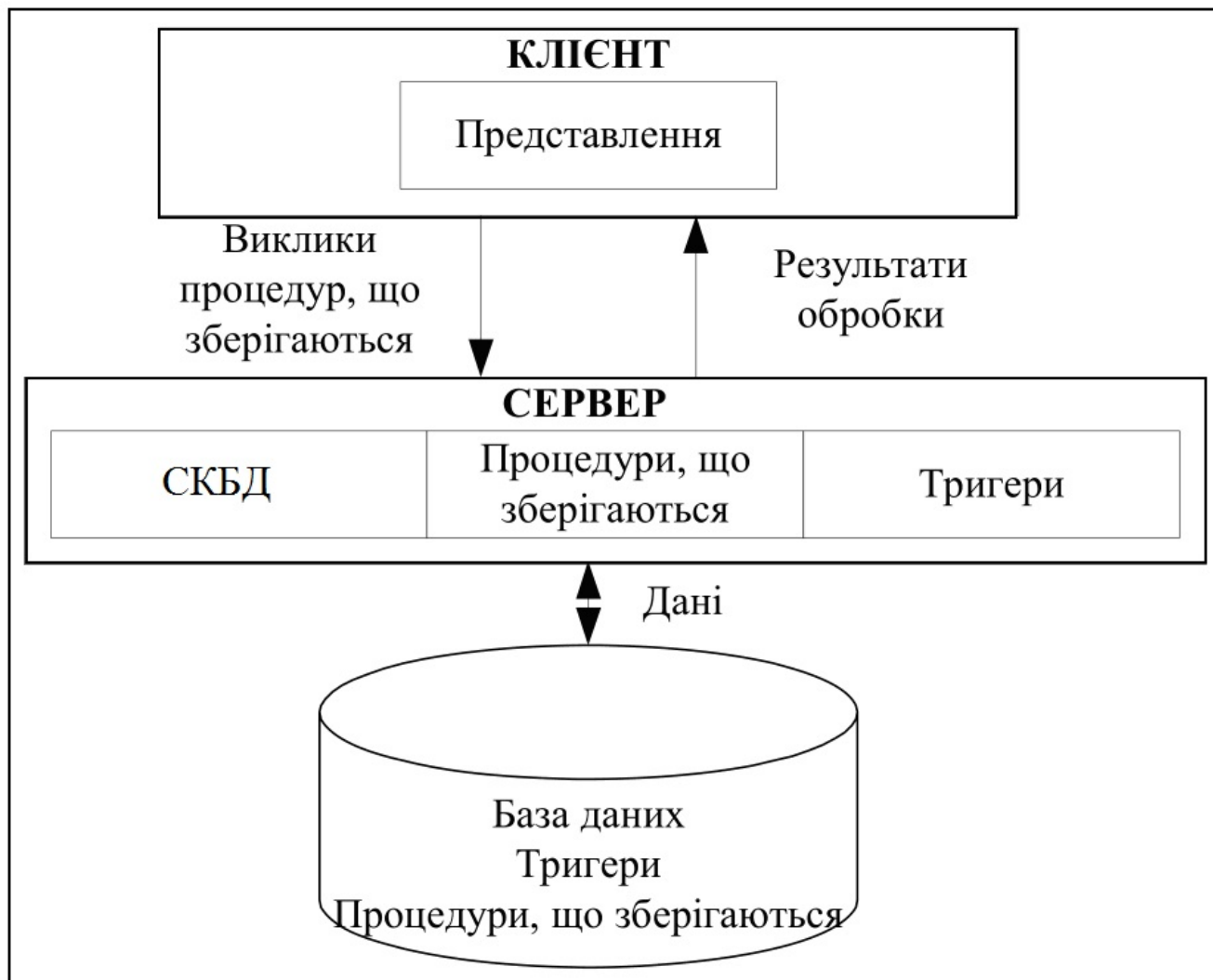


Рис. 12.6. Модель сервера бази даних

Збережені процедури є засобом програмування SQL-сервера і являють собою підпрограми, які можуть викликатися або додатками користувачів, або тригерами.

Тригери являють собою особливий тип процедури, що зберігається, яка самостійно реагує на виникнення певної події в БД. Тригер може активізуватися після операцій додавання, модифікації і вилучення. Застосування тригерів дозволяє зробити сервер активним. Це означає, що сам сервер може бути ініціатором обробки даних в БД.

Застосування процедур, що зберігаються і тригерів на сервері дозволяє їх використовувати різними клієнтами, що суттєво зменшує дублювання алгоритмів обробки даних. Недоліком цієї моделі в першу чергу є велике завантаження сервера.

Подальшим розвитком моделі сервера бази даних є трирівнева архітектура (рис.12.7). Ця архітектура ще називається **моделлю із сервером додатків**.

Сервер додатків – в триланковій архітектурі «клієнт-сервер» компонент прикладної системи, який розташовується між клієнтом і сервером БД і реалізує логіку додатку.

У цій моделі клієнт виконує тільки представлення інформації, сервер БД – виконує функції управління інформаційними ресурсами БД, забезпечує функції створення і ведення БД, підтримує цілісність БД. Сервер додатків виконує загальні функції для клієнтів, забезпечує обмін повідомленнями, підтримує запити, зберігає і виконує найбільш загальні правила бізнес-логіки.

Перевагою моделі із сервером додатків є гнучкість і універсальність внаслідок розділення функцій на три незалежні складові.

Головним недоліком є більш високі витрати ресурсів комп'ютера на обмін інформацією між компонентами додатку у порівнянні з дворівневими моделями.



Рис. 12.7. Модель сервера додатків

12.4. Проектування багатокористувацьких баз даних

Багатокористувацька БД передбачає, що у визначенні вимог до БД задіяні декілька користувачів. Від того, як враховуються вимоги користувачів, розрізняють централізований підхід й інтеграцію представлень користувачів. Централізований підхід передбачає, що вимоги до кожного представлення користувача об'єднуються в загальний набір вимог (рис. 12.8).

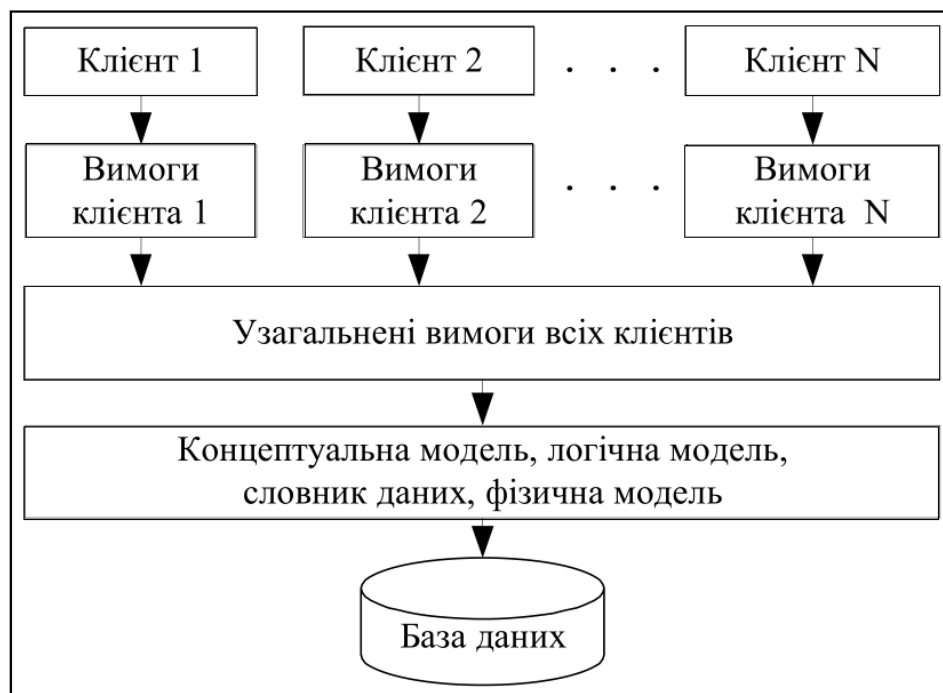


Рис. 12.8. Централізований підхід до проектування багатокористувацької бази даних

На етапі проектування БД створюється глобальна модель даних, яка відповідає загальному представленню. Глобальна модель також перевіряється, як і локальні моделі. Коректність глобальної моделі перевіряється за допомогою правил нормалізації, що дозволяє переконатися в структурній узгодженості, логічній цілісності і мінімальній збитковості прийнятої моделі даних. Модель також перевіряється з метою виявлення можливостей виконання транзакцій, які будуть задаватися користувачами.

Інтеграція представлень користувачів передбачає, що вимоги до кожного представлення користувача застосовуються для створення окремої моделі даних, яка відповідає цьому представленню користувача. У подальшому, на етапі проектування БД, моделі даних, що отримані об'єднуються в єдину глобальну модель даних (рис. 12.9).

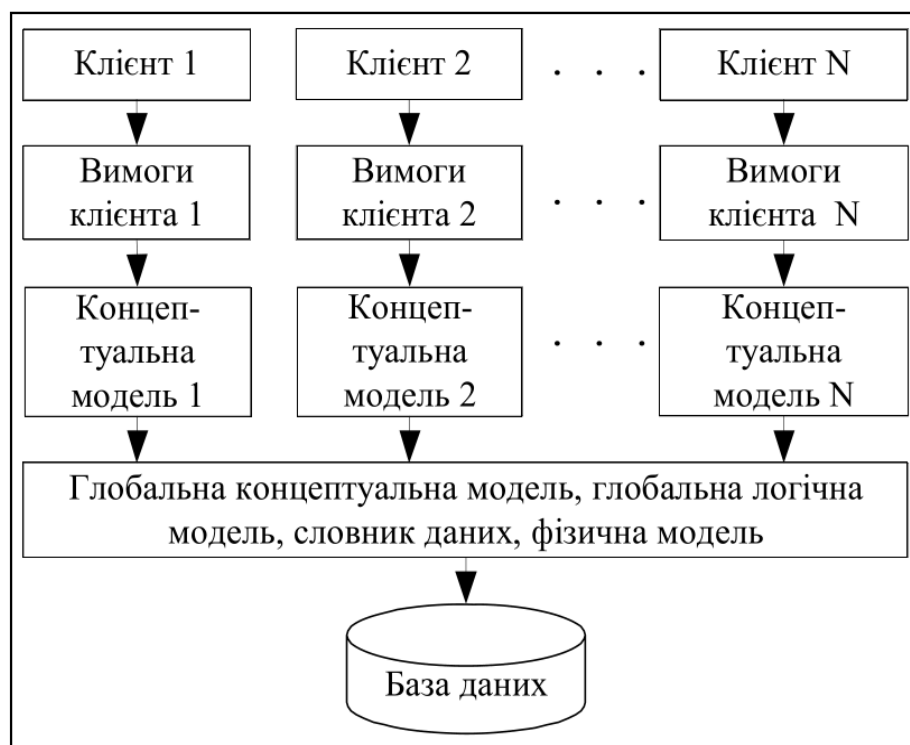


Рис. 12.9. Інтеграція представлень користувачів при проектуванні багатокористувацької бази даних

Після завершення об'єднання локальних моделей може виникнути необхідність перевірити вірність отриманої глобальної моделі, як у відношенні правил нормалізації, так і у відношенні можливості виконання транзакцій, передбачених специфікаціями всіх представлень користувачів. Це викликано тим, що між окремими моделями може існувати несумісність або взаємне перекриття. Простий підхід до об'єднання декількох моделей передбачає, що спочатку об'єднуються дві моделі для отримання нової моделі, а потім до них послідовно додаються інші локальні моделі.

12.5. Проектування розподілених баз даних

На рис. 12.10 показана архітектура розподіленої СКБД. Існують такі схеми розміщення даних в системі:

- централізоване;
- фрагментоване;
- з повною реплікацією;
- з вибірковою реплікацією.

Централізоване розміщення передбачає, що на одному з вузлів створюється і зберігається єдина БД. Доступ до цієї БД мають всі користувачі мережі.

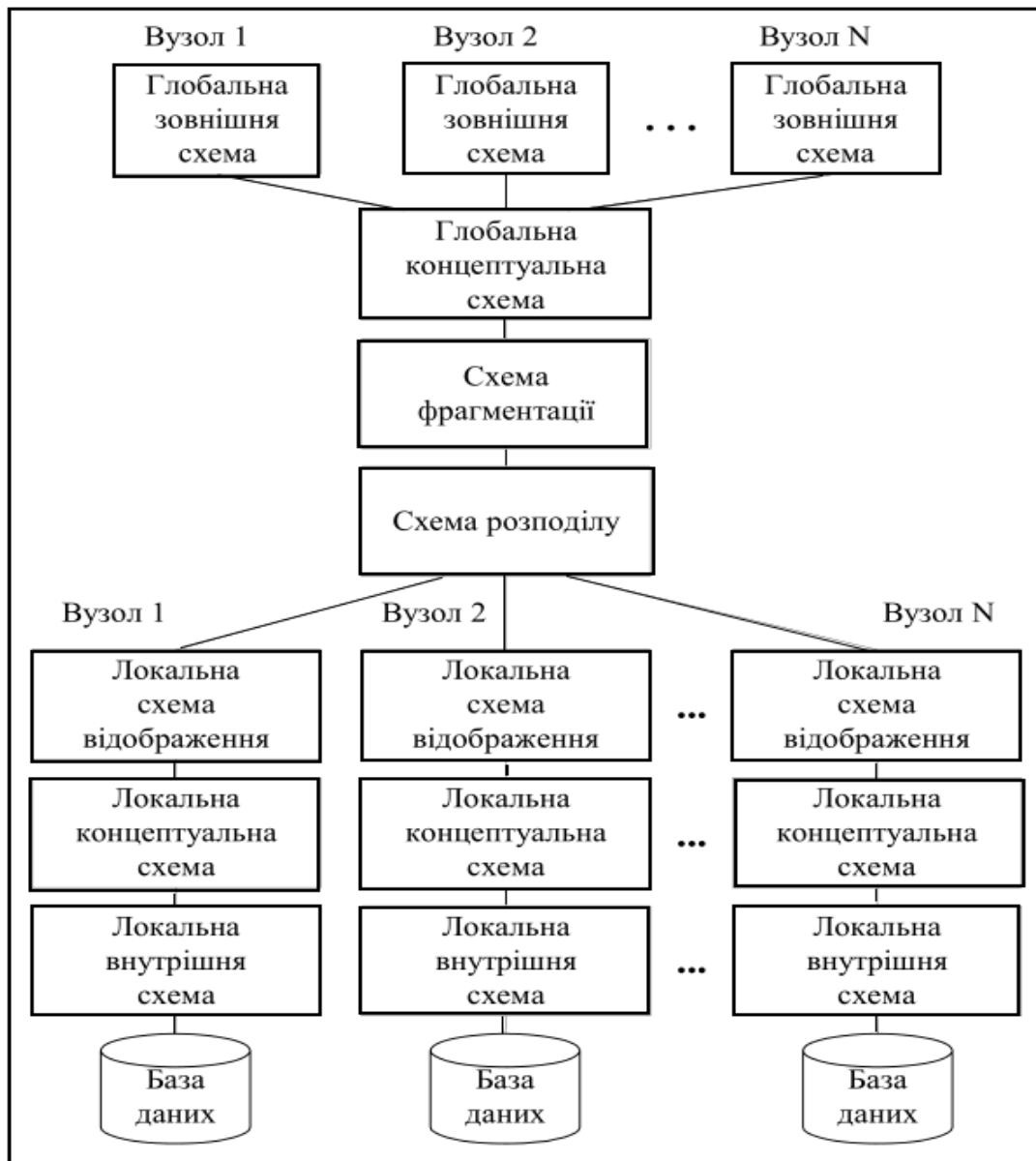


Рис. 12.10. Архітектура розподіленої СКБД

Фрагментоване розміщення передбачає, що БД ділиться на неперетинні фрагменти, кожен з яких розміщується в одному з вузлів системи.

Розміщення з **повною реплікацією** передбачає, що повна копія БД розміщується на кожному вузлі системи.

Розміщення з **вибірковою реплікацією** являє собою комбінацію методів фрагментування, реплікації й централізації. Одні масиви даних поділяються на фрагменти, інші реплікуються, останні зберігаються централізовано.

Реплікація це процес генерації і відтворювання декількох копій даних, які розміщуються на одному або декількох вузлах. Використання реплікацій дозволяє досягнути багатьох переваг (продуктивність, надійність зберігання даних, відновлення даних тощо).

Оновлення даних, що реплікуються, може виконуватися синхронно й асинхронно. Синхронне оновлення передбачає, що всі копії реплікованих даних оновлюються одночасно з оновленням вихідної копії. Асинхронна реплікація передбачає, що оновлення реплікованих даних виконується із затримкою відносно оновлення вихідних даних.

Система підтримує фрагментацію, якщо дане відношення може бути поділене на частини або фрагменти при організації його фізичного зберігання. Фрагментація бажана для підвищення ефективності системи. В цьому випадку дані можуть зберігатися в тому місці, де вони найчастіше використовуються. Це дозволяє досягнути локалізації більшості операцій і зменшити трафік

мережі. Крім того, виключається зберігання даних, які не використовуються локальними додатками, що сприяє підвищенню захисту в системі. Для коректності фрагментації необхідно виконати правила:

- **повнота** – це означає, що кожен елемент даних з вихідного відношення, повинен бути присутнім щонайменше в одному фрагменті;
- **підновленість** – це означає, що вихідне відношення може бути відновлено з його фрагментів за допомогою реляційної алгебри;
- **неперетинність** – це означає, що один елемент не повинен бути присутнім в двох і більше фрагментах.

Фрагментація може бути вертикальна (по атрибутах) і горизонтальна (по кортежах).

На рис.12.11 показана послідовність проектування розподілених БД. Відмінність проектування від централізованих БД полягає в застосуванні фрагментації відношень, реплікації фрагментів і розподілі фрагментів по вузлах мережі. Визначення і розміщення фрагментів повинно виконуватися з урахуванням особливостей використання БД. Під цим розуміється виконання аналізу транзакцій, необхідна продуктивність системи, підтримка цілісності даних тощо.

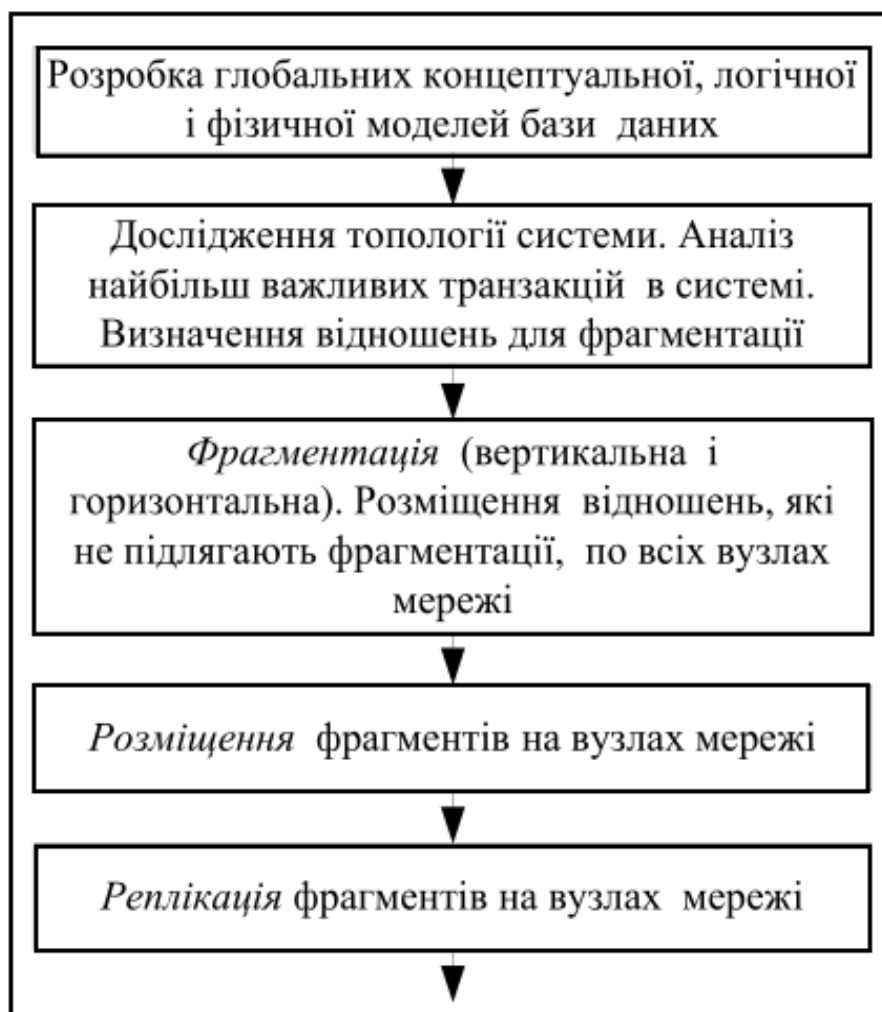


Рис. 12.11. Етапи проектування розподілених баз даних

Згідно з правилами Дейта розподілена СКБД повинна відповідати таким вимогам:

- розподілена система повинна виглядати з точки зору користувача, як звичайна нерозподілена система;
- вузли в розподіленій системі повинні бути незалежні або автономні;
- в системі не повинно бути жодного вузла, без якого система не може функціонувати;
- повинна виконуватись умова незалежності від розташування і користувач може отримувати доступ до БД з будь-якого вузла;

- незалежність від фрагментації – це означає, що користувач може отримати доступ до даних незалежно від засобу їх фрагментації;
- незалежність від реплікації – це означає, що користувач не має засобів доступу до конкретної копії даних і не займається питаннями оновлення всіх копій в БД;
- підтримка обробки розподілених запитів;
- підтримка управління розподіленими транзакціями;
- апаратна, програмна, мережна незалежність, а також незалежність від типу СКБД;
- забезпечення безперервного функціонування.

13. ТРАНЗАКЦІЇ І ЦІЛІСНІСТЬ БАЗ ДАНИХ

Поняття транзакції не входить в реляційну модель даних, тому що транзакції розглядаються не тільки в реляційних СКБД, але і в СКБД інших типів, а також і в інших типах інформаційних систем. Як ми вже знаємо, в багатокористувацьких системах транзакції служать для забезпечення ізольованої роботи окремих користувачів – користувачам, одночасно працюють з однією базою даних, здається, що вони працюють як би в одного користувача системі і не заважають один одному. У той же час транзакції у всіх СКБД використовуються як засоби і методи, які дозволять забезпечувати динамічний відстеження в базі даних узгоджених дій, пов'язаних з узгодженим зміною інформації (цілість даних і відновлення даних після збоїв).

Транзакція – це неподільна, з точки зору впливу на СКБД, послідовність операцій маніпулювання даними. Для користувача транзакція виконується за принципом «все або нічого», тобто або транзакція виконується цілком і переводить базу даних з одного цілісного стану в інший цілісний стан, або, якщо з яких-небудь причин, одна з дій транзакції неможливо, або відбулася яка-небудь порушення роботи системи, база даних повертається в початковий стан, яке було до початку транзакції (відбувається відкат транзакції). З цієї точки зору, транзакції важливі як в багатокористувацьких, так і в одного користувача системах. В одного користувача система транзакції – це логічні одиниці роботи, після виконання яких база даних залишається в цілісному стані. Транзакції також є одиницями відновлення даних після збоїв – відновлюючись, система ліквідує сліди транзакцій, які не встигли успішно завершитися в результаті програмного або апаратного збою. Ці дві властивості транзакцій визначають атомарність (неподільність) транзакції.

13.1. Обмеження цілісності

У реляційній моделі об'єкти реального світу представлені у вигляді сукупності взаємопов'язаних відношень. Під цілісністю будемо розуміти відповідність інформаційної моделі предметної області, що зберігається в базі даних, об'єктам реального світу і їх взаємозв'язкам в кожен момент часу. Будь-яка зміна в предметної області, значуще для побудованої моделі, має відобразитися в базі даних, і при цьому повинна зберігатися однозначна інтерпретація інформаційної моделі в термінах предметної області. Для ілюстрації можливого порушення цілісності бази даних розглянемо наступний приклад.

Приклад 1. Нехай є система, в якій зберігаються дані про підрозділи співробітників, які в них працюють. Список співробітників зберігається в таблиці PERSON (Pers_Id, Pers_Name, Dept_Id), де Pers_Id – ідентифікатор співробітника, Pers_Name – ім'я співробітника, Dept_Id – ідентифікатор підрозділу, в якому працює співробітник. Список підрозділів зберігається в таблиці DEPART (Dep_Id, Dep_Name, Dept_Kol), де Dept_Id – ідентифікатор підрозділу, Dept_Name – найменування підрозділу, Dept_Kol – кількість співробітників в підрозділі.

PERSON		
Pers_Id	Pers_Name	Dept_Id
1	Ковальчук	1
2	Петренко	2
3	Сидорчук	1
4	Радченко	2
5	Шаріпов	1

DEPART

Dept_Id	Dept_Name	Dept_Kol
1	Кафедра інформатики	3
2	Кафедра програмування	2

Обмеження цілісності цієї бази даних полягає в тому, що поле Dept_Kol не може заповнюватися довільними значеннями – це поле повинно містити кількість співробітників, які реально значаться в підрозділі.

З урахуванням цього обмеження можна зробити висновок, що вставка нового співробітника в таблицю не може бути виконана однією операцією. При вставці нового співробітника необхідно одночасно збільшити значення поля Dept_Kol:

Крок 1. Вставити співробітника в таблицю PERSON: INSERT INTO PERSON (6, Марчук, 1)

Крок 2. Збільшити значення поля Dept_Kol: UPDATE DEPART SET Dept = Dept + 1 WHERE Dept_Id = 1

Якщо після виконання першої операції і до виконання другої станеться збій системи, то реально буде виконана тільки перша операція і база даних залишається в нецілісний стані. Для того щоб цього не сталося необхідно виконати ці дві операції як єдине ціле. Ці дві операції виконуються в СКБД атомарно (неподільно) за допомогою транзакції.

Транзакція – це послідовність операторів маніпулювання даними, що виконується як єдине ціле (все або нічого) і переводить базу даних з одного цілісного стану в інший цілісний стан.

Транзакція має чотири важливими властивостями, відомими як властивості Асіда:

1. **(А) Атомарність.** Транзакція виконується як атомарна операція – або виконується вся транзакція повністю, або вона цілком не виконується.
2. **(С) Узгодженість.** Транзакція переводить базу даних з одного узгодженого (цілісного) стану в інший узгоджений (цілісний) стан. Усередині транзакції узгодженість бази даних може порушуватися.
3. **(І) Ізоляція.** Транзакції різних користувачів не повинні заважати один одному (наприклад, як якщо б вони виконувалися строго по черзі).
4. **(Д) Довговічність.** Якщо транзакція виконана, то результати її роботи повинні зберегтися в базі даних, навіть якщо в наступний момент станеться збій системи.

Транзакція зазвичай починається автоматично з моменту приєднання користувача до СКБД і триває до тих пір, поки не відбудеться одна з наступних подій:

1. Подана команда COMMIT WORK (зафіксувати транзакцію).
2. Подана команда ROLLBACK WORK (відкотити транзакцію).
3. Сталося від'єднання користувача від СКБД.
4. Стався збій системи.

Команда COMMIT WORK завершує поточну транзакцію і автоматично починає нову транзакцію. При цьому гарантується, що результати роботи завершеною транзакції фіксуються, тобто зберігаються в базі даних. Деякі системи (наприклад, Visual FoxPro), вимагають подати явну команду BEGIN TRANSACTION для того, щоб почати нову транзакцію.

Команда ROLLBACK WORK призводить до того, що всі зміни, зроблені поточною транзакцією, відкочуються, тобто скасовуються так, як ніби їх взагалі не було. При цьому автоматично починається нова транзакція.

При від'єднанні користувача від СКБД відбувається автоматична фіксація транзакцій. При збої системи відбуваються більш складні процеси. Коротко суть їх зводиться до того, що при наступному запуску системи відбувається аналіз виконувалися до моменту збою транзакцій. Ті транзакції, для яких була подана команда COMMIT WORK, але результати роботи яких не були занесені в базу даних, виконуються знову (накочуються). Ті транзакції, для яких не була подана команда COMMIT WORK, відкочуються. Більш докладно відновлення після збоїв розглядається далі.

Властивості Асіда транзакцій не завжди виконуються в повному обсязі. Особливо це відноситься до властивості І (ізоляція). В ідеалі, транзакції різних користувачів не повинні заважати одна одній, тобто вони повинні бути встановлені таким чином, щоб у користувача створювалася ілюзія, що він в системі один. Найпростіший спосіб забезпечити абсолютну ізолюваність полягає в тому, щоб вибудувати транзакції в чергу і виконувати їх строго одна за одною. Очевидно, при цьому втрачається ефективність роботи системи. Тому реально одночасно виконується кілька транзакцій (суміш транзакцій).

Існує кілька рівнів ізоляції транзакцій. На нижчому рівні ізоляції транзакції можуть реально заважати одна одній, на вищому вони повністю ізолювані. За більшу ізоляцію транзакцій доводиться платити великими накладними витратами системи і уповільненням роботи. Користувачі або адміністратор системи можуть на свій розсуд ставити різні рівні всіх або окремих транзакцій. Більш докладно ізоляція транзакцій розглядається в наступному розділі.

Властивість Д (довговічність) також не є абсолютною властивістю, тому що деякі системи допускають вкладені транзакції. Якщо транзакція Б запущена в транзакції А, і для транзакції Б подана команда COMMIT WORK, то фіксація даних транзакції Б є умовною, тому що зовнішня транзакція А може відкотитися. Результати роботи внутрішньої транзакції Б будуть остаточно зафіксовані тільки якщо буде зафіксована зовнішня транзакція А.

Властивість (С) – узгодженість транзакцій визначається наявністю поняття узгодженості бази даних.

Обмеження цілісності – це деяке твердження, яке може бути істинним або хибним залежно від стану бази даних.

Прикладами обмежень цілісності можуть служити наступні твердження:

1. Вік співробітника не може бути менше 18 і більше 65 років.
2. Кожен співробітник має унікальний табельний номер.
3. Співробітник зобов'язаний числитися в одному відділі.
4. Сума накладної має дорівнювати сумі добутоків цін товарів на кількість товарів для всіх товарів, що входять в накладну.

Як видно з цих прикладів, деякі з обмежень цілісності є обмеженнями реляційної моделі даних. Твердження 2 представляє обмеження, що реалізує цілісність суті. Твердження 3 представляє обмеження, що реалізує кількість посилок цілісності. Інші обмеження є досить довільними твердженнями (1 і 4). Будь-яке обмеження цілісності є семантичним поняттям, тобто з'являється як наслідок певних властивостей об'єктів предметної області та/або їх взаємозв'язків.

База даних знаходиться в узгодженому (цілісному) стані, якщо виконані (задоволені) всі обмеження цілісності, певні для бази даних.

В даному визначенні важливо підкреслити, що повинні бути виконані не всі взагалі обмеження предметної області, а тільки ті, які визначені в базі даних. Для цього необхідно, щоб СКБД володіла розвиненими засобами підтримки обмежень цілісності. Якщо будь-яка СКБД не може відобразити всі необхідні обмеження предметної області, то така база даних хоча і буде перебувати в цілісному стані з точки зору СКБД, але цей стан не буде правильним з точки зору користувача.

Таким чином, узгодженість бази даних є формальна властивість бази даних. База даних не розуміє «сенсу» збережених даних. «Сенсом» даних для СКБД є весь набір обмежень цілісності. Якщо всі обмеження виконані, то СКБД вважає, що дані коректні.

Разом з поняттям цілісності бази даних виникає поняття реакції системи на спробу порушення цілісності. Система повинна не тільки перевіряти, чи не порушуються обмеження в ході виконання різних операцій, а й належним чином реагувати, якщо операція призводить до порушення цілісності. Є два типи реакції на спробу порушення цілісності:

1. **Відмова** виконати «незаконну» операцію.
2. Виконання компенсуючих дій.

Наприклад, якщо система знає, що в поле «Вік_Співробітника» повинні бути цілі числа в діапазоні від 18 до 65, то система відкидає спробу ввести значення віку 66. При цьому може генеруватися яке-небудь повідомлення для користувача.

На противагу цьому, в наведеному вище прикладі 1 система допускає вставку запису про нового співробітника (що призводить до порушення цілісності бази даних), але автоматично виробляє компенсуючі дії, змінюючи значення поля Dept_Kol в таблиці DEPART.

Роботу системи з перевірки обмежень можна зобразити на малюнку 13.1.



Рис. 13.1. Перевірка обмежень при виконанні операції

У деяких випадках система може не виконувати перевірку на порушення обмежень, а відразу виконувати компенсуючі операції. Дійсно, в наведеному прикладі 1 при вставці або видаленні співробітника перевірку проводити не потрібно, тому що результати її відомі заздалегідь – обмеження обов'язково буде порушено. В цьому випадку необхідно відразу приступати до компенсування порушення, яке виникло.

13.2. Класифікація обмежень цілісності

Обмеження цілісності можна класифікувати декількома способами:

1. За способами реалізації.
2. За часом перевірки.
3. По області дії.

13.2.1. Класифікація обмежень цілісності за способами реалізації

Кожна система має свої засоби підтримки обмежень цілісності. Розрізняють два способи реалізації:

1. Декларативна підтримка обмежень цілісності.
2. Процедурна підтримка обмежень цілісності.

Декларативна підтримка обмежень цілісності полягає у визначенні обмежень засобами мови визначення даних (DDL – Data Definition Language). Зазвичай кошти декларативної підтримки цілісності (якщо вони є в СКБД) визначають обмеження на значення доменів і атрибутів, цілісність сутностей (потенційні ключі відношень) і кількість посилань цілісності (цілісність зовнішніх ключів). Декларативні обмеження цілісності можна використовувати при створенні і модифікації таблиць засобами мови DDL або у вигляді окремих тверджень (ASSERTION).

Наприклад, наступний оператор створює таблицю PERSON і визначає для неї деякі обмеження цілісності:

```
CREATE TABLE PERSON
```

```
(Pers_Id INTEGER PRIMARY KEY,
```

```
Pers_Name CHAR (30) NOT NULL,
```

```
Dept_Id REFERENCES DEPART (Dept_Id) ON UPDATE CASCADE ON DELETE CASCADE);
```

Після виконання оператора для таблиці PERSON будуть оголошені такі обмеження цілісності:

1. Поле Pers_Id утворює потенційний ключ відношення.
2. Поле Pers_Name не може містити null-значень.
3. Поле Dept_Id є зовнішнім посиланням на батьківську таблицю DEPART, причому, при зміні або видаленні рядка в батьківській таблиці каскадно повинні бути внесені відповідні зміни в дочірню таблицю.

Засоби декларативної підтримки обмежень описані в стандарті SQL і більш детально розглядаються нижче.

Процедурна підтримка обмежень цілісності полягає у використанні тригерів і збережених процедур.

Не всі обмеження цілісності можна реалізувати декларативно. Прикладом такого обмеження може служити вимога з прикладу 1, яка стверджує, що поле Dept_Kol таблиці DEPART повинно містити кількість співробітників, які реально значаться в підрозділі. Для реалізації цього обмеження необхідно створити тригер, підготовлений до запуску при вставці, модифікації і видалення записів в таблиці PERSON, який коректно змінює значення поля Dept_Kol. Наприклад, при вставці в таблицю PERSON нового рядка, тригер збільшує на одиницю значення поля Dept_Kol, а при видаленні рядка – зменшує. Зауважимо, що при модифікації записів в таблиці PERSON можуть знадобитися навіть більш складні дії. Дійсно, модифікація запису в таблиці PERSON може полягати в тому, що ми переводимо співробітника з одного відділу в інший, змінюючи значення в поле Dept_Id. При цьому необхідно в старому підрозділі зменшити кількість співробітників, а в новому – збільшити.

Крім того, необхідно захиститися від неправильної модифікації рядків таблиці DEPART. Дійсно, користувач може спробувати модифікувати запис про відділ, ввівши невірне значення поля Dept_Kol. Для запобігання подібних дій необхідно створити також тригери, які запускаються при вставці і модифікації записів в таблиці DEPART. Тригер, підготовлений до запуску при видаленні записів з таблиці DEPART не потрібен, тому що вже є обмеження посилальної цілісності, каскадно видаляє записи з таблиці PERSON при видаленні запису з таблиці DEPART.

По суті, наявність обмеження цілісності (як декларативного, так і процедурного характеру) завжди призводить до створення або використання деякого програмного коду, що реалізує це обмеження. Різниця полягає в тому, де такий код зберігається і як він створюється.

Якщо обмеження цілісності реалізовано у вигляді тригерів, то цей програмний код є просто тілом тригера. Якщо використовується декларативне обмеження цілісності, то можливі два підходи:

1. При декларуванні (оголошенні) обмеження текст обмеження зберігається у вигляді деякого об'єкта СКБД, а для реалізації обмеження використовуються вбудовані в СКБД функції, і тоді цей код є внутрішні функції ядра СКБД.
2. При декларуванні обмеження СКБД автоматично генерує тригери, які виконують необхідні дії по перевірці обмежень.

Прикладом використання функцій ядра для перевірки декларативних обмежень є автоматична перевірка унікальності індексів, відповідних потенційним ключам відношень. Як інший приклад можна привести підтримку посилальної цілісності засобами СКБД ORACLE. Обмеження посилальної цілісності є в ORACLE об'єктом бази даних, що зберігає формулювання цього обмеження. Перевірка обмеження виконується функціями ядра ORACLE з посиланням на цей об'єкт. Обмеження цілісності в цьому випадку не можна модифікувати інакше, як використовуючи декларативні оператори створення і модифікації обмежень.

Прикладом генерації нових тригерів для реалізації декларативних обмежень, може служити система Visual FoxPro. Тригери, автоматично згенерували Visual FoxPro при оголошенні обмежень посилальної цілісності можна подивитися і навіть внести в них зміни, так щоб вони могли виконувати деякі додаткові дії.

Якщо система не підтримує ні декларативну підтримку посилальної цілісності, ні тригери (як, наприклад, FoxPro 2.5), то програмний код, що стежить за коректністю бази даних, доводиться розміщувати в призначеному для користувача додатку (такий же код все одно необхідний!). Це сильно ускладнює розробку програм і не захищає від спроб користувачів безпосередньо внести некоректні дані в базу даних. Особливо складно стає в тому випадку, коли є складна база даних і безліч різних додатків, що працюють з нею (наприклад, до бази даних торгового підприємства може звертатися кілька додатків, таких як «Складський облік», «Прийом замовлень», «Головний бухгалтер» тощо). Кожне з таких додатків повинно містити один і той же код, який відповідає за підтримку цілісності бази даних.

13.2.2. Класифікація обмежень цілісності за часом перевірки

За часом перевірки обмеження поділяються на:

1. Негайно перевіряються обмеження.
2. Обмеження з відкладеною перевіркою.

Негайно перевіряються обмеження перевіряються безпосередньо в момент здійснення операції, що може порушити обмеження. Наприклад, перевірка унікальності потенційного ключа перевіряється в момент вставки запису в таблицю. Якщо обмеження порушується, то така операція відкидається. Транзакція, всередині якої відбулося порушення негайно перевіряється затвердження цілісності, зазвичай відкочується.

Обмеження з відкладеною перевіркою перевіряється в момент фіксації транзакції оператором COMMIT WORK. Усередині транзакції обмеження може не виконуватися. Якщо в момент фіксації транзакції виявляється порушення обмеження з відкладеною перевіркою, то транзакція відкочується. Прикладом обмеження, яке не може бути перевірено негайно є обмеження з прикладу 1. Це відбувається тому, що транзакція, яка полягає у вставці нового співробітника в таблицю PERSON, складається не менше ніж з двох операцій – вставки рядка в таблицю PERSON і поновлення рядка в таблиці DEPART. Обмеження, безумовно, є невірним після першої операції і стає вірним після другої операції.

13.2.3. Класифікація обмежень цілісності по області дії

По області дії обмеження поділяються на:

1. Обмеження домену.
2. Обмеження атрибута.
3. Обмеження кортежу.
4. Обмеження відношення.
5. Обмеження бази даних.

Обмеження цілісності домену представляють собою обмеження, що накладаються тільки на допустимі значення домена. Фактично, обмеження домену зобов'язані бути частиною визначення домену.

Наприклад, обмеженням домену «Вік співробітника» може бути умова «Вік співробітника не менше 18 і не більше 65».

Перевірка обмеження. Обмеження домену самі по собі не перевіряються. Якщо на якомусь домені заснований атрибут, то обмеження відповідного домену стає обмеженням цього атрибута.

Обмеження цілісності атрибута представляють собою обмеження, що накладаються на допустимі значення атрибута внаслідок того, що атрибут заснований на якомусь домені. Обмеження атрибута в точності збігаються з обмеженнями відповідного домену. Відмінність обмежень атрибута від обмежень домену в тому, що обмеження атрибута перевіряються.

Якщо логіка предметної області така, що на значення атрибута необхідно накласти додаткові обмеження, крім обмежень домену, то такі обмеження переходять в наступну категорію.

Перевірка обмеження. Обмеження атрибута негайно перевіряється обмеженням. Дійсно, обмеження атрибута не залежить ні від яких інших об'єктів бази даних, крім домену, на якому заснований атрибут. Тому ніякі зміни в інших об'єктах не можуть вплинути на істинність обмеження.

Обмеження цілісності кортежу являють собою обмеження, що накладаються на допустимі значення окремого кортежу відношення, і які не є обмеженням цілісності атрибута. Вимога, що обмеження відноситься до окремого кортежу відношення, означає, що для його перевірки не потрібно ніякої інформації про інші кортежі відношення.

Приклад 2. Атрибут «Вік співробітника» в таблиці «Спецпідрозділ», може мати додаткове обмеження «Вік співробітника не менше 25 і не більше 45», крім того, що цей атрибут вже має обмеження, яке визначається доменом – «Вік співробітника не менше 18 і не більше 65 ».

Наведене обмеження кортежу, по суті, є додатковим обмеженням на значення одного атрибута. У цьому випадку можливі два рішення. Можна оголосити новий домен «Вік співробітника спецпідрозділу» і тоді обмеження кортежу стає обмеженням домену і атрибута, або розглядати це обмеження саме як обмеження кортежу. Обидва рішення мають свої позитивні і негативні сторони.

Тут є деякі можливості для оптимізації. Формально, при зміні значення даного атрибута необхідно перевірити два обмеження – обмеження атрибута і обмеження кортежу. Але в даному випадку обмеження кортежу сильніше обмеження атрибута і досить перевірити тільки обмеження кортежу. Розумно побудована СКБД могла б виявляти такі випадки і зменшувати зайву роботу.

Приклад 3. Для відношення «Співробітники» можна сформулювати наступне обмеження: якщо атрибут «Посада» приймає значення «Директор», то атрибут «Зарплата» містить значення не менше 1000 \$.

Це обмеження пов'язує два атрибути одного кортежу.

Приклад 4. У накладній можна встановити наступну взаємозв'язок атрибутів – «Ціна * Кількість = Сума», що зв'язує атрибути «Ціна», «Кількість», «Сума».

Даний приклад здається неприродним, тому що сума є явно надмірним атрибутом, значення якого просто виводяться з значень інших атрибутів. Тому здається, що краще зберігати тільки два базових атрибута «Ціна» і «Кількість», а суму обчислювати під час виконання запитів в міру необхідності. Так, власне, вимагає реляційна теорія, яка прагне звести надмірність до мінімуму. На практиці, однак, справа йде складніше.

Наприклад, кожен рядок реальної накладної може містити наступні дані про товар (таблиця **АТРИБУТИ ТОВАРУ**):

АТРИБУТИ ТОВАРУ

Найменування атрибуту	Опис атрибуту	Базовий атрибут	Формула обчислюваного атрибуту
Name	Найменування товару	Так	
N	кількість	Так	
P1	Облікова ціна товару	Так	
S1	Облікова сума на всю кількість		$S1 = N * P1$
PerSent	Відсоток націнки на одиницю товару	Так	
P2	Націнка на одиницю товару		$P2 = P1 * PerSent / 100$
S2	Суму націнки на всі кількість		$S2 = N * P2$
P3	Ціну товару з урахуванням націнки		$P3 = P1 + P2$

S3	Суму на всю кількість з урахуванням націнки		$S3 = N * P3$
NDS	відсоток ПДВ	Так	
P4	Сума ПДВ на одиницю товару		$P4 = P2 * NDS / 100$
S4	Сума ПДВ на всю кількість		$S4 = N * P4$
P5	Ціна товару з ПДВ		$P5 = P3 + P4$
S5	Сума на всю кількість з ПДВ		$S5 = N * P5$

Базовими, тобто що вимагають введення даних, є 5 атрибутів. Всі інші атрибути обчислюються з базових. Чи потрібно зберігати відносно тільки базові атрибути, або бажано зберігати всі атрибути, перераховуючи значення обчислюваних атрибутів кожного разу при зміні базових?

Рішення 1. Нехай вирішено зберігати тільки базові атрибути.

Переваги рішення:

1. Структура відношення повністю ненадлишкова.
2. Не потрібно додаткового програмного коду для підтримки цілісності кортежу. Економиться дисковий простір.
3. Зменшується трафік мережі.

Недоліки рішення:

1. У бухгалтерії для формування проводок використовуються, як правило, не базові, а обчислювальні атрибути. Одні і ті ж формули використовуються в багатьох місцях, тому всі оператори відбору даних будуть містити однакові фрагменти коду з одними і тими ж формулами. Є ризик в різних місцях обчислювати одні й ті ж дані по різних формулах.
2. При зміні логіки обчислень (що буває досить часто при зміні законодавства), необхідно змінити одні і ті ж фрагменти коду в усіх місцях, де вони зустрічаються. Це сильно ускладнює модифікацію додатків.
3. Якщо виникає нерегламентований запит, то людина, який формулює запит повинна пам'ятати всі ці формули.

Рішення 2. Припустимо, що відносно вирішено зберігати всі атрибути, в тому числі і обчислювані.

Переваги рішення:

1. Код, що підтримує цілісність кортежу (і містить формули для обчислюваних атрибутів), зберігається в одному місці, наприклад, в тригері, пов'язаному з даним відношенням.
2. При зміні логіки обчислень, зміни в формули потрібно внести тільки в одному місці (в тригері).
3. Запити до бази даних містять менше формул і тому більш прості.
4. Легше формулювати нерегламентовані запити, тому що в запиті використовуються атрибути, що мають для бухгалтера конкретний зміст.

Недоліки рішення:

1. При зміні логіки розрахунку потреба в деяких атрибутах може зникнути, зате може з'явитися потреба в нових атрибутах. Це зажадає перебудови структури відношення, що є досить болючою операцією для працюючої системи.
2. Структура відношення стає більш складною і заплутаною.
3. Збільшується обсяг бази даних.
4. Збільшується трафік мережі.

Як бачимо, обидва рішення мають свої переваги і недоліки. Важливим є те, що програмний код, що містить ці формули, котрі не з'являються ні в якому з цих рішень (це залежить від предметної області). Тільки в одному випадку код зберігається в одному місці, а в іншому може бути «розкиданий» по всьому додатку.

Насправді даний приклад сильно спрощений, тому що ще однією неприємною особливістю наших бухгалтерій є те, що всі розрахунки повинні вестися з певною точністю, а саме – до копійок. Виникає проблема округлення, а це ще більше ускладнює формули для розрахунків цін.

Простий приклад – обчислення ПДВ містить операцію ділення, отже, може призводити до нескінченних дробів типу 15,519999. Такий дріб необхідно округлити до 15.52. Якщо продається одна одиниця товару, то це не страшно, але якщо продається кілька одиниць товару, то суму ПДВ на всю кількість можна обчислювати за різними формулами:

$S4 = N * \text{ROUND}(P2 * \text{NDS} / 100)$ – спочатку округлити при обчисленні ПДВ на одиницю товару, а потім помножити на всю кількість, або $S4 = \text{ROUND}(N * P2 * \text{NDS} / 100)$ – спочатку помножити на всю кількість, а потім округлити до необхідного знаку.

Практика показує, що вірною, безумовно, є перша формула. Дійсно, обчислюючи по першій формулі, ми отримаємо одну і ту ж суму ПДВ незалежно від того, продали ми одну партію товару, що містить 50 одиниць, або продали 50 партій по одній одиниці в кожній.

При обчисленнях по другій формулі сума ПДВ в партії, що складається з 50 одиниць товару відрізняється від суми ПДВ по 50 партіям по одній одиниці товару в кожній.

Обмеження цілісності відношення представляють обмеження, що накладаються тільки на допустимі значення окремого відношення, і які не є обмеженням цілісності кортежу. Вимога, що обмеження відноситься до окремого відношення, означає, що для його перевірки не потрібно інформації про інші відношення (в тому числі не потрібно посилань по зовнішньому ключу на кортежі цього ж відношення).

Приклад 5. Обмеження цілісності сутності, що задається потенційним ключем відношення, є обмеженням відношення, тому що для його перевірки необхідно мати інформацію про всі кортежі відношення (точніше, про всіх зайнятих в даний момент значеннях потенційного ключа).

Приклад 6. Обмеження цілісності, що визначаються наявністю функціональних, багатозначних залежностей і залежностей з'єднання, є обмеженнями відношення.

Приклад 7. Припустимо, що відносно PERSON (див. Приклад 1) задано наступне обмеження – в кожному відділі має бути не менше двох співробітників. Тобто, кількість рядків з однаковим значенням Dept_Id має бути не менше 2.

Для того щоб ввести в дію (оголосити) це обмеження, необхідно, щоб в відношення вже були вставлені деякі кортежі.

Приклад 8. Обмеження цілісності, яке визначається вимогою, що деяка таблиця повинна бути не порожня, є обмеженнями відношення.

Перевірка обмеження. До моменту перевірки обмеження відношення повинні бути перевірені обмеження цілісності кортежів цього відношення.

Обмеження відношення може бути як негайно перевіряється обмеженням, так і обмеженням з відкладеною перевіркою.

Обмеження відношення, що є обмеженням потенційного ключа (приклад 5) є негайно перевіряється обмеженням.

Обмеження, певне наявністю функціональної залежності атрибутів, також є негайно перевіряється обмеженням.

Обмеження ж, певні багатозначною залежністю або залежністю з'єднання, є обмеженнями з відкладеною перевіркою. Дійсно, ці обмеження вимагають, щоб кортежі вставлялися і віддалялися цілими групами. Це неможливо зробити, якщо виконувати перевірку після кожної одиночної вставки або видалення кортежу.

Обмеження в прикладі 7 здається негайно перевіряється. Дійсно, можна відразу після вставки або видалення кортежу перевірити, чи виконується обмеження, і, якщо воно не виконується, то відкотити операцію. Але, проте, в цьому випадку, неможливо вставити жоден новий кортеж для нового відділу. У новий відділ необхідно вставити відразу не менше двох співробітників. Таким чином, це обмеження з відкладеною перевіркою.

Обмеження з прикладу 8 має сенс перевіряти тільки при видаленні кортежів з відношення. Це обмеження може бути як негайно перевіряється, так і відкладеною.

Обмеження цілісності бази даних представляють обмеження, що накладаються на значення двох або більше пов'язаних між собою відношень (в тому числі відношення може бути пов'язано саме з собою).

Приклад 9. Обмеження цілісності посилань, що задається зовнішнім ключем відношення, є обмеженням бази даних.

Приклад 10. Обмеження на таблиці DEPART і PERSON з прикладу 1 є відношенням бази даних, тому що воно пов'язує дані, розміщені в різних таблицях.

Перевірка обмеження. До моменту перевірки обмеження бази даних повинні бути перевірені обмеження цілісності відношень.

Обмеження бази даних може бути як негайно перевіряється обмеженням, так і обмеженням з відкладеною перевіркою.

13.3. Реалізація декларативних обмежень цілісності засобами SQL

13.3.1. Загальні принципи реалізації обмежень засобами SQL

Стандарт SQL не передбачає процедурних обмежень цілісності, що реалізуються за допомогою тригерів і збережених процедур. У стандарті SQL 92 відсутнє поняття «тригер», хоча тригери є у всіх промислових СКБД SQL-типу. Таким чином, реалізація обмежень засобами конкретної СКБД має більшу гнучкість, ніж з використанням виключно стандартних засобів SQL.

Стандарт SQL дозволяє задавати декларативні обмеження такими способами:

1. Як обмеження домену.
2. Як обмеження, що входять до визначення таблиці.
3. Як обмеження, що зберігаються в базі даних у вигляді незалежних тверджень (assertion).

Допускаються як негайно перевіряються, так і обмеження з відкладеною перевіркою. Режим перевірки відкладених обмежень можна в будь-який момент змінити так, щоб обмеження перевірялося:

1. Після виконання кожного оператора, що змінює вміст таблиці, до якої відноситься дане обмеження.
2. При завершенні кожної транзакції, що включає оператори, що змінюють вміст таблиць, до яких відносяться дане обмеження.
3. У будь-який проміжний момент, якщо користувач ініціює перевірку.

При визначенні обмеження вказується тип перевірки обмеження – чи є це обмеження, яке не відкладається (NOT DEFERRED) або може відкладатись (DEFERRED). У другому випадку можна задати процедуру за замовчуванням: перевіряти негайно або перевіряти по завершенні транзакції. Таким чином, можна визначити обмеження, що потенційно відкладається, яке за замовчуванням перевіряється негайно. У будь-який момент режим перевірки такого обмеження можна змінити на відкладений і навпаки. Режим перевірки може бути змінений для одного обмеження або відразу для всіх обмежень, що потенційно відкладаються. Якщо обмеження визначено як невідкладне, то тип такого обмеження змінити не можна і обмеження завжди перевіряється негайно.

Елементи процедурності все ж присутні в стандарті SQL у вигляді так званих дій, виконуваних за посиланням (referential triggered actions). Ці дії визначають, що буде відбуватися при зміні значення батьківського ключа, на який посиляється деякий зовнішній ключ. Ці дії можна задавати незалежно для операцій оновлення (ON UPDATE) або для операцій видалення (ON DELETE) записів у батьківському відношенні. Стандартом SQL визначається 4 типи дій, виконуваних за посиланням:

- **CASCADE**. Зміни значення батьківського ключа автоматично призводять до таких же змін пов'язаного з ним значення зовнішнього ключа. Видалення кортежу в батьківському відношенні призводить до видалення пов'язаних з ним кортежів в дочірньому відношенні.
- **SET NULL**. Всі зовнішні ключі, які посиляються на оновлений або віддалений батьківський ключ отримують значення NULL.

- **SET DEFAULT.** Всі зовнішні ключі, які посилаються на оновлений або віддалений батьківський ключ отримують значення, прийняті за замовчуванням для цих ключів.
- **NO ACTION.** Значення зовнішнього ключа не змінюються. Якщо операція призводить до порушення посилальної цілісності (з'являються «висячі» посилання), то така операція не виконується.

Як видно, дії, виконувані за посиланням, фактично є вбудованими в СКБД тригерами. Дії типу CASCADE, SET NULL і SET DEFAULT є компенсують операціями, викликає при спробі порушити посилальну цілісність.

13.3.2. Синтаксис обмежень стандарту SQL

Поняття обмеження використовується в багатьох операторах визначення даних (DDL).

обмеження check:: =

CHECK предикат

обмеження таблиці:: = [CONSTRAINT Ім'я обмеження] { {PRIMARY KEY (Ім'я стовпця., ..)} | {UNIQUE (Ім'я стовпця., ..)} | {FOREIGN KEY (Ім'я стовпця., ..) REFERENCES Ім'я таблиці [(Ім'я стовпця., ..)] [Посилальна специфікація]} | {Обмеження check}} [Атрибути обмеження]

обмеження стовпця:: = [CONSTRAINT Ім'я обмеження] { {NOT NULL} | {PRIMARY KEY} | {UNIQUE} | {REFERENCES Ім'я таблиці [(Ім'я стовпця)] [Посилальна специфікація]} | {Обмеження check}} [Атрибути обмеження]

посилальна специфікація:: = [MATCH {FULL | PARTIAL}] [ON UPDATE {CASCADE | SET NULL | SET DEFAULT | NO ACTION}] [ON DELETE {CASCADE | SET NULL | SET DEFAULT | NO ACTION}]

атрибути обмеження:: = {DEFERRABLE [INITIALLY DEFERRED | INITIALLY IMMEDIATE]} | {NOT DEFERRABLE}

Обмеження типу CHECK містить предикат, який може приймати значення TRUE, FALSE і UNKNOWN (NULL). Обмеження типу CHECK може бути використано як частина опису домену, таблиці, стовпці таблиці або окремого обмеження цілісності – ASSERTION. Обмеження вважається порушеним, якщо предикат обмеження приймає значення FALSE.

Приклад 11. Приклад обмеження типу CHECK:

CHECK (Salespeople.Salary IS NOT NULL) OR (Salespeople.Commission IS NOT NULL)

Дане обмеження стверджує, що кожен продавець повинен мати або ненульову зарплату, або ненульові комісійні.

CHECK EXIST (SELECT * FROM Salespeople)

Дане обмеження стверджує, що список продавців не може бути порожнім.

Обмеження PRIMARY KEY для таблиці або стовпця означає, що група з одного або декількох стовпців утворюють потенційний ключ таблиці. Це означає, що комбінація значень в PRIMARY KEY повинна бути унікальною для кожного рядка таблиці. Дубльовані значення або значення, що містять NULL, будуть відкинуті. Для однієї таблиці може бути визначено єдине обмеження PRIMARY KEY. У термінах стандарту SQL це називається первинним ключем таблиці.

Обмеження UNIQUE для таблиці або стовпця означає, що група з одного або декількох стовпців утворюють потенційний ключ таблиці, в якому допускаються значення NULL. Це означає, що два рядки, що містять однакові і не рівні NULL-значення, вважаються такими, що порушують унікальність і не допускаються. Два рядки, що містять NULL-значення вважаються різними і допускаються. Для однієї таблиці може бути визначено декілька обмежень UNIQUE.

З точки зору реляційної моделі, обмеження типу UNIQUE не визначає потенційний ключ, тому що потенційний ключ не повинен містити NULL-значень.

Обмеження FOREIGN KEY REFERENCES для таблиці і обмеження REFERENCES для стовпця визначають зовнішній ключ таблиці. Обмеження REFERENCES для стовпця визначає простий зовнішній ключ, тобто ключ, що складається з однієї колонки. Обмеження FOREIGN KEY REFERENCES для таблиці може визначати як простий, так і складний зовнішній ключ, тобто ключ, що складається з декількох колонок таблиці.

Стовпець або група стовпців таблиці, на яку посилається зовнішній ключ, повинна мати обмеження PRIMARY KEY або UNIQUE. Стовпці, на які посилається зовнішній ключ, повинні мати той же тип даних, що і стовпці, що входять до складу зовнішнього ключа. Таблиця може мати посилання на себе. Обмеження зовнішнього ключа порушується, якщо значення, присутні в зовнішньому ключі, не збігаються зі значеннями відповідного ключа батьківської таблиці ні для одного рядка з батьківської таблиці. Операції, що призводять до порушення обмеження зовнішнього ключа, відкидаються. Як повинні збігатися значення зовнішнього ключа і ключа батьківської таблиці, а також, які дії необхідно виконати при змінах ключів в батьківській таблиці, описані нижче в посилальній специфікації.

Обмеження NOT NULL стовпця не допускає появи в стовпці NULL-значень.

Пропозиція MATCH FULL вимагає повного збігу значень зовнішнього і первинного ключів. Пропозиція MATCH PARTIAL допускає частковий збіг значень зовнішнього і первинного ключів. Пропозиція MATCH може бути також пропущеним. Для випадку MATCH PARTIAL в дочірній таблиці можуть з'явитися рядки, що мають значення зовнішнього ключа, неунікальні збігаються зі значеннями батьківського ключа. Тобто, один рядок дочірньої таблиці може мати неунікальні посилання на кілька рядків батьківської таблиці. Це дуже сильно відрізняється від реляційної моделі даних, і ця відмінність провокується допущенням NULL-значень.

14. ТРАНЗАКЦІЇ І ПАРАЛЕЛІЗМ

У даній лекції вивчаються можливості паралельного виконання транзакцій декількома користувачами, тобто властивість (I) – ізолюваність транзакцій.

Сучасні СКБД є багатокористувацькими системами, тобто допускають паралельну одночасну роботу великої кількості користувачів. При цьому користувачі не повинні заважати один одному. Оскільки логічною одиницею роботи для користувача є транзакція, то робота СКБД повинна бути організована так, щоб у користувача складалося враження, що їх транзакції виконуються незалежно від транзакцій інших користувачів.

Найпростіший і очевидний спосіб забезпечити таку ілюзію у користувача полягає в тому, щоб всі, хто вступає в транзакції вибудовувати в єдину чергу і виконувати строго по черзі. Такий спосіб не годиться з очевидних причин – втрачається перевага паралельної роботи. Таким чином, транзакції необхідно виконувати одночасно, але так, щоб результат був би такий же, як якщо б транзакції виконувалися по черзі. Складність полягає в тому, що якщо не вживати ніяких спеціальних заходів, то дані змінені одним користувачем можуть бути змінені транзакцією іншого користувача раніше, ніж закінчиться транзакція першого користувача. В результаті, в кінці транзакції перший користувач побачить не результати своєї роботи, а невідомо що. Одним із способів забезпечити незалежну паралельну роботу декількох транзакцій є метод блокувань.

14.1. Робота транзакцій в суміші

Транзакція розглядається як послідовність елементарних атомарних операцій. Атомарність окремої елементарної операції полягає в тому, що СКБД гарантує, що, з точки зору користувача, будуть виконані дві умови:

1. Ця операція буде виконана повністю або не виконано зовсім (атомарність – все або нічого).
2. Під час виконання цієї операції не виконуються ніякі інші операції інших транзакцій (сувора черговість елементарних операцій).

Наприклад, елементарними операціями транзакції будуть зчитування сторінки даних з диска або запис сторінки даних на диск (сторінка даних – це мінімальна одиниця для дискових операцій СКБД). Умова 2 насправді є саме логічною умовою, тому що реально система може виконувати кілька різних елементарних операцій в один і той же момент. Наприклад, дані можуть зберігатися на декількох фізично різних дисках і операції читання-запису на ці диски можуть виконуватися одночасно.

Елементарні операції різних транзакцій можуть виконуватися в довільній черговості (звичайно, всередині кожної транзакції послідовність елементарних операцій цієї транзакції є строго визначеною). Наприклад, якщо є кілька транзакцій, що складаються з послідовності операцій елементарних:

$$\begin{aligned} T &= \{T_1, T_2, T_3, \dots, T_n\}, \\ Q &= \{Q_1, Q_2, Q_3, \dots, Q_m\}, \\ S &= \{S_1, S_2, S_3, \dots, S_l\} \end{aligned}$$

то реальна послідовність, в якій СКБД виконує ці транзакції може бути, наприклад, такою:

$$\{T_1, Q_1, T_2, S_1, T_3, S_2, S_3, Q_2, \dots\}.$$

Набір з декількох транзакцій, елементарні операції яких чергуються один з одним, називається сумішню транзакцій.

Послідовність, в якій виконуються елементарні операції заданого набору транзакцій, називається графіком запуску набору транзакцій.

Очевидно, що для заданого набору транзакцій може бути кілька (взагалі кажучи, досить багато) різних графіків запуску.

Забезпечення ізолюваності користувачів, таким чином, зводиться до вибору підходящого (в якомусь сенсі правильного) графіка запуску транзакцій. Одночасно з цим графік запуску повинен

бути оптимальним в деякому сенсі, наприклад, давати мінімальне середнє час виконання транзакцій кожним користувачем.

Далі ми уточнимо поняття «правильного» графіка і зробимо деякі зауваження про оптимальність. Яким чином транзакції різних користувачів можуть заважати один одному? Розрізняють три основні проблеми паралелізму:

1. **Проблема втрати результатів поновлення.**
2. **Проблема незафіксованої залежності** (читання «брудних» даних, неакуратне зчитування).
3. **Проблема несумісного аналізу.**

Розглянемо докладно ці проблеми.

Розглянемо дві транзакції, А і В, які запускаються відповідно до деякими графіками. Нехай транзакції працюють з деякими об'єктами бази даних, наприклад, з рядками таблиці. Операцію читання рядка Р будемо позначати $P = P_0$, де P_0 – прочитане значення. Операцію запису значення P_1 до рядка Р будемо позначати $P_1 \rightarrow P$.

Проблема втрати результатів поновлення. Дві транзакції по черзі записують деякі дані в одну і ту ж рядок і фіксують зміни (табл. 14.1).

Таблиця 14.1. Проблема втрати результатів поновлення

Транзакція А	Час	Транзакція В
Читання $P = P_0$	t_1	---
---	t_2	Читання $P = P_0$
Запис $P_1 \rightarrow P$	t_3	---
---	t_4	Запис $P_2 \rightarrow P$
Фіксація транзакції	t_5	---
---	t_6	Фіксація транзакції
Втрата результату поновлення		

Після закінчення обох транзакцій, рядок Р містить значення P_2 , занесене більш пізньою транзакцією В. Транзакція А нічого не знає про існування транзакції В, і природно очікує, що в рядку Р міститься значення P_1 . Таким чином, транзакція А втратила результати своєї роботи.

Проблема незафіксованої залежності (читання «брудних» даних, неакуратне зчитування). Транзакція В змінює дані в рядку. Після цього транзакція А читає змінені дані і працює з ними. Транзакція В відкочується і відновлює старі дані (табл. 14.2).

Таблиця 14.2. Проблема незафіксованою залежності

Транзакція А	Час	Транзакція В
---	t_1	Читання $P = P_0$
---	t_2	Запис $P_1 \rightarrow P$
Читання $P = P_1$	t_3	---
Робота з прочитаними даними P_1	t_4	---
---	t_5	Відкат транзакції $P_0 \rightarrow P$
Фіксація транзакції	t_6	---
Робота з «брудними» даними		

З чим же працювала транзакція А?

Транзакція А в своїй роботі використовувала дані, яких немає в базі даних. Більш того, транзакція А використовувала дані, яких немає, і не було в базі даних! Дійсно, після відкоту транзакції В, повинна відновитися ситуація, як якщо б транзакція В взагалі ніколи не виконувалася. Таким чином, результати роботи транзакції А некоректні, тому що вона працювала з даними, були відсутні в базі даних.

Проблема несумісного аналізу. Проблема несумісного аналізу включає кілька різних варіантів:

1. *Є повторно зчитуваною.*
2. *Фіктивні елементи* (фантоми).
3. *Власне, несумісний аналіз.*

Є повторно зчитуваною. Транзакція А двічі читає один і той же рядок. Між цими читаннями вклинюється транзакція В, яка змінює значення в рядку (табл. 14.3).

Таблиця 14.3. Є повторно зчитуваною

Транзакція А	Час	Транзакція В
Читання $P = P_0$	t_1	---
---	t_2	Читання $P = P_0$
---	t_3	Запис $P_1 \rightarrow P$
---	t_4	Фіксація транзакції
Повторне читання $P = P_1$	t_5	---
Фіксація транзакції	t_6	---
Є повторно зчитуваною		

Транзакція А нічого не знає про існування транзакції В, і, тому що сама вона не змінює значення в рядку, то очікує, що після повторного читання значення буде тим же самим.

Транзакція А працює з даними, які, з точки зору транзакції А, мимовільно змінюються.

Фіктивні елементи (фантоми). Ефект фіктивних елементів дещо відрізняється від попередніх транзакцій тим, що тут за один крок виконується досить багато операцій – читання одночасно декількох рядків, що задовольняють деякій умові.

Транзакція А двічі виконує вибірку рядків з однією і тією ж умовою. Між вибірками вклинюється транзакція В, яка додає новий рядок, що задовольняє умові відбору (табл. 14.4).

Таблиця 14.4. Фіктивні елементи (фантоми)

Транзакція А	Час	Транзакція В
Вибірка рядків, що задовольняють умові α . (Відібрано n рядків)	t_1	---
---	t_2	Вставка нового рядка, що задовольняє умові α .
---	t_3	Фіксація транзакції
Вибірка рядків, що задовольняють умові α . (Відібрано $n + 1$ рядків)	t_4	---
Фіксація транзакції	t_5	---
З'явилися рядки, яких раніше не було		

Транзакція А нічого не знає про існування транзакції В, і, тому що сама вона не змінює нічого в базі даних, то очікує, що після повторного відбору будуть відібрані ті ж самі рядки.

Транзакція А в двох однакових вибірках рядків отримала різні результати.

Власне, несумісний аналіз. Ефект власне несумісного аналізу також відрізняється від попередніх прикладів тим, що в суміші присутні дві транзакції – одна довга, інша коротка (табл. 14.5).

Довга транзакція виконує деякий аналіз по всій таблиці, наприклад, підраховує загальну суму грошей на рахунках клієнтів банку для головного бухгалтера. Нехай на всіх рахунках знаходяться однакові суми, наприклад, по 100 грн. Коротка транзакція в цей момент виконує переклад 50 грн з одного рахунку на інший так, що загальна сума за всіма рахунками не змінюється.

Таблиця 14.5. Власне, несумісний аналіз

Транзакція А	Час	Транзакція В
Читання рахунку $P_1 = 100$ і підсумовування. $SUM = 100$	t_1	---
---	t_2	Зняття грошей з рахунку P_3 . $P_3 : 100 \rightarrow 50$
---	t_3	Приміщення грошей на рахунок P_1 . $P_1 : 100 \rightarrow 150$
---	t_4	Фіксація транзакції
Читання рахунку $P_2 = 100$ і підсумовування. $SUM = 200$	t_5	---
Читання рахунку $P_3 = 50$ і підсумовування. $SUM = 250$	t_6	---
Фіксація транзакції	t_7	---
Сума 250 грн за всіма рахунками неправильна - має бути 300 грн		

Хоча транзакція В все зробила правильно – гроші переведені без втрати, але в результаті транзакція А підрахувала невірну загальну суму.

Так як транзакції з переказу грошей йдуть зазвичай безперервно, то в даній ситуації слід очікувати, що головний бухгалтер ніколи не дізнається, скільки ж грошей в банку.

14.2. Конфлікти між транзакціями

Отже, аналіз проблем паралелізму показує, що якщо не вживати спеціальних заходів, то при роботі в суміші порушується властивість (І) транзакцій – ізолюваність. Транзакції реально заважають одна одній отримувати правильні результати.

Однак не всякі транзакції заважають одна одній. Очевидно, що транзакції не заважають одна одній, якщо вони звертаються до різних даних або виконуються в різний час.

Транзакції називаються конкуруючими, якщо вони перетинаються за часом і звертаються до одних і тих самих даних.

В результаті конкуренції за даними між транзакціями виникають конфлікти доступу до даних. Розрізняють такі види конфліктів:

1. *WW (Запис – Запис)*. Перша транзакція змінила об'єкт і не закінчилася. Друга транзакція намагається змінити цей об'єкт. Результат – втрата поновлення.
2. *RW (Читання – Запис)*. Перша транзакція прочитала об'єкт і не закінчилася. Друга транзакція намагається змінити цей об'єкт. Результат – несумісний аналіз (повторюваною зчитування).
3. *WR (Запис – Читання)*. Перша транзакція змінила об'єкт і не закінчилася. Друга транзакція намагається прочитати цей об'єкт. Результат – читання «брудних» даних.

Конфлікти типу RR (Читання – Читання) відсутні, тому що дані при читанні не змінюються. Інші проблеми паралелізму (фантоми і власне несумісний аналіз) є більш складними, тому що принципова відмінність їх у тому, що вони не можуть виникати при роботі з одним об'єктом. Для виникнення цих проблем потрібно, щоб транзакції працювали з цілими наборами даних.

Графік запуску набору транзакцій називається послідовним, якщо транзакції виконуються строго по черзі, тобто елементарні операції транзакцій чергуються одна з одною.

Якщо графік запуску набору транзакцій містить елементарні операції транзакцій, які чергуються, то такий графік називається *чергованим*.

При виконанні послідовного графіка гарантується, що транзакції виконуються правильно, тобто при послідовному графіку транзакції не «відчувають» присутності одна одної.

Два графіка називаються еквівалентними, якщо при їх виконанні буде отримано один і той же результат, незалежно від початкового стану бази даних.

Графік запуску транзакції називається вірним (серіалізованим), якщо він еквівалентний якому-небудь послідовного графіку.

При виконанні двох різних послідовних (а, отже, вірних) графіків, що містять один і той же набір транзакцій, можуть бути отримані різні результати. Дійсно, нехай транзакція А полягає в дії «Скласти X з 1», а транзакція В – «Подвоїти X». Тоді послідовний графік {А, В} дасть результат 2 (X + 1), а послідовний графік {В, А} дасть результат 2X + 1. Таким чином, може існувати кілька вірних графіків запусків транзакцій, що призводять до різних результатів при одному і тому ж початковому стані бази даних.

Завдання забезпечення ізолюваної роботи користувачів не зводиться просто до знаходження правильних (серіальних) графіків запусків транзакцій. Якби цього було достатньо, то кращим був би найпростіший спосіб серіалізації – ставити транзакції в загальну чергу в міру їх надходжень і виконувати строго послідовно. Таким способом автоматично отримують правильний (серіальний) графік. Проблема в тому, що цей графік буде неоптимальним з точки зору загальної продуктивності системи. Виходить ситуація, в якій борються протилежні сили – з одного боку, прагнення забезпечити серіальність за рахунок погіршення загальної ефективності роботи, з іншого боку, прагнення поліпшити загальну ефективність за рахунок погіршення серіальності.

Один крайній випадок (виконання транзакцій по черзі) ми розглянули. Розглянемо інший крайній випадок – спробуємо досягти оптимального графіка – тобто графіка з максимальною ефективністю виконання транзакцій. Для цього спочатку потрібно уточнити поняття «оптимальність». З кожним можливим графіком запуску транзакцій ми можемо пов'язати значення якоїсь вартісної функції. Як вартісну функцію можна взяти, наприклад, сумарний час виконання всіх транзакцій в наборі. Час виконання однієї транзакції вважається від моменту, коли транзакція виникла і до моменту, коли транзакція виконала свою останню елементарну операцію. Це час складається з наступних компонентів:

1. Час очікування початку транзакції – це час, який проходить від моменту, коли транзакція виникла до моменту, коли почала реально виконуватися її перша елементарна операція.
2. Сума часів виконання елементарних операцій транзакції.
3. Сума часів всіх елементарних операцій інших транзакцій, які вклинились між елементарними операціями транзакції.

Оптимальним буде графік, який дає мінімум вартісної функції. Очевидно, оптимальність графіка запуску залежить від вибору вартісної функції, тобто графік, оптимальний з точки зору одних критеріїв (наприклад, з точки зору наведеної функції вартості) не оптимальний з точки зору інших критеріїв (наприклад, з точки зору досягнення максимально швидкого початку виконання кожної транзакції).

Розглянемо наступну гіпотетичну ситуацію. Припустимо, що нам заздалегідь на деякий проміжок часу наперед відомо, які транзакції в які моменти надійдуть, тобто заздалегідь відома вся майбутня суміш транзакцій і моменти надходження кожної транзакції:

1. Транзакція $T^1 = \{T_1^1, T_2^1, T_3^1, \dots, T_{n_1}^1\}$ надійде в момент t_1 .
2. Транзакція $T^2 = \{T_1^2, T_2^2, T_3^2, \dots, T_{n_2}^2\}$ надійде в момент t_2 .
3. Транзакція $T^m = \{T_1^m, T_2^m, T_3^m, \dots, T_{n_m}^m\}$ надійде в момент t_m .

В цьому випадку, із-за того, що набір всіх транзакцій заздалегідь відомий, теоретично можна перебрати всі можливі варіанти графіків запусків (їх кінцеве число, хоча і дуже велике), і вибрати з них ті графіки, які, по-перше, правильні, а по-друге, оптимальні за обраним критерієм. У цьому випадку оптимальний графік запуску транзакцій можна досягти.

В реальній ситуації, однак, невідомо не лише які транзакції будуть надходити в які моменти часу, але і невідома тривалість періоду часу, що охоплює набір транзакцій. Реально, система може безперервно працювати кілька днів або місяців і в цьому випадку набором транзакцій буде набір всіх транзакцій за цей період. З іншого боку, припинення роботи сервера може статися в будь-який момент або по команді адміністратора системи, або в результаті збою. Необхідно, отже, щоб система працювала так, щоб до будь-якого моменту часу набір виконаних транзакцій і тих, що виконуються в цей момент був би правильним і не дуже далекий від оптимального.

Так як транзакції не заважають одна одній, якщо вони звертаються до різних даних або виконуються в різний час, то є два способи вирішити конкуренцію між вступниками в довільні моменти транзакціями:

«Пригальмовувати» деякі з вступників транзакцій настільки, наскільки це необхідно для забезпечення правильності суміші транзакцій в кожен момент часу (тобто забезпечити, щоб конкуруючі транзакції виконувалися в різний час).

Надати конкуруючим транзакціям «різні» екземпляри даних (тобто забезпечити, щоб конкуруючі транзакції працювали з різними версіями даними).

Перший метод – «пригальмовування» транзакцій – реалізується шляхом використання блокувань різних видів або методом часових міток.

Другий метод – надання різних версій даних – реалізується шляхом використання даних з журналу транзакцій.

14.3. Блокування

Основна ідея блокувань полягає в тому, що якщо для виконання деякої транзакції необхідно, щоб деякий об'єкт не змінювався без відома цієї транзакції, то цей об'єкт повинен бути заблокований, тобто доступ до цього об'єкта з боку інших транзакцій обмежується на час виконання транзакції, що викликала блокування.

Розрізняють два типи блокувань:

1. **Монопольні блокування** (X-блокування, X-locks – eXclusive locks) – блокування без взаємного доступу (блокування запису).
2. **Спільні блокування** (S-блокування, S-locks – Shared locks) – блокування з взаємним доступом (блокування читання).

Якщо транзакція А блокує об'єкт за допомогою X-блокування, то будь-який доступ до цього об'єкта з боку інших транзакцій відкидається.

Якщо транзакція А блокує об'єкт за допомогою S-блокування, то:

- запити з боку інших транзакцій на X-блокування цього об'єкта будуть відкинуті;
- запити з боку інших транзакцій на S-блокування цього об'єкта будуть прийняті.

Правила взаємного доступу до заблокованих об'єктів можна представити у вигляді такої матриці сумісності блокувань. Якщо транзакція А наклала блокування на деякий об'єкт, а транзакція В після цього намагається накласти блокування на цей же об'єкт, то успішність блокування транзакцією В об'єкта описується таблицею 14.6.

Таблиця 14.6. Матриця сумісності S- і X-блокувань

	Транзакція В намагається накласти блокування:	
Транзакція А наклала блокування:	S-Блокування	X-Блокування
S-Блокування	Так	НІ (Конфлікт RW)
X-Блокування	НІ (Конфлікт WR)	НІ (Конфлікт WW)

Три випадки, коли транзакція В не може блокувати об'єкт, відповідають трьом видам конфліктів між транзакціями.

Доступ до об'єктів бази даних на читання і запис повинен здійснюватися відповідно до наступного протоколу доступу до даних:

1. Перш ніж прочитати об'єкт, транзакція повинна накласти на цей об'єкт S-блокування.
2. Перш ніж оновити об'єкт, транзакція повинна накласти на цей об'єкт X-блокування. Якщо транзакція вже заблокувала об'єкт S-блокуванням (для читання), то перед оновленням об'єкта S-блокування повинна бути замінена X-блокуванням.
3. Якщо блокування об'єкта транзакцією В відкидається тому, що об'єкт уже заблокований транзакцією А, то транзакція В переходить в стан очікування. Транзакція В буде перебувати в стані очікування до тих пір, поки транзакція А не зніме блокування об'єкта.
4. X-Блокування, накладені транзакцією А, зберігаються до кінця транзакції А.

14.4. Дозвіл тупикових ситуацій

При використанні протоколу доступу до даних з використанням блокувань частина проблем вирішилося (в повному обсязі), але виникла нова проблема – тупики:

1. Проблема втрати результатів поновлення – виник глухий кут.
2. Проблема незафіксованої залежності (читання «брудних» даних, неакуратне зчитування) – проблема вирішилася.
3. Є повторно зчитуваною – проблема вирішилася.
4. Поява фіктивних елементів – проблема не вирішилася.
5. Проблема несумісного аналізу – виник глухий кут.

Загальний вигляд тупика (dead locks) описується таблицею 14.7.

Таблиця 14.7. Проблема тупиків

Транзакція А	Час	Транзакція В
Блокування об'єкта P_1 – успішна	t_1	---
---	t_2	Блокування об'єкта P_2 – успішна
Блокування об'єкта P_2 – конфліктує з блокуванням, накладеною транзакцією В	t_3	---
Очікування	t_4	Блокування об'єкта P_1 – конфліктує з блокуванням, накладеною транзакцією А
Очікування	t_5	Очікування
Очікування		Очікування

Як видно, ситуація безвиході може виникати при наявності не менше двох транзакцій, кожна з яких виконує не менше двох операцій. Насправді в тупиковій ситуації може брати участь багато транзакцій, які очікують одна одну.

Так як нормального виходу з тупикової ситуації немає, то таку ситуацію необхідно розпізнавати і усувати. Методом вирішення тупикової ситуації є відкат однією з транзакцій (транзакції-жертви) так, щоб інші транзакції продовжили свою роботу. Після вирішення тупикової ситуації, транзакцію, обрану в якості жертви можна повторити заново.

Можна уявити два принципових підходи до виявлення тупикової ситуації і вибору транзакції-жертви:

1. СКБД не стежить за виникненням тупиків. Транзакції самі приймають рішення, чи бути їм жертвою.
2. За виникненням тупикової ситуації стежить сама СКБД, вона ж приймає рішення, якою транзакцією пожертвувати.

Перший підхід характерний для так званих настільних СКБД (FoxPro і т.п.). Цей метод є більш простим і не вимагає додаткових ресурсів системи. Для транзакцій задається час очікування (або число спроб), протягом якого транзакція намагається встановити потрібне блокування. Якщо за вказаний час (або після вказаного числа спроб) блокування не завершується успішно, то транзакція відкочується (або генерується помилкова ситуація). За простоту цього методу доводиться платити тим, що транзакції-жертви обираються, взагалі кажучи, випадковим чином. В результаті через одну просту транзакцію може відкотитися дуже дорога транзакція, на виконання якої вже витрачено багато часу і ресурсів системи.

Другий спосіб характерний для промислових СКБД (ORACLE, MS SQL Server і т.п.). В цьому випадку система сама стежить за виникненням ситуації глухого кута, шляхом побудови (або постійної підтримки) графа очікування транзакцій.

Граф очікування транзакцій – це орієнтований двочастковий граф, в якому існує два типи вершин – вершини, які відповідають транзакціям, і вершини, що відповідають об'єктам захоплення. Ситуація глухого кута виникає, якщо в графі очікування транзакцій є хоча б один цикл. Одну з транзакцій, що потрапила в цикл, необхідно відкинути, причому, система сама може вибрати цю транзакцію у відповідності з деякими вартісними міркуваннями (наприклад, найкоротшу, або з мінімальним пріоритетом і т.п.).

14.5. Метод тимчасових міток

Альтернативний метод серіалізації транзакцій, добре працює в умовах рідкісних конфліктів транзакцій і не вимагає побудови графа очікування транзакцій заснований на використанні тимчасових міток.

Основна ідея методу полягає в наступному: якщо транзакція А почалася раніше транзакції В, то система забезпечує такий режим виконання, як якщо б А була цілком виконана до початку В.

Для цього кожній транзакції Т пропонується тимчасова мітка t , відповідна часу початку Т. При виконанні операції над об'єктом R бази даних транзакція Т позначає його своєю тимчасовою міткою і типом операції (читання або зміна).

Перед виконанням операції над об'єктом R транзакція В виконує наступні дії:

1. Перевіряє, чи не закінчилася транзакція А, помітити цей об'єкт. Якщо А закінчилася, В позначає об'єкт R своєю тимчасовою міткою і виконує операцію.
2. Якщо транзакція А не завершена, то В перевіряє конфліктність операцій. Якщо операції неконфліктні, при об'єкті R залишається або проставляється тимчасова мітка з меншим значенням (більш рання), і транзакція В виконує свою операцію.
3. Якщо операції В і А конфліктують, то якщо $t(A) > t(B)$ (тобто транзакція А є «молодшою», ніж В), то транзакція А відкочується і, одержавши нову тимчасову мітку, починається заново. Транзакція В продовжує роботу.
4. Якщо ж $t(A) < t(B)$ (А «старша» В), то транзакція В відкочується і, одержавши нову тимчасову мітку, починається заново. Транзакція А продовжує роботу.

В результаті система забезпечує таку роботу, при якій при виникненні конфліктів завжди відкочується молодша транзакція (що почалася пізніше).

Очевидним недоліком методу часових міток є те, що може відкотитися дорожча транзакція, що почалася пізніше дешевшої.

До інших недоліків методу часових міток відносяться потенційно частіші відкати транзакцій, ніж в разі використання блокувань. Це пов'язано з тим, що конфліктність транзакцій визначається більш грубо.

14.6. Реалізація ізолюваності транзакцій засобами SQL

Стандарт SQL не передбачає поняття блокувань для реалізації серіалізованої суміші транзакцій. Замість цього вводиться поняття рівнів ізоляції. Цей підхід забезпечує необхідні вимоги до ізолюваності транзакцій, залишаючи можливість виробникам різних СКБД реалізовувати ці вимоги своїми способами (зокрема, з використанням блокувань).

Стандарт SQL передбачає 4 рівня ізоляції:

1. **READ UNCOMMITTED** – рівень незавершеного зчитування.
2. **READ COMMITTED** – рівень завершеного зчитування.
3. **REPEATABLE READ** – рівень повторюваного зчитування.
4. **SERIALIZABLE** – рівень здатності до впорядкування.

Якщо всі транзакції виконуються на рівні здатності до впорядкування (прийнятому за замовчуванням), то виконання будь-якої множини паралельних транзакцій, що чергується, може бути впорядковане. Якщо деякі транзакції виконуються на більш низьких рівнях, то є безліч способів порушити здатність до впорядкування. У стандарті SQL виділені три особливих випадки порушення здатності до впорядкування, фактично саме ті, які були описані вище як проблеми паралелізму:

1. **Неакуратне зчитування** («Брудне» читання, незафіксована залежність).
2. **Повторюване зчитування** (Окремий випадок неспільного аналізу).
3. **Фантоми** (Фіктивні елементи – окремий випадок неспільного аналізу).

Втрата результатів поновлення стандартом SQL не допускається, тобто на найнижчому рівні ізолюваності транзакції повинні працювати так, щоб не допустити втрати результатів оновлення.

Різні рівні ізоляції визначаються по можливості або виключення цих особливих випадків порушення здатності до впорядкування. Ці визначення описуються таблицею 14.8.

Таблиця 14.8. Рівні ізоляції стандарту SQL

Рівень ізоляції	Неакуратне зчитування	Повторюване зчитування	Фантоми
READ UNCOMMITTED	Так	Так	Так
READ COMMITTED	немає	Так	Так
REPEATABLE READ	немає	немає	Так
SERIALIZABLE	немає	немає	немає

Висновки. Сучасні багатокористувацькі системи допускають одночасну роботу великої кількості користувачів. При цьому якщо не вживати спеціальних заходів, транзакції будуть заважати одна одній. Цей ефект відомий як проблеми паралелізму.

Є три основні проблеми паралелізму:

1. **Проблема втрати результатів поновлення.**
2. **Проблема незафіксованої залежності** (Читання «брудних» даних, неакуратне зчитування).
3. **Проблема несумісного аналізу.**

Графік запуску набору транзакцій називається послідовним, якщо транзакції виконуються строго по черзі. Якщо графік запуску набору транзакцій містить елементарні операції транзакцій, які чергуються, то такий графік називається чергованим.

Два графіка називаються еквівалентними, якщо при їх виконанні буде отримано один і той же результат, незалежно від початкового стану бази даних.

Графік запуску транзакції називається вірним (серіалізованим), якщо він еквівалентний якому-небудь послідовному графіку.

Рішення проблем паралелізму полягає в знаходженні такої стратегії запуску транзакцій, щоб забезпечити серіалізованість графіка запуску і не дуже зменшити ступінь паралельності.

Одним з методів забезпечення серіальності графіка запуску є протокол доступу до даних за допомогою блокувань. У найпростішому випадку розрізняють S-блокування (колективні) і X-блокування (монопольні). Протокол доступу до даних має вигляд:

1. Перш ніж прочитати об'єкт, транзакція повинна накласти на цей об'єкт S-блокування.
2. Перш ніж оновити об'єкт, транзакція повинна накласти на цей об'єкт X-блокування.
3. Якщо транзакція вже заблокувала об'єкт S-блокуванням (для читання), то перед оновленням об'єкта S-блокування повинна бути замінена X-блокуванням.
4. Якщо блокування об'єкта транзакцією В відкидається тому, що об'єкт уже заблокований транзакцією А, то транзакція В переходить в стан очікування. Транзакція В буде перебувати в стані очікування до тих пір, поки транзакція А не зніме блокування об'єкта.
5. X-блокування, накладені транзакцією А, зберігаються до кінця транзакції А.

Для руйнування тупиків система періодично або постійно підтримує граф очікування транзакцій. Наявність циклів в графі очікування свідчить про виникнення тупика. Для руйнування тупика одна з транзакцій (найдешевша з точки зору системи) вибирається в якості жертви і відкочується.

15. ЗАХИСТ І ВІДНОВЛЕННЯ ДАНИХ

15.1. Захист інформації в базах даних

Дані є цінним ресурсом, доступ до якого необхідно суворо контролювати і регламентувати, так само як і до будь-яких інших корпоративних ресурсів. В СКБД описані особливості середовища бази даних зокрема типові функції і служби сучасної СКБД. В склад цих функцій і служб входять і кошти авторизації користувачів – механізм, за допомогою якого СКБД гарантує, що доступ до бази даних можуть отримати тільки користувачі, яким було надано відповідне право. Іншими словами, будь-яка СКБД повинна гарантувати, що створена в її середовищі база даних буде надійно захищена. Термін захист відноситься до захисту баз даних від несанкціонованого доступу, як навмисного, так і випадкового. Однак тема захисту баз даних включає не тільки стандартні служби, що надаються цільовою СКБД, але і набагато ширше коло питань, що мають відношення до захищеності самих баз даних і всього їх оточення.

Захист даних – попередження несанкціонованого або випадкового доступу до даних, їх зміни або руйнування даних з боку користувачів; попередження змін або руйнування даних при збоях апаратних і програмних засобів і при помилках в роботі співробітників групи експлуатації.

Наслідками порушення захисту є матеріальні збитки, зниження продуктивності системи, руйнування БД. Головними причинами порушення захисту є несанкціонований доступ, некоректне використання ресурсів, перебої в роботі системи. В БД застосовуються засоби захисту наведені на рис. 15.1.

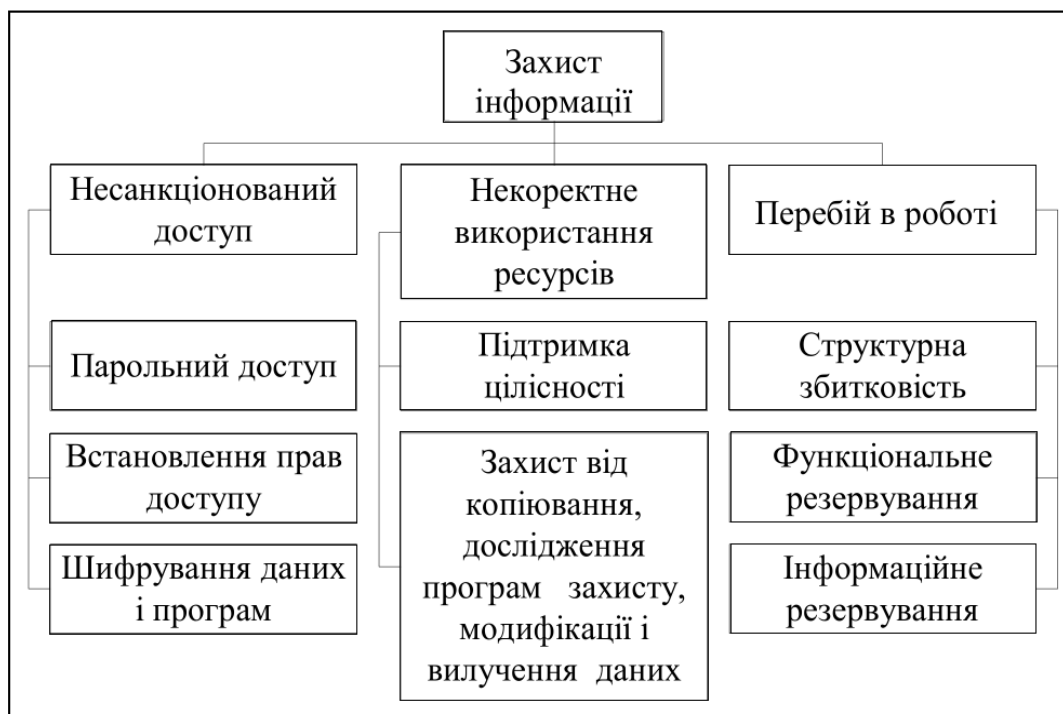


Рис. 15.1. Засоби захисту інформації в базах даних

Парольний захист передбачає встановлення пароля, який являє собою ряд літер, визначених користувачем при вході в систему з метою ідентифікації системними механізмами управління доступом. Після ідентифікації виконується аутентифікація.

Аутентифікація – процедура, яка встановлює автентичність ідентифікатора, який висувається користувачем, програмою, процесом, системою при вході в систему, при запиті деякої послуги або при доступі до ресурсу. Процес аутентифікації – це обмін між користувачем і системою інформацією, яка відома тільки системі і користувачу.

Управління доступом – захист інформаційних ресурсів в системі БД від несанкціонованого доступу. Мета полягає в забезпеченні представлення доступу до ресурсів

тільки користувачам, програмам, процесам, системам, які мають відповідні права доступу. Перевірка повноважень заснована на тому, що кожному користувачеві або процесу інформаційної системи відповідає набір дій, які він може виконувати по відношенню до певних об'єктів.

Як тільки користувач отримає право доступу до СКБД, йому автоматично представляються різні привілеї, пов'язані з його ідентифікатором. Ці привілеї можуть включати дозвіл на доступ до певних таблиць, представлень, індексів, а також на певні операції над цими об'єктами: перегляд (SELECT), додавання (INSERT), оновлення (UPDATE), вилучення (DELETE) (рис. 15.2).

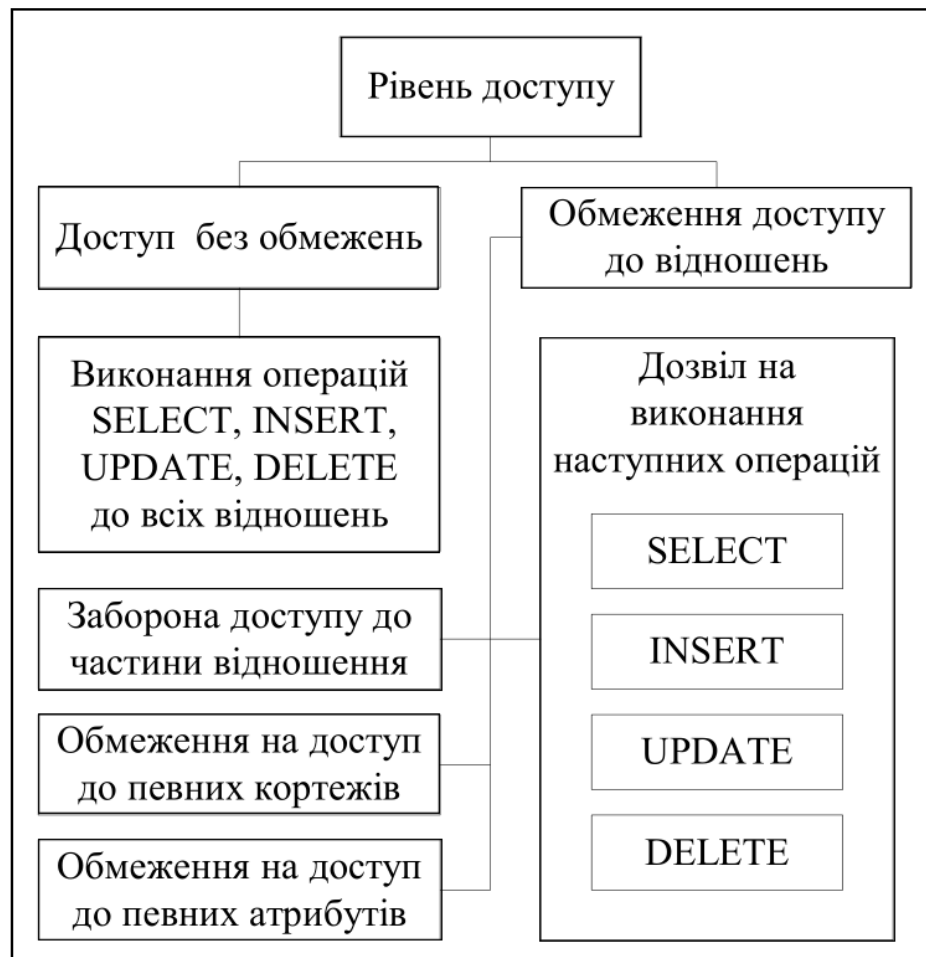


Рис. 15.2. Рівні доступу до бази даних

У сучасних СКБД підтримуються два найбільш загальних підходи до забезпечення безпеки даних: вибіркового підхід і обов'язковий підхід.

Вибірковий підхід передбачає, що кожен користувач має різні права при роботі з даними об'єктами. Різні користувачі можуть мати різні права доступу до одного й того ж об'єкта. Такий підхід це однорівнева модель безпеки (табл. 15.1).

Обов'язковий підхід передбачає, що кожному об'єкту даних надається деякий кваліфікаційний рівень, а кожен користувач має деякий рівень допуску. При такому підході права доступу до певного об'єкта даних мають тільки користувачі з відповідним рівнем доступу. Такий підхід являє собою багаторівневу модель безпеки (рис. 15.3). Приклад. Однорівнева модель безпеки.

Шифрування даних – метод забезпечення таємності даних шляхом генерації нового їх представлення, яке допускає однозначне відновлення вихідного представлення.

Система шифрування включає в себе ключ шифрування, алгоритм шифрування, ключ дешифрування і алгоритм дешифрування.

Захист від збоїв і відмов роботи апаратно-програмного забезпечення є однією з необхідних умов нормальної роботи системи. Головне навантаження по забезпеченню захисту системи від перебоїв і відказів припадає на апаратно-програмні компоненти.

Таблиця 15.1. Однорівнева модель безпеки

	Таблиця 1	Таблиця 2	...	Таблиця N
Користувач 1	Read,Update	-	...	Read, Write,Delete
Користувач 2	Read	Read, Write,Insert	...	Update,Insert
			...	
Користувач N	Read,Insert	Update	...	—

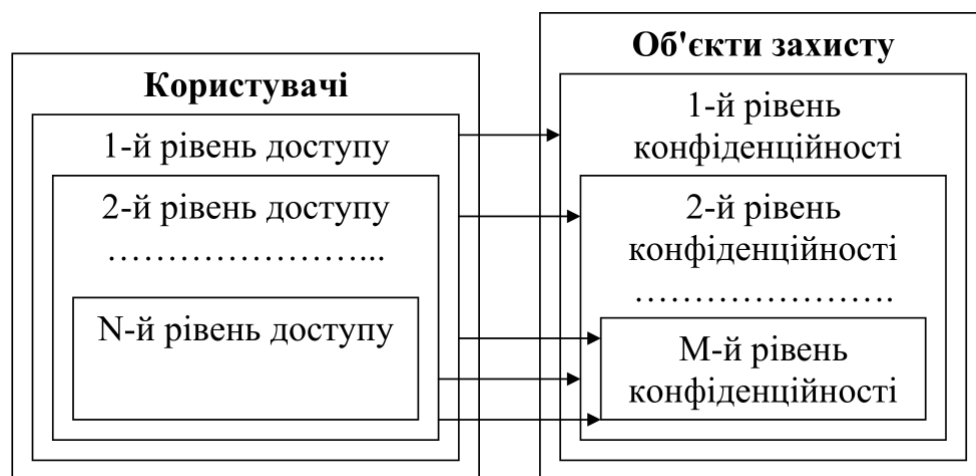


Рис. 15.3. Організація багаторівневої моделі безпеки

Структурна збитковість передбачає резервування апаратних компонентів на різних рівнях: дублювання серверів, процесорів, накопичувачів на магнітних дисках (RAID – масив недорогих дискових накопичувачів зі збитковістю), використання джерел безперебійного живлення.

Функціональне резервування означає і організацію обчислювального процесу, при якій функції обробки даних виконуються декількома елементами системи.

Інформаційне резервування реалізується шляхом періодичного копіювання і архівування даних.

Під **некоректним використанням ресурсів** розуміють некоректні зміни в БД (порушення посилкової цілісності, введення невірних даних тощо), випадковий доступ прикладних програм до чужих розділів основної пам'яті тощо. Вирішуються ці проблеми шляхом підтримки цілісності БД, використанням захисту від копіювання, дослідження програм, модифікації і видалення даних. Тут застосовуються як прикладні програми, так і засоби операційної системи.

Головна вимога довговічності даних полягає в тому, що дані зафіксованих транзакцій повинні зберігатися в системі, навіть якщо в наступний момент станеться збій системи. Здавалося б, найпростіший спосіб забезпечити таку гарантію – це під час кожної операції відразу записувати всі зміни на дискові носії. Такий спосіб не є задовільним, тому що є велика різниця в швидкості роботи з оперативною та із зовнішньою пам'яттю. Єдиний спосіб досягти прийнятної швидкості роботи полягає в буферизації сторінок бази даних в оперативній пам'яті. Це означає, що дані потрапляють у зовнішню довготривалу пам'ять не відразу після внесення змін, а через деякий (досить великий) час. Проте, що у зовнішній пам'яті повинно залишатися, тому що інакше нізвідки отримати інформацію для відновлення.

Вимога атомарності транзакцій стверджує, що не закінчені або відкочені транзакції не повинні залишати слідів в базі даних. Це означає, що дані повинні зберігатися в базі даних з надмірністю, що дозволяє мати інформацію, за якою відновлюється стан бази даних на момент початку невдалої транзакції. Таку надмірність зазвичай забезпечує журнал транзакцій. Журнал транзакцій містить деталі всіх операцій модифікації даних в базі даних, зокрема, старе і нове значення модифікованого об'єкта, системний номер транзакції, модифікований об'єкт і інша інформація.

15.2. Види відновлення даних

Відновлення бази даних може здійснюватися в наступних випадках:

1. **Індивідуальний відкат транзакції.** Відкат індивідуальної транзакції може бути ініційований або самою транзакцією шляхом подачі команди ROLLBACK, або системою. СКБД може ініціювати відкат транзакції в разі виникнення будь-якої помилки в роботі транзакції (наприклад, розподіл на нуль) або якщо ця транзакція обрана в якості жертви при вирішенні тупіка.
2. **М'який збій системи** (аварійна відмова програмного забезпечення). М'який збій характеризується втратою оперативної пам'яті системи. При цьому уражаються всі запущені в момент збою транзакції, втрачається вміст всіх буферів бази даних. Дані, що зберігаються на диску, залишаються неушкодженими. М'який збій може відбутися, наприклад, в результаті аварійного відключення електричного живлення або в результаті незворотного збою процесора.
3. **Жорсткий збій системи** (Аварійна відмова апаратури). Жорсткий збій характеризується пошкодженням зовнішніх носіїв пам'яті. Жорсткий збій може відбутися, наприклад, в результаті поломки головок дискових накопичувачів.

У всіх трьох випадках основою відновлення є надмірність даних, що забезпечується журналом транзакцій.

Як і сторінки бази даних, дані з журналу транзакцій записуються відразу на диск, а попередньо буферизуються в оперативній пам'яті. Таким чином, система підтримує два види буферів – буфери сторінок бази даних і буфери журналу транзакцій.

Сторінки бази даних, вміст яких в буфері (в оперативній пам'яті) відрізняється від вмісту на диску, називаються «брудними» (dirty) сторінками. Система постійно підтримує список «брудних» сторінок – dirty-список. Запис «брудних» сторінок з буфера на диск називається виштовхуванням сторінок в зовнішню пам'ять. Очевидно, необхідно передбачити такі правила виштовхування буферів бази даних і буферів журналу транзакцій, які забезпечували б два вимоги:

1. **Максимальну швидкість виконання транзакцій.** Для цього необхідно виштовхувати сторінки якомога рідше. В ідеалі, якщо оперативна пам'ять була б нескінченною, і збоїв ніколи б не відбувалися, найкращим виходом було б завантаження всієї бази даних в оперативну пам'ять, робота з даними тільки в оперативній пам'яті, і запис змінених сторінок на диск тільки в момент завершення роботи всієї системи.
2. **Гарантію, що при виникненні збою (будь-якого типу), дані завершених транзакцій можна було б відновити, а дані незавершених транзакцій безслідно видалити, тобто забезпечення відновлення останнього узгодженого стану бази даних.** Для цього щось виштовхувати на диск все-таки необхідно, навіть якщо ми володіли б нескінченною оперативною пам'яттю.

Таким чином, є дві причини для періодичного виштовхування сторінок в зовнішню пам'ять – недолік оперативної пам'яті і можливість збоїв.

Основним принципом узгодженої політики виштовхування буфера журналу і буферів сторінок бази даних є те, що запис про зміну об'єкта бази даних повинен потрапляти в зовнішню пам'ять журналу раніше, ніж змінений об'єкт виявиться у зовнішній пам'яті бази даних. Відповідний протокол журналізації (і управління буферизацією) називається Write Ahead Log (WAL) – «пиши спочатку в журнал», і полягає в тому, що якщо потрібно виштовхнути в зовнішню пам'ять змінений об'єкт бази даних, то перед цим потрібно гарантувати виштовхування у зовнішню пам'ять журналу запис про його зміну. Це означає, що якщо у зовнішній пам'яті бази даних міститься об'єкт, до якого застосована певна команда модифікації, то у зовнішній пам'яті журналу транзакцій міститься запис про цю операцію. Зворотне невірно – якщо у зовнішній пам'яті журналу міститься запис про деяку зміну об'єкта, то у зовнішній пам'яті бази даних може і не бути самого зміненого об'єкта.

Додаткова умова на виштовхування буферів накладається тією вимогою, що кожна успішно завершена транзакція повинна бути реально зафіксована у зовнішній пам'яті. Який би збій не стався, система повинна бути в змозі відновити стан бази даних, що містить результати всіх зафіксованих до моменту збою транзакцій.

Третьою умовою виштовхування буферів є обмеженість обсягів буферів бази даних і журналу транзакцій. Періодично або при настанні певної події (наприклад, кількість сторінок в dirty-списку перевищила певний поріг, або кількість вільних сторінок в буфері зменшилася і досягла критичного значення) система приймає так звану контрольну точку. Ухвалення контрольної точки включає виштовхування у зовнішню пам'ять вмісту буферів бази даних і спеціальну фізичну запис контрольної точки, яка представляє собою список всіх здійснюваних в даний момент транзакцій.

Виявляється, що мінімальною вимогою, що гарантує можливість відновлення останнього узгодженого стану бази даних, є виштовхування при фіксації транзакції в зовнішню пам'ять журналу всіх записів про зміну бази даних цієї транзакцією. При цьому останнім записом в журнал, виробленим від імені даної транзакції, є спеціальний запис про кінець цієї транзакції.

15.2.1. Індивідуальний відкат транзакції

Для того щоб можна було виконати за журналом транзакцій індивідуальний відкат транзакції, всі записи в журналі від даної транзакції зв'язуються в зворотний список. Початком списку для не закінчених транзакцій є запис про останню зміну бази даних, зроблену даною транзакцією. Для закінчених транзакцій (індивідуальні відкати яких вже неможливі) початком списку є запис про кінець транзакції, яка обов'язково виштовхнута в зовнішню пам'ять журналу. Кінцем списку завжди служить перший запис про зміну бази даних, зроблену даною транзакцією. У кожного запису є унікальний системний номер транзакції, щоб можна було відновити прямий список записів про зміни бази даних даною транзакцією.

Індивідуальний відкат транзакції виконується наступним чином:

1. Проглядається список записів, зроблених даною транзакцією у журналі транзакцій (від останньої зміни до першого зміни).
2. Обирається черговий запис зі списку даної транзакції.
3. Виконується протилежна за змістом операція: замість операції INSERT виконується відповідна операція DELETE, замість операції DELETE виконується INSERT, і замість прямої операції UPDATE зворотна операція UPDATE, відновлює попередній стан об'єкта бази даних.
4. Будь-яка з цих зворотних операцій також журналізуються. Це необхідно робити, тому що під час виконання індивідуального відкату може статися м'який збій, при відновленні після якого буде потрібно відкотити таку транзакцію, для якої не повністю виконаний індивідуальний відкат.
5. При успішному завершенні відкату в журнал заноситься запис про кінець транзакції.

15.2.2. Відновлення після м'якого збою

Незважаючи на протокол WAL, після м'якого збою не всі фізичні сторінки бази даних містять змінені дані, тому що не всі «брудні» сторінки бази даних були виштовхнуті в зовнішню пам'ять.

Останній момент, коли гарантовано були виштовхнуті «брудні» сторінки – це момент прийняття останньої контрольної точки. Є 5 варіантів стану транзакцій по відношенню до моменту останньої контрольної точки і до моменту збою (рис. 15.4).

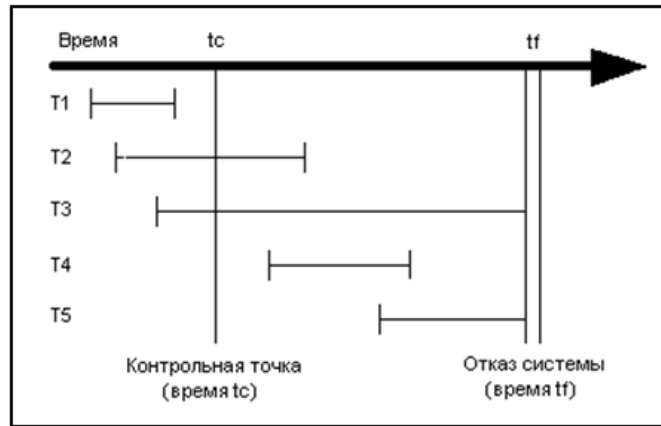


Рис. 15.4. П'ять варіантів транзакцій

Остання контрольна точка приймалася в момент t_c . М'який збій системи стався в момент t_f . Транзакції T1–T5 характеризуються такими властивостями:

1. **T1** – транзакція успішно завершена до прийняття контрольної точки. Всі дані цієї транзакції збережені в довготривалій пам'яті – як записи журналу, так і сторінки даних, змінені цією транзакцією. Для транзакції T1 ніяких операцій по відновленню не потрібно.
2. **T2** – транзакцію розпочато до прийняття контрольної точки і успішно завершено після контрольної точки, але до настання збою. Записи журналу транзакцій, які стосуються цієї транзакції виштовхнуті в зовнішню пам'ять. Сторінки даних, змінені цією транзакцією, тільки частково виштовхнуті в зовнішню пам'ять. Для даної транзакції необхідно повторити заново ті операції, які були виконані після прийняття контрольної точки.
3. **T3** – транзакцію розпочато до прийняття контрольної точки і не завершено в результаті збою. Таку транзакцію необхідно відкинути. Проблема, однак, у тому, що частина сторінок даних, змінених цією транзакцією, вже міститься у зовнішній пам'яті – це ті сторінки, які були оновлені до прийняття контрольної точки. Слідів змін, внесених після контрольної точки в базі даних, немає. Записи журналу транзакцій, зроблені до прийняття контрольної точки, виштовхнуті в зовнішню пам'ять, ті записи журналу, які були зроблені після контрольної точки, відсутні у зовнішній пам'яті журналу.
4. **T4** – транзакцію розпочато після прийняття контрольної точки і успішно завершено до збою системи. Записи журналу транзакцій, які стосуються цієї транзакції виштовхнуті в зовнішню пам'ять журналу. Зміни в базі даних, внесені цією транзакцією, повністю відсутні у зовнішній пам'яті бази даних. Таку транзакцію необхідно повторити цілком.
5. **T5** – транзакцію розпочато після прийняття контрольної точки і не завершено в результаті збою. Ніяких слідів цієї транзакції немає ні у зовнішній пам'яті журналу транзакцій, ні у зовнішній пам'яті бази даних. Для такої транзакції ніяких дій робити не потрібно, її як би і не було зовсім.

Відновлення системи після м'якого збою здійснюється як частина процедури перезавантаження системи. Після перезавантаження системи транзакції T2 і T4 необхідно частково або повністю повторити, транзакцію T3 – частково відкотити, для транзакцій T1 і T5 ніяких дій робити не потрібно. При перезавантаженні система виконує наступні дії:

1. Створюється два списки транзакцій UNDO (скасувати) і REDO (повторити). У список UNDO заносяться всі транзакції з останнього запису контрольної точки (тобто всі транзакції, що виконувалися в момент прийняття контрольної точки). Список REDO залишається порожнім. У нашому випадку буде: $UNDO = \{T2, T3\}$, $REDO = \{\}$.
2. Починаючи з запису контрольної точки проглядається наперед журнал транзакцій.
3. Якщо в журналі транзакцій виявляється запис про початок транзакції, то цю транзакцію буде додано до списку UNDO. У нашому випадку буде: $UNDO = \{T2, T3, T4\}$, $REDO = \{\}$. Зауважимо, що слідів транзакції T5 в журналі транзакцій немає.

4. Якщо у файлі реєстрації виявляється запис COMMIT про закінчення транзакції, то цю транзакцію буде додано до списку REDO. У нашому випадку буде: $UNDO = \{T2, T3, T4\}$, $REDO = \{T2, T4\}$. Зауважимо, що запис про кінець цих транзакцій є у зовнішній пам'яті журналу транзакцій відповідно до мінімальної вимоги виштовхування записів журналу при фіксації транзакції.
5. Коли досягається кінець журналу транзакцій, обидва списки аналізуються. При цьому зі списку UNDO видаляються ті транзакції, які потрапили в список REDO. У нашому випадку буде: $UNDO = \{T3\}$, $REDO = \{T2, T4\}$.
6. Після цього система переглядає журнал транзакцій тому, починаючи з моменту контрольної точки відкочуються всі транзакції зі списку UNDO. У нашому випадку будуть відкочуватися ті операції транзакції T3, які були виконані до прийняття контрольної точки.
7. Остаточнo, система переглядає журнал транзакцій наперед, починаючи з моменту контрольної точки, і повторно виконує всі операції транзакцій зі списку REDO. У нашому випадку, система виконає повторно всі операції транзакції T4 і ті операції транзакції T2, які були виконані після прийняття контрольної точки.

15.2.3. Відновлення після жорсткого збою

При жорстких збоях база даних на диску порушується фізично. Основою відновлення в цьому випадку є журнал транзакцій і архівна копія бази даних. Архівна копія бази даних повинна створюватися періодично, а саме з урахуванням швидкості наповнення журналу транзакцій.

Відновлення починається з зворотного копіювання бази даних з архівної копії. Потім прокрутити журнал транзакцій для виявлення всіх транзакцій, які закінчилися успішно до настання збою. Після цього по журналу транзакцій в прямому напрямку повторюються всі успішно закінчені транзакції. При цьому немає необхідності відкату транзакцій, перерваних в результаті збою, тому що зміни, внесені цими транзакціями, відсутні після відновлення бази даних з резервної копії.

Найгіршим випадком є ситуація, коли зруйновані фізично і база даних, і журнал транзакцій. У цьому випадку єдине, що можна зробити – це відновити стан бази даних на момент останнього резервного копіювання. Для того щоб не допустити виникнення такої ситуації, базу даних і журнал транзакцій зазвичай розташовують на фізично різних дисках, керованих фізично різними контролерами.

Стандарт мови SQL не містить вимог до відновлення даних, залишаючи ці питання на розсуд розробників СКБД.

15.3. Шифрування

Шифрування – перетворення даних з використанням спеціального алгоритму, в результаті чого дані стають недоступними для читання будь-якою програмою, яка не має ключа дешифрування.

Якщо в системі з базою даних міститься дуже важлива конфіденційна інформація, то має сенс зашифрувати її з метою попередження можливої загрози несанкціонованого доступу з зовнішнього боку (по відношенню до СКБД). Деякі СКБД включають засоби шифрування, призначені для використання в подібних цілях. Підпрограми таких СКБД забезпечують санкціонований доступ до даних (після їх дешифрування), хоча це буде пов'язано з деяким зниженням продуктивності, викликаним необхідністю подвійного перетворення.

Шифрування також може використовуватися для захисту даних при їх передачі по лініях зв'язку. Існує безліч різних технологій шифрування даних з метою приховування інформації, що передається, причому одні з них називають необоротними, а інші – оборотними. Необоротні методи, як і випливає з їх назви, не дозволяють встановити вихідні дані, хоча останні можуть використовуватися для збору достовірної статистичної інформації. Оборотні технології використовуються частіше. Для організації захищеної передачі даних по незахищених мережах повинні використовуватися системи шифрування, що включають такі компоненти:

- ключ шифрування, призначений для шифрування вихідних даних (звичайного тексту);
- алгоритм шифрування, який описує, як за допомогою ключа шифрування перетворити звичайний текст в зашифрований;
- ключ дешифрування, призначений для дешифрування зашифрованого тексту;
- алгоритм дешифрування, який описує, як за допомогою ключа дешифрування перетворити зашифрований текст в звичайний вихідний.

Деякі системи шифрування, що називаються симетричними, використовують один і той же ключ як для шифрування, так і для дешифрування, при цьому передбачається наявність захищених ліній зв'язку, призначених для обміну ключами. Однак більшість користувачів не мають доступу до захищених ліній зв'язку, тому для отримання надійного захисту довжина ключа повинна бути не менше довжини самого повідомлення. Проте більшість систем побудовано на використанні ключів, які коротше самих повідомлень.

Одна з поширених систем шифрування називається DES (Data Encryption Standard) [15]. У ній використовується стандартний алгоритм шифрування, розроблений фірмою IBM. У цій схемі для шифрування і дешифрування застосовується один і той же ключ, який повинен зберігатися в секреті, хоча сам алгоритм шифрування не є секретним. Цей алгоритм передбачає перетворення кожного 64-бітового блоку звичайного тексту з використанням 56-бітного ключа шифрування.

Система шифрування DES розцінюється як досить надійна далеко не всіма – деякі розробники вважають, що слід було б використовувати більш довге значення ключа. Так, в системі шифрування PGP (Pretty Good Privacy) [15] використовується 128-бітовий симетричний алгоритм, який застосовується для шифрування блоків переданих даних.

В даний час ключі довжиною до 64 біт розкриваються з достатнім ступенем вірогідності урядовими службами розвинених країн, для чого використовується спеціальне дороге обладнання. Проте, протягом найближчих років ця технологія може потрапити в руки організованої злочинності, великих організацій і урядів інших держав. Хоча, передбачається, що ключі довжиною до 80 біт також виявляться такими, що розкриваються в найближчому майбутньому, є впевненість, що ключі довжиною 128 біт в найближчому майбутньому залишаться надійним засобом шифрування. Терміни «сильне шифрування» і «слабке шифрування» іноді використовуються для підкреслення різниці у способах, які не можуть бути розкриті за допомогою існуючих в даний час технологій і теоретичних методів (сильні), і тими, які допускають подібне розкриття (слабкі).

Інший тип систем шифрування передбачає використання різних ключів для шифрування та дешифрування повідомлень – подібні системи прийнято називати асиметричними. Прикладом такої системи є система з відкритим ключем, що передбачає використання двох ключів, один з яких є відкритим, а інший зберігається в секреті.

Алгоритм шифрування також може бути відкритим, тому будь-який користувач, який бажає направити власнику ключів зашифроване повідомлення, може використовувати його відкритий ключ і відповідний алгоритм шифрування. Однак дешифрувати дане повідомлення зможе тільки той, хто має парний закритий ключ шифрування. Системи шифрування з відкритим ключем можуть також використовуватися для відправки разом з повідомленням «цифровий підпис», який підтверджує, що дане повідомлення було дійсно відправлено власником відкритого ключа. Найбільш популярною асиметричною системою шифрування є RSA (це ініціали трьох розробників даного алгоритму).

Як правило, симетричні алгоритми є більш швидкодіючими, ніж асиметричні, однак на практиці обидві схеми часто застосовуються спільно, коли алгоритм з відкритим ключем використовується для шифрування випадковим чином згенерованого ключа шифрування, а випадковий ключ служить для шифрування самого повідомлення із застосуванням деякого симетричного алгоритму.

16. АДМІНІСТРУВАННЯ БАЗ ДАНИХ

З базою даних, як правило, взаємодіють кілька користувачів. Ці користувачі в організації можуть виконувати абсолютно різні функції, мати різні уявлення про дані, які використовуються, але користуватися ними одночасно. Тому при експлуатації БД вкрай необхідний облік різних вимог і наявність алгоритму розв'язання конфліктів.

Іншими словами, потрібно ввести довгострокову функцію адміністрування, спрямовану на координацію і виконання всіх етапів проектування, реалізації та ведення інтегрованої бази даних. Відповідно до цієї функції на певних осіб покладається відповідальність за збереження такого важливого ресурсу, як дані.

У цій лекції ми познайомимося з завданнями і функціями адміністрування БД, яке необхідно забезпечити на самих ранніх стадіях розробки.

16.1. Цілі адміністрування та його актуальність для сучасних БД

Адміністрування баз даних передбачає виконання функцій, спрямованих на забезпечення надійного та ефективного функціонування системи баз даних, адекватності змісту бази даних інформаційним потребам користувачів, відображення в базі даних актуального стану предметної області.

Необхідність персоналу, який забезпечує адміністрування даними в системі БД в процесі функціонування, є наслідком централізованого характеру управління даними в таких системах, постійно вимагає пошуку компромісу між суперечливими вимогами до системи в соціальному користувацькому середовищі. Хоча така необхідність і визнавалася на ранніх стадіях розвитку технології баз даних, чітке розуміння і структуризація функцій персоналу, зайнятого адмініструванням, склалося тільки разом з визнанням багаторівневої архітектури СКБД в 1975р.

Адміністратор бази даних (АБД) – це фахівець, який відповідає за розробку вимог до різних баз даних організації. Він займається проектуванням, ефективним використанням, підтриманням цілісності і супроводом роботи сховища. Адміністратор управляє записами облікового типу і організовує систему захисту від несанкціонованого використання даних, що знаходяться в базі даних.

Адміністратор БД повинен вміти визначати вузькі місця системи, що обмежують її продуктивність, налаштовувати SQL і програмне забезпечення СКБД і володіти знаннями, необхідними для вирішення питань оптимізації швидкодії БД.

16.2. Процедура адміністрування

Адміністрування баз даних передбачає обслуговування користувачів бази даних. Можна провести аналогію між адміністратором баз даних і ревізором підприємства. Ревізор захищає ресурси підприємства, ресурсами – грошима, а адміністратор – ресурсами, які називаються даними. Не можна розглядати адміністратора баз даних тільки як кваліфікованого технічного спеціаліста, так як це не відповідає цілям адміністрування. Рівень адміністратора баз даних в ієрархії організації досить високий, щоб визначати структуру даних і право доступу до них. Адміністратор повинен знати, як працює підприємство і як використовуються відповідні дані; важливим є не тільки технічна компетентність, а й розуміння предметної області, а також уміння спілкуватися з людьми.

Адміністратор бази даних – не «володар» бази даних, а її «хранитель». З ускладненням предметної області неминуче ускладнюється процес формування інформації і прийняття рішень. В результаті розширюється спектр функцій адміністрування. Так, як у випадку використання бази даних прикладний програміст «усувається» від безпосереднього управління даними, то він втрачає з ними і контакт, а отже, і почуття відповідальності за них. Це вимагає розробки процедур забезпечення несуперечності даних, які повинні бути скоординовані з функцією адміністрування бази даних.

Адміністратор бази даних повинен координувати дії по збору відомостей, проектування і експлуатації бази даних, а також щодо забезпечення захисту даних. Він зобов'язаний враховувати поточні та перспективні інформаційні вимоги предметної області, що є однією з головних задач.

Правильна реалізація функцій адміністрування бази даних істотно покращує контроль і управління ресурсами даних предметної області. З цієї точки зору функції АБД є більше керуючими, ніж технічними. Принципи роботи АБД і його функції визначаються підходом до даних як до ресурсів організації, тому вирішення проблем, пов'язаних з адмініструванням починається з встановлення загальних принципів експлуатації СКБД.

Рівень АБД в ієрархії організації повинен бути досить високим, щоб він міг визначати структуру даних і право доступу до них і нести за це відповідальність. АБД зобов'язаний добре уявляти собі, як працює підприємство і як воно використовує дані. Хоча від АБД і отримати технічну компетентність, не менш важливим є розуміння ним предметної області, а також уміння спілкуватися з людьми і підкоряти альтернативи стандартним процедурам. В іншому випадку АБД не зможе ефективно виконувати свої функції.

В цілому, адміністрування бази даних передбачає виконання функцій адміністратора БД та інших адміністративних функцій, які забезпечують життєдіяльність системи бази даних (рис. 16.1).



Рис. 16.1. Завдання адміністрування бази даних

Завдання адміністратора. Потoki інформації, що передається грають важливу роль в сучасному світі. Всі дані систематизуються в певні групи – бази. Адміністратор – це особа, яка забезпечує кваліфіковане управління цими базами, включаючи їх всебічний захист. Через зв'язку будь-яких процесів, що проходять в організаціях ця професія дуже затребувана на ринку.

Основними завданнями адміністратора база даних є забезпечення безперебійної роботи всього обладнання, що знаходиться в організації (мереж, серверів та електроніки). Діяльність фахівця включає виконання певних алгоритмів для переробки та розподілу всього обсягу інформації на підприємстві (її обслуговування і диспетчеризацію), що дозволить безперервно отримувати і користуватися при необхідності потрібними відомостями. Адміністратор бази даних це фахівець, який більшу частину робочого часу займається саме обслуговуванням готової системи відомостей. Але в деяких випадках перед ним ставляться інші завдання в рамках робочого процесу:

- розробка необхідних вимог;
- нормування продуктивності сховища даних;
- формулювання прав для доступу і базових регламентів;
- копіювання баз даних в резервному режимі і їх відновлення;
- визначення формату облікових записів призначених для користувача;

- дослідження варіантів використання захисту баз даних від несанкціонованих вторгнень в систему;
- розробка варіантів запобігання помилок апаратного типу і збоїв програмного забезпечення з метою підтримки цілісності обсягу даних;
- забезпечення можливості швидкого переходу на оновлену версію системи керування базами даних.

Загальні обов'язки адміністратора. Посадова інструкція для адміністратора баз даних передбачає виконання великої кількості заходів, пов'язаних з системою відомостей в організації. Будь-яка інструкція включає кілька загальних пунктів, характерних для будь-яких різновидів управлінців в сфері інформації.

1. Проведення постійного копіювання баз інформації в резервному режимі. У разі постійного збереження даних при виникненні проблем з серверами або мережами всі дані з інформаційної бази можна легко відновити (або їх більшу частину).
2. Регулярна робота по оновленню програмного забезпечення. Інформаційні масиви часто обробляються не однією програмою, а цілим комплексом софту обслуговуючого характеру. Тому при постійному оновленні програмного забезпечення від адміністратора баз даних потрібна наявність знань про особливості різних програмних послуг, протоколів (мережевих), а також наявність навичок з програмування на різних комп'ютерних мовах. Крім того, кожен адміністратор повинен вміти самостійно написати утиліту, яка потрібна в його діяльності.
3. Адміністратор БД відповідає за забезпечення необхідного рівня продуктивності системи. Ці завдання вирішуються шляхом використання ефективних методів доступу, раціональної стратегії розміщення даних на носіях. Адміністратор БД вирішує завдання, пов'язані з вибором розміщення файлів на диску, визначенням необхідного обсягу дискової пам'яті, розподілом інформації на диску.

Групи специфічних обов'язків. Робота адміністратором передбачає виконання крім загальних обов'язків, однією з п'яти груп специфічних функцій:

- забезпечення безперебійного функціонування систем даних;
- оптимізація роботи інформаційних баз;
- запобігання ушкоджень втрат даних;
- постачання баз даних різними заходами безпеки;
- управління розширенням і розвиток інформаційних баз.

Робота щодо забезпечення функціонування баз даних включає наступні обов'язки.

1. Копіювання інформації з бази в резервному режимі.
2. Відновлення інформації з бази даних.
3. Управління варіантами доступу до інформаційних баз.
4. Установка, налаштування програмного забезпечення для керування базами даних.
5. Аналіз подій, які виникають при роботі баз даних.
6. Протоколювання і фіксація подій, які виникають в процесі обробки інформації в базах.

Оптимізація роботи інформаційних баз включає:

- аналіз роботи баз даних, збір інформації статистичного характеру про роботу інформаційних баз;
- оптимізацію перерозподілу обчислювальних даних, які взаємодіють з базами;
- нормування продуктивності інформаційних баз;
- оптимізацію елементів обчислювальних мереж, які взаємодіють з базами даних;
- оптимізацію здійснення запитів до інформаційних баз;
- оптимізацію контролю життєвого циклу, який зберігається в інформаційних системах.

Запобігання ушкодженям і втрат даних включає наступні обов'язки адміністратора:

1. Розробка положень про копіювання інформаційних баз в резервному режимі. Створення резервних копій (backup) виконується для запобігання можливого руйнування БД і, в разі необхідності, відновлення даних.
2. Контроль за виконанням положень про резервне копіювання.

3. Розробка планів щодо створення резервної копії інформаційних баз.
4. Розробка процедур створення інформаційних копій даних в резервному автоматичному режимі.
5. Здійснення процедур по відновленню даних після «обвалів» інформації.
6. Аналіз, що відбувається в системі збоїв, виявлення причин порушень.
7. Розробка інструкцій та методичних рекомендацій з обслуговування баз даних.
8. Дослідження функціонування програмно-апаратного супроводу баз даних.
9. Налаштування функціонування і працездатності інформаційних баз.
10. Розробка пропозицій щодо модернізації програмно-апаратних засобів.
11. Оцінка і аналіз ризиків виникнення збоїв в діяльності інформаційних баз.
12. Розробка способів автоматичного резервування інформаційних баз.
13. Розробка процедур щодо введення режимів гарячих заміन даних.
14. Складання звітів про роботу баз даних.
15. Проведення консультацій для користувачів при експлуатації інформаційних баз.
16. Контроль цілісності і відновлення бази даних. Проблема фізичної цілісності БД виникає в зв'язку з її можливим руйнуванням в результаті збоїв і відмов обладнання обчислювальної системи. Розвинені СКБД мають у своєму розпорядженні засоби відновлення зруйнованої БД, які засновані на використанні її контрольної копії і журналізації змін.

Забезпечення баз даних різними заходами безпеки включає:

- розробка стратегії інформаційної безпеки баз даних;
- контроль за дотриманням заходів безпеки інформації на базовому рівні;
- оптимізація функціонування системи в сфері безпеки на рівні баз даних;
- аудит інформаційної системи і захист баз даних від зовнішніх загроз;
- складання регламентів, що сприяють забезпеченню безпеки інформаційних систем даних;
- удосконалення роботи системи безпеки для зменшення навантажень на функціонування інформаційних систем;
- підготовка доповідей і звітів про ефективність роботи та стан систем безпеки в інформаційних носіях і сховищах.

Управління розширенням і розвиток інформаційних баз з даними включає наступні обов'язки адміністратора:

1. Аналіз проблем в системі з обробки інформації в базах даних і розробка пропозицій щодо розвитку перспектив у роботі баз даних.
2. Складання регламентів по оновленню програмного системного забезпечення в базах даних, інформаційних баз в нові варіанти програмного забезпечення і їх поєднання з новими платформами.
3. Вивчення та впровадження на практиці нових варіантів і способів роботи з інформаційними базами.
4. Відстеження оновлень варіантів інформаційних баз.
5. Відстеження впровадження сховищ інформації і їх сумісність з новими платформами і новими версіями програмного забезпечення.
6. Реорганізація (реструктуризація) БД. Логічна реструктуризація – модифікація концептуальної схеми з подальшим приведенням БД у відповідність з новосформованою схемою. Реорганізація БД дозволяє підвищити продуктивність системи БД і/або більш ефективно використовувати ресурси простору пам'яті середовища зберігання.
7. Підключення нових розробників і користувачів, приписування їм паролів, привілеїв доступу до конкретних даних.
8. Контроль зростання СКБД; визначення доцільності модернізації обладнання.
9. Консультування аналітиків і програмістів за особливостями використовуваної версії СКБД і інструментів розробки, участь (спільно з аналітиками з проектування бази даних) в логічному проектуванні в тому випадку, якщо необхідно враховувати специфічні для СКБД або режиму обробки даних рекомендації з проектування бази даних.

16.3. Резервування і відновлення БД

У ряді програм збереження і працездатність бази даних є надзвичайно критичним аспектом або в силу технологічних особливостей (системи реального часу), або в силу змістовного характеру даних. У цих випадках застосовуються підходи так званого гарячого резервування даних, коли база даних постійно знаходиться в вигляді двох ідентичних (дзеркальних) і паралельно функціонуючих копій, що розміщуються на двох роздільних системах дискової пам'яті.

В інших ситуаціях для забезпечення збереження даних використовуються операції архівування та резервування даних. У більшості випадків архівування здійснюється звичайними засобами архівації файлів для їх компактного довготривалого зберігання, як правило, на зовнішніх знімних носіях. Функції архівування даних іноді можуть входити і в перелік внутрішніх функцій самих СКБД.

Резервування даних, як правило, не передбачає спеціального стиснення даних, а виробляється через створення спеціальних копій файлів даних в технологічних або інших цілях.

І архівування, і резервування, крім технологічних цілей, переслідують також профілактичні цілі по ще одній надзвичайно важливій операції, що виконується адміністратором ІС – відновлення даних після збоїв і пошкоджень. Наявність резервної або архівної копії бази даних дозволяє відновити працездатність системи при виході з ладу основного файлу (файлів) даних. При цьому, однак, частина даних, або їх змін, які вироблені за час, що минув з моменту останнього архівування або резервування, можуть бути втрачені. Такі ситуації особливо критичні при колективній обробці загальних даних, реалізованих клієнт-серверними системами. Тому в промислових СКБД, що реалізують технології «Клієнт-сервер», в більшості випадків передбачається ведення спеціального журналу поточних змін бази даних, що розміщується окремо від основних даних і, як правило, на окремому носії.

Як вже зазначалося, такий підхід називається *журналізацією*. В журналі змін здійснюються безперервна фіксація і протоколювання всіх маніпуляцій користувачів з базою даних. В результаті при будь-якому збої за допомогою резервної копії та журналу змін адміністратор системи може повністю відновити дані до моменту збою за допомогою відкату-накату операцій транзакцій по журналу поточних змін бази даних.

16.4. Оптимізація роботи БД

У кожного виробника бази даних є цілий розділ в документації, який присвячений питанням продуктивності та оптимізації роботи бази даних. Треба розуміти, що типова установка бази даних зазвичай має на увазі мінімальне серверне обладнання, мінімальну пам'ять і диски. Тобто стандартні конфігурації не враховують можливості конкретного обладнання і конкретного додатка. Тому треба обов'язково налаштовувати базу даних для забезпечення оптимальної роботи.

Основні принципи оптимізації:

- якомога менше дискових операцій читання даних; треба збільшувати розмір буфера кешування, щоб база даних якомога менше читала дані з диска;
- якомога менше угруповань даних на диску; треба збільшити буфер сортування таким чином, щоб уникнути подвійних угруповань і угруповань на диску;
- збільшувати паралельність виконання запитів за рахунок запуску великого числа процесів бази даних, вибору форматів зберігання даних, що забезпечують паралельність роботи;
- відкладена фіксація транзакцій на диску; бажано налаштувати базу даних так, щоб при зміні даних або їх вставці не проводився негайний запис на диск, а зміни збиралися і фіксувалися з деяким інтервалом; це дозволяє значно швидше повертати керування продукту після виконання SQL запитів.

Важливим параметром, що впливає на споживання пам'яті базами даних, є максимальне число одночасних з'єднань. Тому треба прагнути зменшити число одночасних з'єднань і вивільнити більше пам'яті для сортування даних в пам'яті і їх буферизації.

16.5. Безпека даних

В сучасних СКБД підтримується один з двох підходів до питання забезпечення безпеки даних: виборчий або обов'язковий.

Виборчий підхід забезпечує користувачеві різні права (привілей або повноваження) при роботі з різними об'єктами бази даних. Більш того, різні користувачі зазвичай володіють різними правами доступу до одного і того ж об'єкту. Тому виборчі схеми характеризуються значною гнучкістю.

Обов'язковий підхід кожному об'єкту бази даних привласнює певний класифікаційний рівень, а кожному користувачеві забезпечується певний рівень допуску. При такому підході доступом до певного об'єкту володіють тільки користувачі з відповідним рівнем допуску. Обов'язкові схеми досить вузькі і статичні.

Всі рішення щодо вибору того чи іншого підходу до забезпечення прав доступу приймаються на стратегічному, а не на технічному рівні.

1. Результати стратегічних рішень повинні бути відомі системі (виконані на основі тверджень, заданих за допомогою необхідного мови) і зберігатися в ній (збереження їх в каталозі в вигляді правил безпеки – повноважень).
2. Наявність засобів регулювання запитів доступу по відношенню до відповідних правил безпеки (запит доступу – комбінація запитуваної операції, запитуваного об'єкта і запитуваного користувача). Така перевірка виконується підсистемою безпеки СКБД, яка також називається підсистемою повноважень.
3. В системі повинні бути передбачені способи впізнання джерела запиту, тобто, пізнання запитувача користувача. Як правило, це ідентифікатор користувача і пароль. В системі можуть підтримуватися групи користувачів і забезпечуватися однакові права доступу для користувачів однієї групи. Крім того, операції додавання окремих користувачів в групу або їх видалення з неї можуть виконуватися незалежно від операції завдання привілеїв для цієї групи (місцем зберігання інформації про приналежність до групи також є системний каталог або, можливо, БД).

Методи обов'язкового доступу застосовуються в організаціях, що мають досить статичну і жорстку структуру (наприклад, урядові організації або військові). Основна ідея – кожен об'єкт має певний рівень класифікації (наприклад, «таємно», «цілком таємно», «для службового користування» тощо), А кожен користувач має рівень допуску з такими ж градаціями, що і в рівні класифікації. Рівні повинні утворювати строгий ієрархічний порядок (що йде вище чого). Звідси випливають правила безпеки:

- користувач U1 має доступ до об'єкта O1, тільки якщо його рівень допуску більше або дорівнює рівню класифікації об'єкта O1;
- користувач U1 може модифікувати об'єкт O1, тільки якщо його рівень допуску дорівнює рівню класифікації об'єкта O1.

16.6. Підтримка заходів забезпечення безпеки в мові SQL

У мові SQL передбачається підтримка тільки виборчого управління доступом, яка заснована на двох більш-менш незалежних частинах. Механізм уявлень використовується для приховування дуже важливих даних від несанкціонованих користувачів. Підсистема повноважень наділяє користувачів виборчими правами по відношенню до об'єктів БД і діям з даними.

У SQL правила безпеки задаються на основі директиви GRANT (рис. 16.2).

При створенні таблиці за допомогою інструкції CREATE TABLE користувач автоматично отримує для неї все привілеї. Для доступу інших користувачів до таблиці їм слід надати привілеї за допомогою інструкції GRANT.

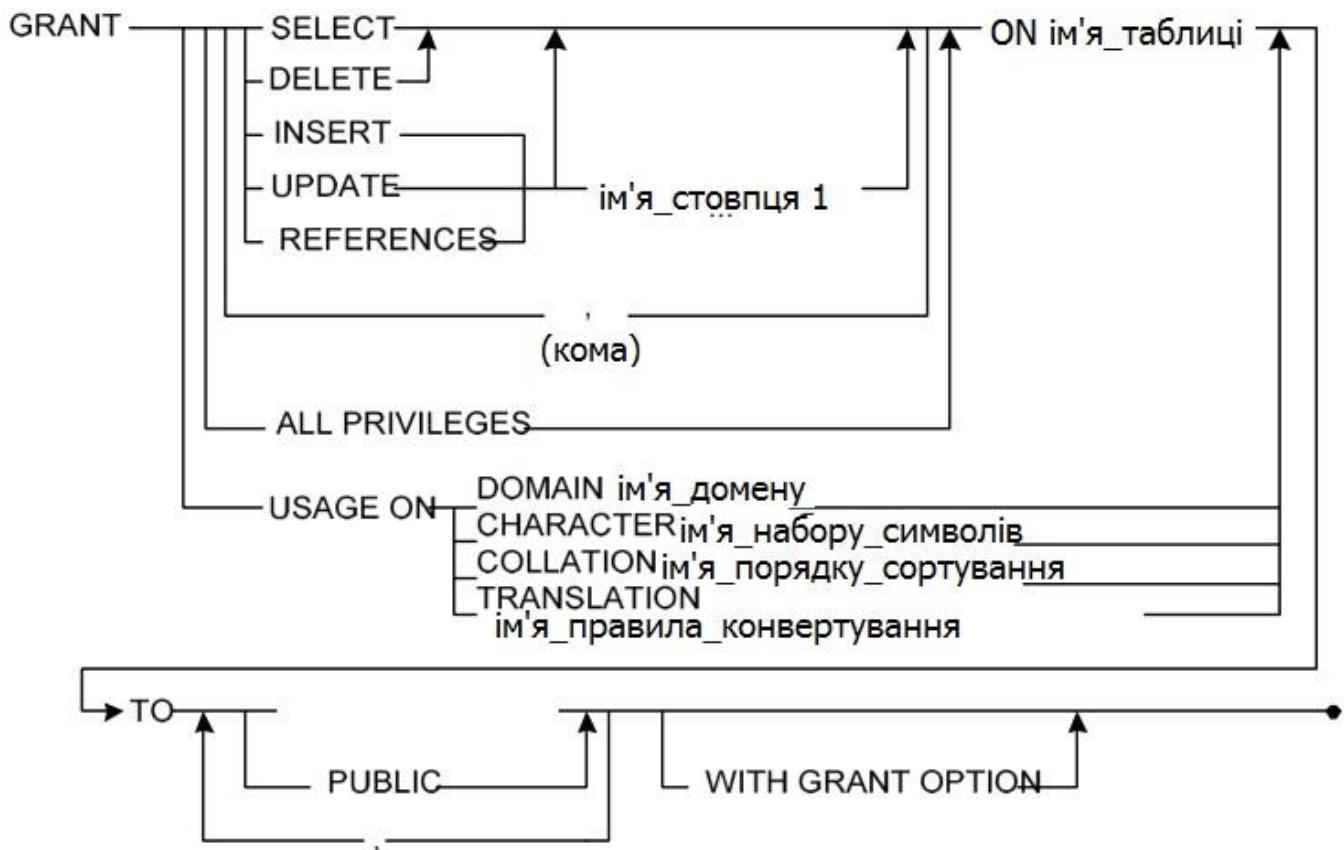


Рис. 16.2. Інструкція надання привілеїв (GRANT)

У спрощеному вигляді синтаксис твердження grant (надання повноважень) можна записати таким чином:

GRANT список_привілеїв_через_кому
ON об'єкт
TO список_користувачів_через_кому
[WITH GRANT OPTION];

Серед допустимих привілеїв, тобто, дій, які користувач має право виконувати над об'єктом БД, можуть бути: операції використання (USAGE), вибору (SELECT), вставки (INSERT), оновлення (UPDATE), видалення (DELETE) і посилання (REFERENCES). Привілей USAGE необхідний для вказівки використання деякого заданого домена, привілей REFERENCES – для звернення до обмеження цілісності до спеціально заданої таблиці. Привілеї INSERT і UPDATE можуть задаватися для спеціально заданих стовпців.

Як об'єкти можуть виступати, наприклад, таблиці та подання.

Список користувачів, розділених комами, може бути замінений ключовим словом public, яке означає всіх користувачів.

Якщо задана директива WITH GRANT OPTION, це означає, що зазначені користувачі наділені особливими повноваженнями для заданого об'єкта – правом надання повноважень. Якщо необхідно дати будь-які одні привілеї з правом надання, а інші – без, то необхідно скористатися двома окремими інструкціями GRANT – GRANT з директивою WITH GRANT OPTION і без.

Скасування привілеїв проводиться за допомогою інструкції REVOKE (рис. 16.3).

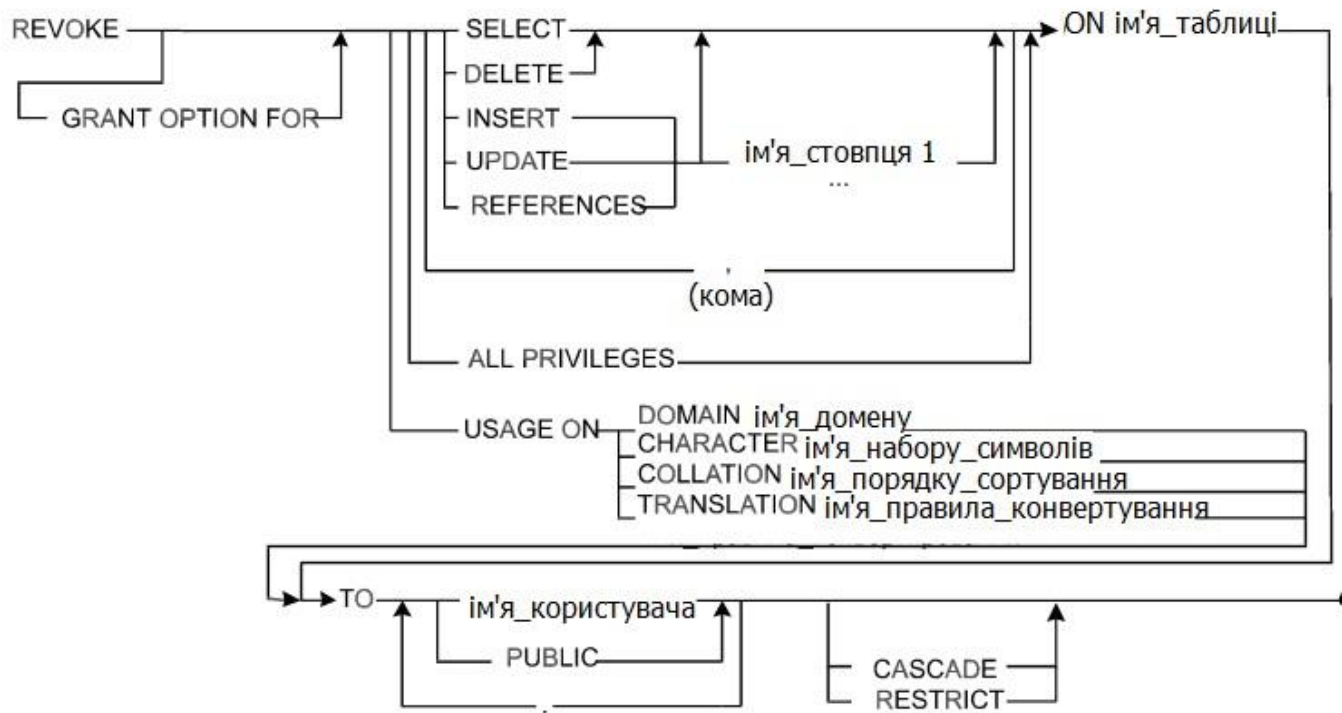


Рис. 16.3. Інструкція скасування привілеїв (REVOKE)

Скорочений вигляд інструкції REVOKE:

REVOKE [GRANT OPTION FOR] список_привілеїв_через_кому
ON об'єкт
FROM список_користувачів_через_кому параметр;

Директива GRANT OPTION FOR означає, що скасовуються тільки повноваження, які призначаються (передані) іншим користувачам; список привілеїв і список користувачів використовуються так само, як і в утвердженні GRANT, а в якості параметра використовується або RESTRICT, або CASCADE. RESTRICT і CASCADE запобігають ситуації, що з'являються з виникненням покинутих привілеїв. При завданні RESTRICT забороняється виконувати операцію скасування привілеїв, якщо вона призводить до появи покинутої привілеї. Параметр CASCADE вказує на послідовну скасування всіх привілеїв, похідних від запропонованої.

При видаленні домену, таблиці, стовпці або подання автоматично будуть видалені також і всі привілеї щодо цих об'єктів з боку всіх користувачів.

Приклад 1. Створити правило безпеки, що дозволяє вилучення та оновлення стовпців SURNAME, NAME, _YEAR (курс), _GROUP, FACULTY таблиці STUDENTS, якщо студенти навчаються з 1 по 4 курс на математичному, фізичному або економічному факультетах і проживають в містах Черкаси або Канів, для користувачів Deanery_1, Deanery_2.

Рішення.

```

CREATE VIEW STUDIES AS
  SELECT SURNAME, NAME, _YEAR, _GROUP, FACULTY
  FROM STUDENTS
  WHERE FACULTY IN ('математичний',
    'Фізичний', 'економічний')
  AND YEAR IN (1, 2, 3, 4)
  AND CITY = 'Черкаси' OR CITY = 'Канів';
GRANT SELECT, INSERT, DELETE ON STUDIES TO
  Deanery_1, Deanery_2;

```

Приклад 2. Створити правило безпеки для користувачів Deanery_3, Deanery_4, що дозволяє витяг, видалення і оновлення стовпців COURSE_NAME, TEACHER, _HOURS таблиці COURSE, якщо курси включені в 1, 2 або 4 семестри, і кількість їх годин знаходиться в межах від 30 до 90. Рішення.

```
CREATE VIEW STUDR AS
  SELECT SUBJECT_NAME, LECTURER, _HOURS
  FROM COURSE
  WHERE SEMESTER IN (1, 2, 4)
  AND _HOURS >= 30 AND _HOURS <= 90;
GRANT SELECT, INSERT, DELETE ON STUDR TO
Deanery_3, Deanery_4;
```

17. БАЗИ ЗНАНЬ

База знань (БЗ, англ. *Knowledge base, KB*) – це особливого роду база даних, розроблена для керування знаннями (**метаданими**), тобто збором, зберіганням, пошуком і видачою знань. Розділ штучного інтелекту, що вивчає бази даних і методи роботи із знаннями, називається **інженерією знань**.

Інше визначення ж говорить, що база знань – це сукупність відомостей (про реальні об'єкти, процес, події або явища), що відносяться до певної теми або задачі, яка організована так, щоб забезпечити зручне представлення цієї сукупності відомостей як в цілому, так і будь-якої її частини. Це означає, що система управління базою знань (саме знань, а не даних) повинна забезпечити представлення й обробку моделі, зіставною за своєю складністю з моделлю, що використовується свідомістю людини.

У штучному інтелекті (ШІ) основна мета – навчитися зберігати знання таким чином, щоб програми могли обробляти їх і досягти подібності людського інтелекту. Дослідники ШІ використовують теорії представлення знань з **когнітології** (це сфера діяльності, пов'язана з аналізом знання і забезпеченням його подальшого розвитку). Такі методи як фрейми, правила, і семантичні мережі прийшли в ШІ з теорій обробки інформації людиною. Так як знання використовується для досягнення розумної поведінки, фундаментальною метою дисципліни уявлення знань є пошук таких способів уявлення, які роблять можливим процес логічного висновку, тобто створення висновків з знань.

Система штучного інтелекту – це система, що оперує знаннями про проблемну область. Без бази знань систем штучного інтелекту не існує. Для формалізації і представлення знань розробляються спеціальні моделі представлення знань і мови для опису знань, виділяються різні типи знань. Моделі подання знань являють собою один з найважливіших напрямків досліджень в області штучного інтелекту.

На сьогоднішній день розроблено вже достатню кількість моделей. Кожна з них володіє своїми плюсами і мінусами, і тому для кожної конкретної задачі необхідно вибрати саме свою модель. Від цього залежатиме не стільки ефективність виконання поставленої задачі, скільки можливість розв'язання її взагалі.

17.1. Коли дані стають знаннями

При вивченні інтелектуальних систем традиційно виникає питання – що ж таке знання і чим вони відрізняються від звичайних даних, які десятиліттями обробляються на ЕОМ. Можна запропонувати кілька робочих визначень, в рамках яких це стає очевидним.

Дані – це окремі факти, що характеризують об'єкти, процеси і явища предметної області, а також їх властивості.

При обробці на ЕОМ дані трансформуються, умовно проходячи наступні етапи:

- дані як результат вимірювань і спостережень;
- дані на матеріальних носіях інформації (таблиці, протоколи, довідники);
- моделі (структури) даних у вигляді діаграм, графіків, функцій;
- дані в комп'ютері мовою опису даних;
- бази даних на машинних носіях інформації.

Знання засновані на даних, отриманих емпіричним шляхом. Вони являють собою результат розумової діяльності людини, спрямованої на узагальнення його досвіду, отриманого в результаті практичної діяльності.

Знання – це закономірності предметної області (принципи, зв'язки, закони), отримані в результаті практичної діяльності і професійного досвіду, що дозволяють фахівцям ставити і розв'язувати задачі в цій області.

При обробці на ЕОМ знання трансформуються аналогічно даним.

- знання в пам'яті людини як результат мислення;
- матеріальні носії знань (підручники, методичні посібники);

- поле знань – умовний опис основних об'єктів предметної області, їх атрибутів і закономірностей, що зв'язують їх;
- знання, описані на мовах подання знань (продукційні мови, семантичні мережі, фрейми – див. далі);
- база знань на машинних носіях інформації.

Часто використовується таке визначення знань.

Знання – це добре структуровані дані, або дані про дані, або метадані.

Метадані – дані про дані. Чим більше ми знаємо про дані, тим інформативнішими вони є. Розширивши відомості про дані, можна надати їм статусу знань.

Нехай у базі є дані «15.01.2018». Якщо дата – це єдина інформація, яку можна почерпнути з бази про ці дані, то вони є малоінформативними. Якщо ж із бази даних можна буде встановити, що це – дата здачі іспитів студентами КМ-16 і КС-16 груп 3 курсу кафедри програмного забезпечення автоматизованих систем факультету ОТІУС Черкаського національного університету, то ця дата стає набагато інформативнішою. У даному прикладі семантика дати розширюється завдяки встановленню зв'язків з іншими поняттями, такими як іспит, студент, група, кафедра, факультет, університет. Звичайна база даних дає змогу встановлювати зв'язки одних сутностей з іншими. Проте в базі не можуть зберігатися неповні, нечіткі, суперечливі дані та інформація про те, що дані є саме такими.

Активність даних. Будь-які зміни певних даних можуть спричинити зміни інших даних, які, у свою чергу, приводять до подальших змін і т. д. Наприклад, поява нового співробітника приведе до зміни кількості співробітників відповідного підрозділу, штатного розкладу і фонду заробітної платні, відбудеться перерозподіл обсягів робіт тощо. Зміна будь-яких даних обумовлюється настанням певної події, яка може не лише приводити до цієї зміни, але й ініціювати сукупність інших дій.

Дані у просторі та часі. Сутності будь-якої предметної області мають багато властивостей і найважливішими з них є ті, що пов'язані з існуванням сутностей у просторі та часі. Такі властивості мають фіксуватися в базі даних незалежно від того, чи вказується це проектувальником системи явно. Якщо йдеться про простір, то СКБД, що претендує на інтелектуальність, має «розуміти», що означає «поставити на», «посунути якомога ближче до», «пересунути в крайній лівий кут», «обернути на певний кут навколо певної осі» тощо. Якщо СКБД відображує часові характеристики, то в ній має фіксуватися час настання подій. Проблема полягає не в тому, щоб запрограмувати ті чи інші просторові операції або явним способом визначити часові атрибути сутностей (дата введення, дата видалення, дата заміни), а в тому, щоби відтворити семантику простору і часу в самій системі.

Ускладнення моделі даних. Реляційна модель відображує найпростіші структури даних і надає високорівневу мову маніпулювання даними. У реальному світі ми маємо справу з об'єктами довільної складності. Отже, в реляційній базі даних доводиться зображувати складні об'єкти у вигляді сукупностей простих, хоча це й не сприяє адекватному відображенню предметної області.

Розглянемо простий приклад. Реляційна модель дає змогу зобразити відношення R (Батько, Дитина), в якому для кожної особи вказується її дитина, причому єдина (відношення має перебувати в 1НФ). Якщо в людини кілька дітей, то ми повинні зображувати це дублюванням значень поля Батько, відтак і дублюванням екземплярів відповідної сутності.

Можна стверджувати, що таке відношення не адекватно відображує предметну область, і про це мають пам'ятати всі, хто працюють з базою даних. Природнішим було б зображувати предметну область у вигляді відношення R (Батько, Діти), коли з кожним батьком асоціюються всі його діти.

Іншими словами, для адекватнішого зображення семантики предметної області бажано відмовитися від нормалізованості відношень, зберігаючи при цьому високорівневу мову маніпулювання даними. Таке ускладнення структури даних насамперед має відображатися на концептуальному й зовнішньому рівнях. Внутрішній рівень СКБД може підтримуватися звичайними нормалізованими відношеннями за умови, що є формальні алгоритми опису відображень між рівнями.

Надання семантики зв'язкам між сутностями. Реляційна модель дає змогу зв'язувати сутності за допомогою зовнішніх ключів, але ці зв'язки можуть мати лише невелику кількість властивостей (наприклад, обов'язковість існування батьківської сутності для підлеглої). Проте багато дослідників зазначають, що реально існує понад 200 типів зв'язків із різною семантикою, які бажано відображувати в базі даних. Уже згадувалося про необхідність відображення семантики двох фундаментальних типів властивостей об'єктів реального світу – просторових і часових. Взагалі, що більше типів зв'язків з притаманними їм властивостями фіксується в базі даних, то більш інформативною є така база.

Виведення нових даних. У базі даних зберігаються лише дані або факти, що стосуються модельованої предметної області. Будь-які дані зв'язані з багатьма похідними даними, які можуть бути отримані шляхом відповідного виведення. Традиційні бази даних не дають змоги описувати правила виведення нових даних (фактів, понять) з наявних. Перші кроки в цьому напрямку були зроблені в дедуктивних базах даних. Проте в них використовується лише один метод виведення – дедуктивний, а людина в своїх міркуваннях застосовує й інші методи.

Дедуктивна база даних реалізується як звичайна (як правило, реляційна) база даних разом із множиною логічних правил, що вказують способи породження нових даних. Наприклад, нехай база даних містить відношення

РОДИЧ{Батько, Син} і такі правила:

Дід – це батько Батька. Онук – це син Сина

До такої бази даних, що містить правила породження понять «Дід» і «Онук», можна ставити запити типу: «*Хто дід Івана?*», «*Визначити (знайти) всіх онуків Петра*» тощо.

Використання ширшого набору методів виведення даних сприяє трансформації баз даних у бази знань.

17.2. Постулати систем баз даних

Традиційні системи керування базами даних висувують кілька вимог до того, якою має бути модель предметної області, що фіксується в базі. Ці вимоги можна охарактеризувати за допомогою чотирьох постулатів.

Постулат єдності. Цей постулат стверджує, що існує єдина предметна область, яка вивчається однією особою, в результаті чого формується єдина база даних. У цьому твердженні слово «єдиний» не можна сприймати категорично. Предметна область може бути поділена на сукупність предметних областей, що в певних ситуаціях можуть розглядатися незалежно. Проте, враховуючи мету вивчення предметної області, вона має розглядатися як єдине ціле. Тобто вивчається не сукупність предметних областей (взаємозв'язаних, але різних), а одна предметна область. Далі база даних проектується колективом, але це колектив однодумців в тому розумінні, що перед ними поставлено завдання сформулювати не множину різних уявлень про предметну область, а єдине адекватне уявлення. Цей постулат не допускає існування, відтак і фіксації, в базі даних множини уявлень про предметну область, сформованих у свідомості різних осіб, оскільки їхні погляди на предметну область, що певним чином доповнюють одне одного, можуть базуватися на різних поняттях і суперечити одне одному. Проте база даних має бути несуперечною.

Постулат про категоричність. Категоричність свідчить про те, що факти, які виводяться з бази даних, можуть бути лише істинними або хибними. Ніяка інша ситуація, окрім «знаю» або «не знаю», неможлива. Більше того, неможливі відтінки істинних значень типу «*цілком можливо*», «*в дев'яти випадках з десяти істинно*», «*ймовірно істинно*» тощо.

Постулат достовірності. Суть постулату полягає в тому, що знання, закладені в базу даних, відповідають дійсності. Даний постулат означає, що істинність або хибність визначень не викликає сумнівів. Усе, що істинне в базі даних, насправді має місце в предметній області, а все що хибне – не має й не може мати місця. Цей постулат відкидає ситуацію, коли знання в базі даних можуть не відповідати реаліям. Із нього також випливає, що самі знання про предметну область є несуперечними (жодний факт не може бути оцінений як істинний і як хибний водночас). Достовірність вимагає беззастережної віри в істинність фактів, що виводяться з бази даних, не

лише тому, що система баз даних не припускається помилок, але й тому, що всі закладені в неї знання вважаються істинними.

Постулат про всезнання. Цей постулат стверджує, що проектувальник бази даних вивчив предметну область у обсязі, достатньому для досягнення поставлених практичних цілей. Він знає все, що йому слід було б знати, і в нього немає жодних сумнівів у достатності своїх знань. Тому вважається, що спроектована база даних повністю відображує предметну область.

Звичайно, розглянуті постулати ідеалізують реальну ситуацію. Колектив, який вивчає предметну область, може й не прийти до спільних позицій. І тоді рішення мають бути компромісними, а компроміс позбавляє впевненості у правильності ухвалених рішень. У цьому випадку було б доцільно організувати базу даних так, щоб вона акумулювала думки всіх експертів і дозволяла вирішувати завдання у неоднозначно визначеному середовищі. Наші знання можуть бути нечіткими, неповними, суперечливими, але традиційні бази даних, як правило, відповідають розглянутим вище постулатам і тому цих властивостей знань не відображують.

17.3. Моделі зображення знань

Існує багато різних моделей, призначених для зображення знань. Класичними серед них можна вважати такі:

- формально-логічна модель;
- продукційна модель (модель, яка базується на правилах);
- семантичні мережі;
- фреймова модель.

Деякі автори включають у цей список об'єктну модель та модель сутностей і зв'язків.

Що стосується класифікації знань, то зазвичай їх поділяють на **декларативні** й **процедурні**. На ці категорії поділяються й моделі зображення знань. Підкласами декларативних моделей є **логічні** та **структурні** моделі.

Раніше в комп'ютерних системах використовувався процедурний спосіб зображення знань. Він передбачав зображення знань у вигляді алгоритмів, за якими укладаються програми. Такі програми керують і обробляють дані, що зберігаються у файлових системах або базах даних. Для зміни чи поповнення «розчинених» у програмі знань, програма має бути переписана.

Під декларативними знаннями розуміють знання типу «*A це B*», зберігання яких є характерним для баз даних. Це факти типу «*у годину пік на вулиці багато машин*», «*запалена плита гаряча*», «*жорсткий диск – середовище для збереження інформації*».

Розглянемо детальніше різновиди моделей зображення знань.

17.3.1. Формально-логічна модель

Логіка предикатів першого порядку стала теоретичним базисом науки про штучний інтелект на початку її становлення. Дослідники в галузі баз знань віддають перевагу математичній логіці у зв'язку з тим, що вона має повну математичну формалізацію, а також тому, що вона дозволяє в межах однієї й тієї ж моделі подавати знання і способи мислення.

Згідно з логічною моделлю база знань розглядається як теорія, що містить багато фактів довільної складності, які є даними бази фактів та аксіомами бази знань. У цій теорії діють механізми виведення, що дають змогу одержувати нові знання з наявних. У класичній логіці використовується дедуктивний механізм виведення (він дає можливість на підставі фактів A і $A \supset B$ зробити висновок про існування факту B). Можуть застосовуватися й інші механізми виведення (абдуктивний, традиктивний та індуктивний), але це вже не буде класичною логікою.

Наприклад, якщо задано факти A , $A \supset B$, $B \supset C$, то за допомогою дедуктивного виведення можна отримати такі нові факти: B , C , $A \vee B$, $A \vee C$, $B \vee C$ тощо.

Обмеження. Застосування логіки для моделювання знань пов'язане з певними обмеженнями:

- відсутністю механізмів додавання й видалення аксіом;
- відсутністю механізмів «відстежування» переходу від однієї теорії до іншої;

- відсутністю принципів керування;
- негнучкістю у тому розумінні, що мова моделювання знань є повністю декларативною. Є альтернативні способи використання логіки, які знімають ці обмеження.

17.3.2. Продукційна модель

База фактів і база правил. У продукційній моделі, або моделі, заснованій на правилах, знання зображуються у вигляді сукупності фактів і правил.

Факти нічим не відрізняються від даних, вони можуть бути лише елементарними. Кажучи про елементарність, ми маємо на увазі відсутність суттєвої структурованості фактів. Максимально допустима структурованість – це можливість зображення у вигляді фактів описів сутностей, що мають атрибути, яким надаються елементарні значення. Факти утворюють *базу фактів*.

Правила описують процедурні знання і подають їх у вигляді пропозицій: «*якщо істинна певна умова, то виконати певну дію*».

Під *умовою* розуміють пропозицію-зразок, за якою виконується пошук у базі фактів. Умову іноді називають *посилкою правила*, а дії, що виконуються в разі успішного завершення пошуку за умовою, *дією правила*. Дію також називають *висновком правила*. Дія може бути *проміжною* або *термінальною (цільовою)*.

Проміжні дії – це окремі кроки в ланцюжку можливих (або необхідних) дій. Такою дією може бути, наприклад, ініціювання діалогу з людиною, внесення змін у базу фактів або породження додаткових умов. Термінальні дії завершують роботу системи. Сукупність правил утворює базу правил. База знань у продукційній моделі – це сукупність бази фактів і бази правил.

Побудова бази знань на основі продукційної моделі є домінуючим підходом в експертних системах. Наприклад, у медичній експертній системі правила *if...then* можуть використовуватися для встановлення взаємозв'язків між симптомами і діагнозами. Під час виведення реальний симптом зіставляється з тими, які є в лівих частинах правил і в разі збігу права частина відповідного правила вважається можливим діагнозом. Якщо є інші правила, що містять у лівих частинах отриманий можливий діагноз, то він розглядається як проміжний симптом. У цьому випадку здійснюється подальше виведення, яке триває доти, доки не буде отримано результат, з якого нічого вже не можна вивести. Якщо більше немає правил, на основі яких можна зробити виведення з отриманого можливого діагнозу, то він розглядається як «остаточний». На будь-якому кроці такого виведення може виявитися кілька застосовних правил і тоді породжується дерево виведення, що визначає множину діагнозів.

Машина виведення. Правила продукційної моделі не впорядковані. Кожне з них існує незалежно від інших правил. У зв'язку з цим потрібний спеціальний механізм, який керуватиме перебиранням правил. Такий механізм називається машиною виведення.

Машина виведення складається з двох компонентів:

1. Компонент виведення реалізує власне дедуктивне виведення. Тобто, якщо в базі фактів є факт *A*, а в базі правил є правило *If A then B*, то робиться висновок про необхідність застосування дії *B*.
2. Компонент керування, або інтерпретатор правил керує процесом перебирання фактів і застосування правил.

Інтерпретатор правил працює за описаним нижче алгоритмом.

1. Зіставлення. Здійснюється пошук множини правил, посилки яких зіставляються хоча б з одним фактом із бази фактів. Усі правила з цієї множини є застосовними до поточної бази фактів. Якщо правил у цій множині більше одного, то кажуть, що множина правил є конфліктною (у тому розумінні, що будь-яке з правил можна застосувати і невідомо, яке саме).

2. Вибір. Алгоритм роботи інтерпретатора є циклічним. На кожній ітерації циклу може бути застосовано лише одне правило. Якщо правил більше одного, інтерпретатор має вирішити конфлікт, тобто обрати з правил найвідповідніше. Вибір здійснюється на основі критерію, який може встановлюватися ззовні.

3. Виконання. Відібране правило запускається на виконання (спрацьовує). Суть спрацьовування полягає у виконанні дії, описаної у висновку правила. Такими діями можуть бути:

- коректування критерію вибору правил;
 - запис, видалення або коректування фактів у базі фактів;
 - запис, видалення або коректування правил у базі правил;
 - виконання інших дій (ведення діалогу з людиною, перевірка цілісності тощо).
- Схематично процес інтерпретації правил зображено на рис. 17.1.

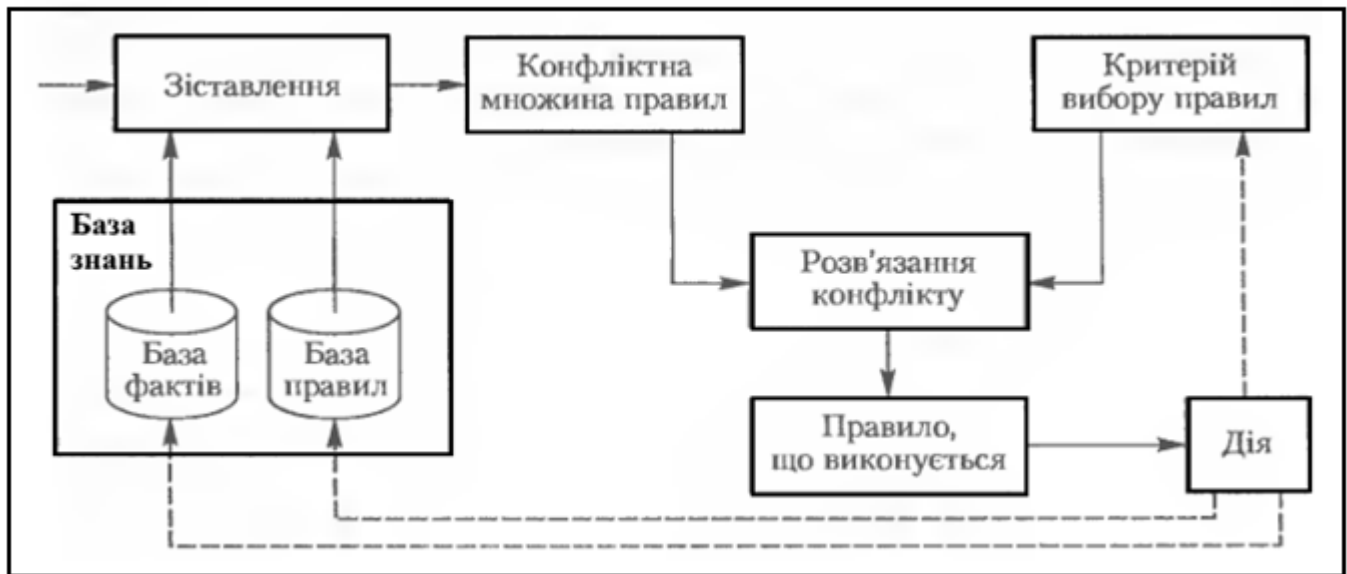


Рис. 17.1. Інтерпретація правил

У цій схемі робота машини виведення залежить від стану бази знань і критерію вибору правил. Існує також варіант організації машини виведення, за якого враховується передісторія її роботи, тобто поведінка механізму виведення на попередніх ітераціях.

Стратегії керування виведенням. Машина виведення має вирішувати, як перебирати факти і правила бази знань, а також на підставі якого критерію слід вибирати правила з конфліктної множини. Відповідні рішення приймаються згідно з обраною стратегією керування виведенням. Зазвичай основні складові цієї стратегії вбудовані в машину виведення і їх не можна змінити.

Розробляючи стратегію керування виведенням, слід вирішити такі питання.

1. Яку точку в просторі станів обрати як початкову? Від вибору цієї точки залежить спосіб виконання пошуку – в прямому чи зворотному напрямку.
2. Якою має бути стратегія перебирання правил – углибину чи вшир?

Пряме і зворотне виведення. У системах з *прямим виведенням* за відомими фактами відшукується факт, який з них впливає (рис. 17.2, а). Якщо такий факт вдається знайти, то він записується в базу фактів. Пряме виведення називають також *виведенням, керованим даними*, або *виведенням, керованим посилками правил*.

Під час *зворотного виведення* спочатку висувається гіпотеза, а потім механізм виведення ніби повертається назад, переходячи до фактів і намагаючись знайти в базі фактів ті, які підтверджують гіпотезу (рис. 17.2, б). Якщо гіпотеза не підтверджується фактами з бази фактів, одна з її можливих посилок вважається гіпотезою, що деталізує початкову і є стосовно неї підціллю. Далі відшукуються факти, які могли б підтвердити дану підціль, тобто процес рекурсивно повторюється. Виведення цього типу називається також *виведенням, керованим цілями*.

Пошук углибину та вшир. Під час пошуку вглибину черговою під ціллю стає та, що відповідає більш детальному рівню опису задачі. Виконуючи пошук ушир, машина виведення спочатку спробує знайти розв'язок серед можливих варіантів одного рівня і потім, за необхідності, перейде на наступний рівень деталізації.

На рис. 17.2 графічно зображено пошук углибину та вшир за умов прямого й зворотного виведення.

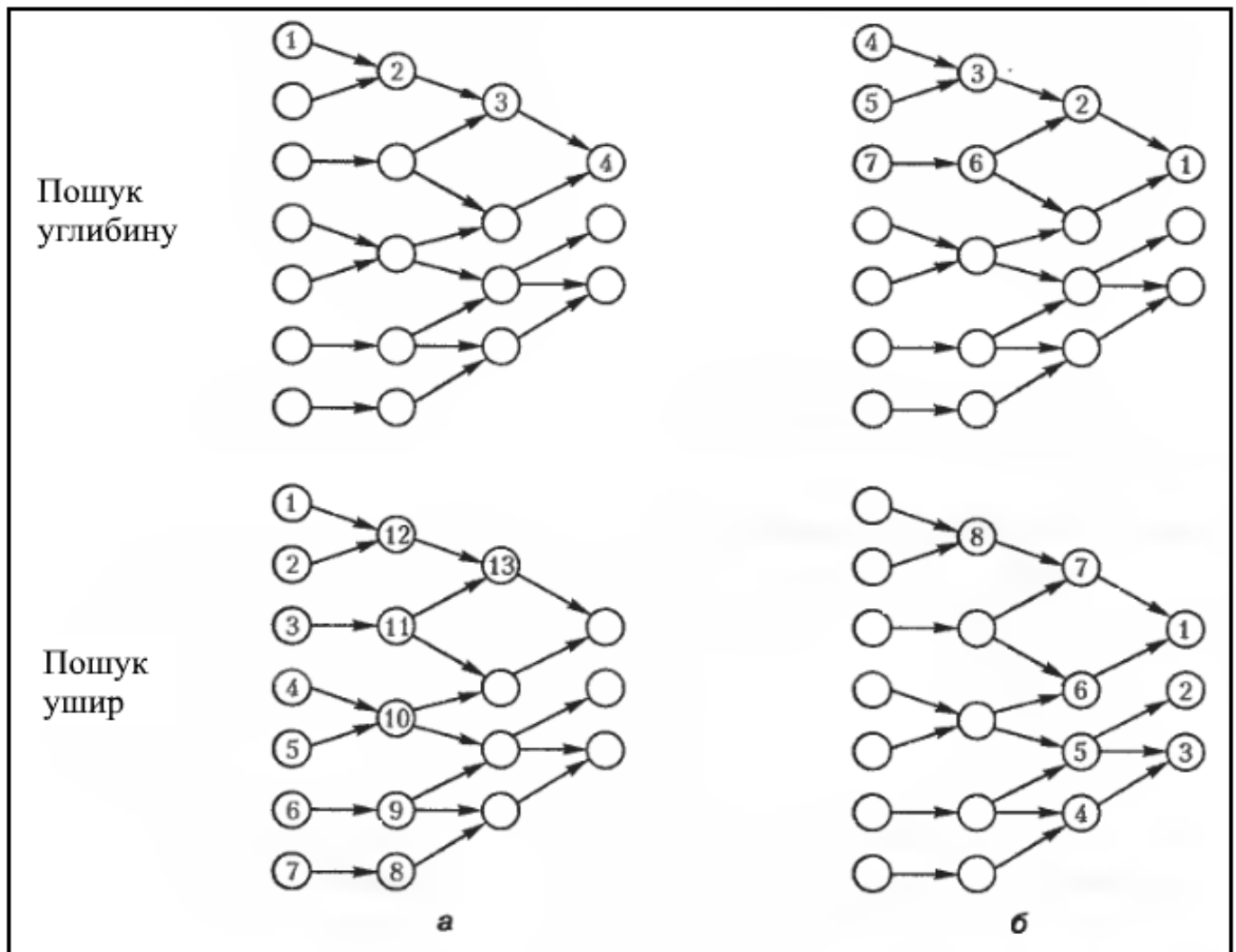


Рис. 17.2. Стратегії виведення: пряме виведення (а); зворотне виведення (б)

Проблеми. Продукційна модель знімає обмеження, характерні для логіки, проте з нею пов'язані інші проблеми: нескінченні цикли, можлива суперечність знань і непрозорість поведінки машини виведення.

Нескінченні цикли виникають у тому випадку, коли машина виведення повертається до правил, які вже були переглянуті. Це можливо, наприклад, за наявності таких правил:

If A then B; If B then C; If C then A.

Суперечливі знання з'являються тоді, коли додавання нових правил призводить до суперечності тим фактам, які можна було отримати раніше.

Непрозорість поведінки обумовлена тим, що немає жодних принципів, які б встановлювали порядок перегляду правил і їхнього застосування в тому випадку, коли може бути застосовано кілька правил. Унаслідок цього досить важко обробляти продукційні бази знань великого обсягу, оскільки навіть за умов коректності всіх наявних правил хибний порядок їхнього виконання може призвести до помилок, що важко виявляються.

Частково зняти обмеження, характерні для формально-логічної та продукційної моделей, можна шляхом структуризації бази знань. Названі моделі допускають зображення в базі знань лише елементарних фактів. Структуризація фактів приводить до створення груп взаємопов'язаних фактів, тобто певних абстракцій. Структурні абстракції можуть мати свою семантику, щодо якої застосовуються правила виведення.

Семантичні мережі та фрейми, що розглядатимуться далі, найчастіше використовуються у моделях, які підтримують структурні абстракції.

17.3.3. Семантичні мережі

Знаннями можна називати описи зв'язків між абстрактними поняттями і сутностями, що є конкретними об'єктами реального світу. Поняття і зв'язки між ними можна описати мережею, що складається з вузлів і дуг. Вузли в такій мережі зображують сутності й поняття, а дуги відповідають зв'язкам між ними; усі вузли й дуги можуть бути позначені мітками, які вказують, що саме вони описують. Така форма зображення знань називається семантичною мережею.

Семантичні мережі як моделі даних були розроблені дослідниками, які працювали над проблемами штучного інтелекту. Базові структури можуть бути зображені у вигляді графу, множина вершин і дуг якого утворює мережу як, наприклад, у мережній моделі даних. Проте семантичні мережі призначені для зображення і систематизації знань загального характеру. Поштовхом до розвитку моделей цього класу стали насамперед проблеми алгоритмізації процесів розпізнавання природної мови. Використання семантичних мереж було спочатку пов'язане з моделюванням людської пам'яті, яка має асоціативну природу. Асоціації зображувалися за допомогою дуг, що з'єднують вершини мережі.

Структура семантичної мережі. Будь-яка семантична мережа є графом. Вершини такого графа можуть зображувати предмети (значення екземплярів або сутностей), а дуги – твердження щодо предметів (зв'язки між предметами). Наприклад, одна вершина може зображувати сутність **Іван**, інша – **Петро**, а дуга між ними – твердження **Іван – брат Петра**.

Одиниця інформації в цій моделі може розглядатися як предмет (сутність) або твердження. Наприклад, колір «зелений» може відповідати предмету або твердженню. Зв'язок «бути братом» може розглядатися як твердження або предмет із певними властивостями (наприклад, «це дійсний факт» або «хтось вірить у це»). Можливість інтерпретації об'єкта як предмета або твердження має аналогії в інших моделях даних, де дані інтерпретуються як типи сутностей, їхні атрибути або зв'язки.

Виразові можливості семантичної мережі, що зображує лише предмети й твердження, достатньо обмежені. Виразова потужність моделі підвищується завдяки диференціації вершин і дуг мережі за категоріями.

Категорії вершин встановлюються відповідно до предметів, що зображуються ними. Наведемо один із найважливіших способів такої категоризації. Виділимо чотири категорії вершин:

- концепти (поняття);
- події;
- характеристики (властивості);
- значення.

Концепти – категорія об'єктів, заради яких будується модель. Концепти використовуються для специфікації сутностей модельованої предметної області (**Іван, Петро, місто Київ**).

Події відповідають діям у предметній області, що зображується. Наприклад, дія **Іван вчить Петра** зображується вершиною події для дії **вчить** і двома вершинами-концептами **Іван** і **Петро** (рис. 17.3).

Дуги, що сполучають вершини-події та вершини-концепти, відповідають ролям концептів у подіях (наприклад, у події вчить концепт **Іван** є агентом, а концепт **Петро** – об'єктом дії).

Характеристики – це вершини, що відповідають властивостям концепту. Наприклад, **вага** і **зріст Івана** – характеристики концепту **Іван**.

Нарешті, **значення** – це вершини, що відповідають значенням, яких можуть набувати характеристики. Так, характеристика **Вага**, що належить концепту **Іван** і має значення 80, визначає, що вага Івана становить 80 кг.

Додатковим засобом забезпечення виразової потужності семантичних мереж даних є розподіл вершин за типами. Зазвичай **вершини-концепти**, що зображують об'єкти, відрізняють від **вершин-класів**, що зображують типи. Наприклад, **Іван** – концепт, а **ОСОБА** – клас. Концепт може належати більш ніж одному класу. Отже, різні ролі об'єкта зображуються його належністю до різних класів.

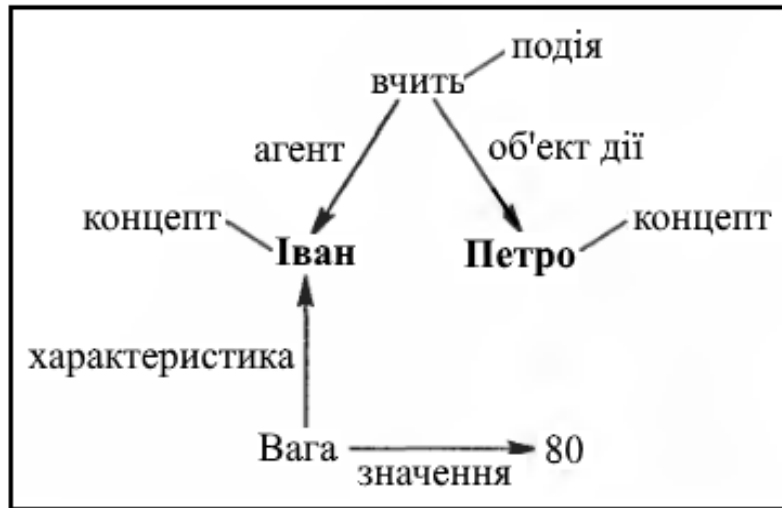


Рис. 17.3. Приклад семантичної мережі

Відмінність між класом і концептом близька до відмінності між типом і екземпляром в інших моделях даних, але у зв'язку з цим слід зробити два зауваження.

По-перше, в семантичних мережах зображуються як класи, так і концепти. В інших моделях даних типи специфікуються в схемі, а екземпляри зображуються в базі даних. По-друге, в семантичних мережах концепти зв'язуються з різними класами. У мережній та ієрархічній моделях даних це не допускається.

Відмінність між вершинами-концептами і вершинами-класами обумовлює виділення трьох різновидів дуг. Дуга, що сполучає два концепти, відповідає *твердженню*. Дуга між класом і концептом зображує *породження екземпляра класу*. Нарешті, дуга, що зв'язує два класи, характеризується як *бінарний зв'язок* (сам по собі він може розглядатися як клас). На рис. 17.4 наведено приклад семантичної мережі. Тут вершини **Іван** і **Петро** – це концепти, **ШКОЛА**, **ДИТИНА** – класи, дуга брат – твердження. Дуги, що зв'язують **Івана** і **Петра** з класом **ДИТИНА**, зображують зв'язок екземплярів з класом. Дуга *відвідує* зображує бінарний зв'язок між класами **ДИТИНА** і **ШКОЛА**.



Рис. 17.4. Різновиди дуг і вершин у семантичній мережі

Класи можуть бути сполучені ієрархічно за допомогою зв'язків «ціле/частина» і «узагальнення/різновид» (рис. 17.5). Наприклад, вік є «частиною» студента, а студент є «різновидом» особи. Ієрархія класів використовується для того, щоб вказати на успадкування властивостей одного класу іншим. Клас успадковує всі атрибути (властивості) класу, розташованого вище за ієрархією «узагальнення/різновид». За зв'язком «клас/екземпляр» концепт успадковує всі атрибути класу, до якого він належить.

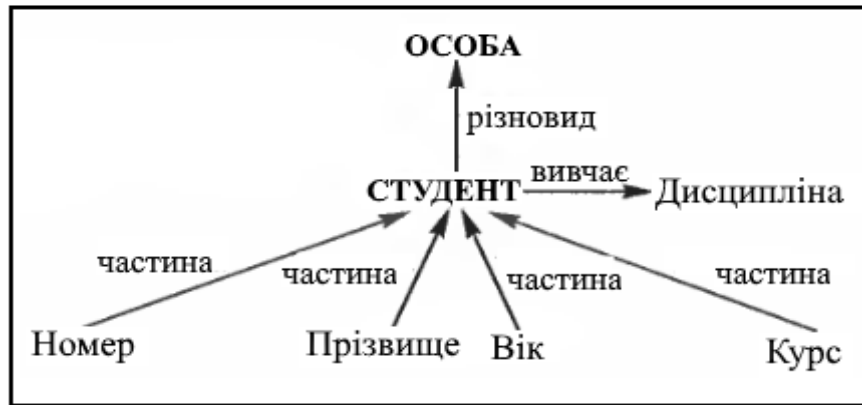


Рис. 17.5. Ієрархія класів

З метою структуризації і забезпечення успадкування властивостей бінарний зв'язок класів можна розглядати, у свою чергу, як клас, і на множині таких класів також можна встановлювати ієрархічні зв'язки «ціле/частина» і «узагальнення/різновид». Наприклад, визначивши на класі **ОСОБА** бінарні зв'язки **Взаємини** і **Шлюб**, можна встановити, що вони, у свою чергу, зв'язані відношенням «узагальнення/різновид» (**Шлюб** є різновидом зв'язку **Взаємини** і навпаки, **Взаємини** є узагальненням зв'язку **Шлюб**). Якщо бінарні зв'язки розглядати як вершини графу семантичної мережі, то їх також можна з'єднувати дугами (рис. 17.6).



Рис. 17.6. Зв'язки між зв'язками

Клас, екземплярами якого є інші класи, називається метакласом. Оскільки метаклас є класом, то він може брати участь в ієрархічних зв'язках типу «ціле/частина» і «узагальнення/різновид». Коренем цієї ієрархії може бути вершина КЛАС, що зображує клас усіх класів.

Отже, семантичні мережі моделі даних достатньо складні. Вершини семантичної мережі можуть зображувати екземпляри, класи і метакласи, а дуги – твердження, породження екземплярів і бінарні зв'язки. Крім того, на бінарних зв'язках, класах і метакласах можуть бути означені ієрархічні зв'язки типу «ціле/частина» і «узагальнення/різновид».

Виведення в семантичних мережах. Виведення створюються за дугами, що мають спільні вузли. Вони базуються переважно на властивостях тих чи інших типів зв'язків між вершинами мережі. Якщо, наприклад, відношення, поименоване є, належить типу «узагальнення/різновид» (яке має властивість транзитивності) і в мережі зафіксовано, що футболіст є спортсмен і спортсмен є людина, то з цього робиться висновок, що футболіст є людина.

Ймовірно, кращі результати в галузі виведення в семантичних мережах можна буде отримати в разі їх комбінування з іншими моделями знань.

17.3.4. Фреймова модель

Фреймова модель даних є методом зображення знань, який базується на теорії фреймів, запропонованій М. Мінським у 1975 році. Ця теорія розроблялася, зокрема, і для вирішення проблеми виходу за межі контексту.

У фреймовій моделі одиницею зображення є фрейм (об'єкт). Він є формою зображення певної ситуації, яку можна (або доцільно) описувати сукупністю понять і сутностей.

У різних галузях науки використовується поняття абстрактного зображення. Наприклад, слово «кімната» породжує зображення кімнати: «житлове приміщення зі стінами, підлогою, стелею, вікнами і дверима». В цьому описі є «гнізда» – невизначені значення певних атрибутів, наприклад, кількість стін, вікон і дверей, висота стелі. У теорії фреймів таке зображення кімнати називається фреймом кімнати.

Розрізняють *фрейми-зразки* (прототипи, що є описами) і *фрейми-екземпляри* (конкретні значення прототипів).

У фреймі сутності предметної області зображуються у вигляді пар «атрибут-значення», або так званих *слотів*. Слоти, у свою чергу, можуть містити *фасети*, або обмеження на множину допустимих значень.

Слот у складі фрейму-екземпляра може набувати значень у такі способи:

- наданням значення за замовчуванням, що вказується у фреймі-зразку;
- через успадкування властивостей від іншого фрейма;
- за формулою, вказаною в описі слота;
- через з'єднану зі слотом процедуру;
- під час діалогу з користувачем;
- з бази даних.

На сьогодні розроблені спеціальні мови для зображення знань у вигляді фреймів: FRL (Frame Representation Language), KRL (Knowledge Representation Language), інструментальні засоби підтримки цих мов, а також фрейм-орієнтовані експертні системи.

Недоліки. Як і інші моделі, фреймова модель має певні недоліки. Перший з них полягає в тому, що фрейми не можуть так легко зображувати евристичні знання, як продукційна модель. Фрейми також не можна пристосувати до нових ситуацій або об'єктів.

Окрім того, підходи, що дають змогу описувати структуру предметної області (семантичні мережі та фрейми), вимагають, щоб механізм виведення залежав від семантики структурних зв'язків. Відтак створення нових зв'язків зі своєю семантикою вимагає заміни або настроювання машини виведення. Одним зі шляхів вирішення цієї проблеми є об'єднання різних підходів (моделей).

17.4. Механізми виведення даних

У базах знань, окрім дедуктивного механізму виведення, про який ми вже згадували в цій лекції, можуть підтримуватися й інші. Далі ми коротко опишемо ще два механізми виведення: індуктивний та за аналогією.

17.4.1. Індуктивне виведення

Індуктивне виведення – це виведення від часткового до загального, виведення з наявних даних (прикладів) загального правила, що їх пояснює.

Розглянемо простий приклад. Нехай $f(x)$ – певний невідомий многочлен. Подивимося, як виводиться $f(x)$, якщо послідовно задаються (як приклади) пари значень $(0, f(0))$, $(1, f(1))$, Припустимо, що цими парами є $(0, 1)$, $(1, 1)$, $(2, 3)$, $(3, 7)$, $(4, 13)$, $(5, 21)$.

Спочатку задається пара $(0, 1)$, і можна покласти $f(x) = 1$. Потім задається пара $(1, 1)$, яка відповідає запропонованій функції $f(x) = 1$. Отже, у цей момент немає необхідності змінювати виведення. Нарешті, нехай задається пара $(2, 3)$. Тепер наше виведення не відповідає вихідним даним, тому відмовимося від нього й після кількох спроб і помилок виведемо нову функцію $f(x)$

$= x^2 - x + 1$, що відповідає всім заданим фактам $(0,1)$, $(1,1)$, $(2,3)$. Далі, ми переконуємося, що й наступні наявні в нас факти $(3, 7)$, $(4, 13)$, $(5, 21)$ підтверджують правильність виведення, тому немає необхідності його змінювати.

Як видно з цього прикладу, під час виведення в кожен момент часу враховуються всі дані, отримані до цього моменту. Дані, отримані пізніше, можуть не відповідати цьому виведенню. У такому разі виведення доводиться змінювати. Отже, у загальному випадку індуктивне виведення – це необмежено довгий процес.

Для здійснення індуктивного виведення необхідно визначити:

- множину правил виведення;
- метод зображення правил;
- спосіб зображення прикладів;
- метод виведення;
- критерій правильності виведення.

Існують такі варіанти індуктивного виведення:

- на повних прикладах;
- на позитивних (правильних) прикладах (тобто тих прикладах, що відповідають шуканому узагальнюючому правилу);
- на негативних прикладах (тобто тих, які спростовують узагальнююче правило).

З повних даних, що містять позитивні й негативні приклади, завжди можна робити індуктивні висновки. Проте, користуючись лише позитивними даними, у деяких випадках індуктивні висновки робити не можна. Тоді слід розглядати негативні дані.

Серед типових областей застосування індуктивного виведення відзначимо напіваавтоматичне зображення знань в експертних системах, автоматичний синтез програм за прикладами в галузі інтелектуального програмування, напіваавтоматичне налагодження програм.

17.4.2. Виведення за аналогією

Принцип аналогії полягає у виведенні висновку, подібного посилці. Пояснимо сформульований принцип на прикладі (рис. 17.7). Нехай задано два об'єкти, S_1 і S_2 . Щодо об'єкта S_1 відомі факти $\alpha_1, \alpha_2, \dots, \alpha_n$, а щодо об'єкта S_2 – факти $\beta_1, \beta_2, \dots, \beta_n$. Нехай між фактами об'єктів S_1 і S_2 задано відношення подібності φ , тобто $\alpha_i \varphi \beta_i$ для $1 \leq i \leq n$. Нехай існує виведення з фактів $\alpha_1, \alpha_2, \dots, \alpha_n$ факту α . Тоді **аналогією** називається виведення з фактів $\beta_1, \beta_2, \dots, \beta_n$ такого факту β , що $\alpha \varphi \beta$.

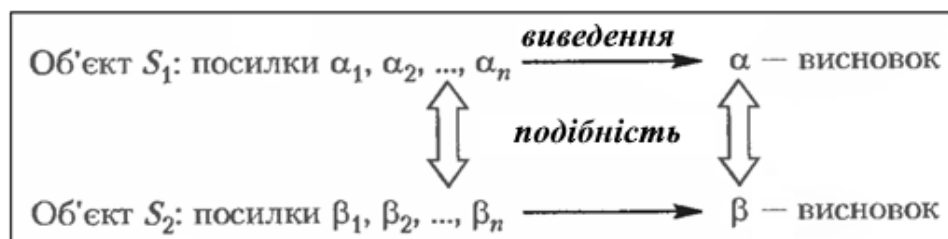


Рис. 17.7. Принцип аналогії

Щоб отримати конкретні механізми виведення за аналогією, слід уточнити такі аспекти розглянутої загальної схеми:

як здійснюється виведення факту α з фактів $\alpha_1, \alpha_2, \dots, \alpha_n$;

чим є відношення подібності φ ;

як за заданим φ здійснюється виведення такого β , що $\alpha \varphi \beta$.

Аналогія – більш природний механізм виведення для людини, ніж індукція. Класичними прикладами виведення за аналогією є прогнози і методи розв'язання задач. Аналогія застосовується в дослідженнях з машинного перекладу, використовується під час автоматизації доведення теорем та розроблення програм. Згідно з принципом аналогії нові факти прогнозуються (чи навіть виводяться) шляхом перетворення уже відомих знань.

17.5. Інтелект людини і штучний інтелект

В основі систем баз знань лежать принципи роботи людського інтелекту. Інтелектом називається здатність підходити до розв'язання будь-якої задачі з урахуванням наявного досвіду. Згідно Хармону і Кінгу (Harmon & King, 1985) для людського інтелекту характерні такі властивості:

- здатність навчатися;
- здатність знаходити аналоги;
- здатність створювати нові поняття на основі відомих понять ефективність обробки неоднозначних і суперечливих повідомлень;
- здатність визначати відносну важливість різних складових частин задачі;
- гнучкість підходу до розв'язання задачі;
- здатність розбиття складної задачі на складові частини;
- здатність моделювання сприйманого світу.

Машинні знання – це те, що і штучний інтелект. Родоначальником в цій області є Алан Тюрінг, британський математик. Однак незважаючи на те, що Тюрінг розробив первісну концепцію штучного інтелекту ще в 1937 р., офіційно штучний інтелект з'явився тільки в 1956 р. Це сталося в Дартмутського коледжі, під час зустрічі групи вчених, на якій обговорювалося потенціал комп'ютерів в області стимуляції когнітивного процесу людини. Термін «штучний інтелект» був запропонований одним з організаторів конференції, Джоном Маккарті.

Штучний інтелект – це одна з гілок інформатики. Він пов'язаний з комп'ютерами, які стимулюють процес розв'язання задачі шляхом дублювання функцій людського мозку. Штучний інтелект включає в себе сукупність програмного та апаратного забезпечення та методів імітації властивості людині діяльності як розумової (мислення, прийняття рішень, міркування, вирішення завдань, навчання і пошук даних), так і фізичної (сенсорні та моторні навички). Комплексне розв'язання задач моделюється за допомогою подання когнітивного процесу людини, а когнітивне моделювання розв'язує задачі, оцінюючи знання як людина.

Когнітивне моделювання і штучний інтелект – родинні, але різні дисципліни. Когнітивне моделювання – це методика моделювання процесу людського пізнання, на якому будуються осмислені роздуми, а штучний інтелект – методика моделювання розумної поведінки, в якій міркування зовсім не обов'язкові. Правда, відмінності між двома цими методиками поступово стираються.

17.6. Застосування баз знань

Прості бази знань можуть використовуватися для створення експертних систем і зберігання даних про організацію: документації, посібників, статей технічного забезпечення. Головна мета створення таких баз – допомогти менш досвідченим людям знайти існуючий опис способу вирішення будь-якої проблеми предметної області.

Онтологія може служити для представлення в базі знань ієрархії понять і їх відношень. Онтологія, яка містить ще й екземпляри об'єктів, не що інше як база знань.

Системи, засновані на знаннях, реалізуються на базі наступних інтелектуальних алгоритмів:

- експертні системи;
- нейронні мережі;
- нечітка логіка;
- генетичні алгоритми.

17.6.1. Експертні системи

База знань – важливий компонент інтелектуальної системи. Найбільш відомий клас таких програм – експертні системи.

Експертна система – комп'ютерна програма, яка здатна частково замінити фахівця-експерта у вирішенні проблемної ситуації. Сучасні експертні системи почали розроблятися дослідниками штучного інтелекту в 1970-х роках, а в 1980-х отримали комерційне підкріплення.

В інформатиці експертні системи розглядаються спільно з базами знань як моделі поведінки експертів у визначеній області знань з використанням процедур логічного висновку і прийняття рішень, а бази знань – як сукупність фактів і правил логічного висновку в обраній предметній галузі діяльності.

Характерними рисами експертної системи є:

- чітка обмеженість предметної області;
- здатність приймати рішення в умовах невизначеності;
- здатність пояснювати хід і результат рішення зрозумілим для користувачів способом;
- чіткий поділ декларативних і процедурних знань (фактів і механізмів виведення);
- здатність поповнювати базу знань, можливість нарощування системи;
- результат видається у вигляді конкретних рекомендацій для дій в ситуації, що склалася, не поступаються рішенням кращих фахівців;
- орієнтація на вирішення неформалізованих (спосіб формалізації поки невідомий) задач;
- алгоритм рішення не описується заздалегідь, а будується самою експертною системою;
- відсутність гарантії знаходження оптимального рішення з можливістю вчитися на помилках.

Структура експертних систем. Структура експертних систем:

1. Інтерфейс користувача.
2. Користувач.
3. Інтелектуальний редактор бази знань.
4. Експерт.
5. Інженер по знанням.
6. Робоча (оперативна) пам'ять.
7. База знань.
8. Вирішувач (механізм виведення).
9. Підсистема пояснень.

База знань складається з правил аналізу інформації від користувача з конкретної проблеми. Експертна система аналізує ситуацію і, в залежності від спрямованості експертної системи, дає рекомендації з вирішення проблеми.

Як правило, база знань експертної системи містить факти (статичні відомості про предметну область) і правила – набір інструкцій, застосовуючи які до відомих фактів можна отримувати нові факти.

Головна мета створення будь-якої бази знань – скоротити час і трудовитрати на вирішення типових інцидентів.

Користувач – фахівець предметної області, для якого призначена система.

Інженер зі знань – фахівець в галузі штучного інтелекту, який виступає в ролі проміжного буфера між експертом і базою знань.

Інтерфейс користувача – комплекс програм, що реалізують діалог користувача з експертною системою.

База знань – ядро експертної системи, сукупність знань предметної області

Вирішувач – програма, що моделює хід міркувань експерта на підставі знань, наявних в БД.

Підсистема пояснень – програма, що дозволяє користувачеві отримати відповіді на питання: «Як була отримана та чи інша рекомендація?» і «Чому система прийняла таке рішення?».

17.6.2. Інші способи застосування штучного інтелекту

Поряд із системами з базою знань існують інші програми штучного інтелекту, такі як ігри, рішення головоломок, обробка природної мови, розпізнавання мови, машинне зір, робототехніка, інтелектуальне навчання, навчання машини і розв'язання загальних задач. Розвиток цих напрямів сприятиме розробці більш досконалих і більш «схожих на людину» систем з базою знань.

Ігри та рішення головоломок (наприклад, шахи) були першою областю програми штучного інтелекту та інженерії знань, де мала місце імітація людського інтелекту і здібностей щодо розв'язання задач.

Засоби обробки природних мов дають можливість комп'ютерам розуміти повідомлення на різних мовах і здійснювати вербальні комунікації з живими користувачами. Вони забезпечені базою знань (словником) і в даний час використовуються для створення інтерактивного інтерфейсу з комп'ютером в таких областях, як електронні таблиці, програми управління базами даних, операційні системи та системи автоматичного перекладу. В майбутньому обробка природних мов буде використовуватись для сканування, інтерпретації та узагальнення масивом даних для різних прикладних систем з базою знань.

Розпізнавання мови і машинне зір імітують два найбільш важливих людських почуття і таким чином спрощують взаємодію живого експерта і комп'ютера.

Робототехніка займається копіюванням фізичних характеристик людини і їх машинною реалізацією.

Інтелектуальне навчання застосовується в основному при навчанні за допомогою комп'ютера. Навчання машини – це спроба імітації навчання людини з використанням дедуктивних і індуктивних процесів.

Системи розв'язання загальних задач призначені для розв'язання різних задач, які представлені на формальній мові, з використанням алгоритмів та евристики.

17.6.3. Погляд у майбутнє

Як і в інших областях, сучасне інженерії знань належить реалістам, які адаптують технології до задоволення існуючих потреб. Однак майбутнє інженерії знань залежить від мрійників, які передбачають появу технологій, які будуть служити людям в майбутньому.

У розпорядженні інженерів по знанням буде більш досконале апаратне і програмне забезпечення. Швидка дія і велика ємність запам'ятовуючих пристроїв дозволить використовувати знання, засновані на здоровому глузді, і надасть можливість одночасно обробляти правила, фрейми та інші структури знань. Стане необхідною обробка даних з масовим паралелізмом і застосування суперкомп'ютерів.

Програмне забезпечення дозволить навчання на базі досвіду і оновлення його бази даних. Також воно буде володіти можливостями динамічного відгуку на мінливі вхідні умови або функцію. Системи з базою знань будуть покладатися на автоматизоване програмне забезпечення з отримання знань. В якості призначених для користувача інтерфейсів будуть використовуватися розпізнавання мови і введення рукописної інформації. Комунікації будуть багатомовними, з'являться можливості машинного перекладу.

Придбання знань – це те, що обмежує розвиток систем з базою знань. Ми зможемо розробити більш ефективні системи з базою знань тільки в тому випадку, якщо ми краще зрозуміємо способи обробки знань, їх зберігання та пошуку, властиві людського розуму, а також принципи накопичення людини досвідом.

У комп'ютера великі можливості штучного інтелекту. Він перетворюється з пристроєм для обробки даних у пристрій для обробки знань. Обладнана сенсорними зв'язками та роботами, система з базою знань може збирати та аналізувати інформацію, а також діяти без втручання людини. Язикове програмне забезпечення буде імітувати інтуїцію. Додаткові технології, такі як нейромережі або «широкомасштабна» паралельна обробка, готують ґрунт для появи інтелектуальних машин більш високого рівня.

18. ПЕРСПЕКТИВИ РОЗВИТКУ БД ТА СКБД

Сучасні бази даних є основою численних інформаційних систем. Інформація, накопичена в них, є надзвичайно цінним матеріалом, і зараз широко поширюються методи обробки баз даних з точки зору вилучення з них додаткових знань, методів, які пов'язані з узагальненням і різними додатковими способами обробки даних. Бази даних в даній концепції виступають як сховища інформації, цей напрямок називається «Сховища даних» (Data Warehouse) [16].

Для роботи з «Сховищами даних» найбільш значущим стає так званий інтелектуальний аналіз даних (ІАД), або data mining, – це процес виявлення значимих кореляцій, зразків і тенденцій у великих обсягах даних. З огляду на високі темпи зростання обсягів накопиченої в сучасних сховищах даних інформації, неможливо недооцінити роль ІАД.

У 2000-х поряд з добре зарекомендованим себе реляційним стандартом розвивалися і інші, які були об'єднані ключовим словом NoSQL. У деяких сферах застосування вони є цікавою альтернативою реляційних баз даних.

18.1. Революція 2005 року в реляційних базах даних

В 2005 році в СКБД MS SQL Server-2005 з'явився тип даних «XML data type», тобто фірма Майкрософт дозволила зберігати в одній комірці таблиці складні структури даних, як завгодно, аж до всієї бази даних (БД). Щоб охарактеризувати революційність цієї події, згадаємо коротко історію розвитку реляційних СКБД (РБД).

Кодд опублікував першу роботу з РБД в 1970 році, а в 1981 році з'явилася перша промислова реалізація – SQL / DS. У 2006 році у всьому світі відсвяткували 25-річчя РБД.

Математик Кодд ввів суворі постулати, що визначають суть реляційних БД. Перший постулат РБД: БД – сукупність відношень, які змінюються в часі (relation) – таблиць, всі осередки яких атомарні (неподільні). Протягом більше 30 років вивчаються нормальні форми РБД, призначені для ефективного опису інформаційних об'єктів.

Перша нормальна форма (НФ) вимагає виконання названого постулату, всі наступні НФ від 2-ї до 6-ї вимагають виконання властивостей всіх попередніх нормальних форм і якихось додаткових властивостей. Тобто всі РБД повинні виконувати перший постулат – атомарності даних у всіх осередках всіх відношень. Цьому вчили і вчать всі професори за тисячами підручників у всіх університетах світу.

Таким чином, Майкрософт, можна сказати, перекреслила, не чекаючи двадцятип'ятирічного ювілею, основу РБД. Тут же, слідом за Майкрософт, всі розробники РСКБД, включаючи ORACLE і IBM з DB2, боячись відстати в конкурентній боротьбі, теж поспішили ввести такі специфікації [1].

Все розвивається по спіралі. В інформаційній системі, створеній в 1996-1998 роках, зафіксовано понад 1000 СКБД [2]. На рис. 18.1 перераховані деякі найбільш значні продукти з роками їх появи. Овалами позначені ті з них, які дожили до наших днів.

Перші промислові СКБД – IMS, що підтримує ієрархічну модель даних, і IDMS, заснована на мережевий моделі, – мали дуже низькорівневі інтерфейси як при описі даних, так і при маніпулюванні. Програмування в середовищі цих СКБД було надзвичайно виснажливим. Тому поява реляційної моделі, спочатку передбачала досить високий рівень абстрагування від фізичної організації даних, які б давали хорошу основу для побудови мови маніпулювання високого рівня, було потужним поштовхом до побудови СКБД наступного покоління. Поява стандарту SQL, що базується на реляційній моделі, закріпило її провідне становище.

Таким чином, можна сказати, що перші півтора десятиліття розвитку СКБД пройшли під знаком спрощення моделі даних і підвищення рівня мови маніпулювання даними.

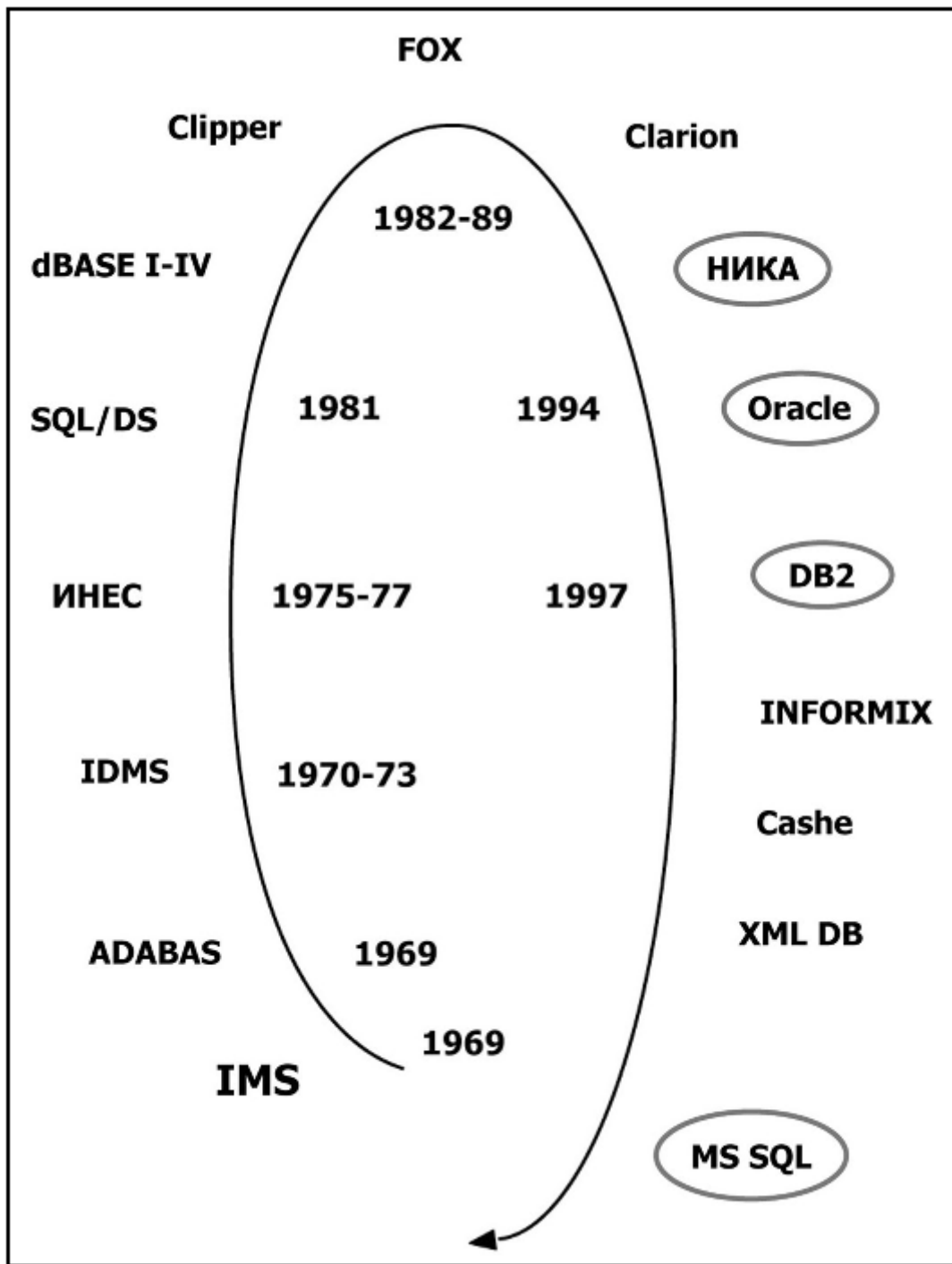


Рис.18.1 СКБД і роки їх створення

На рис. 18.2 наведено умовний графік, який показує за роками спрощення структур даних.

Переваги реляційної моделі полягають в простоті її елементарних структур – таблиць (відношень), і можливість внаслідок цього побудувати математично прозорий метод маніпулювання – реляційну алгебру, яка знайшла своє втілення в мові SQL, багаторазово описану. Менш відомі хронічні дефекти цієї моделі. Головний з них полягає в тому, що основні елементи структур – рядки (кортежі) – незалежні і повинні мати унікальний ідентифікатор – первинний ключ. Саме по цьому ключу даний елемент може бути пов'язаний з іншими елементами бази даних. Зазвичай підручники з РБД наводять приклади, коли цей ключ міститься серед змістовних елементів рядка (унікальну назву, номер паспорта і т. п.). Зрозуміло, це далеко не завжди так. Наведемо кілька простих прикладів.

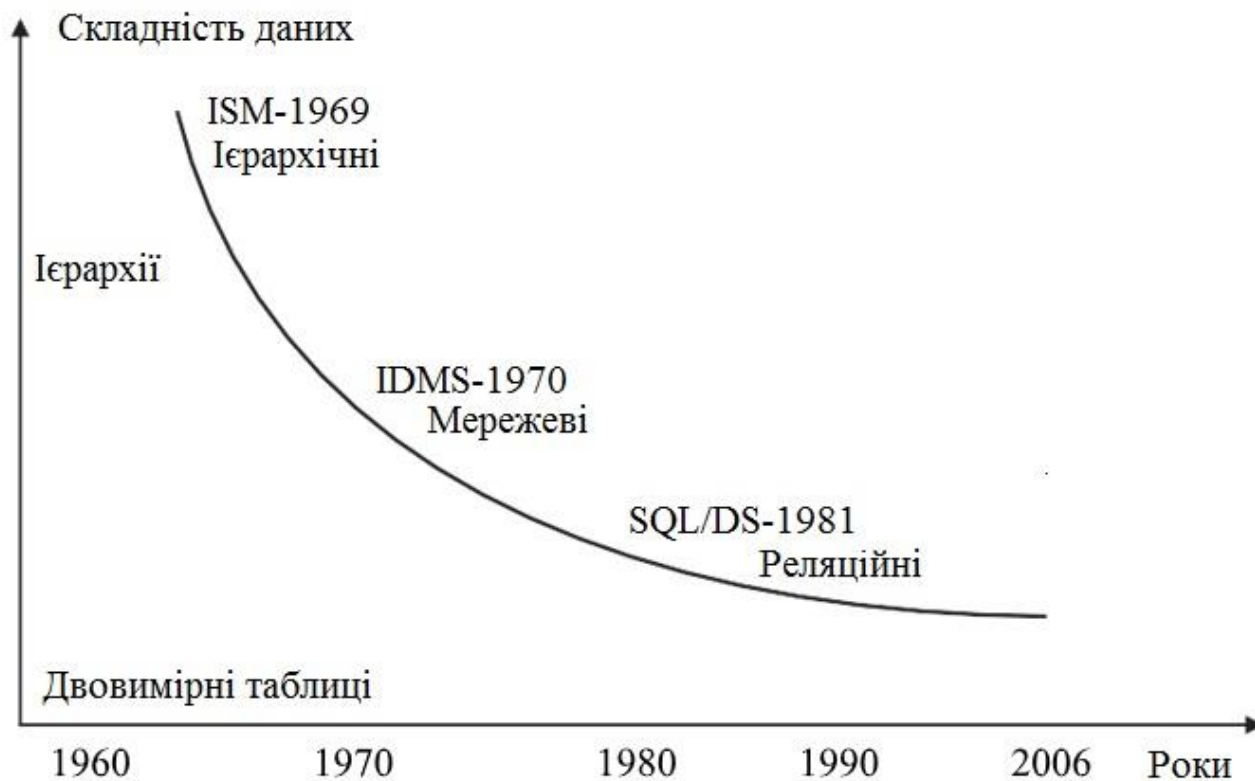


Рис. 18.2 Спрощення структур даних в 1970-2005 роках

Нехай є кімнати, а в них стоять столи, кожен з яких має свої розміри. Якщо столи мають ідентифікатори (наприклад, інвентарні номери, як у багатьох установах), то ми легко побудуємо базу даних з одного відношення, в якій перенесення столу в іншу кімнату відобразиться найпростішою операцією заміни номера кімнати в рядку. А якщо інвентарного номера немає?

Оскільки в кімнаті може стояти декілька однакових столів, нам доведеться ввести в рядок новий елемент-лічильник «кількість столів даного типу», і перенесення столу з кімнати в кімнату відобразиться в цілу програму, приблизно таку:

1. Знайти в цій кімнаті стіл даного типу (кортеж).
2. Якщо лічильник столів в кортежі більше 1, зменшити його на 1.
3. Якщо лічильник столів дорівнює одиниці, викреслити кортеж.
4. Знайти в другій кімнаті стіл даного типу.
5. Якщо він є, додати до відповідного лічильника 1.
6. Якщо такого стола немає, додати новий кортеж.

Інший приклад. Ми хочемо побудувати генеалогічне древо, що відображає зв'язки між батьками і дітьми. Так як будь-які характеристики наявних в базі людей можуть повторюватися, то для опису зв'язку нам обов'язково доведеться придумати і вводити унікальний ідентифікатор кожної людини. Але навіть в цьому випадку найпростіший запит «видати всіх предків даного персонажа» вилається в циклічний пошук по ідентифікаторах.

Розглянемо ще один близький за змістом приклад. Нехай є кілька об'єктів, структура інформації про яких дуже близька, але є деякі відмінності (республіки і області, самостійні відділи, управління та служби і т. п.). В реальності переважна кількість дій з цими об'єктами в БД однакові. Питається, як побудувати концептуальну схему? Можна вибрати один з декількох шляхів. Ми можемо побудувати окреме відношення для кожного типу об'єктів. Але тоді ми змушені будемо забезпечувати спільне відпрацювання їх на рівні операторів під час виконання.

Ми можемо побудувати одне відношення, включивши в нього всі атрибути всіх об'єктів. Однак створення відношень, велика частина атрибутів яких завжди порожня, вважається поганим стилем. Нарешті, ми можемо «відселити» атрибути в окремі відношення, забезпечивши їх ключами основного відношення для встановлення зв'язків. Дивне рішення, але чому б ні?

У всіх розглянутих прикладах проблеми присутні саме в реляційних структурах даних. У будь-якої об'єктної (ієрархічної, мережевої) бази даних ніяких питань навіть не виникне. Дані там визначаються місцем розташування в загальній структурі і ніякої додаткової ідентифікації не потребують.

Звичайно, нічого страшного для реляційних СКБД в наведених прикладах немає. Перший з них легко покривається введенням прихованих від користувача ідентифікаторів, останній – механізмом зовнішніх схем уявлень, що грає взагалі важливу роль в сучасних системах баз даних. Мабуть, лише другий по-справжньому неприємний.

Ми привели ці приклади лише для того, щоб показати, що простота реляційних структур дається зовсім не безкоштовно і призводить до появи абсолютно беззмістовних атрибутів, а то й цілих відношень, які роблять концептуальну схему ще більш непрозорою, що потребує підтримки і ускладнює маніпулювання.

Однак важливий недолік реляційних баз даних – надзвичайна складність зв'язків між відношеннями, що впливає безпосередньо з примітивності основних структур. Навіть найпростіша прикладна база даних містить 15-20 таблиць – відношень. Якщо ж система більш серйозна, число відношень різко зростає, їх стає багато десятків. Це ще терпимо. Однак зі зростанням швидкостей і обсягів зовнішньої пам'яті комп'ютерів зростає і складність вирішуваних завдань. Тепер концептуальна схема інтегрованої інформаційної системи містить сотні, а то й тисячі таблиць.

Укладений на папір опис даних такої системи займає лист формату А1, причому кожне відношення на ньому ледь можна розгледіти. Встежити за стрілками, які позначають зв'язки між відношеннями, практично неможливо. З'явилися найпотужніші засоби проектування баз даних, що дозволяють автоматично формувати її схему на основі опису бізнес-процесів і інформації, що міститься в них. Але і це не допомагає. Багато засновників великих інтегрованих систем (наприклад, ERP) дають користувачам доступ тільки до частини даних, наявних в системі, підриваючи одне з основних гасел батьків-засновників реляційних баз даних: користувач може сам, без допомоги програмістів, маніпулювати своїми даними.

Якщо провести аналогію з програмуванням, то чи можете ви собі уявити, щоб в програмі, написаній на будь-якій з сучасних мов, описи всіх змінних, функцій і об'єктів були винесені на верхній рівень і всі зв'язки між ними намальовані у вигляді стрілок? Це буде безграмотність і нонсенс. Людство придумало модульність, локалізацію областей дій змінних і описів, а потім і об'єктне програмування рівно для того, щоб кожен інформаційний об'єкт, будь то елементарні дані, структура або масив могли (і повинні були) існувати в деякій вкладеній одна в одну системі об'єктів, а не в абстрактному звалищі собі подібних.

Тим часом в РБД не існує ніяких локалізацій, і ми змушені розглядати всі таблиці як потенційно пов'язані між собою, що надзвичайно складно, незважаючи ні на які зовнішні схеми. Тому, вже починаючи з другої половини 1980-х років, робилися спроби побудувати на основі реляційної моделі «об'єктні» бази даних. З'явився навіть термін «реляційно-об'єктні СКБД». Хоча таких спроб за останні 20 років зроблено чимало і деякі з них дуже цікаві, великого успіху вони поки що не принесли.

Новий потужний удар по реляційних БД було завдано в кінці 1990-х років швидким розвитком комп'ютерних мереж і появою мови XML. Комп'ютерні мережі вимагають розвиненого апарату з'єднання і, відповідно, форматів обміну. Чим же повинні обмінюватися комп'ютери? Таблицями? Очевидно, ні. Ясно, що обмін повинен здійснюватися якимись осмисленими одиницями – документами. На роль мови опису змісту документів і претендувала мова XML, скоро стала стандартом.

Досить швидко стало зрозуміло, що XML може використовуватися не тільки для зв'язку між машинами, а й між програмами всередині однієї машини. У його формат зручно переводити дані при введенні, виведенні, в системах документообігу і т. п. З'явилися XML-сховища, парсери і інші засоби обробки. І ось закономірний підсумок.

Розглянемо основні положення порівняння реляційної моделі даних і XML, проведене співробітниками Microsoft [3]:

1. Якщо дані добре структуровані за відомою схемою, то РБД добре працює.

2. Якщо ж дані підлозі (слабо) структуровані, або зовсім не структуровані, або структура не відома, крім того, якщо ви домагаєтеся мобільності системи (або даних), то використання XML-моделі – хороший вибір.
3. XML гарний також, якщо:
 - дані розріджені;
 - структура даних може істотно змінюватися;
 - можлива рекурсія в описі даних;
 - порядок принципів для даних;
 - ви хочете запитувати або оновлювати дані на основі їх структури.
4. Якщо немає жодної з цих умов, то можна (але не обов'язково) використовувати РБД.

Такі висновки потребували революційної модернізації, зокрема введення нового типу даних XML data type.

Звичайно, неможливо ігнорувати мільйони програмістів і користувачів, що освоїли SQL, і реальні розробки, наявні в сотнях тисяч організацій на базі, перш за все, таких систем, як ORACLE і MS SQL. Однак введення в реляційну СКБД нового типу даних «XML data type» – не найкраще рішення проблем роботи з складно структурованими документами. Паралельне узгодження великих обсягів даних викликає багато додаткових ускладнень і має бути викоринене.

Розглянемо аргументи, які представив сам Кодд в статті про «Великий бій», розглядаючи можливі альтернативи майбутнього систем баз даних. У викладі Дейта це виглядає так [4].

1. Якщо ми додаємо операції високого рівня (з'єднання і т. д.), То отримуємо автоматичну навігацію, значить, навіть не програмісти зможуть досягти своїх цілей без будь-чєї допомоги.
2. Навпаки, якщо ми не додаємо такі операції, а додаємо нові структури даних, такі як зв'язки, то безпосередня навігація стає необхідною, значить, для досягнення цілей кінцевих користувачів необхідні програмісти.
3. Якщо ми додаємо і операції, і структури, то отримуємо непотрібну складність, значить, програмістам і адміністратору бази даних доведеться приймати набагато більше рішень.

З 2005 року розвиток РСКБД пішов по третьому шляху, самому абсурдному, з точки зору Кодда. У підручниках з РСКБД часто говориться, що всі документи, з якими працюють люди, – плоскі таблиці, і тому реляційні відношення є ідеальною моделлю даних. Що документи плоскі, це, безумовно, вірно, так як вони представляються на аркуші паперу. Але в той же час треба розуміти, що немає жодного реального документа, рівного реляційному відношенню, ця абстракція в своєму чистому вигляді на практиці ніколи не зустрічається. Заголовки таблиць, підсумки, проміжні заголовки, що повторюються рядки і багато іншого не вкладаються в реляційні відношення.

Справедливо інше твердження: будь-який документ, який зустрічається на практиці можна записати на мові XML. Правильною видається орієнтація не на записи в реляційних двомірних таблицях, а на дерева в XML-документах і операції високого рівня (з'єднання і т. д.) з операндами деревами, такі щоб не лише програмісти змогли досягти своїх цілей, як правило, без безпосередньої навігації. Повинна з'явитися нова модель даних, яка об'єднує переваги реляційної БД і XML DB.

До трьох шляхів розвитку систем БД, розглянутих Коддом, можна додати четвертий: якщо ми додаємо, зберігаючи «спартанську простоту», нові структури даних (XML-документи) і одночасно вводимо операції високого рівня, які забезпечують навігацію і автоматичну обробку таких даних (практично всіх вхідних і вихідних документів), то різні користувачі, від новачків до найдосвідченіших, можуть взаємодіяти з даними на основі загальної моделі, і навіть не програмісти зможуть досягти своїх цілей без будь-чєї допомоги.

18.2. Інтелектуальний аналіз даних

На думку фахівців Gartner Group, вже в 1998 р. ІАД увійшов в десятку найважливіших інформаційних технологій. В останні роки почалося активне впровадження технології ІАД. Її активно використовують як великі корпорації, так і більш дрібні фірми, які серйозно ставляться до

питань аналізу і прогнозування своєї діяльності. Очевидно, що на ринку програмних продуктів стали з'являтися відповідні інструментальні засоби.

Особливо широко методи ІАД застосовуються в бізнес-додатках аналітиками і керівниками компаній. Для цих категорій користувачів розробляються інструментальні засоби високого рівня, що дозволяють вирішувати досить складні практичні завдання без спеціальної математичної підготовки.

Актуальність використання ІАД в бізнесі пов'язана з жорсткою конкуренцією, яка виникла внаслідок переходу від «ринку продавця» до «ринку покупця». У цих умовах особливо важливо якість і обґрунтованість прийнятих рішень, що вимагає суворого кількісного аналізу наявних даних. При роботі з великими обсягами інформації, що накопичується необхідно постійно оперативно відслідковувати динаміку ринку, а це практично неможливо без автоматизації аналітичної діяльності.

У бізнес-додатках найбільший інтерес представляє інтеграція методів інтелектуального аналізу даних з технологією оперативної аналітичної обробки даних (On-Line Analytical Processing, OLAP). OLAP використовує багатовимірне уявлення агрегованих даних для швидкого доступу до важливої інформації і подальшого її аналізу.

Системи OLAP забезпечують аналітикам і керівникам швидкий послідовний інтерактивний доступ до внутрішньої структури даних і можливість перетворення вихідних даних з тим, щоб вони дозволяли відобразити структуру системи потрібним для користувача способом. Крім того, OLAP-системи дозволяють переглядати дані та виявляти наявні в них закономірності або візуально, або найпростішими методами (такими як лінійна регресія), а включення в їх арсенал нейромережевих методів забезпечує суттєве розширення аналітичних можливостей.

В основі концепції оперативної аналітичної обробки лежить багатовимірне представлення даних. Термін OLAP ввів Кодд в 1993 році. У своїй статті він розглянув недоліки реляційної моделі, в першу чергу неможливість «об'єднувати, переглядати і аналізувати дані з точки зору множинності вимірів, тобто найзрозумілішим для корпоративних аналітиків способом», і визначив загальні вимоги до систем OLAP, які розширюють функціональність реляційних СКБД і включають багатовимірний аналіз, як одну зі своїх характеристик.

Слід зауважити, що Кодд позначає терміном OLAP багатовимірний спосіб представлення даних виключно на концептуальному рівні. Які він використовував терміни – «Багатовимірне концептуальне уявлення» («Multidimensional conceptual view»), «Множинні вимірювання даних» («Multiple data dimensions»), «Сервер OLAP» («OLAP server») – не визначають фізичного механізму зберігання даних (терміни «багатовимірна база даних» і «багатовимірна СКБД» невідомі жодного разу).

Часто в публікаціях аббревіатурою OLAP позначається не тільки багатовимірний погляд на дані, але і зберігання самих даних в багатовимірній БД, що в принципі невірно [16].

За Коддом, багатовимірне концептуальне уявлення (multi-dimensional conceptual view) є найбільш природним поглядом керуючого персоналу на об'єкт управління. Воно являє собою множинну перспективу, що складається з декількох незалежних вимірювань, уздовж яких можуть бути проаналізовані певні сукупності даних.

Одночасний аналіз по декількох вимірах даних визначається як багатовимірний аналіз. Кожен вимір включає напрямки консолідації даних, що складаються з серії послідовних рівнів узагальнення, де кожен вищестоячий рівень відповідає більшою мірою агрегації даних по відповідному вимірюванню.

Так, вимір *Виконавець* може визначатися напрямком консолідації, що складається з рівнів узагальнення «підприємство-підрозділ-відділ-службовець». Вимірювання *Час* може навіть включати два напрямки консолідації – «рік-квартал-місяць-день» і «тиждень-день», оскільки відлік часу по місяцях і по тижнях несумісний. У цьому випадку стає можливим довільний вибір бажаного рівня деталізації інформації по кожному з вимірів.

18.3. Об'єктно-орієнтовані бази даних

Наступним новим напрямком у розвитку систем управління базами даних є напрямок, пов'язаний з відмовою від нормалізації відношень. Багато в чому нормалізація відношень порушує природні ієрархічні зв'язки між об'єктами, які досить поширені в нашому світі. Можливість зберігати їх на концептуальному (але не на фізичному) рівні дозволяє користувачам більш природно відображати семантику предметної області. На даний момент вже існує теоретичне обґрунтування роботи з ненормалізованими відношеннями і практичні реалізації подібних систем.

Подальшим розширенням в структурних перетвореннях є об'єктно-орієнтовані бази даних. В об'єктно-орієнтованій парадигмі предметна область моделюється як множина класів взаємодіючих об'єктів. Кожен об'єкт характеризується набором властивостей, які є як би його пасивними характеристиками і набором методів роботи з цим об'єктом. Працювати з об'єктом можна тільки з використанням його методів. Атрибути об'єкта можуть брати певну множину допустимих значень, набір конкретних значень атрибутів об'єкта визначає його стан.

Використовуючи методи роботи з об'єктом, можна змінювати значення його атрибутів і тим самим як би змінювати стан самого об'єкта. Множина об'єктів з одним і тим же набором атрибутів і методів утворює клас об'єктів. Об'єкт повинен належати тільки одному класу (якщо не враховувати можливості успадкування). Допускається наявність примітивних визначених класів, об'єкти-екземпляри яких не мають атрибутів: цілі, рядки і т. д. Клас, об'єкти якого можуть служити значеннями атрибута об'єктів іншого класу, називається доменом цього атрибута.

Однією з найбільш перспективних рис об'єктно-орієнтованої парадигми є принцип успадкування. Допускається породження нового класу на основі вже існуючого класу, і цей процес називається спадкуванням. У цьому випадку новий клас, називається підкласом існуючого класу (суперкласу), успадковує всі атрибути і методи суперкласу. У підкласі, крім того, можуть бути визначені додаткові атрибути і методи. Розрізняються випадки простого і множинного успадкування. У першому випадку підклас може визначатися тільки на основі одного суперкласу, у другому випадку суперкласів може бути декілька. Якщо в мові або системі підтримується одиничне успадкування класів, набір класів утворює деревоподібну ієрархію. При підтримці множинного успадкування класи пов'язані в орієнтований граф з коренем, що називаються графами класів.

Можна вважати, що найбільш важливою якістю ООБД (об'єктно-орієнтованої бази даних), яка дозволяє реалізувати об'єктно-орієнтований підхід, є облік поведінкового аспекту об'єктів.

У прикладних інформаційних системах, що ґрунтуються на БД з традиційною організацією (аж до тих, які базувалися на семантичних моделях даних), існував принциповий розрив між структурною і поведінковою частинами. Структурна частина системи підтримувалася всім апаратом БД, її можна було моделювати, верифікувати і т. д., а поведінкова частина створювалася ізольовано. Зокрема, були відсутні формальний апарат і системна підтримка спільного моделювання і гарантій узгодженості структурної (статичної) і поведінкової (динамічної) частин. У середовищі ООБД проектування, розробка і супровід прикладної системи стають процесом, в якому інтегруються структурний і поведінковий аспекти. Звичайно, для цього потрібні спеціальні мови, що дозволяють визначати об'єкти і створювати на їх основі прикладну систему.

Специфіка застосування об'єктно-орієнтованого підходу для організації та управління БД зажадала уточненого тлумачення класичних концепцій і деякого їх розширення.

Перш за все, виник напрям, який передбачає можливість зберігання об'єктів всередині реляційної БД, тоді додатково необхідно передбачити зберігання і використання специфічних методів роботи з цими об'єктами, а це в свою чергу вимагає розширення стандарту мови SQL. Частково це вже зроблено в новому стандарті SQL3, однак там далеко не всі питання отримали однозначний дозвіл [16].

Однак частина розробників дотримується думки про необхідність повної відмови від реляційної парадигми і переходу на об'єктно-орієнтовану парадигму. Для переходу до об'єктно-орієнтованих БД стандарт об'єктного проектування був доповнений стандартизованими засобами доступу до баз даних (стандарт ODMG93).

Постачальники комерційних СКБД негайно відреагували на цю потребу. Практично кожна поважаюча себе фірма звернулася до об'єктних технологій і продуктивно співпрацює з розробниками об'єктно-орієнтованих СКБД. IBM і Oracle допрацювали свої СКБД (відповідно, DB2 і ORACLE), додавши об'єктну надбудову над реляційним ядром системи. Інший шлях обрав Informix, який придбав серйозну об'єктно-реляційну СКБД Illustra і вмонтував її в свою СКБД. В результаті вийшов продукт, що називається універсальним сервером. Інший лідер ринку СКБД – Computer Associates, вчинив інакше. Він зробив ставку на чисто об'єктну базу Jasmine, активно пропагуючи її переваги. Хто має рацію – покаже майбутнє.

18.4. Темпоральні бази даних

Наступним напрямком розвитку баз даних є виникнення так званих темпоральних баз даних, тобто баз даних, чутливих до часу. Фактично БД моделює стан об'єктів предметної області в певний поточний момент часу. Однак в ряді прикладних областей необхідно досліджувати саме зміну станів об'єктів в часі. Якщо використовувати чисто реляційну модель, то потрібно будувати і зберігати додатково безліч відношень, що мають однакові схеми, що відрізняються часом існування або зняття даних.

Набагато перспективніше і зручніше для цього використовувати спеціальні механізми зняття зрізів за часом для певних об'єктів БД. Основна теза темпоральних систем полягає в тому, що для будь-якого об'єкта даних, створеного в момент часу t_1 і знищеного в момент часу t_2 , в БД зберігаються (і доступні користувачам) всі його стани в тимчасовому інтервалі (t_1, t_2) . При позначенні інтервалу квадратні дужки означають, що межа інтервалу включена в нього, а круглі дужки означають, що точка на тимчасовій осі на межі інтервалу не включається в інтервал. Якщо об'єкт знищений в момент часу t_2 , то в цій точці тимчасової осі він вже не існує, тому права межа тимчасового інтервалу залишається відкритою.

18.5. Дедуктивні бази даних

Ще одним з перспективних напрямків розвитку баз даних є напрямок, пов'язаний з об'єднанням технології експертних систем і баз даних, і розвиток так званих дедуктивних баз даних. Ці бази засновані на виявленні нових знань з баз даних не шляхом запитів або аналітичної обробки, а шляхом використання правил виведення і побудови ланцюжків застосування цих правил для виведення відповідей на запити. Для цих баз даних існують мови запитів, відмінні від класичної мови SQL. В експертних системах також знання експертів зберігаються в формі правил, найчастіше використовуються так звані продукційні правила типу «якщо опис ситуації, то опис дії». Зберігання подібних правил і організація виведення на підставі наявних фактів під силу сучасним СКБД.

У 2000-х поряд з добре зарекомендували себе реляційним стандартом розвивалися і інші, які були об'єднані ключовим словом NoSQL. У деяких сферах застосування вони є цікавою альтернативою реляційних баз даних.

18.6. Бази даних NoSQL

Безпрецедентні обсяги даних змушують розробників і бізнес придивлятися до альтернатив реляційних баз даних, що використовуються ось вже більше тридцяти років. У сукупності всі ці технології відомі як «NoSQL бази даних».

18.6.1. Рух NOSQL

Історично склалося так, що більшість веб-додатків корпоративного рівня використовує реляційні бази даних. Але в останнє десятиліття ми зіткнулися з настільки великими обсягами даних, які так швидко змінюються і такі різноманітні за своєю структурою, що з ними неможливо працювати, використовуючи традиційні реляційні СКБД. Рух NOSQL створено для вирішення цієї проблеми.

Реляційні бази були хорошим рішенням рівно до тих пір, поки потрібно обробляти одні й ті ж дані з фіксованою структурою. Якби лікарняна каса захотіла управляти своїми клієнтами: їх розподілом по філіях, їх медичними рахунками і платежами, то реляційна база була підходящим вибором, тому що вона дозволяє записувати певні однакові властивості, такі як імена та адреси, для кожного клієнта каси під номером учасника. Всі лікарі та філії отримують ідентифікатори, і тоді можна встановити зв'язки між клієнтом і доктором: рахунок 49378 на 729,87 грн для учасника 72627 виданий лікарем 2625. Статистика питомих платежів клієнтів, витрат на лікарів і т. п. Працює відносно швидко і проста в розробці.

Такий тип додатків був і залишається важливим. Він буде існувати і надалі і, ймовірно, буде продаватися на основі реляційних баз даних. Але з появою Інтернет системи управління вмістом працюють не з багатьма структурно однаковими пакетами даних, а з відносно невеликою кількістю сильно структурованих всередині даних. Жорсткий і схематичний спосіб зберігання в реляційних базах поганенько підходить для документів, в яких можуть зустрічатися одна або кілька таблиць, зображення, посилання і різні заголовки. А якщо у користувача до того ж є право придумувати власні елементи, наприклад, вставлені в текст об'єкти з певним форматуванням, то робота розробника стає дуже важкою справою.

У нових пошукових системах, які тоді виникли і в соціальних мережах, навпаки, були протилежні потреби, які теж не дуже добре вирішувалися за допомогою реляційної моделі: потрібно було вміти управляти величезними обсягами даних і відповідати на запити за долю секунди. Для таких додатків сторінка запитів реляційної бази була неоптимальною, оскільки ці бази даних були розроблені, щоб відповідати на запити типу «видай мені актуальні обороти всіх філій за останні 12 місяців». Хоч реляційні бази і здатні підтримувати повнотекстовий пошук, який потрібний для пошукових систем, але їх продуктивність неідеальна для цього. І хоча зв'язки між користувачами соціальних мереж також можуть бути збережені в реляційних базах даних і в них можна виконувати такі запити, як «видай мені всіх користувачів з інтересами як у користувача Данилюка», все ж SQL не дуже добре підходить для програмування таких запитів.

Як відомо, великі набори даних дуже складно зберігати в реляційних базах даних. Зокрема, час виконання запитів збільшується разом зі збільшенням розмірів таблиць і зростанням числа з'єднань. І це не вина самих баз даних. Це один з аспектів, що лежить в основі моделі даних, що полягає в отриманні множини можливих результатів запиту з наступною їх фільтрацією для отримання лише необхідних даних.

Щоб уникнути з'єднань і тим самим поліпшити обробку дуже великих наборів даних, було запропоновано кілька альтернатив реляційної моделі. Хоча вони краще справляються з обробкою дуже великих наборів даних, ці альтернативні моделі, як правило, менш наочні, ніж реляційна (за винятком графової моделі, яка є навіть більш наочною).

Але об'єм – це не єдина проблема, з якою стикаються сучасні веб-системи. Крім свого великого об'єму, сучасні дані зазвичай дуже швидко змінюються. Швидкість – це темп зміни даних з плином часу.

Швидкість рідко залишається статичною. Внутрішні і зовнішні зміни системи і контексту її використання можуть мати значний вплив на швидкість. У поєднанні з великим обсягом даних змінна швидкість вимагає від сховища не тільки справлятися з стійко високим рівнем обсягів запису, але і залишатися працездатною при великих навантаженнях.

Існує ще один аспект швидкості – швидкість зміни структури даних. Іншими словами, на додачу до зміни значень певних властивостей може змінюватися загальна структура елементів, що визначають ці властивості. Це зазвичай відбувається з двох причин. Першою є динамізм прикладної області. При змінах прикладної області змінюються і потреби в даних.

По-друге, збір даних часто є експериментальним процесом. Деякі властивості створюються «про всяк випадок», інші додаються пізніше в зв'язку зі зміною вимог. Ті, що опинилися потрібними, залишаються, інші викидаються. Обидва ці аспекти в реляційному світі викликають проблеми, оскільки великий обсяг запису приносить високі витрати обробки, а висока мінливість схем супроводжується високими експлуатаційними витратами.

Хоча пізніше на чашу терезів були додані і інші цілком обґрунтовані причини, але вирішальною причиною стало усвідомлення, що дані набагато різноманітніші, ніж дані, з якими зазвичай має справу реляційний світ.

Тому в кінці 1990-х і початку 2000-х були розроблені нові концепції баз даних, які більше не покладалися на реляційну модель і не використовували SQL для запитів, їх об'єднали під короткою назвою NoSQL.

Деякі розробники ПЗ відносяться неохотно до терміну NoSQL, так як він звучить, як заснований на тому що ми не хочемо робити, а не на тому ким ми є. Рух NoSQL це не рух проти реляційних баз даних. NoSQL – це «Не тільки SQL» (Not Only SQL), а не «Hi SQL» (No SQL at all).

Оскільки ці бази даних були створені в той час, коли Інтернет вже став звичайною справою, то в них для запитів і зберігання даних ставка була зроблена на веб-технології. Таким чином, відпала необхідність встановлювати для розробки «клієнт» для БД, як це потрібно було робити з РСКБД, і можна працювати безпосередньо з веб-браузером.

Якщо потім потрібно буде звернутися до цих баз даних, можна викликати бібліотеку на кшталт cURL (кросплатформена службова програма командного рядка) з власних функцій або викликати безпосередньо бібліотеку, розроблену під використання NoSQL-базу даних.

Рух NoSQL різко зріс в 2009 році, завдяки захопленню кількості компаній пов'язаних з використанням великих обсягів даних. З'являється все більше систем, що дозволяють організовувати і прозоро підтримувати величезні масиви даних, обробляти і контролювати ці дані. Існує цілий ряд баз даних NoSQL, але з них для розробників-одинаків і невеликих груп особливо цікаві такі дві.

18.6.2. Документоорієнтовані бази даних

Документоорієнтовані бази даних, такі як CouchDB або MongoDB, дозволяють зберігати дані з дуже гнучкою структурою. Це ідеально підходить для сайтів або систем управління контентом, тому що не доводиться мучитися над тим, як відобразити структуру документа в базі даних.

Користувач може за своїм бажанням без особливо трудомісткого програмування додавати до документа абзаци, або посилання, або абсолютно нові елементи, які спочатку не передбачалися.

У таких баз даних немає ні таблиць, ні стовпців, ні SQL. Замість цього в них зберігаються JSON-документи (JavaScript Object Notation – текстовий формат обміну даними, заснований на JavaScript), які можуть містити дочірні елементи. І дочірні елементи, в свою чергу, можуть містити інші дочірні елементи. Як розробник, ви пишете невеликі функції JavaScript для виконання запитів.

А якщо ви хочете дати відвідувачам можливість коментувати пости в блозі? У реляційній моделі вам для цього довелося б створити таблицю COMMENT, що містить коментарі відвідувачів; потім ще одну таблицю, VISITOR, в якій зареєстрована інформація про кожного відвідувача, наприклад, ім'я та електронну адресу, і, нарешті, сполучну таблицю між COMMENT і VISITOR, щоб відстежити, який відвідувач який коментар залишив. І між цими трьома таблицями і таблицею для постів вам довелося б створити ще один зв'язок. У документоорієнтованій моделі це набагато простіше: коментарі можуть бути просто прикріплені до кожного посту у вигляді додаткових дочірніх елементів, і при цьому у вас залишається можливість шляхом порівняно невеликих зусиль заблокувати повідомлення спамерів (користувачі, що розсилають спам через Інтернет).

Такий тип зберігання набагато ближчий до нашого способу мислення: коментарі до одного посту «належать» цьому посту і підкоряються йому. Однак реляційна модель вимагає, щоб коментарі зберігалися окремо від поста і потім знову об'єднувалися з ним через зв'язок.

Документоорієнтовані бази – це хороший варіант в тому випадку, якщо ваші дані в структурі динамічні або якщо ви до початку проекту ще не знаєте точно, яким чином дані будуть структуровані пізніше. Якщо ж змінюється тільки вміст, а структура практично та ж, тоді у документоорієнтованих баз немає великих переваг в порівнянні з реляційними базами даних.

18.6.3. Графові бази даних

Якщо мова йде головним чином про створення зв'язків між пакетами даних – неважливо, будь то шляхи хімічних реакцій, грошові потоки або соціальні мережі, – то хорошим вибором будуть графові бази даних, наприклад, такі як Neo4j. Вони можуть використовуватися, наприклад, в рекомендаційних системах, як у Amazon: якщо система моделює замовників і товари у вигляді вузлів, а покупки – як зв'язки між вузлом клієнта і вузлом товару, тоді стає досить просто на основі покупок клієнта А знайти інших клієнтів зі схожими купівельними звичками. Коли система перевіряє їх покупки, вона може порівнювати їх з покупками клієнта А і пропонувати цьому клієнтові ті товари, які він сам до цього не купував, але які охоче купували клієнти зі схожими потребами.

У графових баз даних виключно спартанська модель: є тільки вузли і зв'язки. Вузли можуть містити такі властивості, як ID, ім'я або певні, обрані розробником, властивості. З'єднання, як правило, підкріплюються дієсловами на кшталт knows, likes (в соціальних мережах) або inhibits, catalyzes (в науці).

Створений Facebook додаток Graph search досить просто реалізується за допомогою графової бази даних. З його допомогою можна переглядати свої контакти і робити запити виду «знайти друзів моїх друзів, які часто відвідують групу «Океан Ельзи» і люблять музику Джонні Кеша».

Крім того, SQL не виробляє пошук в графових базах даних. Частково у них є SQL-подібна мова запитів, але часто ви повинні програмувати запити самі.

Наданий графові базами даних новий потужний метод моделювання даних сам по собі не є достатньою підставою для заміни сталих і зрозумілих платформ обробки даних – від цього повинна бути значна практична користь. Для графових баз даних такою мотивацією може послужити застосування її в тих випадках і до таких моделей даних, коли при переході на графову модель буде досягнуто збільшення продуктивності на один і більше порядків. Разом з виграшем в продуктивності графові бази даних надають надзвичайно гнучку модель даних і спосіб розгортання, який відповідає сучасним способам розгортання програмного забезпечення.

Однією з вагомих причин вибору графової бази даних є великий приріст продуктивності при роботі зі взаємопов'язаними даними, в порівнянні з реляційними базами даних і NoSQL-сховищами. На відміну від реляційних баз даних, де облік взаємозв'язків інтенсивно погіршує продуктивність запитів на великих наборах даних, продуктивність графових баз даних залишається незмінною зі збільшенням обсягу збережених даних. Це пов'язано з тим, що запити локалізуються в певній частині графа. В результаті час виконання кожного запиту залежить від розміру частки графа, яку потрібно обійти для задоволення запиту, а не від загального розміру графа.

Властива графам можливість розширення означає, що можна додавати нові види взаємозв'язків, нові вузли, нові мітки і нові підграфи в існуючу структуру, не порушивши при цьому існуючих запитів і функціоналу програми. Це позитивно впливає на продуктивність розробки і знижує ризики для проекту. Завдяки гнучкості графової моделі не потрібно попередньо моделювати завдання в найдрібніших подробицях, що дуже незручно через швидкі зміни бізнес-вимог. Здатність графів до розширення також дозволяє зменшити кількість міграцій, що знижує навантаження при обслуговуванні даних і зменшує ризик втрати даних.

Неважливо, на який тип зберігання даних ви зважитесь, – приймайте це рішення усвідомлено, а не на основі необґрунтованої неприязні до конкретної технології («XML – ну це просто минуле століття») або страху, що не володієте альтернативними технологіями.

Хоч вивчення нових технологій баз даних і потребують часу і зусиль, але в довгостроковій перспективі набагато більш трудомістким може виявитися багаторічна мука з абсолютно не придатною системою. Кожен наступний фрагмент коду, який ви пишете, щоб утримати свою систему на плаву, ускладнює перехід до кращого варіанта.

18.7. Web-технології і бази даних

І нарешті, останнім, але, може бути, найзначнішим напрямком розвитку баз даних є перспектива взаємодії Web-технології і баз даних. Простота і доступність Web-технології, можливість вільної публікації інформації в Інтернеті, так щоб вона була доступна будь-якій кількості користувачів, безсумнівно, відразу завоювали авторитет у великого числа користувачів. Однак процес накопичення слабоструктурованої інформації швидко проходить і далі настає момент забезпечення ефективного управління цією різноманітною інформацією. І це вже серйозна проблема. Деякі дослідники навіть вивели певну тенденцію, яка виражається в тому, що найбільш популярні сайти з часом стають некерованими, у великій кількості інформації неможливо відшукати те, що потрібно.

З одного боку, Web являє собою одну величезну базу даних. Однак до сих пір, замість того щоб перетворитися на невід'ємну частину інфраструктури Web, бази даних залишаються на других ролях. По-перше, дизайнери найбільших Web-серверів з мільйонами сторінок вмісту поступово перекладають завдання управління сторінками з файлових систем на системи баз даних.

По-друге, системи баз даних використовуються в якості серверів електронної комерції, допомагаючи відстежувати профілі, транзакції, рахунки і інвентарні листи.

По-третє, провідні Web-видавці наблизились до використання систем баз даних для зберігання інформаційного наповнення, що мають складне походження. Однак в переважній частині Web-вузлів, особливо в тих, які належать провайдерам і власникам пошукових машин, технологія баз даних не застосовується. У невеликих Web-вузлах, як правило, використовуються статичні HTML-сторінки, що зберігаються в звичайних файлових системах.

В майбутньому статичні HTML-сторінки все частіше стануть замінювати системами управління з вмістом, що динамічно формується. Вже зараз, наприклад, продавці по каталогам не просто перетворюють паперові каталоги в набори статичних HTML-сторінок. Фактично вони представляють електронний каталог, що дозволяє замовникам оперативно дізнатися те, що їх цікавить, чи не гортаючи непотрібну інформацію: наприклад, чи продає постачальник сірі джемperi великого розміру. Продавці пропонують клієнтам персоналізовані манекени, що дозволяють побачити, як буде сидіти на них одяг. Для персоналізації потрібні досить складні моделі даних.

HTML розширюється до XML, мови розширюваної розмітки, яка краще описує структуровані дані. На жаль, XML, схоже, здатна породити хаос в системах баз даних. Підмова запитів XML, що розвивається, нагадує процедурні мови обробки запитів, що превальювали 25 років тому. Крім того, XML стимулює використання кешів (наборів) даних на стороні клієнта з підтримкою оновлень, що змушує розробників занурюватися в трясину проблем розподілених транзакцій. На жаль, значна частина робіт по XML відбувається без серйозної участі спільноти дослідників систем баз даних.

Автори Web-публікацій потребують інструментів для швидкої й економічної побудови сховищ даних, розрахованих на складні додатки. Це, в свою чергу, формує вимоги до технології баз даних для створення, управління, пошуку і забезпечення безпеки вмісту Web-вузлів.

З іншого боку, універсальність Web-клієнта стає вельми привабливою для розробників нескладних додатків, які зможуть працювати з базами даних. В цьому випадку не потрібна установка кожного клієнта, достатньо вислати код доступу, і клієнт автоматично може вже працювати з базою даних, при цьому вам байдуже, де знаходиться клієнт, він може працювати як в локальній, так і, в глобальній мережі, якщо технологія це дозволяє. А це дуже зручно, якщо ви можете з будь-якого робочого місця, маючи відповідний пароль, отримати доступ до необхідних даних. Подібні системи називаються *системами, розробленими по інтранет-технології*, тобто технології, що використовує принципи технологій Інтернету, але реалізовані у внутрішній локальній мережі.

Для розробки Інтернет-додатків, які пов'язані з базами даних, широко використовуються нові засоби програмування: це мова PERL, мова PHP (Personal Home Page Tools), мова Javascript і ряд інших. Познайомившись з базами даних, ви ще не раз з ними зіткнетеся в житті. Будь-яка база даних може стати вашим помічником або тягарем, це залежить від розробників, тобто від вас.

19. СЛОВНИК ТЕРМІНІВ БАЗ ДАНИХ

А

Агрегація даних – абстракція, що дозволяє трактувати зв'язок між елементами моделі як новий елемент.

Адміністратор бази даних (database administrator) – особа або група осіб, які відповідають за вироблення вимог до бази даних, за її проектування, створення, супроводження та забезпечення необхідного рівня продуктивності системи.

Адміністратор даних (data steward) – особа або група осіб, які відповідають за достовірність і повноту даних, які містяться в базі даних, її узгодженість, а також за дотримання регламенту щодо актуалізації бази даних.

Адміністрування бази даних – управління базою даних як єдиним інформаційним ресурсом організації з метою ефективного обслуговування колективу користувачів. Адміністрування БД передбачає виконання комплексу заходів, що забезпечують точність, ясність, повноту, захист і доступність даних в потрібній формі, в потрібному місці і в потрібний час. Це стосується питань забезпечення інформацією як користувачів, так і додатків, що працюють з БД. В рамках адміністрування БД забезпечується безпека БД, здійснюється відновлення БД, проводиться її обслуговування, здійснюється навчання користувачів, забезпечується резервне копіювання даних, організовується супровід БД, проводиться тестування системи, організовується управління доступом користувачів до даних, забезпечується цілісність даних. Відповідальність за підтриманням ресурсів БД може бути поділена між адміністратором даних і адміністратором бази даних. У деяких організаціях не проводиться відмінності між цими двома ролями адміністрування БД.

Аномалії операцій над даними – ситуації, які призводять до суперечливості даних, їх ненавмисної втрати, неможливості введення даних. Розрізняють аномалії оновлення, видалення, введення даних. Аномалія оновлення – це суперечливість даних, викликана їх надмірністю і частковим оновленням. Такого роду аномалія може виникнути, якщо одні й ті ж дані зберігаються в кількох таблицях. При оновленні такого роду даних в одній з таблиць, ці дані можуть бути оновлені в інших таблицях. Аномалія видалення – ненавмисна втрата даних, викликана видаленням інших даних. Якщо, наприклад, при видаленні записів з таблиці ЗАМОВЛЕННЯ КЛІЄНТІВ видаляються замовлення будь-якого клієнта, то може виникнути ситуація, коли загубляться потрібні дані – інформація про клієнта. Аномалія введення – неможливість ввести дані в таблицю, викликана відсутністю інших даних. Наприклад, якщо в таблиці ЗАМОВЛЕННЯ КЛІЄНТІВ встановлена необхідність наявності замовлення, то не представляється можливим додати в цю таблицю інформації про нового клієнта, якщо він поки не зробив замовлення. Виключення можливих аномалій операцій над даними забезпечується розбиттям таблиць, в яких можуть відбутися аномалії, на дві або більше таблиць. Для цього використовуються нормальні форми або правила структурування таблиць.

Архітектура бази даних (архітектура ANSI-SPARC) – архітектура бази даних, що складається з концептуального, зовнішнього і внутрішнього рівнів. Три рівні абстракції представлення БД дозволяють реалізувати ідею відділення логічної структури і маніпуляції даними, як вони розуміються користувачами, від фізичної подання, необхідного комп'ютерним обладнанням. На концептуальному рівні здійснюється концептуальне проектування БД. Воно включає аналіз інформаційних потреб користувачів і визначення необхідних їм елементів даних. Зовнішній рівень становить користувацьке представлення даних БД. Кожна спеціальна група отримує своє уявлення даних в БД. Кожне уявлення даних дає орієнтоване на користувача опис елементів даних і відносин між ними. Внутрішній рівень відображає представлення БД на фізичному рівні. За цей рівень відповідають розробники БД. На цьому рівні визначаються індекси, покажчики, методи доступу. Вигода трирівневої архітектури БД є незалежність фізичного та логічного представлення даних. Трирівневий підхід до побудови архітектури БД був запропонований в 1975 році Комітетом планування стандартів і норм SPARC (Standards Planning

and Requirements Committee) Національного Інституту Стандартизації США (American National Standard Institute – ANSI).

Архітектура системи (System Architecture) – представлення про сукупність функціональних компонентів системи та їх взаємозв'язках.

Атомарність даних – властивість даних, що означає їх неподільність для операцій використовуваної моделі представлення і обробки даних. Тобто дані розглядаються, адресуються і обробляються як єдине ціле (як одне значення).

Атрибут – поіменована характеристика сутності. Наприклад, сутність ГРОМАДЯНИН може включати в себе наступні характеристики (атрибути): НОМЕР ПАСПОРТА, ПРИЗВИЩЕ, ІМ'Я, по БАТЬКОВІ, РІК НАРОДЖЕННЯ тощо. За допомогою атрибутів може бути ідентифікований екземпляр сутності. Атрибут може бути використаний для представлення зв'язків між сутностями. Наприклад, сутності громадянин і замовлення можуть бути пов'язані між собою за допомогою атрибута НОМЕР ПАСПОРТА. При цьому атрибут НОМЕР ПАСПОРТА повинен бути включений в сутність замовлення. Атрибути можуть бути ключовими та описовими. Ключовий атрибут ідентифікує сутність. Описовий атрибут описує одну з його характеристик. У розглянутому прикладі виправдано вибрати в якості ключового атрибута атрибут НОМЕР ПАСПОРТА, а в якості описових атрибутів всі інші. Кожен атрибут має набір властивостей. Так наприклад: розмір, формат, маска введення, значення за замовчуванням, умова на значення, обов'язковість для введення, список допустимих значень.

Б

База даних (Database) – організована згідно з певними правилами і збереженої у пам'яті комп'ютера сукупність даних, яка характеризує актуальний стан деякої предметної області і використовується для задоволення інформаційних потреб користувачів.

Базова мова – мова програми, в яку можуть бути включені команди структурованої мови запитів SQL. Для кожної СУБД в якості базової мови можуть виступати різні мови.

Бази даних клієнт/сервер – база даних, що функціонує в локальній мережі, що складається з клієнтських комп'ютерів, які обслуговує комп'ютер-сервер. На серверному комп'ютері запускається програма – сервер бази даних. Ця програма обслуговує доступ клієнтів до бази даних. На клієнтських комп'ютерах запускаються прикладні програми, які виконують запити до БД, встановленої на сервері.

Безпека даних – властивість БД, яке характеризується зведенням до мінімуму неправильного використання і пошкодження даних користувачами. Адміністратор бази даних розробляє процедури контролю, що запобігають неправильному використанню даних. Він призначає права вихідної групи користувачів. Потім власники даних можуть надавати доступ до даних іншим користувачам. Доступи розрізняються: доступ тільки до деякої частини даних, доступ тільки для читання, доступ з правом оновлення. Доступ до БД контролюється спеціальним механізмом, наприклад з використанням паролів. Таким чином знижується ризик того, що дані однієї групи користувачів будуть пошкоджені іншою групою. Із забезпеченням безпеки даних пов'язана проблема цілісності даних – точності і узгодженості даних, що зберігаються в БД. Цілісність даних забезпечується написанням і використанням програм, контролюючих Введення даних в БД. У разі відмови обладнання для забезпечення безпеки даних передбачають використання процедур резервного копіювання і відновлення БД. У процесі паралельної обробки даних при багатокористувацькому режимі дані можуть бути пошкоджені. У зв'язку з цим необхідно реалізувати заходи, що не допускають пошкодження даних в процесі їх паралельної обробки. Для невразливості даних з боку несанкціонованого доступу до них використовуються засоби шифрування даних.

Блокування записів – запобігає доступ до запису іншої транзакції, поки перша транзакція не виконає своїх дій. Засоби блокування забезпечують безпеку даних у разі паралельної обробки даних.

Буфер (buffer) – область оперативної пам'яті, яка використовується для тимчасового розміщення оброблюваних даних, які передбачається використовувати при зверненні до СУБД, або планується записати в БД після обробки, і яка призначена для прискорення обміну між зовнішньою і оперативною пам'яттю.

В

Відновлення БД – приведення БД в стан, що існувало незадовго до її відмови. База даних приводиться в стан, який вона мала на момент початку незавершених транзакцій, а потім виконуються ці транзакції. У відновленні після відмов ключову роль відіграє протокол, в якому фіксуються всі зміни вносяться в БД, а також стан кожної транзакції.

Вторинний ключ – це атрибут або сукупність атрибутів, які використовуються в операції пошуку необхідних даних у таблиці.

Д

Дані (Data) – представлення фактів про предметну область системи баз даних в формі, яка допускає їх зберігання і обробку на комп'ютері.

Дані неструктуровані (Unstructured Data) – дані, які не описуються засобами будь-якої типизованої моделі даних, для них не існує схеми, що описує їх структуру і багатьох інших властивостей.

Дані структуровані (Structured Data) – дані, які організовані згідно з функціональними можливостями будь-якої типизованої моделі даних, яка має регулярну статичну структуру, яка чітко відповідає заданій схемі.

Датологічне проектування БД – побудова моделей даних і їх взаємозв'язків в термінах інструментальних засобів проектування БД. Будується на базі інформаційно-логічної моделі даних. В якості моделей даних виступають самі дані, їх структурна композиція, правила побудови. При побудові моделі використовуються формати даних і склад операцій конкретної СКБД. В процесі датологічного проектування використовуються дві групи конструкцій мови: декларативного і процедурного типів, Сукупність операцій мови визначає модель даних і є формалізм для опису структур даних і процесів зміни їх станів для моделювання реальних процесів. При датологічному проектуванні будуються схеми даних. В процесі датологічного проектування визначається склад БД, перевіряється адекватність моделі предметної області, оцінюється ефективність запропонованих структур даних.

Декомпозиція схем відношень – операція заміни схеми відношення сукупністю її підсхем, причому таких, що їх об'єднання знову призводить до вихідної схеми.

Ділення відношень – реляційний оператор, результатом виконання якого є відношення, що містить тільки ті атрибути діленого, яких немає в дільнику. Розподіл відношень – бінарний оператор. Атрибути відношення дільника повинні міститися у відношенні – діленому і базуватися на однакових доменах. Наприклад, якщо в якості діленого використовується відношення ІСПИТОВІ ВІДОМОСТІ, в якій представлені атрибути ПРИЗВИЩЕ, ДИСЦИПЛІНА і ОЦІНКА, а в якості дільника відношення ОЦІНКИ з атрибутами ДИСЦИПЛІНА і ОЦІНКА, то результатом поділу буде третє відношення з єдиним атрибутом ПРИЗВИЩЕ. В цьому відношенні будуть присутні прізвища студентів, які отримали оцінки з дисциплін, які представлені у відношенні дільника.

Додаток (Application) – програма або комплекс програм, які забезпечують автоматизацію обробки інформації для прикладної задачі.

Додаток бази даних (Database Application) – програма або комплекс програм, які використовують базу даних і забезпечують автоматизацію обробки інформації з деякої предметної області.

Домен відношення (Domain of relation) – множина всіх можливих значень певного атрибуту відношення.

Друга нормальна форма – відношення наведено до другої нормальної форми, якщо кожен неключовий атрибут функціонально повно залежить від ключового атрибута. Під функціональною залежністю розуміють ситуацію, коли значення атрибута в кортежі однозначно визначає значення атрибута в кортежі. Наприклад, відношення відомість=(СТУДЕНТ, дисципліна, викладач, оцінка) не знаходиться в другій нормальній формі, тому що атрибут викладач не знаходиться в функціонально повній залежності від складеного ключа СТУДЕНТ + дисципліна. Атрибут викладач залежить тільки від атрибуту дисципліна і не знаходиться у функціонально повній залежності від складеного ключа. Для приведення відношення ВІДОМІСТЬ до другої нормальної форми його треба розбити на два відношення УСПІШНІСТЬ і ВИКЛАДАЧ.

Ж

Життєвий цикл бази даних – процес проектування, реалізації та підтримки системи баз даних.

Журнал (Log) – функціональний компонент СУБД, який забезпечує реєстрацію в процесі функціонування системи бази даних відомостей про виконання транзакцій, про працюючих користувачів, про застосування, що виконуються тощо.

З

Запис (Record) – структура даних, яка являє собою сукупність взаємозв'язаних елементів даних різних типів.

Запит (Query) – повідомлення кінцевого користувача або застосування, що направляється до СУБД і активізує в системі бази даних дії, які забезпечують вибірку, вставку, вилучення або оновлення вказаних у ньому даних.

Збитковість даних – дублювання даних в базі даних. Збитковість даних може виникнути при непередбаченому проектуванні реляційних таблиць. Наприклад, в таблиці КНИГИ зберігається інформація про видавництва, в яких видані книги. Так як кожне видавництво випустило не одну книгу, то інформація про одне і те ж видавництво в таблиці КНИГИ буде дублюватися стільки разів скільки книг це видавництво випустило. У результаті цього невинновано витрачається пам'ять комп'ютера. Для виключення збитковості даних в наведеному прикладі необхідно спроектувати другу таблицю ВИДАВНИЦТВА, а потім зв'язати дві таблиці між собою. Для виключення збитковості даних виконується процес нормалізації відношень. У деяких випадках збитковість може вводиться штучно при проектуванні БД з метою підвищення швидкості обробки запитів користувачів, або з метою підвищення надійності системи в умовах роботи зі збоями.

З'єднання відношень – бінарний реляційний оператор. У кожному вихідному відношенні виділяється атрибут, що базується на одному домені, за яким буде проводиться з'єднання. З'єднанням є безліч всіх кортежів, кожен з яких отриманий об'єднанням одного кортежу з першого відношення і одного кортежу другого відношення таких, що виконується умова з'єднання. Якщо умова з'єднання *рівність*, то з'єднання називається еквівалентним. Наприклад, в якості вихідних відношень виступають відношення СПІВРОБІТНИК = (ПІРІЗВИЩЕ, НОМЕР, ПОСАДА) і ВИД ДІЯЛЬНОСТІ = (ПРОЕКТ, НОМЕР). В якості умови з'єднання візьмемо рівність атрибутів НОМЕР. В результаті виконання операції з'єднання цих відношень формується відношення УЧАСНИКИ ПРОЕКТУ = (ПІРІЗВИЩЕ, НОМЕР, ПОСАДА, ПРОЕКТ). В остаточному відношенні представлені тільки ті кортежі першого і другого відношень, в яких значення атрибута НОМЕР збігаються.

І

Ієрархічна модель даних – модель даних, в яких зв'язку між даними мають вигляд деревовидної ієрархічної структури. Наприклад, база даних КЛІЄНТИ може будуватися на основі наступної моделі. На верхньому рівні ієрархії дані про клієнтів. Клієнту підпорядковані його рахунки, рахункам у свою чергу підпорядковані рядки рахунків. В ієрархічній базі даних ці файли зв'язуються між собою фізичними покажчиками, або поля даних, доданих до окремих записів. Кожен запис про клієнта містить вказівник першого запису рахунку цього клієнта. У свою чергу, записи рахунків містять покажчики на інші записи рахунків і на записи рядків рахунків.

Інтеграція даних (Data Integration) – формування єдиного інформаційного ресурсу на основі різних, можливо неоднорідних джерел даних, які базуються на різних технологіях, і забезпечення його використання в деяких застосуваннях.

Індекс (index) – допоміжна структура даних в середовищі зберігання бази даних, яка призначена для підвищення продуктивності виконання операцій пошуку даних, що задовольняють заданому критерію, забезпечення обробки даних відповідно до порядку значень ключа або обчислення різних статистичних характеристик, які залежать від значень ключів.

Інтерфейс прикладного програмування (Application Programming Interface) – сукупність програмних засобів, які забезпечують прикладній програмі на деякій традиційній мові програмування доступ до систем баз даних, які підтримуються в середовищі конкретної СУБД.

Інформаційна система (Information System) – система, яка реалізує автоматизоване збирання, обробку і маніпулювання даними і включає технічні засоби обробки даних, програмне забезпечення і обслуговуючий персонал.

Інформаційна технологія (Information Technology) – комплекс методів, підходів, стандартів й інструментальних засобів, які використовуються для створення, підтримки і застосування комп'ютерних систем будь-якого класу в деякому середовищі функціонування.

К

Клієнт БД – це так званий «клієнтський» процес, що вимагає для своєї роботи певних ресурсів, в тому числі, як правило, і мережних (проте, в деяких випадках клієнтський процес може виконуватися на тому ж комп'ютері, що і серверний процес). У контексті бази даних клієнт БД управляє призначеним для користувача інтерфейсом і логікою програми. Клієнт БД приймає від користувача запит, перевіряє синтаксис і генерує запит до бази даних відповідно до логіки програми. Потім передає запит серверному процесу і очікує відповіді. Сервер приймає запит від клієнта, обробляє надійшов до бази даних запит і потім посилає відповідь на запит назад клієнту. Клієнт БД, отримавши відповідь, форматує надійшли від сервера дані і представляє їх користувачеві.

Ключ первинний (Primary Key) – атрибут або підмножина атрибутів, які унікальним чином визначають кортеж даного відношення.

Ключ зовнішній (Foreign Key) – сукупність атрибутів відношення, значення яких є одночасно значеннями первинного ключа іншого відношення.

Контроль доступу – комплекс засобів і заходів, що дозволяють уникнути несанкціонований доступ до даних. Засоби контролю особливо важливі при спільному використанні даних.

Контроль паралельної обробки – засоби підтримки цілісності даних при багатокористувацькому режимі роботи. Можливі ситуації, коли двоє користувачів одночасно звертаються до одного запису, щоб зробити над нею деякі транзакції. СКБД повинна обмежити доступ одного з користувачів, поки транзакція другого не завершиться. Без таких засобів порушується точність і несуперечливість даних.

Концептуальне проектування БД – створення моделі предметної області у формі, доступній для розуміння користувачем. Ця модель служить для досягнення взаєморозуміння між різними групами користувачів, а також між користувачами та розробниками БД. Така модель будується без урахування фізичного представлення даних в конкретній СКБД і називається інфологічною моделлю предметної області. Для опису інфологічної моделі можуть бути використані різні засоби, в тому числі і природна мова. Але через громіздкість опису моделі природною мовою і неоднозначності його трактування використовуються формалізовані мови. Так як модель повинна забезпечувати введення нових даних без зміни існуючих, то вона повинна мати властивість розширення. Інфологічна модель зазвичай будується спільно потенційними користувачами і розробниками БД. На етапі її побудови здійснюється структуризація даних і опис зв'язків між ними. Подання інформаційної структури реалізується в графічній формі. Інфологічна модель БД будується на основі етапу формулювання та аналізу вимог до БД. У свою чергу вона використовується для етапу датологічного проектування БД.

М

Метадані (Meta Data) – дані про дані, які описують їх склад і структуру, формат представлення, методи доступу і необхідні для цього повноваження користувачів, місце зберігання, їх семантику, володаря тощо.

Мережева модель даних – модель підтримує таке відношення між даними, коли кожен запис може бути підпорядкована записам більш ніж з одного файлу. На відміну від ієрархічної моделі даних, в якій у кожній підлеглої записи в ієрархії може бути тільки одна підпорядковує запис в ієрархії, в мережевої моделі даних до у кожній підлеглої записи в ієрархії може бути кілька підпорядкованих записів в ієрархії.

Мова визначення даних (Data Definition Language) – мова декларативного типу, яка призначена для опису структури бази даних, обмежень цілісності, а іноді для специфікації тригерів, процедур, що зберігаються тощо.

Мова маніпулювання даними (Data Manipulation Language) – мова декларативного типу, яка призначена для виконання основних операцій по роботі з базою даних: введення, модифікація і вибір даних за запитом.

Модель даних (Data model) – сукупність правил структурування даних в базах даних, допустимих операцій над ними і обмежень цілісності, яким вони повинні задовольняти.

Модель предметної області (Enterprise Model) – опис структури предметної області разом із сукупністю зв'язаних з нею обмежень цілісності.

Можливий ключ – це атрибут або сукупність атрибутів, які є ідентифікуючими атрибутами сутності і можуть виступати в якості первинного ключа для сутності після відповідного їх опису.

Н

Нормалізація відношень – процес перетворення реляційних таблиць в стандартну форму. Одні й ті ж дані можуть групуватися у відношення різними способами. Певний набір відношень має кращі властивості при зберіганні та обробці даних, якщо він приведений до нормальних форм. Таблиця може бути представлена у вигляді декількох таблиць, а потім, при необхідності, з цих таблиць за допомогою операцій реляційної алгебри можна відновити первісну таблицю. Нормалізація відношень – формальний апарат обмежень на формування відношень, який дозволяє усунути дублювання даних, забезпечити їх несуперечливість, зменшити витрати на ведення БД.

Нормальна форма – сукупність вимог до відношень, виконання яких забезпечує їх нормалізацію. Як правило, для нормалізації відношень використовуються три нормальні форми. У тих випадках, коли необхідно отримати відношення без залежності від з'єднання, використовується 4-а і 5-я нормальна форма.

О

Операції з даними – будь-яка СКБД дозволяє виконати чотири найпростіші операції з даними: додати в таблицю одну або декілька записів, видалити з таблиці одну або декілька записів, оновити значення деяких полів в одному або декількох записах, знайти одну або декілька записів, що задовольняють заданій умові. Для виконання цих операцій використовується механізм запитів.

Однокористувацькі БД. Бази даних встановлені, і які використовуються на одному комп'ютері. Такі БД використовуються в разі відсутності необхідності доступу до даних з інших комп'ютерів, а також через міркування забезпечення секретності збереженої в БД інформації. Проектування однокористувацьких БД менш трудомістко в порівнянні з проектуванням розподілених БД або БД типу клієнт-сервер.

П

Паралельна обробка – виникає, коли два або більше користувача вимагають доступу до однієї і тієї ж записи БД приблизно в один і той же час.

Перша нормальна форма – відношення приведено до першої нормальної форми, якщо всі його атрибути атомарні. Іншими словами, жоден з елементів відношення сам не повинен бути відношенням. Наприклад, якщо в процесі опису відношення ПОСТАВКА один з його атрибутів ПОСТАЧАЛЬНИК є в свою чергу відношенням, що включає атрибути ПІБ, ТЕЛЕФОН, то таке відношення не приведено до першої нормальної форми. Для приведення відношення до першої нормальної форми потрібно позбутися від складного атрибута ПОСТАЧАЛЬНИК і замінити цей атрибут на два простих ПІБ ПОСТАЧАЛЬНИКА і ТЕЛЕФОН ПОСТАЧАЛЬНИКА.

Поле – стовпець таблиці. Таблиці можна представити як реалізацію відношення. Вони являють собою набір стовпців і рядків. Записи – це рядки таблиці, які включають в себе одне або більше значень полів. Поля описують ті характеристики таблиці, які розробники БД вважають важливими.

Предметна область – частина реального світу, яка представляє інтерес для дослідження, використання. В якості предметної області можуть виступати люди організації, явища природи, історичні дані та багато іншого. На початковому етапі проектування БД проводиться аналіз предметної області, здійснюється виділення її об'єктів і зв'язків між ними.

Проекція відношень – бінарний реляційний оператор. Результатом його виконання є викреслення з вихідної таблиці стовпців, відповідним зазначеним атрибутам, які не входять в зазначений набір. Якщо після викреслення виявилися, що деякі рядки збігаються, то повторювані

рядки викреслюються. Наприклад, якщо виконується оператор проекції до відношення СПІВРОБІТНИК=(ВІДДІЛ, ПРИЗВИЩЕ, ПРОЕКТ) з набором атрибутів ВІДДІЛ, ПРОЕКТ, то в результаті буде отримано нове відношення РОЗПОДІЛ ПРОЕКТІВ ПО ВІДДІЛАХ=(ВІДДІЛ, ПРОЕКТ). В остаточному відношенні відсутні прізвища співробітників і однакові кортежі.

Процедура, що зберігається (Storage Procedure) – програма або процедура обробки даних, що зберігається і виконується на комп'ютері-сервері.

Р

Розподілені БД – система, у яких дані розміщені на декількох комп'ютерах. В розподілених БД не всі дані зберігаються централізовано, вони розподілені по мережі вузлів. Кожен вузол має свою БД і він може звертатися до даних, що зберігаються на інших вузлах. Програмне забезпечення БД організовує узгоджену взаємодію даних при роботі користувачів. Використання БД такого роду дозволяє домагатися швидкого доступу до найбільш часто використовуваних даних, підвищує надійність зберігання даних і незалежність від збоїв роботи інших комп'ютерів.

Резервне копіювання та відновлення даних – засоби, які забезпечують відновлення БД в разі відмови системи.

Реляційна модель даних – модель даних, що представляють дані у вигляді таблиць. В основі побудови моделі лежить ідея, що дані треба пов'язувати згідно з їх внутрішніми логічними відносинами, а не фізичними показниками. Зовнішнє подання даних і простота проектування БД найкращим чином реалізується, коли дані представлені у вигляді відношень (реляційне подання). Реляційна таблиця має такі властивості: кожен елемент таблиці являє собою один елемент даних; всі елементи в стовпці мають однаковий тип; кожен стовпець має унікальне ім'я; однакові рядки в таблиці відсутні; порядок проходження стовпців і рядків може бути довільним. В реляційній моделі цілі файли можуть оброблятися однією командою. Сьогодні реляційні БД розглядаються як стандарт для сучасних комерційних систем роботи з даними.

Реляційні оператори – набір операторів, які описують зміну стану відношень. Операндами реляційних операторів є одне або два відношення. Вони називаються відповідно унарним і бінарним операторами. Результатом застосування реляційного оператора є єдине відношення.

С

Сервер (Server) – програма, яка представляє деякі послуги по запитах інших програм, що називаються клієнтами.

Сервер бази даних (Database Server) – програма, яка виконує функції управління і захисту бази даних, якщо виклик функцій виконується на мові SQL, то його називають SQL-сервером.

Система управління базами даних (Data Base Management System) – комплекс мовних і програмних засобів, призначений для створення, ведення і сумісного використання баз даних, забезпечення логічної і фізичної цілісності даних, надійного і ефективного використання ресурсів, представлення санкціонованого доступу для застосувань і кінцевих користувачів, а також підтримки функцій адміністрування бази даних.

Словник даних (Data Dictionary) – програмна система призначена для збереження інформації про структури даних, взаємозв'язках файлів бази даних, типах даних і форматах їх представлення, належності користувачам, кодах захисту, засобах ідентифікації тощо.

Структурована мова запитів SQL. Мова, призначена для взаємодії з реляційними БД. SQL прийнята в якості стандарту мови комунікації реляційних БД. Конкретні реалізації SQL дещо відрізняються один від одного і використовують розширення стандарту. Типи команд SQL: мова визначення даних, мова маніпулювання даними, мова запитів, мова управління даними. У SQL задіяні команди адміністрування даних, команди управління транзакціями. Команди визначення даних використовуються для визначення структур БД, дозволяють створювати, змінювати видаляти таблиці. Команди маніпулювання даними використовуються для маніпулювання інформацією всередині об'єктів реляційної БД. Мова запитів служить для вибірки даних. Це найбільш відома мова для розробників БД. Мова управління даними дозволяє управляти доступом до інформації знаходиться в БД. Команди адміністрування даних дозволяють здійснювати контроль за виконуваними діями користувачами, можуть використовуватися для аналізу продуктивності системи.

Сутність – основний зміст явища або процесу, про який збирається інформація. Сутності виділяються в процесі аналізу предметної області. Сутність це предмет інтересу розроблюваної і використовуваної БД, то про що можна збирати інформацію. Наприклад, якщо в якості предметної області розглядається підприємство, то в ній можна виділити наступні сутності: співробітники, підрозділи, приміщення та ін. Сутності можуть бути реальними і абстрактними. Наприклад, напрямки діяльності підприємства абстрактні сутності. Тип сутності це поняття, яке відноситься до набору однорідних об'єктів. Примірник сутності – конкретний об'єкт в наборі. Сутність характеризується набором властивостей, атрибутів. Наприклад, сутність СПІВРОБІТНИК може характеризуватися атрибутами ТАБЕЛЬНИЙ НОМЕР, ПРІЗВИЩЕ, ПОСАДА тощо. Сутність часто ототожнюють з об'єктом.

Схема бази даних (Database Schema) – опис бази даних засобами мови опису даних будь-якого архітектурного рівня СКБД.

Схема відношення (Relation Schema) – список імен атрибутів відношення.

Сховище даних (Data Warehouse) – система, що містить несуперечливу інтегровану предметно-орієнтовану сукупність історичних даних з метою аналізу даних і прийняття рішень.

Т

Таблиця – спосіб реалізації об'єкта реляційної моделі. В таблиці дані розташовані у вигляді полів і записів, відповідно у вигляді стовпців і рядків. В процесі опису таблиць в СКБД їм присвоюють імена і визначають структуру, тобто визначають поля, з яких складається таблиця. Для кожного поля визначаються його властивості: розмір, формат, маска введення, початкове значення, обов'язковість введення та інші. Визначають індексні і ключові поля.

Транзакція (Transaction) – сукупність операцій маніпулювання даними в системі баз даних, яка переводить базу даних з одного цілісного стану в інший.

Третя нормальна форма – відношення приведено до третьої нормальної форми, якщо усунені транзитивні залежності між атрибутами відношення. Транзитивна залежність має місце в тому випадку, якщо один неключовий атрибут залежить від іншого неключового атрибута. Наприклад, відношення СПИСОК СПІВРОБІТНИКІВ=(СПІВРОБІТНИК, ПОСАДА, ПІДРОЗДІЛ, ТЕЛЕФОН) не приведено до третьої нормальної форми, тому що атрибут ТЕЛЕФОН залежить від неключового атрибута ПІДРОЗДІЛ. Для приведення відношення до третьої нормальної форми відношення СПИСОК СПІВРОБІТНИКІВ потрібно розбити на два відношення СПІВРОБІТНИКИ і ПІДРОЗДІЛИ.

Тригер (Trigger) – різновид процедури, що зберігається, яка автоматично викликається при виникненні певних подій в базі даних. Тригери можуть використовуватися для виконання будь-яких розрахунків в таблицях. Основне призначення тригерів – забезпечення цілісності даних. У цьому випадку вони використовуються для підтримки єдності значень первинних ключів і цілісності на рівні посилань. Кожен раз, коли над таблицею виконується операція додавання, зміни або видалення даних, створюються нові керовані системою версії таблиць. Такі таблиці, які відображають введені в таблицю і видалені з рядка при кожному оновленні відповідної БД, називаються тригерними таблицями.

Ф

Файл плоский (Flat File) – файл, записи якого не можуть містити яких-небудь агрегати даних – елементів даних або груп, що повторюються, а також покажчики на будь-які інші записи цього файлу.

Файл-сервер (File Server) – комп'ютер, що забезпечує зберігання файлів і представлення санкціонованого доступу до них користувачів і застосувань в мережі.

Фізичне проектування БД. Етап проектування БД, на якому моделі, побудовані на попередніх етапах проектування, прив'язуються до середовища зберігання. В якості вихідних даних для етапу фізичного проектування використовуються результати датовісного проектування. На етапі фізичного проектування БД визначаються використовувані запам'ятовуючі пристрої, способи фізичної організації даних. На цьому етапі вибираються формати збережених записів, розробляються методи доступу, методи стиснення інформації, вирішуються питання безпеки даних.

Формулювання та аналіз вимог – початковий етап проектування БД. На цьому етапі на основі аналізу предметної області визначаються цілі розроблюваної БД і вимоги до неї. Проводиться експертування фахівців предметної області, визначається семантика інформаційної моделі, виділяються інформаційні потоки. Створюється інформаційна схема організації, в якій повинна функціонувати БД. На основі аналізу цієї схеми визначаються сфери застосування БД на протязі всього життєвого циклу. Здійснюється збір інформації про використання даних і визначаються їх функції. Перетворюються вимоги до інформації в форму прийнятну для використання на наступних етапах проектування БД.

Функціональна залежність – описує зв'язок між атрибутами відношень. Якщо у відношенні R атрибут Y функціонально залежить від атрибута X , то кожне значення атрибута X пов'язано тільки з одним значенням атрибута Y .

Ц

Централізована база даних – база даних фізично зосереджена в одному місці.

Цілісність бази даних (Database Integrity) – властивість бази даних, яка означає, що вона містить повну, несуперечливу і адекватно відображаючу предметну область інформацію.

Цілісність сутностей – ніякий ключовий атрибут рядка не може бути порожнім. Ключ реляційної таблиці однозначно визначає кожен рядок. Таким чином, якщо користувач хоче отримати конкретну рядок таблиці або виконати з нею маніпуляції, він повинен знати значення ключа цього рядка. Тому запис не повинен записуватися в БД поки значення її ключових атрибутів не будуть визначені.

Етапи проектування баз даних – У процесі проектування БД можна виділити кілька етапів. Зокрема: формулювання та аналіз вимог, концептуальне проектування, проектування реалізації, фізичне проектування. На кожному етапі розробляються моделі, які використовуються для формального представлення даних і зв'язків між ними. Ці моделі дозволяють розв'язувати задачі проектування БД характерні для кожного етапу, а також виступають в якості вихідних для наступних етапів.

CASE-технологія (Computer Aided System Engineering і Computer Aided Software Engineering) – методологія проектування інформаційних систем й інструментальні засоби, які дозволяють наочно моделювати предметну область, аналізувати її модель на всіх етапах розробки і супроводження інформаційної системи і розробляти застосування.

CGI (Common Gateway Interface – інтерфейс загальний шлюзовий) – стандарт, що визначає взаємодію між WWW-сервером і модулями розширення, які можуть застосовуватися для виконання додаткових функцій, які не підтримуються сервером.

HTML (HyperText Markup Language – мова розмітки гіпертекстових документів) – стандартна мова розмітки веб-сторінок в Інтернеті.

ODBC (Open Database Connectivity – інтерфейс відкритий для підключення до баз даних) – інтерфейс прикладного програмування у вигляді бібліотек функцій, що викликаються з різних програмних середовищ і дозволяють застосуванням уніфікованим чином звертатися на мові SQL до баз даних різних форматів.

OLAP (On-Line Analytical Processing – аналітична обробка інформації) – технологія інтерактивної аналітичної обробки даних в системах баз даних, яка призначена для підтримки прийняття рішень і орієнтована головним чином на нерегламентовані інтерактивні запити.

OLE DB (Object Linking and Embedding Database – зв'язування і вбудовування об'єктів баз даних) – стандартний інтерфейс, що являє собою універсальну технологію доступу до будь-яких джерел даних через інтерфейс COM (Common Object Model).

OLTP (On-Line Transaction Processing – оперативна обробка транзакцій) – клас застосувань систем баз даних, які призначені для підтримки поточної діяльності різного роду організацій.

SQL (Structured Query Language – структурована мова запитів) – мова баз даних, яка являє собою стандартизований засіб опису запитів до баз даних.

XML (eXtensible Markup Language – розширювана мова розмітки) – мова розмітки для документів WEB, яка дозволяє визначати структуру документа і типи даних, які в ньому зберігаються.

**Авраменко Валентин Семенович
Розломій Інна Олександрівна**

ОРГАНІЗАЦІЯ БАЗ ДАНИХ І ЗНАНЬ

Курс лекцій

Загальний редактор *В. І. Салапатов*
Комп'ютерне верстання *А. С. Авраменко*

Підписано до друку __. __. 2019. Формат 60х84/16.
Ум. друк. арк. 16,64. Тираж 60 пр. Зам. № __

Видавець і виготовлювач

Черкаський національний університет імені Богдана Хмельницького

Адреса: бульвар Шевченка, 81, м. Черкаси, Україна, 18031

Тел. (0472) 37-13-16, факс (0472) 37-22-33,

e-mail: vydav@cdu.edu.ua, <http://www.cdu.edu.ua>

Свідоцтво про внесення до державного реєстру
суб'єктів видавничої справи ДК №3427 від 17.03.2009 р.